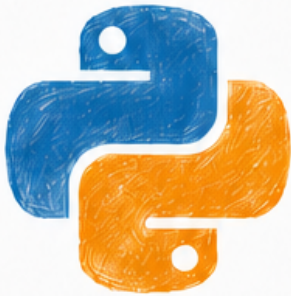
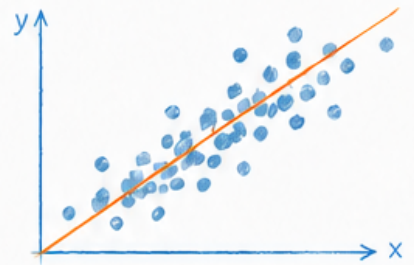


CIENCIA DE DATOS CON PYTHON

— DE LOS DATOS BRUTOS AL CONOCIMIENTO —



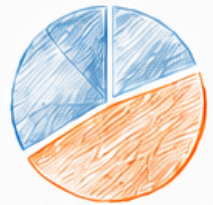
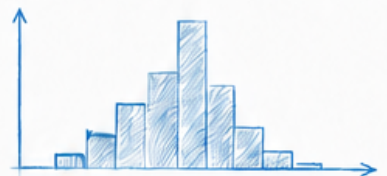
Alan Turing
(1912–1954)



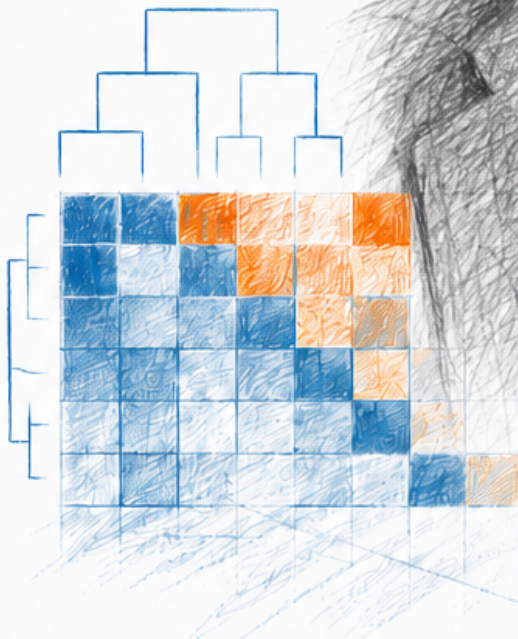
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

df = pd.read_csv('datos.csv')
df.describe()

plt.figure(figsize=(8,5))
sns.heatmap(df.corr(),
            annot=True, cmap='coolwarm')
```



$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$



Vol. III

MATERIAL GRATUITO

Última versión y apoyo al proyecto

Este documento forma parte de VintaBytes y se comparte de manera gratuita.

Escaneá el código QR para acceder a la última versión disponible, encontrar otros recursos del proyecto o colaborar de forma voluntaria.

Si este material te resultó útil, cualquier aporte ayuda a seguir preparando libros, cuadernos y ejemplos en español. También ayuda compartirlo, recomendarlo o avisarme si encontrás algo para mejorar.

Y si querés colaborar con el café que acompaña muchas de estas páginas, prometo usarlo con responsabilidad pedagógica.

Muchas gracias por formar parte de VintaBytes.



Escaneá para ir a la página oficial

También podés entrar desde:

<https://vintabytes.github.io/>

Alan Turing

Alan Turing fue un matemático, lógico e investigador británico considerado una de las figuras fundamentales en la historia de la computación moderna. Nacido en Londres en 1912, desarrolló ideas que ayudaron a definir qué significa calcular, procesar información y construir máquinas capaces de ejecutar procedimientos formales.

Su aporte más importante para la ciencia de datos no está en haber trabajado con bases de datos o modelos estadísticos como los entendemos hoy, sino en haber establecido parte de la base conceptual sobre la que se apoyan la informática, la inteligencia artificial y el aprendizaje automático. En 1936 propuso el modelo teórico conocido como **máquina de Turing**, una idea central para comprender qué problemas pueden resolverse mediante algoritmos.

Durante la Segunda Guerra Mundial trabajó en Bletchley Park, donde participó en el desarrollo de métodos para descifrar mensajes codificados por la máquina Enigma. Ese trabajo combinaba matemática, lógica, análisis de patrones y procesamiento sistemático de información, elementos que hoy reconocemos como parte del pensamiento computacional aplicado a datos.

Después de la guerra, Turing también reflexionó sobre la posibilidad de que las máquinas pudieran mostrar comportamientos inteligentes. En 1950 publicó el artículo *Computing Machinery and Intelligence*, donde propuso una forma de pensar la inteligencia artificial que más tarde sería conocida como el **test de Turing**.

Por todo esto, Alan Turing ocupa un lugar central en la historia de la ciencia de datos, aunque haya vivido antes de que ese campo tuviera su nombre actual. Sus ideas ayudaron a construir el marco intelectual que hizo posible la computación moderna: representar información, diseñar algoritmos, automatizar razonamientos y pensar en máquinas capaces de aprender o tomar decisiones a partir de datos.

Créditos y uso del material

Ciencia de Datos con Python - Tomo 3

Machine Learning

Autor: **Ariel Palazzesi**

Año: **2026**

Versión actualizada de este documento:

<https://vintabytes.github.io/>

Este libro forma parte de una serie de materiales educativos sobre ciencia de datos con Python. La presente versión está en desarrollo, y además de este documento PDF incluye, en cada capítulo, un cuadernos de trabajo para Google Colab.

Los cuadernos de trabajo están disponibles de forma gratuita en el repositorio del proyecto:

Repositorio:

<https://github.com/VintaBytes/Ciencia-de-datos>

Desde allí, el lector puede acceder a la última versión de cada notebooks, abrirlos en Google Colab, ejecutar el código, modificar ejemplos y experimentar con los datos como parte del proceso de aprendizaje.

Derechos de autor

© 2026 Ariel Palazzesi. Todos los derechos reservados.

Este material se publica con fines educativos. Se permite su lectura, descarga y uso personal o educativo, así como la ejecución y modificación de los cuadernos en Google Colab con fines de aprendizaje.

No se permite la venta, redistribución, publicación de versiones modificadas, incorporación en cursos comerciales, reproducción total o parcial con fines comerciales, ni la creación de obras derivadas para su distribución pública sin autorización previa y por escrito del autor.

La disponibilidad pública de este libro o de sus cuadernos complementarios no implica cesión de derechos ni autorización para usos distintos de los indicados en esta página.

Para solicitar permisos de uso, adaptación o distribución, contactar al autor.

Uso educativo

Este material puede utilizarse como apoyo para el estudio personal, la consulta docente o el acompañamiento de cursos introductorios de programación, análisis de datos o ciencia de datos con Python, siempre respetando las condiciones de uso indicadas anteriormente.

Se recomienda acompañar la lectura del PDF con la ejecución de los cuadernos online, ya que buena parte del aprendizaje ocurre al probar instrucciones, observar resultados, modificar el código y revisar cómo cambian los datos después de cada operación.

Herramientas utilizadas

Los ejemplos y actividades de este tomo utilizan principalmente:

- Python;
- Pandas;
- Google Colab;
- Jupyter Notebooks.

Las menciones a tecnologías, bibliotecas, plataformas o servicios de terceros se realizan con fines educativos y descriptivos. Cada herramienta conserva sus propias marcas, licencias y condiciones de uso.

Nota sobre el alcance del material

Este tomo está pensado como una introducción progresiva al trabajo con datos tabulares en Python. No pretende cubrir todos los temas de ciencia de datos, sino construir una base sólida para continuar luego con análisis exploratorio, visualización, comunicación de resultados y machine learning.

Introducción

Después de aprender a cargar, inspeccionar, limpiar, transformar, resumir, agrupar y visualizar datos, aparece una nueva etapa en el trabajo con Python: construir modelos capaces de aprender patrones a partir de esos datos. Ya no se trata solo de describir lo que ocurrió, comparar grupos o comunicar hallazgos exploratorios. Ahora comenzamos a preguntarnos si, a partir de la información disponible, es posible hacer predicciones, clasificar casos, estimar valores numéricos y evaluar qué tan bien funciona un modelo frente a datos que no había visto antes.

Este tercer tomo de **Ciencia de Datos con Python** continúa el recorrido iniciado en los volúmenes anteriores. Si el primer tomo estuvo centrado en comprender y preparar datos tabulares, y el segundo avanzó hacia el análisis exploratorio, este volumen introduce los conceptos fundamentales del **Machine Learning**. El objetivo es que el lector pueda comprender qué significa entrenar un modelo, qué tipos de problemas pueden abordarse, cómo se preparan los datos para aprender, cómo se evalúan los primeros resultados y qué precauciones deben tomarse para no extraer conclusiones engañosas.

Machine Learning suele presentarse como una disciplina compleja, rodeada de términos técnicos, modelos sofisticados y promesas exageradas. En este libro vamos a recorrer el tema desde una perspectiva introductoria, pero cuidadosa. No empezaremos por los modelos más avanzados ni por fórmulas desconectadas de la práctica, sino por las preguntas básicas: qué es aprender a partir de datos, qué diferencia hay entre un problema de clasificación y uno de regresión, qué significa trabajar con datos etiquetados, por qué es necesario separar datos de entrenamiento y prueba, y cómo sabemos si un modelo está funcionando mejor que una simple suposición.

El contenido de este libro incluye una serie de cuadernos de trabajo preparados para Google Colab. En el libro en formato PDF se presentan los temas como texto organizado para facilitar la lectura continua, la consulta y el estudio. Sin embargo, la experiencia más completa se obtiene al trabajar también con los cuadernos disponibles en línea: allí es posible ejecutar el código, modificarlo, observar los resultados, cambiar parámetros, comparar modelos y experimentar con distintos datasets.

Siempre que sea posible, los ejemplos se apoyan en situaciones reales o en datasets disponibles públicamente, incluyendo conjuntos de datos clásicos y datasets provenientes de plataformas como Kaggle. La intención no es trabajar con ejemplos artificiales demasiado simples, sino acercar al lector a problemas parecidos a los que

puede encontrar en contextos de estudio, investigación, análisis profesional o proyectos personales.

Qué vas a encontrar en este tomo

Este tomo se concentra en los fundamentos iniciales del Machine Learning aplicado con Python. A lo largo de los capítulos se presentan los conceptos centrales necesarios para comenzar: modelos, entrenamiento, predicción, variables de entrada, variable objetivo, datos etiquetados, aprendizaje supervisado, clasificación, regresión y evaluación de resultados.

En los primeros capítulos se construye una base conceptual. Antes de entrenar modelos, es necesario comprender qué tipo de problema tenemos entre manos y qué esperamos que el modelo aprenda. No es lo mismo predecir si un correo es spam que estimar el precio de una vivienda. Cada problema requiere una forma distinta de pensar los datos, elegir modelos y evaluar resultados.

Luego se trabaja sobre la preparación de datos para Machine Learning. Se introducen ideas fundamentales como la separación entre variables de entrada y variable objetivo, la división entre datos de entrenamiento y prueba, el tratamiento de valores faltantes, la codificación de variables categóricas, el escalado de variables numéricas y el uso de herramientas como `ColumnTransformer` y `Pipeline`. También aparece una advertencia clave que acompañará todo el recorrido: la fuga de datos. Un modelo puede parecer muy bueno si, sin darnos cuenta, le damos información que no debería tener disponible al momento de predecir.

Más adelante se presentan los primeros modelos de clasificación, comenzando por enfoques intuitivos como K-Nearest Neighbors y avanzando hacia la regresión logística. La intención no es memorizar instrucciones, sino entender cómo piensa cada modelo, qué tipo de frontera de decisión construye, qué ventajas ofrece, en qué situaciones puede fallar y qué cuidados requiere.

Una parte importante del recorrido está dedicada a la evaluación de modelos de clasificación. En Machine Learning no alcanza con entrenar un modelo y obtener una predicción: necesitamos saber si esa predicción es confiable, qué tipos de errores comete y si la métrica utilizada realmente tiene sentido para el problema. Por eso se estudian la matriz de confusión, la accuracy, precision, recall, F1-score, curvas ROC, AUC y el uso de umbrales de decisión.

El libro también aborda problemas de regresión, donde la variable objetivo no es una categoría sino un valor numérico. Allí se introduce la regresión lineal y se estudian métricas como MAE, MSE, RMSE y R^2 . El objetivo es que el lector pueda distinguir con claridad cuándo está frente a un problema de clasificación y cuándo frente a un

problema de regresión, y que pueda interpretar los errores del modelo en unidades reales.

Este primer volumen llega hasta ese punto del recorrido. Al finalizarlo, el lector contará con una base para comprender qué es Machine Learning, cómo se organizan los datos para entrenar modelos, cómo se construyen modelos iniciales de clasificación y regresión, y cómo se evalúan sus resultados principales.

Los temas más avanzados continuarán en el volumen siguiente. Allí se profundizará en la generalización, la validación cruzada, el sobreajuste, el subajuste, la búsqueda de hiperparámetros y el control de la complejidad. También se presentarán modelos más potentes, como Random Forest, Gradient Boosting, Support Vector Machines y Naive Bayes, junto con una introducción al aprendizaje no supervisado, la interpretación de modelos, los sesgos, los errores frecuentes y la comunicación responsable de resultados.

De esta manera, este tomo funciona como una primera etapa completa: una base sólida para construir, entrenar y evaluar modelos iniciales. El volumen siguiente retomará ese punto para avanzar hacia preguntas más profundas: cómo mejorar modelos, cómo compararlos con mayor criterio, cómo interpretarlos y cómo usarlos de manera responsable.

Requisitos previos

Para aprovechar este libro conviene haber trabajado previamente con los contenidos de los tomos anteriores de Ciencia de Datos con Python o contar con conocimientos equivalentes. En particular, se recomienda conocer:

- nociones básicas de programación en Python;
- uso general de Google Colab o notebooks;
- estructura de un DataFrame de Pandas;
- selección de filas y columnas;
- filtros con condiciones;
- ordenamiento de datos;
- creación y modificación de columnas;
- detección básica de valores faltantes, duplicados y tipos de datos;
- cálculo de resúmenes estadísticos simples;
- uso básico de gráficos con Matplotlib;
- lectura e interpretación inicial de tablas, distribuciones y comparaciones.

No es necesario tener experiencia previa en Machine Learning. Este libro está pensado para lectores que se acercan al tema por primera vez. Tampoco se requiere un dominio avanzado de matemática, aunque sí se presentarán fundamentos matemáticos cuando sean necesarios para comprender una métrica, una distancia, un error o una medida de evaluación.

El objetivo no es evitar la matemática, sino incorporarla de forma gradual y conectada con problemas concretos. Cuando una fórmula aparezca, lo hará para explicar una idea útil: cómo se mide un error, cómo se resume el desempeño de un modelo o por qué una métrica puede ser más adecuada que otra.

A quién está dirigido

Este material está pensado para personas que ya conocen Python y han trabajado con datos tabulares, pero no tienen experiencia previa en Machine Learning. Puede ser útil para estudiantes, docentes, programadores, analistas, profesionales que trabajan con datos, personas que preparan proyectos académicos y cualquier lector que quiera comprender los fundamentos de los modelos predictivos sin comenzar directamente por herramientas avanzadas.

El enfoque es introductorio, pero no superficial. Se busca que el lector pueda entrenar modelos iniciales, evaluar resultados y comparar alternativas simples, pero también que entienda las limitaciones de cada procedimiento. En Machine Learning, obtener una métrica no significa necesariamente haber resuelto un problema. Hay que mirar los datos, revisar supuestos, analizar errores, evitar fugas de información y comunicar conclusiones con prudencia.

Este libro puede acompañar a quienes estén comenzando un curso de Machine Learning, preparando un proyecto académico, explorando datasets de Kaggle o intentando comprender mejor cómo se aplican modelos predictivos en problemas reales. No está pensado como un manual exhaustivo de todos los algoritmos existentes, sino como una base sólida para empezar a construir criterio.

Cómo trabajar con este libro y los cuadernos

Cada capítulo puede leerse directamente en el PDF, pero fue pensado también como una experiencia práctica apoyada en cuadernos ejecutables. La recomendación es acompañar la lectura con la versión online del cuaderno correspondiente siempre que sea posible.

En los cuadernos de Google Colab conviene ejecutar cada celda, observar la salida y preguntarse qué aporta ese resultado al problema que se está resolviendo. En Machine Learning, muchas veces el valor no está solo en que el código funcione, sino

en interpretar qué ocurrió: cómo quedaron separados los datos, qué variables usó el modelo, qué métrica se obtuvo, qué errores cometió y si el resultado parece razonable.

También es recomendable modificar los ejemplos. Cambiar una variable, probar otro modelo, ajustar un parámetro, alterar el tamaño del conjunto de prueba o comparar distintas métricas ayuda a comprender que los modelos no son cajas mágicas. Cada decisión del proceso puede influir en el resultado final.

Los capítulos fueron diseñados como una secuencia progresiva. Algunos conceptos reaparecen en distintos momentos porque forman parte del trabajo cotidiano con modelos. La separación entre entrenamiento y prueba, la evaluación sobre datos no vistos, la importancia de elegir métricas adecuadas y el cuidado frente a la fuga de datos no se aprenden en una única aparición: se consolidan al verlos funcionar en distintos contextos.

Enfoque del libro

El objetivo principal de este tomo es introducir el Machine Learning como una extensión natural del trabajo con datos. Antes de construir modelos, necesitamos comprender los datos; antes de confiar en una predicción, necesitamos evaluarla; antes de comunicar un resultado, necesitamos conocer sus límites.

A lo largo del recorrido se priorizan cuatro ideas:

- un modelo aprende patrones a partir de datos, no verdades absolutas;
- evaluar correctamente es tan importante como entrenar;
- no todos los problemas requieren el mismo tipo de modelo ni la misma métrica;
- una predicción útil debe interpretarse dentro del contexto del problema, los datos disponibles y sus limitaciones.

Con esa base, el lector queda preparado para avanzar hacia el volumen siguiente, donde se trabajarán temas como generalización, validación cruzada, sobreajuste, búsqueda de hiperparámetros, modelos más potentes, aprendizaje no supervisado, interpretación, sesgos y comunicación responsable de resultados. Pero, sobre todo, queda mejor preparado para formular buenas preguntas, elegir herramientas con criterio y evitar algunos de los errores más comunes al comenzar a trabajar con Machine Learning.

Índice general

Parte I · Comprender qué es Machine Learning

Capítulo 1 · ¿Qué es Machine Learning?

- Programación tradicional vs. Machine Learning.
- ¿Qué significa que un modelo “aprenda” de los datos?.
- Patrones, predicciones y generalización.
- Qué puede resolver el Machine Learning y qué no puede resolver.
- El rol central de los datos.
- Algunos ejemplos del mundo real.

Capítulo 2 · Tipos de problemas en Machine Learning

- Aprendizaje supervisado.
- Aprendizaje no supervisado.
- Aprendizaje por refuerzo.
- Problemas de clasificación.
- Problemas de regresión.
- Problemas de agrupamiento.
- Problemas de detección de anomalías.
- No todos los modelos sirven para todos los problemas.

Capítulo 3 · El flujo general de un proyecto de Machine Learning

- Definir el problema.
- Conseguir y explorar los datos.
- Elegir variables.
- Separar datos de entrenamiento y prueba.
- Entrenar un modelo.
- Evaluar resultados.
- Mejorar el modelo.
- Comunicar conclusiones.

Parte II · Preparar datos para aprender

Capítulo 4 · Variables de entrada y variable objetivo

- El puente hacia scikit-learn.
- ¿Qué son X e y?.
- Features o variables predictoras.
- Target, etiqueta o variable objetivo.
- Variables numéricas y categóricas.
- ¿Qué significa que una variable sea útil para predecir?.
- Variables disponibles antes de la predicción vs. variables conocidas después.
- Primer acercamiento a la fuga de datos.

Capítulo 5 · Separar datos: entrenamiento y prueba

- ¿Por qué no evaluamos el modelo con los mismos datos con los que entrenamos?.
- `train_test_split`.
- Conjuntos de entrenamiento y prueba.
- Proporciones habituales.
- Aleatoriedad y `random_state`.
- Estratificación en clasificación.
- Separar para evaluar mejor.

Capítulo 6 · Limpieza y transformación de datos

- Valores faltantes.
- Variables categóricas.
- One-Hot Encoding.
- Escalado de variables numéricas.
- Transformaciones que deben aprenderse solo con entrenamiento.
- Fuga de datos durante el preprocesamiento.
- Introducción a `ColumnTransformer`.
- Introducción a `Pipeline`.

Parte III · Primeros modelos de clasificación

Capítulo 7 · Clasificación: predecir categorías

- ¿Qué significa clasificar?.

- Clasificación binaria.
- Clasificación multiclase.
- Clases, etiquetas y probabilidades.
- Frontera de decisión.
- Ejemplos reales de clasificación.
- Clasificar es decidir entre categorías.

Capítulo 8 · K-Nearest Neighbors

- La idea de aprender por cercanía.
- Distancias entre puntos.
- El parámetro k.
- Por qué el escalado importa.
- Ventajas del modelo.
- Limitaciones.
- Cuándo puede funcionar bien y cuándo no.
- Clasificar mirando a los vecinos.

Capítulo 9 · Regresión logística: clasificar con probabilidades

- Por qué se llama regresión si sirve para clasificar.
- La idea de estimar una probabilidad.
- La función sigmoide.
- El umbral de decisión.
- Interpretar la influencia de las variables.
- Ventajas del modelo.
- Limitaciones.
- Clasificar estimando probabilidades.

Capítulo 10 · Evaluar modelos de clasificación

- Por qué accuracy no siempre alcanza.
- Aciertos y errores en clasificación.
- Matriz de confusión.
- Verdaderos positivos, falsos positivos, verdaderos negativos y falsos negativos.
- Precisión.
- Recall o sensibilidad.
- F1-score.
- El impacto del desbalance de clases.

Parte IV · Evaluar modelos de clasificación

Capítulo 11 · Accuracy y matriz de confusión

- Qué mide la accuracy.
- Cuándo puede ser útil.
- Cuándo puede engañar.
- Matriz de confusión.
- Verdaderos positivos.
- Falsos positivos.
- Verdaderos negativos.-
- Falsos negativos.
- Problemas con clases desbalanceadas.

Capítulo 12 · Precision, recall y F1-score

- Qué mide precision.
- Qué mide recall.
- El equilibrio entre ambas.
- F1-score.
- Cuándo conviene priorizar precision.
- Cuándo conviene priorizar recall.
- Ejemplos reales: medicina, fraude, spam, seguridad.

Capítulo 13 · Probabilidades, umbrales, ROC y AUC

- Predicción de clases vs predicción de probabilidades.
- Umbral de clasificación.
- Sensibilidad y especificidad.
- Curva ROC.
- AUC.
- Limitaciones de ROC-AUC.
- Breve mención a Precision-Recall Curve.

Parte V · Modelos de regresión

Capítulo 14 · Regresión: predecir valores numéricos

- Diferencia entre clasificación y regresión.
- Variable objetivo continua.

- Error de predicción.
- Ejemplos reales: precios, ventas, demanda, temperaturas, consumo.
- Cuándo un problema no debería tratarse como clasificación.

Capítulo 15 · Regresión Lineal

- La idea de ajustar una recta o un plano.
- Pendiente e intercepto.
- Regresión lineal simple.
- Regresión lineal múltiple.
- Coeficientes.
- Supuestos básicos.
- Ventajas y limitaciones.

Capítulo 16 · Métricas para regresión

- Error absoluto.
- MAE.
- MSE.
- RMSE.
- R^2 .
- Interpretar errores en unidades reales.
- Por qué no existe una única métrica perfecta.
- Comparación entre modelos de regresión.

Capítulo 1 · ¿Qué es Machine Learning?

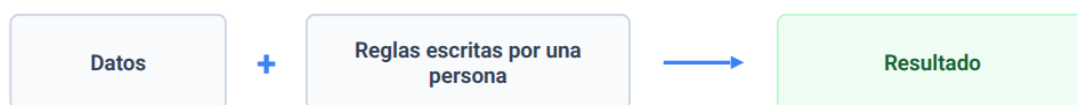
Programación tradicional vs. Machine Learning

Durante mucho tiempo, la forma más habitual de resolver problemas con una computadora fue escribir programas siguiendo una lógica bastante directa: una persona analiza el problema, piensa una solución, la traduce a instrucciones precisas y la computadora ejecuta esas instrucciones. A esta forma de trabajo podemos llamarla, de manera general, **programación tradicional**.

En la programación tradicional, el programador define explícitamente las reglas que el programa debe seguir. Si queremos construir una calculadora, escribimos las instrucciones para sumar, restar, multiplicar o dividir. Si queremos validar una contraseña, indicamos cuáles son las condiciones que debe cumplir: una longitud mínima, la presencia de números, letras mayúsculas o símbolos. Si queremos calcular el promedio de notas de un estudiante, indicamos cómo sumar los valores y dividirlos por la cantidad de notas.

En todos estos casos, la computadora no “descubre” cómo resolver el problema. Simplemente ejecuta una serie de pasos que alguien escribió previamente. La inteligencia, por decirlo de algún modo, está puesta principalmente en el diseño de las reglas. El programa recibe datos de entrada, aplica instrucciones definidas por una persona y produce una salida.

Podemos representar esta idea de una forma sencilla:



Programación tradicional.

Por ejemplo, imaginemos un sistema que decide si una persona puede acceder a un descuento por edad. La regla podría ser:

```
if edad >= 65:
    print("Corresponde descuento")
else:
    print("No corresponde descuento")
```

Este programa funciona porque el criterio es claro, directo y fácil de expresar mediante una regla. No hace falta que la computadora “aprenda” nada. La persona que programó el sistema ya conoce la condición que debe evaluarse.

El problema aparece cuando intentamos resolver situaciones en las que las reglas no son tan fáciles de escribir. Pensemos, por ejemplo, en un sistema que debe detectar si un correo electrónico es spam. Al principio podríamos intentar resolverlo con programación tradicional. Podríamos escribir reglas como: “si el correo contiene la palabra gratis, marcarlo como sospechoso”, o “si contiene muchos signos de exclamación, considerarlo posible spam”. Estas reglas podrían funcionar en algunos casos, pero rápidamente mostrarían sus límites.

No todos los correos con la palabra “gratis” son spam. Una universidad podría enviar un mensaje legítimo sobre una inscripción gratuita. Una empresa podría mandar una factura o una promoción válida. Al mismo tiempo, muchos correos no deseados pueden evitar palabras obvias y estar escritos de forma bastante parecida a un mensaje normal. Si intentáramos cubrir todos los casos posibles escribiendo reglas manualmente, el programa se volvería cada vez más complejo, más frágil y más difícil de mantener.

Algo similar ocurre con otros problemas: recomendar películas, estimar el precio de una vivienda, detectar operaciones bancarias sospechosas, reconocer si una imagen contiene determinado objeto o predecir si un cliente podría dejar de usar un servicio. En todos estos casos, suele haber muchos factores involucrados. Además, esos factores no siempre se combinan de una manera simple.

Tomemos el caso del precio de una vivienda. Una regla tradicional podría decir que una casa más grande vale más que una casa más pequeña. En general, eso parece razonable. Pero no alcanza. También importa la ubicación, la antigüedad, el estado de conservación, la cantidad de ambientes, la cercanía a transporte público, la demanda de la zona, la presencia de jardín, cochera o terraza, y muchos otros elementos. Incluso puede ocurrir que una vivienda más pequeña tenga un precio mayor que otra más grande si está mejor ubicada.

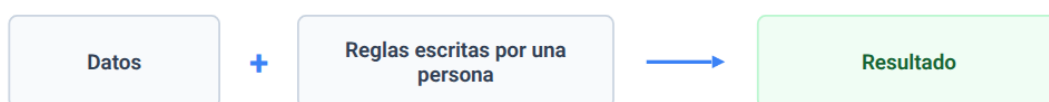
Frente a problemas de este tipo, escribir reglas manuales puede volverse poco práctico. No porque sea imposible escribir reglas, sino porque las reglas necesarias serían demasiadas, demasiado rígidas o demasiado difíciles de ajustar a la realidad. Además, la realidad cambia. Las palabras usadas en correos no deseados cambian, los gustos de los usuarios cambian, los precios del mercado cambian, los patrones de consumo cambian.

Aquí aparece una idea distinta: en lugar de escribir todas las reglas a mano, podemos darle a la computadora muchos ejemplos y permitir que encuentre regularidades en esos datos. Esta es una de las ideas centrales del **machine learning**.

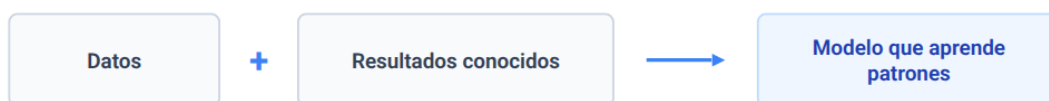
En machine learning, el punto de partida no suele ser una lista cerrada de reglas escritas por una persona, sino un conjunto de datos. A partir de esos datos, un algoritmo busca patrones que le permitan producir una respuesta útil ante casos nuevos. En lugar de indicarle explícitamente al sistema todas las condiciones que debe evaluar, le mostramos ejemplos del problema que queremos resolver.

Podemos representar esta diferencia así:

Programación tradicional



Machine Learning



Programación tradicional vs. Machine Learning

La diferencia es importante. En programación tradicional, el programador escribe las reglas. En machine learning, el programador prepara los datos, elige una estrategia de aprendizaje y entrena un modelo para que encuentre patrones. El resultado de ese proceso es un modelo capaz de realizar predicciones, clasificaciones o estimaciones sobre datos nuevos.

Volvamos al ejemplo del spam. En lugar de escribir manualmente todas las reglas posibles, podríamos reunir una gran cantidad de correos electrónicos ya clasificados: algunos marcados como spam y otros marcados como correos legítimos. El algoritmo analizaría esos ejemplos y buscaría características que suelen aparecer en cada grupo. Tal vez encuentre ciertas palabras frecuentes, estructuras de mensaje, tipos de enlaces, remitentes sospechosos o combinaciones de elementos que, en conjunto, aumentan la probabilidad de que un correo sea spam.

Lo interesante es que el modelo no necesita que una persona le diga cada regla exacta. Aprende a partir de ejemplos. Esto no significa que aprenda como aprende una persona, ni que comprenda el contenido del correo del mismo modo en que lo

haríamos nosotros. Significa que puede encontrar relaciones estadísticas útiles dentro de los datos y usarlas para tomar decisiones o hacer predicciones.

Esta diferencia cambia también el rol de quien desarrolla el sistema. En programación tradicional, gran parte del trabajo consiste en definir instrucciones precisas. En machine learning, una parte fundamental del trabajo consiste en comprender el problema, reunir datos adecuados, preparar esos datos, elegir el tipo de modelo apropiado, evaluar su comportamiento y analizar si sus resultados son confiables. Algunos de estos temas se estudiarán en profundidad más adelante; por ahora, lo importante es notar que el centro del proceso se desplaza: ya no se trata solamente de escribir reglas, sino de trabajar con datos para que un modelo pueda aprender de ellos.

Esto no significa que el machine learning reemplace a la programación tradicional. De hecho, todo proyecto de machine learning necesita programación tradicional en muchas partes: para cargar datos, limpiarlos, organizarlos, construir aplicaciones, mostrar resultados, automatizar procesos o integrar el modelo dentro de un sistema más grande. La diferencia está en qué parte del problema resolvemos mediante reglas explícitas y qué parte intentamos resolver mediante aprendizaje a partir de datos.

Conviene pensar el machine learning no como una forma “mágica” de programar, sino como una herramienta especialmente útil cuando las reglas son difíciles de escribir, pero existen datos suficientes para aprender patrones. Si el problema puede resolverse con una regla simple, probablemente no necesitemos machine learning. Si queremos calcular el impuesto a partir de una fórmula conocida, validar una edad mínima o convertir grados Celsius a Fahrenheit, usar machine learning sería innecesario. Pero si queremos estimar el riesgo de que un crédito no sea pagado, recomendar productos según el comportamiento de usuarios o detectar patrones complejos en grandes volúmenes de información, entonces el machine learning puede aportar mucho valor.

Una forma sencilla de resumir la diferencia es esta: la programación tradicional funciona muy bien cuando conocemos las reglas; el machine learning resulta útil cuando tenemos ejemplos, pero las reglas son difíciles de escribir de manera completa y precisa.

En los próximos apartados vamos a profundizar en esta idea: qué significa exactamente que un modelo “aprenda”, qué tipo de patrones puede encontrar, por qué es tan importante que pueda generalizar a casos nuevos y por qué los datos ocupan un lugar central en todo el proceso.

¿Qué significa que un modelo “aprenda” de los datos?

Cuando decimos que un modelo de machine learning “aprende” de los datos, no estamos usando la palabra aprender exactamente en el mismo sentido en que la usamos para una persona. Un modelo no comprende el mundo, no tiene intención, no razona con sentido común ni entiende el significado profundo de aquello que procesa. Lo que hace es encontrar relaciones, regularidades o patrones dentro de un conjunto de datos, y ajustar su comportamiento para producir mejores respuestas ante situaciones similares.

En términos simples, aprender significa **modificar una forma inicial de responder a partir de ejemplos**. Al principio, un modelo no sabe resolver el problema. Luego observa datos, compara sus respuestas con resultados conocidos o con ciertas señales presentes en esos datos, y ajusta sus parámetros internos para mejorar. Más adelante veremos con mayor detalle qué son esos parámetros y cómo se entrena un modelo. Por ahora, alcanza con pensar que el modelo va afinando una especie de criterio interno a partir de la experiencia que le proporcionan los datos.

Volvamos a un ejemplo clásico, como es la detección de correos spam. Supongamos que tenemos miles o millones de correos electrónicos ya clasificados en dos grupos: correos legítimos y correos no deseados. Cada correo contiene información: palabras, remitente, enlaces, asunto, estructura del mensaje, frecuencia de ciertos términos, presencia de archivos adjuntos, entre muchos otros elementos. A partir de esos ejemplos, un modelo puede empezar a detectar qué características aparecen con más frecuencia en los mensajes spam y cuáles aparecen con más frecuencia en mensajes legítimos.

El modelo podría descubrir, por ejemplo, que ciertos tipos de enlaces, ciertas frases insistentes, determinadas combinaciones de palabras o ciertos patrones en el remitente aparecen con mayor frecuencia en correos no deseados. Pero lo importante es que no analiza cada elemento de manera aislada como una regla rígida. No se trata simplemente de decir: “si aparece la palabra gratis, entonces es spam”. Un modelo puede aprender combinaciones más sutiles. Puede reconocer que una palabra determinada no alcanza por sí sola para decidir, pero que esa palabra, junto con un enlace extraño, un remitente desconocido y una estructura repetitiva del mensaje, aumenta la probabilidad de que el correo sea no deseado.

Esto es muy distinto de escribir manualmente una lista de reglas. En lugar de decirle al sistema exactamente qué debe mirar y cómo debe decidir, le damos muchos ejemplos para que encuentre relaciones útiles. El modelo aprende porque ajusta su manera de clasificar correos según la evidencia que encuentra en los datos.

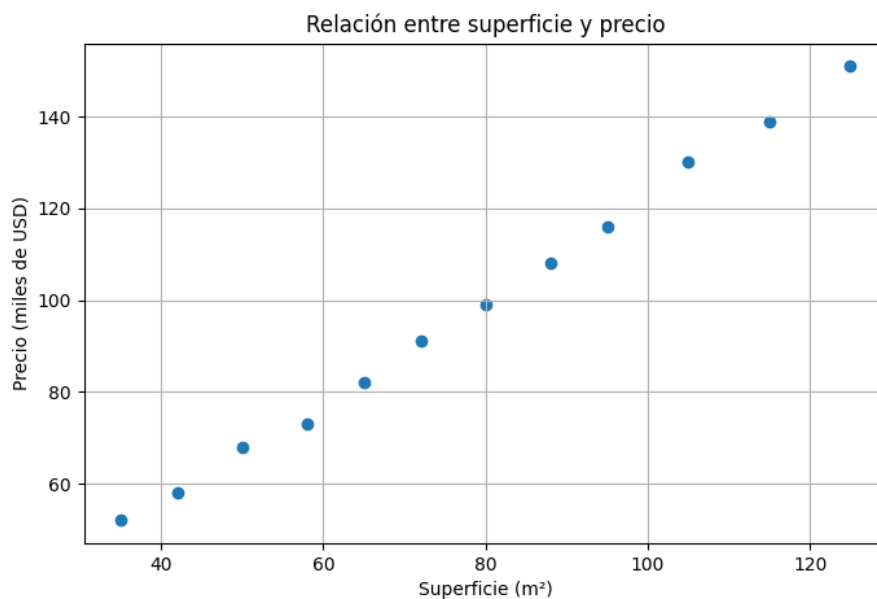
Otro ejemplo interesante aparece en las aplicaciones de navegación, como Google Maps, Waze u otros sistemas similares. Cuando una aplicación informa que una avenida está congestionada o que el viaje tardará más de lo habitual, no está mirando la calle del mismo modo en que lo haría una persona parada en una esquina. Está trabajando con datos: ubicación de teléfonos móviles, velocidades promedio, tiempos históricos de viaje, cambios bruscos en el movimiento de muchos usuarios, horarios, días de la semana, eventos cercanos y otros factores.

Si en una avenida donde normalmente los vehículos circulan a cierta velocidad, de pronto muchos dispositivos comienzan a moverse mucho más lento, el sistema puede inferir que hay un atasco. Si ese patrón se repite en ciertos días y horarios, también puede aprender que en esa zona suele haber congestión a la salida del trabajo o durante el ingreso a una escuela. El sistema no “entiende” el tránsito como lo entiende una persona, pero aprende relaciones entre datos de movimiento, ubicación, horario y duración de los recorridos.

En este caso, aprender de los datos significa poder estimar mejor qué está ocurriendo ahora y qué podría ocurrir durante los próximos minutos. Si muchas personas toman una ruta y tardan más de lo esperado, el sistema incorpora esa información para ajustar sus predicciones. Si otra ruta comienza a mostrar mejores tiempos, puede recomendarla. La aplicación no necesita que alguien escriba manualmente todas las reglas posibles para cada esquina de cada ciudad. Utiliza datos reales y cambiantes para mejorar sus estimaciones.

También podemos pensar en un ejemplo más cercano al análisis de datos: estimar el precio de una vivienda. Si reunimos información sobre muchas propiedades vendidas, podríamos tener columnas como superficie, cantidad de ambientes, barrio, antigüedad, estado general, cercanía a transporte, presencia de cochera y precio final de venta. Un modelo puede analizar esos datos y aprender qué características suelen estar asociadas con precios más altos o más bajos.

Quizás descubra que, en general, una mayor superficie se relaciona con un precio mayor. Pero también puede aprender que la ubicación tiene un peso muy importante, que una cochera aumenta el valor en ciertas zonas más que en otras, o que dos propiedades de la misma superficie pueden tener precios muy diferentes según el barrio y el estado del inmueble. El aprendizaje consiste en encontrar esas relaciones a partir de ejemplos previos y usarlas para estimar el precio de una propiedad nueva.



El gráfico muestra una tendencia ascendente: las viviendas con mayor superficie suelen tener precios más altos.

Este punto es clave: un modelo aprende de casos anteriores para responder sobre casos nuevos. No memoriza simplemente una tabla de datos. Si solo recordara los ejemplos exactos que vio, no sería demasiado útil. Lo valioso es que pueda identificar patrones que le permitan actuar ante situaciones que no estaban exactamente en los datos originales. Si entrenamos un modelo con viviendas ya vendidas, esperamos que pueda estimar razonablemente el precio de una vivienda que todavía no conoce. Si entrenamos un modelo con correos ya clasificados, esperamos que pueda evaluar un correo nuevo. Si un sistema de tránsito analiza viajes previos y datos actuales, esperamos que pueda estimar el tiempo de llegada de un viaje que está ocurriendo ahora.

Aprender, entonces, no es copiar. Aprender es **extraer regularidades**. El modelo observa ejemplos, ajusta su comportamiento y construye una forma de responder que intenta funcionar más allá de los casos ya vistos.

Por supuesto, este aprendizaje depende profundamente de la calidad de los datos. Si los ejemplos son pocos, incompletos, incorrectos o poco representativos, el modelo puede aprender patrones equivocados. Si todos los correos de entrenamiento están mal clasificados, el modelo aprenderá a partir de etiquetas incorrectas. Si los datos de tránsito provienen de muy pocos usuarios, las estimaciones pueden ser poco confiables. Si los precios de viviendas corresponden solo a una zona específica, el modelo puede fallar al aplicarse a otra ciudad o a otro tipo de propiedad.

Esto nos lleva a una idea que aparecerá muchas veces a lo largo del libro: un modelo de machine learning no aprende en el vacío. Aprende a partir de los datos que le damos. Sus aciertos y errores están estrechamente relacionados con la información

disponible, con cómo fue recolectada, con qué tan bien representa el problema real y con qué decisiones tomamos durante el desarrollo del modelo.

También es importante entender que el aprendizaje de un modelo no garantiza verdad absoluta. Un modelo puede producir predicciones útiles, pero no perfectas. Puede decir que un correo tiene alta probabilidad de ser spam y equivocarse. Puede estimar que un viaje tardará veinte minutos y que luego un accidente inesperado cambie por completo la situación. Puede calcular un precio probable para una vivienda, pero no anticipar una negociación particular entre comprador y vendedor.

Por eso, cuando hablamos de aprendizaje en machine learning, hablamos de un proceso basado en evidencia, probabilidad y patrones, no de certeza total. El modelo aprende una relación aproximada entre los datos de entrada y las respuestas esperadas. Esa relación puede ser muy útil, incluso extremadamente poderosa, pero siempre debe evaluarse con cuidado.

Una buena forma de resumirlo es la siguiente: un modelo aprende de los datos cuando, a partir de ejemplos, ajusta su manera de responder para reconocer patrones y producir resultados útiles ante casos nuevos. Ese aprendizaje no es comprensión humana, sino detección de regularidades. Y su valor depende, en gran medida, de que los datos usados para aprender sean adecuados para el problema que queremos resolver.

Patrones, predicciones y generalización

Para entender mejor cómo funciona el machine learning, conviene detenerse en tres ideas que aparecen una y otra vez: **patrones**, **predicciones** y **generalización**. Son conceptos muy relacionados entre sí. Un modelo aprende patrones en los datos, usa esos patrones para hacer predicciones y resulta realmente útil cuando puede generalizar, es decir, cuando puede funcionar razonablemente bien con casos nuevos.

Un **patrón** es una regularidad que se repite en los datos. No necesariamente es una regla perfecta, ni algo que se cumpla siempre sin excepción. Muchas veces, un patrón es una tendencia. Algo que ocurre con cierta frecuencia, bajo ciertas condiciones, y que permite anticipar parcialmente lo que podría suceder.

Por ejemplo, si analizamos datos de viviendas, podríamos encontrar que las propiedades con mayor superficie suelen tener precios más altos. Esa relación no es absoluta: una casa pequeña en una zona muy demandada puede valer más que una casa grande en una zona menos buscada. Pero aun así existe un patrón útil. La superficie aporta información sobre el precio probable, aunque no sea el único factor.

Lo mismo ocurre con los correos electrónicos. Algunos mensajes con ciertos tipos de enlaces, asuntos llamativos, remitentes extraños o estructuras repetitivas pueden tener mayor probabilidad de ser spam. Ninguna de esas características por separado garantiza una respuesta correcta, pero en conjunto pueden formar un patrón. El modelo no necesita encontrar una condición perfecta; necesita encontrar señales que ayuden a distinguir un caso de otro.

En machine learning, los patrones suelen surgir de la relación entre los datos de entrada y algún resultado que nos interesa. Si queremos estimar precios de viviendas, los datos de entrada pueden ser la superficie, la ubicación, la cantidad de ambientes y la antigüedad, mientras que el resultado esperado es el precio. Si queremos detectar spam, los datos de entrada pueden ser las características del correo, y el resultado esperado es si el mensaje es spam o no. Si queremos predecir riesgo crediticio, los datos de entrada pueden incluir ingresos, historial de pagos, edad de la deuda y otros indicadores, mientras que el resultado buscado puede ser la probabilidad de incumplimiento.

A partir de esos patrones, el modelo produce **predicciones**. En este contexto, predecir no siempre significa anticipar el futuro. A veces sí: por ejemplo, estimar la demanda de un producto para la semana próxima o calcular cuánto tardará un viaje según el tránsito actual. Pero muchas veces “predecir” significa producir una respuesta sobre un caso que todavía no conocemos completamente.

Cuando un sistema decide si un correo nuevo es spam, está haciendo una predicción. Cuando un modelo estima el precio probable de una vivienda, está haciendo una predicción. Cuando una aplicación recomienda una película, también está haciendo una predicción: intenta anticipar qué contenido podría interesarle a una persona. Incluso cuando un sistema clasifica una imagen como “perro”, “gato” o “auto”, está prediciendo cuál es la categoría más probable para esa imagen.

Por eso, en machine learning, la palabra predicción debe entenderse en un sentido amplio. Una predicción puede ser un número, como el precio estimado de una casa. Puede ser una categoría, como “spam” o “no spam”. Puede ser una probabilidad, como “hay un 80% de probabilidad de que este cliente abandone el servicio”. Puede ser una recomendación, como una serie, un producto o una canción.

Ahora bien, encontrar patrones en datos conocidos y hacer predicciones sobre esos mismos datos no alcanza. Un modelo podría funcionar muy bien con los ejemplos que ya vio, pero fallar cuando se enfrenta a situaciones nuevas. Por eso aparece una tercera idea fundamental: la **generalización**.

Generalizar significa aplicar lo aprendido a casos nuevos, distintos de los usados durante el aprendizaje. Es una de las condiciones más importantes para que un

modelo de machine learning sea útil. No nos interesa que el modelo simplemente “recuerde” los ejemplos anteriores; nos interesa que pueda usar lo aprendido para responder frente a datos que no estaban en el conjunto original.

Imaginemos un estudiante que memoriza las respuestas exactas de diez ejercicios de matemática, pero no entiende el procedimiento. Si en la evaluación aparecen los mismos ejercicios, puede responder bien. Pero si aparecen ejercicios parecidos con otros números, probablemente falle. Algo similar puede ocurrir con un modelo. Si solo memoriza los datos de entrenamiento, puede parecer muy bueno al evaluarlo sobre esos mismos casos, pero comportarse mal ante ejemplos nuevos.

En cambio, si el estudiante comprende la idea detrás del procedimiento, puede resolver ejercicios nuevos. En machine learning buscamos algo parecido: que el modelo no copie simplemente los ejemplos, sino que capture regularidades útiles. Esa capacidad de funcionar con casos nuevos es lo que llamamos generalización.

Veamos un ejemplo. Supongamos que entrenamos un modelo para estimar precios de viviendas usando datos de ventas anteriores. Durante el aprendizaje, el modelo observa muchas propiedades: algunas grandes, otras pequeñas; algunas céntricas, otras alejadas; algunas nuevas, otras antiguas. Si el modelo aprende relaciones útiles, luego podrá recibir los datos de una vivienda que no estaba en el conjunto original y producir una estimación razonable de su precio.

Pero si el modelo se limita a memorizar casos específicos, tendrá problemas. Tal vez recuerde que una vivienda de 80 metros cuadrados en cierto barrio se vendió a determinado precio, pero no sabrá qué hacer con una vivienda de 82 metros cuadrados, en una calle cercana, con otra distribución y otro estado de conservación. La utilidad del modelo depende de que pueda moverse más allá de los ejemplos exactos.

La generalización también es central en la detección de spam. Los mensajes no deseados cambian todo el tiempo. Quienes los producen modifican palabras, formatos, remitentes y estrategias para evitar ser detectados. Un modelo útil no puede depender solamente de una lista fija de frases conocidas. Debe aprender patrones más generales que le permitan identificar mensajes sospechosos aunque no sean idénticos a los anteriores.

Esto no significa que un modelo pueda generalizar a cualquier situación. Un modelo aprende a partir de los datos disponibles, y su capacidad de generalización suele estar limitada por ellos. Si entrenamos un modelo de precios de viviendas usando solo datos de una ciudad, no podemos asumir que funcionará igual de bien en otro país, con otro mercado inmobiliario y otras condiciones económicas. Si entrenamos un detector de spam solo con correos en un idioma, puede fallar al recibir mensajes

en otro. Si un sistema de diagnóstico asistido fue entrenado con datos de una población muy específica, puede no comportarse igual en pacientes con características diferentes.

Por eso, generalizar no significa “acertar siempre”. Significa que el modelo logró aprender regularidades que se mantienen, al menos en parte, cuando aparecen nuevos casos similares al problema original. La palabra “similares” es importante. Un modelo suele funcionar mejor cuando los datos nuevos se parecen, en aspectos relevantes, a los datos con los que fue entrenado.

Aquí aparece una tensión muy importante, que se verá con más detalle más adelante: un modelo debe aprender lo suficiente como para reconocer patrones reales, pero no tanto detalle irrelevante como para quedar atado a los ejemplos exactos que vio. Si aprende demasiado poco, sus predicciones serán pobres. Si memoriza demasiado, puede parecer preciso en los datos conocidos, pero fallar en la práctica. Por ahora, alcanza con dejar planteada esta idea: el objetivo no es recordar, sino aprender patrones que sirvan para responder ante nuevos casos.

En la vida real, la generalización es lo que convierte al machine learning en una herramienta valiosa. Una empresa no entrena un modelo para saber lo que ya ocurrió, sino para tomar mejores decisiones sobre situaciones nuevas. Un banco quiere estimar el riesgo de un nuevo solicitante de crédito. Una plataforma quiere recomendar contenido a un usuario en su próxima sesión. Una aplicación de mapas quiere estimar cuánto tardará un viaje que todavía está ocurriendo. Un sistema médico de apoyo quiere señalar posibles riesgos en un paciente actual, no solo describir pacientes del pasado.

Por eso, patrones, predicciones y generalización forman una cadena conceptual muy importante. Los datos contienen patrones. El modelo intenta aprender esos patrones. Luego utiliza lo aprendido para producir predicciones. Y esas predicciones solo son realmente útiles si el modelo puede generalizar a casos nuevos.

Una forma sencilla de resumirlo sería decir que el machine learning no busca solamente encontrar relaciones en datos históricos. Busca usar esas relaciones para actuar sobre nuevos datos. Esa diferencia es fundamental: un modelo no se entrena para explicar únicamente el pasado, sino para ofrecer respuestas razonables frente a situaciones que todavía no conoce.

Qué puede resolver el Machine Learning y qué no puede resolver

Una forma frecuente de presentar el machine learning es a través de ejemplos llamativos: sistemas que recomiendan contenidos, modelos que predicen precios,

aplicaciones que reconocen imágenes, herramientas que detectan transacciones sospechosas o algoritmos que ayudan a priorizar diagnósticos. Estos casos son importantes porque muestran el alcance práctico de la disciplina. Sin embargo, también pueden generar una impresión equivocada: la idea de que el machine learning puede resolver cualquier problema si se le entregan suficientes datos.

Esa idea es peligrosa. El machine learning es una herramienta poderosa, pero no mágica. Funciona especialmente bien en ciertos tipos de problemas y resulta inadecuado, innecesario o incluso riesgoso en otros. Para comprender cuándo tiene sentido usarlo, conviene mirar no solo los ejemplos, sino la estructura del problema que queremos resolver.

En términos generales, el machine learning puede ser útil cuando existe una relación entre los datos disponibles y la respuesta que queremos obtener, pero esa relación es demasiado compleja, variable o difícil de expresar mediante reglas manuales. Es decir, hay señales en los datos que permiten aprender algo, aunque esas señales no sean evidentes a simple vista o no puedan resumirse fácilmente en una fórmula simple.

Por ejemplo, si una organización quiere estimar la demanda futura de un producto, puede reunir datos de ventas anteriores, fechas, temporadas, promociones, precios, ubicación de los locales y otros factores. Tal vez no exista una regla sencilla que diga exactamente cuántas unidades se venderán la semana próxima, pero sí pueden existir patrones históricos útiles. Si ciertos productos aumentan sus ventas en determinados meses, si algunas promociones tienen efectos repetidos o si ciertos días de la semana muestran comportamientos similares, un modelo puede aprender parte de esas relaciones y producir estimaciones razonables.

Lo mismo ocurre cuando se quiere segmentar clientes según comportamientos de compra, estimar la probabilidad de abandono de un servicio, detectar consumos anómalos de energía, priorizar reclamos, clasificar documentos, reconocer defectos en imágenes industriales o anticipar fallas en una máquina a partir de sensores. En todos estos casos, el machine learning puede aportar valor porque trabaja con datos en los que existen regularidades. No necesita que esas regularidades sean perfectas; necesita que sean lo suficientemente estables como para que aprenderlas ayude a mejorar una decisión.

Desde un punto de vista más teórico, el machine learning necesita tres condiciones básicas para tener sentido. Primero, debe existir algún patrón en el fenómeno que se quiere modelar. Si los resultados son completamente aleatorios, no hay nada que aprender. Segundo, deben existir datos que capturen, al menos parcialmente, ese patrón. Si la información disponible no contiene señales relevantes, el modelo no puede inventarlas. Tercero, el patrón aprendido debe tener cierta continuidad entre

los datos usados para entrenar el modelo y los casos nuevos donde se lo quiere aplicar. Si el mundo cambia por completo, lo aprendido puede dejar de servir.



Machine Learning necesita tres condiciones básicas para tener sentido.

Estas tres condiciones ayudan a distinguir problemas adecuados de problemas inadecuados. El machine learning puede aprender relaciones aproximadas, tendencias, asociaciones y probabilidades. Puede ayudarnos cuando el problema tiene incertidumbre, pero no cuando no hay información suficiente para reducir esa incertidumbre. Puede encontrar patrones complejos, pero no puede crear conocimiento verdadero a partir de datos vacíos, erróneos o irrelevantes.

Por eso, una primera limitación importante es que el machine learning no resuelve problemas donde no hay datos útiles. Si queremos predecir algo, pero no tenemos registros previos, mediciones confiables o variables relacionadas con el resultado, el modelo no tiene una base desde la cual aprender. Podríamos entrenar un algoritmo de todos modos, pero sus respuestas serían frágiles o engañosas. El hecho de usar una herramienta sofisticada no compensa la ausencia de información.

Una segunda limitación es que el machine learning no reemplaza la definición del problema. Antes de entrenar un modelo, alguien debe decidir qué se quiere resolver, qué datos se van a usar, qué resultado se considera correcto, qué errores son aceptables y cómo se evaluará si el sistema funciona. Un modelo puede optimizar una tarea definida, pero no puede decidir por sí mismo cuál es el objetivo adecuado en un contexto humano, institucional o ético.

Esto es especialmente importante en problemas sensibles. Por ejemplo, si una institución quiere usar datos para priorizar la asignación de recursos, no alcanza con entrenar un modelo que produzca una lista ordenada. Hay que discutir qué significa “priorizar”, qué criterios son legítimos, qué sesgos pueden existir en los datos, qué consecuencias puede tener una decisión equivocada y quién será responsable de revisar los resultados. El machine learning puede asistir una decisión, pero no elimina la necesidad de criterio humano.

Una tercera limitación es que el machine learning no garantiza causalidad. Un modelo puede encontrar que ciertas variables están asociadas con un resultado, pero eso no significa necesariamente que una cause la otra. Esta distinción es

fundamental. Si un modelo observa que dos fenómenos aparecen juntos con frecuencia, puede usar esa relación para predecir. Pero predecir bien no siempre equivale a entender por qué ocurre algo.

Por ejemplo, un sistema podría encontrar que cierto comportamiento de los usuarios se asocia con una mayor probabilidad de cancelar una suscripción. Esa relación puede servir para anticipar abandonos, pero no demuestra automáticamente cuál es la causa. Tal vez el comportamiento detectado sea una consecuencia del desinterés, no su origen. Tal vez dependa de otro factor no registrado. El modelo puede ser útil para señalar un riesgo, pero no necesariamente explica el mecanismo profundo que lo produce.

También hay problemas que no requieren machine learning. Si una tarea puede resolverse mediante una regla clara, estable y conocida, usar machine learning suele ser innecesario. Calcular un recargo, aplicar una condición legal explícita, convertir unidades, validar un formato o ejecutar una fórmula definida son casos donde la programación tradicional es más simple, transparente y confiable. No hay ventaja en entrenar un modelo para descubrir una regla que ya conocemos.

Esta idea es importante porque a veces se confunde complejidad técnica con valor. Un sistema más sofisticado no siempre es mejor. Si una solución basada en reglas es suficiente, más fácil de auditar y menos propensa a errores inesperados, entonces probablemente sea la solución adecuada. El machine learning conviene cuando aporta una capacidad que las reglas manuales no pueden ofrecer con la misma flexibilidad o precisión.

Otra frontera importante aparece en los problemas donde el futuro no se parece al pasado. Como los modelos aprenden a partir de datos históricos o disponibles, su comportamiento depende de que esos datos sigan siendo relevantes. Si cambia el contexto económico, social, tecnológico o cultural, los patrones aprendidos pueden debilitarse. Un modelo que funcionaba bien durante cierto período puede perder calidad cuando aparecen nuevas condiciones.

Esto no significa que el machine learning sea inútil en contextos cambiantes, pero sí que requiere vigilancia, evaluación y actualización. Un modelo no queda “terminado” para siempre. Si el fenómeno que intenta representar cambia, el modelo debe revisarse. La idea de aprender de datos no elimina el paso del tiempo; al contrario, lo vuelve central.

También debemos reconocer que el machine learning no puede resolver por sí solo problemas mal formulados. Si el objetivo es ambiguo, contradictorio o imposible de medir, el modelo hereda esa confusión. Preguntas como “¿quién merece una oportunidad?”, “¿qué contenido es valioso?” o “¿qué decisión es justa?” no se

convierten automáticamente en problemas técnicos por estar acompañadas de datos. Pueden tener componentes medibles, pero también involucran valores, criterios sociales y decisiones humanas.

Por eso, una parte importante del trabajo con machine learning consiste en traducir una necesidad real en una tarea que pueda ser abordada con datos, sin perder de vista lo que queda fuera del modelo. Esa traducción nunca es neutral. Elegir qué se mide, qué se ignora, qué se considera éxito y qué se considera error forma parte del diseño del sistema.

En resumen, el machine learning puede resolver problemas donde existen datos relevantes, patrones aprendibles y una tarea suficientemente definida. Puede ayudar a clasificar, estimar, recomendar, detectar anomalías, agrupar comportamientos o anticipar ciertos resultados. Es especialmente útil cuando las reglas son difíciles de escribir manualmente, cuando hay muchos factores interactuando y cuando se dispone de ejemplos suficientes para aprender de ellos.

Pero no puede resolver problemas donde no hay información significativa, donde el resultado es puramente aleatorio, donde el objetivo no está claro o donde se necesita una regla exacta ya conocida. Tampoco reemplaza la comprensión del contexto, el juicio humano, la responsabilidad ética ni el análisis causal profundo.

Una buena pregunta antes de usar machine learning no es “¿podemos aplicar un modelo a esto?”, porque casi siempre es posible aplicar algún modelo. La pregunta más importante es: “¿este problema tiene datos adecuados, patrones que puedan aprenderse y una forma razonable de evaluar si el modelo ayuda?”. Si la respuesta es sí, el machine learning puede ser una herramienta valiosa. Si la respuesta es no, probablemente necesitemos otro enfoque, o tal vez debemos trabajar primero en comprender mejor el problema y construir mejores datos.

El rol central de los datos

Si hay una idea que conviene dejar clara desde el comienzo, es esta: en machine learning, los datos no son un elemento secundario. Son el punto de partida, el material de trabajo y una de las principales condiciones que determinan la calidad del resultado final.

Un modelo de machine learning aprende a partir de datos. Esto significa que sus respuestas no surgen de una comprensión abstracta del mundo, sino de los ejemplos, registros, mediciones o descripciones que se le presentan durante el proceso de entrenamiento. Por eso, cuando un modelo funciona bien, no se debe solamente al algoritmo utilizado. También se debe, y muchas veces sobre todo, a que los datos

contienen información relevante, están suficientemente bien preparados y representan de manera razonable el problema que se quiere resolver.

Esta idea puede parecer sencilla, pero tiene consecuencias muy importantes. Un algoritmo avanzado no compensa automáticamente datos pobres. Si los datos son incompletos, confusos, erróneos, contradictorios o poco representativos, el modelo aprenderá a partir de esas limitaciones. Puede encontrar patrones donde solo hay ruido, reproducir errores de medición, amplificar sesgos existentes o comportarse mal cuando se lo aplique a casos nuevos.

En este sentido, los datos funcionan como una especie de experiencia previa para el modelo. Así como una persona que solo conoce una parte muy limitada de una realidad puede sacar conclusiones equivocadas, un modelo entrenado con datos parciales puede aprender una versión incompleta del problema. No porque el modelo “quiera” equivocarse, sino porque su aprendizaje depende de aquello que los datos le permiten ver.

Por ejemplo, si queremos construir un modelo para estimar la demanda de un servicio, los datos deberían reflejar distintos períodos, condiciones, niveles de actividad y situaciones relevantes. Si solo tenemos registros de una temporada excepcional, el modelo puede aprender patrones que no se sostienen en condiciones normales. Si queremos desarrollar un sistema de clasificación de reclamos, pero los reclamos históricos fueron registrados de forma desordenada o con criterios cambiantes, el modelo puede confundirse. Si queremos anticipar fallas en equipos, pero solo registramos los casos exitosos y no los problemas, la información disponible será insuficiente.

Esto muestra que los datos no son simplemente “números en una tabla”. Son una representación de algún aspecto de la realidad. Y toda representación tiene límites. Puede dejar cosas afuera, puede estar construida con errores, puede reflejar decisiones humanas anteriores, puede estar condicionada por la forma en que se recolectó la información o por los objetivos de la institución que la generó.

Por eso, antes de pensar en modelos, es necesario preguntarse por los datos. ¿De dónde vienen? ¿Quién los registró? ¿Con qué criterio? ¿Qué significan sus columnas? ¿Qué período cubren? ¿Qué casos incluyen y cuáles quedan afuera? ¿Hay valores faltantes? ¿Hay errores evidentes? ¿Las categorías son consistentes? ¿La información disponible se relaciona realmente con el problema que queremos resolver?

Muchas de estas preguntas ya pertenecen al terreno de la ciencia de datos y del análisis exploratorio. En los tomos anteriores trabajamos justamente con herramientas para cargar, inspeccionar, limpiar, resumir, agrupar y visualizar datos. En machine learning, esas habilidades no desaparecen; al contrario, se vuelven

indispensables. No hay una frontera rígida donde termina el análisis de datos y empieza el aprendizaje automático. En la práctica, un proyecto de machine learning se apoya constantemente en tareas de exploración, preparación e interpretación de datos.

También es importante distinguir entre tener muchos datos y tener buenos datos. La cantidad puede ayudar, pero no garantiza calidad. Un conjunto de datos grande puede estar lleno de registros duplicados, variables irrelevantes, mediciones incorrectas o información que no representa el problema actual. En cambio, un conjunto más pequeño pero bien construido, coherente y relacionado con la pregunta que queremos responder puede ser mucho más útil.

Esto no significa que la cantidad no importe. En muchos problemas, contar con más ejemplos permite que el modelo observe mayor variedad de situaciones y aprenda patrones más estables. Pero la cantidad solo aporta valor si los datos son pertinentes. Más datos defectuosos pueden llevar a errores más persistentes, no necesariamente a mejores modelos.

Otro aspecto central es que los datos determinan qué puede aprender el modelo. Un modelo no puede utilizar información que no está presente, ni recuperar por sí solo una variable que nunca fue registrada. Si queremos predecir cierto resultado, pero los datos disponibles no contienen señales relacionadas con ese resultado, el aprendizaje será débil. Esto es una limitación fundamental: el modelo no inventa evidencia. Trabaja con la información que recibe.

Además, los datos influyen en los tipos de errores que puede cometer un modelo. Si ciertos grupos, situaciones o casos aparecen poco representados, el modelo puede funcionar peor para ellos. Si los datos históricos reflejan decisiones injustas o criterios sesgados, el modelo puede aprender esos patrones como si fueran normales. Por eso, trabajar con datos en machine learning no es solo una cuestión técnica; también implica responsabilidad. Hay que observar qué mundo está siendo representado en los datos y qué consecuencias puede tener usar esa representación para tomar decisiones.

En este punto conviene evitar una mirada ingenua. Los datos no son neutrales por el solo hecho de estar almacenados en una tabla. Son el resultado de procesos de selección, medición, clasificación y registro. Alguien decidió qué medir, cómo medirlo, cuándo registrarlo y con qué categorías describirlo. Incluso los datos generados automáticamente dependen de sensores, sistemas, diseños de software y criterios previos.

Por eso, un buen proyecto de machine learning no empieza entrenando un modelo, sino comprendiendo los datos. Esa comprensión incluye aspectos técnicos, como

tipos de variables, valores faltantes, distribución de los datos o presencia de valores atípicos. Pero también incluye aspectos conceptuales: qué significa cada variable, qué representa cada fila, qué historia hay detrás de los registros y qué relación existe entre los datos y el problema real.

A medida que avancemos en el libro, veremos métodos concretos para preparar datos y entrenar modelos. Pero desde ahora es importante sostener esta idea: el modelo es solo una parte del sistema. Los datos, el problema, el contexto y la forma de evaluar los resultados son igualmente importantes.

En machine learning, una tabla no es solo una tabla. Es la forma en que el modelo va a conocer el problema. Si esa representación es pobre, el aprendizaje también lo será. Si es cuidadosa, relevante y suficientemente representativa, el modelo tendrá mejores condiciones para encontrar patrones útiles.

Por eso, el rol central de los datos puede resumirse así: los datos definen qué experiencia recibe el modelo, qué patrones puede encontrar, qué límites tendrá su aprendizaje y qué tipo de decisiones podrá apoyar. Antes de confiar en un modelo, tenemos que confiar razonablemente en los datos que lo formaron.

Algunos ejemplos del mundo real

Para terminar de comprender qué tipo de problemas puede abordar el machine learning, conviene recorrer algunos ejemplos concretos. No se trata todavía de estudiar cómo se construye cada modelo, qué algoritmo se utiliza o cómo se evalúan sus resultados. Eso aparecerá más adelante. Por ahora, lo importante es observar qué tienen en común estos casos: en todos ellos existen datos, patrones posibles y una tarea que puede formularse como una predicción, una clasificación, una estimación o una recomendación.

Uno de los ejemplos más conocidos es la detección de **correo no deseado o spam**. Cada vez que recibimos un correo electrónico, el sistema puede analizar distintas características del mensaje: palabras utilizadas, asunto, remitente, enlaces incluidos, archivos adjuntos, estructura del texto, idioma, frecuencia de envío o comportamiento previo de mensajes similares. A partir de muchos correos ya clasificados como legítimos o no deseados, un modelo puede aprender patrones que le permitan estimar si un nuevo mensaje probablemente sea spam.

Este ejemplo es útil porque muestra que el modelo no necesita una única regla perfecta. No alcanza con decir que todo mensaje con determinada palabra es spam, ni que todo mensaje enviado desde cierto dominio debe bloquearse. El problema es más sutil. Un correo legítimo puede contener palabras promocionales, y un correo malicioso puede estar redactado con cuidado para parecer confiable. El modelo

trabaja combinando señales. Algunas serán fuertes, otras débiles, y muchas solo serán útiles cuando aparecen junto con otras características.

Otro caso muy presente en la vida cotidiana son los sistemas de **recomendación**. Plataformas de películas, música, libros, cursos, noticias o compras en línea utilizan modelos para sugerir contenidos o productos que podrían interesarle a cada usuario. Para hacerlo, pueden analizar datos como reproducciones anteriores, búsquedas, valoraciones, tiempo de visualización, productos comprados, contenidos guardados, elementos ignorados o comportamientos de usuarios con gustos similares.

Cuando una plataforma recomienda una película, no está “conociendo” al usuario en un sentido humano. Está estimando, a partir de datos, qué contenido tiene mayor probabilidad de resultar relevante. Puede encontrar patrones como: usuarios que vieron ciertos documentales también suelen ver determinadas series; personas que compraron un producto suelen necesitar otro complementario; estudiantes que completaron cierto curso suelen interesarse por un tema relacionado.

Las recomendaciones muestran una característica importante del machine learning: muchas veces no se busca una única respuesta correcta, sino una respuesta razonablemente útil. No existe una verdad absoluta sobre cuál es “la película correcta” para una persona. El sistema intenta ordenar opciones según una probabilidad de interés. Puede acertar, puede equivocarse y puede mejorar a medida que recibe más señales del comportamiento del usuario.

Un tercer ejemplo es la estimación de **precios**. El precio de una vivienda, de un automóvil usado, de un producto o de un servicio puede depender de muchos factores. En el caso de una propiedad, pueden intervenir la ubicación, la superficie, la cantidad de ambientes, la antigüedad, el estado general, la cercanía a medios de transporte, la disponibilidad de cochera, la demanda de la zona y el momento del mercado. En el caso de un vehículo, podrían importar la marca, el modelo, el año, el kilometraje, el estado, la versión y el historial de mantenimiento.

Estos problemas son adecuados para machine learning cuando existen registros históricos suficientes. Si contamos con muchos ejemplos de propiedades vendidas, junto con sus características y precios finales, un modelo puede aprender relaciones entre esas características y el valor de mercado. No encontrará una fórmula universal e infalible, pero puede producir una estimación útil.

Este tipo de aplicación también permite ver una diferencia importante entre una predicción y una decisión. Un modelo puede estimar que una vivienda vale cierta cantidad, pero eso no significa que ese será necesariamente el precio final. El valor real puede depender de una negociación, de una urgencia de venta, de una situación económica particular o de factores difíciles de registrar. El modelo aporta una

referencia basada en datos, no reemplaza por completo el juicio humano ni las condiciones concretas del contexto.

Otro uso frecuente aparece en el análisis de **riesgo crediticio**. Cuando una persona o una empresa solicita un crédito, una institución necesita estimar la probabilidad de que ese crédito sea devuelto en tiempo y forma. Para eso puede analizar ingresos, historial de pagos, deudas previas, antigüedad laboral, relación entre cuota e ingreso, comportamiento financiero anterior y otras variables disponibles.

En este caso, el modelo intenta aprender patrones asociados con pagos cumplidos o incumplimientos. Si en los datos históricos ciertas combinaciones de factores aparecen con mayor frecuencia en casos de mora, el modelo puede asignar un nivel de riesgo más alto a solicitudes similares. Si otras combinaciones aparecen asociadas con buen comportamiento de pago, puede estimar un riesgo menor.

Este ejemplo es especialmente importante porque muestra que el machine learning puede tener consecuencias significativas sobre la vida de las personas. Un modelo de riesgo crediticio puede influir en la aprobación de un préstamo, en el monto ofrecido o en las condiciones financieras. Por eso, no alcanza con que el modelo sea técnicamente preciso. También es necesario preguntarse qué datos utiliza, si esos datos son justos y representativos, qué errores puede cometer, cómo se revisan sus decisiones y qué lugar ocupa la supervisión humana.

Un quinto ejemplo es el **diagnóstico asistido** en contextos médicos. En este tipo de aplicaciones, los modelos pueden ayudar a analizar imágenes, resultados de laboratorio, historias clínicas, señales fisiológicas o síntomas registrados, con el objetivo de detectar indicios de una enfermedad, priorizar casos o apoyar el trabajo profesional.

Por ejemplo, un modelo podría analizar imágenes médicas para señalar zonas sospechosas que deberían ser revisadas por un especialista. Otro modelo podría identificar pacientes con mayor probabilidad de desarrollar una complicación, a partir de variables clínicas disponibles. En estos casos, el machine learning no debería entenderse como un reemplazo del criterio médico, sino como una herramienta de apoyo.

Este ejemplo también permite ver con claridad los límites y responsabilidades del uso de modelos. Un error en un sistema de recomendación puede sugerir una película poco interesante. Un error en un contexto médico puede tener consecuencias mucho más graves. Por eso, en áreas sensibles se requieren datos de alta calidad, evaluaciones rigurosas, controles profesionales, criterios éticos y mucha prudencia al interpretar los resultados.

A pesar de sus diferencias, todos estos ejemplos comparten una misma estructura. Hay datos disponibles sobre casos anteriores o situaciones actuales. Hay una tarea que se quiere resolver: clasificar un correo, recomendar un contenido, estimar un precio, calcular un riesgo o asistir un diagnóstico. Y hay patrones que pueden aprenderse a partir de la relación entre los datos y los resultados observados.

También comparten una misma advertencia: el modelo no trabaja fuera del contexto. Sus respuestas dependen de los datos con los que fue entrenado, de cómo se definió el problema, de qué variables se tuvieron en cuenta y de cómo se evalúan sus errores. El machine learning puede ser muy valioso, pero su valor no está solamente en el algoritmo. Está en la relación entre el problema, los datos, el modelo y el uso responsable de sus resultados.

Capítulo 2 · Tipos de problemas en Machine Learning

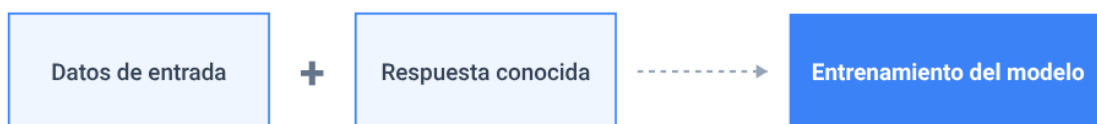
Aprendizaje supervisado

Una de las primeras formas de ordenar el mundo del machine learning es observar **cómo aprende un modelo**. No todos los problemas se presentan de la misma manera, y no todos los datos ofrecen el mismo tipo de información. En algunos casos, contamos con ejemplos donde ya conocemos la respuesta correcta. En otros, solo tenemos datos sin una respuesta explícita. Esa diferencia cambia por completo el tipo de aprendizaje que podemos utilizar.

El **aprendizaje supervisado** es el tipo de aprendizaje que usamos cuando tenemos datos de entrada y, además, conocemos el resultado que corresponde a cada caso. Dicho de otro modo: el modelo aprende a partir de ejemplos ya resueltos.

Imaginemos una tabla con información de muchas viviendas vendidas. Para cada vivienda tenemos datos como la superficie, la cantidad de ambientes, el barrio, la antigüedad, si tiene cochera o no, y también conocemos el precio final de venta. En ese caso, el modelo puede observar la relación entre las características de cada vivienda y su precio. Luego, cuando reciba los datos de una vivienda nueva cuyo precio todavía no conocemos, intentará estimarlo a partir de lo aprendido.

Entrenamiento



Predicción



Antes y después del entrenamiento.

Esa “respuesta conocida” es una pieza central del aprendizaje supervisado. Puede recibir distintos nombres según el contexto: **variable objetivo**, **resultado esperado**, **etiqueta** o, en inglés, *target*. No hace falta obsesionarse todavía con la terminología, pero sí conviene comprender la idea: el modelo aprende porque puede comparar los datos de cada ejemplo con una respuesta que ya conocemos.

Por eso se llama “supervisado”. No significa que una persona esté corrigiendo manualmente cada paso del modelo mientras aprende, sino que los datos incluyen una especie de guía: para cada ejemplo, sabemos cuál debería ser la salida correcta.

Un ejemplo clásico es la detección de correos no deseados. Si tenemos miles de correos ya clasificados como “spam” o “no spam”, podemos entrenar un modelo para que observe sus características: palabras utilizadas, remitentes, enlaces, asuntos, longitud del texto, frecuencia de ciertos términos, entre otras. A partir de esos ejemplos, el modelo aprende patrones asociados con cada categoría. Luego, cuando llega un correo nuevo, intenta predecir si pertenece a la categoría “spam” o “no spam”.

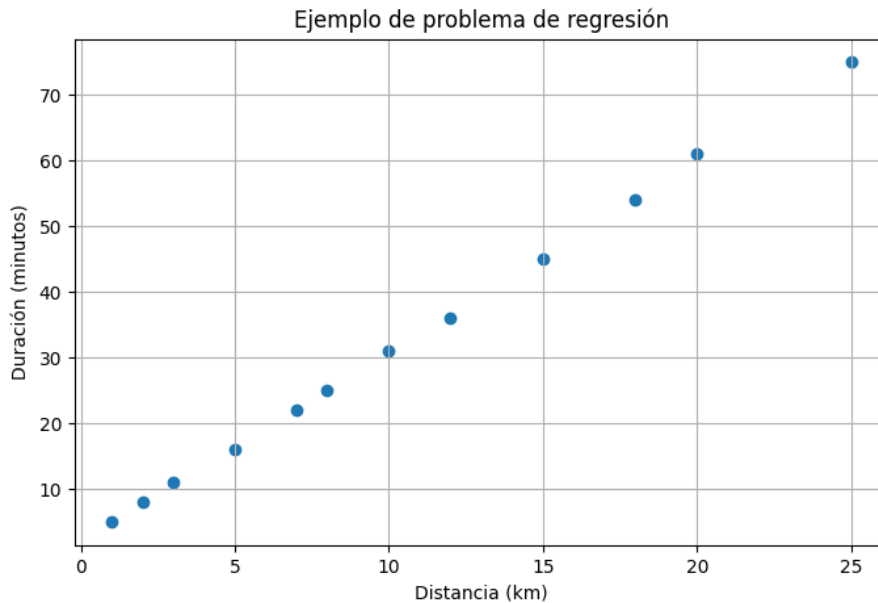
Otro ejemplo puede ser la predicción de abandono de clientes. Una empresa puede tener datos históricos de usuarios: antigüedad, frecuencia de uso, cantidad de reclamos, cambios en el consumo, pagos realizados, interacción con promociones y otros indicadores. Si además sabe qué clientes abandonaron el servicio y cuáles continuaron, puede entrenar un modelo supervisado. El objetivo sería estimar qué clientes actuales tienen mayor probabilidad de abandonar en el futuro.

En ambos casos, el aprendizaje supervisado se apoya en la misma lógica: usar casos pasados donde el resultado ya se conoce para aprender una relación que pueda aplicarse a casos nuevos.

Dentro del aprendizaje supervisado aparecen dos grandes familias de problemas: **clasificación** y **regresión**.

En los problemas de **clasificación**, la respuesta que queremos predecir es una categoría. Por ejemplo, “spam” o “no spam”, “cliente que abandona” o “cliente que permanece”, “riesgo bajo”, “riesgo medio” o “riesgo alto”. El modelo debe elegir entre un conjunto de clases posibles.

En los problemas de **regresión**, la respuesta que queremos predecir es un valor numérico. Por ejemplo, el precio de una vivienda, la demanda esperada de un producto, el consumo eléctrico de una fábrica o el tiempo estimado de entrega de un pedido. En estos casos, el modelo no asigna una categoría, sino que estima una cantidad.



En general, los viajes más largos duran más minutos.

Más adelante vamos a estudiar clasificación y regresión con mayor detalle. Por ahora, lo importante es reconocer que ambas pertenecen al aprendizaje supervisado porque en los dos casos necesitamos ejemplos previos con respuestas conocidas.

El aprendizaje supervisado es muy útil cuando queremos construir sistemas capaces de predecir resultados futuros o desconocidos a partir de experiencia acumulada. Si una organización tiene datos históricos confiables, puede usarlos para entrenar modelos que ayuden a tomar mejores decisiones. Pero esta posibilidad depende de una condición fundamental: que los datos realmente contengan información útil sobre el problema.

No alcanza con tener una tabla grande. Para que el aprendizaje supervisado tenga sentido, la variable objetivo debe estar bien definida y los datos de entrada deben estar relacionados con ella. Si queremos predecir el precio de una vivienda, necesitamos variables que ayuden a explicar ese precio. Si queremos estimar el abandono de clientes, necesitamos señales del comportamiento de esos clientes antes de que abandonen. Si queremos clasificar correos, necesitamos características del mensaje que puedan asociarse con una u otra categoría.

También es importante que las respuestas conocidas sean confiables. Si los datos históricos contienen errores, el modelo puede aprender esos errores. Si muchos correos fueron etiquetados incorrectamente, el modelo aprenderá a partir de una guía defectuosa. Si los precios registrados no corresponden a operaciones reales, las estimaciones futuras pueden ser débiles. Si los diagnósticos usados para entrenar un sistema médico no fueron confirmados adecuadamente, el modelo puede incorporar esa incertidumbre.

Además, los ejemplos deben ser representativos del tipo de casos donde luego queremos usar el modelo. Un modelo entrenado solo con datos de una región puede no funcionar igual en otra. Un modelo entrenado con datos de una época muy particular puede fallar si cambian las condiciones. Un modelo entrenado con pocos casos de cierto grupo puede tener peor desempeño justamente en ese grupo.

Esto muestra que el aprendizaje supervisado no es simplemente “darle datos a un algoritmo”. Implica definir con cuidado qué queremos predecir, qué ejemplos tenemos, qué tan confiables son las respuestas conocidas y qué tan parecidos serán los casos futuros a los casos usados durante el entrenamiento.

También conviene señalar una limitación importante: el aprendizaje supervisado aprende a partir de lo que ya fue registrado. Si una variable importante nunca fue medida, el modelo no podrá usarla. Si una decisión histórica estuvo sesgada, el modelo puede reproducir ese sesgo. Si el contexto cambia demasiado, los patrones aprendidos pueden perder validez. Por eso, aunque el aprendizaje supervisado es una de las herramientas más poderosas del machine learning, siempre debe estar acompañado por análisis, criterio y evaluación.

El aprendizaje supervisado es una forma de machine learning en la que el modelo aprende a partir de ejemplos con respuestas conocidas. Es adecuado cuando queremos predecir una categoría o un valor numérico para casos nuevos, siempre que tengamos datos históricos relevantes, una variable objetivo clara y ejemplos suficientemente confiables. Su potencia está en convertir casos ya resueltos en una capacidad de predicción; su límite está en que solo puede aprender aquello que los datos permiten aprender.

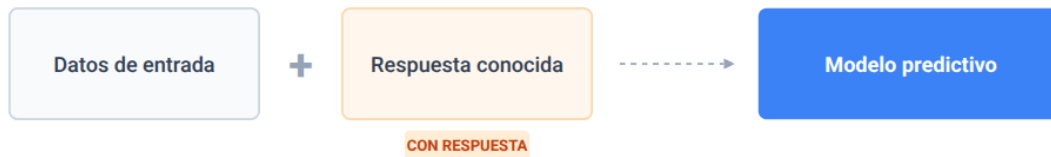
Aprendizaje no supervisado

El **aprendizaje no supervisado** es otra forma importante de machine learning, pero parte de una situación diferente a la del aprendizaje supervisado. En este caso, tenemos datos, pero no tenemos una respuesta conocida para cada ejemplo. Es decir, no contamos con una variable objetivo que indique qué debería predecir el modelo.

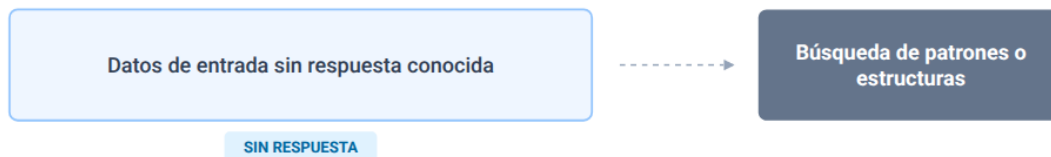
En el aprendizaje supervisado, los datos vienen acompañados por una especie de guía: sabemos cuál es la respuesta correcta para cada caso. En cambio, en el aprendizaje no supervisado, el modelo recibe los datos y debe buscar por sí mismo alguna estructura interna: grupos, similitudes, patrones, relaciones o comportamientos poco habituales.

La diferencia puede resumirse así:

Aprendizaje supervisado



Aprendizaje no supervisado



Aprendizaje Supervisado vs. Aprendizaje No Supervisado.

Imaginemos una empresa que tiene información de muchos clientes: edad, ubicación, frecuencia de compra, monto promedio gastado, categorías de productos elegidas, antigüedad como cliente y canales de contacto utilizados. Pero supongamos que no tiene una etiqueta previa que diga “este cliente pertenece al grupo A”, “este al grupo B” o “este abandonará el servicio”. Solo tiene los datos.

En ese caso, un enfoque no supervisado podría ayudar a descubrir grupos de clientes con comportamientos parecidos. Tal vez aparezca un grupo de clientes que compren poco, pero de manera muy frecuente; otro que compra pocas veces al año, pero con tickets altos; otro que solo responde a promociones; y otro que consume productos muy específicos. Esos grupos no estaban definidos de antemano. El modelo los propone a partir de las similitudes que encuentra en los datos.

Este tipo de problema se conoce como **agrupamiento** o *clustering*, y lo veremos más adelante con mayor detalle. Por ahora, lo importante es entender que el modelo no está intentando predecir una respuesta conocida. Está intentando organizar los datos de una manera útil.

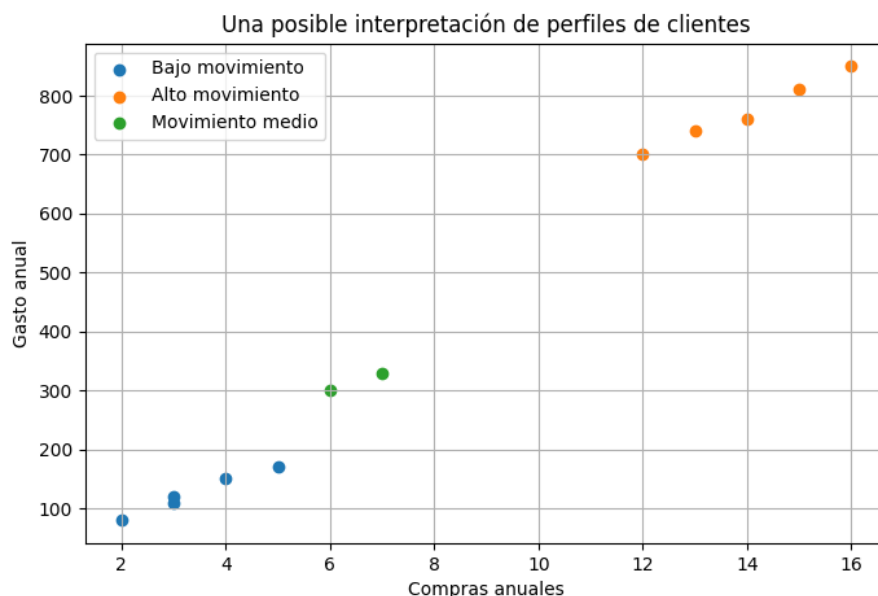
Otro ejemplo podría aparecer en una biblioteca digital o una plataforma educativa. Supongamos que tenemos información sobre los materiales que consultan los estudiantes, el tiempo que dedican a cada recurso, los temas que visitan con más frecuencia y el orden en que recorren los contenidos. Si no tenemos una etiqueta que clasifique a cada estudiante, podemos usar aprendizaje no supervisado para explorar patrones de uso. Quizás descubramos grupos de estudiantes con recorridos similares, materiales que suelen consultarse juntos o comportamientos que sugieren distintas formas de interactuar con la plataforma.

También puede utilizarse aprendizaje no supervisado para detectar casos extraños o poco frecuentes. Por ejemplo, en un sistema industrial, podríamos tener mediciones

de temperatura, presión, vibración y consumo energético de una máquina. Si la mayoría de los registros muestran comportamientos estables, pero algunos se alejan mucho del patrón habitual, esos casos podrían indicar una anomalía. No necesariamente sabemos de antemano qué registros corresponden a fallas confirmadas, pero podemos detectar observaciones que no se parecen al comportamiento normal.

Esto muestra una idea importante: el aprendizaje no supervisado no produce necesariamente una “respuesta correcta” en el mismo sentido que el aprendizaje supervisado. Muchas veces produce una interpretación posible de la estructura de los datos. Por eso requiere todavía más cuidado al analizar sus resultados. Si un algoritmo separa clientes en cuatro grupos, esos grupos no vienen con un significado automático. Debemos observarlos, describirlos, interpretarlos y decidir si tienen sentido para el problema real.

Por ejemplo, un modelo puede agrupar clientes según sus comportamientos de compra, pero luego una persona debe mirar esos grupos y preguntarse: ¿qué caracteriza a cada grupo?, ¿son grupos útiles?, ¿permiten tomar mejores decisiones?, ¿son estables?, ¿tienen sentido desde el punto de vista del negocio o del análisis? El algoritmo puede encontrar una estructura matemática, pero el significado de esa estructura debe construirse con conocimiento del contexto.



Posible separación entre clientes de bajo, medio y alto movimiento.

Esta es una diferencia fundamental respecto del aprendizaje supervisado. En un problema supervisado, podemos comparar la predicción del modelo con una respuesta conocida. Si el modelo predice que un correo es spam y sabemos que efectivamente era spam, podemos evaluar si acertó. Si estima un precio y conocemos el precio real, podemos medir el error. En el aprendizaje no supervisado, esa

comparación directa muchas veces no existe, porque no hay una respuesta previa contra la cual contrastar.

Eso no significa que no podamos evaluar los resultados, sino que la evaluación suele ser más compleja. Podemos analizar si los grupos son coherentes, si los casos dentro de un grupo se parecen entre sí, si los grupos son suficientemente distintos unos de otros, si el resultado se mantiene al cambiar algunos datos o si la segmentación sirve para tomar decisiones. Pero no siempre hay una única respuesta correcta.

El aprendizaje no supervisado es especialmente útil cuando queremos explorar datos y descubrir estructuras que todavía no conocemos. Puede funcionar como una herramienta de investigación inicial, de organización, de reducción de complejidad o de detección de patrones. En lugar de partir de una pregunta como “¿qué valor tendrá esta variable?”, muchas veces parte de preguntas más abiertas: “¿hay grupos naturales en estos datos?”, “¿qué casos se parecen entre sí?”, “¿hay comportamientos fuera de lo común?”, “¿existen relaciones ocultas que conviene investigar?”.

Esto lo vuelve muy valioso en etapas exploratorias. Cuando todavía no sabemos exactamente qué categorías existen, qué comportamientos son relevantes o cómo se organiza un conjunto de datos, el aprendizaje no supervisado puede ayudarnos a mirar el problema desde otro ángulo. No reemplaza al análisis exploratorio tradicional, pero puede ampliarlo cuando los datos son numerosos, complejos o difíciles de interpretar a simple vista.

Sin embargo, también tiene límites importantes. El primero es que encontrar grupos no significa necesariamente encontrar grupos reales o útiles. Un algoritmo de agrupamiento siempre puede dividir datos en grupos si se lo pedimos, pero eso no garantiza que esos grupos tengan significado. A veces la estructura encontrada refleja diferencias importantes; otras veces puede estar capturando ruido, escalas mal preparadas, variables irrelevantes o decisiones técnicas poco adecuadas.

El segundo límite es que los resultados dependen mucho de cómo representamos los datos. Si incluimos variables poco relevantes, si dejamos variables en escalas muy diferentes o si no preparamos adecuadamente la información, el modelo puede encontrar patrones engañosos. Por ejemplo, si agrupamos clientes usando una variable con valores muy grandes junto con otras de valores pequeños, esa variable puede dominar el resultado. Estos detalles técnicos se verán más adelante, pero conviene anticipar la idea: en aprendizaje no supervisado, la preparación de los datos es especialmente importante.

El tercer límite es interpretativo. Como no hay una etiqueta previa, el modelo puede sugerir una estructura, pero no explicarla por completo. La interpretación queda en manos del analista. Dos grupos pueden ser matemáticamente distintos, pero quizás

esa diferencia no sea relevante para el objetivo del proyecto. O, al revés, una diferencia pequeña en los datos puede ser muy importante desde el punto de vista del contexto.

Por eso, el aprendizaje no supervisado debe usarse con una actitud exploratoria y crítica. Sus resultados suelen ser puntos de partida para pensar, no conclusiones automáticas. Puede ayudarnos a descubrir algo interesante, pero luego debemos revisar, describir, contrastar y decidir si eso que encontramos realmente aporta valor.

El aprendizaje no supervisado se utiliza cuando tenemos datos sin respuestas conocidas y queremos descubrir patrones, grupos, similitudes o comportamientos inusuales. A diferencia del aprendizaje supervisado, no aprende a predecir una variable objetivo previamente definida, sino que busca estructuras dentro de los datos. Su potencia está en revelar organizaciones posibles que no estaban etiquetadas de antemano; su desafío está en interpretar esas estructuras con cuidado y no confundir cualquier patrón matemático con una conclusión significativa.

Aprendizaje por refuerzo

El **aprendizaje por refuerzo** es otra forma de machine learning, distinta del aprendizaje supervisado y del aprendizaje no supervisado. En este libro lo mencionaremos solo como panorama, porque no será nuestro foco principal. Aun así, vale la pena conocer su idea general, ya que aparece en áreas importantes como robótica, videojuegos, sistemas de recomendación avanzados, control automático y entrenamiento de agentes que toman decisiones en entornos cambiantes.

En el aprendizaje supervisado, el modelo aprende a partir de ejemplos con respuestas conocidas. En el aprendizaje no supervisado, busca estructuras en datos sin etiquetas. En el aprendizaje por refuerzo, en cambio, el centro está en la **toma de decisiones**. Un agente interactúa con un entorno, realiza acciones y recibe recompensas o penalizaciones según los resultados de esas acciones. A partir de esa experiencia, intenta aprender qué decisiones conviene tomar para obtener mejores resultados a lo largo del tiempo.

La estructura general puede pensarse así:



Aprendizaje por refuerzo.

El término **agente** se usa para referirse al sistema que toma decisiones. El **entorno** es el mundo, real o simulado, donde esas decisiones tienen consecuencias. La **acción** es lo que el agente decide hacer. La **recompensa** es una señal que indica si el resultado fue bueno, malo o más o menos conveniente según el objetivo definido.

Un ejemplo simple puede ser un juego. Imaginemos un agente que aprende a jugar un videojuego. En cada momento observa el estado del juego: la posición del personaje, los obstáculos, los enemigos, el puntaje, los objetos disponibles. Luego elige una acción: moverse a la izquierda, saltar, avanzar, retroceder. Si la acción permite ganar puntos, avanzar de nivel o evitar perder una vida, el agente recibe una recompensa positiva. Si la acción provoca una pérdida, recibe una penalización. Con muchas partidas, el agente puede aprender qué acciones suelen conducir a mejores resultados.

Este tipo de aprendizaje no se basa en mostrarle al modelo una lista de jugadas correctas para cada situación. Tampoco se limita a agrupar datos. El agente aprende probando acciones y observando sus consecuencias. Por eso, el aprendizaje por refuerzo suele estar asociado a la idea de **ensayo y error**, aunque en sistemas reales ese ensayo y error debe estar cuidadosamente controlado para evitar riesgos.

Otro ejemplo aparece en robótica. Un robot que aprende a desplazarse, tomar objetos o mantener el equilibrio puede recibir recompensas cuando se acerca al objetivo y penalizaciones cuando se cae, se aleja o realiza movimientos ineficientes. A través de muchas interacciones con el entorno, o con una simulación del entorno, puede aprender estrategias de movimiento cada vez mejores.

También puede pensarse en sistemas de control, como administrar el uso de energía en un edificio, ajustar parámetros en un proceso industrial o decidir cuándo encender o apagar ciertos dispositivos. En estos casos, el agente no busca simplemente predecir un resultado, sino elegir acciones que produzcan buenos resultados acumulados: menor consumo, mayor estabilidad, menor costo o mayor eficiencia.

Una idea clave del aprendizaje por refuerzo es que una acción no siempre puede evaluarse solo por su consecuencia inmediata. A veces una decisión parece buena en el corto plazo, pero mala a largo plazo. O al revés: una acción puede tener un costo inicial, pero permitir mejores resultados después. Por ejemplo, en un juego, tomar un camino más largo puede ser conveniente si permite conseguir un recurso importante. En un sistema de energía, reducir demasiado el consumo ahora puede generar problemas de confort o rendimiento más tarde.

Por eso, el aprendizaje por refuerzo trabaja mucho con la relación entre decisiones presentes y recompensas futuras. El objetivo no es solo obtener una recompensa inmediata, sino aprender una estrategia de acción que funcione bien a lo largo del tiempo.

Esto lo diferencia bastante de los problemas que vamos a trabajar principalmente en este libro. En clasificación, regresión o agrupamiento, normalmente tenemos un conjunto de datos y buscamos entrenar un modelo para hacer predicciones o encontrar estructuras. En aprendizaje por refuerzo, en cambio, el sistema aprende mientras interactúa con un entorno. La experiencia se genera a partir de las acciones del propio agente.

Esta diferencia hace que el aprendizaje por refuerzo sea muy potente, pero también más complejo. Requiere definir cuidadosamente el entorno, las acciones posibles, las recompensas, las penalizaciones y las condiciones de aprendizaje. Si la recompensa está mal diseñada, el agente puede aprender comportamientos no deseados. Puede encontrar formas de maximizar la recompensa sin cumplir realmente el objetivo que las personas tenían en mente.

Por ejemplo, si se entrena un agente para obtener la mayor cantidad de puntos en un juego, puede aprender estrategias muy efectivas para sumar puntos, aunque esas estrategias no sean las que una persona consideraría “inteligentes” o elegantes. Si se diseña mal la recompensa en un sistema real, el agente podría optimizar una métrica y perjudicar otras dimensiones importantes.

También hay un problema práctico: aprender por prueba y error puede requerir muchísimos intentos. En un videojuego o una simulación, eso puede ser aceptable. En un entorno real, como un robot físico, un sistema financiero o una planta industrial, probar acciones al azar puede ser costoso o peligroso. Por eso, muchas aplicaciones

de aprendizaje por refuerzo dependen de simulaciones, restricciones de seguridad y supervisión cuidadosa.

En este libro no profundizaremos en los algoritmos de aprendizaje por refuerzo, porque nuestro recorrido se centrará principalmente en problemas supervisados y no supervisados con datos tabulares, análisis, preparación, entrenamiento y evaluación de modelos. Sin embargo, conocer su existencia ayuda a completar el mapa general del machine learning.

El aprendizaje por refuerzo se utiliza cuando queremos que un agente aprenda a tomar decisiones mediante interacción con un entorno, recibiendo recompensas o penalizaciones según las consecuencias de sus acciones. Su foco no está solamente en predecir, sino en actuar. Es una rama muy importante del machine learning, pero más avanzada y con desafíos propios, especialmente cuando las decisiones tienen efectos acumulados, riesgos o consecuencias en el mundo real.

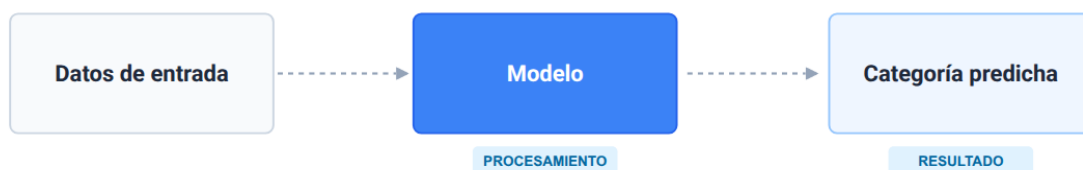
Problemas de clasificación

Los **problemas de clasificación** son uno de los tipos de problemas más frecuentes dentro del aprendizaje supervisado. Aparecen cuando queremos que un modelo tome un caso nuevo y lo asigne a una de varias categorías posibles.

La idea central es sencilla: tenemos ejemplos anteriores, cada uno con sus datos de entrada y una categoría conocida. El modelo aprende a relacionar las características de cada caso con la categoría correspondiente. Luego, cuando recibe un caso nuevo, intenta predecir a qué categoría pertenece.

En una clasificación, la respuesta no es un número continuo como un precio, una temperatura o una cantidad vendida. La respuesta es una **clase**. Por ejemplo: “spam” o “no spam”, “aprobado” o “rechazado”, “cliente que abandona” o “cliente que permanece”, “transacción normal” o “transacción sospechosa”, “riesgo bajo”, “riesgo medio” o “riesgo alto”.

Podemos representarlo de esta manera:



Problemas de clasificación.

Si el modelo analiza un correo electrónico, la salida puede ser “spam”. Si analiza una solicitud de crédito, la salida puede ser “riesgo alto”. Si analiza una imagen médica, la salida puede ser “requiere revisión”. En todos esos casos, el modelo está clasificando.

Para entrenar un modelo de clasificación necesitamos datos donde las categorías ya estén registradas. Si queremos clasificar correos, necesitamos correos previamente etiquetados. Si queremos detectar transacciones sospechosas, necesitamos ejemplos históricos de transacciones normales y sospechosas. Si queremos estimar si un estudiante está en riesgo de abandonar un curso, necesitamos datos de estudiantes anteriores y saber qué ocurrió con ellos.

Esa categoría conocida suele llamarse **etiqueta** o **clase**. En una tabla, podría estar representada por una columna. Las demás columnas describen el caso: edad, cantidad de compras, monto, frecuencia de uso, palabras del mensaje, resultados de laboratorio, características del producto o cualquier otra información relevante para el problema.

Por ejemplo, imaginemos una tabla con datos de estudiantes de una plataforma educativa. Cada fila representa un estudiante y contiene variables como cantidad de clases vistas, actividades entregadas, días de inactividad, participación en foros y promedio de calificaciones. Además, tenemos una columna que indica si ese estudiante finalizó o no finalizó el curso. En ese contexto, podríamos entrenar un modelo de clasificación para estimar si un estudiante actual está en riesgo de no finalizar.

El modelo no decide por intuición. Aprende a partir de patrones observados en estudiantes anteriores. Tal vez encuentre que muchos días de inactividad, pocas actividades entregadas y baja participación se asocian con mayor probabilidad de abandono. Pero también podría aprender combinaciones más sutiles: por ejemplo, estudiantes con pocas participaciones en foros, pero con entregas regulares, quizás no estén en riesgo; mientras que estudiantes con muchas visualizaciones, pero sin entregas, podrían requerir atención. El modelo busca relaciones entre las variables disponibles y la clase conocida.

Los problemas de clasificación pueden tener dos categorías o más de dos. Cuando solo hay dos clases posibles, hablamos de **clasificación binaria**. Algunos ejemplos son: spam o no spam, fraude o no fraude, enfermo o sano, aprueba o no aprueba, abandona o permanece.

Cuando hay más de dos clases, hablamos de **clasificación multiclase**. Por ejemplo, clasificar una noticia como política, economía, deportes, ciencia o cultura; identificar

si una imagen contiene un perro, un gato, un auto o una bicicleta; asignar un reclamo a distintas áreas de atención; o clasificar un nivel de riesgo como bajo, medio o alto.

También existen problemas donde un mismo caso puede pertenecer a más de una categoría al mismo tiempo. Por ejemplo, una noticia podría estar relacionada tanto con ciencia como con economía; una película podría etiquetarse como drama y suspenso; un correo podría requerir varias etiquetas internas. Ese tipo de situación suele llamarse clasificación multietiqueta. No hace falta profundizar ahora, pero conviene saber que no todos los problemas de clasificación obligan a elegir una única clase.

Una característica importante de muchos modelos de clasificación es que no solo pueden devolver una clase final, sino también una estimación de probabilidad o confianza. Por ejemplo, un modelo podría indicar que un correo tiene 92% de probabilidad de ser spam, o que una solicitud tiene 78% de probabilidad de pertenecer al grupo de alto riesgo. Luego, según el contexto, se decide qué hacer con esa información.

Esto es muy importante porque en muchos problemas reales no alcanza con saber la clase más probable. También importa cuán segura parece la predicción. Un sistema de correos podría enviar directamente a la carpeta de spam los mensajes con probabilidad muy alta, pero dejar en la bandeja principal aquellos casos dudosos. Un sistema de apoyo médico podría marcar ciertos casos para revisión profesional sin tomar una decisión automática. Una institución educativa podría usar el modelo para priorizar acompañamiento, no para etiquetar de manera definitiva a un estudiante.

En clasificación, los errores no siempre tienen el mismo peso. Confundir un correo legítimo con spam puede ser molesto o incluso grave si se pierde un mensaje importante. Pero dejar pasar un correo no deseado también tiene consecuencias. En un sistema de detección de fraude, bloquear una operación legítima puede afectar a un cliente, mientras que permitir una operación fraudulenta puede generar una pérdida económica. En un contexto médico, no detectar un posible problema puede ser mucho más grave que solicitar una revisión adicional.

Por eso, evaluar un modelo de clasificación no consiste solamente en contar cuántas veces acierta. También debemos analizar qué tipos de errores comete. Más adelante estudiaremos métricas específicas para esto, pero desde ahora conviene incorporar la idea: en clasificación, el significado de un error depende del problema.

Otro aspecto importante es que no todas las clases aparecen con la misma frecuencia. En muchos problemas reales, una categoría puede ser mucho más común que otra. Por ejemplo, la mayoría de las transacciones bancarias son legítimas y solo una pequeña proporción es fraudulenta. La mayoría de los pacientes puede no tener

cierta enfermedad, mientras que los casos positivos son menos frecuentes. La mayoría de los clientes puede continuar usando un servicio, mientras que una minoría lo abandona en un período determinado.

Esto puede generar dificultades. Un modelo podría parecer bueno simplemente porque predice siempre la clase mayoritaria. Si el 99% de las transacciones son normales, un modelo que diga “normal” en todos los casos tendría 99% de aciertos, pero sería inútil para detectar fraude. Por eso, en clasificación debemos mirar con cuidado la distribución de las clases y no quedarnos solo con una medida general de acierto.

También debemos recordar que una clasificación no siempre describe una verdad absoluta. A veces las categorías son claras, como “aprobado” o “no aprobado” según una regla formal. Pero otras veces las etiquetas dependen de criterios humanos, decisiones institucionales o diagnósticos que pueden tener incertidumbre. Si las etiquetas de entrenamiento son discutibles o inconsistentes, el modelo aprenderá a partir de esa ambigüedad.

Por ejemplo, si distintos evaluadores clasifican reclamos con criterios diferentes, el modelo puede aprender una mezcla confusa de esos criterios. Si una etiqueta de riesgo fue asignada con reglas antiguas o sesgadas, el modelo puede reproducirlas. Por eso, antes de entrenar un clasificador, es importante revisar qué significa cada clase y cómo fue construida.

Los problemas de clasificación aparecen cuando queremos asignar casos a categorías. Son problemas supervisados porque necesitamos ejemplos previos con clases conocidas. Pueden ser binarios, multiclase o, en algunos casos, multietiqueta. Su utilidad es enorme, pero deben analizarse con cuidado: no todas las clases tienen la misma frecuencia, no todos los errores tienen la misma gravedad y no todas las etiquetas representan una verdad simple. Un buen modelo de clasificación no solo debe acertar muchas veces; debe cometer errores aceptables para el contexto en el que será usado.

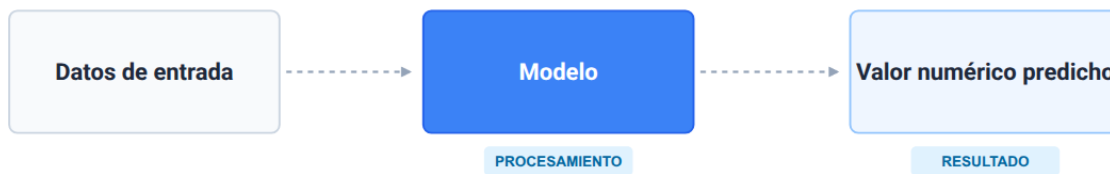
Problemas de regresión

Los **problemas de regresión** son otro gran tipo de problema dentro del aprendizaje supervisado. Aparecen cuando queremos que un modelo prediga un valor numérico. A diferencia de la clasificación, donde la respuesta es una categoría, en la regresión la respuesta es una cantidad.

Si en clasificación el modelo responde algo como “spam” o “no spam”, “riesgo bajo” o “riesgo alto”, en regresión responde con un número: un precio, una temperatura, una

duración, una demanda esperada, un consumo, una distancia, un ingreso probable o cualquier otra magnitud que pueda expresarse numéricamente.

Podemos representarlo así:



Problemas de regresión.

Por ejemplo, un modelo de regresión podría recibir información sobre una vivienda y estimar su precio. Podría recibir datos históricos de ventas y predecir la demanda de un producto para la semana próxima. Podría analizar características de un viaje y estimar su duración. Podría usar mediciones de sensores industriales para predecir el consumo energético de una máquina.

La estructura de aprendizaje es similar a la que vimos en clasificación. Necesitamos ejemplos anteriores donde conocemos tanto los datos de entrada como el resultado correcto. La diferencia está en la naturaleza de ese resultado. En lugar de una clase, tenemos un valor numérico.

Imaginemos una empresa de logística que quiere estimar el tiempo de entrega de sus pedidos. Para cada entrega histórica podría tener datos como distancia recorrida, zona, día de la semana, horario, tipo de vehículo, cantidad de paquetes, condiciones del tránsito y tiempo real de entrega. Con esos ejemplos, un modelo de regresión puede aprender relaciones entre las características del envío y la duración final. Luego, ante un nuevo pedido, puede estimar cuántos minutos tardará aproximadamente.

Este ejemplo permite notar algo importante: una predicción de regresión suele ser una aproximación. Si el modelo estima que una entrega tardará 42 minutos, no significa que necesariamente tardará exactamente 42 minutos. Tal vez tarde 39, 45 o 50. En muchos problemas reales, el objetivo no es acertar el número exacto, sino producir una estimación suficientemente útil para tomar decisiones.

Esto ocurre también con precios. Si un modelo estima que una vivienda vale cierta cantidad, ese número debe interpretarse como una referencia basada en datos, no como una verdad absoluta. El precio final puede depender de una negociación, de la urgencia de venta, de la disponibilidad de compradores, del contexto económico o de detalles que no están en la tabla de datos. El modelo puede acercarse, pero no controlar todos los factores del mundo real.

Por eso, en regresión es fundamental pensar en el **error**. Como la respuesta es numérica, el modelo casi nunca acierta exactamente. Lo importante es medir qué tan lejos están sus predicciones de los valores reales. Si queremos estimar el consumo eléctrico de una vivienda, no esperamos que el modelo acierte cada decimal. Queremos saber si sus errores son pequeños, razonables o demasiado grandes para el uso que queremos darle.

Más adelante veremos métricas específicas para evaluar modelos de regresión. Por ahora, alcanza con entender la idea general: en clasificación observamos si el modelo eligió la categoría correcta; en regresión observamos cuánto se equivocó numéricamente.

La magnitud aceptable del error depende del contexto. Un error de 5 minutos puede ser irrelevante en la estimación de duración de un viaje largo, pero importante en una intervención médica o en un proceso industrial sensible. Un error de 10 pesos puede ser insignificante en el precio de una vivienda, pero enorme si estamos prediciendo el precio de un producto muy barato. No existe una definición universal de “buen error”: siempre depende del problema.

También es importante distinguir entre valores numéricos y categorías representadas con números. No todo número implica regresión. Por ejemplo, si tenemos una columna donde 0 significa “no abandonó” y 1 significa “abandonó”, no estamos frente a un problema de regresión, sino de clasificación. Lo mismo ocurre si usamos 1, 2 y 3 para representar “riesgo bajo”, “riesgo medio” y “riesgo alto”. Aunque aparezcan números, el modelo está prediciendo categorías.

En regresión, el número tiene sentido como cantidad. Si predecimos 100, ese valor puede compararse con 90 o 110. Tiene sentido hablar de distancia entre valores, de diferencia, de error numérico. En cambio, en una clasificación codificada como 0 y 1, no tiene sentido decir que “spam” está a una distancia numérica de “no spam”. Los números son solo códigos para representar clases.

Esta distinción es muy importante al formular un problema de machine learning. Si definimos mal el tipo de problema, podemos elegir modelos, métricas o interpretaciones incorrectas. Antes de entrenar cualquier modelo, debemos preguntarnos: ¿la respuesta que quiero predecir es una categoría o una cantidad? Si es una categoría, probablemente sea clasificación. Si es una cantidad numérica, probablemente sea regresión.

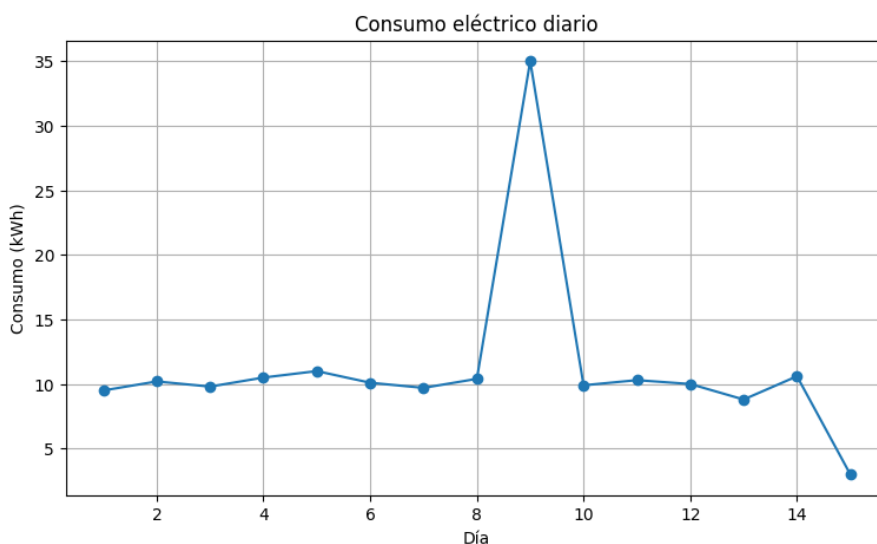
Los problemas de regresión pueden involucrar una sola variable de entrada o muchas. Un ejemplo simple sería estimar el precio de un producto usando solo su tamaño. Pero en la mayoría de los casos reales intervienen múltiples variables. Para estimar la demanda de un producto, podrían importar el precio, la temporada, el

stock disponible, las promociones, la ubicación, el día de la semana, el comportamiento histórico y la competencia. Para estimar el rendimiento de un cultivo, podrían intervenir el clima, el suelo, la humedad, el tipo de semilla, la fertilización y el manejo agrícola.

El modelo intenta aprender cómo se relacionan esas variables con el valor que queremos predecir. Algunas relaciones pueden ser directas: a mayor distancia, mayor tiempo de entrega. Otras pueden ser más complejas: una promoción puede aumentar la demanda, pero solo en ciertos productos o en ciertos momentos del año. La utilidad del modelo está en combinar muchas señales para producir una estimación más informada.

Sin embargo, como ocurre con cualquier forma de aprendizaje supervisado, la regresión depende de la calidad de los datos. Si los valores históricos están mal medidos, si hay errores de carga, si faltan variables importantes o si el contexto cambió demasiado, las predicciones pueden ser poco confiables. Un modelo de regresión no entiende por sí mismo el significado del precio, del tiempo o del consumo: aprende relaciones estadísticas a partir de los datos disponibles.

También debemos tener cuidado con los valores extremos. En muchos problemas numéricos, algunos casos pueden ser muy diferentes del resto. Una vivienda de lujo puede tener un precio muchísimo mayor que las demás. Un pedido puede tardar muchas horas por un evento excepcional. Un consumo eléctrico puede dispararse por una falla. Estos casos pueden afectar el aprendizaje del modelo y la evaluación de sus errores. No siempre deben eliminarse, pero sí deben comprenderse.



El gráfico permite identificar rápidamente dos días que el consumo eléctrico se aleja del comportamiento habitual.

Otro punto importante es que una predicción numérica puede dar una falsa sensación de precisión. Si un modelo estima que la demanda será de 12.347 unidades, el número parece muy exacto. Pero esa cantidad de decimales no significa necesariamente mayor certeza. En muchos contextos, sería más honesto comunicar un rango aproximado o acompañar la predicción con una medida de incertidumbre. Aunque estos temas se verán más adelante, conviene recordar desde ahora que una predicción numérica no es una garantía.

Los problemas de regresión aparecen cuando queremos predecir una cantidad. Son problemas supervisados porque necesitamos ejemplos previos con valores conocidos. Se utilizan para estimar precios, tiempos, consumos, demandas, temperaturas, ingresos y muchas otras magnitudes. Su evaluación se basa en analizar cuánto se alejan las predicciones de los valores reales, y la gravedad de ese error depende siempre del contexto. Un buen modelo de regresión no es el que promete exactitud absoluta, sino el que produce estimaciones suficientemente confiables y útiles para el problema que se quiere resolver.

Problemas de agrupamiento

Los **problemas de agrupamiento** aparecen cuando queremos encontrar grupos dentro de los datos, pero no contamos con una categoría conocida de antemano. A diferencia de la clasificación, donde cada ejemplo usado para entrenar el modelo ya tiene una clase asignada, en el agrupamiento no partimos de etiquetas previas. El objetivo es descubrir si los datos parecen organizarse en conjuntos de casos parecidos entre sí.

Por eso, el agrupamiento pertenece al aprendizaje no supervisado. No le decimos al modelo cuál es la respuesta correcta. Le damos datos y le pedimos que busque similitudes, diferencias y posibles estructuras internas.

Podemos pensarlo así:



Problemas de agrupamiento.

Imaginemos una tabla con información de clientes. Cada fila representa una persona y cada columna describe algún aspecto de su comportamiento: frecuencia de compra, monto promedio gastado, tipo de productos elegidos, antigüedad como cliente, uso de promociones o canal de compra preferido. Si no tenemos una columna que clasifique previamente a esos clientes, no podemos entrenar un modelo supervisado para predecir una categoría. Pero sí podemos intentar descubrir si existen grupos de clientes con comportamientos similares.

Un modelo de agrupamiento podría sugerir, por ejemplo, que hay clientes que compran poco pero con mucha frecuencia, otros que compran pocas veces pero gastan más, otros que aparecen casi exclusivamente durante promociones y otros que mantienen un comportamiento estable durante todo el año. Esos grupos no estaban escritos en los datos como una etiqueta previa. Surgen del análisis de semejanzas entre los casos.

La idea de “parecido” es central en los problemas de agrupamiento. El modelo necesita alguna forma de comparar casos entre sí. Dos clientes pueden considerarse parecidos si tienen hábitos de compra similares. Dos canciones pueden considerarse parecidas si comparten características de ritmo, género, duración o patrones de escucha. Dos productos pueden agruparse si suelen ser comprados por perfiles similares de usuarios o si tienen características técnicas cercanas.

Pero esta idea también exige cuidado. Los datos no “dicen” por sí solos qué significa parecerse. Esa semejanza depende de las variables que elegimos, de cómo están medidas y de cómo fueron preparadas. Si agrupamos personas usando edad, ingresos y frecuencia de compra, obtendremos una estructura. Si usamos ubicación, preferencias de producto y canal de contacto, quizás obtengamos otra. El agrupamiento no descubre una verdad única e inevitable; descubre una organización posible según la representación de los datos que le ofrecemos.

Esto vuelve al agrupamiento muy útil, pero también delicado. Puede ayudarnos a explorar un conjunto de datos cuando todavía no sabemos qué categorías existen. Puede servir para segmentar clientes, organizar documentos, agrupar productos similares, identificar perfiles de usuarios, ordenar materiales educativos o analizar comportamientos dentro de una plataforma. En muchos casos, permite convertir una tabla grande y difícil de interpretar en un conjunto más manejable de grupos que luego pueden describirse y estudiarse.

Por ejemplo, en una plataforma educativa podríamos agrupar estudiantes según sus patrones de interacción: quienes ingresan todos los días y avanzan de manera regular, quienes se conectan poco pero entregan actividades, quienes miran muchos contenidos pero no resuelven ejercicios, o quienes tienen actividad intensa al inicio y

luego desaparecen. El modelo no sabría por sí mismo qué nombre darle a cada grupo. Esa interpretación la debe realizar una persona que conozca el contexto educativo.

Este punto es importante: en agrupamiento, el resultado necesita interpretación. Si un algoritmo devuelve tres, cuatro o cinco grupos, esos grupos todavía no significan nada por sí solos. Debemos observar qué características tienen, qué variables los distinguen, qué casos quedaron dentro de cada uno y si esa división aporta algo al problema que queremos comprender. Muchas veces, el trabajo más valioso empieza después de obtener los grupos.

Una buena práctica conceptual es no pensar los grupos como “verdades naturales”, sino como herramientas de lectura. Un agrupamiento puede ser útil si permite entender mejor los datos, tomar decisiones más claras o formular nuevas preguntas. Pero puede no ser útil si genera divisiones artificiales, difíciles de explicar o irrelevantes para el objetivo del análisis.

También debemos recordar que distintos métodos de agrupamiento pueden producir resultados diferentes. Algunos tienden a formar grupos compactos y separados; otros pueden detectar formas más irregulares; otros pueden identificar casos que no encajan bien en ningún grupo. No todos los algoritmos buscan la misma idea de grupo. Por eso, más adelante, cuando estudiemos técnicas concretas, será importante mirar no solo el resultado, sino también qué tipo de estructura supone cada método.

Otro aspecto a tener en cuenta es la cantidad de grupos. En algunos problemas, podríamos tener una idea previa de cuántos grupos esperamos encontrar. En otros, no. Pedirle a un algoritmo que divida los datos en cuatro grupos puede producir una segmentación, pero eso no garantiza que cuatro sea el número más adecuado. Tal vez los datos se organicen mejor en tres grupos, o tal vez no haya grupos claros. El hecho de que un algoritmo pueda separar los casos no significa que esa separación tenga sentido.

Por eso, el agrupamiento suele combinarse con análisis exploratorio. Después de obtener los grupos, conviene resumirlos, compararlos, visualizarlos y revisar ejemplos concretos. Podemos preguntarnos: ¿qué caracteriza a cada grupo?, ¿qué variables explican mejor la separación?, ¿hay grupos muy pequeños?, ¿hay casos mezclados?, ¿los grupos son estables?, ¿tienen sentido para alguien que conoce el problema?

Los problemas de agrupamiento también pueden ser útiles como paso previo para otros análisis. Por ejemplo, una empresa podría segmentar clientes y luego diseñar estrategias distintas para cada segmento. Un equipo educativo podría detectar perfiles de participación y pensar intervenciones diferenciadas. Un catálogo digital podría agrupar productos similares para mejorar la navegación. En estos casos, el

agrupamiento no es el final del trabajo, sino una forma de organizar la información para actuar mejor.

Sin embargo, hay que evitar una confusión común: agrupar no es lo mismo que explicar. Si encontramos un grupo de usuarios con bajo uso de una plataforma, el agrupamiento nos muestra que esos usuarios se parecen en ciertos comportamientos, pero no nos dice automáticamente por qué ocurre eso. Tal vez tienen poco tiempo, tal vez el contenido no les resulta claro, tal vez tuvieron problemas técnicos o tal vez ingresaron solo por curiosidad. El modelo puede señalar una estructura; la explicación requiere análisis adicional.

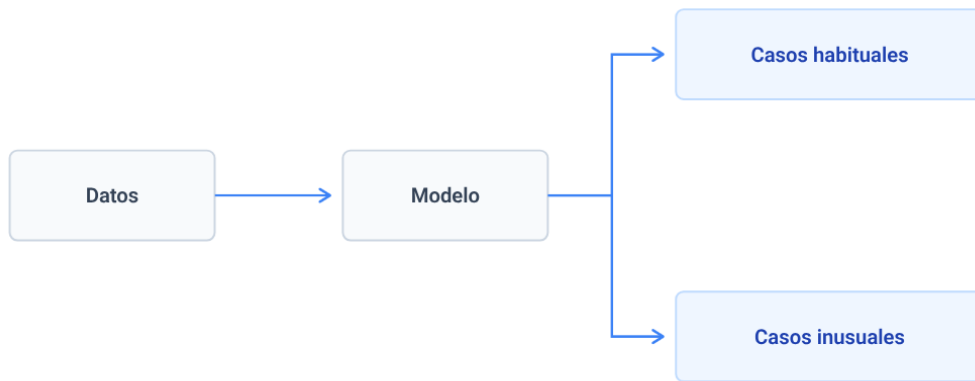
Los problemas de agrupamiento buscan descubrir grupos dentro de datos que no tienen etiquetas previas. Son útiles cuando queremos explorar, segmentar u organizar casos según sus similitudes. Su valor depende de que los grupos encontrados sean interpretables y relevantes para el contexto. Un agrupamiento no debe tomarse como una verdad automática, sino como una propuesta de organización que debemos examinar, describir y validar con criterio.

Problemas de detección de anomalías

Los **problemas de detección de anomalías** aparecen cuando queremos identificar casos que se apartan de lo habitual. Una anomalía es una observación que resulta poco frecuente, extraña o inesperada en relación con el comportamiento general de los datos.

A diferencia de otros problemas donde buscamos predecir una categoría o estimar un valor, aquí el interés está puesto en detectar situaciones que no encajan bien con el patrón dominante. No siempre sabemos de antemano qué forma tendrá una anomalía. A veces solo sabemos cómo se ve el comportamiento normal, y queremos marcar aquello que se aleja demasiado de ese comportamiento.

Podemos pensarlo así:



Identificar casos que se apartan de lo habitual.

Un ejemplo claro aparece en sistemas industriales. Una máquina puede generar mediciones continuas de temperatura, vibración, presión, velocidad o consumo eléctrico. Durante la mayor parte del tiempo, esos valores se mantienen dentro de ciertos rangos esperables. Si de pronto aparece una combinación extraña, como un aumento de vibración junto con una temperatura fuera de lo habitual, el sistema podría marcar ese registro como anómalo. Esa anomalía no prueba por sí sola que haya una falla, pero indica que conviene revisar la situación.

También puede aplicarse en seguridad informática. En una red, la mayoría de los accesos, transferencias o patrones de uso siguen comportamientos relativamente conocidos. Si una cuenta que normalmente se conecta desde una ubicación y horario determinados comienza a realizar acciones inusuales, o si aparece un volumen de tráfico muy diferente del habitual, puede ser razonable marcar ese evento para investigación.

Otro ejemplo aparece en control de calidad. Si una línea de producción genera piezas con medidas, pesos o características visuales esperadas, un caso que se aleja mucho de esos valores puede indicar un defecto de fabricación. En contextos financieros, operaciones con montos, horarios, ubicaciones o frecuencias poco comunes pueden requerir revisión. En salud, ciertos valores clínicos o combinaciones de síntomas pueden llamar la atención si se apartan de lo esperable para un paciente o una población.

La detección de anomalías puede abordarse de distintas maneras. En algunos casos, tenemos ejemplos históricos de anomalías ya identificadas. Si contamos con registros etiquetados como “normal” y “anómalo”, el problema puede parecerse a una clasificación supervisada. El modelo aprende a distinguir ambos tipos de casos a partir de ejemplos conocidos.

Pero muchas veces no tenemos suficientes ejemplos de anomalías. Esto ocurre porque, por definición, las anomalías suelen ser poco frecuentes. Una falla grave en

una máquina puede haber ocurrido pocas veces. Un ataque informático nuevo quizás no tenga antecedentes claros. Un fraude puede cambiar de forma justamente para no parecerse a fraudes anteriores. En esas situaciones, el enfoque no puede depender únicamente de ejemplos etiquetados.

Por eso, muchos problemas de detección de anomalías se trabajan con ideas cercanas al aprendizaje no supervisado o semisupervisado. En lugar de aprender todas las formas posibles de anomalía, el modelo intenta aprender cómo es el comportamiento normal. Luego marca como sospechosos los casos que se alejan demasiado de ese patrón.

Esto es útil, pero también trae una dificultad: no todo lo raro es necesariamente problemático. Un caso puede ser inusual y aun así ser perfectamente válido. Una venta excepcionalmente alta puede deberse a una promoción exitosa. Un consumo eléctrico elevado puede corresponder a un evento planificado. Un acceso desde otra ciudad puede ser simplemente un viaje del usuario. Una medición fuera de rango puede deberse a un error del sensor, no a una falla real del sistema.

Por eso, detectar anomalías no significa tomar decisiones automáticas sin revisión. Muchas veces el modelo funciona como un sistema de alerta. Señala casos que merecen atención, pero la interpretación final requiere contexto. La anomalía es una pista, no una conclusión definitiva.

También ocurre lo contrario: no todo problema importante se ve como una anomalía evidente. Algunos fraudes pueden parecer normales. Algunas fallas se desarrollan lentamente y no generan cambios bruscos. Algunos errores están dentro de rangos aparentemente aceptables, pero forman parte de una tendencia preocupante. Por eso, un sistema de detección de anomalías puede ayudar mucho, pero no garantiza detectar todos los casos relevantes.

Una cuestión central en estos problemas es definir qué consideramos “normal”. Esa definición no siempre es fija. El comportamiento normal puede cambiar según el horario, la temporada, el lugar, el tipo de usuario, el estado de una máquina o el contexto económico. Lo que es normal en diciembre puede no serlo en marzo. Lo que es normal para un cliente mayorista puede ser extraño para un cliente ocasional. Lo que es normal durante el arranque de una máquina puede ser anómalo durante su operación estable.

Esto significa que la detección de anomalías depende mucho del contexto. Un valor aislado puede parecer raro, pero tal vez sea esperable en ciertas condiciones. Por eso, en muchos casos no alcanza con mirar una variable individual; conviene analizar combinaciones de variables, patrones temporales y condiciones específicas.

También debemos cuidar la cantidad de alertas. Si un sistema marca demasiados casos como anómalos, puede volverse poco útil. Las personas encargadas de revisar las alertas pueden saturarse y dejar de prestar atención. Si marca muy pocos, puede pasar por alto situaciones importantes. Encontrar un equilibrio entre sensibilidad y precisión es uno de los desafíos prácticos de este tipo de problemas.

En contextos reales, los errores tienen consecuencias distintas. Marcar como sospechosa una operación normal puede molestar a un usuario o generar trabajo innecesario. No detectar una operación realmente problemática puede provocar pérdidas o riesgos mayores. Como en otros tipos de problemas, no existe una única configuración ideal: depende del costo de cada tipo de error.

La detección de anomalías también está muy relacionada con el monitoreo. En muchos sistemas, no basta con entrenar un modelo una vez. Hay que observar cómo cambian los datos, revisar si el comportamiento normal se modifica, actualizar criterios y analizar nuevas formas de anomalía. En entornos dinámicos, lo normal se mueve, y el modelo debe acompañar ese cambio.

Los problemas de detección de anomalías buscan identificar casos que se apartan del comportamiento habitual. Pueden aparecer en industria, seguridad, finanzas, salud, control de calidad y muchos otros ámbitos. Su dificultad está en que las anomalías suelen ser raras, variadas y dependientes del contexto. Un modelo puede ayudar a señalar casos sospechosos, pero esos señalamientos deben interpretarse con cuidado: una anomalía no siempre es un problema, y un problema no siempre aparece como una anomalía evidente.

No todos los modelos sirven para todos los problemas

Después de recorrer clasificación, regresión, agrupamiento y detección de anomalías, aparece una idea fundamental: no todos los modelos sirven para todos los problemas. Esta afirmación puede parecer obvia, pero es una de las fuentes más frecuentes de errores cuando se empieza a trabajar con machine learning.

Un modelo no es simplemente una herramienta genérica que se aplica a cualquier tabla. Cada modelo fue diseñado para cierto tipo de tarea, trabaja bajo ciertas suposiciones y produce cierto tipo de salida. Algunos modelos están pensados para predecir categorías, otros para estimar valores numéricos, otros para encontrar grupos y otros para detectar observaciones inusuales.

Por eso, antes de elegir un modelo, debemos entender qué tipo de problema tenemos entre manos. Si la respuesta que buscamos es una categoría, estamos probablemente frente a un problema de clasificación. Si buscamos una cantidad numérica, estamos

frente a regresión. Si no tenemos etiquetas y queremos descubrir grupos, hablamos de agrupamiento. Si queremos encontrar casos inusuales, hablamos de detección de anomalías.

El tipo de salida esperada ya condiciona la elección. No tendría sentido usar un modelo de regresión para predecir directamente una etiqueta como “spam” o “no spam”, salvo que lo transformemos de alguna manera y asumamos consecuencias técnicas que todavía no necesitamos abordar. Tampoco tendría sentido usar un algoritmo de agrupamiento si tenemos una variable objetivo clara y queremos predecirla en casos nuevos. En ese caso, estaríamos desaprovechando información supervisada.

También importa la forma de los datos. Algunos modelos funcionan mejor con datos numéricos ordenados en tablas. Otros pueden adaptarse a texto, imágenes, series temporales o datos con estructuras más complejas. Algunos modelos son sensibles a la escala de las variables; otros toleran mejor diferentes tipos de datos. Algunos capturan relaciones simples; otros pueden representar relaciones más complejas, pero pueden requerir más datos, más cuidado y más capacidad de cómputo.

Además, no todos los problemas tienen la misma necesidad de interpretación. En ciertos contextos, no alcanza con que un modelo prediga bien; también necesitamos explicar por qué produjo una respuesta. Esto puede ser importante en educación, salud, finanzas, políticas públicas o cualquier ámbito donde una decisión afecte a personas. En otros casos, puede priorizarse la precisión predictiva, siempre que el sistema sea evaluado y controlado adecuadamente. Esa diferencia también influye en la elección del modelo.

El tamaño y la calidad de los datos también condicionan qué modelos conviene usar. Un modelo muy flexible puede aprender patrones complejos, pero si los datos son pocos o ruidosos, puede terminar aprendiendo detalles accidentales. Un modelo más simple puede capturar menos matices, pero ofrecer resultados más estables e interpretables. No existe un modelo universalmente superior; existe una relación entre modelo, datos, problema y objetivo.

Otro factor importante es el costo de los errores. En algunos problemas, todos los errores tienen consecuencias parecidas. En otros, ciertos errores son mucho más graves que otros. Por ejemplo, en detección de anomalías industriales, no detectar una falla crítica puede ser más grave que revisar una falsa alarma. En una recomendación de contenidos, sugerir algo poco interesante puede ser aceptable. En un sistema de apoyo médico, un error puede requerir controles mucho más estrictos. El modelo y la forma de evaluarlo deben ajustarse a esas diferencias.

También debemos considerar si el problema requiere predicción, descripción, organización o decisión. A veces se quiere anticipar un resultado concreto. Otras veces se quiere explorar datos y entender segmentos. Otras, detectar comportamientos extraños. Otras, recomendar una acción. Cada objetivo pide herramientas diferentes. Usar el modelo equivocado puede llevar a resultados que parecen técnicos, pero que no responden realmente a la pregunta original.

Por eso, una parte esencial del trabajo en machine learning ocurre antes de entrenar cualquier modelo. Consiste en formular correctamente el problema. Debemos preguntarnos: ¿qué queremos obtener?, ¿tenemos una variable objetivo?, ¿esa variable es categórica o numérica?, ¿tenemos etiquetas confiables?, ¿queremos predecir, agrupar o detectar rarezas?, ¿cómo vamos a evaluar si el resultado sirve?, ¿qué errores son aceptables y cuáles no?

Estas preguntas pueden parecer menos atractivas que entrenar modelos, pero son decisivas. Un modelo sofisticado aplicado a un problema mal formulado puede producir una solución inútil. En cambio, una formulación clara permite elegir herramientas más adecuadas, evaluar mejor los resultados y evitar confusiones.

No todos los modelos sirven para todos los problemas porque cada problema tiene una estructura distinta, cada dato contiene información diferente y cada modelo produce un tipo particular de respuesta. Elegir un modelo no es solo una decisión técnica; es una consecuencia de haber entendido qué queremos resolver. En machine learning, el camino correcto no empieza preguntando “¿qué algoritmo uso?”, sino “¿qué tipo de problema tengo?”.

Elegir el enfoque según el problema

Después de recorrer los principales tipos de aprendizaje y algunos de los problemas más frecuentes en machine learning, conviene detenerse en una idea central: antes de elegir un modelo, necesitamos entender qué tipo de problema tenemos.

Cuando se comienza a estudiar machine learning, es común sentir la tentación de empezar por los algoritmos. Aparecen nombres, técnicas, bibliotecas y métodos que parecen ser el centro de la disciplina. Sin embargo, en un proyecto real, la primera decisión importante no suele ser qué algoritmo usar, sino cómo formular correctamente la pregunta.

No es lo mismo querer predecir si un correo es spam que estimar el precio de una vivienda. No es lo mismo buscar grupos de clientes con comportamientos parecidos que detectar operaciones poco habituales. No es lo mismo aprender a partir de

ejemplos con respuestas conocidas que explorar datos donde todavía no tenemos etiquetas. Cada situación pide un enfoque diferente.

Una forma útil de comenzar es preguntarse qué tenemos y qué queremos obtener. Si tenemos datos históricos con una respuesta conocida y queremos predecir esa respuesta en casos nuevos, probablemente estemos frente a un problema de aprendizaje supervisado. Si esa respuesta es una categoría, hablaremos de clasificación. Si es un valor numérico, hablaremos de regresión.

Si, en cambio, tenemos datos pero no una respuesta conocida, el enfoque puede ser no supervisado. En ese caso, quizás queramos agrupar casos parecidos, explorar estructuras internas o detectar comportamientos inusuales. El objetivo no será predecir una etiqueta previamente definida, sino descubrir una forma útil de organizar o interpretar los datos.

El aprendizaje por refuerzo queda en un lugar diferente. Allí el problema se plantea en términos de acciones, entorno y recompensas. No se trata solamente de predecir o agrupar, sino de aprender a tomar decisiones a partir de la interacción con un sistema. Aunque en este libro no lo abordaremos en profundidad, conocer su existencia ayuda a completar el mapa general.

Este recorrido nos deja una advertencia importante: usar machine learning no significa aplicar cualquier modelo disponible a cualquier conjunto de datos. Un modelo puede ser técnicamente correcto y, aun así, no responder a la pregunta que nos interesa. También puede producir resultados numéricos o gráficos que parecen sofisticados, pero que no tienen valor si el problema fue mal planteado.

Por eso, una buena práctica es formular el problema con claridad antes de avanzar. Algunas preguntas iniciales pueden ayudar mucho: ¿qué queremos predecir, estimar, agrupar o detectar?, ¿tenemos una variable objetivo?, ¿esa variable es una categoría o un número?, ¿contamos con ejemplos confiables?, ¿los datos representan el fenómeno que queremos analizar?, ¿cómo sabremos si el resultado del modelo es útil?

Estas preguntas no son un paso burocrático. Son parte del trabajo central. Un proyecto de machine learning empieza a tomar forma cuando logramos conectar una necesidad real con una tarea concreta que pueda abordarse con datos.

A medida que avancemos, estudiaremos modelos específicos, formas de preparar datos, métodos de evaluación y herramientas de Python que nos permitirán construir soluciones. Pero esa parte técnica siempre deberá apoyarse en una comprensión previa del problema. Si entendemos qué queremos resolver, podremos elegir mejor el

enfoque. Si no lo entendemos, ningún algoritmo podrá corregir por completo esa confusión inicial.

Este capítulo nos permite construir un primer mapa. El aprendizaje supervisado usa ejemplos con respuestas conocidas. El aprendizaje no supervisado busca estructuras en datos sin etiquetas. El aprendizaje por refuerzo se orienta a decisiones mediante recompensas. La clasificación predice categorías, la regresión predice valores numéricos, el agrupamiento busca grupos y la detección de anomalías señala casos inusuales.

Elegir el enfoque adecuado no garantiza por sí solo que el proyecto salga bien, pero evita empezar desde el lugar equivocado. En machine learning, entender el tipo de problema es el primer paso para elegir herramientas con sentido.

Capítulo 3 · El flujo general de un proyecto de Machine Learning

Definir el problema

Todo proyecto de machine learning debería comenzar antes de abrir nuestro IDE favorito, importar una biblioteca o entrenar un modelo. El primer paso consiste en **definir con claridad el problema que queremos resolver**.

Esta afirmación puede parecer obvia, pero en la práctica es una de las etapas más importantes y también una de las más descuidadas. Muchas veces se empieza un proyecto desde los datos disponibles o desde el algoritmo que se quiere probar. Se descarga un conjunto de datos, se cargan algunas librerías, se entrena un modelo y recién después se intenta explicar qué se estaba buscando. Ese camino puede servir como ejercicio exploratorio, pero no es una buena forma de pensar un proyecto de machine learning.

Un modelo no tiene sentido por sí solo. Tiene sentido en relación con una pregunta, una necesidad o una decisión. Antes de preguntarnos qué modelo usar, deberíamos preguntarnos qué queremos lograr. ¿Queremos anticipar un resultado? ¿Clasificar casos? ¿Estimar una cantidad? ¿Detectar comportamientos inusuales? ¿Agrupar observaciones parecidas? ¿Apoyar una decisión humana? ¿Automatizar una tarea repetitiva? Cada una de esas posibilidades conduce a enfoques distintos.

Definir el problema significa traducir una necesidad general en una tarea concreta que pueda abordarse con datos. Esta traducción no siempre es inmediata. Una necesidad real suele expresarse en lenguaje amplio: “queremos mejorar las ventas”, “queremos reducir abandonos”, “queremos detectar riesgos”, “queremos conocer mejor a nuestros usuarios”, “queremos optimizar tiempos”, “queremos priorizar casos”. Todas esas frases pueden ser puntos de partida, pero todavía no son problemas de machine learning bien definidos.

Por ejemplo, “mejorar las ventas” es un objetivo demasiado general. Puede abordarse de muchas maneras. Tal vez se quiera predecir la demanda de ciertos productos. Tal vez se quiera identificar clientes con alta probabilidad de comprar. Tal vez se quiera recomendar productos complementarios. Tal vez se quiera segmentar clientes según sus hábitos. Tal vez se quiera detectar zonas con bajo rendimiento comercial. Cada formulación lleva a un problema diferente, con datos diferentes, modelos diferentes y formas de evaluación diferentes.

Algo similar ocurre con una frase como “reducir el abandono de clientes”. Esa necesidad puede transformarse en un problema supervisado si contamos con datos históricos de clientes que abandonaron y clientes que permanecieron. En ese caso, podríamos formular la tarea como una clasificación: estimar qué clientes actuales tienen mayor probabilidad de abandonar. Pero también podría formularse de otra manera. Tal vez queremos agrupar clientes según patrones de uso para comprender distintos perfiles de comportamiento. O tal vez queremos detectar cambios inusuales en la actividad de ciertos usuarios. La necesidad general es la misma, pero la formulación técnica puede variar.

Por eso, una buena definición del problema debe aclarar qué queremos predecir, estimar, clasificar, agrupar o detectar. También debe indicar cuál será la unidad de análisis. Esto significa decidir qué representa cada caso. En algunos proyectos, cada fila puede representar una persona. En otros, una compra, una vivienda, una transacción, un paciente, un producto, un mensaje, una sesión de uso, una máquina en un momento determinado o una entrega realizada. Esta decisión parece simple, pero condiciona todo el trabajo posterior.

Imaginemos un proyecto educativo en el que se quiere detectar estudiantes en riesgo de abandonar un curso. La unidad de análisis podría ser “estudiante”, pero también podría ser “estudiante por semana”. En el primer caso, cada fila resume la información general de un estudiante durante todo el curso. En el segundo, cada fila describe el estado de un estudiante en una semana específica. La diferencia es importante, porque cambia qué datos podemos usar, qué significa la predicción y cuándo esa predicción sería útil.

Si el objetivo es intervenir a tiempo, no tendría sentido construir un modelo que use información disponible solo al final del curso. Tal modelo podría explicar el abandono después de ocurrido, pero no ayudaría a prevenirlo. Este ejemplo muestra que definir el problema no es solo elegir una variable objetivo; también implica pensar en el momento en que se hará la predicción y en el uso real que tendrá el resultado.

Una parte central de la definición del problema consiste en formular la pregunta de manera operativa. No alcanza con decir “queremos predecir riesgo”. Debemos precisar qué significa riesgo en ese contexto. ¿Riesgo de qué? ¿En qué período? ¿Medido cómo? ¿Con qué consecuencias? Un banco puede definir riesgo como la probabilidad de incumplir el pago de un crédito durante los próximos doce meses. Una empresa de mantenimiento puede definir riesgo como la probabilidad de que una máquina falle durante la próxima semana. Una institución educativa puede definir riesgo como la probabilidad de que un estudiante no complete el curso. La palabra es la misma, pero el problema es distinto.

También es necesario distinguir entre el objetivo del proyecto y la salida del modelo. El objetivo puede ser reducir costos, mejorar la atención, priorizar recursos, aumentar la seguridad o apoyar decisiones. La salida del modelo, en cambio, suele ser algo más específico: una categoría, un número, una probabilidad, un grupo o una alerta. Confundir estas dos cosas puede llevar a expectativas incorrectas.

Por ejemplo, un modelo puede estimar la probabilidad de abandono de clientes, pero no reduce el abandono por sí mismo. Para lograr ese objetivo, alguien debe decidir qué acciones tomar con esa información: contactar a ciertos clientes, ofrecer asistencia, mejorar el servicio, revisar precios o modificar procesos. El modelo produce una señal; el impacto depende de cómo se use esa señal.

Esto es especialmente importante porque machine learning no reemplaza la estrategia ni la toma de decisiones. Puede aportar información, priorizar casos, automatizar partes de un proceso o descubrir patrones, pero necesita estar conectado con una acción concreta. Si no sabemos qué haremos con la predicción, tal vez el problema todavía no está suficientemente definido.

Otra pregunta importante es cómo sabremos si el modelo ayuda. Desde el comienzo conviene pensar qué significa un buen resultado. En algunos proyectos, un buen modelo será aquel que comete errores pequeños al estimar un valor numérico. En otros, será aquel que detecta la mayor cantidad posible de casos importantes, aunque genere algunas falsas alarmas. En otros, será aquel que produce grupos interpretables y útiles para planificar acciones. La evaluación no debería pensarse recién al final; debe estar relacionada con la forma en que definimos el problema.

También debemos considerar el costo de los errores. No todos los errores son equivalentes. En un sistema que prioriza reclamos, marcar como urgente un caso que no lo era puede generar trabajo adicional. Pero no marcar como urgente un caso realmente grave puede tener consecuencias mayores. En un modelo que estima demanda, equivocarse por exceso puede generar exceso de stock; equivocarse por defecto puede provocar faltantes. Estos costos deben formar parte de la definición del problema, porque influyen en cómo se evaluará el modelo y en qué tipo de solución será aceptable.

Definir el problema también implica reconocer límites. Hay preguntas que no pueden responderse con los datos disponibles. Hay objetivos que requieren información que no fue registrada. Hay fenómenos que cambian demasiado rápido. Hay decisiones que no deberían automatizarse sin revisión humana. Una formulación responsable no solo dice qué se quiere resolver, sino también qué no se puede prometer.

Por ejemplo, un modelo puede ayudar a estimar la probabilidad de que un cliente abandone un servicio, pero no puede explicar por completo todas las razones

personales, económicas o emocionales detrás de esa decisión si esos factores no están representados en los datos. Puede detectar patrones asociados con un resultado, pero no necesariamente identificar causas profundas. Esta distinción evita transformar un modelo predictivo en una explicación total del problema.

Una buena definición del problema suele incluir varias decisiones: cuál es la necesidad real, qué tarea de machine learning se ajusta mejor a esa necesidad, cuál es la unidad de análisis, qué resultado se quiere obtener, en qué momento se usará la predicción, qué datos podrían estar disponibles, cómo se evaluará el resultado y qué acciones se tomarán a partir de él.

No siempre podremos responder todas estas preguntas de manera perfecta al inicio. En muchos proyectos, la definición se ajusta a medida que conocemos mejor los datos. Pero empezar con una formulación clara evita avanzar a ciegas. También permite detectar temprano si el proyecto es viable, si faltan datos importantes o si el enfoque elegido no responde realmente a la necesidad original.

Definir el problema es convertir una necesidad general en una tarea concreta que pueda trabajarse con datos. Es decidir qué queremos que el modelo haga, para quién será útil, cuándo se usará, cómo se evaluará y qué decisiones podrá apoyar. Sin esta etapa, el proyecto corre el riesgo de convertirse en una sucesión de pruebas técnicas sin dirección. Con una buena definición, en cambio, cada paso posterior tiene un propósito: conseguir los datos adecuados, explorarlos, elegir variables, entrenar un modelo, evaluar sus resultados y comunicar conclusiones con sentido.

Conseguir y explorar los datos

Una vez definido el problema, el siguiente paso consiste en conseguir los datos y comenzar a explorarlos. Esta etapa es fundamental porque el machine learning no trabaja sobre intenciones, sino sobre datos concretos. Podemos tener una pregunta muy interesante y una formulación clara, pero si no contamos con información adecuada, el proyecto no podrá avanzar de manera sólida.

Conseguir los datos no significa simplemente descargar un archivo o conectarse a una base de datos. Significa reunir información que tenga sentido para el problema que queremos resolver. Los datos deben estar relacionados con la tarea planteada, deben representar casos relevantes y deben estar disponibles en el momento en que se necesitarán para entrenar o usar el modelo.

Por ejemplo, si queremos estimar la demanda futura de un producto, necesitamos datos históricos de ventas y posiblemente información sobre fechas, precios,

promociones, temporadas, stock, ubicación o eventos relevantes. Si queremos predecir abandono de clientes, necesitamos registros anteriores de comportamiento y una forma clara de saber qué clientes abandonaron. Si queremos detectar anomalías en una máquina, necesitamos mediciones que describan su funcionamiento habitual y, si es posible, registros de eventos problemáticos.

En esta etapa aparece una pregunta importante: ¿los datos que tenemos realmente permiten responder el problema que formulamos? A veces la respuesta es sí. Otras veces descubrimos que los datos disponibles son insuficientes, incompletos o no fueron registrados con ese objetivo. Esto no significa necesariamente que el proyecto deba abandonarse, pero sí que debemos ajustar las expectativas, redefinir el problema o buscar nuevas fuentes de información.

También puede ocurrir que los datos existan, pero estén distribuidos en distintos lugares. Una parte puede estar en archivos CSV, otra en una base de datos, otra en planillas de cálculo, otra en sistemas internos o incluso en registros manuales. En proyectos reales, reunir los datos suele ser una tarea considerable. Antes de entrenar cualquier modelo, hay que entender dónde está la información, cómo se relaciona, qué formato tiene y qué tan confiable es.

Una vez que tenemos un conjunto inicial de datos, comienza la exploración. Explorar los datos significa observarlos con atención antes de tomar decisiones más avanzadas. No se trata todavía de construir el modelo, sino de comprender con qué material estamos trabajando.

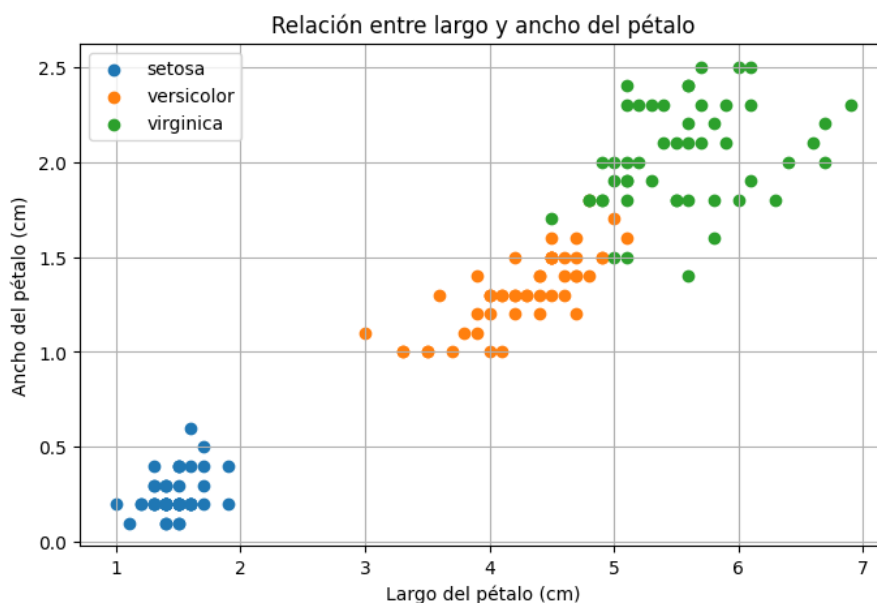
En los tomos anteriores de Ciencia de Datos con Python ya trabajamos muchas de estas tareas: cargar datos, mirar las primeras filas, revisar dimensiones, identificar tipos de variables, detectar valores faltantes, calcular resúmenes estadísticos, contar categorías, agrupar información y construir visualizaciones simples. En machine learning, esas habilidades se vuelven parte natural del flujo de trabajo.

Una exploración inicial suele comenzar con preguntas básicas. ¿Cuántas filas y columnas tenemos? ¿Qué representa cada fila? ¿Qué significa cada columna? ¿Qué tipos de datos aparecen? ¿Hay variables numéricas, categóricas, fechas o textos? ¿Hay valores faltantes? ¿Hay registros duplicados? ¿Hay valores que parecen imposibles o sospechosos? ¿Las categorías están escritas de manera consistente? ¿Las unidades de medida son claras?

Estas preguntas pueden parecer simples, pero evitan muchos problemas posteriores. Un modelo puede entrenarse con datos mal interpretados y producir resultados aparentemente válidos, pero engañosos. Si una columna fue leída como texto cuando debería ser numérica, si una fecha está en formato incorrecto, si una categoría

aparece escrita de varias formas o si una variable contiene errores de carga, el modelo puede aprender patrones distorsionados.

Explorar los datos también implica mirar la distribución de las variables. En una variable numérica, nos interesa saber si los valores están concentrados en cierto rango, si hay valores extremos, si existen ceros inesperados o si aparecen cantidades negativas donde no deberían existir. En una variable categórica, conviene revisar cuántas categorías hay, cuáles son las más frecuentes y si existen categorías con muy pocos casos.



Podemos usar gráficas para "ver" la distribución de los datos.

Estas observaciones no son detalles menores. Supongamos que queremos entrenar un modelo para clasificar transacciones sospechosas. Si descubrimos que casi todas las transacciones pertenecen a la clase normal y solo una proporción muy pequeña corresponde a casos sospechosos, eso condicionará la forma de evaluar el modelo. Si queremos estimar precios y encontramos valores extremadamente altos o bajos, tendremos que analizar si son casos reales, errores o situaciones excepcionales. Si queremos predecir abandono y la variable objetivo fue registrada con criterios distintos a lo largo del tiempo, tendremos que tenerlo en cuenta.

La exploración también permite detectar si los datos son representativos. Un conjunto de datos puede estar limpio desde el punto de vista técnico, pero aun así ser limitado. Tal vez contiene información de una sola región, de un período muy particular, de un grupo específico de usuarios o de condiciones que ya no se repiten. En ese caso, el modelo podría aprender patrones válidos para ese contexto, pero no necesariamente para otros.

Por eso, explorar datos no es solo mirar números. Es preguntarse qué parte de la realidad está representada en la tabla y qué parte queda fuera. Los datos siempre tienen una historia: fueron generados por algún proceso, registrados con ciertos criterios, almacenados en ciertos sistemas y filtrados de alguna manera antes de llegar a nosotros. Comprender esa historia ayuda a interpretar mejor lo que el modelo podrá aprender.

Otro aspecto importante es revisar la variable objetivo, cuando estamos frente a un problema supervisado. Si queremos predecir una categoría o un valor, debemos observar esa columna con especial cuidado. ¿Tiene valores faltantes? ¿Está correctamente definida? ¿Su distribución es razonable? ¿Hay categorías demasiado desbalanceadas? ¿Los valores parecen coherentes? ¿Fue construida antes o después del momento en que se haría la predicción?

Esta última pregunta es especialmente importante. En un proyecto predictivo, debemos evitar usar información que no estaría disponible en el momento real de tomar la decisión. Por ejemplo, si queremos predecir al inicio de un curso qué estudiantes están en riesgo de abandonar, no podemos usar como dato una columna que solo se conoce al final del curso. Hacerlo produciría un modelo artificialmente bueno, pero inútil en la práctica. Más adelante veremos este problema con mayor detalle bajo la idea de fuga de datos.

La exploración inicial también ayuda a decidir si el problema debe reformularse. A veces descubrimos que la variable que queríamos predecir no está disponible, que los datos son demasiado pocos, que ciertas columnas no significan lo que creíamos o que el período registrado no sirve para el objetivo original. Estos hallazgos no son fracasos; son parte normal del trabajo. Detectar temprano una limitación es mucho mejor que construir un modelo sobre una base equivocada.

También puede ocurrir lo contrario: al explorar los datos, encontramos oportunidades que no habíamos previsto. Tal vez aparecen variables muy informativas, patrones interesantes, segmentos claros o problemas de calidad que, una vez corregidos, abren nuevas posibilidades. La exploración no solo sirve para detectar errores; también sirve para comprender mejor el problema.

En esta etapa conviene mantener una actitud cuidadosa. No debemos apresurarnos a entrenar modelos antes de conocer los datos. Tampoco debemos confiar únicamente en una inspección superficial. Ver las primeras filas de una tabla puede dar una impresión inicial, pero no alcanza. Los problemas suelen aparecer al revisar distribuciones, conteos, valores faltantes, relaciones entre variables y casos extremos.

La exploración de datos cumple entonces una doble función. Por un lado, nos permite preparar el terreno técnico para el modelado. Por otro, nos ayuda a verificar si la formulación del problema sigue teniendo sentido a la luz de la información disponible. Es una etapa de diálogo entre la pregunta y los datos.

Conseguir y explorar los datos significa reunir información relevante, comprender su estructura, revisar su calidad y analizar si realmente puede apoyar el problema definido. Antes de pedirle a un modelo que aprenda, debemos saber de qué datos aprenderá. Un proyecto de machine learning sólido no empieza con un algoritmo, sino con una mirada atenta sobre los datos que harán posible, o limitarán, todo lo demás.

Elegir variables

Después de definir el problema y explorar los datos, llega una decisión clave: elegir qué variables usará el modelo para aprender. Esta etapa puede parecer simplemente técnica, pero en realidad combina conocimiento del problema, comprensión de los datos y criterio analítico.

En un conjunto de datos, no todas las columnas son necesariamente útiles para un modelo de machine learning. Algunas pueden aportar información relevante. Otras pueden ser redundantes, confusas, demasiado incompletas o directamente irrelevantes. También puede haber columnas que parecen muy predictivas, pero que no deberían usarse porque contienen información que no estaría disponible en el momento real de hacer la predicción.

Elegir variables significa decidir qué información recibirá el modelo como entrada. En un problema supervisado, también implica distinguir esas variables de la variable objetivo, es decir, de aquello que queremos predecir. Si queremos estimar el precio de una vivienda, el precio será la variable objetivo, mientras que la superficie, la ubicación, la cantidad de ambientes, la antigüedad o la presencia de cochera podrían ser variables de entrada. Si queremos predecir abandono de clientes, la variable objetivo podría indicar si el cliente abandonó o no, mientras que las variables de entrada describirían su comportamiento previo.

Esta distinción es fundamental. El modelo no debe recibir como entrada la respuesta que intentamos predecir. Puede parecer obvio, pero en la práctica no siempre es tan simple. Algunas columnas pueden estar muy relacionadas con la respuesta porque fueron generadas después del evento que queremos anticipar. Si las usamos, el modelo puede obtener resultados artificialmente buenos, pero fallará cuando lo apliquemos en una situación real.

Imaginemos un proyecto para predecir si un estudiante abandonará un curso durante las próximas semanas. Sería incorrecto usar como variable de entrada una columna que indique la fecha final de baja, la cantidad total de clases no asistidas al cierre del curso o el estado administrativo final. Esa información solo se conoce después de que el abandono ocurrió. Si el modelo la utiliza, no está aprendiendo a anticipar el problema; está recibiendo pistas del futuro.

Este tipo de error se conoce como fuga de datos, y más adelante lo estudiaremos con mayor profundidad. Por ahora, alcanza con retener la idea: al elegir variables, debemos preguntarnos si esa información estaría disponible en el momento en que el modelo debería hacer la predicción.

También debemos preguntarnos si cada variable tiene sentido para el problema. Una columna puede estar técnicamente disponible, pero no aportar información útil. Por ejemplo, un identificador interno, como un número de cliente o un código de registro, normalmente no describe una característica real del caso. Puede servir para ubicar una fila en una base de datos, pero no para aprender un patrón general. Si un modelo usa identificadores de manera inadecuada, puede terminar memorizando detalles del conjunto de datos en lugar de aprender relaciones útiles.

Otras variables pueden estar relacionadas con el problema, pero requerir interpretación cuidadosa. Por ejemplo, la ubicación de una vivienda puede ser muy importante para estimar su precio, pero no siempre aparece en una forma directamente utilizable. Puede estar registrada como barrio, código postal, coordenadas, ciudad o zona. Cada representación ofrece información distinta y puede requerir transformaciones diferentes. Lo mismo ocurre con fechas, textos, categorías, rangos o variables numéricas con unidades específicas.

Elegir variables no significa necesariamente quedarse solo con columnas tal como aparecen en la tabla original. A veces una variable útil se construye a partir de otras. Si tenemos una fecha de compra, quizá podamos derivar el mes, el día de la semana o si corresponde a una temporada especial. Si tenemos ingresos y monto de cuota, quizá podamos construir una relación entre cuota e ingreso. Si tenemos registros de actividad de usuarios, quizá podamos calcular días desde el último ingreso, frecuencia de uso o cantidad de interacciones recientes.

Este proceso de construir o transformar variables suele llamarse ingeniería de características. No necesitamos profundizar todavía en el término, pero sí conviene comprender la idea: muchas veces el modelo aprende mejor cuando representamos los datos de una manera más cercana al problema. Las variables no son solo columnas disponibles; son formas de describir los casos para que el modelo pueda encontrar patrones relevantes.

Sin embargo, agregar variables no siempre mejora el modelo. Más información no significa automáticamente mejor información. Una variable mal medida, muy ruidosa o poco relacionada con la tarea puede introducir confusión. Muchas variables similares pueden aportar información repetida. Variables con demasiados valores faltantes pueden dificultar el trabajo. Variables construidas sin cuidado pueden incluir información indebida o generar interpretaciones equivocadas.

Por eso, elegir variables también implica descartar. A veces conviene excluir columnas que no aportan, columnas con demasiados problemas de calidad o columnas cuyo uso no sería válido en el contexto real. Descartar una variable no significa que sea inútil en absoluto; significa que, para este problema y en esta etapa, no conviene usarla.

Otro aspecto importante es considerar el tipo de variable. Algunas variables son numéricas, como edad, precio, cantidad, distancia o tiempo. Otras son categóricas, como ciudad, tipo de producto, nivel educativo o método de pago. Otras pueden ser fechas, textos, identificadores, coordenadas o señales temporales. Los modelos no siempre pueden trabajar con todos estos tipos de la misma manera. Más adelante veremos cómo preparar cada tipo de dato, pero desde ahora es útil reconocer que la elección de variables también anticipa decisiones de preparación.

También debemos prestar atención a variables que pueden introducir sesgos o consecuencias no deseadas. En algunos contextos, ciertos datos pueden estar asociados con desigualdades históricas, decisiones institucionales previas o diferencias sensibles entre grupos. Usar una variable porque mejora una métrica no siempre significa que sea apropiado usarla. La elección de variables no es una decisión puramente matemática; también puede tener implicancias éticas, legales o sociales.

Por ejemplo, en un sistema que apoya decisiones sobre personas, debemos preguntarnos si las variables elegidas representan información pertinente para la tarea o si podrían llevar a decisiones injustas. Incluso variables aparentemente neutrales pueden actuar como aproximaciones de otras características sensibles. Este tema merece un tratamiento más profundo en capítulos posteriores, pero es importante mencionarlo desde el inicio: elegir variables es también elegir qué aspectos de la realidad entran en el modelo y cuáles quedan afuera.

La elección de variables suele mejorar con el conocimiento del dominio. Un analista puede detectar relaciones estadísticas, pero una persona que conoce el contexto puede ayudar a distinguir qué variables tienen sentido, cuáles son sospechosas, cuáles no estarán disponibles en producción y cuáles podrían generar decisiones problemáticas. En proyectos reales, esta etapa suele ser colaborativa.

También es habitual que la selección de variables no se resuelva de una vez para siempre. Podemos comenzar con un conjunto inicial, entrenar un modelo, evaluar resultados, revisar errores y luego ajustar. Tal vez descubramos que algunas variables no aportan demasiado, que faltan señales importantes o que cierta transformación mejora la representación del problema. El flujo de machine learning suele ser iterativo: se avanza, se revisa y se mejora.

Elegir variables significa decidir qué información usará el modelo para aprender. Esta decisión debe respetar la definición del problema, la disponibilidad real de los datos, la calidad de cada columna y el sentido que tiene cada variable en el contexto. Un modelo no aprende directamente de la realidad, sino de la representación que le damos de esa realidad. Elegir buenas variables es construir una representación más útil, más honesta y más adecuada para el problema que queremos resolver.

Separar datos de entrenamiento y prueba

Una vez que tenemos una formulación clara del problema, conocemos mejor los datos y elegimos las variables que usará el modelo, aparece una etapa fundamental: separar los datos en, al menos, dos partes. Una parte se usará para entrenar el modelo y otra parte se reservará para probarlo.

Esta idea puede parecer extraña al principio. Si ya tenemos datos disponibles, ¿por qué no usarlos todos para entrenar? La respuesta está relacionada con uno de los objetivos centrales del machine learning: no queremos que el modelo funcione bien solamente con los casos que ya conoce. Queremos que pueda generalizar a casos nuevos.

Cuando entrenamos un modelo, le mostramos ejemplos para que aprenda patrones. Pero si después evaluamos el modelo usando esos mismos ejemplos, corremos el riesgo de obtener una impresión demasiado optimista. El modelo podría haber aprendido detalles específicos de esos datos, incluso aspectos accidentales, y parecer muy bueno al responder sobre casos que ya vio. Eso no garantiza que funcione igual de bien cuando reciba datos nuevos.

Por eso, una práctica básica consiste en dividir el conjunto de datos. La parte de **entrenamiento** se usa para que el modelo aprenda. La parte de **prueba** se mantiene separada y se utiliza después, cuando el modelo ya fue entrenado, para evaluar cómo se comporta frente a casos que no vio durante el aprendizaje.

Podemos pensarlo así:



Separar datos de entrenamiento y prueba.

La separación cumple una función muy importante: simular, de manera controlada, lo que ocurrirá cuando el modelo se enfrente a datos nuevos en el mundo real. Como todavía no tenemos esos datos futuros, reservamos una parte de los datos actuales y la tratamos como si fuera nueva para el modelo.

Imaginemos que queremos construir un modelo para estimar el precio de viviendas. Tenemos datos de muchas propiedades ya vendidas. Si entrenamos el modelo con todas las viviendas y luego medimos su desempeño sobre esas mismas viviendas, el resultado puede ser engañoso. Tal vez el modelo aprendió muy bien esos casos particulares, pero no necesariamente aprendió relaciones generales que sirvan para estimar el precio de una vivienda distinta.

En cambio, si reservamos una parte de las viviendas antes de entrenar, podremos usar esos casos como una prueba más honesta. El modelo no habrá visto esas propiedades durante el entrenamiento, de modo que sus predicciones sobre ellas nos darán una idea más razonable de cómo podría comportarse con propiedades nuevas.

Lo mismo ocurre en clasificación. Si queremos predecir si una solicitud pertenece a cierta categoría, necesitamos saber si el modelo puede clasificar correctamente casos que no formaron parte del entrenamiento. Evaluarlo solo con los datos usados para aprender sería parecido a tomar una prueba con las mismas preguntas que se usaron durante la práctica. Puede servir para revisar si el modelo incorporó algo, pero no alcanza para estimar su desempeño real.

La separación entre entrenamiento y prueba es, entonces, una forma de protegernos contra una ilusión: creer que el modelo aprendió a resolver el problema cuando en realidad solo se ajustó demasiado a los ejemplos conocidos. Más adelante estudiaremos con mayor detalle este fenómeno, pero desde ahora conviene comprender la idea general. Un modelo debe ser evaluado en datos que no haya usado para aprender.

La proporción exacta entre entrenamiento y prueba puede variar. Es común reservar una parte menor para prueba y usar el resto para entrenamiento. Por ejemplo, podríamos usar el 70% u 80% de los datos para entrenar y el 30% o 20% para probar. No existe una única proporción correcta para todos los casos. La decisión depende del tamaño del conjunto de datos, del tipo de problema y de las necesidades de evaluación.

Si tenemos pocos datos, separar una parte demasiado grande para prueba puede dejar al modelo con pocos ejemplos para aprender. Si tenemos muchísimos datos, podemos reservar una porción pequeña y aun así contar con una prueba amplia. Lo importante no es memorizar una proporción, sino entender la razón de la separación.

También debemos cuidar cómo hacemos esa división. En algunos problemas, una separación aleatoria puede ser adecuada. Mezclamos los casos y asignamos una parte al entrenamiento y otra a la prueba. Pero en otros problemas, especialmente cuando interviene el tiempo, debemos ser más cuidadosos.

Por ejemplo, si queremos predecir ventas futuras, no tendría sentido entrenar con datos de meses posteriores y probar con datos de meses anteriores. En la realidad, el modelo siempre se usará para anticipar lo que viene después, no para predecir el pasado usando información del futuro. En problemas temporales, muchas veces conviene entrenar con datos más antiguos y probar con datos más recientes.

Esta idea está muy relacionada con el uso real del modelo. La separación entre entrenamiento y prueba debe parecerse, en la medida de lo posible, a la situación donde el modelo será aplicado. Si el modelo se usará para hacer predicciones sobre clientes futuros, la prueba debería representar clientes que el modelo no conoce. Si se usará en una nueva etapa temporal, la prueba debería reflejar datos posteriores. Si se aplicará en ciertos grupos o regiones, debemos revisar si esos grupos están representados adecuadamente.

En problemas de clasificación, además, suele ser importante observar la distribución de las clases en ambas partes. Si una clase es poco frecuente, una división descuidada podría dejar muy pocos casos de esa clase en el conjunto de entrenamiento o en el conjunto de prueba. Eso dificultaría tanto el aprendizaje como la evaluación. Por ejemplo, si estamos detectando eventos raros, necesitamos asegurarnos de que la prueba incluya suficientes casos relevantes para poder analizar el comportamiento del modelo.

La división de datos también debe hacerse antes de ciertas decisiones de preparación. Este punto puede parecer técnico, pero es conceptualmente importante. Si usamos toda la información disponible para decidir transformaciones, calcular valores de reemplazo o seleccionar variables, podemos introducir información del

conjunto de prueba en el proceso de entrenamiento. Eso haría que la evaluación fuera menos honesta. Más adelante veremos cómo evitarlo mediante flujos de trabajo adecuados.

Por ahora, la idea principal es esta: el conjunto de prueba debe mantenerse como una referencia externa. No debería influir en las decisiones que el modelo usa para aprender. Su función es ayudarnos a responder una pregunta: ¿qué tan bien funciona el modelo cuando se enfrenta a casos que no vio?

También conviene aclarar que separar entrenamiento y prueba no garantiza por sí solo una evaluación perfecta. Es una práctica necesaria, pero debe acompañarse con una buena formulación del problema, datos representativos, variables bien elegidas y métricas adecuadas. Si los datos de prueba no representan el contexto real, la evaluación seguirá siendo limitada. Si el problema fue mal definido, la separación no corregirá ese error.

Aun así, esta etapa marca una diferencia profunda entre simplemente ajustar un modelo y construir una solución evaluable. Reservar datos para prueba nos obliga a aceptar que el objetivo no es explicar perfectamente el pasado, sino funcionar razonablemente bien frente a datos nuevos.

Separar datos de entrenamiento y prueba permite entrenar el modelo con una parte de los datos y evaluar su desempeño con casos que no participen del aprendizaje. Esta separación es esencial para estimar la capacidad de generalización del modelo y evitar una confianza exagerada en resultados obtenidos sobre datos ya conocidos. Un proyecto de machine learning necesita aprender del pasado, pero también demostrar que puede responder ante información nueva.

Entrenar un modelo

Después de definir el problema, conseguir y explorar los datos, elegir las variables y separar los datos de entrenamiento y prueba, llegamos a una etapa que suele asociarse directamente con el machine learning: **entrenar un modelo**.

Entrenar un modelo significa permitir que un algoritmo observe los datos de entrenamiento y ajuste su comportamiento para encontrar relaciones útiles entre las variables de entrada y el resultado que queremos obtener. En un problema supervisado, esto implica aprender a relacionar las características de cada caso con una respuesta conocida. En un problema no supervisado, implica buscar estructuras, grupos o patrones sin una respuesta previa.

Aunque esta etapa suele verse como el centro del proyecto, en realidad depende profundamente de todo lo anterior. Un modelo no puede corregir por sí solo un problema mal definido, datos poco representativos o variables elegidas sin criterio. El entrenamiento es importante, pero no ocurre en el vacío. Es el momento en que el modelo trabaja con la representación del problema que nosotros construimos.

En un problema supervisado, el proceso puede pensarse de esta manera:



Entrenamiento del modelo.

Supongamos que queremos estimar el precio de una vivienda. Durante el entrenamiento, el modelo recibe ejemplos de viviendas ya vendidas. Para cada una conoce sus características, como superficie, ubicación, cantidad de ambientes o antigüedad, y también conoce el precio real de venta. A partir de esos ejemplos, intenta aprender una relación entre las características y el precio.

Si el problema fuera clasificar correos electrónicos, el modelo recibiría mensajes ya etiquetados como “spam” o “no spam”. Si el problema fuera detectar estudiantes en riesgo de abandonar un curso, recibiría datos históricos de estudiantes junto con información sobre si finalizaron o no. En todos estos casos, el modelo aprende usando ejemplos donde la respuesta ya está disponible.

La palabra “entrenar” puede generar una imagen engañosa. No estamos enseñando al modelo del mismo modo en que una persona enseña a otra mediante explicaciones, conceptos o razonamientos. El modelo no entiende el precio de una vivienda, el significado de abandonar un curso o las razones profundas por las que un cliente deja un servicio. Lo que hace es ajustar una estructura matemática para producir respuestas cada vez más cercanas a los resultados observados en los datos de entrenamiento.

Ese ajuste depende del tipo de modelo. Algunos modelos buscan relaciones simples y fáciles de interpretar. Otros pueden capturar patrones más complejos. Algunos funcionan mejor con ciertos tipos de datos, otros requieren más preparación, otros son más sensibles a valores extremos o a variables en distintas escalas. No necesitamos entrar todavía en los detalles de cada algoritmo, pero sí conviene entender que entrenar no es una acción única e idéntica para todos los modelos. Cada modelo aprende de acuerdo con sus propias reglas internas.

Aun así, la idea general se mantiene: el modelo parte de una forma inicial de responder y la modifica a partir de los datos. Si sus respuestas están lejos de los

valores o clases esperadas, debe ajustar su comportamiento. Si se acercan más, ese ajuste va en una dirección útil. Con muchos ejemplos, el modelo intenta encontrar una forma de responder que funcione razonablemente bien.

En problemas de regresión, entrenar significa aprender a estimar un número. El modelo observa casos donde conoce el valor real e intenta reducir la diferencia entre sus predicciones y esos valores. Si estima tiempos de entrega, compara sus tiempos predichos con los tiempos reales. Si estima precios, compara sus precios estimados con los precios observados. El aprendizaje se orienta a producir valores cada vez más cercanos.

En problemas de clasificación, entrenar significa aprender a asignar categorías. El modelo observa casos ya etiquetados y ajusta su criterio para distinguir mejor una clase de otra. Si clasifica solicitudes como aprobadas o rechazadas, intenta aprender qué características se asocian con cada resultado. Si clasifica reclamos por área, intenta reconocer patrones que diferencian un tipo de reclamo de otro.

En problemas no supervisados, el entrenamiento tiene otra lógica. No hay una variable objetivo contra la cual comparar cada respuesta. El modelo busca estructuras internas en los datos: grupos de casos parecidos, observaciones poco habituales o formas de resumir información. En estos casos, entrenar no significa aprender una respuesta correcta previamente definida, sino construir una organización posible de los datos.

Algo importante es que el entrenamiento utiliza solamente los datos de entrenamiento. Los datos de prueba deben quedar separados. Si permitimos que el modelo, directa o indirectamente, use información del conjunto de prueba durante el entrenamiento, la evaluación posterior deja de ser honesta. El conjunto de prueba debe representar casos no vistos, y para eso debe mantenerse fuera del proceso de aprendizaje.

También conviene entender que entrenar un modelo no garantiza que el modelo sea bueno. El entrenamiento siempre produce algún resultado, pero ese resultado puede ser útil o no. Un modelo puede aprender relaciones reales, pero también puede aprender patrones débiles, accidentales o poco generalizables. Puede funcionar bien con los datos de entrenamiento y fallar con datos nuevos. Por eso, entrenar es apenas una parte del proceso; después necesitaremos evaluar.

Esta idea es clave. Muchas personas que comienzan en machine learning sienten que el objetivo es lograr que el modelo “corra” sin errores. Pero que el código funcione no significa que el modelo sirva. Un modelo puede entrenarse correctamente desde el punto de vista técnico y aun así ser una mala solución para el problema. Puede usar

variables inadecuadas, datos insuficientes, una formulación incorrecta o una métrica que no refleja lo que realmente importa.

Durante el entrenamiento también aparecen decisiones prácticas. Debemos elegir qué tipo de modelo probar, con qué datos entrenarlo, qué transformaciones aplicar antes, qué parámetros usar y cómo organizar el flujo de trabajo. Algunas de estas decisiones se verán con detalle más adelante. En este capítulo, lo importante es ubicar el entrenamiento dentro del proceso completo y comprender su función.

Un error común es pensar que entrenar modelos consiste en probar muchos algoritmos hasta que alguno dé un número alto en una métrica. Ese enfoque puede llevar a resultados engañosos. La elección del modelo debe estar guiada por el tipo de problema, la naturaleza de los datos, la necesidad de interpretación, el costo de los errores y el uso que tendrá la predicción. No todos los modelos son igualmente adecuados para todos los contextos.

También es útil pensar el entrenamiento como una primera versión, no como un cierre definitivo. En muchos proyectos se entrena un modelo inicial para establecer una base de comparación. Luego se evalúa, se revisan errores, se ajustan variables, se mejora la preparación de datos o se prueban otros modelos. El flujo suele ser iterativo. Entrenar una vez rara vez es el final del trabajo.

Entrenar un modelo significa usar los datos de entrenamiento para que el modelo aprenda patrones relacionados con la tarea que queremos resolver. En problemas supervisados, aprende a relacionar variables de entrada con una respuesta conocida. En problemas no supervisados, busca estructuras dentro de los datos. Pero el entrenamiento solo tiene valor si forma parte de un proceso bien definido. Un modelo entrenado no es automáticamente un buen modelo; es una hipótesis aprendida a partir de datos, que luego deberá evaluarse con cuidado.

Evaluar resultados

Entrenar un modelo no alcanza. Después de entrenarlo, necesitamos evaluar sus resultados. Esta etapa responde una pregunta fundamental: ¿el modelo funciona lo suficientemente bien para el problema que queremos resolver?

La evaluación es una parte central del flujo de machine learning porque nos ayuda a distinguir entre un modelo que simplemente fue entrenado y un modelo que realmente puede ser útil. Un algoritmo puede producir predicciones, clasificaciones o agrupamientos sin errores técnicos en el código, pero eso no significa que sus respuestas sean buenas, confiables o adecuadas para el contexto.

Evaluar un modelo significa comparar su comportamiento con algún criterio de calidad. En problemas supervisados, ese criterio suele construirse comparando las predicciones del modelo con valores reales conocidos. Por eso reservamos previamente un conjunto de prueba: para tener casos que el modelo no haya visto durante el entrenamiento y poder observar cómo responde ante ellos.

En un problema de clasificación, el modelo predice categorías. La evaluación consiste en revisar cuántas veces acertó, cuántas veces se equivocó y, sobre todo, qué tipos de errores cometió. Si el modelo intenta clasificar mensajes como “spam” o “no spam”, no alcanza con saber el porcentaje total de aciertos. También importa saber cuántos correos legítimos fueron enviados por error a la carpeta de spam y cuántos correos no deseados lograron pasar como legítimos.

En un problema de regresión, el modelo predice valores numéricos. La evaluación no se basa en preguntar si acertó exactamente, porque en muchos casos eso sería poco realista. En cambio, se analiza qué tan lejos quedaron sus predicciones de los valores reales. Si el modelo estima tiempos de entrega, precios, consumos o demandas, necesitamos medir la magnitud de sus errores y decidir si esos errores son aceptables para el uso que queremos darle.

Esta última parte es muy importante: un resultado no es bueno o malo de manera absoluta. Un mismo nivel de error puede ser aceptable en un contexto e inaceptable en otro. Si un modelo estima la duración de un viaje urbano con un error promedio de dos minutos, tal vez sea bastante útil. Si un modelo estima una dosis médica con un error proporcionalmente similar, la situación sería mucho más delicada. La evaluación siempre debe interpretarse dentro del problema.

Por eso, evaluar no es solamente calcular una métrica. Las métricas son necesarias porque nos permiten resumir el desempeño del modelo, comparar alternativas y seguir un criterio más objetivo. Pero una métrica no reemplaza el juicio. Debemos preguntarnos qué significa esa métrica, qué errores oculta, a quién afectan esos errores y si el modelo mejora realmente la forma de tomar decisiones.

Supongamos que una empresa quiere detectar clientes con alta probabilidad de abandonar un servicio. Un modelo podría lograr un buen porcentaje general de aciertos, pero fallar justamente en los clientes que más interesaba detectar. Si la mayoría de los clientes permanece, un modelo que predice siempre “permanece” podría parecer correcto muchas veces, pero no serviría para anticipar abandonos. En este caso, mirar solo el acierto general puede ser engañoso.

Algo similar ocurre en problemas donde los casos importantes son poco frecuentes. Fraudes, fallas críticas, enfermedades raras o incidentes de seguridad pueden representar una pequeña proporción de los datos. Un modelo puede obtener una

métrica aparentemente alta sin detectar bien esos casos. Por eso, la evaluación debe mirar más allá de un único número.

También es necesario analizar los errores concretos. Revisar ejemplos donde el modelo se equivocó puede ser muy útil. A veces los errores revelan problemas en los datos, etiquetas incorrectas, variables insuficientes, casos ambiguos o situaciones que el modelo no puede distinguir con la información disponible. Otras veces muestran que la formulación del problema debe ajustarse.

Por ejemplo, si un modelo que estima precios comete errores muy grandes en propiedades de lujo, quizás ese segmento tenga características distintas y necesite un tratamiento especial. Si un clasificador de reclamos se equivoca con frecuencia entre dos áreas, tal vez esas categorías están mal definidas o se superponen. Si un modelo educativo falla en ciertos grupos de estudiantes, tal vez los datos disponibles no representan adecuadamente sus trayectorias.

Evaluar también implica comparar el modelo con una referencia. Antes de celebrar un resultado, conviene preguntarse: ¿mejora algo simple? A veces un modelo complejo apenas supera una estrategia básica, o incluso funciona peor. Si queremos predecir ventas, una referencia simple podría ser usar el promedio histórico o el valor de la semana anterior. Si queremos clasificar, una referencia mínima podría ser predecir siempre la clase más frecuente. El modelo debe aportar algo más que complejidad.

Esta comparación ayuda a evitar una trampa frecuente: asumir que cualquier modelo entrenado ya representa un avance. En realidad, un modelo debe justificar su uso. Si una regla sencilla, transparente y fácil de mantener ofrece resultados similares, tal vez sea preferible. Machine learning tiene sentido cuando mejora la capacidad de responder al problema, no simplemente cuando produce una solución más sofisticada.

En problemas no supervisados, la evaluación suele ser más difícil porque no siempre existe una respuesta correcta contra la cual comparar. Si agrupamos clientes, documentos o estudiantes, no tenemos necesariamente una etiqueta previa que indique cuál era el grupo verdadero. En esos casos, evaluamos si los grupos son coherentes, interpretables, estables y útiles para el objetivo del análisis. La pregunta no es solo si el algoritmo produjo grupos, sino si esos grupos ayudan a comprender o actuar mejor.

También debemos considerar que la evaluación inicial no es una garantía permanente. Un modelo puede funcionar bien en un conjunto de prueba y perder calidad con el tiempo si cambian los datos, el contexto o el comportamiento de las personas. Un modelo de demanda puede verse afectado por cambios económicos. Un

modelo de abandono puede degradarse si la empresa modifica sus planes o servicios. Un detector de anomalías puede perder utilidad si cambia el funcionamiento normal del sistema.

Por eso, evaluar resultados no debería pensarse como una etapa única y definitiva. En proyectos reales, la evaluación continúa después del primer entrenamiento. Se revisan nuevas predicciones, se monitorean errores, se comparan resultados con datos recientes y se decide si el modelo necesita ajustes. Un modelo útil no es solo el que obtiene un buen resultado una vez, sino el que mantiene un desempeño razonable en las condiciones donde será usado.

La evaluación también debe estar conectada con la comunicación. No alcanza con decir que un modelo “tiene buen desempeño”. Debemos poder explicar qué se evaluó, sobre qué datos, con qué criterio, qué errores se observaron y qué límites tiene el resultado. Una evaluación honesta permite tomar decisiones informadas; una evaluación superficial puede generar confianza excesiva.

Evaluar resultados significa analizar si el modelo cumple su propósito en datos que no usó para aprender. En clasificación, implica revisar aciertos y tipos de errores. En regresión, implica medir la distancia entre predicciones y valores reales. En problemas no supervisados, implica interpretar la utilidad y coherencia de las estructuras encontradas. Pero en todos los casos, evaluar es más que calcular una métrica: es preguntarse si el modelo ayuda realmente a resolver el problema definido.

Mejorar el modelo

Después de evaluar los resultados, suele comenzar una nueva etapa: mejorar el modelo. Esta parte del flujo es importante porque rara vez el primer modelo entrenado es la mejor versión posible. En muchos proyectos, el primer modelo funciona como una referencia inicial. Nos permite comprobar que el proceso está encaminado, detectar problemas y abrir preguntas para seguir trabajando.

Mejorar un modelo no significa simplemente buscar un número más alto en una métrica. Significa revisar el sistema completo para entender dónde están sus límites y qué cambios podrían hacerlo más útil para el problema real. A veces la mejora vendrá de elegir otro algoritmo. Pero muchas otras veces vendrá de corregir datos, ajustar variables, revisar la definición del problema, cambiar la forma de evaluación o interpretar mejor los errores.

Un error común al comenzar es pensar que mejorar el modelo consiste en probar modelos cada vez más complejos. Ese camino puede ser tentador, porque existen

muchos algoritmos disponibles y las bibliotecas de Python permiten entrenarlos con pocas líneas de código. Sin embargo, un modelo más complejo no garantiza mejores resultados. Si los datos tienen problemas, si las variables no representan bien el fenómeno o si la evaluación está mal planteada, cambiar de algoritmo puede aportar poco.

Por eso, una buena mejora empieza mirando los errores. ¿Dónde se equivoca el modelo? ¿Se equivoca de manera uniforme o falla especialmente en ciertos tipos de casos? ¿Predice peor para un grupo, una categoría, una región, un rango de valores o un período determinado? ¿Sus errores son pequeños y aceptables, o hay errores grandes concentrados en situaciones específicas?

Supongamos que un modelo estima tiempos de entrega y, en promedio, parece funcionar razonablemente bien. Pero al revisar los errores descubrimos que falla mucho en pedidos realizados durante días de lluvia o en zonas alejadas. Esa observación es más valiosa que mirar solo una métrica general. Tal vez el modelo necesita variables relacionadas con clima, zona, tipo de recorrido o disponibilidad de transporte. La mejora no surge de cambiar el algoritmo a ciegas, sino de comprender qué información le falta al modelo.

En un problema de clasificación puede ocurrir algo parecido. Un modelo puede tener un buen porcentaje de aciertos, pero equivocarse demasiado en una clase minoritaria. Si el objetivo principal era detectar justamente esa clase, entonces la mejora debe orientarse a ese problema. Quizás haya que revisar el equilibrio entre clases, elegir otra métrica, ajustar el criterio de decisión o conseguir más ejemplos relevantes. La pregunta no es solo “¿cómo subo la exactitud?”, sino “¿qué errores necesito reducir para que el modelo sea útil?”.

También puede ser necesario mejorar los datos. Si durante la evaluación aparecen errores que parecen inexplicables, puede que algunas etiquetas estén mal asignadas, que existan registros duplicados, que ciertas variables tengan valores incorrectos o que algunas columnas hayan sido interpretadas de manera equivocada. En esos casos, la mejora no está en el modelo, sino en la calidad de la información.

Otras veces el problema está en las variables disponibles. El modelo puede estar limitado porque no recibe señales suficientes para distinguir los casos. Si queremos predecir abandono de clientes, pero solo tenemos datos muy generales y no sabemos nada sobre el uso reciente del servicio, es probable que el modelo tenga poca capacidad predictiva. Si queremos estimar precios, pero no contamos con variables sobre ubicación o estado del producto, la estimación será limitada. El modelo no puede aprender relaciones que no están representadas en los datos.

Mejorar el modelo puede implicar entonces construir nuevas variables. A partir de los datos existentes, podemos crear formas más informativas de representar los casos. En lugar de usar únicamente una fecha, podemos calcular antigüedad, días desde la última actividad o temporada del año. En lugar de usar montos aislados, podemos calcular promedios, variaciones o proporciones. En lugar de registrar solo si un usuario ingresó alguna vez, podemos medir frecuencia reciente o cambios en su comportamiento.

Estas transformaciones deben hacerse con cuidado. Una nueva variable puede aportar información útil, pero también puede introducir fuga de datos si utiliza información que no estaría disponible en el momento real de la predicción. Por eso, cada mejora debe evaluarse no solo por su efecto en las métricas, sino también por su validez dentro del flujo real del proyecto.

Otra forma de mejorar consiste en ajustar el modelo elegido. Muchos algoritmos tienen configuraciones internas que modifican su comportamiento. Más adelante veremos esto con mayor detalle. Por ahora, alcanza con entender que un mismo tipo de modelo puede comportarse de manera distinta según sus ajustes. Sin embargo, esos ajustes no deben realizarse mirando directamente el conjunto de prueba una y otra vez, porque eso puede hacer que terminemos adaptando nuestras decisiones a esos datos reservados. La prueba debe seguir funcionando como una evaluación honesta.

También es posible comparar distintos modelos. Un modelo simple puede servir como punto de partida. Luego podemos probar alternativas y observar si realmente mejoran el desempeño. Pero la comparación debe ser cuidadosa. No se trata solo de elegir el modelo con mejor número en una tabla, sino de considerar estabilidad, interpretabilidad, costo computacional, facilidad de mantenimiento y adecuación al contexto.

En algunos proyectos, un modelo apenas menos preciso pero mucho más interpretable puede ser preferible. En otros, puede valer la pena usar un modelo más complejo si la mejora es importante y el contexto lo permite. La mejor elección no siempre es la que maximiza una métrica aislada. Es la que ofrece el mejor equilibrio entre desempeño, comprensión, confiabilidad y utilidad.

También puede ocurrir que la mejora más honesta sea reconocer que el modelo no está listo. Si los resultados son débiles, si los errores son demasiado costosos o si los datos no permiten aprender patrones útiles, forzar una solución puede ser peligroso. Parte del trabajo profesional consiste en saber cuándo un modelo necesita más datos, una mejor formulación o incluso cuándo machine learning no es el enfoque adecuado.

Mejorar el modelo es, por lo tanto, un proceso iterativo. Entrenamos, evaluamos, revisamos errores, ajustamos datos o variables, probamos cambios y volvemos a evaluar. Cada vuelta del proceso debería estar guiada por una hipótesis. No cambiamos cosas al azar; intentamos responder preguntas concretas. ¿Faltan variables relevantes? ¿Hay problemas de calidad? ¿El modelo es demasiado simple? ¿Está aprendiendo detalles accidentales? ¿La métrica refleja lo que importa? ¿La formulación del problema sigue siendo adecuada?

Esta etapa también requiere registrar decisiones. Si probamos varias versiones del modelo, conviene documentar qué cambió en cada una y qué resultados produjo. Sin ese registro, es fácil perderse entre pruebas y terminar sin saber qué decisión mejoró realmente el desempeño. Un proyecto de machine learning necesita experimentación, pero una experimentación ordenada.

Mejorar el modelo significa revisar críticamente los resultados y realizar cambios fundamentados para obtener una solución más útil. A veces la mejora vendrá de los datos, otras de las variables, otras del algoritmo, otras de la evaluación y otras de una mejor definición del problema. El objetivo no es perseguir una métrica de manera ciega, sino construir un modelo que generalice mejor, cometa errores más aceptables y aporte valor real al problema planteado.

Comunicar conclusiones

Después de entrenar, evaluar y mejorar un modelo, todavía queda una etapa fundamental: comunicar los resultados. Un proyecto de machine learning no termina cuando obtenemos una métrica aceptable o cuando el código funciona. Termina, al menos en una primera versión, cuando podemos explicar qué se hizo, qué se encontró, qué tan confiables son los resultados y cómo deberían interpretarse.

Comunicar conclusiones no significa decorar el trabajo con gráficos o escribir una frase final optimista. Significa transformar el proceso técnico en información comprensible para quienes necesitan tomar decisiones, revisar el proyecto o continuar trabajando sobre él. Un modelo puede tener valor técnico, pero si sus resultados no se entienden, no se pueden discutir o no se conectan con el problema original, ese valor queda limitado.

La comunicación debe volver siempre a la pregunta inicial. Si el proyecto comenzó con la intención de estimar demanda, detectar riesgo, clasificar casos, agrupar comportamientos o señalar anomalías, las conclusiones deben responder a esa necesidad. No alcanza con decir “el modelo obtuvo buen desempeño”. Debemos explicar qué significa ese desempeño en el contexto del problema.

Por ejemplo, si se construyó un modelo para estimar tiempos de entrega, una conclusión útil no debería limitarse a mencionar una métrica. Debería indicar si los errores observados son aceptables para planificar recorridos, informar a clientes o asignar recursos. Si el modelo se equivoca más en ciertas zonas, horarios o tipos de pedido, eso también debe comunicarse. Esos detalles pueden ser más importantes para la toma de decisiones que el resultado promedio.

En un problema de clasificación ocurre algo similar. Supongamos que un modelo intenta detectar casos que requieren atención prioritaria. No basta con informar cuántas veces acertó. Debemos explicar qué tipos de errores comete, cuántos casos importantes logra identificar, cuántas falsas alarmas produce y qué consecuencias podrían tener esos errores. Una persona que use el modelo necesita saber si puede confiar en sus alertas, si debe revisarlas manualmente o si solo deben funcionar como una señal orientativa.

También es importante comunicar los límites del modelo. Todo modelo tiene límites. Puede haber sido entrenado con datos de cierto período, de una región específica, de un conjunto particular de usuarios o bajo determinadas condiciones. Puede funcionar bien en casos frecuentes y peor en situaciones poco representadas. Puede depender de variables que no siempre estarán disponibles con la misma calidad. Puede producir estimaciones útiles, pero no certezas.

Explicitar esos límites no debilita el trabajo; lo vuelve más serio. Una conclusión honesta ayuda a evitar usos indebidos. Si un modelo fue entrenado con datos de un contexto determinado, no deberíamos presentarlo como una herramienta válida para cualquier contexto. Si los resultados son prometedores pero todavía iniciales, conviene decirlo. Si ciertos errores requieren revisión humana, eso debe quedar claro.

También debe comunicarse qué datos se usaron. No es necesario repetir cada detalle técnico, pero sí describir de manera comprensible la fuente de los datos, qué representa cada registro, qué período cubre la información y qué variables principales se tuvieron en cuenta. Esto permite que otras personas entiendan de qué experiencia aprendió el modelo.

Del mismo modo, es útil explicar qué quedó fuera. Tal vez no se contaba con información sobre clima, ubicación precisa, historial completo, condiciones económicas o características personales relevantes para el fenómeno. Estas ausencias ayudan a interpretar los errores y a pensar mejoras futuras. A veces una buena conclusión no solo dice qué logró el modelo, sino qué datos harían falta para avanzar.

La comunicación también debe adaptar el lenguaje a la audiencia. No es lo mismo escribir para un equipo técnico que para una dirección institucional, un equipo comercial, un grupo docente o una persona usuaria del sistema. Un equipo técnico puede necesitar detalles sobre preparación de datos, métricas, validación y versiones del modelo. Una audiencia no técnica puede necesitar una explicación más centrada en el problema, los resultados, las limitaciones y las decisiones posibles.

Esto no significa simplificar hasta perder rigor. Significa elegir qué información es necesaria para cada lector. La comunicación de un proyecto de machine learning debe ser clara, pero no engañosamente simple. Debe evitar tanto el exceso de jerga como las afirmaciones exageradas.

Un riesgo frecuente es presentar el modelo como si “descubriera la verdad” sobre el problema. En realidad, un modelo aprende patrones a partir de datos disponibles. Sus resultados son aproximaciones, probabilidades, estimaciones o agrupamientos posibles. Por eso, frases como “el modelo demuestra que...” deben usarse con cuidado. Muchas veces es mejor decir “el modelo sugiere”, “los datos muestran una asociación”, “se observa un patrón” o “la predicción indica un mayor riesgo según las variables disponibles”.

También conviene evitar conclusiones demasiado generales. Si el modelo funcionó bien en el conjunto de prueba, eso no significa automáticamente que funcionará siempre igual. Si una variable aparece asociada con el resultado, eso no implica necesariamente causalidad. Si un grupo tiene cierto comportamiento promedio, eso no significa que todos sus integrantes se comporten de la misma manera. Comunicar bien implica respetar el alcance real de lo que se analizó.

Los gráficos y tablas pueden ayudar mucho, pero deben usarse con intención. Una tabla comparativa de modelos puede mostrar cuál tuvo mejor desempeño. Un gráfico de errores puede revelar dónde falla más. Una matriz de confusión puede ayudar a entender los tipos de aciertos y errores en clasificación. Una visualización de grupos puede facilitar la interpretación de un agrupamiento. Pero cada elemento visual debe responder a una pregunta. No se trata de mostrar todo, sino de mostrar lo necesario para comprender.

Una buena comunicación suele incluir una síntesis del problema, una descripción breve de los datos, el enfoque usado, los resultados principales, las limitaciones detectadas y las recomendaciones o próximos pasos. Las recomendaciones son importantes porque conectan el análisis con la acción. Si el modelo sirve para priorizar casos, conviene explicar cómo podría usarse. Si todavía no es suficientemente confiable, conviene indicar qué habría que mejorar antes de usarlo.

También puede ser necesario comunicar incertidumbre. En muchos contextos, una predicción no debería presentarse como una respuesta única y definitiva. Si el modelo estima una demanda, puede ser útil acompañar esa estimación con una advertencia sobre variabilidad. Si clasifica un caso como riesgoso, puede ser importante indicar que se trata de una probabilidad, no de una certeza. Si detecta una anomalía, debe quedar claro que se trata de una alerta que requiere interpretación.

La comunicación cumple además una función de documentación. Quien revise el proyecto más adelante necesita saber qué decisiones se tomaron y por qué. Qué datos se usaron, qué variables se descartaron, cómo se separaron los datos, qué modelos se probaron, qué resultados se obtuvieron y qué límites se identificaron. Sin documentación, un proyecto puede volverse difícil de reproducir, mejorar o auditar.

Comunicar conclusiones es convertir el trabajo técnico en conocimiento útil. Implica explicar qué problema se abordó, qué datos sostienen el análisis, qué aprendió el modelo, cómo se evaluó, qué tan confiables son los resultados y qué límites deben tenerse en cuenta. Un buen proyecto de machine learning no solo produce predicciones; también permite que otras personas comprendan esas predicciones y decidan cómo usarlas de manera responsable.

Errores comunes al comenzar

Al llegar al final del flujo general de un proyecto de machine learning, conviene detenerse en algunos errores frecuentes. No para presentar una lista de advertencias aisladas, sino para reforzar una idea que atraviesa todo el capítulo: construir un modelo es solo una parte del trabajo. Un proyecto puede fallar aunque el código funcione, aunque el algoritmo sea conocido y aunque las métricas parezcan buenas.

Uno de los errores más comunes es empezar por el modelo antes de definir el problema. Esto ocurre cuando la atención se concentra demasiado rápido en la herramienta: qué algoritmo usar, qué biblioteca importar, qué función ejecutar. El riesgo es construir una solución técnicamente correcta para una pregunta que nunca fue formulada con claridad.

Por ejemplo, una persona podría entrenar varios modelos con una tabla de clientes y comparar resultados, pero sin haber definido si quiere predecir abandono, estimar valor futuro, segmentar comportamientos o detectar casos inusuales. En ese contexto, cualquier resultado queda flotando. Puede haber números, gráficos y predicciones, pero no una respuesta clara a una necesidad concreta.

Otro error frecuente es asumir que tener datos alcanza. Como vimos, los datos son centrales, pero no todos los datos sirven para cualquier problema. Una tabla puede tener muchas filas y columnas, y aun así no contener la información necesaria para responder la pregunta planteada. También puede contener datos incompletos, mal registrados, poco representativos o generados en condiciones que ya no se mantienen.

A veces se confunde cantidad con calidad. Más registros pueden ayudar, pero solo si aportan información relevante. Si queremos predecir fallas en una máquina, pero nuestros datos no incluyen mediciones relacionadas con esas fallas, el modelo tendrá poco para aprender. Si queremos estimar riesgo y los datos históricos reflejan decisiones tomadas con criterios inconsistentes, el modelo puede aprender esa inconsistencia. Si queremos anticipar un evento, pero las variables disponibles solo se conocen después de que el evento ocurrió, el proyecto queda mal planteado.

Esto nos lleva a otro error muy importante: usar información que no estaría disponible en el momento real de la predicción. Este problema suele ser especialmente engañoso porque produce modelos que parecen funcionar muy bien durante la evaluación inicial. El modelo obtiene resultados altos, pero no porque haya aprendido patrones generalizables, sino porque recibió información indebida.

Imaginemos un modelo que intenta predecir si un cliente abandonará un servicio, pero entre sus variables incluye una columna generada después de la baja. O un modelo que intenta anticipar si un estudiante aprobará, pero usa como dato una calificación final. En ambos casos, el modelo está usando una pista del futuro. En la práctica, esa información no estaría disponible cuando se necesita tomar la decisión. Por eso, el buen resultado es artificial.

Otro error habitual es evaluar el modelo con los mismos datos usados para entrenarlo. Si hacemos esto, no estamos midiendo su capacidad de generalizar, sino su comportamiento sobre ejemplos que ya conoce. El modelo puede parecer muy bueno porque se ajustó a los datos de entrenamiento, pero fallar cuando se enfrenta a casos nuevos. Separar datos de entrenamiento y prueba no es un formalismo: es una condición básica para obtener una evaluación más honesta.

También es común mirar una única métrica y sacar conclusiones apresuradas. Una métrica puede resumir parte del desempeño, pero rara vez cuenta toda la historia. En clasificación, un porcentaje general de aciertos puede ocultar que el modelo falla en la clase más importante. En regresión, un error promedio puede esconder errores muy grandes en ciertos grupos de casos. En agrupamiento, obtener grupos no significa que esos grupos tengan significado real.

Por eso, además de calcular métricas, conviene revisar errores, comparar resultados con una referencia simple y preguntarse si el desempeño es útil para el contexto. Un modelo no es bueno porque una métrica sea alta en abstracto; es bueno si ayuda a resolver el problema definido con errores aceptables.

Otro error frecuente es pensar que un modelo más complejo será necesariamente mejor. Esta idea aparece mucho al comenzar, porque los algoritmos más sofisticados suelen resultar atractivos. Sin embargo, complejidad no equivale a calidad. Un modelo más complejo puede capturar relaciones difíciles, pero también puede ajustarse demasiado a detalles accidentales, ser más difícil de interpretar, requerir más datos o ser más costoso de mantener.

Muchas veces conviene comenzar con modelos simples. No porque sean siempre superiores, sino porque permiten construir una base clara. Un modelo simple ayuda a entender el problema, detectar errores en los datos, establecer una referencia y comprobar si una solución más compleja realmente aporta una mejora. Si un modelo sofisticado apenas mejora a uno simple, o lo hace a costa de perder interpretabilidad en un contexto sensible, tal vez no sea la mejor elección.

También puede ser un error ignorar el contexto de uso. Un modelo no vive aislado dentro de un cuaderno. Sus resultados pueden alimentar decisiones, alertas, recomendaciones, informes o sistemas automáticos. Por eso, es necesario pensar quién usará el modelo, cuándo lo usará, qué información recibirá, qué acciones tomará y qué consecuencias tendrán los errores.

Un modelo que predice riesgo puede ser útil si se usa para priorizar una revisión humana, pero problemático si se usa para tomar decisiones automáticas sin supervisión. Un modelo que detecta anomalías puede ser valioso como sistema de alerta, pero inútil si genera tantas alertas que nadie puede revisarlas. Un modelo de recomendación puede mejorar una experiencia, pero también puede limitar la diversidad de opciones si se usa sin cuidado.

Otro error común es no documentar el proceso. Durante un proyecto se toman muchas decisiones: qué datos se usaron, qué filas se excluyeron, qué variables se eligieron, cómo se separaron los datos, qué modelos se probaron, qué métricas se analizaron y qué limitaciones se detectaron. Si esas decisiones no quedan registradas, el proyecto se vuelve difícil de revisar, explicar o mejorar.

La documentación no debe pensarse como una tarea administrativa separada del análisis. Es parte del trabajo. Permite reconstruir el camino, justificar decisiones y evitar que los resultados dependan de pasos que nadie recuerda. En un contexto educativo, además, ayuda a que el lector o evaluador comprenda el razonamiento detrás del modelo, no solo el resultado final.

También es frecuente comunicar los resultados de manera exagerada. Decir que un modelo “demuestra” algo, que “predice con certeza” o que “resuelve” un problema puede generar una confianza indebida. La mayoría de los modelos producen estimaciones, probabilidades o clasificaciones con cierto margen de error. Comunicar resultados con honestidad implica explicar qué se logró, bajo qué condiciones, con qué datos y con qué limitaciones.

Otro error es olvidar que el flujo es iterativo. Las etapas que vimos en este capítulo no siempre ocurren de manera perfectamente lineal. Podemos definir un problema, explorar datos y descubrir que necesitamos reformularlo. Podemos entrenar un modelo, evaluar errores y volver a revisar variables. Podemos comunicar resultados iniciales y recibir preguntas que obliguen a mejorar el análisis. Esto no significa que el proyecto esté mal organizado; significa que el aprendizaje sobre el problema también avanza durante el proceso.

El error sería pensar que cada etapa se completa una vez y para siempre. En la práctica, un proyecto de machine learning suele avanzar mediante revisiones sucesivas. Lo importante es que esas revisiones tengan sentido y no sean cambios al azar. Cada vuelta debería estar guiada por lo aprendido en la etapa anterior.

Finalmente, uno de los errores más importantes es olvidar que machine learning no siempre es la respuesta adecuada. Si el problema puede resolverse con una regla clara, una consulta simple, una fórmula conocida o un análisis descriptivo, quizás no haga falta entrenar un modelo. Usar machine learning cuando no aporta valor puede producir soluciones más difíciles de entender y mantener.

Un buen proyecto no se define por haber usado un algoritmo avanzado, sino por haber elegido una herramienta adecuada para una necesidad concreta. A veces esa herramienta será un modelo predictivo. Otras veces será una visualización, una tabla de resumen, una regla de negocio, una mejora en la recolección de datos o una pregunta mejor formulada.

Los errores comunes al comenzar suelen surgir de avanzar demasiado rápido hacia el modelo y demasiado poco hacia la comprensión del problema. Definir mal la tarea, confiar ciegamente en los datos, usar información del futuro, evaluar de manera superficial, perseguir complejidad sin necesidad o comunicar resultados con exceso de seguridad son formas distintas de perder el sentido del proyecto.

El flujo general que vimos en este capítulo ayuda justamente a evitar esos errores. Primero definimos el problema, luego conseguimos y exploramos datos, elegimos variables, separamos entrenamiento y prueba, entrenamos, evaluamos, mejoramos y

comunicamos. Cada etapa cumple una función. Respetarlas no garantiza el éxito, pero aumenta mucho las posibilidades de construir modelos útiles, evaluables y responsables.

Capítulo 4 · Variables de entrada y variable objetivo

El puente hacia scikit-learn

A partir de este capítulo comenzamos a acercarnos a la forma concreta en que suelen construirse modelos de machine learning con Python. En los capítulos anteriores hablamos de problemas, datos, modelos, entrenamiento y evaluación desde una perspectiva conceptual. Ahora necesitamos empezar a traducir esas ideas a una organización práctica de los datos.

En el ecosistema de Python, una de las librerías más utilizadas para trabajar con machine learning es `scikit-learn`. No es la única herramienta disponible, pero sí una de las más importantes para aprender y aplicar modelos clásicos de machine learning, especialmente cuando trabajamos con datos tabulares, es decir, con datos organizados en filas y columnas.

En los tomos anteriores de Ciencia de Datos con Python trabajamos principalmente con herramientas como `pandas`, que nos permiten cargar, explorar, limpiar, transformar y analizar datos. `scikit-learn` aparece en una etapa posterior del recorrido: cuando ya tenemos datos preparados y queremos usarlos para entrenar modelos, hacer predicciones, evaluar resultados y construir flujos de trabajo más ordenados.

Esto no significa que `pandas` deje de ser importante. Al contrario. En muchos proyectos, los datos se cargan y se exploran con `pandas`, se preparan cuidadosamente, y luego se organizan en la forma que espera `scikit-learn`. Por eso, el paso entre ciencia de datos y machine learning no es un salto brusco. Es una continuidad: primero entendemos los datos; luego los convertimos en una representación adecuada para que un modelo pueda aprender.

Una de las convenciones más importantes de `scikit-learn` es separar los datos en dos partes: las variables de entrada y la variable objetivo. Esta separación suele expresarse con dos nombres muy breves: `X` e `y`.

La letra `X` se usa para representar el conjunto de variables que el modelo recibe como entrada. Son las columnas que describen cada caso y que el modelo utilizará para aprender patrones. La letra `y` se usa para representar aquello que queremos predecir, es decir, la respuesta conocida en los datos de entrenamiento.

Por ejemplo, si tenemos una tabla de viviendas y queremos predecir el precio, `X` podría contener columnas como superficie, cantidad de ambientes, ubicación y

antigüedad. `y` sería la columna precio. Si tenemos una tabla de clientes y queremos predecir abandono, `X` podría contener datos de uso, antigüedad, cantidad de reclamos o frecuencia de compra, mientras que `y` indicaría si cada cliente abandonó o no.

En código, esta idea puede verse de una forma muy simple:

```
X = df[["superficie", "ambientes", "antigüedad"]]  
y = df["precio"]
```

Este fragmento no debe leerse solamente como una receta de sintaxis. Lo importante es la idea que representa. Estamos separando lo que el modelo va a mirar de lo que el modelo debe aprender a predecir. `X` contiene la información disponible para describir cada caso. `y` contiene la respuesta que queremos que el modelo aprenda a estimar o clasificar.

Esta separación es tan importante que aparece una y otra vez en el trabajo con `scikit-learn`. Cuando entrenamos un modelo supervisado, normalmente usamos una instrucción con esta forma general:

```
modelo.fit(X, y)
```

El método `.fit()` representa el entrenamiento. Le estamos diciendo al modelo: “observá estas variables de entrada y aprendé su relación con esta variable objetivo”. Más adelante veremos muchos ejemplos concretos, pero desde ahora conviene reconocer la estructura: un modelo supervisado necesita recibir tanto los datos de entrada como las respuestas conocidas.

Luego, cuando el modelo ya fue entrenado, podemos usarlo para hacer predicciones sobre nuevos datos de entrada:

```
predicciones = modelo.predict(X_nuevo)
```

En este caso, el modelo recibe solamente `X_nuevo`. No recibe `y`, porque justamente la idea es que produzca una predicción sobre casos cuya respuesta no conocemos o queremos estimar. Esta diferencia es fundamental. Durante el entrenamiento, el modelo aprende con ejemplos que incluyen respuestas conocidas. Durante la predicción, el modelo recibe casos nuevos y genera sus propias respuestas.

Esta lógica permite entender por qué insistimos tanto en organizar bien los datos. Si mezclamos variables de entrada con la variable objetivo, si elegimos columnas incorrectas o si incluimos información que no estaría disponible en el momento real

de la predicción, el modelo puede aprender de manera engañosa. Puede obtener buenos resultados durante una prueba inicial, pero fallar cuando se lo use de verdad.

`scikit-learn` no puede saber por sí solo qué columnas son apropiadas para el problema. La librería puede entrenar modelos, aplicar transformaciones y calcular métricas, pero la decisión de qué representa cada variable, cuál es el objetivo y qué información debe quedar fuera depende de quien desarrolla el proyecto. Por eso, antes de escribir código, necesitamos comprender con claridad qué son `X` e `y`.

Otro aspecto interesante de `scikit-learn` es que muchos modelos comparten una forma de uso parecida. Aunque los algoritmos sean distintos, la lógica general se mantiene: crear un modelo, entrenarlo con `.fit()`, usarlo para predecir con `.predict()` y evaluar sus resultados comparando las predicciones con valores reales cuando corresponda. Esta consistencia hace que la librería sea especialmente útil para aprender, porque una vez entendida la estructura general, podemos explorar distintos modelos sin tener que aprender desde cero una sintaxis completamente diferente cada vez.

Por supuesto, detrás de esa aparente simplicidad hay muchas decisiones importantes. No todos los modelos sirven para todos los problemas. No todos los datos pueden usarse tal como están. Muchas variables necesitan preparación. Algunas columnas deben transformarse. Otras deben descartarse. También debemos separar datos de entrenamiento y prueba, elegir métricas adecuadas y revisar errores. Todos esos temas aparecerán gradualmente en los próximos capítulos.

Por ahora, el objetivo es construir una base clara. En machine learning supervisado, el modelo aprende a partir de una relación entre entradas y salidas conocidas. En `scikit-learn`, esa relación se expresa de manera habitual como `X` e `y`. Esa convención no es un detalle menor ni una costumbre arbitraria. Es una forma compacta de representar la estructura del problema: qué información tenemos para describir los casos y qué resultado queremos aprender a predecir.

En el cuaderno Colab de este capítulo vamos a practicar esta idea con un DataFrame real. Veremos cómo observar sus columnas, decidir cuál será la variable objetivo, seleccionar variables de entrada y construir `X` e `y`. Pero antes de llegar a la práctica, necesitamos desarrollar bien cada concepto. Comprender esta separación es uno de los primeros pasos para trabajar con modelos de manera ordenada y evitar errores que pueden afectar todo el proyecto.

¿Qué son `X` e `y`?

En machine learning supervisado, una de las primeras separaciones que debemos hacer es distinguir entre la información que el modelo usará para aprender y la

respuesta que queremos que aprenda a predecir. Esa separación suele expresarse con dos nombres muy breves: `X` e `y`.

Aunque parezcan nombres demasiado simples, representan una idea fundamental. `X` contiene las variables de entrada, es decir, los datos que describen cada caso. `y` contiene la variable objetivo, es decir, el resultado que queremos predecir.

Podemos pensarlo de esta manera:

```
X = datos que el modelo usa para aprender
y = respuesta que queremos que el modelo aprenda a predecir
```

Si tenemos una tabla donde cada fila representa una vivienda, `X` podría estar formado por columnas como superficie, cantidad de ambientes, barrio, antigüedad y presencia de cochera. La variable `y` podría ser el precio de venta. El modelo observa muchos ejemplos donde conoce tanto las características de las viviendas como sus precios, y a partir de esa relación intenta aprender a estimar el precio de una vivienda nueva.

En una tabla de clientes, `X` podría contener información como antigüedad, cantidad de compras, monto promedio, frecuencia de uso, cantidad de reclamos o días desde la última actividad. `y` podría indicar si el cliente abandonó el servicio o no. En ese caso, el modelo intentaría aprender qué patrones en las variables de entrada se relacionan con el abandono.

La idea no depende del ejemplo. En cualquier problema supervisado necesitamos diferenciar entre aquello que describe los casos y aquello que queremos predecir. Esa diferencia es conceptual antes que técnica. Después la escribimos en Python, pero primero debemos entenderla.

Supongamos que tenemos una tabla simplificada como esta:

superficie	ambientes	antigüedad	precio
80	3	15	95000
120	4	8	150000
45	2	30	62000

Si nuestro objetivo es predecir el precio, entonces las tres primeras columnas podrían formar parte de `X`, mientras que la columna `precio` sería `y`.

```
X: superficie, ambientes, antigüedad
y: precio
```

En código, usando un DataFrame de `pandas`, podríamos escribir algo así:

```
X = df[["superficie", "ambientes", "antiguedad"]]  
y = df["precio"]
```

La variable `X` suele escribirse con mayúscula porque normalmente representa una tabla, una matriz o un conjunto de varias columnas. Cada fila de `X` corresponde a un caso, y cada columna corresponde a una variable de entrada. En cambio, `y` suele escribirse con minúscula porque normalmente representa una sola columna: la respuesta asociada a cada fila.

Esto no es una regla matemática obligatoria, sino una convención muy extendida. Podríamos usar otros nombres, como `variables_entrada` y `objetivo`, pero `X` e `y` aparecen en documentación, ejemplos, cursos, libros y bibliotecas de machine learning. Acostumbrarse a esta convención ayuda a leer código escrito por otras personas y a comprender mejor la sintaxis de herramientas como `scikit-learn`.

Lo importante es que las filas de `X` y los valores de `y` deben corresponderse. Si la primera fila de `X` describe una vivienda de 80 metros cuadrados, el primer valor de `y` debe ser el precio de esa misma vivienda. Si la fila número diez de `X` describe a cierto cliente, el valor número diez de `y` debe indicar el resultado correspondiente a ese cliente. Perder esta correspondencia sería un error grave, porque el modelo estaría intentando aprender relaciones entre datos y respuestas que no pertenecen al mismo caso.

Esta relación entre filas puede representarse así:

```
Fila 1 de X → valor 1 de y  
Fila 2 de X → valor 2 de y  
Fila 3 de X → valor 3 de y
```

Durante el entrenamiento, el modelo recibe `X` e `y` juntos. Necesita ver tanto las características como las respuestas conocidas para aprender la relación entre ambas. Esa es la lógica detrás de una instrucción como:

```
modelo.fit(X, y)
```

La expresión puede leerse como: “entrenar el modelo usando estas variables de entrada y esta variable objetivo”. El modelo ajusta su comportamiento a partir de esa información.

Pero cuando queremos predecir sobre casos nuevos, la situación cambia. En ese momento solo tenemos las variables de entrada. Justamente porque no conocemos la

respuesta, queremos que el modelo la produzca. Por eso, para predecir usamos solamente `X`:

```
predicciones = modelo.predict(X_nuevo)
```

Aquí `X_nuevo` contiene casos nuevos descritos con las mismas variables de entrada que usamos durante el entrenamiento. Si entrenamos el modelo con superficie, ambientes y antigüedad, no podemos luego pedirle que prediga usando columnas completamente distintas. El modelo aprendió a partir de una cierta representación de los datos, y las predicciones deben respetar esa misma estructura.

Esta idea es sencilla, pero muy importante. El modelo no aprende de una tabla cualquiera; aprende de una tabla organizada de una forma específica. Las columnas de `X` son la manera en que describimos cada caso. La columna `y` es la respuesta que usamos como guía durante el entrenamiento. Si esa separación está mal hecha, todo el proyecto queda comprometido.

También conviene notar que `y` no siempre contiene el mismo tipo de información. En un problema de regresión, `y` contiene valores numéricos, como precios, tiempos, consumos o cantidades. En un problema de clasificación, `y` contiene categorías, como “spam” o “no spam”, “abandona” o “permanece”, “riesgo bajo”, “riesgo medio” o “riesgo alto”.

Por ejemplo:

```
# Problema de regresión
X = df[["superficie", "ambientes", "antigüedad"]]
y = df["precio"]
```

```
# Problema de clasificación
X = df[["antigüedad_cliente", "compras_mes", "dias_sin_actividad"]]
y = df["abandono"]
```

En ambos casos usamos `X` e `y`, pero el tipo de problema cambia según lo que representa la variable objetivo. Si `y` es un número que queremos estimar, estamos frente a una regresión. Si `y` es una categoría que queremos asignar, estamos frente a una clasificación.

Esta separación también nos ayuda a pensar con más claridad qué información debería conocer el modelo y qué información no. `X` debe contener solamente datos disponibles antes de hacer la predicción. `y` representa aquello que queremos conocer, estimar o anticipar. Si incluimos dentro de `X` una columna que revela directa o indirectamente el valor de `y`, el modelo puede parecer muy preciso, pero

estará aprendiendo de manera incorrecta. Este problema se relaciona con la fuga de datos, que veremos más adelante en este mismo capítulo como una primera aproximación.

X e y son una forma compacta de representar la estructura básica de un problema supervisado. X contiene las variables que describen cada caso. y contiene la respuesta conocida que queremos aprender a predecir. Separarlas correctamente es uno de los primeros pasos prácticos para trabajar con machine learning, pero también es una decisión conceptual: define qué información usará el modelo y cuál será el objetivo de su aprendizaje.

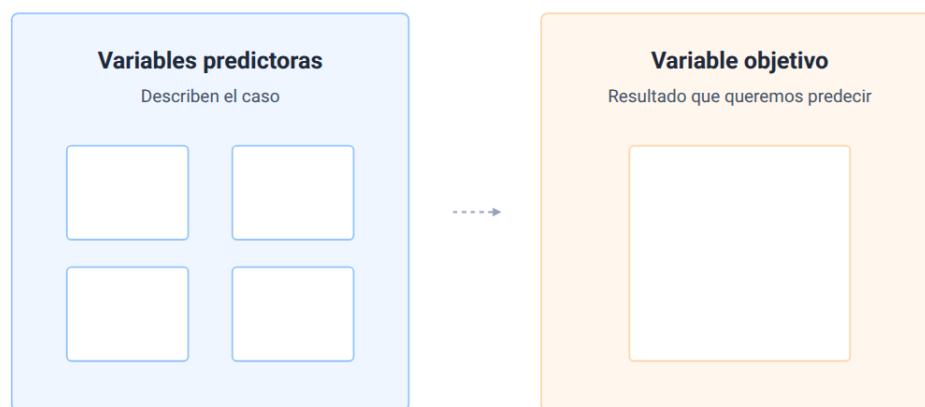
Features o variables predictoras

Las variables que forman parte de X suelen llamarse **features**, **variables predictoras**, **variables de entrada** o **características**. Todos estos nombres apuntan a la misma idea general: son los datos que describen cada caso y que el modelo utilizará para intentar aprender un patrón.

La palabra *feature* aparece mucho en la documentación, en ejemplos de código y en textos de machine learning escritos en inglés. En español, podemos traducirla como característica, atributo o variable predictora. En este libro vamos a usar estas expresiones de manera bastante cercana, aunque conviene notar un pequeño matiz: cuando decimos “variable predictora”, estamos destacando que esa columna se usa para ayudar a predecir algo.

Una variable predictora no es la respuesta del problema. Es parte de la información disponible para intentar llegar a esa respuesta. Si queremos estimar el precio de una vivienda, el precio no es una feature, sino la variable objetivo. En cambio, la superficie, la ubicación, la cantidad de ambientes, la antigüedad o la presencia de cochera pueden funcionar como variables predictoras. Son datos que el modelo puede mirar para intentar aprender cómo se relacionan con el precio.

Podemos pensarlo así:



Features o variables predictoras.

En una tabla, las variables predictoras suelen ocupar varias columnas. Cada fila representa un caso, y cada columna de **X** representa una característica de ese caso. Si tenemos una tabla de clientes, una fila puede representar a un cliente específico, y las columnas pueden describir su antigüedad, su frecuencia de compra, el monto promedio de sus operaciones, la cantidad de reclamos realizados o los días desde su última actividad.

En código, la selección de features puede verse de esta manera:

```
features = ["antigüedad_cliente", "compras_mes", "monto_promedio", "dias_sin_actividad"]  
  
X = df[features]
```

Aquí **features** es una lista con los nombres de las columnas que decidimos usar como variables de entrada. Luego, **X = df[features]** construye un nuevo DataFrame con esas columnas. Esta forma de escribirlo suele ser práctica porque deja explícita la decisión tomada: esas son las columnas que el modelo recibirá como información.

La elección de variables predictoras no debería hacerse de manera automática ni descuidada. Una columna no se vuelve útil solo porque esté disponible. Debemos preguntarnos qué representa, si está relacionada con el problema, si tiene calidad suficiente, si estará disponible en el momento real de la predicción y si su uso tiene sentido dentro del contexto.

Por ejemplo, supongamos que queremos predecir si un cliente abandonará un servicio durante el próximo mes. Algunas variables podrían ser útiles: cantidad de ingresos recientes a la plataforma, cambios en la frecuencia de uso, cantidad de reclamos, antigüedad como cliente o días desde la última interacción. Estas variables describen comportamientos que podrían estar asociados con el abandono.

En cambio, una columna como **id_cliente** probablemente no debería usarse como variable predictora. Aunque identifica a cada cliente, no describe una característica

generalizable. El modelo podría encontrar relaciones accidentales entre ciertos identificadores y ciertos resultados, pero esas relaciones no tendrían valor para nuevos clientes. Un identificador sirve para ubicar un registro, no para aprender un patrón del fenómeno.

Algo similar puede ocurrir con nombres propios, números de factura, códigos internos o campos administrativos. Pueden ser necesarios para gestionar datos, pero no necesariamente son adecuados para entrenar un modelo. La pregunta importante es si la variable aporta información que pueda generalizar a casos nuevos.

También debemos distinguir entre variables predictoras directas e indirectas. Algunas variables tienen una relación bastante clara con el objetivo. Si queremos estimar el precio de una vivienda, la superficie parece una señal naturalmente relevante. Si queremos estimar el tiempo de entrega de un pedido, la distancia parece importante. Si queremos predecir consumo eléctrico, la temperatura exterior o el horario pueden tener sentido.

Otras variables pueden ser útiles aunque su relación no sea tan evidente. En un modelo de abandono de clientes, la cantidad de veces que una persona consultó la sección de ayuda podría tener alguna relación con problemas de uso. En un modelo de demanda, el día de la semana puede aportar información aunque no cause directamente la compra. En machine learning, una variable predictora no necesita explicar por sí sola el resultado; puede aportar una señal parcial que, combinada con otras, ayude al modelo.

Esta idea es importante porque muchos modelos aprenden a partir de combinaciones de variables. Una sola columna puede no ser suficiente para predecir bien, pero varias columnas juntas pueden formar un patrón más útil. En el caso de una vivienda, la superficie por sí sola no determina el precio. La ubicación por sí sola tampoco. Pero superficie, ubicación, antigüedad, estado y cantidad de ambientes pueden ofrecer una descripción más completa.

Sin embargo, incluir muchas variables tampoco garantiza un mejor modelo. Una variable irrelevante puede agregar ruido. Una variable mal registrada puede introducir errores. Una variable con demasiados valores faltantes puede complicar la preparación. Varias variables muy parecidas pueden aportar información repetida. Elegir features es encontrar un equilibrio entre ofrecerle al modelo información suficiente y evitar información confusa, inválida o innecesaria.

También existen variables que pueden parecer predictoras, pero que en realidad revelan la respuesta. Supongamos que queremos predecir si un préstamo será aprobado y entre las variables de entrada incluimos una columna llamada `estado_final_solicitud`. Esa columna no ayuda a predecir: contiene, directa o

indirectamente, lo que queremos saber. Lo mismo ocurriría si queremos predecir abandono e incluimos una columna que solo se genera cuando el cliente ya abandonó.

Este cuidado no es un detalle técnico. Es una condición para que el modelo aprenda de manera válida. Una buena variable predictora debe estar disponible antes de la predicción y debe describir el caso sin revelar el resultado que intentamos anticipar.

En algunos proyectos, las variables predictoras se toman directamente de las columnas originales del DataFrame. En otros, se construyen nuevas variables a partir de información existente. Por ejemplo, a partir de una fecha de alta de cliente podemos calcular la antigüedad. A partir de un historial de compras podemos calcular el monto promedio o la cantidad de compras recientes. A partir de registros de actividad podemos calcular días desde la última conexión.

Estas variables derivadas pueden ser muy valiosas porque resumen información de una manera más cercana al problema. Un modelo no siempre necesita todos los registros originales de actividad de un usuario; quizás le resulte más útil recibir un resumen como “cantidad de actividades en los últimos siete días” o “variación respecto del mes anterior”. La construcción de buenas variables suele ser una de las partes más importantes de un proyecto.

De todos modos, cada variable construida debe respetar la misma regla: debe poder calcularse con información disponible antes del momento de la predicción. Si para construir una feature usamos datos posteriores al evento que queremos predecir, volvemos a caer en un problema de fuga de datos.

También conviene recordar que las features pueden tener distintos tipos de datos. Algunas son numéricas, como edad, precio, distancia, cantidad de compras o temperatura. Otras son categóricas, como ciudad, tipo de producto, método de pago o nivel educativo. Otras pueden ser fechas, textos, coordenadas o indicadores booleanos. No todas se preparan de la misma manera antes de entrenar un modelo, pero todas pueden funcionar como variables predictoras si aportan información válida para el problema.

Las features o variables predictoras son las columnas que forman X y que el modelo usará como entrada. Describen cada caso desde distintos aspectos y permiten que el modelo busque patrones relacionados con la variable objetivo. Elegirlas bien implica comprender el problema, revisar los datos, evitar columnas irrelevantes o inválidas y asegurarse de que la información esté disponible en el momento correcto. Un modelo aprende a partir de las features que le damos; por eso, la calidad de esa elección influye directamente en lo que el modelo podrá aprender.

Target, etiqueta o variable objetivo

Así como X representa las variables de entrada, y representa la **variable objetivo**. Esta variable también puede llamarse **target**, **etiqueta**, **salida esperada** o **resultado a predecir**, según el contexto. Todos estos nombres se refieren a la misma idea central: es aquello que queremos que el modelo aprenda a predecir.

En un problema supervisado, la variable objetivo es la guía del aprendizaje. El modelo observa las variables predictoras y compara esa información con el valor de y correspondiente a cada caso. A partir de esa relación, intenta aprender un patrón que luego pueda aplicar a casos nuevos.

Si tenemos una tabla de viviendas, la variable objetivo podría ser el precio. Si tenemos una tabla de clientes, podría ser si abandonaron o no un servicio. Si tenemos registros de ventas, podría ser la cantidad vendida durante la semana siguiente. Si tenemos datos de estudiantes, podría ser si finalizaron o no un curso. En todos los casos, y responde a la pregunta principal del problema.

Podemos pensarlo así:



X representa las variables de entrada, y representa la variable objetivo

Esta distinción es sencilla, pero muy importante. La variable objetivo no es una columna más. Es la columna que define la tarea de aprendizaje. Cambiar la variable objetivo cambia el problema completo, aunque usemos el mismo conjunto de datos.

Imaginemos una tabla de clientes de una empresa. Esa tabla puede contener edad, antigüedad, compras realizadas, monto promedio, ciudad, cantidad de reclamos, canal de contacto preferido y fecha de última actividad. Con esos mismos datos podríamos plantear distintos problemas. Si y indica si el cliente abandonó el servicio, tendremos un problema de clasificación. Si y representa el monto que gastará el próximo mes, tendremos un problema de regresión. Si y indica si

responderá o no a una campaña, tendremos otro problema de clasificación, distinto del anterior.

Esto muestra que el target no se elige de manera mecánica. Se elige en función de la pregunta que queremos responder. La tabla puede ser la misma, pero la tarea cambia según cuál sea la variable objetivo.

En código, la variable objetivo suele seleccionarse como una sola columna del DataFrame:

```
y = df["abandono"]
```

o bien:

```
y = df["precio"]
```

En el primer caso, si `abandono` contiene categorías como “sí” y “no”, o valores como 1 y 0 que representan esas categorías, estamos frente a un problema de clasificación. En el segundo caso, si `precio` contiene valores numéricos que representan cantidades, estamos frente a un problema de regresión.

La variable objetivo puede contener distintos tipos de valores. En clasificación, suele contener clases o etiquetas. Por ejemplo, “spam” y “no spam”, “aprobado” y “rechazado”, “bajo”, “medio” y “alto”, o valores numéricos usados como códigos de categorías. En regresión, contiene valores numéricos con significado cuantitativo, como precios, tiempos, consumos, temperaturas o cantidades.

Es importante no confundirse con los números. Una variable objetivo codificada como 0 y 1 no necesariamente indica regresión. Si 0 significa “no abandonó” y 1 significa “abandonó”, esos números representan clases. El modelo está aprendiendo a clasificar, no a estimar una cantidad continua. En cambio, si `y` contiene el monto de una compra, el precio de una propiedad o la duración de un viaje, entonces el valor numérico tiene significado como cantidad.

Una buena variable objetivo debe estar claramente definida. Si queremos predecir “abandono”, necesitamos saber qué significa abandonar. ¿Un cliente abandonó si canceló formalmente la cuenta? ¿Si no compró durante tres meses? ¿Si no ingresó a la plataforma durante cierto período? ¿Si dejó de pagar? Distintas definiciones pueden producir distintos valores de `y`, y por lo tanto distintos modelos.

Lo mismo ocurre con “riesgo”. La palabra riesgo no alcanza por sí sola. Debemos definir riesgo de qué, en qué plazo y con qué criterio. Riesgo de impago durante los próximos seis meses no es lo mismo que riesgo de atraso en la próxima cuota. Riesgo

de abandono en una plataforma educativa no es lo mismo que riesgo de bajo rendimiento. La variable objetivo debe convertir una idea general en un resultado observable o medible.

Esta definición es especialmente importante porque el modelo aprenderá a predecir exactamente aquello que hayamos puesto en **y**, no necesariamente aquello que teníamos en mente de manera informal. Si definimos abandono como “no ingresar durante 30 días”, el modelo aprenderá patrones relacionados con esa definición. Si lo definimos como “cancelar la cuenta”, aprenderá otra cosa. Puede haber relación entre ambas, pero no son idénticas.

También debemos revisar la calidad de la variable objetivo. Si **y** contiene errores, el modelo aprenderá a partir de errores. Si las etiquetas fueron asignadas con criterios inconsistentes, el modelo aprenderá esa inconsistencia. Si algunos casos están mal clasificados, si hay valores faltantes o si distintas personas usaron criterios diferentes para registrar el resultado, el aprendizaje puede verse afectado.

Por ejemplo, en una base de reclamos, una misma situación podría haber sido etiquetada como “problema técnico” por una persona y como “consulta de uso” por otra. Si esa columna se usa como variable objetivo, el modelo intentará aprender una frontera entre categorías que quizás no está bien definida. Antes de entrenar, necesitamos entender cómo se construyó **y** y qué tan confiable es.

También es importante considerar cuándo se conoce la variable objetivo. En un conjunto histórico, **y** ya está registrada porque estamos mirando casos pasados. Pero en el momento real de usar el modelo, esa variable no estará disponible. Justamente queremos predecirla. Esta diferencia entre datos históricos y casos nuevos es una de las claves del aprendizaje supervisado.

Durante el entrenamiento, el modelo ve **X** e **y**:

```
modelo.fit(X, y)
```

Pero durante la predicción, el modelo solo recibe nuevas variables de entrada:

```
predicciones = modelo.predict(X_nuevo)
```

El valor real de **y** para esos nuevos casos puede conocerse más tarde, o quizás nunca de manera directa. Por ejemplo, podemos predecir el riesgo de abandono de un cliente este mes y saber recién en el futuro si efectivamente abandonó. Podemos estimar la demanda de la semana próxima y comparar después con las ventas reales. Podemos predecir si una operación es sospechosa y confirmar más adelante si se trataba de un fraude.

En muchos proyectos, la variable objetivo se construye a partir de eventos posteriores. Eso no es un problema en sí mismo, siempre que respetemos la lógica temporal. Para entrenar un modelo de abandono, podemos usar datos históricos de clientes y mirar luego si abandonaron. Pero las variables predictoras deben corresponder a información disponible antes del abandono, no después. El target puede mirar el resultado final; las features no deben contener pistas del futuro.

La elección de `y` también condiciona la evaluación. Si elegimos una variable objetivo mal definida, la evaluación del modelo puede ser engañosa. Un modelo puede predecir muy bien una etiqueta administrativa, pero esa etiqueta quizá no represente el fenómeno que nos interesa. Puede estimar una cantidad registrada en el sistema, pero esa cantidad tal vez no sea la mejor medida del problema real. Por eso, definir el target es una decisión conceptual, no solo una selección de columna.

El target, etiqueta o variable objetivo es la respuesta que queremos que el modelo aprenda a predecir. En `scikit-learn`, suele representarse con `y`. Puede ser una categoría en problemas de clasificación o un valor numérico en problemas de regresión. Su definición debe ser clara, coherente y alineada con el problema real. Elegir bien `y` es tan importante como elegir buenas variables predictoras, porque el modelo no aprende lo que deseamos de manera abstracta: aprende aquello que la variable objetivo le muestra.

Variables numéricas y categóricas

Cuando elegimos las variables que formarán parte de `X`, no alcanza con saber qué columnas parecen importantes para el problema. También necesitamos comprender qué tipo de información contiene cada una. En machine learning, una distinción inicial muy importante es la diferencia entre **variables numéricas** y **variables categóricas**.

Una **variable numérica** es aquella cuyos valores representan cantidades. Son datos con los que tiene sentido realizar operaciones matemáticas como sumar, restar, calcular promedios, comparar magnitudes o medir distancias entre valores. La edad de una persona, el precio de un producto, la superficie de una vivienda, la cantidad de compras, el tiempo de entrega, la temperatura o el consumo eléctrico son ejemplos de variables numéricas.

Si una vivienda tiene 80 metros cuadrados y otra tiene 120, tiene sentido decir que la segunda tiene 40 metros cuadrados más que la primera. Si un cliente hizo 10 compras y otro hizo 5, tiene sentido comparar esas cantidades. Si un pedido tardó 30

minutos y otro tardó 45, podemos calcular una diferencia de 15 minutos. En todos estos casos, los números expresan magnitudes.

Una **variable categórica**, en cambio, representa grupos, clases, tipos o etiquetas. Sus valores no expresan una cantidad, sino una pertenencia. Por ejemplo, ciudad, barrio, tipo de producto, método de pago, género de una película, estado civil, categoría de cliente, nivel educativo, turno, día de la semana o tipo de reclamo.

Si una columna contiene valores como “efectivo”, “tarjeta” y “transferencia”, esos valores representan categorías de método de pago. No tiene sentido calcular un promedio entre “efectivo” y “tarjeta”. Si una columna contiene barrios, no tiene sentido decir que un barrio es numéricamente mayor que otro. Las categorías pueden compararse por igualdad o diferencia, pero no suelen tener una distancia matemática directa.

Esta distinción parece sencilla, pero en la práctica puede generar confusiones. Una variable puede estar escrita con números y, sin embargo, ser categórica. Por ejemplo, un código postal, un número de documento, un identificador de cliente o un código de producto pueden estar formados por dígitos, pero no representan cantidades. No tiene sentido decir que el cliente 2040 es “el doble” del cliente 1020, ni calcular el promedio de los códigos de producto vendidos.

Lo mismo ocurre con variables codificadas. Supongamos que una columna llamada `nivel_riesgo` usa los valores 1, 2 y 3 para representar riesgo bajo, medio y alto. Aunque la columna contenga números, esos valores pueden estar funcionando como categorías. En algunos casos existe un orden entre ellas, pero eso no significa automáticamente que la distancia entre 1 y 2 sea igual a la distancia entre 2 y 3. Es una representación codificada, no necesariamente una magnitud continua.

Por eso, al revisar un DataFrame, no debemos confiar únicamente en el tipo técnico que muestra Python. Una columna puede aparecer como `int` o `float`, pero eso no garantiza que sea una variable numérica en sentido conceptual. Debemos preguntarnos qué representa. ¿Es una cantidad medible? ¿Un código? ¿Una etiqueta? ¿Un orden? ¿Una categoría disfrazada de número?

También existen variables categóricas con orden. Por ejemplo, nivel educativo, nivel de satisfacción, tamaño de producto o categoría de riesgo pueden tener un orden natural: bajo, medio, alto; primario, secundario, universitario; pequeño, mediano, grande. Estas variables suelen llamarse **ordinales**, porque sus categorías pueden ordenarse. Aun así, deben tratarse con cuidado. Que exista un orden no significa que la distancia entre categorías sea numéricamente precisa.

Por otro lado, hay variables categóricas sin orden natural, llamadas a veces **nominales**. Ciudad, color, método de pago, tipo de producto o canal de venta son ejemplos habituales. Podemos decir que dos valores son iguales o distintos, pero no que uno sea mayor o menor que otro en sentido cuantitativo.

Esta diferencia importa porque los modelos de machine learning necesitan recibir los datos en una forma que puedan procesar. Muchos modelos trabajan internamente con números. Las variables numéricas suelen poder usarse de manera más directa, aunque a veces necesiten transformaciones o escalado. Las variables categóricas, en cambio, muchas veces deben convertirse a una representación numérica antes de entrenar un modelo.

Por ejemplo, una columna `metodo_pago` con valores como “efectivo”, “tarjeta” y “transferencia” no puede ser interpretada directamente por muchos algoritmos. Necesitamos transformarla de alguna manera. Más adelante veremos técnicas concretas para hacerlo. Por ahora, lo importante es comprender que transformar una categoría en números no significa convertirla mágicamente en una cantidad. La transformación debe respetar el significado de la variable.

Un error frecuente es asignar números arbitrarios a categorías sin pensar en las consecuencias. Por ejemplo:

```
efectivo = 1
tarjeta = 2
transferencia = 3
```

Esta codificación puede parecer cómoda, pero puede inducir una interpretación equivocada. Un modelo podría tratar esos números como si tuvieran orden o distancia. Podría interpretar que “transferencia” es mayor que “tarjeta”, o que la distancia entre efectivo y tarjeta es igual a la distancia entre tarjeta y transferencia. En realidad, esos números solo son códigos.

Esto no significa que nunca podamos codificar categorías con números. Significa que debemos hacerlo de manera adecuada al tipo de variable y al modelo que vamos a usar. Algunas técnicas respetan mejor la naturaleza nominal de las categorías; otras son apropiadas para variables ordinales. Estos detalles aparecerán más adelante, cuando trabajemos con preparación de datos.

También debemos considerar que las variables categóricas pueden tener muchas categorías distintas. Una columna como `pais` puede tener decenas de valores. Una columna como `producto_id` puede tener miles. Una variable con demasiadas categorías puede complicar el aprendizaje, especialmente si muchas categorías aparecen pocas veces. No todas las categorías ofrecen la misma cantidad de

información, y algunas pueden estar demasiado poco representadas como para que el modelo aprenda patrones confiables.

Las variables numéricas también requieren atención. Que una variable sea numérica no significa que esté lista para usarse sin revisión. Puede tener valores faltantes, valores extremos, unidades mezcladas, errores de carga o distribuciones muy desbalanceadas. Una columna de edad con valores negativos, una columna de precio con ceros imposibles o una columna de temperatura con unidades mezcladas puede afectar seriamente el aprendizaje.

Además, algunas variables numéricas pueden estar en escalas muy diferentes. Una columna puede medir edad en años, otra ingresos en pesos, otra distancia en kilómetros y otra cantidad de compras. Según el modelo utilizado, esas diferencias de escala pueden influir en el resultado. Más adelante veremos cuándo y cómo conviene tratar este problema. Por ahora, alcanza con entender que el tipo de variable condiciona su preparación.

En el armado de `X`, entonces, no solo elegimos columnas. También empezamos a reconocer qué tipo de trabajo necesitará cada una. Las variables numéricas pueden requerir limpieza, revisión de valores extremos, imputación de faltantes o cambios de escala. Las variables categóricas pueden requerir normalización de nombres, agrupamiento de categorías poco frecuentes o codificación. La preparación adecuada depende del significado de cada columna.

Esta mirada conecta directamente con el análisis exploratorio. Antes de entrenar un modelo, conviene revisar las variables numéricas y categóricas por separado. Para las numéricas, podemos observar promedios, medianas, mínimos, máximos y distribuciones. Para las categóricas, podemos revisar conteos, proporciones y categorías únicas. Estas tareas no son un paso decorativo: ayudan a detectar problemas que podrían afectar al modelo.

Las variables numéricas representan cantidades; las variables categóricas representan grupos, tipos o etiquetas. La diferencia no depende solamente de cómo están almacenadas en Python, sino de qué significan en el problema. Identificar correctamente el tipo de cada variable es esencial para preparar los datos, elegir transformaciones adecuadas y evitar interpretaciones equivocadas. Un modelo aprende a partir de la forma en que representamos la información; por eso, comprender la naturaleza de cada variable es una parte central del trabajo previo al entrenamiento.

¿Qué significa que una variable sea útil para predecir?

Una variable es útil para predecir cuando aporta información relevante sobre la variable objetivo. Dicho de otra manera, una variable predictora es útil si ayuda al modelo a distinguir casos, estimar valores o reconocer patrones relacionados con aquello que queremos anticipar.

Esta idea parece simple, pero conviene desarrollarla con cuidado. Una variable no es útil solamente porque esté disponible en la tabla. Tampoco es útil porque tenga un nombre interesante, porque sea fácil de medir o porque parezca importante a primera vista. Una variable es útil para un modelo si contiene alguna señal que permita mejorar la predicción.

Supongamos que queremos estimar el precio de una vivienda. La superficie probablemente sea una variable útil, porque en muchos mercados existe alguna relación entre el tamaño de una propiedad y su precio. Esa relación no será perfecta: una vivienda pequeña en una zona muy demandada puede valer más que una grande en una zona menos buscada. Pero la superficie aporta información. No determina por sí sola el precio, pero ayuda.

La ubicación también puede ser útil. La antigüedad puede ser útil. La cantidad de ambientes, el estado general o la presencia de cochera pueden ser útiles. Cada una de estas variables aporta una parte de la descripción del caso. El modelo intentará combinar esas señales para producir una estimación mejor que la que podría obtener mirando una sola variable aislada.

En cambio, una columna como el número interno de registro de la vivienda probablemente no sea útil para predecir el precio. Puede servir para identificar la fila dentro de una base de datos, pero no describe una característica real de la propiedad. Si el modelo encuentra alguna relación entre ciertos números de registro y ciertos precios, esa relación probablemente sea accidental y no se mantenga en casos nuevos.

Esta diferencia es muy importante. En machine learning no buscamos que el modelo memorice datos particulares, sino que aprenda relaciones que puedan generalizar. Una variable útil debe ayudar a reconocer patrones que tengan sentido más allá de los ejemplos exactos usados durante el entrenamiento.

También puede ocurrir que una variable sea útil solo en combinación con otras. Por sí sola, quizá no diga demasiado. Por ejemplo, el día de la semana puede aportar poca información para predecir ventas si lo miramos de manera aislada. Pero combinado con el tipo de producto, la temporada, las promociones y el canal de venta, puede volverse relevante. Algunos productos se venden más los fines de semana, otros

durante días laborales, otros cerca de ciertas fechas especiales. La utilidad de una variable muchas veces depende del contexto y de su relación con otras variables.

Esto muestra que la utilidad predictiva no siempre es evidente. Algunas variables parecen importantes, pero terminan aportando poco. Otras parecen secundarias, pero ayudan al modelo a mejorar. Por eso, en un proyecto real, la elección de variables combina conocimiento del problema, análisis exploratorio y evaluación empírica. Primero pensamos qué variables tienen sentido; luego observamos los datos; finalmente evaluamos si el modelo mejora al usarlas.

Una variable puede ser útil porque tiene una relación directa con el objetivo. Si queremos estimar el consumo eléctrico de una vivienda, la cantidad de habitantes puede estar relacionada con el consumo. Si queremos predecir el tiempo de entrega de un pedido, la distancia puede estar relacionada con la duración. Si queremos estimar el rendimiento de un cultivo, la cantidad de lluvia puede ser relevante.

Pero una variable también puede ser útil aunque no represente una causa directa. Puede funcionar como señal indirecta. Por ejemplo, en una plataforma digital, los días desde la última actividad de un usuario pueden ayudar a predecir abandono. Esa variable no “causa” necesariamente el abandono, pero puede estar asociada con desinterés, problemas de uso o pérdida de hábito. El modelo puede usar esa señal para anticipar un resultado, aunque la explicación completa del fenómeno sea más amplia.

Esta distinción entre asociación y causa es fundamental. Para predecir, muchas veces alcanza con encontrar variables asociadas al resultado. Pero eso no significa que esas variables expliquen causalmente lo que ocurre. Un modelo puede detectar que ciertos comportamientos aparecen antes de un abandono, pero eso no prueba que esos comportamientos sean la causa. Puede haber otros factores no registrados. Puede tratarse de una consecuencia temprana del problema. Puede ser una señal útil, pero no una explicación completa.

También debemos tener cuidado con variables que parecen demasiado útiles. Si una columna permite predecir la variable objetivo casi perfectamente, conviene revisar por qué. Puede ser que realmente sea una variable muy informativa, pero también puede ser una señal de fuga de datos. Tal vez la columna fue generada después del evento que queremos predecir, o contiene una codificación directa del resultado, o resume información que no estaría disponible en el momento real de la predicción.

Por ejemplo, si queremos predecir si un cliente abandonará un servicio y encontramos una columna llamada `fecha_baja`, esa columna está directamente relacionada con el target. Usarla como variable predictora sería incorrecto, porque

solo se conoce cuando el abandono ya ocurrió. El modelo parecería excelente, pero no estaría anticipando nada. Estaría usando una pista del futuro.

Una variable útil, entonces, no solo debe estar relacionada con el objetivo. También debe ser válida para el momento en que se hará la predicción. Esta condición es tan importante que la desarrollaremos con más detalle en las próximas secciones.

Otra cuestión relevante es la calidad de la variable. Una variable puede ser conceptualmente útil, pero estar mal registrada. Si muchos valores faltan, si hay errores de carga, si las unidades están mezcladas o si las categorías se escriben de manera inconsistente, su utilidad real puede disminuir. Por ejemplo, la ubicación puede ser muy importante para estimar precios de viviendas, pero si aparece escrita de formas diferentes, con errores o con niveles de detalle incompatibles, el modelo puede tener dificultades para aprovecharla.

También puede ocurrir que una variable sea útil para algunos casos y poco útil para otros. La presencia de cochera puede influir mucho en el precio de una vivienda en una zona con poco estacionamiento, pero tener menor impacto en otra zona. La antigüedad de un cliente puede ser relevante en ciertos servicios, pero no en otros. Una variable no tiene una utilidad universal; su valor depende del problema, del contexto y de los datos disponibles.

En algunos casos, la utilidad de una variable puede cambiar con el tiempo. Una señal que era predictiva en un período puede dejar de serlo si cambian las condiciones. Por ejemplo, los patrones de compra pueden modificarse por cambios económicos, nuevas políticas comerciales o cambios en los hábitos de los usuarios. Un modelo aprende relaciones presentes en los datos históricos, pero esas relaciones deben seguir siendo razonablemente válidas para que la variable continúe siendo útil.

También debemos evitar incluir variables solo porque aumentan una métrica durante una prueba inicial. Una mejora numérica puede ser real, pero también puede deberse a casualidad, fuga de datos, sobreajuste o particularidades del conjunto de prueba. La utilidad de una variable debe analizarse con criterio. Debe aportar información válida, interpretable en el contexto y disponible cuando corresponda.

Una variable es útil para predecir cuando aporta una señal relevante sobre la variable objetivo y ayuda al modelo a mejorar sus respuestas en casos nuevos. Esa utilidad puede ser directa o indirecta, individual o combinada con otras variables. Pero no basta con que la variable esté asociada al resultado: también debe tener calidad suficiente, representar información válida y estar disponible antes de la predicción. Elegir variables útiles es uno de los pasos donde más claramente se combinan conocimiento del problema, análisis de datos y responsabilidad en el diseño del modelo.

Variables disponibles antes de la predicción vs. variables conocidas después

Cuando armamos `X`, una de las preguntas más importantes no es solamente qué variables parecen relacionadas con la variable objetivo, sino **cuándo están disponibles esas variables**. Esta dimensión temporal es fundamental para construir modelos válidos.

En un conjunto de datos histórico, muchas columnas pueden aparecer juntas en la misma tabla. Podemos ver datos iniciales, datos intermedios, resultados finales, fechas de cierre, estados administrativos y valores calculados después de que ocurrió el evento que queremos predecir. Como todo está disponible en el DataFrame, puede parecer que cualquier columna puede usarse como variable predictora. Pero desde el punto de vista del modelo, eso no es cierto.

Una variable solo debería formar parte de `X` si estaría disponible en el momento real en que queremos hacer la predicción. Si una columna se conoce recién después del resultado que queremos anticipar, no debe usarse como entrada del modelo, aunque esté presente en la tabla histórica.

Esta idea es clave porque el objetivo de un modelo predictivo no es explicar un resultado cuando ya ocurrió, sino anticiparlo o estimarlo en una situación donde todavía no lo conocemos. Para que el entrenamiento sea válido, debemos recrear esa situación de la manera más honesta posible.

Imaginemos un proyecto educativo. Queremos predecir, durante las primeras semanas de cursada, qué estudiantes tienen mayor riesgo de abandonar. En una base histórica puede haber muchas columnas: edad, curso elegido, fecha de inscripción, cantidad de accesos durante la primera semana, actividades entregadas al inicio, promedio final, cantidad total de inasistencias, estado final del estudiante y fecha de baja.

Algunas de esas variables podrían estar disponibles al momento de la predicción. Por ejemplo, la edad, el curso elegido, la fecha de inscripción, la cantidad de accesos durante la primera semana o las primeras actividades entregadas. Esos datos podrían usarse si realmente están disponibles cuando queremos generar la alerta.

Otras variables no deberían usarse. El promedio final, la cantidad total de inasistencias al cierre del curso, el estado final o la fecha de baja se conocen después. Si las incluimos en X , el modelo no estará aprendiendo a anticipar el abandono. Estará usando información posterior al evento.

La diferencia puede representarse así:



Variables válidas y no válidas.

Esta misma idea aparece en muchos contextos. Si queremos predecir si un cliente abandonará un servicio el próximo mes, podemos usar información de uso anterior: frecuencia de ingreso, cantidad de reclamos previos, antigüedad, cambios recientes en el consumo o días desde la última actividad. Pero no deberíamos usar una columna como `fecha_baja`, `motivo_baja` o `estado_final_cliente`, porque esas variables se conocen cuando el abandono ya ocurrió.

Si queremos estimar el precio de venta de una vivienda antes de publicarla, podemos usar superficie, ubicación, antigüedad, cantidad de ambientes o características del inmueble. Pero no deberíamos usar el precio final de venta, el descuento negociado al cierre o los días reales que tardó en venderse, si esos datos solo se conocen después de la operación.

Si queremos predecir la demanda de un producto para la semana próxima, podemos usar ventas históricas, calendario, precios vigentes, promociones planificadas o stock disponible antes de esa semana. Pero no podemos usar las ventas reales de la semana que queremos predecir. Esa sería justamente la respuesta.

En todos los casos, la pregunta central es la misma: **¿esta información existiría cuando el modelo tenga que hacer la predicción?**

Esta pregunta ayuda a ordenar el armado de X . No alcanza con mirar la tabla como una foto completa del pasado. Hay que reconstruir la línea de tiempo del problema. Primero ocurre cierta información inicial. Luego llega el momento en que queremos predecir. Más tarde se conoce el resultado. Las variables de entrada deben pertenecer al período anterior a la predicción o, como máximo, al momento exacto en que se predice. La variable objetivo pertenece al resultado que queremos conocer después.

Podemos pensarlo como una línea temporal:



Hay que reconstruir la línea de tiempo del problema.

Esta forma de pensar evita muchos errores. En los datos históricos, X e y pueden estar en la misma fila, pero conceptualmente pertenecen a momentos distintos. La fila resume un caso completo, pero el modelo debe aprender como si estuviera parado en el momento de la predicción, sin acceso al futuro.

También debemos tener cuidado con variables derivadas. A veces una columna no revela directamente el resultado, pero fue calculada usando información posterior. Por ejemplo, una variable llamada `total_compras_ultimos_12_meses` puede ser válida o no según el momento de predicción. Si se calcula usando los doce meses anteriores al momento en que se predice, puede ser válida. Pero si se calcula usando meses posteriores, incluiría información futura.

Lo mismo ocurre con promedios, acumulados, estados finales, indicadores de cierre o resúmenes históricos. Una variable construida puede parecer razonable por su nombre, pero debemos revisar cómo fue calculada. La disponibilidad temporal no depende solo de la columna, sino también del proceso que la generó.

Este cuidado es especialmente importante cuando trabajamos con datos ya preparados por otra persona o por un sistema. Muchas bases de datos no fueron diseñadas pensando en machine learning. Pueden incluir columnas administrativas que resumen el estado final de un caso, campos que se actualizan con el tiempo o

indicadores que mezclan información de diferentes momentos. Antes de usarlas como features, debemos entender su significado.

También puede ocurrir que una variable esté disponible en teoría, pero no en la práctica. Por ejemplo, una organización puede registrar cierta información, pero con varios días de demora. Si el modelo debe predecir en tiempo real, esa variable quizá no esté disponible cuando se necesita. O puede existir en una base interna, pero no estar accesible para el sistema que usará el modelo. La disponibilidad no es solo conceptual; también es operativa.

Por eso, una buena selección de variables debe tener en cuenta el escenario real de uso. Debemos imaginar el momento exacto en que el modelo recibirá datos y preguntarnos qué información tendrá efectivamente a mano. Ese ejercicio conecta la preparación de datos con la definición del problema.

Esta distinción también ayuda a evitar modelos engañosamente buenos. Cuando usamos variables conocidas después del resultado, el modelo suele obtener métricas muy altas durante la evaluación. Parece haber aprendido muy bien, pero en realidad recibió pistas que no existirán en producción. El problema no se nota mirando solo la métrica; se detecta comprendiendo la lógica temporal del problema.

No todas las columnas disponibles en una tabla histórica son variables válidas para entrenar un modelo predictivo. Una feature debe estar disponible antes o en el momento de la predicción. Las variables conocidas después del resultado deben quedar fuera de X , aunque estén muy relacionadas con y . Esta diferencia es una de las bases para evitar fuga de datos y construir modelos que puedan funcionar en situaciones reales, no solo en una tabla del pasado.

Primer acercamiento a la fuga de datos

La **fuga de datos** es uno de los errores más importantes y más engañosos en machine learning. Ocurre cuando, durante el entrenamiento o la evaluación, el modelo recibe información que no debería estar disponible en el momento real de hacer una predicción.

El problema es que esa información indebida puede hacer que el modelo parezca mucho mejor de lo que realmente es. Sus métricas pueden ser altas, sus errores pueden parecer bajos y sus predicciones pueden dar una sensación de gran precisión. Pero esa calidad es artificial. El modelo no aprendió a resolver el problema de manera válida; aprendió usando pistas que en la práctica no tendrá.

La fuga de datos puede pensarse como una filtración de información desde el futuro hacia el entrenamiento. En los datos históricos, muchas columnas aparecen juntas porque ya conocemos toda la historia del caso. Pero cuando usamos un modelo en una situación real, no estamos parados al final de la historia. Estamos parados en un momento anterior, tratando de anticipar algo que todavía no sabemos.

Supongamos que queremos predecir si un estudiante abandonará un curso. En una base histórica podríamos tener columnas como cantidad de accesos durante la primera semana, actividades entregadas al inicio, mensajes enviados al docente, promedio final, fecha de baja y estado final. Si usamos `estado_final` o `fecha_baja` como variables predictoras, el modelo obtendrá resultados excelentes, porque esas columnas revelan directamente el resultado. Pero no nos sirven para anticipar nada. Cuando queramos usar el modelo con estudiantes actuales, esas columnas todavía no existirán.

En ese caso, el modelo no aprendió señales tempranas de abandono. Aprendió a leer una consecuencia del abandono. Es como intentar predecir quién ganará un partido usando como variable el resultado final. La predicción parece perfecta, pero no tiene ningún valor predictivo real.

La fuga de datos también puede ser menos evidente. No siempre aparece con columnas tan obvias como `fecha_baja`, `estado_final` o `resultado`. A veces se esconde en variables calculadas. Por ejemplo, si queremos predecir abandono durante las primeras semanas de un curso, una variable como `total_actividades_entregadas` puede ser problemática si fue calculada usando todo el curso completo. Aunque el nombre parezca razonable, incluye información posterior al momento de la predicción.

Lo mismo puede ocurrir en un proyecto comercial. Si queremos predecir qué clientes abandonarán el próximo mes, no podemos usar variables construidas con información de ese próximo mes. Si una columna resume “cantidad de reclamos del mes siguiente” o “consumo total del período posterior”, está trayendo información del futuro. El modelo puede aprovecharla durante el entrenamiento histórico, pero no la tendrá cuando se use para anticipar nuevos casos.

También puede haber fuga de datos cuando se usan variables que son consecuencia directa de la decisión o del resultado. Imaginemos un modelo que intenta predecir si una solicitud de crédito será aprobada. Si entre las variables incluimos el monto finalmente otorgado, la tasa asignada o el estado final de la solicitud, probablemente estemos usando información posterior a la decisión. Esas columnas no describen la solicitud antes de evaluarla; describen lo que ocurrió después.

La fuga de datos es peligrosa porque no siempre se detecta mirando el código. Un modelo con fuga puede entrenarse correctamente, sin errores técnicos. La sintaxis puede estar perfecta. Las métricas pueden ser muy buenas. Justamente por eso es un problema tan engañoso: desde el punto de vista del programa, todo parece funcionar.

La forma de detectarla requiere comprender el problema y la línea temporal de los datos. Hay que preguntarse, para cada variable: ¿esta información estaría disponible cuando el modelo tenga que predecir? ¿Fue registrada antes del resultado o después? ¿Fue calculada usando datos futuros? ¿Resume información que no existiría en el momento real? ¿Revela directa o indirectamente la variable objetivo?

Estas preguntas son más importantes que cualquier automatismo. Ninguna librería puede saber por sí sola qué columnas son válidas en el contexto real de uso. `scikit-learn` puede entrenar un modelo con las columnas que le entregamos, pero no puede advertirnos que una variable administrativa contiene información posterior al evento. Esa responsabilidad queda en manos de quien diseña el proyecto.

Una señal de alerta aparece cuando el modelo obtiene resultados demasiado buenos para ser creíbles. No siempre significa que haya fuga de datos, pero conviene revisar. Si un problema difícil se resuelve con una precisión casi perfecta usando pocas variables, tal vez alguna columna esté revelando la respuesta. Si una variable tiene una relación extremadamente fuerte con el target, conviene examinar cómo se construyó y cuándo se conoce.

Otra forma de fuga puede aparecer durante la preparación de los datos. Por ejemplo, si usamos todo el conjunto de datos, incluyendo los datos de prueba, para tomar ciertas decisiones de transformación, podemos contaminar la evaluación. El conjunto de prueba debe simular datos no vistos. Si usamos información de esa parte para preparar el entrenamiento, aunque sea de manera indirecta, la evaluación deja de ser completamente honesta. Más adelante veremos técnicas para organizar correctamente estas transformaciones y evitar este problema.

Por ahora, el punto principal es entender que la fuga de datos rompe la separación entre lo que el modelo debería saber y lo que intenta predecir. El modelo recibe información que no tendría en una situación real, y por eso su desempeño estimado deja de representar su verdadero comportamiento futuro.

La fuga de datos no es solo un problema técnico. También puede generar decisiones equivocadas. Una organización puede confiar en un modelo que parece excelente durante las pruebas, implementarlo y descubrir que no funciona. Esto puede producir pérdida de tiempo, costos económicos, errores operativos o decisiones injustas si el modelo se aplica sobre personas. Por eso, detectar y prevenir fuga de datos es una práctica central en machine learning responsable.

Volvamos a la estructura `X` e `y`. La variable objetivo, `y`, representa lo que queremos predecir. Las variables de entrada, `X`, deben contener solamente información válida para hacer esa predicción. Si alguna columna de `X` contiene la respuesta, una parte de la respuesta o una consecuencia posterior de la respuesta, estamos en riesgo de fuga.

Una buena regla inicial es revisar cada variable candidata para `X` con una pregunta simple: **¿podría conocer este dato antes de necesitar la predicción?** Si la respuesta es no, esa variable debe quedar fuera. Si la respuesta es dudosa, conviene investigar cómo se genera y en qué momento está disponible.

La fuga de datos ocurre cuando el modelo aprende usando información que no estaría disponible en el momento real de predecir. Puede aparecer por columnas posteriores al evento, variables calculadas con datos futuros, consecuencias del resultado o transformaciones mal organizadas. Su efecto es especialmente peligroso porque produce modelos que parecen buenos, pero que no generalizan de manera válida. Evitarla empieza con una separación cuidadosa entre `X` e `y`, una comprensión clara del problema y una revisión temporal de cada variable que pensamos usar.

Preparar la información que verá el modelo

A lo largo de este capítulo trabajamos una idea que será central en todo el libro: un modelo de machine learning no aprende directamente de la realidad, sino de la información que le entregamos. Esa información debe estar organizada, seleccionada y separada de manera cuidadosa.

La separación entre `X` e `y` es uno de los primeros pasos para construir esa organización. `X` contiene las variables que describen cada caso. `y` contiene la respuesta que queremos aprender a predecir. Esta convención, muy usada en `scikit-learn`, no es solamente una cuestión de nombres. Es una forma de representar la estructura del problema: qué datos tendrá disponible el modelo y qué resultado intentará estimar o clasificar.

Al construir `X`, elegimos las variables predictoras o features. Esa elección no debería hacerse simplemente tomando todas las columnas disponibles. Cada variable debe ser revisada: qué significa, qué tipo de dato contiene, si tiene relación con el problema, si aporta información útil, si está bien registrada y si estará disponible en el momento real de la predicción. Una columna puede estar presente en el `DataFrame` y, aun así, no ser adecuada para entrenar el modelo.

También vimos que las variables pueden ser numéricas o categóricas, y que esta diferencia depende del significado de la información, no solo de cómo aparece almacenada en Python. Una variable numérica representa una cantidad. Una variable categórica representa una clase, grupo o etiqueta. Identificar correctamente esta diferencia será importante más adelante, cuando preparemos los datos para que distintos modelos puedan utilizarlos.

La variable objetivo, por su parte, define qué problema estamos resolviendo. Si `y` contiene un valor numérico que queremos estimar, estaremos frente a un problema de regresión. Si contiene una categoría que queremos predecir, estaremos frente a un problema de clasificación. Cambiar la variable objetivo cambia la tarea, incluso si usamos el mismo conjunto de datos.

Uno de los aprendizajes más importantes del capítulo es que una variable predictora no solo debe estar relacionada con el objetivo. También debe ser válida. Para que una variable pueda formar parte de `X`, debe estar disponible antes o en el momento en que el modelo hará la predicción. Las columnas conocidas después del resultado, o calculadas con información futura, pueden producir una fuga de datos.

La fuga de datos es peligrosa porque genera modelos que parecen muy buenos, pero que aprendieron usando información que no tendrán en una situación real. Por eso, al preparar `X`, debemos mirar cada variable con una pregunta clave: ¿este dato existiría cuando el modelo tenga que predecir? Si la respuesta es no, esa columna no debe usarse como variable de entrada.

Este capítulo prepara el terreno para el trabajo práctico que haremos en los próximos pasos. Antes de entrenar modelos, calcular métricas o comparar algoritmos, necesitamos construir correctamente la información que verá el modelo. Una buena separación entre `X` e `y`, una elección cuidadosa de features y una revisión atenta de la variable objetivo son condiciones básicas para que el aprendizaje tenga sentido.

Preparar la información que verá el modelo significa decidir qué datos usará para aprender y qué resultado intentará predecir. Es una etapa conceptual y técnica al mismo tiempo. Técnica, porque luego se traduce en código con DataFrames, columnas y métodos de `scikit-learn`. Conceptual, porque define la forma en que el problema real será representado para el modelo. Si esa representación es clara y válida, el proyecto puede avanzar sobre una base sólida.

Capítulo 5 · Separar datos: entrenamiento y prueba

¿Por qué no evaluamos el modelo con los mismos datos con los que entrenamos?

Cuando entrenamos un modelo de machine learning, nuestro objetivo no es que funcione bien solamente con los datos que ya vio. Queremos que pueda responder de manera razonable frente a casos nuevos. Esta idea, que ya apareció en capítulos anteriores bajo el concepto de generalización, se vuelve especialmente importante cuando llega el momento de evaluar un modelo.

Un modelo aprende a partir de ejemplos. Durante el entrenamiento observa datos, ajusta su comportamiento y busca patrones que le permitan producir mejores predicciones. Pero si después medimos su desempeño usando exactamente esos mismos datos, corremos el riesgo de obtener una evaluación engañosa. El modelo puede parecer muy bueno porque está respondiendo sobre casos que ya formaron parte de su aprendizaje.

Esto se parece a evaluar a un estudiante con los mismos ejercicios que memorizó durante la práctica. Si responde bien, no sabemos si comprendió el procedimiento o si simplemente recuerda esos casos particulares. Para saber si realmente aprendió, necesitamos presentarle ejercicios nuevos, parecidos en estructura, pero no idénticos. En machine learning ocurre algo similar: para evaluar si el modelo aprendió patrones útiles, necesitamos probarlo con datos que no haya usado para entrenar.

Supongamos que tenemos un conjunto de datos de viviendas y entrenamos un modelo para estimar precios. Si usamos todas las viviendas disponibles para entrenar y luego evaluamos el modelo sobre esas mismas viviendas, el resultado puede ser demasiado optimista. El modelo ya tuvo acceso a esos ejemplos. Tal vez aprendió relaciones generales, pero también pudo ajustarse a detalles específicos de esas propiedades, a particularidades del conjunto de datos o incluso a ruido presente en los registros.

El problema no es que el modelo “haga trampa” de manera intencional. Un modelo no tiene intención. El problema es que, durante el entrenamiento, ajusta su comportamiento para responder bien sobre los datos disponibles. Si lo evaluamos sobre esos mismos datos, estamos midiendo en parte qué tan bien se adaptó a ellos, no necesariamente qué tan bien funcionará con casos nuevos.

En machine learning, esta diferencia es fundamental. Un modelo puede cometer pocos errores en los datos de entrenamiento y, sin embargo, tener un desempeño

pobre cuando se aplica a nuevos datos. Esto ocurre cuando aprendió demasiado los detalles particulares del conjunto de entrenamiento y no logró capturar patrones generales. Más adelante estudiaremos este fenómeno con mayor profundidad, pero por ahora basta con entender la idea: un buen resultado sobre los datos de entrenamiento no garantiza un buen resultado en la práctica.

Por esa razón, necesitamos reservar una parte de los datos para evaluar. Esa parte no debe participar del entrenamiento. Se mantiene separada hasta que el modelo ya fue ajustado. Luego se usa para simular lo que ocurriría cuando el modelo reciba casos nuevos.

Podemos pensarlo así:



Separamos los datos en conjuntos de entrenamiento y prueba.

La separación entre entrenamiento y prueba nos permite hacer una evaluación más honesta. El modelo aprende con un conjunto de casos y luego se prueba con otros. Si funciona bien en esos datos no vistos, tenemos una señal más razonable de que aprendió patrones que pueden generalizar. Si funciona bien en entrenamiento pero mal en prueba, eso nos indica que algo no está funcionando como esperamos.

Esta separación no convierte la evaluación en una verdad absoluta. El conjunto de prueba también es una muestra limitada de la realidad. Puede estar mejor o peor construido, puede representar ciertos casos y dejar otros afuera, puede pertenecer a un período particular o a una población específica. Pero aun con esas limitaciones, evaluar en datos separados es mucho más confiable que evaluar sobre los mismos datos usados para entrenar.

Imaginemos un modelo que clasifica solicitudes como aprobadas o rechazadas. Si lo entrenamos y evaluamos con las mismas solicitudes históricas, tal vez obtengamos un porcentaje de aciertos muy alto. Pero cuando lleguen solicitudes nuevas, el desempeño podría caer. Tal vez el modelo había aprendido patrones demasiado específicos de los casos anteriores. Tal vez ciertas combinaciones de variables aparecían en el conjunto histórico, pero no se repiten de la misma manera en los nuevos datos. Tal vez había ruido o decisiones inconsistentes que el modelo memorizó.

En cambio, si reservamos una parte de las solicitudes como conjunto de prueba, podemos observar cómo se comporta el modelo con casos que no formaron parte del aprendizaje. Esa evaluación se parece más a la situación real: el modelo recibe información de un caso nuevo y debe producir una predicción sin conocer de antemano la respuesta.

En problemas de regresión ocurre lo mismo. Si un modelo estima tiempos, precios, consumos o demandas, no alcanza con medir el error sobre los datos usados para entrenar. Necesitamos saber qué tan lejos quedan sus predicciones en casos que no vio. Un error bajo en entrenamiento puede ser alentador, pero solo el desempeño en datos no vistos nos ayuda a estimar su utilidad real.

Esta idea también nos protege de una ilusión frecuente: creer que un modelo es mejor solo porque se adapta mucho a los datos disponibles. En realidad, el objetivo no es ajustarse perfectamente al pasado. El objetivo es aprender regularidades que sigan siendo útiles más allá de los ejemplos concretos. Un modelo que memoriza demasiado puede parecer excelente en entrenamiento y fallar en prueba. Un modelo que generaliza bien puede cometer algunos errores en entrenamiento, pero mantener un desempeño razonable en datos nuevos.

Por eso, cuando evaluamos modelos, debemos distinguir entre desempeño en entrenamiento y desempeño en prueba. El desempeño en entrenamiento nos dice qué tan bien el modelo se ajustó a los datos con los que aprendió. El desempeño en prueba nos da una aproximación más importante: qué tan bien podría funcionar con datos nuevos similares.

Esta distinción también ayuda a tomar decisiones. Si el modelo funciona mal tanto en entrenamiento como en prueba, probablemente no está capturando patrones suficientes. Tal vez faltan variables, el modelo es demasiado simple, los datos no contienen señales relevantes o el problema está mal formulado. Si funciona muy bien en entrenamiento pero mucho peor en prueba, tal vez se ajustó demasiado a los datos de entrenamiento. Si funciona razonablemente bien en ambos, podríamos tener una primera señal de que el modelo está aprendiendo patrones más estables.

No necesitamos resolver todavía todos esos casos. Lo importante en este capítulo es comprender por qué la evaluación debe hacerse sobre datos separados. Entrenar y evaluar con los mismos casos mezcla dos preguntas distintas: “¿el modelo pudo ajustarse a estos datos?” y “¿el modelo puede generalizar a datos nuevos?”. Para machine learning, la segunda pregunta es la más importante.

No evaluamos un modelo con los mismos datos con los que lo entrenamos porque eso puede generar una confianza exagerada en su desempeño. Un modelo debe demostrar que puede responder ante casos no vistos. Separar datos de entrenamiento y prueba nos permite construir una evaluación más honesta y acercarnos mejor al objetivo real del machine learning: aprender patrones que sean útiles más allá de los ejemplos usados durante el entrenamiento.

train_test_split

En Python, cuando trabajamos con `scikit-learn`, una de las formas más habituales de separar los datos en entrenamiento y prueba es usar la función `train_test_split`.

Esta función forma parte del módulo `model_selection`, que reúne herramientas relacionadas con la división de datos, la evaluación de modelos y la organización de experimentos. Su tarea básica es tomar nuestros datos y devolver dos subconjuntos: uno para entrenar el modelo y otro para evaluarlo.

La idea general es esta:

```
from sklearn.model_selection import train_test_split
```

Luego, cuando ya tenemos contruidos `X` e `y`, podemos separarlos así:

```
X_train, X_test, y_train, y_test = train_test_split(X, y)
```

A primera vista, esta línea puede parecer un poco cargada, porque aparecen cuatro variables nuevas. Pero cada una tiene un significado muy claro.

`X_train` contiene las variables de entrada que usaremos para entrenar el modelo. `y_train` contiene las respuestas correspondientes a esas filas. Es decir, `X_train` e `y_train` forman juntos el conjunto de entrenamiento.

`X_test` contiene variables de entrada de casos que el modelo no verá durante el entrenamiento. `y_test` contiene las respuestas reales de esos casos, que reservaremos para evaluar las predicciones. Es decir, `X_test` e `y_test` forman juntos el conjunto de prueba.

Podemos leer la estructura completa de esta manera:



train_test_split se encarga de hacer la división.

La función `train_test_split` se encarga de hacer la división manteniendo la correspondencia entre las filas de `X` y los valores de `y`. Esto es fundamental. Si una fila de `X` describe una vivienda determinada, el valor correspondiente de `y` debe ser el precio de esa misma vivienda. Si una fila describe a cierto cliente, el valor correspondiente de `y` debe ser el resultado de ese mismo cliente. La separación no puede desordenar esa relación.

Por ejemplo, si antes de dividir teníamos esta correspondencia:

```
Fila 1 de X → valor 1 de y
Fila 2 de X → valor 2 de y
Fila 3 de X → valor 3 de y
```

Después de dividir, esa correspondencia debe mantenerse tanto en entrenamiento como en prueba. `train_test_split` hace esa tarea por nosotros. Puede mezclar las filas antes de separarlas, pero mantiene alineadas las entradas y sus respuestas.

Esto es una ventaja importante frente a separar manualmente las columnas sin cuidado. Si mezcláramos `X` por un lado y `y` por otro, podríamos romper la relación entre los datos y la respuesta. En ese caso, el modelo intentaría aprender a partir de ejemplos mal emparejados, lo que volvería inválido el entrenamiento.

Una separación típica podría escribirse así:

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
```

```
y,  
test_size=0.2,  
random_state=42  
)
```

En esta versión aparecen dos argumentos que usaremos con frecuencia. `test_size=0.2` indica que el 20% de los datos se reservará para prueba y el 80% se usará para entrenamiento. `random_state=42` permite controlar la aleatoriedad de la división para que el resultado pueda reproducirse. Más adelante en este mismo capítulo vamos a detenernos en estos argumentos con más detalle.

Por ahora, lo importante es comprender qué devuelve la función y por qué necesitamos esas cuatro partes.

Durante el entrenamiento, usamos solamente `X_train` e `y_train`:

```
modelo.fit(X_train, y_train)
```

El modelo aprende a partir de esos casos. Observa las variables de entrada de entrenamiento y las compara con sus respuestas conocidas. A partir de esa información, ajusta su comportamiento.

Después, para evaluar, usamos `X_test`:

```
predicciones = modelo.predict(X_test)
```

El modelo recibe datos de entrada que no vio durante el entrenamiento y genera predicciones. Luego comparamos esas predicciones con `y_test`, que contiene los valores reales reservados para la evaluación.

La lógica completa puede representarse así:



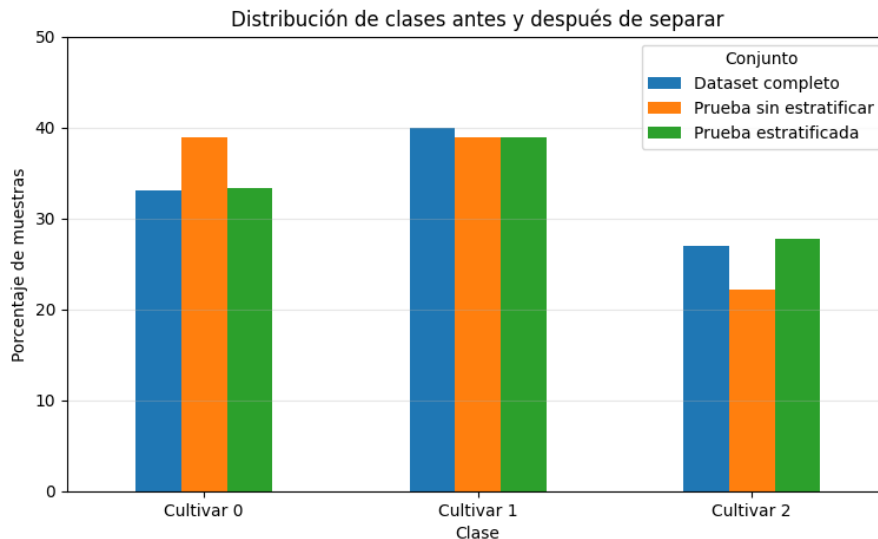
Estructura básica de scikit-learn.

Esta estructura es una de las bases del trabajo con `scikit-learn`. Aparecerá muchas veces a lo largo del libro, con distintos modelos y distintos conjuntos de datos. Aunque cambien los algoritmos, la lógica suele mantenerse: separar datos, entrenar con la parte de entrenamiento, predecir sobre la parte de prueba y comparar contra las respuestas reales.

Es importante notar que `train_test_split` no mejora el modelo por sí misma. Su función no es entrenar, limpiar datos ni elegir variables. Su función es organizar el conjunto de datos para que podamos evaluar de manera más honesta. Es una herramienta sencilla, pero cumple un rol metodológico muy importante.

También conviene recordar que esta función realiza una separación general. En muchos problemas tabulares simples, una división aleatoria puede ser adecuada. Pero no todos los problemas deben dividirse de la misma manera. Si los datos tienen una estructura temporal, por ejemplo, puede no ser correcto mezclarlos al azar. Si queremos predecir el futuro, debemos cuidar que los datos de prueba representen un período posterior al de entrenamiento. Este tipo de situaciones requiere criterios adicionales.

En problemas de clasificación, además, puede ser necesario cuidar la proporción de clases en entrenamiento y prueba. Si una clase es poco frecuente, una separación aleatoria podría dejar pocos ejemplos de esa clase en alguno de los conjuntos. Para eso existe la estratificación, que veremos más adelante en este capítulo.



Una separación correcta queda mucho más cerca de la distribución original.

Por eso, aunque `train_test_split` sea una función muy usada, no debe aplicarse sin pensar. Siempre debemos preguntarnos si la forma de separar los datos tiene sentido para el problema. La herramienta facilita la división, pero el criterio sobre cómo dividir pertenece al diseño del proyecto.

`train_test_split` es una función de `scikit-learn` que nos permite separar `X` e `y` en conjuntos de entrenamiento y prueba. Devuelve cuatro partes: `X_train`, `X_test`, `y_train` e `y_test`. Con las primeras entrenamos el modelo; con las segundas evaluamos su comportamiento en casos no vistos. Esta separación es uno de los pasos prácticos más importantes para construir una evaluación más confiable y evitar medir el modelo únicamente sobre los datos que usó para aprender.

Conjuntos de entrenamiento y prueba

Cuando usamos `train_test_split`, el resultado no es solamente una división técnica de una tabla. Estamos creando dos conjuntos con funciones diferentes dentro del proyecto: el conjunto de entrenamiento y el conjunto de prueba.

El **conjunto de entrenamiento** es la parte de los datos que el modelo usa para aprender. En un problema supervisado, está formado por `X_train` e `y_train`. `X_train` contiene las variables de entrada de los casos de entrenamiento, mientras que `y_train` contiene las respuestas conocidas correspondientes a esos casos.

Podemos pensarlo así:

```
modelo.fit(X_train, y_train)
```

Esta línea resume una idea muy importante. El modelo observa los casos de entrenamiento, compara las variables de entrada con las respuestas conocidas y ajusta su comportamiento para encontrar patrones. Si estamos frente a una regresión, intentará aprender a estimar un valor numérico. Si estamos frente a una clasificación, intentará aprender a asignar una categoría.

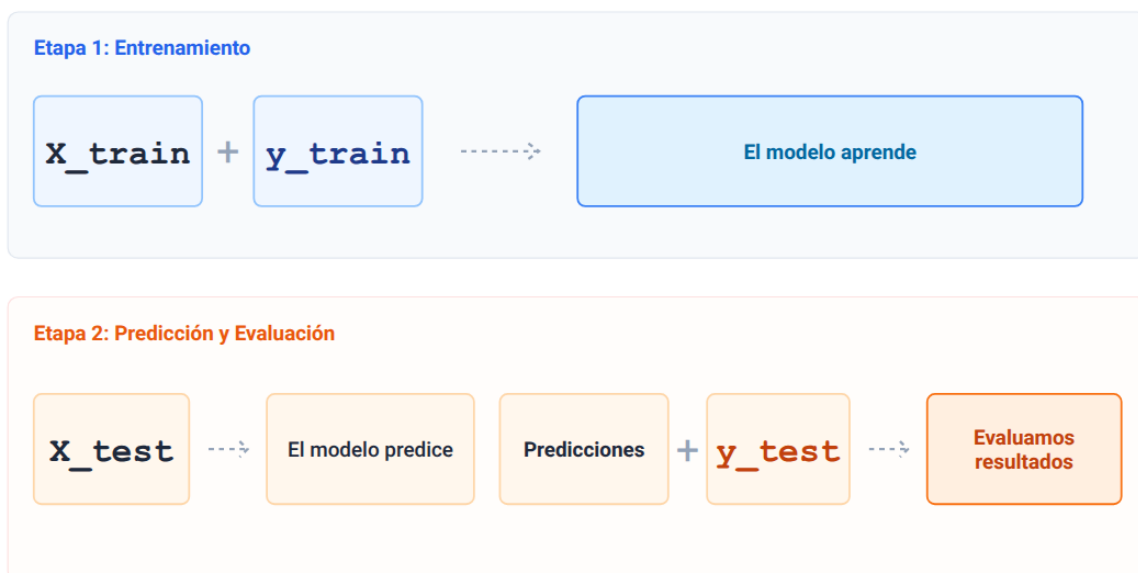
El **conjunto de prueba**, en cambio, cumple otra función. No se usa para entrenar. Se reserva para evaluar. Está formado por `X_test` e `y_test`. `X_test` contiene casos que el modelo no vio durante el entrenamiento. `y_test` contiene las respuestas reales de esos casos, pero esas respuestas no se le muestran al modelo cuando hace sus predicciones.

La evaluación suele seguir esta lógica:

```
predicciones = modelo.predict(X_test)
```

El modelo recibe `X_test`, produce predicciones y luego esas predicciones se comparan con `y_test`. Esa comparación nos permite estimar qué tan bien se comporta el modelo con casos no vistos.

La diferencia entre estos conjuntos puede representarse de manera sencilla:



Entrenamiento y prueba.

Esta separación es uno de los pilares del flujo de trabajo en machine learning. El conjunto de entrenamiento responde a la pregunta: "¿con qué datos va a aprender el

modelo?”. El conjunto de prueba responde a otra pregunta: “¿cómo se comporta el modelo con datos que no usó para aprender?”.

Es importante mantener esta diferencia clara. `y_test` no se utiliza para entrenar el modelo. Se guarda para evaluar. Si durante el entrenamiento el modelo tuviera acceso directo o indirecto a `y_test`, la evaluación perdería valor. Dejaría de representar una prueba sobre datos no vistos y pasaría a estar contaminada por información que debía permanecer reservada.

Esto también ayuda a entender por qué aparecen cuatro variables y no solo dos. No alcanza con separar `X` por un lado e `y` por otro. Necesitamos conservar la estructura completa del problema en ambos subconjuntos. En entrenamiento necesitamos entradas y respuestas para que el modelo aprenda. En prueba necesitamos entradas para generar predicciones y respuestas reales para medir los errores.

Un error frecuente al comenzar es pensar que `X_test` e `y_test` son datos “sobrantes”. No lo son. Son una parte esencial del proyecto. Aunque el modelo no aprenda directamente de ellos, nos permiten saber si el aprendizaje fue útil. Sin conjunto de prueba, solo podríamos medir qué tan bien el modelo se ajustó a los datos de entrenamiento, pero no tendríamos una estimación razonable de su capacidad de generalización.

También conviene recordar que el conjunto de prueba debe parecerse al tipo de datos que el modelo encontrará en su uso real. Si el conjunto de prueba no representa adecuadamente los casos futuros, la evaluación puede ser limitada. Por ejemplo, si entrenamos y probamos un modelo con datos de una sola región, no deberíamos asumir automáticamente que funcionará igual en otras regiones. Si los datos de prueba corresponden a un período muy estable, quizás no reflejen lo que ocurrirá en un período de cambios fuertes.

En muchos ejemplos iniciales, la división entre entrenamiento y prueba se hace al azar. Esto puede funcionar bien cuando los datos son independientes y no tienen una estructura temporal importante. Pero la idea de “datos no vistos” debe interpretarse según el problema. En algunos casos, los datos de prueba deben representar un período posterior. En otros, deben preservar proporciones de clases. En otros, deben cuidar que casos relacionados no queden repartidos de manera inapropiada entre entrenamiento y prueba.

Por ejemplo, si tenemos varias filas para un mismo usuario, una separación aleatoria podría dejar algunas filas de ese usuario en entrenamiento y otras en prueba. Según el problema, eso podría ser aceptable o podría generar una evaluación demasiado optimista, porque el modelo estaría siendo evaluado sobre usuarios que ya

aparecieron parcialmente durante el entrenamiento. Este tipo de detalles depende del diseño del proyecto y debe revisarse con cuidado.

La separación entre entrenamiento y prueba también influye en las decisiones posteriores. Las transformaciones que aprendan algo de los datos, como ciertos cálculos de preparación, deben ajustarse usando el conjunto de entrenamiento y luego aplicarse al conjunto de prueba. Esta idea se verá con mayor detalle cuando trabajemos preprocesamiento y pipelines, pero conviene anticiparla: el conjunto de prueba debe mantenerse lo más independiente posible del proceso de aprendizaje.

Podemos pensar el conjunto de prueba como una evaluación final dentro de una primera etapa de desarrollo. No debería usarse para tomar todas las decisiones del modelo una y otra vez, porque entonces terminaríamos adaptando nuestras decisiones a ese conjunto. Si probamos muchas versiones y elegimos siempre la que mejor funciona en `X_test` e `y_test`, el conjunto de prueba deja de ser una prueba completamente independiente. Más adelante veremos formas más cuidadosas de organizar estas comparaciones.

Por ahora, alcanza con comprender el rol de cada parte. El conjunto de entrenamiento es el espacio donde el modelo aprende. El conjunto de prueba es el espacio donde observamos si ese aprendizaje se sostiene en casos no vistos. Ambos son necesarios, pero cumplen funciones distintas.

Al separar los datos construimos dos conjuntos principales. `X_train` e `y_train` forman el conjunto de entrenamiento y se usan con `.fit()` para que el modelo aprenda. `X_test` e `y_test` forman el conjunto de prueba y se usan para evaluar predicciones sobre casos no vistos. Esta separación permite pasar de una pregunta limitada, “¿el modelo se ajusta a los datos que vio?”, a una pregunta mucho más importante: “¿el modelo puede generalizar a datos nuevos?”.

Proporciones habituales

Al separar los datos en entrenamiento y prueba, una decisión práctica importante es definir qué proporción de los datos se usará para cada conjunto. Esta proporción influye en dos aspectos del proyecto: cuántos ejemplos tendrá el modelo para aprender y cuántos casos reservaremos para evaluar su desempeño.

No existe una única división correcta para todos los problemas. Sin embargo, en muchos proyectos iniciales con datos tabulares se utilizan separaciones como 80% para entrenamiento y 20% para prueba, o 70% para entrenamiento y 30% para prueba. Estas proporciones son habituales porque buscan un equilibrio razonable:

darle al modelo suficientes datos para aprender y, al mismo tiempo, reservar una cantidad adecuada de casos para evaluar.

En `scikit-learn`, esta decisión suele expresarse mediante el argumento `test_size` de `train_test_split`. Por ejemplo:

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2
)
```

En este caso, `test_size=0.2` indica que el 20% de los datos se reservará para prueba. El 80% restante se usará para entrenamiento. Si escribiéramos `test_size=0.3`, estaríamos reservando el 30% para prueba y usando el 70% para entrenar.

También es posible indicar el tamaño del conjunto de entrenamiento con `train_size`, aunque en la práctica suele ser más común especificar `test_size`. La idea es la misma: estamos decidiendo cómo repartir los datos disponibles entre aprendizaje y evaluación.

La proporción elegida debe interpretarse en relación con el tamaño total del conjunto de datos. Si tenemos 10.000 registros, reservar un 20% para prueba significa contar con 2.000 casos para evaluar, lo cual puede ser suficiente en muchos escenarios. Pero si tenemos solo 100 registros, un 20% deja apenas 20 casos para prueba. Esa evaluación puede ser muy inestable: unos pocos aciertos o errores cambian mucho el resultado.

Por eso, cuando el conjunto de datos es pequeño, la separación entrenamiento-prueba debe pensarse con especial cuidado. Si reservamos demasiados datos para prueba, el modelo tendrá pocos ejemplos para aprender. Si reservamos muy pocos, la evaluación será débil y dependerá demasiado de los casos particulares que hayan quedado en el conjunto de prueba.

Imaginemos un conjunto de datos con 50 observaciones. Una división 80/20 dejaría 40 casos para entrenar y 10 para probar. Si el modelo acierta 8 de esos 10 casos, podríamos decir que logró un 80% de aciertos en la prueba. Pero esa cifra se apoya en muy pocos ejemplos. Si por azar hubieran quedado otros casos en el conjunto de prueba, el resultado podría cambiar bastante. En conjuntos pequeños, una métrica calculada sobre pocos casos debe interpretarse con prudencia.

En cambio, cuando el conjunto de datos es grande, podemos reservar una proporción menor para prueba y aun así obtener una evaluación razonable. Con cientos de miles

de registros, un 10% puede representar una cantidad enorme de casos. En esos escenarios, no siempre es necesario reservar un 30% para prueba, porque el conjunto de evaluación puede ser suficientemente amplio incluso con una proporción más pequeña.

La elección también depende del tipo de problema. En clasificación, especialmente cuando una clase es poco frecuente, necesitamos asegurarnos de que el conjunto de prueba contenga suficientes ejemplos de cada clase. Si la clase que más nos interesa aparece muy pocas veces, una división aparentemente razonable puede dejar muy pocos casos positivos en la prueba. En esos problemas, no basta con mirar la proporción general; debemos revisar la distribución de clases en cada conjunto.

Por ejemplo, si tenemos 1.000 registros y solo 20 corresponden a una clase minoritaria, una división aleatoria podría dejar muy pocos casos de esa clase en el conjunto de prueba, o incluso ninguno en situaciones extremas. Eso haría difícil evaluar si el modelo detecta bien esos casos. Más adelante en este mismo capítulo veremos la estratificación, que ayuda a conservar proporciones similares de clases al separar los datos.

En problemas de regresión, la preocupación puede ser distinta. Tal vez nos interese que el conjunto de prueba incluya valores de distintos rangos: casos pequeños, medianos y grandes; precios bajos y altos; tiempos cortos y largos. Si por azar la prueba queda formada solo por casos muy parecidos, la evaluación no reflejará bien el comportamiento general del modelo.

También debemos considerar el costo de conseguir más datos. En algunos proyectos educativos o experimentales, trabajamos con datasets ya disponibles y la proporción se elige por conveniencia. En proyectos reales, en cambio, puede ser posible recolectar más información, esperar nuevos datos o ampliar el período analizado. Si los datos son escasos, a veces la mejor mejora no está en cambiar la proporción, sino en conseguir más ejemplos o redefinir el alcance del problema.

Una práctica habitual al comenzar es usar `test_size=0.2` como punto de partida. No porque sea una regla universal, sino porque suele ofrecer un equilibrio razonable en muchos ejemplos. A medida que el lector avance, encontrará estrategias más completas para evaluar modelos, como validación cruzada o conjuntos adicionales de validación. Pero para construir una primera comprensión, separar entrenamiento y prueba con una proporción clara es un paso suficiente y muy importante.

Conviene recordar que la proporción no debe elegirse mirando qué resultado da mejor después de muchas pruebas. Si probamos varias divisiones hasta encontrar una que favorece al modelo, podemos terminar obteniendo una evaluación demasiado

optimista. La división debe responder a un criterio razonable antes de evaluar, no a la búsqueda del resultado más conveniente.

Las proporciones más habituales para separar datos suelen ser 80/20 o 70/30 entre entrenamiento y prueba. La elección depende del tamaño del dataset, del tipo de problema, de la distribución de los casos y de la necesidad de contar con una evaluación confiable. El objetivo es equilibrar dos necesidades: darle al modelo suficientes datos para aprender y reservar suficientes casos para estimar cómo se comporta ante datos que no vio durante el entrenamiento.

Aleatoriedad y `random_state`

Cuando usamos `train_test_split`, normalmente la separación entre entrenamiento y prueba se realiza de manera aleatoria. Esto significa que la función mezcla los casos y luego asigna una parte al conjunto de entrenamiento y otra al conjunto de prueba.

La aleatoriedad cumple una función importante. Si los datos estuvieran ordenados de alguna manera, tomar simplemente las primeras filas para entrenar y las últimas para probar podría producir una división poco representativa. Por ejemplo, una tabla podría estar ordenada por fecha, por precio, por ciudad, por tipo de cliente o por valor de la variable objetivo. Si separáramos sin mezclar, podríamos terminar entrenando con un tipo de casos y evaluando con otro muy diferente, no porque esa sea la intención del proyecto, sino por el orden accidental de la tabla.

Imaginemos una tabla de viviendas ordenada de menor a mayor precio. Si usamos las primeras filas para entrenar y las últimas para probar, el modelo aprendería principalmente con viviendas baratas y se evaluaría con viviendas caras. Esa separación no sería una muestra equilibrada del conjunto original. Algo parecido podría ocurrir si una tabla de clientes está ordenada por antigüedad, por región o por estado final.

La mezcla aleatoria ayuda a evitar que el orden original de los datos determine la separación. Al distribuir casos de manera aleatoria, es más probable que tanto el conjunto de entrenamiento como el de prueba contengan una variedad de observaciones parecida a la del dataset completo. No es una garantía perfecta, pero suele ser una buena práctica en muchos problemas tabulares donde las filas pueden considerarse independientes.

En `scikit-learn`, `train_test_split` mezcla los datos de manera aleatoria por defecto. Por eso, si ejecutamos dos veces la misma separación sin controlar esa

aleatoriedad, es posible que obtengamos divisiones distintas. Algunas filas que antes habían quedado en entrenamiento podrían quedar en prueba, y al revés.

Esto puede afectar los resultados. Si la división cambia, el modelo se entrena con casos distintos y se evalúa con casos distintos. Las métricas pueden variar, especialmente cuando el conjunto de datos no es muy grande. El modelo no necesariamente cambió porque cambiamos el algoritmo; cambió porque cambiaron los datos usados para entrenar y probar.

Para controlar esa aleatoriedad usamos el argumento `random_state`.

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

El valor de `random_state` fija una semilla para el proceso aleatorio. En términos prácticos, permite que la división sea reproducible. Si ejecutamos el código varias veces con el mismo `random_state`, obtendremos la misma separación entre entrenamiento y prueba.

Esto es muy útil cuando estamos aprendiendo, escribiendo un cuaderno, comparando modelos o compartiendo resultados con otras personas. Si no fijamos la semilla, cada ejecución puede producir una división distinta y, por lo tanto, métricas distintas. Eso vuelve más difícil analizar si una mejora se debe realmente a un cambio en el modelo o simplemente a que la separación de datos fue diferente.

El número elegido para `random_state` no tiene un significado especial desde el punto de vista del problema. Es común ver ejemplos con `random_state=42`, pero podría usarse otro número entero. Lo importante no es el valor en sí, sino que quede fijo para poder reproducir la división.

Podemos pensarlo así:



El valor de `random_state` fija una semilla para el proceso aleatorio.

Esto no significa que `random_state` haga que el modelo sea mejor. No mejora los datos, no mejora el algoritmo y no garantiza una evaluación más justa por sí solo. Su función principal es controlar la aleatoriedad para que el experimento sea reproducible.

La reproducibilidad es importante en machine learning. Si entrenamos un modelo y obtenemos cierto resultado, necesitamos poder volver a ejecutar el proceso y obtener lo mismo, o al menos saber qué parte del proceso puede variar. Esto permite revisar errores, comparar alternativas, documentar decisiones y compartir el trabajo de manera más clara.

Supongamos que entrenamos un modelo con una división aleatoria y obtenemos una métrica determinada. Luego probamos otro modelo, pero al ejecutar nuevamente `train_test_split` se genera una división distinta. Si el segundo modelo obtiene una métrica mejor, no sabremos con claridad si mejoró porque el modelo es superior o porque tuvo una división más favorable. Fijar `random_state` ayuda a que ambos modelos se comparen sobre la misma separación de datos.

También es importante recordar que una única división, aunque sea reproducible, sigue siendo una sola forma de partir los datos. En algunos casos, el resultado puede depender bastante de qué casos quedaron en entrenamiento y cuáles en prueba. Más adelante veremos estrategias más robustas para evaluar modelos, pero en esta etapa inicial `train_test_split` con `random_state` nos ofrece una forma clara y ordenada de comenzar.

Hay situaciones en las que la mezcla aleatoria no es adecuada. Si los datos tienen una estructura temporal y queremos predecir el futuro, mezclar aleatoriamente puede generar problemas, porque podríamos entrenar con datos posteriores y probar

con datos anteriores. En esos casos, la separación debe respetar el tiempo. Por ejemplo, podríamos entrenar con los primeros meses y probar con meses posteriores. La aleatoriedad debe usarse cuando tiene sentido para el problema, no como una regla automática.

En otros contextos, la aleatoriedad puede requerir cuidados adicionales. Si hay varias filas relacionadas con una misma persona, producto, paciente o dispositivo, mezclar filas al azar puede dejar información del mismo individuo en entrenamiento y prueba. Según el problema, esto podría producir una evaluación demasiado optimista. En esos casos, la separación debe considerar grupos, no solo filas individuales. No profundizaremos todavía en estos escenarios, pero conviene saber que existen.

`train_test_split` suele separar los datos de manera aleatoria para evitar que el orden original de la tabla condicione los conjuntos de entrenamiento y prueba. El argumento `random_state` permite fijar esa aleatoriedad y obtener siempre la misma división al ejecutar el código. Esto no mejora el modelo por sí mismo, pero hace que el trabajo sea reproducible y facilita la comparación entre pruebas. Como siempre, la decisión de mezclar aleatoriamente debe tener sentido para el tipo de datos y el problema que queremos resolver.

Estratificación en clasificación

En problemas de clasificación, además de separar los datos en entrenamiento y prueba, debemos prestar atención a la distribución de las clases. Esto es especialmente importante cuando una clase aparece mucho más que otra.

Supongamos que tenemos un conjunto de datos para predecir si un cliente abandonó o no un servicio. Si el 80% de los clientes permaneció y el 20% abandonó, esa proporción describe la distribución general de la variable objetivo. Al separar los datos, sería conveniente que tanto el conjunto de entrenamiento como el conjunto de prueba mantengan una proporción parecida.

Si no cuidamos este detalle, una división aleatoria podría producir conjuntos desequilibrados. Tal vez el conjunto de entrenamiento quede con una proporción razonable de clientes que abandonaron, pero el conjunto de prueba quede con muy pocos. O al revés. Esto puede afectar tanto el entrenamiento como la evaluación.

El problema se vuelve más evidente cuando una clase es poco frecuente. Imaginemos un dataset donde solo el 5% de los casos corresponde a la clase que más nos interesa detectar. Si hacemos una separación aleatoria sin ningún control, puede ocurrir que en el conjunto de prueba queden muy pocos ejemplos de esa clase. Entonces,

cualquier métrica calculada sobre ese conjunto será poco estable. Unos pocos aciertos o errores pueden cambiar mucho la interpretación del modelo.

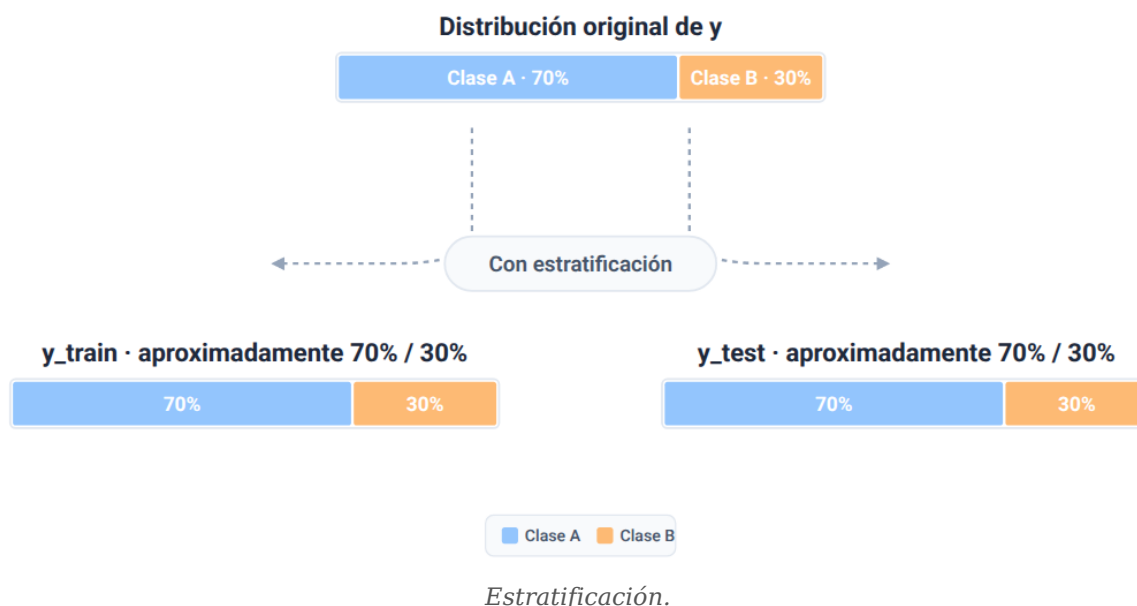
La **estratificación** busca evitar este problema. Al usar estratificación, le pedimos a la división que conserve, en entrenamiento y prueba, proporciones similares de las clases presentes en la variable objetivo.

En `scikit-learn`, esto se puede hacer con el argumento `stratify` de `train_test_split`:

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

El argumento `stratify=y` indica que la separación debe respetar la distribución de clases de la variable objetivo. Si en `y` el 70% de los casos pertenece a una clase y el 30% a otra, la función intentará mantener una proporción similar en `y_train` y en `y_test`.

Podemos pensarlo así:



Esto no significa que las proporciones serán siempre exactamente idénticas, especialmente si el dataset es pequeño o si algunas clases tienen muy pocos casos. Pero la estratificación ayuda a que la división sea más representativa que una separación aleatoria sin control.

Veamos un ejemplo conceptual. Supongamos que tenemos 1.000 registros y una variable objetivo con dos clases: 900 casos negativos y 100 casos positivos. La distribución original es 90% negativos y 10% positivos. Si usamos una división 80/20 con estratificación, esperaríamos algo parecido a esto:



Ejemplo del uso de la estratificación.

El conjunto de entrenamiento conserva ejemplos de ambas clases en proporción similar. El conjunto de prueba también. Esto permite entrenar el modelo con una muestra más representativa y evaluar su desempeño sobre una prueba que refleja mejor la distribución original.

La estratificación es especialmente recomendable en clasificación binaria con clases desbalanceadas. Por ejemplo, cuando queremos detectar abandono, fraude, enfermedad, fallas, incumplimiento, reclamos críticos o cualquier evento poco frecuente. En esos casos, la clase minoritaria suele ser muy importante, aunque aparezca poco.

Sin estratificación, podríamos terminar con una prueba que no permite evaluar adecuadamente esa clase. Si estamos construyendo un modelo para detectar fraudes y en el conjunto de prueba quedan apenas dos fraudes, cualquier conclusión será débil. Si el modelo detecta uno, parecerá tener cierto desempeño; si no detecta ninguno, parecerá pésimo. Pero el problema principal es que la muestra de evaluación es demasiado pequeña para esa clase.

También puede afectar el entrenamiento. Si por azar quedan muy pocos ejemplos de la clase minoritaria en `y_train`, el modelo tendrá menos oportunidades de aprender

sus patrones. En problemas donde la clase de interés ya es escasa, perder ejemplos durante la división puede ser especialmente perjudicial.

Esto no significa que la estratificación resuelva todos los problemas de clases desbalanceadas. Mantener proporciones similares ayuda, pero no cambia el hecho de que una clase puede tener pocos ejemplos. Más adelante estudiaremos con mayor detalle cómo evaluar modelos en situaciones de desbalance y qué estrategias pueden utilizarse. Por ahora, lo importante es entender que la separación de datos debe cuidar la distribución de la variable objetivo.

La estratificación también puede usarse en problemas multiclase. Si `y` contiene varias categorías, `train_test_split` intentará conservar la proporción de cada una en entrenamiento y prueba. Esto puede ser útil cuando algunas categorías aparecen mucho más que otras.

Por ejemplo, si queremos clasificar reclamos en distintas áreas y algunas áreas concentran la mayoría de los casos, una división sin estratificar podría dejar mal representadas las categorías menos frecuentes. Con estratificación, reducimos ese riesgo y obtenemos conjuntos más equilibrados respecto de la distribución original.

Sin embargo, hay que tener cuidado cuando algunas clases tienen muy pocos ejemplos. Si una categoría aparece solo una o dos veces, puede ser imposible repartirla adecuadamente entre entrenamiento y prueba. En esos casos, el problema no se resuelve solo con una opción de `train_test_split`; quizás haya que revisar la calidad del dataset, agrupar categorías, conseguir más datos o reformular la tarea.

También es importante aclarar que la estratificación se aplica sobre la variable objetivo en problemas de clasificación. En regresión, donde `y` es un valor numérico continuo, no se usa de la misma manera en una división básica. Allí pueden existir otras estrategias si queremos preservar rangos de valores, pero no corresponde hablar de clases en el mismo sentido.

En un cuaderno Colab, una forma simple de comprobar si la estratificación funcionó es comparar la distribución de `y`, `y_train` y `y_test`. Por ejemplo, podríamos revisar proporciones con `value_counts(normalize=True)`:

```
y.value_counts(normalize=True)
```

```
y_train.value_counts(normalize=True)
```

```
y_test.value_counts(normalize=True)
```

La idea no es solo ejecutar código, sino verificar que la separación tenga sentido. Si las proporciones de clases cambian demasiado entre entrenamiento y prueba, la evaluación puede ser menos confiable.

La estratificación es una técnica que ayuda a mantener proporciones similares de clases al separar datos de clasificación en entrenamiento y prueba. Se usa en `train_test_split` mediante el argumento `stratify=y`. Es especialmente importante cuando las clases están desbalanceadas o cuando una categoría minoritaria tiene mucha importancia para el problema. Separar datos no es solo dividir filas: también implica cuidar que los conjuntos resultantes representen adecuadamente la variable que queremos predecir.

Separar para evaluar mejor

La separación entre entrenamiento y prueba es una de las primeras prácticas metodológicas importantes en machine learning. No se trata simplemente de dividir filas en dos partes, sino de construir una forma más honesta de evaluar si el modelo puede generalizar.

El conjunto de entrenamiento permite que el modelo aprenda a partir de ejemplos conocidos. Allí observa las variables de entrada y, en problemas supervisados, las compara con la variable objetivo. El conjunto de prueba, en cambio, debe permanecer separado del entrenamiento. Su función es representar casos no vistos, para que podamos estimar cómo se comporta el modelo cuando recibe información nueva.

Esta separación nos ayuda a evitar una evaluación engañosa. Si entrenamos y evaluamos con los mismos datos, podemos creer que el modelo funciona muy bien, cuando en realidad tal vez solo se ajustó demasiado a los ejemplos que ya conocía. Evaluar sobre datos reservados no garantiza que el modelo funcionará perfectamente en el mundo real, pero ofrece una señal mucho más razonable que medirlo únicamente sobre los datos de entrenamiento.

En `scikit-learn`, la función `train_test_split` facilita esta tarea. A partir de `X` e `y`, genera cuatro partes: `X_train`, `X_test`, `y_train` e `y_test`. Con `X_train` e `y_train` entrenamos el modelo. Con `X_test` generamos predicciones. Luego comparamos esas predicciones con `y_test`, que contiene las respuestas reales reservadas para la evaluación.

También vimos que la proporción entre entrenamiento y prueba debe elegirse con criterio. Divisiones como 80/20 o 70/30 son habituales, pero no son reglas universales. La elección depende del tamaño del dataset, del tipo de problema y de la necesidad de contar con suficientes casos tanto para aprender como para evaluar.

La aleatoriedad ayuda a que la separación no dependa del orden original de la tabla. Sin embargo, esa aleatoriedad debe poder controlarse para que el trabajo sea reproducible. Por eso usamos `random_state`: no mejora el modelo, pero permite repetir la misma división y comparar resultados con mayor claridad.

En problemas de clasificación, además, es importante cuidar la distribución de las clases. Cuando una clase aparece mucho más que otra, una separación aleatoria puede dejar conjuntos poco representativos. La estratificación permite conservar proporciones similares de clases en entrenamiento y prueba, lo que vuelve más confiable tanto el aprendizaje como la evaluación.

Este capítulo deja una idea que será clave en todo el recorrido: evaluar bien requiere separar correctamente. Antes de confiar en un modelo, debemos observar cómo responde ante datos que no usó para aprender. Esa distancia entre entrenamiento y prueba es lo que nos permite empezar a hablar de generalización, no solo de ajuste a los datos conocidos.

Capítulo 6 · Limpieza y transformación de datos

Valores faltantes

Al preparar datos para machine learning, uno de los primeros problemas que suelen aparecer son los **valores faltantes**. Un valor faltante aparece cuando una observación no tiene información registrada en una variable determinada. En una tabla, esto puede verse como una celda vacía, como NaN, como None, como un guion, como la palabra “desconocido” o incluso como algún código especial que se usó para indicar ausencia de información.

En los capítulos anteriores insistimos en que un modelo aprende a partir de la información que le damos. Si parte de esa información falta, debemos decidir qué hacer. No podemos simplemente ignorar el problema y esperar que el modelo lo resuelva por sí solo. Muchos algoritmos no aceptan valores faltantes directamente, y aunque algunos métodos puedan tolerarlos mejor que otros, siempre conviene entender por qué faltan y qué consecuencias puede tener tratarlos de una u otra manera.

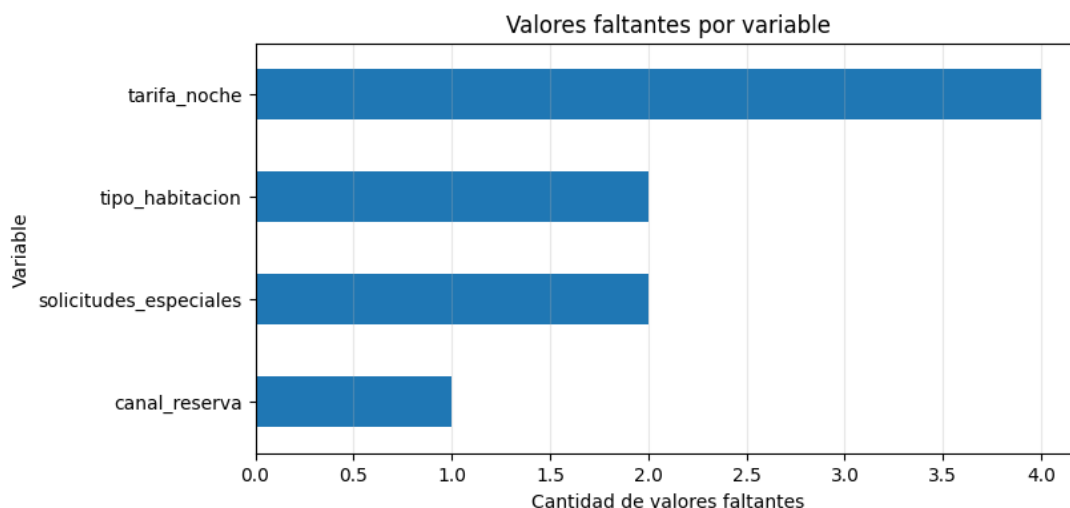
Un valor faltante no es solo un inconveniente técnico. También puede ser una señal sobre el proceso que generó los datos. A veces falta información por un error de carga. Otras veces porque una pregunta no fue respondida, porque un sensor no registró una medición, porque un sistema cambió su forma de guardar datos, porque cierta variable no aplica a todos los casos o porque la información todavía no estaba disponible al momento del registro.

Estas situaciones no son equivalentes. No es lo mismo que falte la edad de una persona porque alguien olvidó completar un formulario, que falte el valor de cochera en una vivienda porque esa vivienda no tiene cochera, o que falte una medición de temperatura porque un sensor dejó de funcionar durante varias horas. En cada caso, el significado del faltante puede ser distinto.

Por eso, antes de reemplazar o eliminar valores faltantes, conviene preguntarse qué representan. ¿Son errores? ¿Son ausencias legítimas? ¿Indican que una característica no aplica? ¿Aparecen de manera aleatoria o se concentran en ciertos grupos? ¿Faltan en pocas filas o en una gran parte de la columna? ¿La ausencia de ese dato podría estar relacionada con la variable objetivo?

Esta última pregunta es especialmente importante. A veces, que un dato falte puede contener información. Por ejemplo, en un sistema de créditos, la ausencia de cierto antecedente financiero puede tener un significado diferente según el contexto. En

una plataforma digital, que no exista registro de una actividad puede indicar que el usuario nunca la realizó. En datos médicos, la ausencia de un estudio puede deberse a que no fue necesario, a que no se solicitó o a que no se pudo realizar. El faltante no siempre es ruido; a veces forma parte del fenómeno.



El gráfico permite comparar rápidamente la magnitud del problema. En este ejemplo los faltantes son pocos, pero igualmente deben tratarse antes de usar muchos modelos de Machine Learning..

En términos prácticos, hay varias estrategias para tratar valores faltantes. Una posibilidad es eliminar filas que contienen faltantes. Esto puede ser razonable si son muy pocos casos y si eliminarlos no altera demasiado la representatividad del dataset. Sin embargo, puede ser una mala decisión si los faltantes son muchos o si se concentran en un grupo particular. En ese caso, eliminar filas puede sesgar los datos y dejar fuera justamente casos importantes.

Otra posibilidad es eliminar columnas con demasiados valores faltantes. Si una variable tiene información ausente en la mayor parte de las filas, quizá no sea suficientemente confiable para entrenar un modelo. Pero tampoco conviene aplicar una regla automática sin pensar. Una columna con muchos faltantes podría ser irrelevante, o podría contener una señal muy valiosa en los casos donde sí está presente. La decisión debe combinar análisis de cantidad, significado y utilidad para el problema.

Una estrategia muy habitual es **imputar** valores faltantes. Imputar significa reemplazar los valores ausentes por algún valor definido. En variables numéricas, una opción común es reemplazar por la media o la mediana de la columna. En variables categóricas, puede reemplazarse por la categoría más frecuente o por una categoría explícita como “desconocido”. Estas decisiones no son neutras: cada una introduce una suposición sobre los datos.

Por ejemplo, si en una variable numérica reemplazamos los faltantes por la media, estamos colocando esos casos en un valor promedio. Esto puede funcionar en algunas situaciones, pero también puede reducir artificialmente la variabilidad. Si usamos la mediana, la imputación puede ser más robusta frente a valores extremos. Si en una variable categórica usamos la categoría más frecuente, estamos suponiendo que los casos faltantes se parecen a la mayoría. Si creamos una categoría “desconocido”, estamos permitiendo que el modelo trate la ausencia como una categoría propia.

No existe una estrategia universalmente correcta. La mejor decisión depende del problema, del tipo de variable, de la cantidad de faltantes, del motivo por el que faltan y del modelo que vamos a utilizar. En un capítulo introductorio, lo importante es comprender que los valores faltantes deben tratarse de manera consciente, no automática.

También debemos tener en cuenta una cuestión metodológica fundamental: cuando se imputan valores faltantes, los valores usados para imputar deben aprenderse a partir del conjunto de entrenamiento, no del dataset completo. Esta idea se conecta directamente con la fuga de datos.

Supongamos que separamos los datos en entrenamiento y prueba, y luego queremos reemplazar los valores faltantes de una variable numérica por su mediana. Si calculamos la mediana usando todos los datos, incluyendo el conjunto de prueba, estamos utilizando información de la prueba para preparar el entrenamiento. Puede parecer un detalle menor, pero rompe la separación que queríamos preservar. El conjunto de prueba debe simular datos no vistos, y por eso no debería influir en las decisiones aprendidas durante el preprocesamiento.

La forma correcta es calcular la mediana en `X_train` y luego usar ese mismo valor para transformar tanto `X_train` como `X_test`. De esta manera, el conjunto de prueba recibe el mismo tratamiento que recibirían datos nuevos en una situación real: se transforman con reglas aprendidas previamente, no con información extraída de ellos mismos.

En `scikit-learn`, este tipo de tratamiento suele organizarse con herramientas como `SimpleImputer`, que aprende el valor de reemplazo durante el ajuste y luego lo aplica durante la transformación. Más adelante en este capítulo presentaremos la idea de `Pipeline`, que permite ordenar estas etapas para evitar errores. Por ahora, lo importante es la lógica: algunas transformaciones no son simples cambios mecánicos; necesitan aprender valores a partir de los datos, y esos valores deben aprenderse solo del entrenamiento.

Los valores faltantes también pueden afectar la interpretación del modelo. Si una variable tiene muchos faltantes y los imputamos sin analizar, el modelo puede

aprender patrones difíciles de entender. Por ejemplo, si reemplazamos muchos valores por una misma mediana, podríamos crear una concentración artificial en torno a ese valor. Si creamos una categoría “desconocido”, el modelo podría aprender que esa categoría tiene relación con el objetivo. Eso puede ser útil si la ausencia realmente contiene información, pero peligroso si refleja un error de registro.

Por eso, una buena práctica conceptual es documentar cómo se trataron los faltantes. Conviene registrar qué columnas tenían valores ausentes, en qué proporción, qué estrategia se usó y por qué. Esta documentación ayuda a interpretar los resultados y a revisar el proyecto más adelante.

Los valores faltantes son una parte normal del trabajo con datos reales. Pueden aparecer por errores, ausencias legítimas, cambios en los sistemas, mediciones incompletas o condiciones propias del problema. Tratarlos requiere más que una instrucción de código: exige comprender qué significan, decidir si conviene eliminar, imputar o representar la ausencia, y cuidar que cualquier valor aprendido para completar datos se calcule solo con el conjunto de entrenamiento. Preparar bien los valores faltantes es uno de los primeros pasos para construir un modelo confiable.

Variables categóricas

Las variables categóricas aparecen con mucha frecuencia en los datos reales. Son aquellas variables cuyos valores representan clases, grupos, tipos o etiquetas, en lugar de cantidades numéricas. Una columna como `ciudad`, `barrio`, `tipo_producto`, `metodo_pago`, `estado_civil`, `nivel_educativo`, `categoria_cliente` o `tipo_reclamo` contiene información categórica.

En un `DataFrame`, estas variables suelen verse como texto. Por ejemplo, una columna `metodo_pago` podría contener valores como “efectivo”, “tarjeta” y “transferencia”. Una columna `tipo_cliente` podría contener “nuevo”, “frecuente” y “premium”. Una columna `barrio` podría contener nombres de zonas o localidades.

Desde el punto de vista conceptual, estas variables pueden ser muy valiosas. Muchas veces describen aspectos importantes del caso que queremos modelar. El método de pago puede estar relacionado con el comportamiento de compra. El tipo de producto puede influir en la demanda. El barrio puede aportar información para estimar precios de viviendas. El tipo de reclamo puede ayudar a predecir tiempos de resolución.

Sin embargo, desde el punto de vista técnico, las variables categóricas requieren preparación. Muchos modelos de machine learning no pueden trabajar directamente

con texto. Un algoritmo no sabe qué hacer, en términos matemáticos, con valores como “efectivo”, “tarjeta” o “transferencia”. Para que el modelo pueda utilizar esa información, debemos convertirla a una representación numérica adecuada.

El punto delicado es que convertir categorías en números no debe cambiar su significado. No queremos introducir relaciones artificiales que no existen en los datos. Si asignamos números de manera arbitraria, podemos darle al modelo una idea equivocada.

Por ejemplo, supongamos que tenemos una variable `metodo_pago` con tres categorías:

```
efectivo
tarjeta
transferencia
```

Una codificación ingenua podría ser:

```
efectivo      → 1
tarjeta       → 2
transferencia → 3
```

Esta transformación parece simple, pero puede ser problemática. Al convertir las categorías en 1, 2 y 3, estamos usando números que tienen orden y distancia. Un modelo podría interpretar que “transferencia” es mayor que “tarjeta”, que “tarjeta” es mayor que “efectivo”, o que la distancia entre efectivo y tarjeta es la misma que entre tarjeta y transferencia. En realidad, nada de eso forma parte del significado original de la variable. Son métodos de pago distintos, pero no cantidades ordenadas.

Este problema no aparece en todas las variables categóricas por igual. Algunas categorías sí tienen un orden natural. Por ejemplo, una variable `nivel_satisfaccion` con valores “bajo”, “medio” y “alto” tiene una estructura ordinal. Lo mismo puede ocurrir con “pequeño”, “mediano” y “grande”, o con niveles educativos ordenados. En esos casos, el orden puede tener sentido, aunque igual debemos ser cuidadosos con la distancia entre categorías.

Pero muchas variables categóricas no tienen orden. Ciudad, color, método de pago, tipo de producto o canal de venta son ejemplos de variables nominales. Sus valores identifican grupos, pero no establecen una jerarquía numérica. Para este tipo de variables, necesitamos una transformación que permita al modelo usar la información sin inventar un orden inexistente.

Antes de transformar variables categóricas, también conviene revisarlas. Los datos reales suelen tener inconsistencias. Una misma categoría puede aparecer escrita de varias formas: “Tarjeta”, “tarjeta”, “TARJETA”, “tarjeta de crédito” o “credito”. Para

una persona, esos valores pueden parecer equivalentes. Para un modelo, son categorías distintas si no las normalizamos antes.

También puede haber espacios extra, errores de tipeo, abreviaturas, valores faltantes o categorías muy poco frecuentes. Una columna de ciudad puede contener “Buenos Aires”, “Bs As”, “CABA” y “Ciudad Autónoma de Buenos Aires”, según cómo se haya registrado la información. Una columna de producto puede tener cientos o miles de categorías, algunas con muy pocos casos. Si no revisamos estos detalles, la transformación posterior puede producir una representación más confusa que útil.

Las variables categóricas con muchas categorías distintas pueden traer desafíos adicionales. Si una columna tiene tres o cuatro categorías, suele ser manejable. Pero si tiene miles de valores posibles, como un código de producto, un identificador de usuario o una dirección exacta, la preparación se vuelve más compleja. No siempre conviene usar esas columnas tal como están. A veces es mejor agrupar categorías, trabajar con niveles más generales o descartar variables que no aportan una señal generalizable.

También debemos distinguir entre una variable categórica real y un identificador. Un identificador puede estar escrito como texto o como número, pero su función principal es distinguir registros individuales. Un `id_cliente`, un `numero_factura` o un `codigo_operacion` no suelen ser buenas variables predictoras, porque no describen una característica general del caso. Incluirlos puede favorecer la memorización de ejemplos, no el aprendizaje de patrones útiles.

En machine learning, la preparación de variables categóricas busca convertir información cualitativa en una representación numérica que conserve el significado relevante para el problema. Esta preparación puede ser muy simple en algunos casos y más delicada en otros. Depende de cuántas categorías hay, si tienen orden, si están bien registradas, si hay valores faltantes y si la variable tiene sentido para la predicción.

Una de las técnicas más usadas para variables categóricas nominales es el One-Hot Encoding, que veremos en la próxima sección. Su idea básica es crear columnas indicadoras para representar la presencia o ausencia de cada categoría, evitando asignar un orden artificial entre ellas. Antes de llegar a esa técnica, conviene retener la idea central: las categorías no deben tratarse como números comunes si esos números no representan cantidades reales.

Las variables categóricas representan grupos, tipos o etiquetas, y pueden aportar información muy valiosa para un modelo. Pero necesitan preparación antes de ser utilizadas por muchos algoritmos. Esa preparación debe respetar el significado de la variable, corregir inconsistencias cuando sea necesario y evitar codificaciones que introduzcan relaciones artificiales. Trabajar bien con variables categóricas es una parte esencial del preprocesamiento de datos.

One-Hot Encoding

Una de las técnicas más utilizadas para preparar variables categóricas nominales es el **One-Hot Encoding**. Su objetivo es convertir una variable categórica en una representación numérica que pueda ser utilizada por un modelo, sin introducir un orden artificial entre las categorías.

La idea es simple: en lugar de reemplazar cada categoría por un número, creamos una nueva columna para cada categoría posible. Luego, para cada fila, indicamos con un 1 si esa categoría está presente y con un 0 si no lo está.

Supongamos que tenemos una variable llamada `metodo_pago` con tres categorías posibles:

```
efectivo
tarjeta
transferencia
```

En lugar de codificarla como 1, 2 y 3, podemos crear tres columnas nuevas:

```
metodo_pago_efectivo
metodo_pago_tarjeta
metodo_pago_transferencia
```

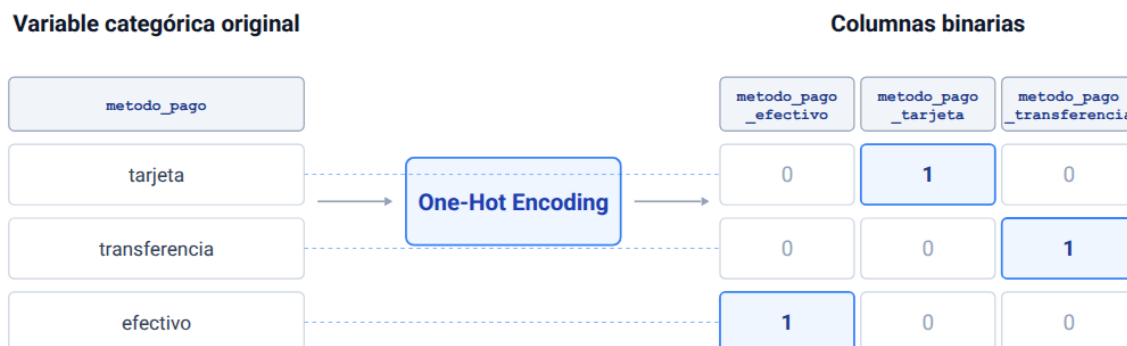
Si una fila corresponde a un pago con tarjeta, la representación quedaría así:

```
metodo_pago_efectivo    0
metodo_pago_tarjeta     1
metodo_pago_transferencia 0
```

Si otra fila corresponde a una transferencia, quedaría así:

```
metodo_pago_efectivo    0
metodo_pago_tarjeta     0
metodo_pago_transferencia 1
```

De esta manera, cada categoría se representa mediante una columna indicadora. El modelo ya no recibe palabras como “tarjeta” o “efectivo”, pero tampoco recibe números que sugieran un orden inexistente. Recibe señales binarias que indican qué categoría aparece en cada caso.



Una columna por categoría, sin introducir un orden artificial.

Esta forma de representación es especialmente útil para variables categóricas nominales, es decir, variables cuyas categorías no tienen un orden natural. Método de pago, ciudad, tipo de producto, barrio, canal de venta o tipo de reclamo son ejemplos habituales. En esos casos, One-Hot Encoding permite transformar las categorías en números sin decirle al modelo que una categoría es mayor, menor o más cercana que otra.

La información original se conserva, pero ahora está expresada en una forma numérica. Cada fila sigue representando el mismo caso. Lo que cambió es la forma en que describimos la variable categórica.

En `scikit-learn`, esta transformación suele realizarse con `OneHotEncoder`. No necesitamos profundizar todavía en todos sus parámetros, pero sí conviene reconocer su lugar dentro del flujo de trabajo. `OneHotEncoder` aprende qué categorías existen en los datos de entrenamiento y luego transforma la columna categórica en columnas numéricas indicadoras.

Una idea importante es que el encoder debe aprender las categorías a partir del conjunto de entrenamiento. Esto significa que no deberíamos mirar todo el dataset completo, incluyendo prueba, para decidir qué columnas crear. Si usamos el conjunto de prueba para aprender categorías, estaríamos dejando que información reservada influya en el preprocesamiento. En capítulos anteriores ya vimos que el conjunto de prueba debe mantenerse separado para simular datos no vistos.

Por eso, en un flujo correcto, el One-Hot Encoding se ajusta con `X_train` y luego se aplica tanto a `X_train` como a `X_test`. En términos generales, la lógica sería:

```
encoder.fit(X_train_categoricas)
X_train_codificado = encoder.transform(X_train_categoricas)
X_test_codificado = encoder.transform(X_test_categoricas)
```

La función `.fit()` aprende las categorías presentes en el entrenamiento. La función `.transform()` aplica esa transformación. Esta separación entre aprender una transformación y aplicarla será muy importante durante todo el preprocesamiento.

También puede aparecer una situación frecuente: una categoría que no estaba en el entrenamiento aparece luego en los datos de prueba o en datos nuevos. Por ejemplo, imaginemos que durante el entrenamiento la variable `metodo_pago` tenía las categorías “efectivo”, “tarjeta” y “transferencia”, pero más adelante aparece una nueva categoría llamada “billetera virtual”. El modelo no vio esa categoría al entrenar, y el encoder no había creado una columna para ella.

Para manejar estos casos, `OneHotEncoder` permite configurar qué hacer con categorías desconocidas. Una opción habitual es indicarle que ignore categorías no vistas, en lugar de producir un error. En código, esto suele aparecer como:

```
OneHotEncoder(handle_unknown="ignore")
```

La idea conceptual es que, si aparece una categoría nueva, el transformador no rompe el flujo de predicción. Simplemente no activa ninguna de las columnas conocidas para esa variable. Esto no significa que el modelo entienda la nueva categoría, pero permite que el sistema siga funcionando. En proyectos reales, la aparición de categorías nuevas puede ser una señal de que el modelo o el preprocesamiento necesitan revisarse.

One-Hot Encoding tiene muchas ventajas, pero también tiene límites. Uno de ellos aparece cuando una variable tiene demasiadas categorías. Si una columna tiene tres categorías, se crean tres columnas nuevas. Si tiene veinte, se crean veinte. Pero si tiene miles de categorías, el número de columnas puede crecer mucho. Esto puede hacer que el conjunto de datos sea más grande, más disperso y más difícil de manejar.

Por ejemplo, una variable `producto_id` con diez mil productos distintos produciría una cantidad enorme de columnas si se codifica directamente con One-Hot Encoding. Además, muchas de esas categorías podrían aparecer en muy pocos casos, lo que dificulta que el modelo aprenda patrones confiables. En situaciones así, puede ser necesario agrupar categorías poco frecuentes, usar una representación más general o reconsiderar si esa variable es adecuada para el modelo.

Otro punto importante es que One-Hot Encoding transforma una sola columna categórica en varias columnas numéricas. Esto cambia la forma de X . Si antes teníamos diez columnas originales, después del preprocesamiento podríamos tener muchas más. Esto no es un problema en sí mismo, pero debemos recordarlo cuando interpretemos la estructura de los datos preparados.

También debemos tener cuidado con los valores faltantes. Si una variable categórica tiene valores ausentes, debemos decidir cómo tratarlos antes o durante la codificación. En algunos casos, podemos imputar la categoría más frecuente. En otros, podemos crear una categoría explícita como “desconocido”. La decisión depende del significado del faltante y del problema. Lo importante es que la estrategia sea consistente entre entrenamiento y prueba.

Existe además una cuestión técnica que aparece en algunos modelos: cuando una variable categórica se transforma en varias columnas, puede generarse redundancia entre ellas. Por ejemplo, si sabemos que `metodo_pago_efectivo` y `metodo_pago_tarjeta` valen 0, entonces podemos deducir que `metodo_pago_transferencia` vale 1, siempre que esas sean las únicas categorías posibles. Algunos enfoques eliminan una de las columnas para evitar esta redundancia. Sin embargo, en muchos flujos iniciales con `scikit-learn`, conservar todas las columnas es una opción clara y fácil de entender. Más adelante, si el modelo lo requiere, podremos profundizar en estos detalles.

Lo esencial es comprender por qué usamos One-Hot Encoding. No lo hacemos simplemente porque los modelos necesitan números. Lo hacemos porque necesitamos una representación numérica que respete el significado de las categorías. Reemplazar categorías nominales por números arbitrarios puede introducir relaciones falsas. One-Hot Encoding evita ese problema creando columnas separadas para cada categoría.

One-Hot Encoding es una técnica para transformar variables categóricas nominales en variables numéricas binarias. Cada categoría se convierte en una columna, y cada fila indica con 1 o 0 si esa categoría está presente. Es una herramienta fundamental en el preprocesamiento de datos para machine learning, especialmente cuando trabajamos con modelos que no aceptan texto directamente. Su uso debe integrarse correctamente al flujo de entrenamiento y prueba: las categorías se aprenden con los datos de entrenamiento y la misma transformación se aplica luego a los datos de prueba o a datos nuevos.

Escalado de variables numéricas

Después de tratar valores faltantes y preparar variables categóricas, otro paso importante del preprocesamiento es revisar las variables numéricas. Muchas veces, esas variables están medidas en escalas muy diferentes. Una columna puede representar edad en años, otra ingresos en pesos, otra distancia en kilómetros, otra cantidad de compras y otra tiempo en minutos. Aunque todas sean numéricas, sus valores pueden tener magnitudes muy distintas.

Por ejemplo, imaginemos un conjunto de datos de clientes con estas variables:

edad	ingresos_mensuales	compras_mes	dias_sin_actividad
35	450000	4	12
52	900000	1	45
28	320000	8	3

Todas las columnas contienen números, pero no están en la misma escala. La edad se mueve quizás entre 18 y 90. Los ingresos pueden moverse entre decenas de miles y millones. La cantidad de compras puede estar entre 0 y 20. Los días sin actividad pueden tener otro rango distinto. Si un modelo compara numéricamente estas variables, las columnas con valores más grandes pueden influir demasiado simplemente por su magnitud, no porque sean más importantes para el problema.

El **escalado** busca transformar variables numéricas para que queden en rangos o distribuciones más comparables. No cambia la información esencial de la variable, pero sí cambia la forma numérica en que se presenta al modelo.

Una técnica muy común es la **estandarización**, que transforma una variable para que tenga media 0 y desviación estándar 1. En términos intuitivos, cada valor deja de expresarse en su unidad original y pasa a expresarse en relación con la distribución de esa columna. Un valor positivo indica que está por encima del promedio; un valor negativo indica que está por debajo; un valor cercano a 0 está cerca de la media.

En `scikit-learn`, una herramienta habitual para esta transformación es `StandardScaler`. Su lógica general es aprender la media y la desviación estándar de cada variable numérica durante el entrenamiento, y luego usar esos valores para transformar los datos.

Conceptualmente, ocurre algo así:

```
scaler.fit(X_train_numericas)
X_train_escalado = scaler.transform(X_train_numericas)
X_test_escalado = scaler.transform(X_test_numericas)
```

La primera línea, `.fit()`, aprende los valores necesarios para escalar: la media y la desviación estándar de cada columna numérica del conjunto de entrenamiento. Luego, `.transform()` aplica la transformación. Es importante notar que el conjunto de prueba no se usa para aprender esos valores. Se transforma usando los valores aprendidos del entrenamiento.

Esta regla es fundamental. Si calculamos la media y la desviación estándar usando todo el dataset, incluyendo los datos de prueba, estaríamos dejando que el conjunto de prueba influya en el preprocesamiento. Aunque parezca un detalle pequeño, sería una forma de fuga de datos. El conjunto de prueba debe comportarse como datos nuevos: no participa en el aprendizaje de transformaciones, solo recibe transformaciones aprendidas previamente.

El escalado no siempre es obligatorio para todos los modelos. Algunos modelos son muy sensibles a la escala de las variables, especialmente aquellos que se basan en distancias, márgenes, gradientes o combinaciones numéricas donde la magnitud de las columnas tiene mucho peso. Otros modelos, como ciertos métodos basados en árboles de decisión, suelen verse menos afectados por las diferencias de escala. Más adelante, cuando estudiemos modelos concretos, retomaremos esta idea con ejemplos.

Por ahora, lo importante es comprender que el escalado no es una cuestión estética. Puede afectar la forma en que algunos modelos aprenden. Si una variable tiene valores mucho más grandes que otra, un modelo sensible a la escala puede darle más importancia simplemente porque sus números son mayores. El escalado ayuda a que las variables numéricas participen en condiciones más comparables.

Además de `StandardScaler`, existen otras formas de escalar. Una de ellas es llevar los valores a un rango determinado, por ejemplo entre 0 y 1. Esa transformación suele asociarse con herramientas como `MinMaxScaler`. En lugar de centrar los datos alrededor de la media, toma el valor mínimo y máximo de la variable y reubica los valores dentro de un intervalo. Esta estrategia puede ser útil en ciertos contextos, aunque también puede ser sensible a valores extremos.

La elección del tipo de escalado depende del modelo, de los datos y del problema. No hay una transformación universalmente mejor. La estandarización suele ser una opción muy común como punto de partida, pero no reemplaza la necesidad de explorar las variables. Si una columna tiene valores extremos muy marcados, una distribución muy asimétrica o errores de carga, escalarla no resuelve esos problemas. Simplemente transforma la escala de una variable que tal vez necesita revisión previa.

Por ejemplo, si una variable de edad contiene valores como 250 o -5, el problema no es la escala. El problema es que hay datos inválidos. Si una variable de ingresos tiene algunos valores extremadamente altos, debemos decidir si son casos reales, errores o situaciones excepcionales. Escalar no significa limpiar automáticamente. Primero debemos comprender la variable; luego decidir cómo transformarla.

También conviene recordar que el escalado cambia la interpretación directa de los valores. Antes del escalado, una edad de 35 significa 35 años. Después de estandarizar, ese valor puede convertirse en algo como -0.4 o 0.2, dependiendo de la media y la desviación estándar del conjunto de entrenamiento. Para el modelo, esa representación puede ser más útil. Para una persona, en cambio, puede ser menos intuitiva. Por eso, cuando comunicamos resultados, muchas veces necesitamos volver a pensar en las unidades originales o explicar claramente qué transformaciones se aplicaron.

En un flujo de trabajo ordenado, el escalado suele integrarse junto con otras transformaciones. Las variables numéricas pueden recibir imputación y escalado. Las variables categóricas pueden recibir imputación y One-Hot Encoding. Como cada tipo de variable necesita un tratamiento distinto, más adelante en este capítulo presentaremos `ColumnTransformer`, una herramienta que permite aplicar transformaciones diferentes a columnas diferentes.

El escalado de variables numéricas consiste en transformar columnas cuantitativas para que sus valores queden en una escala más adecuada para ciertos modelos. Es especialmente importante cuando las variables tienen magnitudes muy distintas y el modelo es sensible a esas diferencias. Como toda transformación que aprende información de los datos, el escalado debe ajustarse solo con el conjunto de entrenamiento y luego aplicarse al conjunto de prueba. Escalar no reemplaza la exploración ni la limpieza, pero puede ser un paso clave para que muchos modelos aprendan de manera más equilibrada.

Transformaciones que deben aprenderse solo con entrenamiento

En machine learning, algunas transformaciones de datos son simples cambios de formato, pero otras necesitan “aprender” información a partir del conjunto de datos. Esta diferencia es muy importante para evitar evaluaciones engañosas.

Por ejemplo, si cambiamos el nombre de una columna, corregimos un tipo de dato o reemplazamos una coma decimal por un punto, estamos haciendo una transformación mecánica. No necesitamos observar la distribución completa de los datos para decidir

cómo hacerlo. Pero si imputamos valores faltantes usando la media, si escalamos una variable usando su promedio y desviación estándar, o si identificamos las categorías disponibles para aplicar One-Hot Encoding, la transformación necesita calcular algo a partir de los datos.

En esos casos, la transformación tiene dos momentos distintos. Primero, aprende cierta información del conjunto de entrenamiento. Luego, aplica esa misma regla a otros datos.

Esta lógica puede representarse así:

```
transformador.fit(X_train)
X_train_transformado = transformador.transform(X_train)
X_test_transformado = transformador.transform(X_test)
```

El método `.fit()` aprende lo necesario a partir de los datos de entrenamiento. El método `.transform()` aplica lo aprendido. Esta separación entre ajustar y transformar es una de las ideas más importantes del preprocesamiento en `scikit-learn`.

Pensemos en la imputación de valores faltantes. Si una variable numérica tiene valores ausentes y decidimos reemplazarlos por la mediana, esa mediana debe calcularse usando solo `X_train`. Luego, el mismo valor se usa para completar faltantes tanto en `X_train` como en `X_test`. No deberíamos calcular la mediana usando todo el dataset antes de separar, ni tampoco recalculer una mediana distinta para el conjunto de prueba.

La razón es conceptual. El conjunto de prueba debe simular datos nuevos. Cuando el modelo se use en una situación real, no podremos mirar todos los datos futuros para calcular una mediana nueva antes de predecir. Tendremos una regla aprendida previamente durante el entrenamiento y la aplicaremos a los nuevos casos. Por eso, el preprocesamiento debe imitar esa situación.

Lo mismo ocurre con el escalado. Si usamos una técnica como la estandarización, necesitamos calcular la media y la desviación estándar de cada variable numérica. Esos valores deben aprenderse solo con el conjunto de entrenamiento. Luego se aplican tanto al entrenamiento como a la prueba.

```
scaler.fit(X_train_numericas)

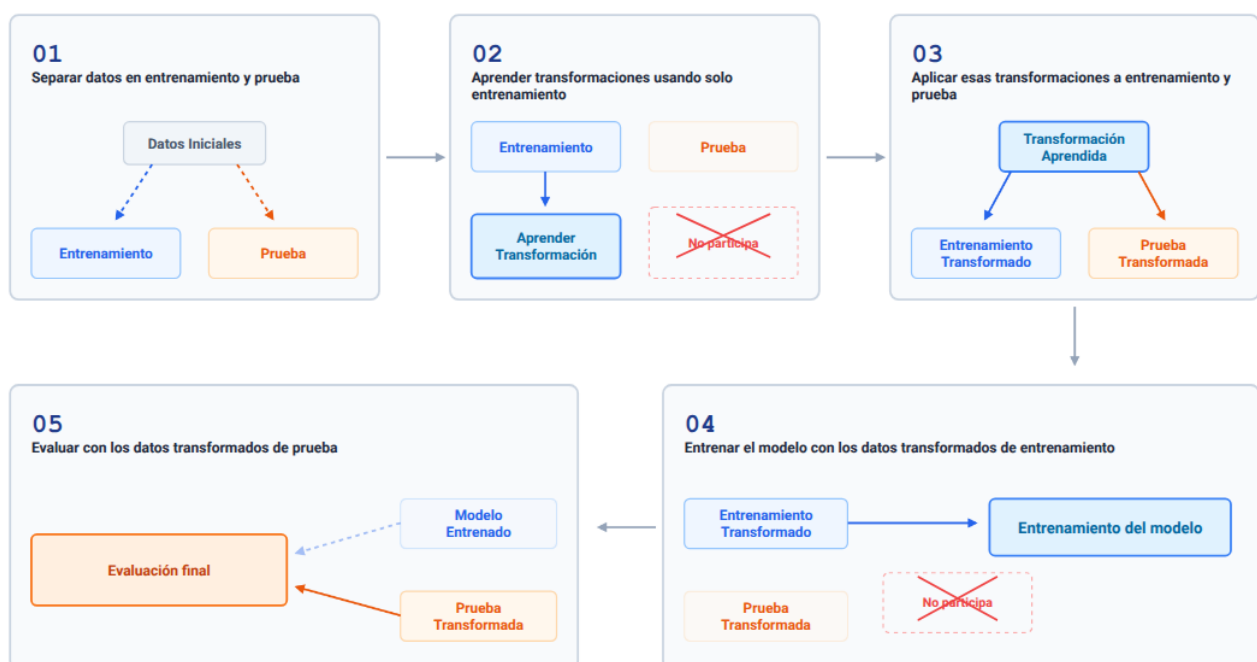
X_train_escalado = scaler.transform(X_train_numericas)
X_test_escalado = scaler.transform(X_test_numericas)
```

Si calculáramos la media y la desviación estándar usando también `X_test`, estaríamos dejando que los datos de prueba influyan en una transformación que forma parte del aprendizaje. Aunque el modelo no vea directamente `y_test`, la evaluación ya no sería completamente limpia, porque los datos reservados habrían participado en la preparación.

Con One-Hot Encoding sucede algo parecido. El encoder aprende qué categorías existen en el conjunto de entrenamiento. A partir de esas categorías, crea una estructura de columnas. Luego, esa misma estructura debe aplicarse al conjunto de prueba. Si en prueba aparece una categoría nueva, el flujo debe manejarla según la configuración elegida, por ejemplo ignorándola con `handle_unknown="ignore"`. Pero no deberíamos usar el conjunto de prueba para decidir de antemano todas las categorías posibles.

Esta idea puede resultar sutil al principio. Muchas personas preparan todo el DataFrame completo y recién después separan entrenamiento y prueba. Ese procedimiento parece cómodo, pero puede introducir fuga de datos. Si imputamos, escalamos o codificamos usando la tabla completa, el conjunto de prueba deja de ser completamente independiente. Parte de su información fue usada para construir las reglas de transformación.

La forma correcta es separar primero y ajustar las transformaciones después, usando solamente el conjunto de entrenamiento. Luego, las reglas aprendidas se aplican a los demás conjuntos. Podemos pensarlo como un flujo:



Flujo completo.

Este orden no es un detalle de estilo. Es una condición para que la evaluación sea más honesta. El modelo y las transformaciones que lo acompañan deben aprender únicamente del conjunto de entrenamiento. El conjunto de prueba debe quedar reservado para evaluar cómo funciona todo el proceso sobre datos que no participaron del aprendizaje.

También debemos notar que, en la práctica, el preprocesamiento y el modelo forman parte de un mismo sistema. Si entrenamos un modelo con variables escaladas, los datos nuevos también deberán escalarse antes de predecir. Si entrenamos con variables categóricas codificadas mediante One-Hot Encoding, los datos nuevos deberán pasar por el mismo encoder. Si imputamos faltantes durante el entrenamiento, también deberemos imputarlos cuando lleguen nuevos casos.

Por eso, no alcanza con guardar solamente el modelo. También necesitamos conservar las reglas de preprocesamiento aprendidas. De lo contrario, podríamos transformar los nuevos datos de una manera distinta a la usada durante el entrenamiento, y las predicciones dejarían de ser confiables.

`scikit-learn` está diseñado para ayudarnos a organizar esta lógica. Muchos objetos de preprocesamiento siguen la misma estructura: `.fit()` para aprender de los datos, `.transform()` para aplicar la transformación y, en algunos casos, `.fit_transform()` para hacer ambas cosas sobre el conjunto de entrenamiento en un solo paso. Más adelante veremos que `Pipeline` permite encadenar transformaciones y modelos para que este orden se respete con mayor facilidad.

Una forma habitual de preparar el conjunto de entrenamiento es:

```
X_train_transformado = transformador.fit_transform(X_train)
```

Esto ajusta el transformador con `X_train` y devuelve `X_train` transformado. Luego, para el conjunto de prueba, usamos solo `.transform()`:

```
X_test_transformado = transformador.transform(X_test)
```

La diferencia es clave. En entrenamiento usamos `.fit_transform()` porque queremos aprender la transformación y aplicarla. En prueba usamos `.transform()` porque no queremos aprender nada nuevo de esos datos; solo queremos aplicar lo que ya fue aprendido.

Algunas transformaciones de datos necesitan aprender información del conjunto sobre el que se ajustan. Imputar valores faltantes, escalar variables numéricas y codificar categorías son ejemplos habituales. Para evitar fuga de datos, esas transformaciones deben aprenderse solo con el conjunto de entrenamiento y luego aplicarse al conjunto de prueba o a datos nuevos. Esta regla mantiene la independencia de la evaluación y ayuda a construir modelos que puedan funcionar fuera del dataset histórico.

Fuga de datos durante el preprocesamiento

La fuga de datos no aparece solamente cuando incluimos en `X` una columna que contiene información del futuro o una consecuencia directa de la variable objetivo. También puede aparecer durante el **preprocesamiento**, es decir, durante las transformaciones que aplicamos a los datos antes de entrenar el modelo.

Este tipo de fuga puede ser más difícil de detectar, porque no siempre se ve como una columna obviamente incorrecta. A veces ocurre por el orden en que organizamos el trabajo. El código puede ejecutarse sin errores, las métricas pueden parecer buenas y, aun así, la evaluación puede estar contaminada.

Recordemos una idea central: el conjunto de prueba debe simular datos nuevos. El modelo no debe aprender de esos datos, ni directa ni indirectamente. Si usamos información del conjunto de prueba para preparar transformaciones, seleccionar reglas, calcular estadísticas o decidir cómo representar los datos, estamos debilitando esa separación.

Un ejemplo sencillo aparece con los valores faltantes. Supongamos que tenemos una variable numérica llamada `ingresos` y decidimos completar sus valores faltantes usando la media. Una forma incorrecta de hacerlo sería calcular la media usando todo el DataFrame antes de separar entrenamiento y prueba. En ese caso, la media usada para imputar incluye información de todos los casos, incluso de aquellos que luego quedarán en el conjunto de prueba.

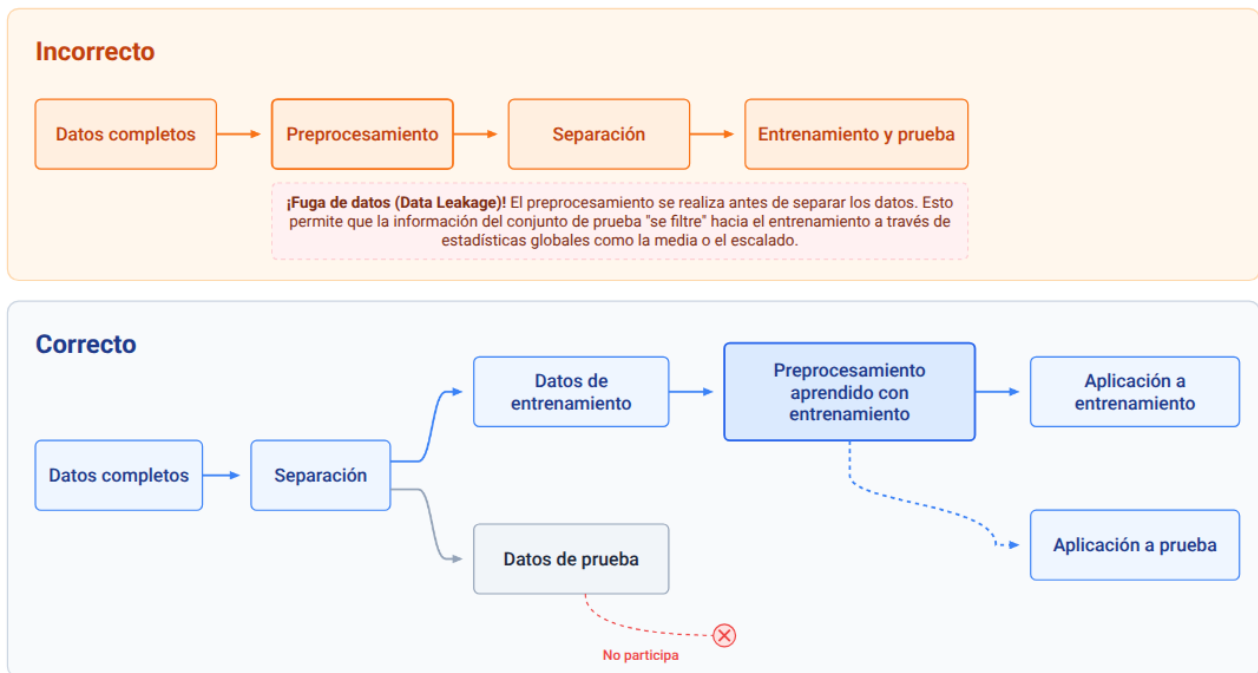
A primera vista, esto puede parecer poco importante. Después de todo, solo estamos calculando una media. Pero desde el punto de vista metodológico, el conjunto de prueba participó en una decisión del preprocesamiento. La evaluación ya no representa completamente el comportamiento del modelo ante datos no vistos, porque parte de la información de esos datos fue usada antes de entrenar.

El mismo problema ocurre con el escalado. Si calculamos la media y la desviación estándar de una variable numérica usando todo el dataset, y luego separamos entrenamiento y prueba, el escalador ya aprendió información del conjunto de

prueba. Después, cuando evaluamos el modelo, parece estar trabajando con datos no vistos, pero esos datos ya influyeron en la transformación.

La forma correcta es separar primero y ajustar las transformaciones después. El escalador debe aprender la media y la desviación estándar usando solamente `X_train`. Luego, esos valores se aplican tanto a `X_train` como a `X_test`.

La diferencia puede resumirse así:



La forma correcta es separar primero y ajustar las transformaciones después.

Este orden es una de las reglas más importantes para construir evaluaciones confiables. Primero separamos. Luego aprendemos las transformaciones con el conjunto de entrenamiento. Después aplicamos esas transformaciones al conjunto de prueba.

Con variables categóricas ocurre algo similar. Si aplicamos One-Hot Encoding antes de separar los datos, el codificador puede aprender categorías que solo aparecen en el conjunto de prueba. Esto significa que la estructura de columnas fue construida usando información de casos que deberían permanecer reservados. En muchos ejemplos pequeños puede no parecer grave, pero el principio es el mismo: el conjunto de prueba no debe ayudar a construir el preprocesamiento.

En un flujo correcto, el codificador aprende las categorías presentes en `X_train`. Luego transforma `X_train` y `X_test` usando esa estructura. Si en `X_test` aparece una categoría nueva, el sistema debe manejarla como una categoría desconocida, por ejemplo ignorándola si usamos una configuración adecuada. Lo que no debe ocurrir es que el conjunto de prueba defina de antemano qué categorías conoce el modelo.

También puede haber fuga de datos durante la selección de variables. Supongamos que antes de separar los datos analizamos todo el dataset para elegir las variables más relacionadas con `y`. Si usamos también los datos que luego serán de prueba para tomar esa decisión, la evaluación puede volverse optimista. En cierto modo, usamos el conjunto de prueba para diseñar el modelo, aunque no lo hayamos usado directamente en `.fit()`.

Esto muestra que la fuga durante el preprocesamiento no siempre depende de una única instrucción, sino del flujo completo. Debemos preguntarnos qué decisiones se aprendieron a partir de los datos y sobre qué parte de los datos se aprendieron. Si una decisión forma parte del proceso de entrenamiento, debe basarse solo en entrenamiento.

Una forma práctica de pensar este problema es imaginar qué ocurriría cuando el modelo se use con datos completamente nuevos. En ese momento, no podremos recalcular la media usando también los nuevos casos antes de predecir. No podremos reconstruir el conjunto completo de categorías mirando el futuro. No podremos elegir variables otra vez usando respuestas que todavía no conocemos. Tendremos un proceso ya aprendido y deberemos aplicarlo a los nuevos datos.

Por eso, el preprocesamiento debe formar parte del sistema que se entrena. No es una tarea externa que hacemos una vez sobre toda la tabla sin consecuencias. Si una transformación aprende estadísticas, categorías, parámetros o criterios, entonces esa transformación debe ajustarse con los datos de entrenamiento y conservarse para transformar datos futuros.

Esta idea también ayuda a entender por qué herramientas como `Pipeline` son tan importantes en `scikit-learn`. Un pipeline permite encadenar transformaciones y modelo en un único flujo. De esa manera, cuando entrenamos, las transformaciones se ajustan correctamente sobre el conjunto de entrenamiento. Y cuando predecimos, las mismas transformaciones se aplican a los datos nuevos antes de generar la predicción. Más adelante en este capítulo retomaremos esta idea.

La fuga de datos durante el preprocesamiento suele ser menos visible que una columna mal elegida, pero puede afectar seriamente la evaluación. Un modelo puede parecer más preciso de lo que realmente será en la práctica. Esto es especialmente problemático porque lleva a tomar decisiones con una confianza injustificada. Cuando el modelo se aplica a datos realmente nuevos, el desempeño puede caer y resultar muy distinto del observado durante el desarrollo.

No siempre el impacto de esta fuga será enorme. En algunos datasets, calcular una media con todos los datos o solo con entrenamiento puede producir valores muy parecidos. Pero el problema no debe evaluarse únicamente por cuánto cambia una

métrica en un ejemplo específico. La regla metodológica es importante porque protege la validez del proceso. Si la respetamos desde el comienzo, evitamos errores más graves en proyectos futuros.

La fuga de datos durante el preprocesamiento ocurre cuando usamos información del conjunto de prueba, o de datos que deberían simular el futuro, para aprender transformaciones o tomar decisiones antes de entrenar y evaluar. Puede aparecer al imputar faltantes, escalar variables, codificar categorías o seleccionar variables usando todo el dataset. La forma de evitarla es separar primero, ajustar las transformaciones solo con el conjunto de entrenamiento y aplicar luego esas transformaciones al conjunto de prueba. Así mantenemos una evaluación más honesta y un flujo de trabajo más parecido al uso real del modelo.

Introducción a ColumnTransformer

A esta altura ya vimos que no todas las columnas de un conjunto de datos necesitan el mismo tratamiento. Algunas variables son numéricas y pueden requerir imputación de valores faltantes, escalado u otras transformaciones. Otras variables son categóricas y pueden necesitar imputación, limpieza de categorías y One-Hot Encoding. También puede haber columnas que decidimos no usar, columnas que se mantienen sin cambios o columnas que requieren transformaciones especiales.

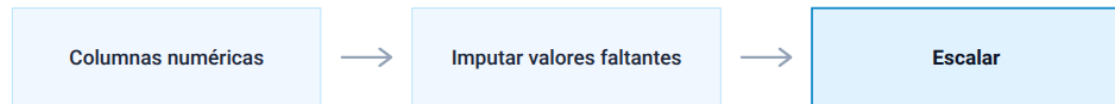
Este es un punto clave del preprocesamiento: **no preparamos todas las variables de la misma manera**. La transformación adecuada depende del tipo de dato y del significado de cada columna.

Imaginemos un conjunto de datos de clientes con estas variables:

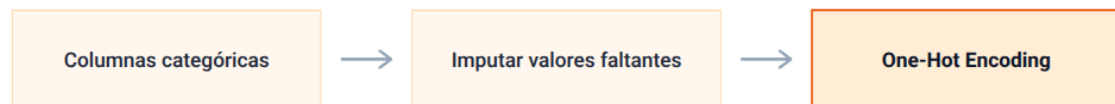
```
edad
ingresos_mensuales
compras_mes
metodo_pago
ciudad
tipo_cliente
```

Las primeras tres columnas son numéricas. Podríamos querer completar valores faltantes con la mediana y luego escalar sus valores. Las últimas tres son categóricas. Tal vez queramos completar faltantes con la categoría más frecuente y luego aplicar One-Hot Encoding. Conceptualmente, el preprocesamiento podría pensarse así:

Columnas numéricas



Columnas categóricas

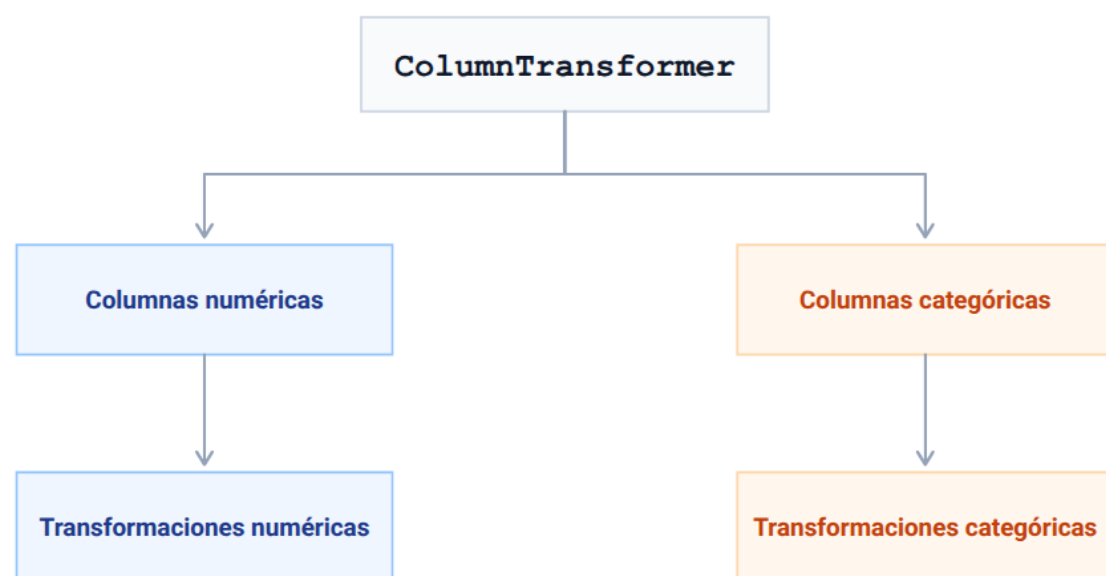


Etapas del preprocesamiento.

El problema es que, si hacemos todo esto manualmente, el flujo puede volverse desordenado. Tendríamos que separar columnas numéricas y categóricas, aplicar transformaciones distintas, asegurarnos de aprender esas transformaciones solo con entrenamiento, transformar luego el conjunto de prueba y finalmente unir todas las columnas transformadas en una única matriz para el modelo. Todo eso es posible, pero aumenta el riesgo de errores.

`scikit-learn` ofrece una herramienta pensada justamente para este tipo de situación: `ColumnTransformer`.

Un `ColumnTransformer` permite aplicar transformaciones diferentes a distintos grupos de columnas dentro de un mismo flujo. En lugar de transformar toda la tabla de la misma manera, podemos decir: “a estas columnas aplicales este tratamiento; a estas otras columnas aplicales este otro”. La idea general puede representarse así:



Un `ColumnTransformer` permite aplicar transformaciones diferentes a distintos grupos de columnas.

En código, una estructura inicial podría verse así:

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
```

Luego podríamos definir, por separado, un tratamiento para columnas numéricas y otro para columnas categóricas:

```
transformador_numerico = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

transformador_categorico = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore"))
])
```

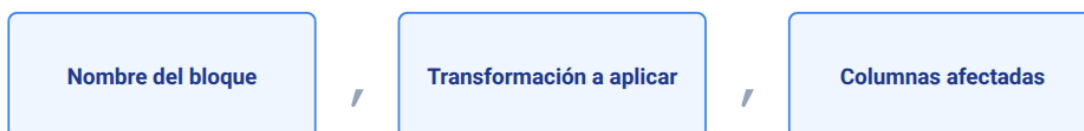
Todavía no necesitamos detenernos en cada detalle de esta sintaxis. Lo importante es observar la estructura. Para las variables numéricas, primero se imputan faltantes y luego se escalan. Para las categóricas, primero se imputan faltantes y luego se codifican. Cada grupo de columnas recibe un tratamiento distinto.

Después, `ColumnTransformer` permite unir esos tratamientos:

```
preprocesamiento = ColumnTransformer(
    transformers=[
        ("num", transformador_numerico, columnas_numericas),
        ("cat", transformador_categorico, columnas_categoricas)
    ]
)
```

Esta expresión puede leerse de manera bastante natural. Estamos creando un objeto llamado `preprocesamiento` que aplicará el transformador numérico a las columnas numéricas y el transformador categórico a las columnas categóricas.

Cada elemento dentro de `transformers` tiene tres partes:



Identifica la transformación

Indica qué operación se realizará

Define dónde se aplicará

Partes de transformers.

Por ejemplo:

```
("num", transformador_numerico, columnas_numericas)
```

significa: “llamemos `num` a este bloque; usemos el transformador numérico; apliquémoslo sobre las columnas numéricas”.

Y:

```
("cat", transformador_categorico, columnas_categoricas)
```

significa: “llamemos `cat` a este bloque; usemos el transformador categórico; apliquémoslo sobre las columnas categóricas”.

La ventaja de esta herramienta no es solamente que el código queda más prolijo. La ventaja principal es metodológica. `ColumnTransformer` ayuda a organizar el preprocesamiento de forma coherente con la separación entre entrenamiento y prueba. Cuando se ajusta sobre `X_train`, cada transformación aprende lo que necesita aprender solamente del conjunto de entrenamiento. Luego, cuando se aplica sobre `X_test`, utiliza esas mismas reglas aprendidas.

Por ejemplo, el imputador numérico aprende la mediana de cada columna numérica usando `X_train`. El escalador aprende la media y la desviación estándar usando `X_train`. El codificador categórico aprende las categorías presentes en `X_train`. Luego, esas decisiones se aplican al conjunto de prueba sin volver a aprender desde cero.

Conceptualmente, el flujo correcto sería:

```
preprocesamiento.fit(X_train)

X_train_preparado = preprocesamiento.transform(X_train)
X_test_preparado = preprocesamiento.transform(X_test)
```

También es común usar:

```
X_train_preparado = preprocesamiento.fit_transform(X_train)
X_test_preparado = preprocesamiento.transform(X_test)
```

La diferencia entre `fit_transform()` y `transform()` vuelve a ser importante. En entrenamiento, ajustamos y transformamos. En prueba, solamente transformamos. El conjunto de prueba no debe enseñar nada nuevo al preprocesamiento.

`ColumnTransformer` también permite mantener juntos pasos que de otro modo quedarían dispersos en el código. Esto ayuda a que el proyecto sea más fácil de leer, reproducir y revisar. Si alguien quiere saber cómo se prepararon las variables numéricas, puede mirar el bloque correspondiente. Si quiere saber qué se hizo con las categóricas, puede mirar el otro bloque. La lógica del preprocesamiento queda explícita.

Otra ventaja es que esta herramienta prepara el camino para usar `Pipeline`, que veremos en la próxima sección. Un pipeline puede unir el preprocesamiento y el modelo en un único flujo. De ese modo, el modelo no se entrena sobre datos preparados manualmente en pasos sueltos, sino dentro de una secuencia ordenada que incluye transformación y aprendizaje.

Esto es especialmente útil porque, en la práctica, el modelo nunca debería pensarse separado de su preprocesamiento. Si entrenamos un modelo con columnas imputadas, escaladas y codificadas, entonces cada dato nuevo deberá pasar por exactamente las mismas transformaciones antes de llegar al modelo. `ColumnTransformer` ayuda a definir esas transformaciones de manera clara.

También es importante aclarar que `ColumnTransformer` no decide por nosotros qué columnas son numéricas, qué columnas son categóricas ni qué estrategia conviene usar. Es una herramienta para aplicar decisiones, no para reemplazar el criterio del analista. Antes de construirlo, necesitamos haber revisado los datos, entendido las variables y elegido qué tratamiento corresponde a cada grupo.

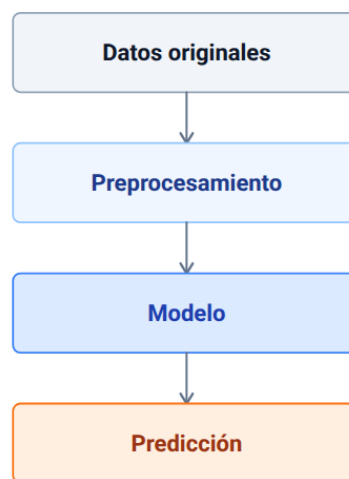
`ColumnTransformer` permite aplicar transformaciones diferentes a diferentes columnas dentro de un mismo flujo de preprocesamiento. Es especialmente útil cuando trabajamos con datos tabulares que mezclan variables numéricas y categóricas. Su uso ayuda a mantener el código ordenado, evita transformaciones manuales dispersas y facilita respetar una regla fundamental: aprender las transformaciones solo con los datos de entrenamiento y aplicarlas luego a prueba o a datos nuevos.

Introducción a `Pipeline`

Cuando trabajamos con machine learning, el modelo no debería verse como una pieza aislada. Antes de que los datos lleguen al modelo, normalmente deben pasar por varias transformaciones: completar valores faltantes, escalar variables numéricas, codificar variables categóricas, seleccionar columnas y aplicar otros pasos de preparación. Todo ese proceso forma parte de la solución.

Un error frecuente es pensar que el preprocesamiento es algo que hacemos “aparte” y que luego el modelo empieza recién cuando llamamos a `.fit()`. En realidad, si el modelo fue entrenado con datos imputados, escalados y codificados, entonces cualquier dato nuevo deberá recibir exactamente el mismo tratamiento antes de ser usado para predecir. Si esa secuencia no se respeta, las predicciones pueden ser incorrectas o inconsistentes.

`Pipeline` es una herramienta de `scikit-learn` que permite encadenar varios pasos en un único flujo ordenado. Cada paso recibe los datos, realiza una transformación o un entrenamiento, y pasa el resultado al siguiente paso. De esta manera, el preprocesamiento y el modelo quedan unidos dentro de una misma estructura. La idea general puede representarse así:



Etapas del pipeline.

En lugar de transformar manualmente los datos por un lado y entrenar el modelo por otro, un pipeline nos permite definir la secuencia completa. Por ejemplo, si ya tenemos un objeto de preprocesamiento construido con `ColumnTransformer`, podríamos unirlo con un modelo de esta manera:

```
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression

modelo = Pipeline(steps=[
    ("preprocesamiento", preprocesamiento),
    ("clasificador", LogisticRegression())
])
```

En este ejemplo, `modelo` no es solamente el algoritmo de regresión logística. Es un flujo completo que primero aplica el preprocesamiento y luego entrena el clasificador. El nombre `modelo` puede parecer amplio, pero en la práctica muchas veces usamos

esa palabra para referirnos al sistema completo que transforma datos y produce predicciones.

Una vez definido el pipeline, podemos entrenarlo usando la misma lógica general de `scikit-learn`:

```
modelo.fit(X_train, y_train)
```

Esta línea ejecuta varias cosas en orden. Primero, el paso de preprocesamiento se ajusta usando `X_train`: los imputadores aprenden medianas o categorías frecuentes, los escaladores aprenden medias y desviaciones estándar, y los codificadores aprenden las categorías disponibles en las variables categóricas. Luego, los datos transformados llegan al modelo final, que aprende la relación entre las variables preparadas y `y_train`.

Lo importante es que todo ese aprendizaje ocurre usando solamente el conjunto de entrenamiento. El pipeline respeta la separación entre ajustar transformaciones y aplicarlas. Esto reduce mucho el riesgo de cometer errores manuales que produzcan fuga de datos.

Después, cuando queremos predecir sobre el conjunto de prueba, usamos:

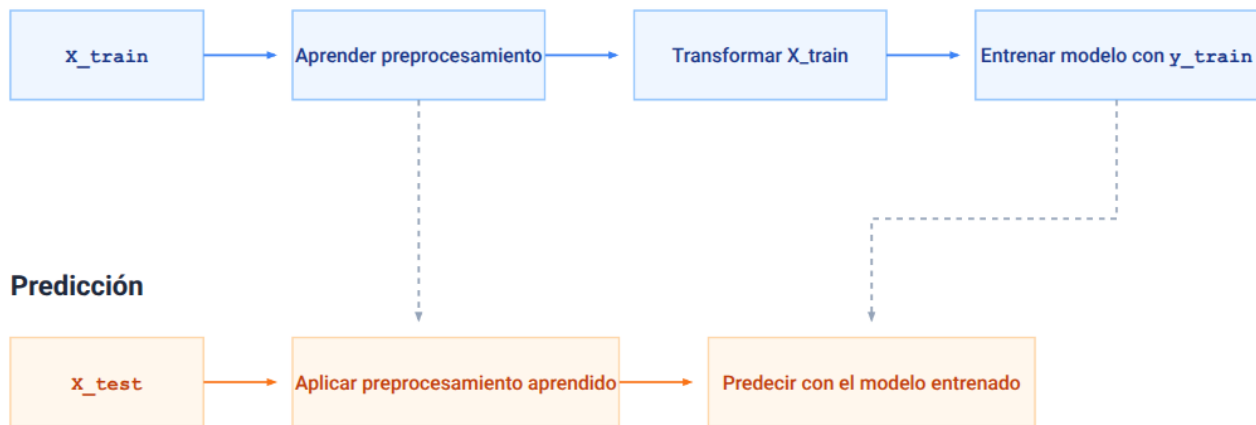
```
predicciones = modelo.predict(X_test)
```

Esta línea también ejecuta varios pasos en orden. Primero aplica a `X_test` el preprocesamiento ya aprendido durante el entrenamiento. No vuelve a calcular medianas, medias, desviaciones estándar ni categorías usando el conjunto de prueba. Solo aplica las reglas que ya aprendió con `X_train`. Luego, los datos transformados pasan al modelo entrenado, que genera las predicciones.

Esa diferencia es fundamental. Durante `.fit()`, el pipeline aprende transformaciones y entrena el modelo. Durante `.predict()`, aplica transformaciones ya aprendidas y produce predicciones. El conjunto de prueba no participa del aprendizaje.

Podemos pensarlo así:

Entrenamiento



El conjunto de prueba no participa del aprendizaje.

Esta estructura hace que el flujo sea más seguro y más fácil de reproducir. Si el preprocesamiento queda escrito en pasos sueltos, es más fácil olvidar una transformación, aplicarla en otro orden o calcular estadísticas usando datos que no corresponden. Con un pipeline, la secuencia queda definida en un solo objeto.

También mejora la claridad del proyecto. Al mirar la definición del pipeline, podemos ver qué pasos forman parte del proceso completo. Por ejemplo, podemos observar que primero se aplica un `ColumnTransformer` y luego un modelo. Si dentro del `ColumnTransformer` las columnas numéricas reciben imputación y escalado, y las categóricas reciben imputación y One-Hot Encoding, toda la preparación queda integrada en una estructura coherente.

Un pipeline también es útil cuando queremos comparar modelos. Podemos mantener el mismo preprocesamiento y cambiar solo el modelo final. Por ejemplo, podríamos probar una regresión logística, un árbol de decisión o un bosque aleatorio usando el mismo tratamiento de datos. Esto permite comparar alternativas de manera más ordenada, sin repetir manualmente todos los pasos de transformación.

Además, cuando el proyecto crece, el pipeline facilita guardar y reutilizar el sistema completo. No alcanza con guardar solamente el modelo final si las predicciones dependen de transformaciones previas. Si en el entrenamiento se usó un imputador, un escalador y un codificador, esas piezas también deben conservarse. Un pipeline agrupa todo eso en una única estructura que puede usarse luego para predecir sobre datos nuevos siguiendo exactamente el mismo proceso.

Esto es especialmente importante en situaciones reales. Imaginemos que entrenamos un modelo con variables categóricas codificadas mediante One-Hot Encoding. Si más adelante llega un nuevo caso para predecir, no podemos codificarlo de cualquier manera. Debemos usar el mismo encoder aprendido durante el entrenamiento, con

las mismas columnas y el mismo orden. Si entrenamos con variables numéricas escaladas, tampoco podemos escalar los nuevos datos recalculando medias distintas. Debemos aplicar el mismo escalador ya aprendido. El pipeline ayuda a preservar esa coherencia.

También debemos recordar que un pipeline no decide por nosotros qué pasos usar. Sigue siendo necesario comprender los datos, elegir variables, distinguir numéricas de categóricas y definir estrategias adecuadas para cada caso. Pipeline organiza el flujo, pero no reemplaza el criterio. Una mala decisión dentro de un pipeline sigue siendo una mala decisión, aunque esté escrita de manera prolija.

En este libro usaremos Pipeline de manera gradual. Al principio puede parecer una estructura más compleja que aplicar transformaciones manualmente, pero su ventaja aparece rápidamente. Permite ordenar el preprocesamiento, evitar fuga de datos, repetir experimentos con mayor seguridad y construir flujos más cercanos a lo que se necesita en un proyecto real.

Pipeline permite encadenar pasos de preprocesamiento y modelado dentro de un único flujo. Durante el entrenamiento, ajusta las transformaciones usando solo los datos de entrenamiento y entrena el modelo final. Durante la predicción, aplica las transformaciones ya aprendidas y luego genera predicciones. Su uso ayuda a mantener el proceso ordenado, reproducible y coherente entre entrenamiento, prueba y datos nuevos.

Preprocesar sin romper la evaluación

En este capítulo trabajamos una idea que será cada vez más importante a medida que avancemos: preparar los datos no consiste solamente en “dejarlos prolijos” para que el modelo pueda entrenarse. El preprocesamiento forma parte del proceso de aprendizaje y, si se organiza mal, puede afectar la validez de la evaluación.

Los datos reales rara vez llegan listos para usar. Pueden tener valores faltantes, variables categóricas expresadas como texto, columnas numéricas en escalas muy diferentes, categorías inconsistentes o información que necesita transformarse antes de llegar al modelo. Cada uno de estos problemas requiere decisiones específicas. No todas las columnas se tratan igual, y no todas las transformaciones tienen las mismas consecuencias.

Los valores faltantes deben analizarse antes de reemplazarse. A veces representan errores; otras veces, ausencia legítima de información; y en algunos casos, la propia ausencia puede tener significado. Las variables categóricas necesitan convertirse a una forma numérica adecuada, pero esa conversión debe respetar su significado.

One-Hot Encoding permite representar categorías nominales sin imponer un orden artificial. Las variables numéricas, por su parte, pueden necesitar escalado para que ciertos modelos no se vean dominados por columnas con valores de mayor magnitud.

Pero el punto central del capítulo no es solo qué transformaciones existen, sino **cuándo y con qué datos deben aprenderse**. Algunas transformaciones calculan información a partir del conjunto de datos: una media, una mediana, una desviación estándar, una categoría frecuente o una lista de categorías posibles. Esas transformaciones no deben aprenderse usando todo el dataset, porque eso permitiría que el conjunto de prueba influya en el proceso.

La regla metodológica es clara: primero separamos entrenamiento y prueba; luego ajustamos el preprocesamiento usando solo el conjunto de entrenamiento; finalmente aplicamos esas transformaciones al conjunto de prueba. De esta manera, la evaluación sigue representando mejor el comportamiento del modelo frente a datos no vistos.

La fuga de datos durante el preprocesamiento puede ser difícil de detectar porque no siempre aparece como una columna obviamente incorrecta. A veces surge de aplicar una imputación, un escalado o una codificación antes de separar los datos. El modelo puede obtener buenos resultados, pero esos resultados estarán apoyados en una evaluación contaminada. Por eso, el orden del flujo importa tanto como las transformaciones elegidas.

Herramientas como `ColumnTransformer` y `Pipeline` ayudan a organizar este proceso. `ColumnTransformer` permite aplicar tratamientos distintos a distintos grupos de columnas, como variables numéricas y categóricas. `Pipeline` permite encadenar el preprocesamiento y el modelo dentro de un único flujo, reduciendo el riesgo de aplicar pasos en un orden incorrecto o de aprender transformaciones con datos que deberían permanecer reservados.

Preprocesar bien significa transformar los datos de una manera que sea útil para el modelo, pero también válida desde el punto de vista metodológico. Un preprocesamiento correcto no solo mejora la calidad de la entrada; protege la evaluación, mantiene la separación entre entrenamiento y prueba y prepara el camino para construir modelos que puedan aplicarse luego a datos nuevos con el mismo tratamiento aprendido durante el entrenamiento.

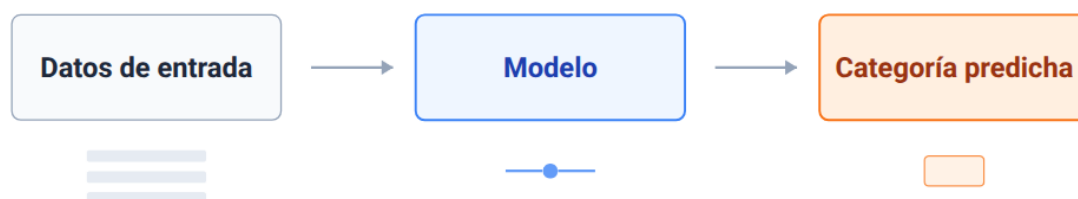
Capítulo 7 · Clasificación: predecir categorías

¿Qué significa clasificar?

Clasificar significa asignar un caso a una categoría. En machine learning, hablamos de clasificación cuando queremos que un modelo reciba datos sobre un caso nuevo y produzca como salida una clase posible.

La clasificación es una de las tareas más habituales del aprendizaje supervisado. Esto significa que, para entrenar un modelo de clasificación, necesitamos ejemplos anteriores donde la respuesta ya esté conocida. Cada ejemplo contiene variables de entrada, que describen el caso, y una etiqueta o clase, que indica a qué categoría pertenece.

Podemos pensarlo de esta manera:



La salida del modelo no es una cantidad numérica continua, sino una categoría.

Si el modelo recibe información sobre un cliente, puede predecir si pertenece a la categoría “abandona” o “permanece”. Si recibe información sobre una solicitud de crédito, puede predecir si corresponde a “aprobada” o “rechazada”. Si recibe datos clínicos de un paciente, puede predecir si un caso requiere revisión médica prioritaria o no. En todos estos ejemplos, la salida del modelo no es una cantidad numérica continua, sino una categoría.

Esta diferencia es importante. En un problema de regresión, el modelo intenta estimar un número: un precio, una duración, una temperatura, una demanda esperada. En clasificación, en cambio, el modelo intenta elegir entre clases. La pregunta no es “¿cuánto vale?”, sino “¿a qué categoría pertenece?”.

Por ejemplo, imaginemos una empresa que quiere anticipar si ciertos clientes podrían abandonar un servicio. Para cada cliente, la empresa puede tener información como antigüedad, frecuencia de uso, cantidad de reclamos, días desde la última actividad, tipo de plan contratado y cambios recientes en el consumo. Además, en datos históricos, sabe qué clientes finalmente abandonaron y cuáles permanecieron.

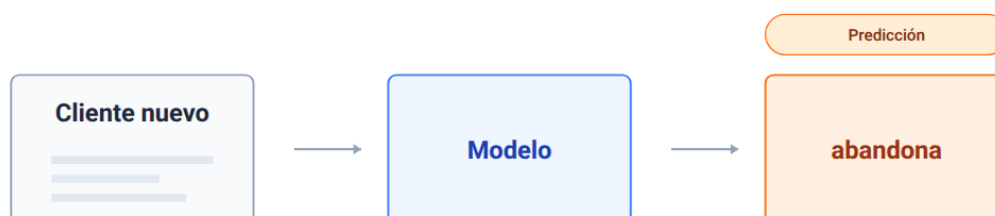
En ese caso, la variable objetivo podría tener dos clases:



La variable objetivo puede tener dos clases.

El modelo de clasificación observa los ejemplos históricos y busca patrones en las variables de entrada que se relacionen con cada clase. Tal vez encuentre que muchos días sin actividad, reclamos recientes y una caída brusca en el uso suelen aparecer con mayor frecuencia entre clientes que abandonan. Pero no necesita basarse en una sola regla. Puede combinar varias señales para estimar a qué clase pertenece un cliente actual.

Cuando el modelo se aplica a un nuevo cliente, no conoce todavía el resultado real. Solo recibe sus variables de entrada. A partir de lo aprendido, produce una predicción:



El modelo realiza una predicción.

Esa predicción no significa que el abandono sea seguro. Significa que, según los patrones aprendidos en los datos, el modelo considera que esa clase es la más probable o la más adecuada. Esta idea será importante cuando hablemos de probabilidades, porque muchos clasificadores no solo pueden devolver una clase final, sino también una estimación de confianza o probabilidad asociada a cada clase.

Otro ejemplo aparece en la aprobación de créditos. Una institución puede tener datos históricos de solicitudes: ingresos declarados, historial de pagos, deudas existentes, antigüedad laboral, monto solicitado, plazo del crédito y otros indicadores. Para

solicitudes anteriores, también conoce el resultado: aprobada o rechazada, o quizás cumplió o incumplió el pago después de recibir el crédito.

Si el problema se formula como clasificación, el modelo intentará aprender qué características se asocian con cada categoría. Luego, ante una nueva solicitud, podrá predecir una clase o asignar un nivel de riesgo. Nuevamente, el modelo no comprende la vida financiera de una persona como lo haría un analista humano. Aprende relaciones estadísticas presentes en los datos históricos.

La clasificación también se usa en contextos médicos, aunque allí el cuidado debe ser mucho mayor. Un modelo podría recibir variables como edad, resultados de laboratorio, síntomas registrados, antecedentes, mediciones clínicas o características de una imagen médica. Su salida podría ser una categoría como “bajo riesgo” o “alto riesgo”, “requiere revisión” o “no requiere revisión inmediata”, “compatible con cierta condición” o “no compatible”.

En este tipo de aplicación, es fundamental entender que clasificar no equivale a emitir una verdad absoluta. Un modelo puede apoyar una decisión, priorizar casos o señalar alertas, pero sus predicciones deben interpretarse dentro del contexto profesional adecuado. La clasificación puede ser útil, pero también puede cometer errores, y esos errores tienen consecuencias distintas según el ámbito.

En términos de datos, un problema de clasificación supervisada se organiza con la estructura que ya conocemos:

```
X = variables de entrada  
y = clase o etiqueta
```

X contiene la información que describe cada caso. **y** contiene la clase conocida en los datos históricos. Durante el entrenamiento, el modelo aprende a relacionar **X** con **y**. Durante la predicción, recibe nuevos valores de **X** y produce una clase estimada.

Por ejemplo:

```
X = df[["antigüedad", "reclamos", "dias_sin_actividad", "tipo_plan"]]  
y = df["abandono"]
```

Aquí **abandono** sería la variable objetivo. Si contiene valores como “sí” y “no”, o 1 y 0 representando esas clases, estamos frente a un problema de clasificación. Las demás columnas son variables predictoras que el modelo usará para aprender patrones asociados con cada etiqueta.

Un punto importante es que las clases deben estar bien definidas. Si queremos clasificar clientes como “en riesgo” o “sin riesgo”, necesitamos saber qué significa

exactamente “en riesgo”. ¿Riesgo de abandonar en los próximos 30 días? ¿Riesgo de no pagar una cuota? ¿Riesgo de presentar un reclamo? La palabra puede parecer clara, pero para entrenar un modelo necesitamos una definición concreta y observable.

Lo mismo ocurre con etiquetas como “aprobado”, “rechazado”, “urgente”, “normal”, “positivo” o “negativo”. Un modelo aprende a partir de esas etiquetas. Si las etiquetas fueron asignadas con criterios inconsistentes, el modelo aprenderá esa inconsistencia. Si distintas personas clasificaron casos de manera diferente, el modelo puede encontrar patrones confusos. Si la etiqueta no representa bien el problema real, el modelo puede obtener buenos resultados técnicos pero ser poco útil.

También debemos recordar que clasificar implica simplificar. La realidad suele ser más compleja que una etiqueta. Un cliente no es simplemente “abandona” o “permanece” desde el punto de vista humano; hay motivos, contextos, dudas y cambios posibles. Una solicitud de crédito no se reduce completamente a “aprobada” o “rechazada”; hay políticas, riesgos, condiciones y criterios institucionales. Un caso médico no debería reducirse sin cuidado a una etiqueta automática. La clasificación convierte una situación compleja en una decisión categórica para poder trabajar con datos y modelos.

Esa simplificación puede ser útil, pero debe ser consciente. La calidad del problema de clasificación depende de que las clases tengan sentido, de que los datos sean adecuados, de que las etiquetas sean confiables y de que los resultados se usen con prudencia.

Otro aspecto clave es que una predicción de clasificación puede equivocarse. Un modelo puede clasificar como “abandona” a un cliente que finalmente permanece, o clasificar como “permanece” a uno que abandona. Puede marcar como riesgosa una solicitud que habría sido cumplida, o considerar segura una que luego presenta problemas. Estos errores no son todos iguales. Más adelante estudiaremos cómo evaluarlos con herramientas específicas, pero desde ahora conviene entender que clasificar no es solo acertar o fallar: también importa qué tipo de error se comete.

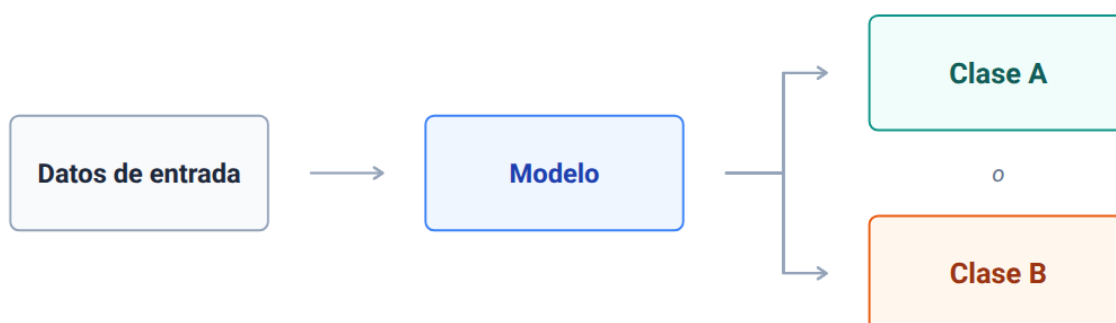
Clasificar significa asignar casos a categorías. En machine learning, un modelo de clasificación aprende a partir de ejemplos históricos con etiquetas conocidas y luego usa lo aprendido para predecir la clase de casos nuevos. La clasificación es útil cuando la respuesta que buscamos es una categoría, no una cantidad. Pero para que tenga sentido, las clases deben estar bien definidas, los datos deben ser adecuados y las predicciones deben interpretarse como resultados probabilísticos o aproximados, no como certezas absolutas.

Clasificación binaria

La **clasificación binaria** aparece cuando el modelo debe elegir entre dos clases posibles. Es uno de los casos más frecuentes y también uno de los más útiles para comenzar a estudiar clasificación, porque su estructura es sencilla de entender: para cada caso, la respuesta pertenece a una de dos categorías.

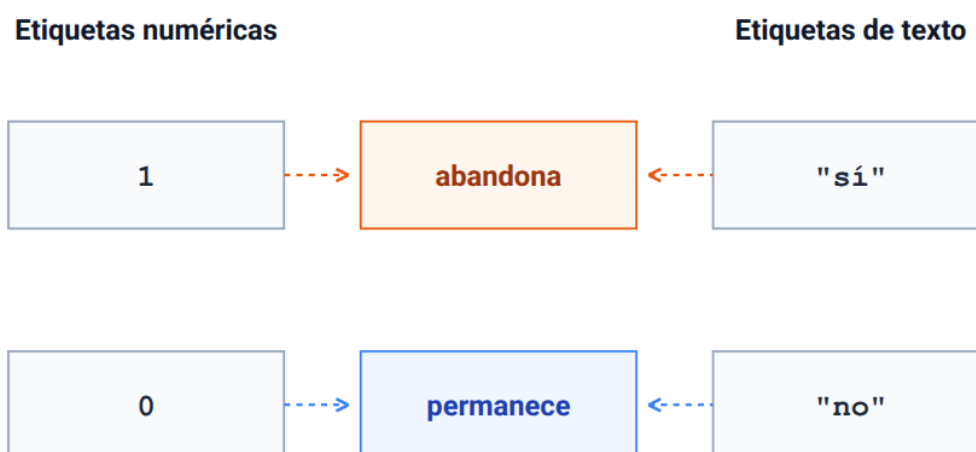
Algunos ejemplos típicos son “abandona” o “permanece”, “aprobado” o “rechazado”, “positivo” o “negativo”, “riesgo alto” o “riesgo bajo”, “requiere revisión” o “no requiere revisión”. En todos estos casos, el modelo no intenta estimar una cantidad, sino decidir cuál de las dos clases parece más adecuada para un caso nuevo.

Podemos representar la idea de esta manera:



Clasificación binaria.

Por ejemplo, una empresa puede querer anticipar si un cliente abandonará un servicio durante el próximo mes. La variable objetivo podría tener dos valores:



La variable objetivo podría tener dos valores. Numéricos o textos.

La forma exacta de escribir las clases puede variar. Lo importante es que hay dos resultados posibles. El modelo aprende a partir de clientes históricos, donde ya se conoce si abandonaron o permanecieron, y luego intenta clasificar clientes actuales.

En código, la variable objetivo podría verse así:

```
X = df[["antiguedad", "reclamos", "dias_sin_actividad", "uso_reciente"]]
y = df["abandono"]
```

Si `abandono` contiene solamente dos clases, estamos frente a un problema de clasificación binaria. El modelo aprenderá a distinguir, a partir de las variables de entrada, qué patrones se asocian más con una clase y cuáles se asocian más con la otra.

Otro ejemplo puede ser la aprobación de créditos. Una institución podría construir un modelo para clasificar solicitudes como “aprobadas” o “rechazadas”, o para estimar si una persona cumplirá o no con el pago de un crédito. En ese caso, la salida también puede organizarse en dos clases: “cumple” o “incumple”, “bajo riesgo” o “alto riesgo”, “aprobado” o “no aprobado”.

Es importante notar que la forma de definir las dos clases depende del problema. No es lo mismo predecir si una solicitud será aprobada que predecir si un crédito ya otorgado será pagado correctamente. Ambos problemas pueden ser binarios, pero no son el mismo problema. Cambia la variable objetivo, cambian los datos necesarios y cambia el significado de los errores.

En clasificación binaria, muchas veces una de las clases se considera la clase de interés. Por ejemplo, en abandono de clientes, la clase de interés suele ser “abandona”. En detección de fraude, suele ser “fraude”. En diagnóstico asistido, puede ser “requiere revisión” o “posible caso positivo”. En incumplimiento crediticio, puede ser “incumple”. Esa clase suele llamarse a veces clase positiva, aunque el nombre “positiva” no significa que sea algo bueno. Solo indica la clase que estamos tratando de detectar o identificar especialmente.

La otra clase suele llamarse clase negativa. En abandono, podría ser “permanece”. En fraude, “no fraude”. En diagnóstico, “no requiere revisión inmediata”. En crédito, “cumple”. Esta forma de nombrar las clases es útil porque muchas herramientas de evaluación distinguen entre casos positivos y negativos. Más adelante veremos métricas que dependen de esta distinción.

Por ahora, lo importante es comprender que una clasificación binaria no siempre tiene errores equivalentes. Si un modelo predice que un cliente abandonará, pero finalmente permanece, comete un tipo de error. Si predice que permanecerá, pero finalmente abandona, comete otro. Los dos son errores, pero pueden tener consecuencias distintas.

Imaginemos un sistema que identifica clientes con riesgo de abandonar para ofrecerles una acción de retención. Si el modelo marca como riesgoso a un cliente que no iba a abandonar, quizá la empresa le ofrezca un beneficio innecesario. Eso tiene un costo. Pero si el modelo no detecta a un cliente que efectivamente abandona, la empresa pierde la oportunidad de intervenir. Ese error puede tener otro costo, quizás más importante.

En un contexto médico, la diferencia puede ser todavía más sensible. Si un modelo marca un caso como sospechoso cuando no lo era, puede generar una revisión adicional. Eso puede implicar tiempo, preocupación o costos. Pero si el modelo no marca un caso que sí requería atención, la consecuencia puede ser mucho más grave. Por eso, en clasificación binaria, no alcanza con saber cuántas veces el modelo acierta. Debemos mirar qué clase de errores comete.

También es frecuente que las clases no estén equilibradas. En muchos problemas reales, una clase aparece mucho más que la otra. La mayoría de los clientes quizás no abandona en un mes determinado. La mayoría de las operaciones financieras no son fraudulentas. La mayoría de las solicitudes puede no presentar incumplimiento. La mayoría de los casos médicos evaluados puede no corresponder a una condición específica.

Esto produce un desafío importante. Un modelo puede parecer bueno si predice siempre la clase mayoritaria. Si el 95% de los clientes permanece, un modelo que siempre predice “permanece” acertaría muchas veces, pero no serviría para detectar abandonos. Si el 99% de las operaciones no son fraudulentas, predecir siempre “no fraude” daría una métrica general alta, pero sería inútil para el objetivo principal.

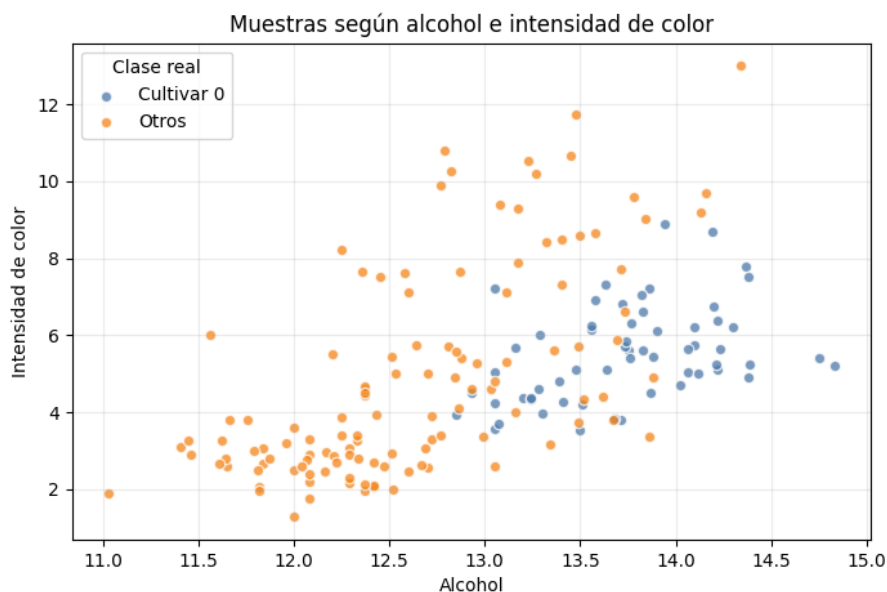
Por eso, la clasificación binaria debe evaluarse con cuidado, especialmente cuando la clase de interés es poco frecuente. La pregunta no es solo “¿cuántas predicciones fueron correctas?”, sino también “¿detectamos los casos que realmente importan?”, “¿cuántas falsas alarmas generamos?” y “¿qué costo tiene cada tipo de error?”. Estos temas se desarrollarán con más detalle cuando estudiemos métricas de clasificación.

Otra característica de muchos modelos de clasificación binaria es que no solo pueden devolver una clase final, sino también una probabilidad estimada. Por ejemplo, un modelo puede indicar que cierto cliente tiene una probabilidad de abandono de 0.78. Luego, el sistema puede convertir esa probabilidad en una clase usando un umbral. Si la probabilidad supera cierto valor, se clasifica como “abandona”; si no, como “permanece”.

Esta idea abre una posibilidad importante: la decisión final puede ajustarse según el contexto. No siempre conviene usar el mismo umbral. Si queremos ser muy sensibles y detectar la mayor cantidad posible de casos de riesgo, podríamos aceptar más

falsas alarmas. Si queremos reducir intervenciones innecesarias, podríamos exigir una probabilidad más alta antes de marcar un caso. Más adelante veremos esta idea con más detalle al estudiar probabilidades y umbrales.

En este punto, alcanza con reconocer que la clasificación binaria no siempre es una decisión rígida desde el comienzo. Muchos modelos producen una puntuación o probabilidad, y la clase final surge de una regla de decisión. Esa regla debe elegirse teniendo en cuenta el problema real.



Suele ocurrir que las clases ocupan regiones diferentes, pero no están separadas perfectamente.

También debemos cuidar la interpretación de las etiquetas 0 y 1. En muchos datasets, las clases binarias se codifican con números. Esto es práctico para trabajar con modelos, pero no debemos olvidar que esos números representan categorías. Si `0` significa “permanece” y `1` significa “abandona”, no estamos prediciendo una cantidad. Estamos prediciendo una clase. El número es una codificación, no una magnitud.

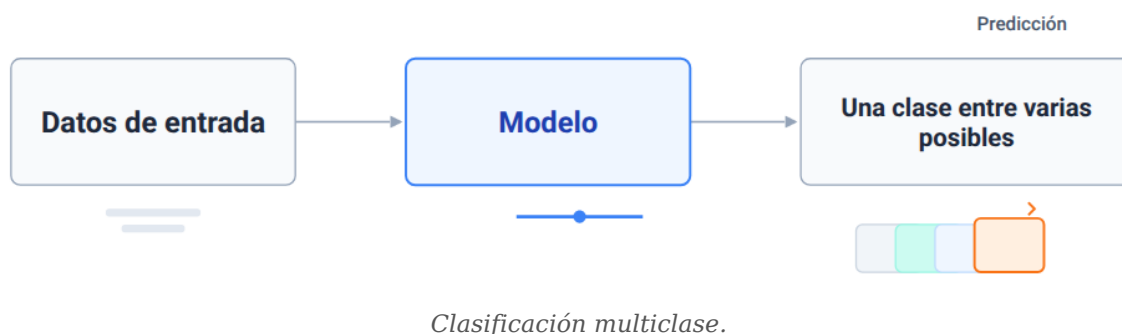
La clasificación binaria es un tipo de clasificación en el que existen dos clases posibles. Es útil para problemas como abandono de clientes, aprobación o rechazo de solicitudes, detección de eventos, identificación de riesgo o diagnóstico asistido. Aunque su estructura parece simple, requiere mucho cuidado: las clases deben estar bien definidas, los errores pueden tener consecuencias distintas y las proporciones entre clases pueden estar muy desbalanceadas. Comprender estas características es fundamental antes de entrenar y evaluar modelos binarios.

Clasificación multiclase

La **clasificación multiclase** aparece cuando el modelo debe elegir entre más de dos clases posibles. A diferencia de la clasificación binaria, donde la salida pertenece a una de dos categorías, en un problema multiclase existen tres o más opciones.

Por ejemplo, un modelo puede clasificar una solicitud de crédito como “riesgo bajo”, “riesgo medio” o “riesgo alto”. Una plataforma de atención al cliente puede clasificar un reclamo como “facturación”, “soporte técnico”, “envíos”, “devoluciones” o “consulta general”. Un sistema educativo puede clasificar el estado de un estudiante como “avance regular”, “riesgo académico”, “baja participación” o “abandono probable”. En todos estos casos, el modelo debe seleccionar una clase entre varias alternativas.

Podemos representar la idea así:



La estructura sigue siendo supervisada. Para entrenar un modelo multiclase necesitamos ejemplos históricos donde cada caso tenga una etiqueta conocida. El modelo observa las variables de entrada y aprende patrones asociados con cada clase. Luego, ante un caso nuevo, intenta decidir cuál de las categorías disponibles parece más adecuada.

Por ejemplo, imaginemos un sistema que recibe reclamos de usuarios y quiere derivarlos automáticamente al área correspondiente. Cada reclamo puede estar descrito por variables como texto del mensaje, canal de ingreso, tipo de cliente, producto asociado, historial previo o palabras clave detectadas. La variable objetivo podría indicar el área que resolvió el reclamo en casos anteriores:



La variable objetivo podría indicar el área que resolvió el reclamo.

El modelo aprende a partir de reclamos ya clasificados y luego intenta asignar una categoría a reclamos nuevos. Si funciona bien, puede ayudar a ordenar la atención, reducir tiempos de derivación y priorizar el trabajo de cada equipo. Pero, como siempre, su utilidad depende de la calidad de las etiquetas históricas y de que las categorías estén bien definidas.

La clasificación multiclase puede parecer una extensión sencilla de la binaria, pero presenta desafíos propios. Cuando hay más clases, también hay más formas posibles de equivocarse. En un problema binario, el modelo puede confundir una clase con la otra. En un problema multiclase, puede confundir una clase con cualquiera de las demás.

Esto importa porque no todos los errores tienen el mismo significado. Si un reclamo de facturación se deriva a ventas, quizás el error sea evidente y se corrija rápidamente. Pero si un reclamo técnico complejo se deriva a una categoría demasiado general, puede demorarse su resolución. En un sistema de riesgo, confundir “riesgo alto” con “riesgo medio” no tiene las mismas consecuencias que confundir “riesgo alto” con “riesgo bajo”. La distancia conceptual entre clases puede importar, aunque el modelo las trate como categorías.

También puede ocurrir que algunas clases sean muy parecidas entre sí. En atención al cliente, “problema técnico” y “consulta de uso” pueden superponerse. En educación, “baja participación” y “riesgo académico” pueden estar relacionados. En diagnóstico asistido, dos categorías clínicas pueden compartir síntomas o indicadores. Si las fronteras entre clases son ambiguas incluso para personas expertas, el modelo también tendrá dificultades.

Por eso, antes de entrenar un clasificador multiclase, conviene revisar si las categorías tienen sentido. ¿Son mutuamente excluyentes? ¿Cada caso debe

pertenecer a una sola clase? ¿Las etiquetas fueron asignadas con criterios consistentes? ¿Hay clases demasiado generales y otras demasiado específicas? ¿Existen categorías con muy pocos casos? Estas preguntas ayudan a detectar problemas en la formulación del modelo.

En muchos problemas multiclase, cada caso pertenece a una sola clase. Por ejemplo, una solicitud se clasifica en un único nivel de riesgo, o un reclamo se deriva a un área principal. Sin embargo, hay situaciones donde un caso podría pertenecer a varias categorías al mismo tiempo. Una consulta de usuario podría tratar sobre facturación y soporte técnico. Un documento podría estar relacionado con economía y salud. Una noticia podría pertenecer a política internacional y tecnología. Ese tipo de problema se conoce como clasificación multietiqueta, y no es exactamente lo mismo que clasificación multiclase.

En este capítulo nos concentramos en la clasificación multiclase tradicional, donde el modelo debe elegir una categoría entre varias posibles. La distinción es importante porque cambia la forma de representar la variable objetivo y de evaluar el modelo.

En código, una variable objetivo multiclase podría verse así:

```
X = df[["monto_solicitado", "ingresos", "historial_pagos", "deuda_actual"]]
y = df["nivel_riesgo"]
```

Si `nivel_riesgo` contiene valores como “bajo”, “medio” y “alto”, estamos frente a un problema multiclase. Aunque esas categorías tengan un orden conceptual, el modelo de clasificación debe aprender a asignar cada caso a una de ellas.

Otro ejemplo podría ser:

```
X = df[["canal", "producto", "antigüedad_cliente", "palabras_clave"]]
y = df["area_reclamo"]
```

Si `area_reclamo` contiene varias áreas posibles, también estamos frente a clasificación multiclase.

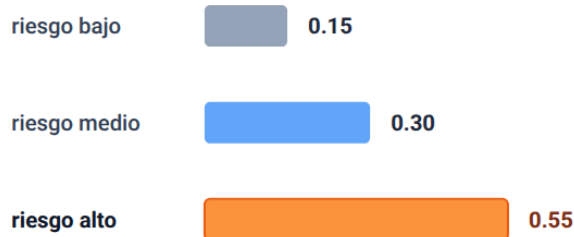
Un aspecto importante es la distribución de las clases. En problemas reales, rara vez todas las categorías aparecen con la misma frecuencia. Puede haber clases muy comunes y clases poco frecuentes. En un sistema de reclamos, “consulta general” puede concentrar muchos casos, mientras que “problema legal” aparece muy pocas veces. En un modelo de riesgo, puede haber muchos casos de riesgo bajo y pocos de riesgo alto. En un sistema educativo, algunos estados pueden ser habituales y otros excepcionales.

Esto afecta tanto el entrenamiento como la evaluación. Si una clase tiene muy pocos ejemplos, el modelo tendrá menos oportunidades de aprender sus patrones. Además, una métrica general puede ocultar malos resultados en clases minoritarias. Un modelo podría funcionar bien para las categorías más frecuentes y muy mal para las menos frecuentes. Si esas clases minoritarias son importantes, no podemos ignorarlas.

La evaluación multiclase requiere mirar el desempeño por clase. No basta con saber cuántas predicciones fueron correctas en total. Debemos observar qué categorías se confunden entre sí, cuáles se predicen bien, cuáles quedan subrepresentadas y qué errores son más graves para el problema. Más adelante veremos herramientas concretas para hacer este análisis, como matrices de confusión y métricas por clase.

También es frecuente que los modelos multiclase produzcan probabilidades o puntuaciones para cada clase. Por ejemplo, ante una solicitud, un modelo podría estimar:

Puntuaciones del modelo



Los modelos multiclase pueden producir probabilidades.

La clase predicha sería “riesgo alto”, porque tiene la probabilidad más alta. Pero las otras probabilidades también aportan información. No es lo mismo que el modelo elija “riesgo alto” con 0.55 que con 0.95. En el primer caso, la decisión parece menos segura. En el segundo, la separación respecto de las demás clases es mayor.

Esta información puede ser útil para decidir acciones. Si el modelo está muy seguro, el caso podría seguir un flujo automático. Si las probabilidades están repartidas entre varias clases, quizás convenga una revisión humana. En problemas sensibles, las probabilidades pueden ser más informativas que una etiqueta final única.

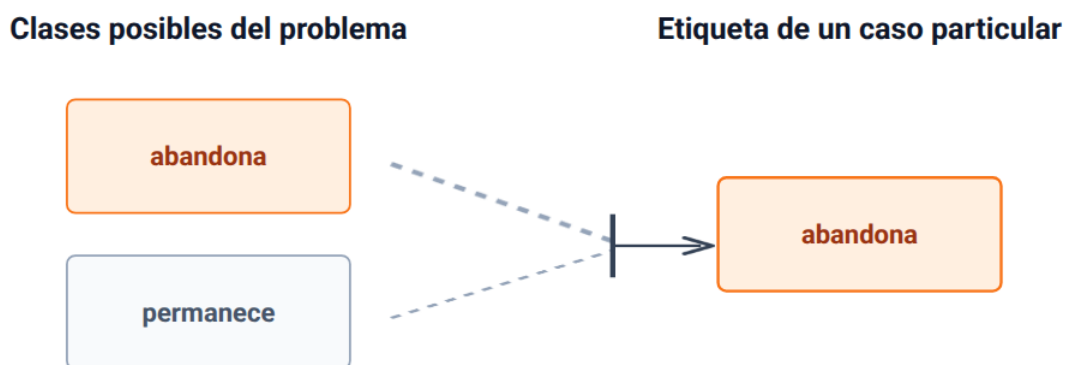
La clasificación multiclase aparece cuando un modelo debe elegir entre tres o más categorías posibles. Es útil para problemas como derivación de reclamos, niveles de riesgo, estados de estudiantes, tipos de solicitudes o categorías de diagnóstico asistido. Su dificultad aumenta porque hay más formas de error, las clases pueden estar desbalanceadas y algunas categorías pueden superponerse conceptualmente. Por eso, un buen modelo multiclase no solo debe acertar muchas veces: también debe mostrar un comportamiento razonable en cada clase y cometer errores aceptables para el contexto en el que será usado.

Clases, etiquetas y probabilidades

En un problema de clasificación, la salida que queremos predecir se expresa mediante **clases** o **etiquetas**. Estos términos suelen usarse de manera muy cercana. Una clase es una categoría posible del problema; una etiqueta es el valor concreto que aparece registrado en los datos para indicar a qué clase pertenece cada caso.

Por ejemplo, en un problema de abandono de clientes, las clases pueden ser “abandona” y “permanece”. En una tabla histórica, cada cliente tendrá una etiqueta que indica cuál fue su resultado. Algunos registros tendrán la etiqueta “abandona” y otros la etiqueta “permanece”. En un problema de riesgo crediticio, las clases podrían ser “bajo”, “medio” y “alto”. Cada solicitud histórica tendrá una etiqueta correspondiente a una de esas clases.

Podemos pensarlo así:



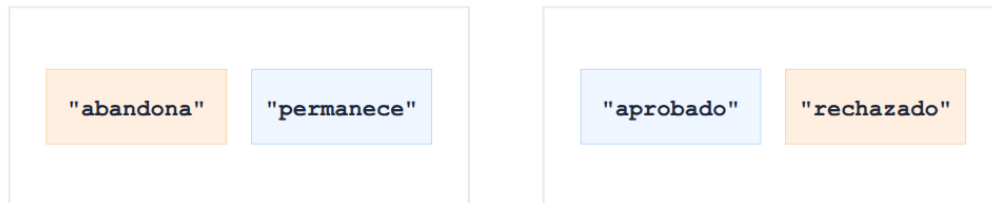
Clases, etiquetas y probabilidades.

La variable objetivo **y** contiene esas etiquetas. En clasificación, **y** no representa una cantidad numérica continua, sino una categoría. A veces esa categoría está escrita como texto. Otras veces está codificada con números.

Por ejemplo:



La categoría puede ser numérica.



La categoría puede ser también un texto.

El hecho de que una etiqueta esté codificada como número no cambia la naturaleza del problema. Si 0 representa “permanece” y 1 representa “abandona”, el modelo no está estimando una cantidad. Está prediciendo una categoría. Los números son códigos convenientes para trabajar con datos, pero su significado sigue siendo categórico.

Esta distinción es importante porque evita confusiones. Si una variable objetivo contiene valores como 0 y 1, no debemos asumir automáticamente que estamos frente a una regresión. Debemos preguntar qué significan esos valores. Si representan clases, estamos trabajando con clasificación.

Las clases deben estar bien definidas. Un modelo aprende a partir de las etiquetas que le damos. Si esas etiquetas son ambiguas, inconsistentes o poco confiables, el modelo aprenderá sobre una base débil. Por ejemplo, si queremos clasificar solicitudes como “riesgo bajo”, “riesgo medio” y “riesgo alto”, necesitamos saber cómo se asignaron esos niveles. ¿Responden a una regla clara? ¿Fueron definidos por especialistas? ¿Se asignaron de manera uniforme? ¿Cambió el criterio con el tiempo?

Lo mismo ocurre en diagnóstico asistido. Una etiqueta como “requiere revisión” debe tener un significado claro. ¿Quién decidió que requería revisión? ¿Con qué criterio? ¿A partir de qué información? ¿La etiqueta fue confirmada luego por un profesional? Si la variable objetivo contiene decisiones humanas previas, el modelo puede aprender tanto los criterios correctos como las inconsistencias o sesgos presentes en esas decisiones.

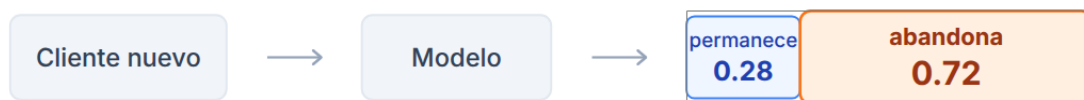
Una vez entrenado, un clasificador puede producir una clase final. Por ejemplo:



Una vez entrenado, un clasificador puede producir una clase final.

Pero muchos modelos de clasificación también pueden producir **probabilidades** o puntuaciones asociadas a cada clase. En lugar de devolver solamente una etiqueta, pueden estimar qué tan probable parece cada clase según los patrones aprendidos.

Por ejemplo, para un cliente nuevo, un modelo podría devolver:

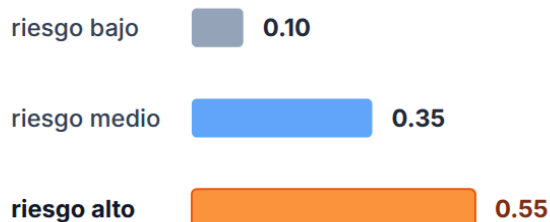


Muchos modelos de clasificación pueden producir probabilidades asociadas a cada clase.

En ese caso, la clase más probable es “abandona”. Si el sistema necesita una predicción final, probablemente elegiría esa clase. Pero la probabilidad también aporta información. No es lo mismo clasificar a un cliente como “abandona” con una probabilidad estimada de 0.72 que con una probabilidad de 0.51. En ambos casos la clase más probable puede ser la misma, pero la confianza de la predicción no parece igual.

En clasificación multiclase, la idea es similar. Un modelo podría estimar probabilidades para varias clases:

Probabilidades estimadas



Un modelo puede estimar probabilidades para varias clases.

La clase predicha sería “riesgo alto”, porque tiene la probabilidad más alta. Sin embargo, el modelo no está completamente separado de las otras opciones: “riesgo medio” también tiene una probabilidad considerable. Esto podría indicar que el caso merece una revisión más cuidadosa.

Las probabilidades permiten pensar la clasificación de una manera menos rígida. Una etiqueta final resume la decisión, pero las probabilidades muestran matices. En muchos contextos, esos matices son importantes. Un sistema puede tratar de manera distinta los casos donde el modelo está muy seguro y aquellos donde las probabilidades están repartidas.

Por ejemplo, una empresa que quiere reducir abandono de clientes podría usar las probabilidades para priorizar acciones. No necesariamente tratará igual a todos los clientes clasificados como “abandona”. Podría ordenar los casos según la probabilidad estimada de abandono y contactar primero a los de mayor riesgo. También podría decidir intervenir solo cuando la probabilidad supere cierto valor, para no gastar recursos en casos poco claros.

En aprobación de créditos, una probabilidad estimada de incumplimiento puede funcionar como una señal de riesgo. Pero esa señal no debería confundirse con una decisión automática sin contexto. La institución puede usarla junto con reglas, revisión humana, políticas internas y criterios legales o éticos. El modelo aporta una estimación; la decisión final puede depender de muchos elementos adicionales.

En diagnóstico asistido, las probabilidades pueden ayudar a priorizar casos para revisión profesional. Un modelo puede señalar que cierto caso tiene alta probabilidad de requerir atención, pero eso no equivale por sí solo a un diagnóstico definitivo. En áreas sensibles, una probabilidad debe interpretarse como una ayuda para decidir qué revisar, no como una verdad absoluta.

También debemos ser prudentes con la palabra “probabilidad”. Cuando un modelo devuelve 0.80 para una clase, no significa necesariamente que exista una certeza matemática perfecta de que el evento ocurrirá en el 80% de los casos exactamente iguales. Esa cifra es una estimación producida por el modelo a partir de los datos y del algoritmo utilizado. Algunas probabilidades están mejor calibradas que otras. Más adelante podremos estudiar esta idea con mayor profundidad. Por ahora, conviene entender que una probabilidad predicha es una señal útil, pero no una garantía.

Además, para convertir probabilidades en clases finales suele usarse un **umbral de decisión**. En clasificación binaria, un criterio común es clasificar como clase positiva si la probabilidad supera 0.5. Por ejemplo, si la probabilidad de abandono es mayor o igual que 0.5, el modelo predice “abandona”; si es menor, predice “permanece”.

Pero ese umbral no tiene por qué ser siempre 0.5. Si el costo de no detectar un caso importante es muy alto, puede convenir usar un umbral más bajo para marcar más casos como positivos. Esto aumentará la cantidad de casos detectados, pero también puede aumentar las falsas alarmas. Si el costo de una falsa alarma es alto, puede convenir un umbral más exigente. Esta decisión depende del problema y se relaciona con la evaluación de errores, que veremos más adelante.

Esta idea muestra que las clases y las probabilidades cumplen funciones distintas. La clase final sirve para tomar una decisión categórica. La probabilidad ayuda a comprender el grado de confianza o prioridad asociado a esa decisión. En muchos proyectos, quedarse solamente con la clase final puede ocultar información valiosa.

Las clases son las categorías posibles de un problema de clasificación, y las etiquetas son los valores registrados en los datos para indicar la clase de cada caso. Un modelo puede devolver una etiqueta final, pero muchos clasificadores también pueden estimar probabilidades para cada clase. Estas probabilidades permiten interpretar la predicción con más matices, priorizar casos, ajustar umbrales y tomar decisiones más cuidadosas. Sin embargo, deben entenderse como estimaciones basadas en datos y modelos, no como certezas absolutas.

Frontera de decisión

Cuando un modelo de clasificación asigna casos a categorías, necesita algún criterio para decidir dónde termina una clase y dónde empieza otra. A esa separación se la suele llamar **frontera de decisión**.

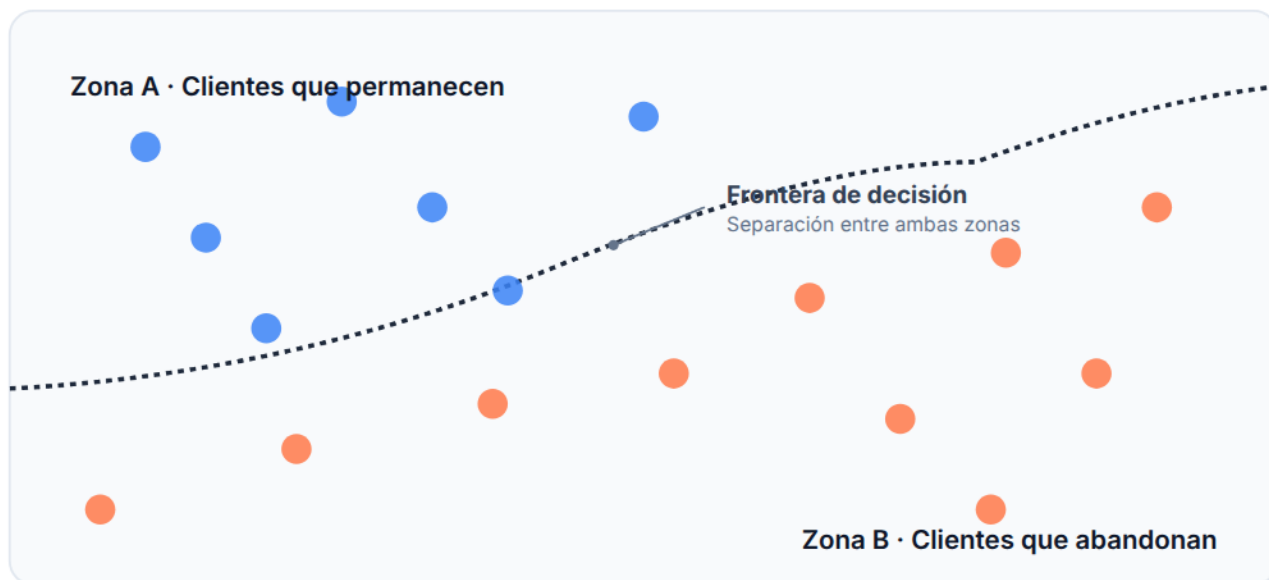
La frontera de decisión es una idea conceptual muy importante en clasificación. No siempre la veremos dibujada, y en muchos problemas reales puede ser demasiado compleja para visualizarla directamente. Pero nos ayuda a entender qué hace un modelo cuando clasifica: divide el espacio de los datos en regiones, y cada región queda asociada con una clase.

Imaginemos un problema muy simple con solo dos variables de entrada. Por ejemplo, queremos clasificar clientes según dos características: cantidad de reclamos recientes y días desde la última actividad. La variable objetivo indica si el cliente abandonó o permaneció. Cada cliente puede representarse como un punto en un plano: una coordenada para los reclamos y otra para los días de inactividad.

En ese plano, quizás observemos que muchos clientes que permanecen se ubican en una zona: pocos reclamos y pocos días sin actividad. En cambio, muchos clientes que abandonan aparecen en otra zona: más reclamos y más días sin actividad. Si

entrenamos un modelo de clasificación, este intentará aprender una separación entre ambas clases.

Podemos imaginarlo así:



La frontera de decisión es la separación entre ambas zonas.

La frontera de decisión es, entonces, la línea o superficie que marca el límite entre las regiones donde el modelo predice una clase u otra. Si un nuevo cliente cae de un lado de esa frontera, el modelo predice “permanece”. Si cae del otro lado, predice “abandona”.

En un ejemplo con dos variables, la frontera podría dibujarse como una línea recta, una curva o una forma más irregular. En problemas con tres variables, podríamos imaginar una superficie en tres dimensiones. En problemas reales, donde puede haber decenas o cientos de variables, ya no podemos visualizar fácilmente esa frontera, pero la idea sigue siendo la misma: el modelo construye una regla que separa regiones del espacio de entrada.

Esta idea se vuelve más clara con un ejemplo sencillo. Supongamos que un modelo de aprobación de créditos usa dos variables: relación entre cuota e ingreso, y cantidad de atrasos previos. En los datos históricos, las solicitudes con baja relación cuota-ingreso y pocos atrasos suelen haber sido aprobadas o cumplidas correctamente. Las solicitudes con alta relación cuota-ingreso y varios atrasos previos suelen estar asociadas con mayor riesgo.

Un clasificador podría aprender una frontera que separe una región de menor riesgo de una región de mayor riesgo. Una nueva solicitud se ubicará en algún punto de ese espacio. Según de qué lado de la frontera quede, el modelo la clasificará como “riesgo bajo” o “riesgo alto”, o como “aprobable” o “requiere revisión”.

Es importante notar que la frontera de decisión no necesariamente coincide con una regla humana simple. A veces podría parecerse a una regla clara, como “si los días sin actividad superan cierto valor, marcar riesgo”. Pero muchas veces depende de combinaciones de variables. Un cliente con muchos días sin actividad quizás no sea considerado de alto riesgo si tiene una larga antigüedad y no presentó reclamos. Otro cliente con menos días de inactividad podría ser marcado como riesgoso si además redujo mucho su uso y realizó varios reclamos recientes.

El modelo no siempre decide por una única variable. Puede construir una frontera basada en la interacción entre varias señales.

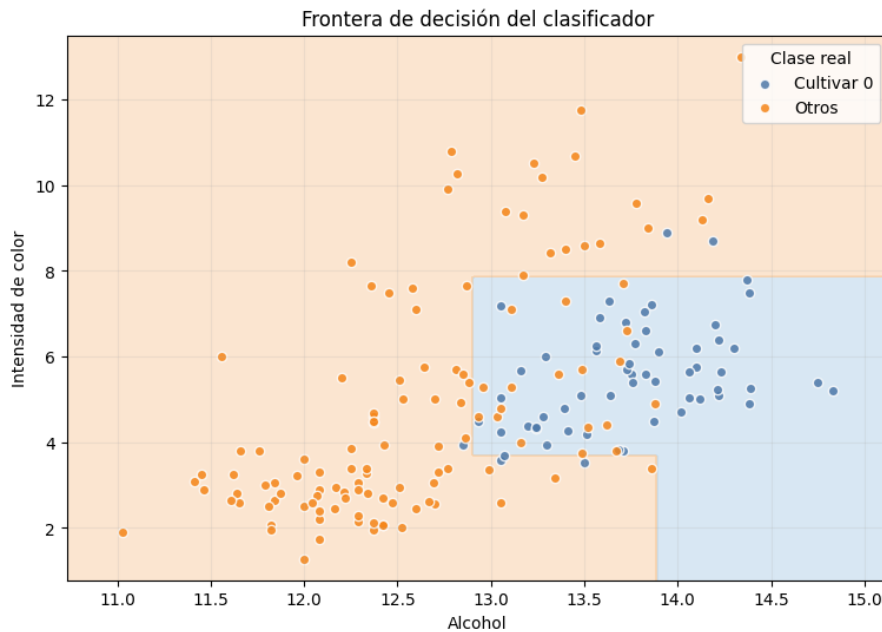
Distintos modelos pueden construir fronteras de decisión distintas. Algunos modelos tienden a producir fronteras simples. Por ejemplo, ciertos modelos lineales separan las clases mediante líneas, planos o hiperplanos. Otros modelos pueden generar fronteras más flexibles, capaces de adaptarse a formas complejas en los datos. Más adelante, cuando estudiemos K-Nearest Neighbors, regresión logística y árboles de decisión, veremos que cada uno tiene una manera diferente de separar clases.

Esta diferencia es importante porque una frontera demasiado simple puede no capturar bien la estructura real del problema. Si las clases están mezcladas de forma compleja, una línea recta puede ser insuficiente. Pero una frontera demasiado flexible también puede ser peligrosa: puede adaptarse demasiado a los casos particulares del entrenamiento y fallar con datos nuevos.

Imaginemos un conjunto de puntos donde las clases están parcialmente mezcladas. Un modelo muy flexible podría trazar una frontera muy irregular para clasificar correctamente casi todos los puntos de entrenamiento. Sin embargo, esa frontera podría estar siguiendo ruido, errores o particularidades accidentales de esos datos. Cuando aparezcan nuevos casos, esa separación tan ajustada puede no funcionar bien. Esta idea se relaciona con el sobreajuste, que estudiaremos con más detalle más adelante.

Por el contrario, una frontera demasiado rígida puede cometer muchos errores porque no logra adaptarse a patrones reales. El desafío consiste en encontrar una separación que capture regularidades útiles sin memorizar detalles irrelevantes. En otras palabras, la frontera de decisión debe ayudar al modelo a generalizar.

También debemos recordar que la frontera de decisión no siempre separa clases de manera perfecta. En muchos problemas reales, las clases se superponen. Clientes que abandonan y clientes que permanecen pueden tener comportamientos parecidos. Solicitudes de crédito que cumplen y solicitudes que incumplen pueden compartir características. Casos clínicos de distinta categoría pueden presentar síntomas similares. Cuando las clases se mezclan, ningún límite podrá separarlas sin errores.



El límite entre regiones es la frontera de decisión.

Esto es una idea fundamental. La clasificación no siempre consiste en encontrar una línea perfecta que divide dos mundos claramente separados. Muchas veces consiste en tomar decisiones bajo incertidumbre, usando señales que aumentan o disminuyen la probabilidad de pertenecer a una clase. La frontera de decisión marca el criterio que el modelo usa para convertir esas señales en una clasificación final.

La relación entre probabilidades y frontera de decisión es especialmente importante. En clasificación binaria, un modelo puede estimar la probabilidad de pertenecer a la clase positiva. Luego, se elige un umbral. Si la probabilidad supera ese umbral, el caso se clasifica como positivo; si no, como negativo. La frontera de decisión aparece justamente en el lugar donde el modelo cambia de una decisión a otra.

Si usamos un umbral de 0.5, la frontera separa los casos cuya probabilidad estimada está por debajo o por encima de ese valor. Si cambiamos el umbral, la frontera se desplaza. Esto significa que la frontera no depende solo del modelo, sino también de la regla de decisión que elegimos.

Por ejemplo, en un sistema de diagnóstico asistido, podríamos querer detectar la mayor cantidad posible de casos que requieren revisión. Para eso, quizás usemos un umbral más bajo. El modelo marcará más casos como positivos, y la frontera se moverá para incluir más observaciones dentro de la región de alerta. Esto puede reducir la cantidad de casos importantes no detectados, pero también aumentar las falsas alarmas.

En cambio, si el costo de una falsa alarma es muy alto, podríamos usar un umbral más exigente. La frontera se moverá en sentido contrario: menos casos serán

clasificados como positivos, pero aquellos que lo sean tendrán una probabilidad estimada más alta. La decisión adecuada depende del contexto.

En clasificación multiclase, la frontera de decisión puede pensarse como la separación entre varias regiones. Cada región corresponde a una clase. Si el modelo clasifica solicitudes en riesgo bajo, medio y alto, habrá zonas del espacio donde predice cada una de esas categorías. Los límites entre esas zonas son las fronteras de decisión entre clases.

En problemas reales, estas fronteras pueden ser difíciles de interpretar. Por eso, cuando trabajamos con modelos, no siempre intentamos dibujar la frontera completa. A veces analizamos ejemplos, probabilidades, errores, métricas o importancia de variables. Pero como idea conceptual, la frontera de decisión nos ayuda a entender que clasificar implica dividir el espacio de posibles casos según el criterio aprendido por el modelo.

La frontera de decisión es el límite que usa un modelo para separar regiones asociadas con distintas clases. En problemas simples puede visualizarse como una línea o una curva; en problemas reales suele existir en un espacio de muchas variables. Distintos modelos construyen fronteras diferentes: algunas más simples, otras más flexibles. Una buena frontera no es necesariamente la que clasifica perfectamente los datos de entrenamiento, sino la que representa patrones útiles y permite generalizar a casos nuevos.

Ejemplos reales de clasificación

Antes de cerrar este primer capítulo sobre clasificación, conviene mirar algunos ejemplos reales con un poco más de detalle. No se trata todavía de construir modelos concretos, sino de reconocer cómo se formula un problema de clasificación en distintos contextos: qué representa cada caso, qué variables podrían formar parte de X , cuál sería la variable objetivo y y qué cuidados aparecen al interpretar las predicciones.

Un primer ejemplo es el **abandono de clientes**, también conocido como *churn*. Muchas empresas necesitan anticipar qué clientes tienen mayor probabilidad de dejar de usar un servicio. Esto puede ocurrir en plataformas digitales, servicios de suscripción, telefonía, bancos, seguros, aplicaciones educativas o comercios con programas de fidelización.

En este tipo de problema, cada fila del dataset podría representar un cliente en un momento determinado. Las variables de entrada podrían incluir antigüedad como cliente, frecuencia de uso, cantidad de compras recientes, días desde la última

actividad, cambios en el consumo, cantidad de reclamos, tipo de plan contratado, interacción con promociones o historial de pagos. La variable objetivo podría indicar si el cliente abandonó o no durante un período posterior.

La formulación podría pensarse así:



Este es un problema de clasificación binaria.

Este es un problema de clasificación binaria, porque la salida tiene dos clases posibles. Pero detrás de esa aparente simplicidad hay varias decisiones importantes. Primero, debemos definir qué significa “abandonar”. En algunos servicios, abandonar puede significar cancelar formalmente una cuenta. En otros, puede significar no comprar durante cierta cantidad de meses, no ingresar a una plataforma durante un período prolongado o dejar de pagar una suscripción.

Esa definición no es un detalle menor. Si definimos abandono como “no comprar durante 30 días”, el modelo aprenderá un tipo de patrón. Si lo definimos como “cancelar la cuenta”, aprenderá otro. La variable objetivo debe representar con claridad el fenómeno que queremos anticipar.

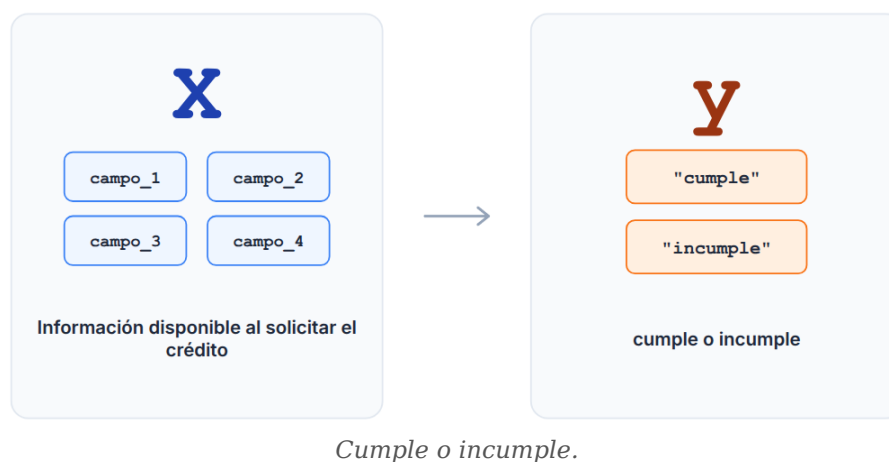
También importa el momento de la predicción. Si queremos detectar riesgo de abandono a tiempo para intervenir, las variables predictoras deben estar disponibles antes de que el abandono ocurra. No tendría sentido usar como entrada una columna que indique la fecha de baja, el motivo de cancelación o el estado final del cliente. Esas variables pueden existir en la base histórica, pero no estarán disponibles cuando queramos anticipar el riesgo en clientes actuales.

El resultado de un modelo de abandono puede usarse de distintas maneras. Una empresa podría contactar a clientes con mayor probabilidad de abandono, ofrecer asistencia, revisar problemas frecuentes o priorizar acciones de retención. Pero el modelo no resuelve por sí solo el abandono. Solo señala casos que, según los datos, parecen tener mayor riesgo. La decisión sobre qué hacer con esa señal sigue siendo humana, comercial y estratégica.

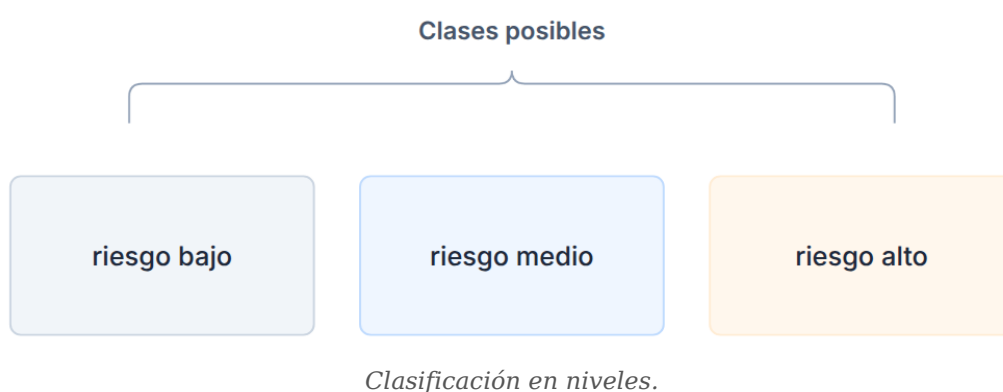
Otro ejemplo importante es la **aprobación de créditos** o, más precisamente, la estimación de riesgo en solicitudes crediticias. Una institución financiera puede recibir solicitudes de préstamos y necesitar evaluar si existe un riesgo elevado de incumplimiento. En un enfoque de clasificación, cada fila podría representar una solicitud o una persona solicitante, y la variable objetivo podría indicar si un crédito histórico fue pagado correctamente o si presentó incumplimiento.

Las variables de entrada podrían incluir ingresos, monto solicitado, relación entre cuota e ingreso, historial de pagos, deudas existentes, antigüedad laboral, edad del crédito, tipo de empleo, cantidad de productos financieros previos u otros indicadores disponibles al momento de la solicitud.

La estructura podría representarse así:



También podría formularse como una clasificación en niveles:



En ese caso, estaríamos frente a una clasificación multiclase. El modelo no elegiría solamente entre dos opciones, sino entre varias categorías de riesgo.

Este ejemplo muestra con claridad que una predicción de clasificación puede tener consecuencias importantes. Un modelo que estima riesgo crediticio puede influir en decisiones que afectan el acceso de una persona a un préstamo, las condiciones

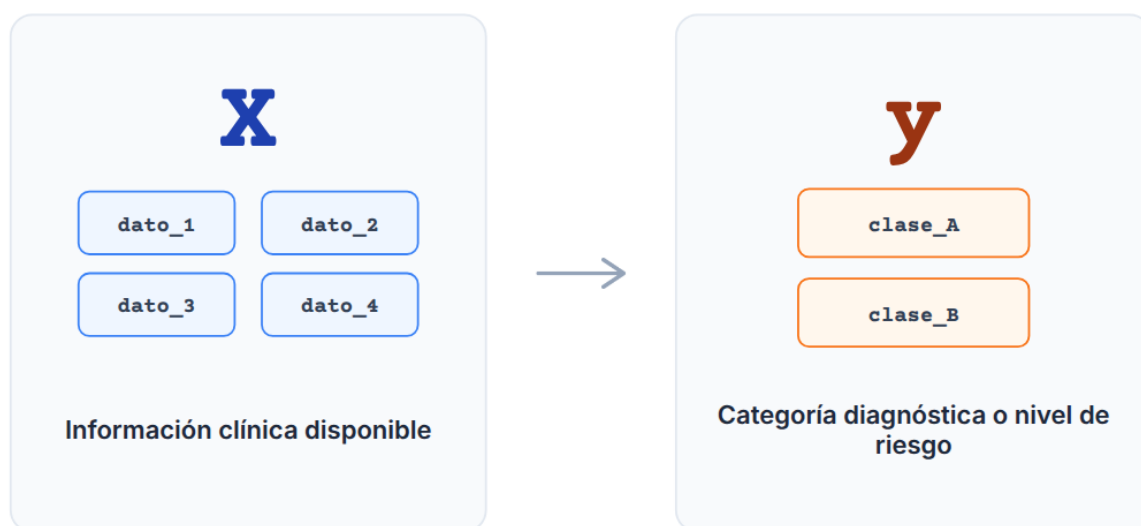
ofrecidas o la necesidad de revisión adicional. Por eso, no alcanza con mirar una métrica general. Es necesario analizar qué errores comete el modelo, si los datos históricos son representativos, si las variables utilizadas son pertinentes y si existen sesgos en las decisiones pasadas que el modelo podría aprender.

Además, es fundamental distinguir entre predicción y decisión. Un modelo puede estimar la probabilidad de incumplimiento o asignar una categoría de riesgo, pero la decisión final puede depender de políticas institucionales, regulaciones, revisión humana y criterios éticos. El modelo aporta una señal, no debería convertirse automáticamente en una autoridad incuestionable.

Un tercer ejemplo aparece en el **diagnóstico asistido**. En este tipo de aplicación, un modelo puede ayudar a clasificar casos clínicos, imágenes, resultados de laboratorio o registros de pacientes. La salida puede ser una categoría como “requiere revisión prioritaria” o “no requiere revisión prioritaria”, “posible presencia de cierta condición” o “sin indicios suficientes”, “riesgo bajo”, “riesgo medio” o “riesgo alto”.

Cada fila del dataset podría representar un paciente, una consulta, una imagen médica, un estudio o un episodio clínico. Las variables de entrada podrían incluir mediciones, antecedentes, síntomas registrados, resultados de análisis, edad, evolución temporal o características extraídas de imágenes. La variable objetivo podría provenir de diagnósticos confirmados, revisiones profesionales o resultados posteriores.

La formulación podría ser:



Clasificación aplicada al diagnóstico.

Este ejemplo debe tratarse con especial prudencia. Un modelo de clasificación no reemplaza el criterio profesional. Puede ayudar a priorizar casos, señalar patrones difíciles de revisar manualmente o funcionar como una segunda lectura, pero sus

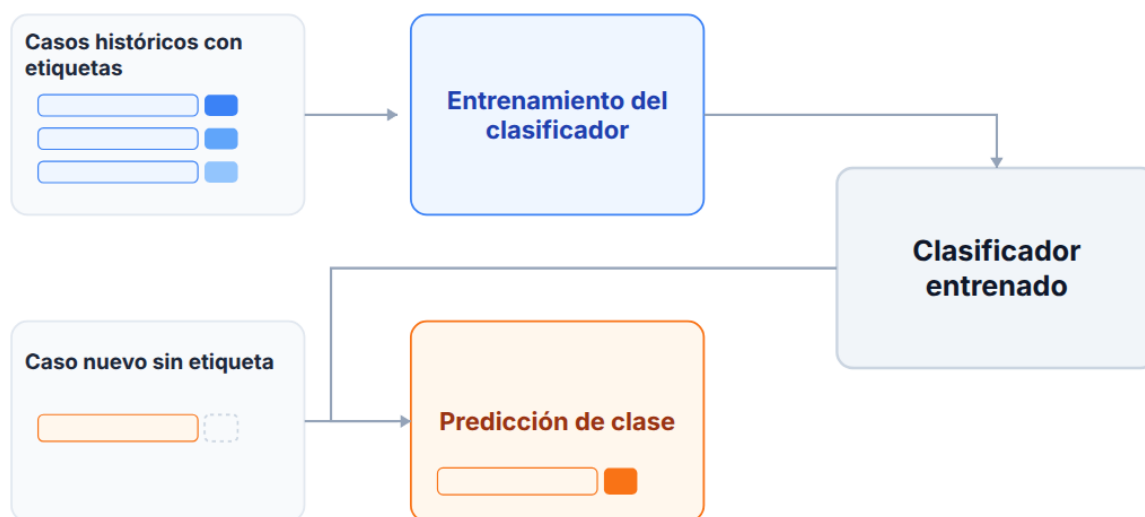
predicciones deben interpretarse dentro de un sistema de atención y supervisión. El costo de los errores puede ser alto, y no todos los errores tienen la misma gravedad.

Por ejemplo, clasificar como “no requiere revisión” un caso que sí necesitaba atención puede tener consecuencias mucho más graves que marcar para revisión un caso que finalmente no presenta un problema. Esto influye en la forma de evaluar el modelo y en el umbral de decisión que se elige. En contextos sensibles, muchas veces se prefiere aumentar la detección de posibles casos importantes, aunque eso genere más revisiones innecesarias. Pero esa decisión no es puramente técnica: depende del contexto clínico, de los recursos disponibles y de criterios profesionales.

También debemos prestar atención a la calidad de las etiquetas. Si la variable objetivo se construyó a partir de diagnósticos incompletos, criterios cambiantes o registros administrativos poco precisos, el modelo aprenderá de esa información. En problemas médicos, la procedencia y confiabilidad de las etiquetas es tan importante como el algoritmo utilizado.

Estos ejemplos muestran que la clasificación puede adoptar formas muy distintas, pero mantiene una estructura común. En todos los casos hay datos de entrada que describen un caso, una variable objetivo que indica la clase conocida en ejemplos históricos y un modelo que intenta aprender patrones para clasificar casos nuevos.

La estructura general puede resumirse así:



Estructura del proceso de clasificación.

También comparten una advertencia: clasificar no significa conocer la verdad absoluta sobre un caso. Significa asignar una categoría según los datos disponibles y los patrones aprendidos. Esa categoría puede ser útil para priorizar, decidir, revisar o automatizar parte de un proceso, pero siempre debe interpretarse considerando el problema real.

En abandono de clientes, una predicción puede orientar acciones de retención. En riesgo crediticio, puede apoyar una evaluación más informada. En diagnóstico asistido, puede ayudar a priorizar revisiones. Pero en todos los casos debemos mirar de dónde vienen los datos, cómo se definieron las clases, qué errores puede cometer el modelo y qué consecuencias tiene usar sus predicciones.

Los problemas de clasificación aparecen cuando queremos predecir categorías. Pueden ser binarios o multiclase, pueden devolver etiquetas finales o probabilidades, y pueden aplicarse en contextos muy variados. Lo esencial es formular bien la tarea: definir qué representa cada caso, cuáles son las clases posibles, qué información estará disponible antes de predecir y cómo se usarán las predicciones. Esa formulación será la base para entrenar y evaluar clasificadores en los próximos capítulos.

Clasificar es decidir entre categorías

A lo largo de este capítulo nos concentramos en una de las tareas más importantes del aprendizaje supervisado: la clasificación. Clasificar significa tomar un caso, observar sus variables de entrada y asignarlo a una categoría posible. Esa categoría puede representar una decisión, un estado, un nivel de riesgo, una alerta o una etiqueta descriptiva.

A diferencia de la regresión, donde el objetivo es estimar una cantidad numérica, en clasificación el modelo debe elegir entre clases. Si hay solo dos clases, hablamos de clasificación binaria. Si hay tres o más, hablamos de clasificación multiclase. En ambos casos, la estructura general se mantiene: el modelo aprende a partir de ejemplos históricos con etiquetas conocidas y luego usa lo aprendido para clasificar casos nuevos.

También vimos que las etiquetas no son simples nombres. Definen qué está aprendiendo el modelo. Si las clases están mal definidas, si se asignaron con criterios inconsistentes o si no representan bien el problema real, el modelo aprenderá sobre una base débil. Por eso, antes de entrenar un clasificador, debemos comprender qué significa cada clase, cómo se construyó la variable objetivo y qué consecuencias tiene cada tipo de error.

Una idea importante del capítulo fue que muchos clasificadores no trabajan solamente con etiquetas finales. También pueden producir probabilidades o puntuaciones asociadas a cada clase. Esa información permite interpretar la predicción con más matices. No es lo mismo clasificar un caso como riesgoso con una probabilidad apenas superior al umbral que hacerlo con una probabilidad muy alta.

En muchos contextos, las probabilidades ayudan a priorizar casos, ajustar decisiones y definir cuándo conviene una revisión humana.

La frontera de decisión nos permitió pensar cómo un modelo separa las clases. En problemas simples puede imaginarse como una línea o una curva. En problemas reales, donde hay muchas variables, esa frontera puede ser imposible de dibujar, pero sigue existiendo como criterio aprendido por el modelo. De un lado de la frontera, el modelo predice una clase; del otro lado, predice otra. En clasificación multiclase, el espacio se divide en varias regiones asociadas con diferentes categorías.

Pero una frontera de decisión no es una verdad absoluta sobre el mundo. Es una forma aprendida de separar casos según los datos disponibles. Si las clases se superponen, si las variables no contienen suficiente información o si los datos históricos tienen errores, el modelo cometerá errores. Por eso, clasificar siempre implica trabajar con incertidumbre.

Los ejemplos reales mostraron que la clasificación puede aplicarse a problemas muy distintos: abandono de clientes, riesgo crediticio, derivación de casos, diagnóstico asistido y muchos otros. En todos ellos, el modelo puede aportar valor si ayuda a ordenar, priorizar o anticipar decisiones. Pero también en todos ellos es necesario interpretar sus resultados con cuidado, especialmente cuando las predicciones afectan a personas o decisiones sensibles.

Clasificar es decidir entre categorías a partir de patrones aprendidos en datos históricos. Esa decisión puede ser binaria o multiclase, puede expresarse como una etiqueta final o apoyarse en probabilidades, y depende de una frontera de decisión construida por el modelo. En el próximo capítulo estudiaremos nuestro primer modelo concreto de clasificación: K-Nearest Neighbors, una técnica que clasifica casos nuevos observando qué ejemplos conocidos se encuentran más cerca.

Capítulo 8 · K-Nearest Neighbors

La idea de aprender por cercanía

K-Nearest Neighbors, o **KNN**, es uno de los modelos más intuitivos para comenzar a estudiar clasificación. Su nombre puede traducirse como “k vecinos más cercanos”, y esa traducción ya adelanta la idea principal: para clasificar un caso nuevo, el modelo observa cuáles son los casos conocidos más parecidos o más cercanos, y usa sus clases para decidir.

A diferencia de otros modelos que construyen una regla interna más abstracta durante el entrenamiento, KNN se basa en una idea muy directa: los casos parecidos suelen tener respuestas parecidas. Si un nuevo caso se parece mucho a varios casos históricos de una clase, entonces es razonable pensar que también podría pertenecer a esa clase.

Imaginemos un conjunto de datos de clientes. Cada cliente está descrito por variables como frecuencia de compra, monto promedio, días desde la última actividad y cantidad de reclamos. Además, en los datos históricos sabemos si cada cliente abandonó o permaneció. Si aparece un cliente nuevo y queremos clasificarlo, KNN buscará en los datos de entrenamiento los clientes más cercanos a ese nuevo caso, según sus variables de entrada. Luego observará sus etiquetas.

Si la mayoría de los vecinos cercanos abandonó, el modelo clasificará al nuevo cliente como “abandona”. Si la mayoría permaneció, lo clasificará como “permanece”.

La lógica puede representarse así:



La mayoría de los vecinos cercanos abandonó.

Esta forma de razonar es bastante cercana a ciertas decisiones humanas cotidianas. Si queremos estimar cómo se comportará una situación nueva, muchas veces buscamos casos parecidos que ya conocemos. Si una solicitud de crédito se parece a

muchas solicitudes anteriores que tuvieron buen comportamiento de pago, podríamos considerarla menos riesgosa. Si se parece a solicitudes que terminaron en incumplimiento, podríamos verla con más cuidado. Si un reclamo se parece a reclamos anteriores que pertenecían al área técnica, probablemente lo derivemos allí.

KNN formaliza esa intuición mediante una regla: mirar los k casos más cercanos. La letra k indica cuántos vecinos se tendrán en cuenta. Si $k = 3$, el modelo mira los tres casos más cercanos. Si $k = 5$, mira los cinco más cercanos. Luego decide según la clase predominante entre esos vecinos.

Por ejemplo, supongamos que queremos clasificar un nuevo caso y usamos $k = 5$. El modelo busca sus cinco vecinos más cercanos en el conjunto de entrenamiento. Las clases de esos vecinos son:



Si $k = 5$, mira los cinco más cercanos.

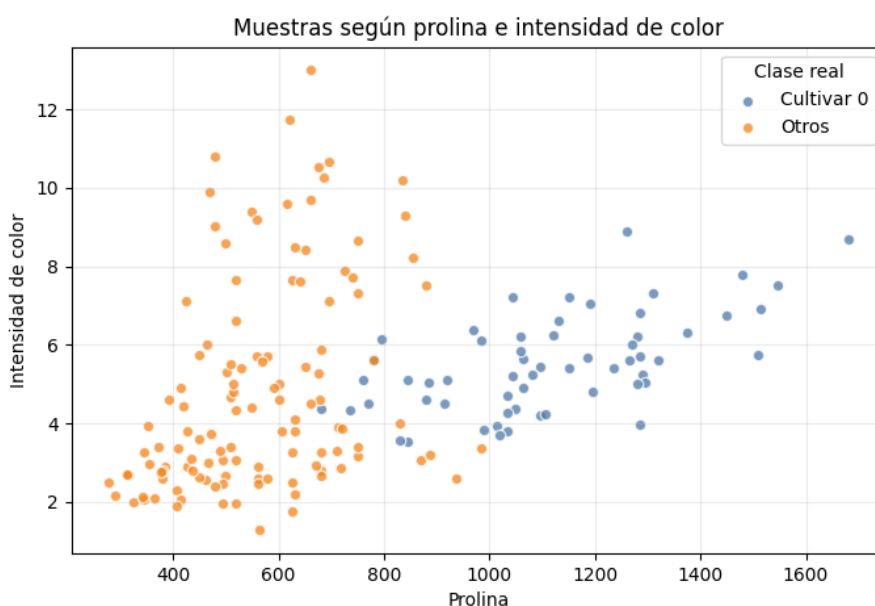
En este grupo hay tres vecinos de la clase "abandona" y dos de la clase "permanece". Como la mayoría corresponde a "abandona", KNN clasifica el caso nuevo como "abandona".

En clasificación multiclase la idea es la misma. Si un reclamo puede pertenecer a "facturación", "soporte técnico", "envíos" o "devoluciones", KNN buscará reclamos similares ya etiquetados y asignará la clase más frecuente entre los vecinos cercanos. No importa si hay dos clases o varias: la decisión se apoya en las etiquetas de los casos cercanos.

Una característica interesante de KNN es que el entrenamiento, en un sentido intuitivo, es muy simple. El modelo necesita conservar los datos de entrenamiento para consultarlos cuando llegue un caso nuevo. No construye una ecuación compleja ni un árbol de decisiones en esta etapa. Su trabajo principal ocurre al momento de predecir: compara el caso nuevo con los casos conocidos, calcula cercanías y decide según los vecinos.

Esto hace que KNN sea fácil de entender, pero también introduce desafíos. Si el dataset es grande, buscar vecinos puede ser costoso. Si las variables están en escalas muy diferentes, la idea de cercanía puede distorsionarse. Si hay muchas variables irrelevantes, los vecinos elegidos pueden no ser realmente parecidos en lo importante. Por eso, aunque KNN sea conceptualmente sencillo, no debe usarse sin preparación.

La palabra “cercanía” es central. Para que KNN funcione, necesitamos una forma de medir qué tan parecido es un caso a otro. En datos numéricos, esto suele hacerse mediante alguna medida de distancia. Si dos casos tienen valores muy similares en sus variables, estarán cerca. Si tienen valores muy diferentes, estarán lejos. En la próxima sección veremos esta idea con más detalle.



Este gráfico muestra regiones donde predominan distintas clases, pero también zonas cercanas o parcialmente mezcladas. KNN clasificará cada caso nuevo consultando las etiquetas de los puntos próximos.

Por ahora, pensemos en un ejemplo simple con solo dos variables: días desde la última actividad y cantidad de reclamos. Cada cliente puede ubicarse como un punto en un plano. Un cliente nuevo también será un punto. KNN buscará los puntos históricos que estén más cerca de ese nuevo punto. Si alrededor del cliente nuevo predominan clientes que abandonaron, el modelo tenderá a predecir abandono. Si predominan clientes que permanecieron, tenderá a predecir permanencia.

Esta idea también ayuda a visualizar la frontera de decisión. En KNN, la frontera entre clases depende de cómo están distribuidos los casos de entrenamiento. Las zonas del espacio donde predominan vecinos de una clase serán clasificadas como esa clase. Las zonas donde predominan vecinos de otra clase recibirán otra

clasificación. Por eso, KNN puede construir fronteras flexibles, adaptadas a la distribución local de los datos.

Pero esa flexibilidad puede ser una ventaja o un problema. Si usamos un valor de k muy pequeño, el modelo puede ser muy sensible a casos individuales, errores o ruido. Si usamos un valor de k muy grande, puede volverse demasiado general y perder detalles locales importantes. Más adelante en este capítulo analizaremos el papel del parámetro k con más cuidado.

KNN también nos permite ver con claridad por qué el preprocesamiento importa. Si una variable tiene valores muy grandes y otra valores pequeños, la distancia puede quedar dominada por la variable de mayor escala. Por ejemplo, si comparamos clientes usando ingresos en pesos y cantidad de reclamos, los ingresos pueden tener valores de cientos de miles, mientras que los reclamos pueden ir de 0 a 10. Sin escalado, la distancia puede estar mucho más influida por ingresos que por reclamos, aunque reclamos sea una variable muy importante para el problema. Por eso, el escalado suele ser especialmente importante para KNN.

KNN clasifica un caso nuevo mirando los casos conocidos más cercanos y observando sus etiquetas. Su intuición principal es que casos parecidos tienden a pertenecer a clases parecidas. Esta simplicidad lo convierte en un excelente primer modelo para estudiar clasificación, porque permite conectar datos, distancia, vecinos, clases y frontera de decisión de una manera muy visual. Sin embargo, esa misma dependencia de la cercanía hace que la elección de variables, el escalado y el valor de k sean decisiones fundamentales.

Distancias entre puntos

Para que KNN pueda decidir cuáles son los vecinos más cercanos de un caso nuevo, necesita una forma de medir cercanía. Esa cercanía se expresa mediante una **distancia**. Cuanto menor sea la distancia entre dos casos, más parecidos se consideran según las variables utilizadas. Cuanto mayor sea la distancia, más diferentes se consideran.

Esta idea es fácil de visualizar cuando trabajamos con dos variables. Imaginemos que queremos clasificar clientes usando solamente dos columnas: cantidad de reclamos recientes y días desde la última actividad. Cada cliente puede representarse como un punto en un plano. Un eje representa los reclamos; el otro, los días de inactividad. Si dos clientes tienen valores similares en ambas variables, sus puntos estarán cerca. Si tienen valores muy distintos, estarán lejos.

Por ejemplo:



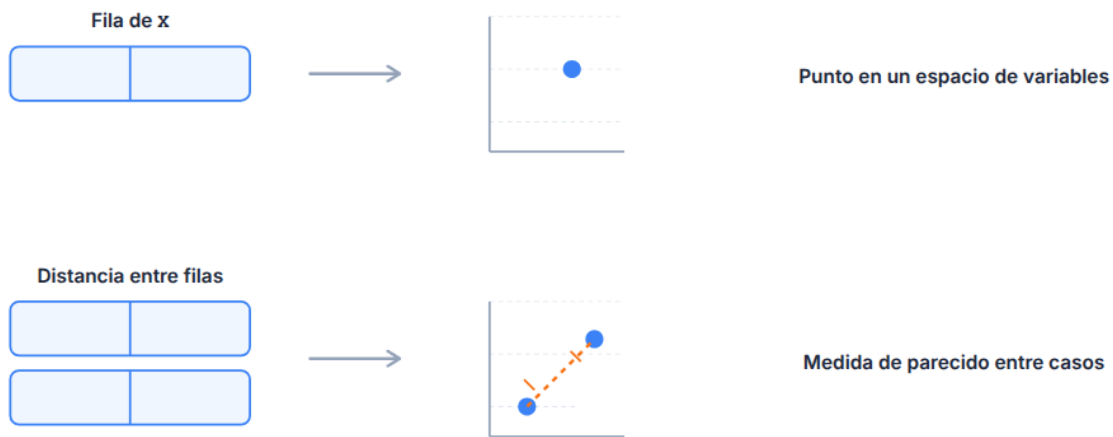
Cada cliente puede representarse como un punto en un plano.

El cliente A se parece más al cliente B que al cliente C. A y B tienen pocos reclamos y pocos días sin actividad. C, en cambio, tiene muchos más reclamos y muchos más días sin actividad. Para KNN, A y B estarían cerca; A y C estarían lejos.

En un plano, podemos imaginar la distancia como la longitud de una línea que une dos puntos. Si los puntos están próximos, la línea es corta. Si están separados, la línea es larga. KNN usa esta idea para ordenar los casos conocidos desde el más cercano hasta el más lejano respecto del caso nuevo.

Cuando hay más de dos variables, la visualización se vuelve más difícil, pero la idea se mantiene. Cada caso ya no se ubica en un plano de dos dimensiones, sino en un espacio con tantas dimensiones como variables usemos. Si usamos cinco variables, cada caso queda representado por cinco valores. Si usamos veinte variables, queda representado por veinte valores. Aunque no podamos dibujar ese espacio fácilmente, el modelo puede calcular distancias dentro de él.

Esto nos lleva a una idea importante: para KNN, cada fila de **X** puede pensarse como un punto. Las columnas de **X** son las coordenadas de ese punto. El modelo compara puntos entre sí para decidir cuáles son vecinos.

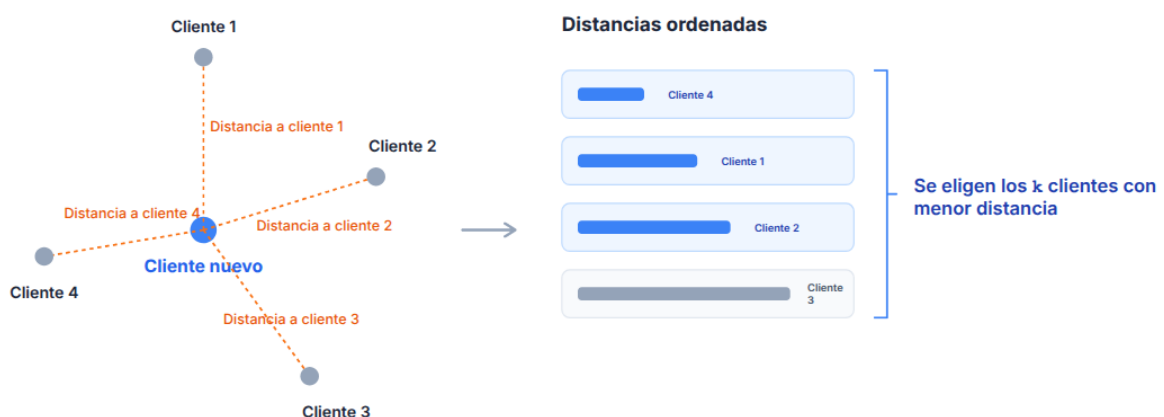


Para KNN, cada fila de X puede pensarse como un punto.

Una de las distancias más conocidas es la **distancia euclidiana**. Es la distancia que solemos imaginar en geometría: la línea recta entre dos puntos. En dos dimensiones, si dos casos difieren poco en ambas variables, su distancia euclidiana será pequeña. Si difieren mucho, será grande.

No necesitamos profundizar ahora en la fórmula, pero sí conviene entender su comportamiento. La distancia euclidiana combina las diferencias entre las variables. Si dos clientes son parecidos en reclamos y también parecidos en días sin actividad, estarán cerca. Si difieren mucho en una o ambas variables, estarán más lejos.

Por ejemplo, si comparamos un cliente nuevo con varios clientes históricos, KNN calcula una distancia entre el nuevo cliente y cada cliente del conjunto de entrenamiento. Luego ordena esas distancias. Los casos con distancias más pequeñas son los vecinos más cercanos.



Se eligen los k clientes con menor distancia.

La distancia no mira la etiqueta directamente. Primero compara las variables de entrada. Después, una vez elegidos los vecinos cercanos, observa sus etiquetas para decidir la clase. Este orden es importante: KNN no busca casos con la misma clase; busca casos cercanos en X y luego usa sus clases para predecir.

La calidad de esa distancia depende mucho de las variables elegidas. Si las variables representan bien el problema, la cercanía puede ser significativa. Si las variables son irrelevantes, ruidosas o están mal preparadas, los vecinos cercanos pueden no ser realmente parecidos en el sentido que nos interesa.

Imaginemos que queremos predecir abandono de clientes, pero usamos variables poco relacionadas con el comportamiento, como un identificador interno o un código administrativo sin significado predictivo. KNN puede calcular distancias de todos modos, pero esas distancias no representarán una cercanía útil. El modelo encontrará vecinos, pero quizás no vecinos relevantes.

También puede ocurrir que algunas variables sean útiles y otras introduzcan ruido. Si agregamos muchas columnas irrelevantes, la distancia puede empezar a reflejar diferencias que no importan para el problema. Dos casos podrían estar lejos porque difieren en variables poco importantes, aunque sean muy parecidos en las señales que realmente anticipan la clase. Por eso, en KNN, elegir buenas variables es especialmente importante.

Otro aspecto fundamental es la escala de las variables. Supongamos que usamos dos columnas: ingresos mensuales y cantidad de reclamos. Los ingresos pueden tener valores como 300000, 600000 o 1200000, mientras que los reclamos pueden ser 0, 1, 2 o 5. Si calculamos distancias directamente, la variable ingresos puede dominar el resultado solo porque sus números son mucho más grandes.

Esto no significa necesariamente que ingresos sea más importante. Significa que está medido en una escala mayor. KNN, al basarse en distancias, puede verse muy afectado por esa diferencia. Por eso, el escalado de variables numéricas suele ser una etapa clave antes de usar KNN. Más adelante en este capítulo volveremos sobre este punto, porque es una de las condiciones prácticas más importantes para que el modelo funcione razonablemente bien.

Las variables categóricas también requieren cuidado. Una distancia entre valores numéricos tiene sentido cuando esos números representan cantidades comparables. Pero si una variable categórica fue codificada de manera arbitraria, como “efectivo = 1”, “tarjeta = 2” y “transferencia = 3”, la distancia puede interpretar diferencias numéricas que no existen realmente. En esos casos, necesitamos transformar las categorías de una manera adecuada antes de usar un modelo basado en distancias.

Esto muestra que la distancia no es una propiedad mágica de los datos. Es una forma de medir parecido según la representación que construimos. Si representamos mal los casos, la distancia medirá mal la cercanía. Si representamos bien los casos, la distancia puede ayudar a encontrar vecinos significativos.

También es importante recordar que estar cerca no garantiza pertenecer a la misma clase. En muchos problemas reales, las clases se mezclan. Puede haber clientes con comportamientos similares donde algunos abandonan y otros permanecen. Puede haber solicitudes de crédito parecidas con resultados distintos. Puede haber casos clínicos con mediciones cercanas pero diagnósticos diferentes. La cercanía aporta una señal, no una certeza.

KNN trabaja con esa señal de manera local. No intenta construir una regla global para todo el conjunto de datos. Para cada caso nuevo, mira su entorno cercano. Si en ese entorno predominan casos de una clase, predice esa clase. Por eso, la forma en que medimos la distancia define qué significa “entorno cercano” para el modelo.

KNN necesita una medida de distancia para identificar los vecinos más cercanos de un caso nuevo. Esa distancia compara los valores de las variables de entrada y permite ordenar los casos conocidos según su cercanía. La calidad de la clasificación depende de que esa distancia represente una semejanza útil para el problema. Por eso, en KNN son especialmente importantes la elección de variables, el tratamiento de categorías y el escalado de variables numéricas.

El parámetro k

El nombre K-Nearest Neighbors incluye una letra fundamental: **k**. Ese parámetro indica cuántos vecinos cercanos se tendrán en cuenta para clasificar un caso nuevo.

Si usamos **k = 1**, el modelo mira solamente el vecino más cercano. La clase de ese vecino se convierte directamente en la predicción. Si el vecino más cercano pertenece a la clase “abandona”, el caso nuevo se clasifica como “abandona”. Si pertenece a “permanece”, se clasifica como “permanece”.

La idea puede representarse así:

k = 1



Si usamos $k = 1$, el modelo mira solamente el vecino más cercano.

Este enfoque es muy sensible a los datos. Puede funcionar bien si los datos están limpios, las clases están bien separadas y el vecino más cercano es realmente representativo. Pero también puede ser frágil. Si el vecino más cercano es un caso atípico, un error de registro o un ejemplo mal etiquetado, el modelo tomará una decisión equivocada.

Por eso, muchas veces se usan valores de k mayores que 1. Si usamos $k = 3$, el modelo mira los tres vecinos más cercanos. Si usamos $k = 5$, mira los cinco más cercanos. La clase final se decide por mayoría.

Por ejemplo, con $k = 5$, podríamos tener estos vecinos:

k = 5



Si usamos $k = 1$, el modelo mira los cinco más cercanos.

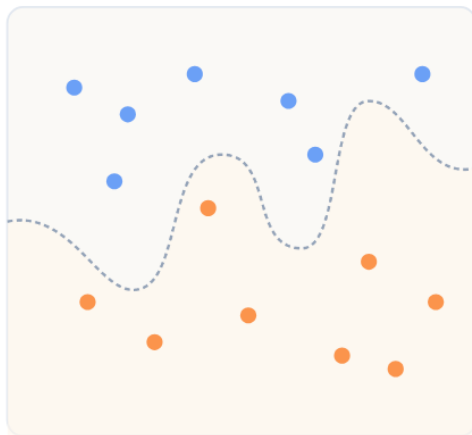
En este caso, la clase más frecuente es “riesgo alto”, porque aparece tres veces. Entonces, el modelo predice “riesgo alto”.

El valor de k controla cuán local o general será la decisión. Con un k pequeño, el modelo se apoya en un entorno muy cercano al caso nuevo. Esto puede permitir capturar detalles locales de los datos, pero también aumenta la sensibilidad al ruido. Con un k grande, el modelo toma una decisión más amplia, basada en una región mayor del espacio. Esto puede hacerlo más estable, pero también puede perder patrones locales importantes.

Podemos pensarlo así:

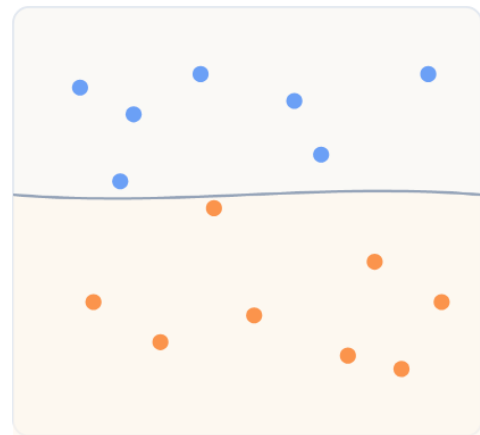
k pequeño

Decisiones más locales, más sensibles al ruido



k grande

Decisiones más generales, más estables, pero menos detalladas



k pequeño vs. k grande.

Imaginemos un conjunto de datos donde la mayoría de los clientes con muchos días sin actividad y varios reclamos abandonan, pero hay algunos casos aislados mal etiquetados como “permanece”. Si usamos $k = 1$, un cliente nuevo que caiga cerca de uno de esos casos aislados podría ser clasificado como “permanece”, aunque su entorno general indique abandono. Si usamos $k = 5$ o $k = 7$, la decisión puede ser menos vulnerable a ese único vecino extraño.

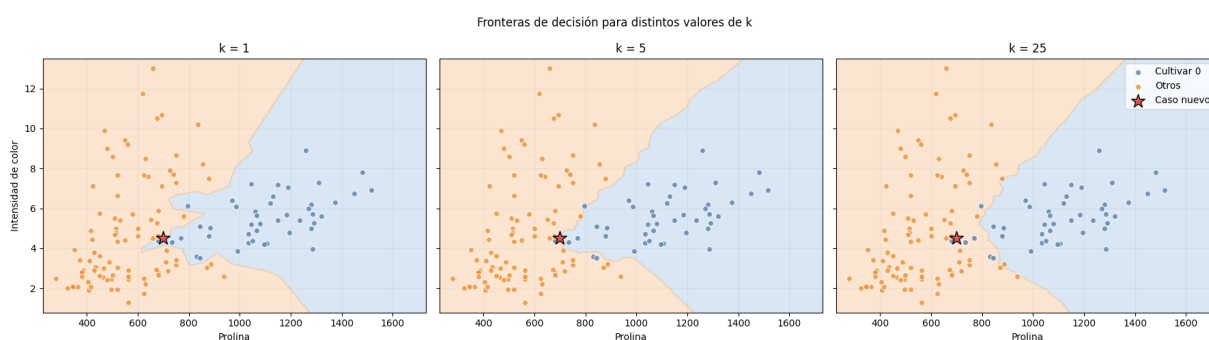
Pero aumentar k demasiado también puede ser un problema. Si el modelo mira demasiados vecinos, puede incluir casos que ya no son realmente parecidos al caso nuevo. La decisión deja de depender del entorno cercano y empieza a reflejar la distribución general del dataset. En un extremo, si k fuera casi igual al número total de casos de entrenamiento, el modelo tendería a predecir la clase mayoritaria con demasiada frecuencia.

Por ejemplo, si en un dataset el 80% de los clientes permanece y el 20% abandona, un k demasiado grande puede favorecer constantemente la clase “permanece”, incluso en zonas donde hay señales locales de abandono. El modelo se vuelve demasiado general y pierde capacidad para detectar regiones específicas.

Esto muestra que no existe un valor de k universalmente correcto. La elección depende del problema, de la cantidad de datos, de la distribución de las clases, del ruido, de la escala de las variables y de la forma en que se distribuyen los casos en el espacio de entrada.

En la práctica, solemos probar varios valores de k y evaluar cuál funciona mejor. Por ejemplo, podemos comparar $k = 1$, $k = 3$, $k = 5$, $k = 7$ y $k = 9$. Pero esa comparación debe hacerse con cuidado, usando datos de validación o estrategias de evaluación adecuadas, para no elegir un valor que funcione bien solo por casualidad en una división particular. Más adelante veremos formas más robustas de comparar configuraciones.

También es común usar valores impares de k en clasificación binaria, como 3, 5, 7 u 11. La razón es simple: si hay dos clases y usamos un número par de vecinos, puede aparecer un empate. Por ejemplo, con $k = 4$, dos vecinos podrían pertenecer a una clase y dos a la otra. Usar valores impares reduce esa posibilidad, aunque no la elimina completamente en problemas multiclase o en situaciones con distancias iguales.



Con $k=1$, la frontera es muy irregular y responde a casos individuales. Con $k=5$, pequeñas irregularidades se suavizan. Con $k=25$, las regiones son más generales y pueden perder detalles locales.

Algunas implementaciones de KNN también permiten ponderar los vecinos por distancia. En lugar de contar todos los vecinos por igual, los vecinos más cercanos pueden tener más peso que los más alejados. La idea es razonable: si dos vecinos están dentro del grupo de los k seleccionados, pero uno está mucho más cerca del caso nuevo que el otro, quizás convenga darle más influencia al más cercano. No necesitamos profundizar todavía en esta variante, pero es útil saber que existe.

El parámetro k también influye en la frontera de decisión. Con valores pequeños, la frontera puede ser muy irregular, porque el modelo responde a detalles locales y casos individuales. Con valores más grandes, la frontera suele volverse más suave, porque la decisión depende de grupos más amplios de vecinos. Esta relación entre k y la frontera de decisión será muy visible en el cuaderno Colab, cuando comparemos distintos valores y observemos cómo cambian las predicciones.

k define cuántos vecinos usa KNN para clasificar un caso nuevo. Un valor pequeño hace que el modelo sea más local y sensible a detalles; un valor grande lo vuelve más estable, pero puede hacerlo demasiado general. Elegir k es una decisión importante porque afecta directamente la frontera de decisión, la sensibilidad al ruido y la capacidad del modelo para generalizar.

Por qué el escalado importa

KNN es un modelo basado en distancias. Esta característica lo hace muy intuitivo, pero también muy sensible a la forma en que están representadas las variables numéricas. Si las variables tienen escalas muy diferentes, la distancia puede quedar dominada por algunas columnas simplemente porque sus valores son más grandes, no porque sean más importantes para el problema.

Recordemos la idea central: para clasificar un caso nuevo, KNN busca los casos de entrenamiento más cercanos. Esa cercanía se calcula comparando los valores de las variables de entrada. Si una variable está medida en una escala mucho mayor que otra, sus diferencias numéricas pueden pesar mucho más dentro del cálculo de distancia.

Imaginemos un conjunto de datos de clientes con dos variables: ingresos mensuales y cantidad de reclamos. Los ingresos pueden tomar valores como 300000, 600000 o 1200000. La cantidad de reclamos, en cambio, puede tomar valores como 0, 1, 2 o 5. Si calculamos distancias usando estas variables tal como están, una diferencia de 100000 pesos en ingresos puede ser enorme comparada con una diferencia de 3 reclamos.

El problema es que esa diferencia de magnitud no significa necesariamente que ingresos sea más importante que reclamos. Simplemente está medido en otra unidad. Los ingresos están en pesos; los reclamos son conteos pequeños. KNN no entiende por sí mismo el significado de esas unidades. Solo ve números y calcula distancias.

Supongamos que tenemos dos clientes cercanos en comportamiento, pero con ingresos distintos. También tenemos otro cliente con ingresos parecidos, pero con una cantidad de reclamos muy diferente. Sin escalado, KNN podría considerar más cercano al cliente con ingresos parecidos, aunque desde el punto de vista del problema la cantidad de reclamos sea una señal más importante. La distancia quedaría dominada por la variable de mayor escala.

Podemos pensarlo así:



El escalado importa.

Si usamos ambas variables sin transformación, las diferencias en ingresos pueden tener un peso mucho mayor en la distancia que las diferencias en reclamos. El modelo puede terminar encontrando vecinos parecidos en ingresos, pero no necesariamente parecidos en comportamiento.

El escalado busca reducir este problema. Al escalar variables numéricas, transformamos sus valores para que queden en una escala más comparable. Una técnica común es la estandarización, que expresa cada valor en relación con la media y la desviación estándar de su columna. Después de estandarizar, una diferencia de una unidad ya no significa “un peso” o “un reclamo”, sino una diferencia medida en términos de variación relativa dentro de esa variable.

Esto permite que las variables participen en el cálculo de distancia de una manera más equilibrada. No significa que todas las variables pasen a ser igualmente importantes desde el punto de vista del problema, pero evita que una columna domine la distancia solo por la magnitud de sus unidades.

En `scikit-learn`, una herramienta habitual para estandarizar variables numéricas es `StandardScaler`. Su uso conceptual sigue la misma lógica que vimos en el capítulo anterior: el escalador debe ajustarse con los datos de entrenamiento y luego aplicarse tanto al entrenamiento como a la prueba.

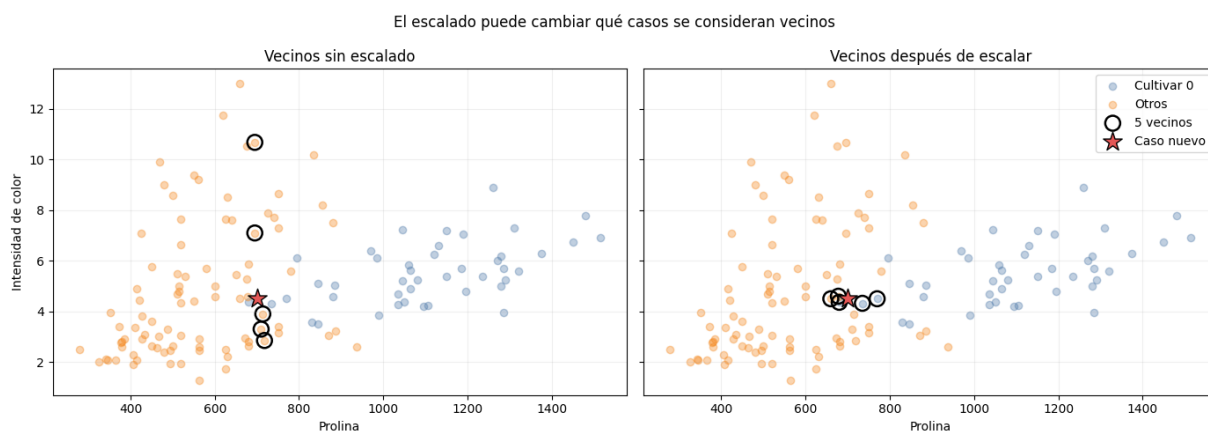
```
scaler.fit(X_train)

X_train_escalado = scaler.transform(X_train)
X_test_escalado = scaler.transform(X_test)
```

En un flujo bien organizado, esta transformación suele formar parte de un `Pipeline`, para asegurar que el escalado se aprenda solamente con `X_train` y se aplique después a los datos de prueba o a nuevos casos.

El orden correcto es importante. No debemos escalar usando todo el dataset antes de separar entrenamiento y prueba, porque eso permitiría que los datos de prueba influyan en el cálculo de medias y desviaciones estándar. Aunque parezca un detalle técnico pequeño, sería una forma de fuga de datos durante el preprocesamiento. La evaluación dejaría de representar completamente el comportamiento del modelo ante datos no vistos.

En KNN, el efecto del escalado puede ser muy notable. A diferencia de modelos menos sensibles a la escala, KNN depende directamente de distancias. Si las distancias cambian, cambian los vecinos elegidos. Si cambian los vecinos, puede cambiar la clase predicha. Por eso, escalar no es un detalle opcional menor cuando trabajamos con KNN; muchas veces es una condición necesaria para que el modelo tenga sentido.



Los puntos del dataset son los mismos en ambos paneles. Lo que cambia son los cinco casos rodeados en negro. El escalado modifica la distancia y, por lo tanto, modifica el vecindario utilizado para votar.

Imaginemos que clasificamos solicitudes de crédito usando ingresos, monto solicitado, cantidad de atrasos previos y antigüedad laboral. Ingresos y monto solicitado pueden tener valores grandes; atrasos previos puede ser un número pequeño; antigüedad laboral puede medirse en años o meses. Sin escalado, las variables monetarias podrían dominar la distancia. Después del escalado, el modelo puede comparar diferencias relativas en todas las variables de manera más equilibrada.

Esto no significa que el escalado solucione todos los problemas. Si una variable es irrelevante, escalarla no la vuelve útil. Si contiene errores, escalarlos no los corrige. Si hay valores extremos muy marcados, el escalado puede verse afectado. Si las variables categóricas fueron codificadas de manera inadecuada, el problema sigue existiendo. El escalado es una transformación importante, pero no reemplaza la exploración ni la selección cuidadosa de variables.

También debemos recordar que no todas las variables deberían escalarse del mismo modo. Las variables categóricas transformadas mediante One-Hot Encoding suelen quedar como columnas binarias, con valores 0 y 1. Dependiendo del flujo y del modelo, puede no ser necesario escalarlas de la misma manera que las variables numéricas continuas. En proyectos reales, por eso usamos herramientas como `ColumnTransformer`, que permiten aplicar escalado a columnas numéricas y One-Hot Encoding a columnas categóricas dentro de un mismo preprocesamiento.

El escalado también afecta la visualización de KNN. Cuando representamos puntos en un plano, cambiar la escala cambia la forma en que medimos cercanía. Dos puntos que parecían cercanos en una escala pueden no serlo después de transformar las variables. En el cuaderno Colab de este capítulo, esta idea se podrá ver con más claridad al comparar el comportamiento del modelo con y sin escalado.

Una buena forma de resumirlo es esta: KNN no sabe qué significa cada unidad. No sabe si una diferencia de 1000 en una variable es pequeña o grande desde el punto de vista del problema. Solo sabe calcular distancias. Por eso, debemos preparar las variables para que esas distancias representen mejor la semejanza que nos interesa.

El escalado importa en KNN porque el modelo clasifica usando distancias entre casos. Si las variables numéricas tienen escalas muy diferentes, las columnas con valores más grandes pueden dominar el cálculo de cercanía. Escalar las variables ayuda a que la distancia sea más equilibrada y a que los vecinos elegidos reflejen mejor la similitud entre casos. En KNN, preparar correctamente la escala de los datos no es un detalle secundario: forma parte central del funcionamiento del modelo.

Ventajas del modelo

KNN tiene varias ventajas que lo convierten en un buen modelo para comenzar a estudiar clasificación. La primera es su intuición. La idea de clasificar un caso nuevo mirando casos parecidos resulta fácil de comprender, incluso antes de entrar en detalles matemáticos. No necesitamos imaginar una ecuación compleja ni una estructura interna difícil de explicar. El modelo se apoya en una regla directa: observar los vecinos cercanos y decidir según sus etiquetas.

Esta simplicidad conceptual es muy valiosa en una primera etapa de aprendizaje. KNN permite conectar de manera clara varias ideas fundamentales de machine learning: variables de entrada, distancia, vecinos, clases, frontera de decisión, escalado y generalización. Cuando usamos KNN, podemos visualizar con bastante

facilidad qué significa que un caso esté cerca de otro y cómo esa cercanía influye en la predicción.

Otra ventaja es que KNN puede adaptarse a fronteras de decisión flexibles. Algunos modelos separan las clases mediante reglas más rígidas, como líneas o planos. KNN, en cambio, decide localmente. Para cada caso nuevo, mira qué ocurre en su entorno cercano. Esto permite que el modelo capture estructuras donde las clases no están separadas por una línea simple.

Imaginemos un conjunto de datos donde los casos de una clase aparecen en varias regiones distintas del espacio de variables. Un modelo demasiado rígido podría tener dificultades para separar esas zonas. KNN puede funcionar mejor en algunos de esos escenarios porque no intenta imponer una única forma global de separación. La clase predicha depende de los vecinos cercanos al punto que queremos clasificar.

Esta capacidad local puede ser útil cuando los datos tienen patrones irregulares. Por ejemplo, en un problema de abandono de clientes, puede haber distintos perfiles de abandono. Algunos clientes abandonan después de muchos reclamos; otros después de una caída brusca en el uso; otros después de un cambio de plan o de una inactividad prolongada. Si esos perfiles forman regiones distintas en los datos, KNN puede aprovechar la cercanía local para reconocerlos.

Otra ventaja es que KNN no requiere una etapa de entrenamiento compleja en el sentido tradicional. En muchos modelos, entrenar implica ajustar parámetros internos a partir de los datos. En KNN, el modelo conserva los casos de entrenamiento y los utiliza cuando necesita predecir. Por eso suele decirse que KNN es un método basado en instancias: las observaciones del entrenamiento siguen siendo protagonistas durante la predicción.

Esto puede hacerlo sencillo de implementar y de explicar. Para clasificar un nuevo caso, el modelo compara ese caso con los ejemplos conocidos, calcula distancias, selecciona los vecinos más cercanos y vota según sus clases. El procedimiento es transparente en su lógica general.

También es un modelo naturalmente aplicable tanto a clasificación binaria como a clasificación multiclase. Si hay dos clases, la mayoría de vecinos decide entre esas dos opciones. Si hay varias clases, la mayoría decide entre varias categorías. La idea no cambia demasiado. Esto permite usar KNN como una herramienta introductoria para distintos tipos de problemas de clasificación.

Además, KNN puede funcionar como una referencia inicial en ciertos proyectos. Aunque no siempre será el modelo más eficiente ni el más preciso, puede servir para tener una primera aproximación. Si con una preparación adecuada de los datos KNN

logra resultados razonables, eso puede indicar que existen patrones locales útiles en las variables. Si funciona muy mal, puede ser una señal para revisar las variables, el escalado, la representación de los datos o la propia formulación del problema.

Otra ventaja importante es que KNN puede producir una idea intuitiva de confianza a partir de la composición de los vecinos. Si usamos $k = 5$ y los cinco vecinos pertenecen a la misma clase, la predicción parece más clara que si tres vecinos pertenecen a una clase y dos a otra. Aunque esta no reemplaza una evaluación formal de probabilidades, ayuda a comprender que no todas las predicciones tienen el mismo grado de claridad local.

Por ejemplo:



KNN puede producir una idea intuitiva de confianza .

En el segundo caso, el modelo predice “abandona”, pero el entorno cercano está más dividido. Este tipo de observación puede ayudar a interpretar predicciones individuales, especialmente en contextos exploratorios o educativos.

KNN también permite visualizar muy bien el efecto de cambiar k . Con valores pequeños, la frontera de decisión suele volverse más irregular y sensible a los casos particulares. Con valores más grandes, la frontera se suaviza. Esta relación entre parámetro, frontera y comportamiento del modelo es muy útil para comprender conceptos que luego aparecerán en otros modelos con nombres diferentes.

Desde el punto de vista pedagógico, KNN tiene una ventaja adicional: obliga a prestar atención al preprocesamiento. Como depende de distancias, deja muy claro por qué el escalado de variables numéricas importa. También muestra por qué las

variables irrelevantes pueden perjudicar un modelo, y por qué una mala codificación de categorías puede distorsionar la idea de cercanía. En este sentido, KNN es un buen modelo para aprender no solo clasificación, sino también la importancia de preparar bien los datos.

Sin embargo, estas ventajas deben entenderse dentro de sus límites. Que KNN sea intuitivo no significa que siempre sea adecuado. Que pueda construir fronteras flexibles no significa que siempre generalice bien. Que sea fácil de explicar no significa que sea eficiente en grandes volúmenes de datos. Sus ventajas son reales, pero dependen del contexto.

KNN es un modelo intuitivo, flexible y útil para aprender clasificación. Su lógica basada en vecinos permite comprender de manera visual cómo un modelo puede usar ejemplos históricos para clasificar casos nuevos. Puede adaptarse a patrones locales, funcionar en clasificación binaria y multiclase, servir como referencia inicial y mostrar con claridad la importancia del escalado y la representación de los datos. Por eso es un excelente primer modelo concreto para estudiar, aunque no siempre sea la mejor opción en proyectos grandes o complejos.

Limitaciones

Aunque KNN es un modelo intuitivo y útil para aprender clasificación, también tiene limitaciones importantes. Comprender esas limitaciones es tan necesario como conocer sus ventajas, porque nos ayuda a decidir cuándo puede ser una buena opción y cuándo conviene buscar otro enfoque.

La primera limitación surge de su propia lógica: KNN necesita comparar cada caso nuevo con los casos de entrenamiento para encontrar los vecinos más cercanos. Esto significa que, al momento de predecir, el modelo debe consultar los datos históricos. Si el conjunto de entrenamiento es pequeño, esto no suele ser un problema. Pero si tenemos cientos de miles o millones de registros, calcular distancias contra muchos casos puede volverse costoso.

En otros modelos, gran parte del trabajo ocurre durante el entrenamiento. Una vez entrenados, pueden predecir con rapidez porque ya aprendieron una estructura interna más compacta. En KNN, en cambio, los datos de entrenamiento siguen siendo necesarios para cada predicción. Por eso, puede ser poco eficiente cuando el volumen de datos es grande o cuando necesitamos respuestas muy rápidas.

Otra limitación es su dependencia de la escala de las variables. Como vimos, KNN usa distancias. Si una variable tiene valores mucho más grandes que otra, puede

dominar el cálculo de cercanía. Esto obliga a prestar especial atención al escalado de variables numéricas. Sin ese paso, el modelo puede elegir vecinos que parecen cercanos desde el punto de vista numérico, pero que no son realmente parecidos para el problema.

Esta dependencia de las distancias también hace que KNN sea sensible a la forma en que representamos las variables. Si incluimos columnas irrelevantes, la distancia puede deteriorarse. Dos casos pueden parecer lejanos porque difieren en variables que no aportan nada al objetivo, o parecer cercanos porque coinciden en aspectos poco importantes. En modelos basados en distancia, agregar más columnas no siempre ayuda; a veces empeora la calidad de la cercanía.

Esto se relaciona con un problema más general: en espacios con muchas variables, la idea de cercanía puede volverse menos útil. Cuando aumenta mucho la cantidad de dimensiones, los puntos tienden a separarse de maneras difíciles de interpretar. Las distancias entre casos pueden volverse menos informativas, especialmente si muchas variables aportan poco o tienen ruido. Este fenómeno suele mencionarse como parte de la “maldición de la dimensionalidad”. No necesitamos profundizar ahora en sus detalles matemáticos, pero sí conviene retener la idea: KNN puede sufrir cuando hay muchas variables y no todas son relevantes.

Otra limitación aparece cuando las clases están muy mezcladas. Si casos de distintas clases aparecen muy próximos entre sí, KNN puede tener dificultades para decidir. Esto ocurre en muchos problemas reales. Clientes que abandonan y clientes que permanecen pueden tener comportamientos similares. Solicitudes que cumplen y solicitudes que incumplen pueden compartir características. Casos clínicos de distintas categorías pueden parecerse en sus variables disponibles. Cuando los datos no separan bien las clases, mirar vecinos cercanos no garantiza una buena predicción.

KNN también puede ser sensible al ruido y a los valores atípicos, especialmente cuando usamos valores pequeños de k . Si $k = 1$, un único vecino decide la clase. Si ese vecino está mal etiquetado, es un caso raro o tiene errores de registro, la predicción será afectada directamente. Aumentar k puede reducir esta sensibilidad, pero también puede hacer que el modelo pierda detalles locales. Por eso, elegir k implica siempre un equilibrio.

Otra dificultad aparece con las clases desbalanceadas. Si una clase es mucho más frecuente que otra, los vecinos cercanos pueden estar dominados por la clase mayoritaria, incluso en zonas donde la clase minoritaria es importante. Por ejemplo, si estamos detectando incumplimiento crediticio y la mayoría de los casos históricos corresponde a créditos cumplidos, KNN puede tender a predecir “cumple” con

demasiada frecuencia. En problemas donde la clase minoritaria es la más importante, esto puede ser una limitación seria.

También debemos considerar el tratamiento de variables categóricas. KNN funciona de manera natural con variables numéricas donde tiene sentido calcular distancias. Pero muchas bases de datos reales combinan variables numéricas y categóricas. Si transformamos categorías de manera inadecuada, podemos introducir distancias artificiales. One-Hot Encoding puede ayudar, pero también puede aumentar mucho la cantidad de columnas, especialmente cuando hay variables con muchas categorías. Esto puede afectar tanto el rendimiento como la calidad de la distancia.

Otra limitación es que KNN no ofrece, por sí mismo, una explicación global simple del problema. Puede explicar una predicción individual mirando los vecinos que la justifican, pero no produce una regla general tan directa como “si ocurre esto, entonces decide aquello”. Tampoco entrega coeficientes interpretables como otros modelos. Para algunas aplicaciones, esta interpretación local puede ser suficiente; para otras, puede ser insuficiente.

Por ejemplo, si una institución necesita justificar de manera clara por qué una solicitud fue considerada riesgosa, decir que “sus vecinos más cercanos tuvieron cierto comportamiento” puede no ser una explicación aceptable. En contextos donde la interpretabilidad global, la auditoría o la trazabilidad son muy importantes, otros modelos pueden resultar más apropiados.

También hay que recordar que KNN depende fuertemente de la calidad del conjunto de entrenamiento. Como conserva los ejemplos y los usa para predecir, los errores, duplicados, inconsistencias o sesgos presentes en esos datos pueden influir directamente en las predicciones. Si los datos históricos no representan bien los casos futuros, los vecinos encontrados pueden no ser una buena guía.

KNN es fácil de entender, pero no está libre de problemas. Puede ser costoso al predecir con datasets grandes, depende mucho del escalado, sufre con variables irrelevantes o demasiadas dimensiones, puede verse afectado por ruido, valores atípicos y clases desbalanceadas, y no siempre ofrece una explicación global clara. Estas limitaciones no lo vuelven inútil; simplemente nos recuerdan que ningún modelo sirve para todo. KNN funciona mejor cuando la idea de cercanía tiene sentido, los datos están bien preparados y el tamaño del problema permite usarlo de manera razonable.

Cuándo puede funcionar bien y cuándo no

KNN puede funcionar bien cuando la idea de cercanía tiene sentido para el problema. Es decir, cuando los casos que son parecidos en sus variables de entrada tienden también a tener respuestas parecidas. Esta condición parece sencilla, pero es el corazón del modelo. Si los vecinos cercanos realmente aportan información sobre la clase de un caso nuevo, KNN puede ser una alternativa razonable.

Por ejemplo, puede funcionar bien en problemas donde los datos forman regiones relativamente coherentes. Si los clientes con patrones similares de uso suelen tener comportamientos similares de permanencia o abandono, KNN puede aprovechar esa estructura. Si las solicitudes de crédito con características parecidas tienden a tener niveles de riesgo parecidos, la cercanía puede ser útil. Si ciertos casos de atención al cliente se agrupan naturalmente según variables que describen el reclamo, el modelo puede clasificar nuevos casos observando ejemplos cercanos.

También puede funcionar bien cuando el dataset no es demasiado grande y las variables están bien preparadas. Como KNN necesita buscar vecinos en el conjunto de entrenamiento, un volumen moderado de datos facilita su uso. Además, si las variables numéricas están escaladas, las categóricas están correctamente transformadas y las columnas irrelevantes fueron descartadas, la distancia tiene más posibilidades de representar una semejanza útil.

KNN suele ser una buena opción exploratoria cuando queremos construir un primer clasificador intuitivo. Nos permite observar si existe cierta estructura local en los datos. Si con un preprocesamiento adecuado el modelo obtiene resultados razonables, eso puede indicar que los casos cercanos comparten información relevante sobre la clase. En ese sentido, KNN puede ayudarnos a aprender sobre el dataset, incluso si luego decidimos usar otro modelo.

También puede ser útil en problemas donde la frontera de decisión no es lineal ni demasiado simple. Como KNN toma decisiones locales, puede adaptarse a formas más irregulares. Si las clases no se separan con una línea recta, pero sí forman zonas locales reconocibles, KNN puede capturar parte de esa estructura. Esto lo diferencia de modelos más rígidos, que pueden tener dificultades cuando la separación entre clases es más compleja.

Sin embargo, KNN puede funcionar mal cuando la cercanía no representa una relación útil. Si los casos cercanos según las variables disponibles no tienden a compartir clase, el modelo no tiene una buena base para decidir. Esto puede ocurrir cuando faltan variables importantes, cuando las variables elegidas son poco informativas o cuando el fenómeno depende de factores que no están registrados en los datos.

Por ejemplo, si queremos predecir abandono de clientes, pero solo tenemos datos administrativos generales y no contamos con información sobre uso reciente, reclamos, satisfacción o cambios de comportamiento, los vecinos cercanos pueden no ser realmente parecidos en lo que importa. KNN calculará distancias de todos modos, pero esas distancias no necesariamente ayudarán a clasificar.

También puede funcionar mal cuando las variables están en escalas muy diferentes y no se aplica escalado. En ese caso, el modelo puede elegir vecinos según la variable de mayor magnitud, aunque esa variable no sea la más relevante. Este problema es especialmente importante en KNN porque la distancia es el mecanismo central del modelo.

Otra situación problemática aparece cuando hay muchas variables, especialmente si varias son irrelevantes o ruidosas. En espacios de muchas dimensiones, la noción de cercanía puede perder claridad. Los casos pueden parecer todos relativamente lejos entre sí, o las distancias pueden estar influenciadas por diferencias poco importantes. En esos escenarios, KNN puede volverse menos confiable.

KNN también puede funcionar mal con datasets muy grandes si necesitamos predicciones rápidas. Como el modelo consulta los datos de entrenamiento para buscar vecinos, el costo de predicción puede crecer. Existen optimizaciones y estructuras de búsqueda más eficientes, pero la idea general sigue siendo una limitación frente a modelos que, una vez entrenados, predicen de manera más directa.

Los datos con mucho ruido o etiquetas incorrectas también pueden perjudicarlo. Si hay casos mal registrados cerca de un nuevo punto, pueden influir en la decisión. Esto es especialmente grave con valores pequeños de k . Aumentar k puede ayudar a suavizar el efecto de casos aislados, pero no elimina el problema si el ruido está muy extendido o si las clases están muy mezcladas.

Las clases desbalanceadas son otro desafío. Si una clase aparece con mucha más frecuencia que otra, es probable que muchos vecindarios estén dominados por la clase mayoritaria. Esto puede llevar al modelo a ignorar casos de la clase minoritaria, aunque esa clase sea la más importante para el problema. En detección de eventos raros, como ciertos riesgos o incumplimientos, KNN debe evaluarse con especial cuidado.

También hay que considerar la interpretabilidad requerida. KNN permite mirar vecinos concretos para explicar una predicción individual, lo cual puede ser útil. Pero no ofrece una regla global simple sobre cómo se toman todas las decisiones. En contextos donde se necesita una explicación general clara, auditable y fácil de comunicar, puede no ser la mejor opción.

Una buena forma de decidir si KNN es razonable es preguntarse: ¿tiene sentido que casos parecidos compartan clase?, ¿tenemos variables que representen bien esa semejanza?, ¿las variables están preparadas para calcular distancias?, ¿el dataset tiene un tamaño manejable?, ¿las clases están suficientemente representadas?, ¿el costo de predicción es aceptable?

Si varias respuestas son afirmativas, KNN puede ser una opción interesante para probar. Si no, probablemente convenga usar otro modelo o trabajar primero en mejorar la representación de los datos.

KNN puede funcionar bien cuando los datos están bien preparados, la cercanía entre casos tiene significado, las variables relevantes están presentes y el tamaño del dataset permite buscar vecinos de manera razonable. Puede funcionar mal cuando las distancias no representan semejanza útil, las variables están mal escaladas, hay demasiadas dimensiones irrelevantes, las clases están muy desbalanceadas, los datos tienen mucho ruido o se requieren predicciones muy rápidas. Su valor depende menos de la complejidad del algoritmo y más de que la idea de “vecinos parecidos” sea realmente adecuada para el problema.

Clasificar mirando a los vecinos

KNN nos permitió estudiar una primera forma concreta de clasificación: decidir la categoría de un caso nuevo observando casos conocidos que se encuentran cerca. La idea central es sencilla: si un caso se parece a otros casos históricos, sus etiquetas pueden ofrecer una pista razonable sobre la clase que debería recibir.

Esa simplicidad hace que KNN sea un modelo muy útil para aprender. Nos obliga a pensar qué significa que dos casos sean parecidos, cómo se mide una distancia, qué papel cumplen las variables de entrada y cómo una decisión local puede construir una frontera de decisión. También muestra con claridad que un modelo no trabaja con la realidad directamente, sino con la representación numérica que preparamos.

El parámetro `k` controla cuántos vecinos se tienen en cuenta. Con valores pequeños, el modelo responde de manera muy local y puede ser sensible al ruido. Con valores grandes, las decisiones se vuelven más estables, pero también pueden perder detalles importantes. Elegir `k` no es un detalle menor: modifica la frontera de decisión y afecta la capacidad del modelo para generalizar.

También vimos que el escalado es especialmente importante en KNN. Como el modelo se basa en distancias, las variables con valores numéricos más grandes pueden dominar la comparación si no se transforman adecuadamente. Escalar no

mejora mágicamente los datos, pero ayuda a que la distancia refleje mejor la semejanza entre casos cuando las variables están medidas en unidades diferentes.

KNN tiene ventajas claras: es intuitivo, flexible, aplicable a clasificación binaria y multiclase, y permite razonar de manera visual sobre vecinos y fronteras. Pero también tiene limitaciones: puede ser costoso con datasets grandes, sensible a variables irrelevantes, ruido, valores atípicos, clases desbalanceadas y problemas de escala. Además, no siempre ofrece una explicación global simple del comportamiento del modelo.

La pregunta más importante antes de usar KNN es si la cercanía realmente tiene sentido para el problema. Si casos parecidos tienden a compartir clase, y si las variables representan bien esa semejanza, KNN puede ser una buena opción o al menos una referencia inicial interesante. Si las distancias no capturan una similitud relevante, el modelo puede encontrar vecinos, pero esos vecinos no ayudarán demasiado a predecir.

KNN clasifica mirando a los vecinos. Su fuerza está en una intuición simple y poderosa; su debilidad, en depender completamente de cómo definimos y medimos la cercanía. En el próximo capítulo estudiaremos otro enfoque para clasificación: la regresión logística, un modelo que ya no decide por cercanía, sino estimando probabilidades a partir de una combinación de variables.

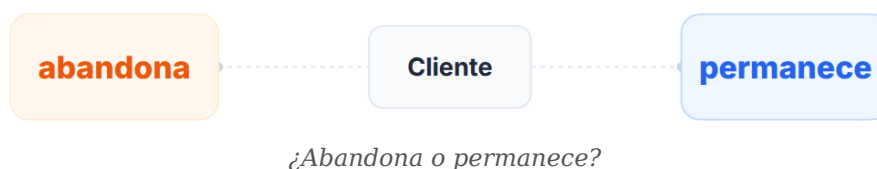
Capítulo 9 · Regresión logística: clasificar con probabilidades

Por qué se llama regresión si sirve para clasificar

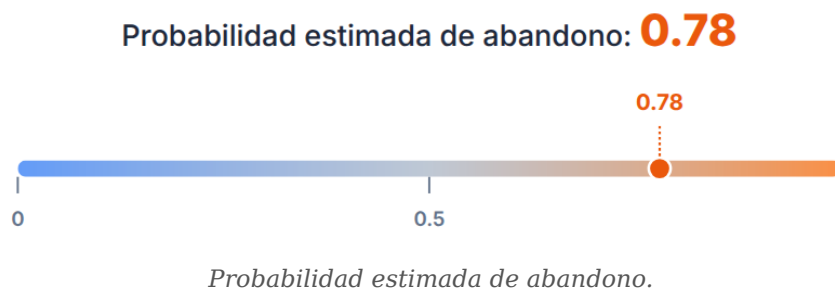
El nombre **regresión logística** puede generar confusión al principio. Venimos de distinguir entre problemas de regresión y problemas de clasificación: en regresión queremos predecir una cantidad numérica, mientras que en clasificación queremos predecir una categoría. Entonces, es razonable preguntarse por qué un modelo llamado “regresión” aparece dentro de una parte dedicada a clasificación.

La respuesta está en la forma matemática del modelo, no en el tipo de problema que resuelve habitualmente. La regresión logística utiliza una combinación de variables de entrada para estimar una probabilidad. Esa probabilidad luego se transforma en una clase. Por eso, aunque su nombre incluya la palabra “regresión”, en machine learning se usa principalmente como modelo de clasificación, especialmente para clasificación binaria.

Imaginemos un problema de abandono de clientes. Queremos predecir si un cliente abandonará o permanecerá. La salida final tiene dos clases posibles:

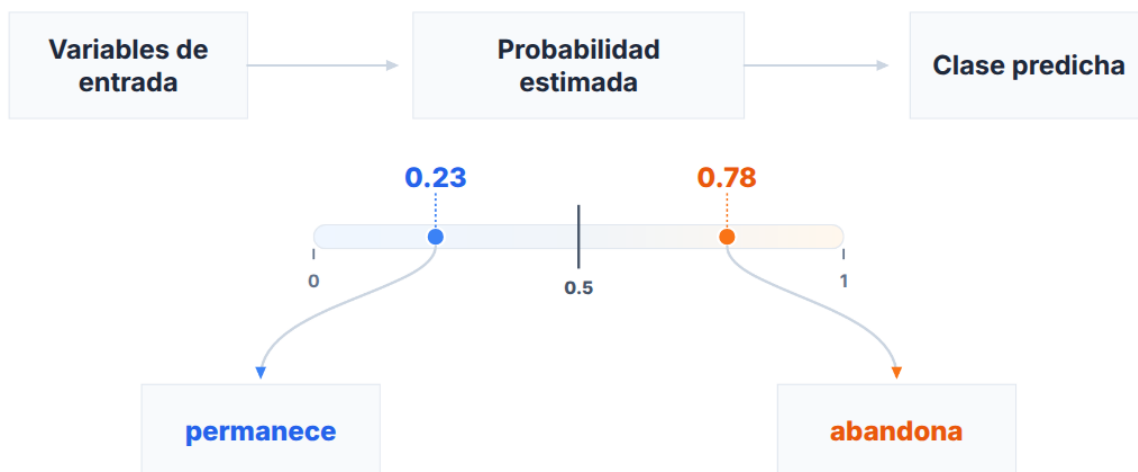


Pero la regresión logística no empieza directamente diciendo “abandona” o “permanece”. Primero estima una probabilidad. Por ejemplo:



Luego, a partir de esa probabilidad, se toma una decisión. Si usamos un umbral de 0.5, una probabilidad de 0.78 llevaría a clasificar al cliente como “abandona”. Si la probabilidad fuera 0.23, el modelo lo clasificaría como “permanece”.

La estructura general puede pensarse así:

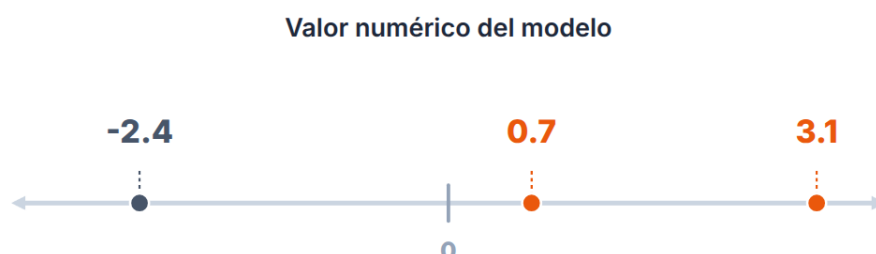


A partir de la probabilidad, se toma una decisión.

Esta idea la conecta con lo que vimos en el capítulo anterior sobre clases, etiquetas y probabilidades. Muchos clasificadores pueden producir una etiqueta final, pero también una probabilidad asociada a cada clase. La regresión logística es especialmente importante porque su funcionamiento se entiende justamente desde esa lógica: estimar probabilidades para tomar decisiones de clasificación.

El término “logística” aparece porque el modelo utiliza una función especial, llamada función logística o sigmoide, que permite convertir un valor numérico cualquiera en un valor entre 0 y 1. Ese rango es fundamental, porque una probabilidad debe estar entre 0 y 1. No puede ser menor que 0 ni mayor que 1.

No necesitamos entrar todavía en todos los detalles matemáticos, pero sí conviene entender el problema que intenta resolver. Si combinamos variables de entrada mediante una operación lineal, el resultado podría ser cualquier número: negativo, positivo, muy pequeño o muy grande. Por ejemplo, un modelo podría producir valores como:



El resultado podría ser cualquier número.

Esos números no son probabilidades por sí mismos. Para interpretarlos como una probabilidad, necesitamos transformarlos. La función logística cumple ese papel: toma un valor numérico y lo comprime dentro del intervalo entre 0 y 1.



La función logística devuelve valores entre 0 y 1.

Por eso la regresión logística no predice una cantidad continua como el precio de una vivienda o la duración de un viaje. Predice una probabilidad asociada a una clase. Luego, esa probabilidad se convierte en una etiqueta final.

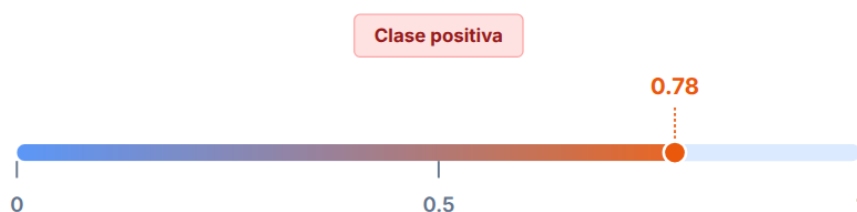
En clasificación binaria, normalmente se elige una clase como la clase positiva. Por ejemplo, en un problema de abandono, podríamos definir:



Clasificación binaria.

La regresión logística puede estimar la probabilidad de la clase positiva:

$$P(\text{abandona}) = 0.78$$



Probabilidad de la clase positiva.

Si esa probabilidad supera cierto umbral, se predice la clase positiva. Si no lo supera, se predice la clase negativa.

Este mecanismo es muy útil porque separa dos ideas distintas: estimar el riesgo y tomar una decisión. El modelo puede decir que un cliente tiene una probabilidad estimada de abandono de 0.62. Luego, la organización decide qué hacer con esa información. Puede clasificarlo como “abandona” usando un umbral de 0.5, puede priorizarlo en una lista de clientes en riesgo, o puede intervenir solamente si la probabilidad supera 0.8.

Lo mismo ocurre en otros contextos. En riesgo crediticio, la regresión logística puede estimar la probabilidad de incumplimiento. En diagnóstico asistido, puede estimar la probabilidad de que un caso requiera revisión. En detección de fraude, puede estimar la probabilidad de que una transacción sea sospechosa. En todos esos casos, la salida probabilística permite tomar decisiones con más matices que una etiqueta aislada.

El nombre “regresión logística”, entonces, no debe llevarnos a pensar que estamos resolviendo un problema de regresión en el sentido que usamos en machine learning. El modelo tiene una raíz matemática relacionada con la regresión, porque calcula una combinación de variables y ajusta parámetros. Pero su uso más frecuente en este contexto es clasificar casos, especialmente cuando queremos estimar la probabilidad de pertenecer a una clase.

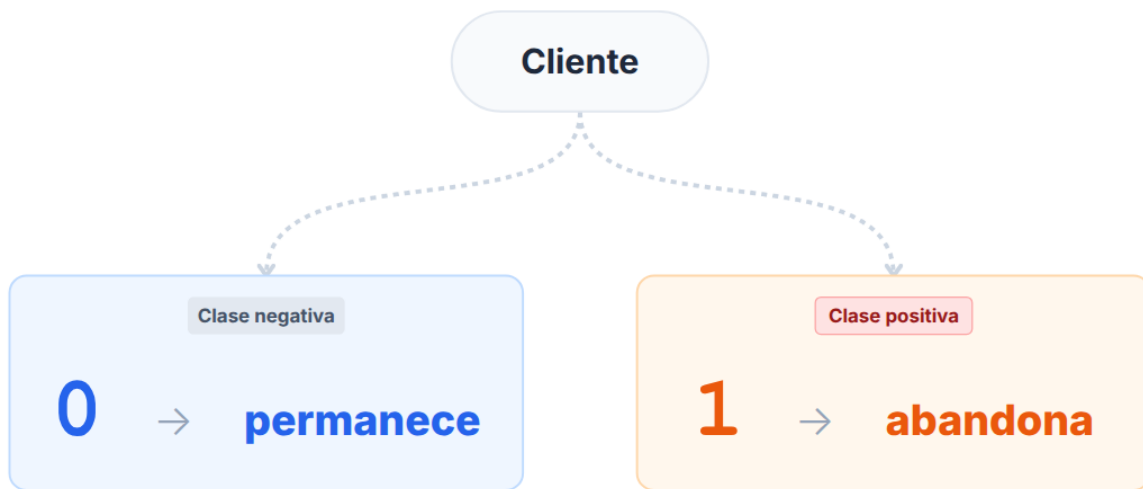
También es importante notar que la regresión logística suele ser un modelo relativamente simple e interpretable en comparación con otros métodos más complejos. Esto la convierte en una herramienta muy usada como primer modelo serio de clasificación. No solo permite obtener predicciones, sino también analizar cómo las variables influyen en la probabilidad estimada, aunque esa interpretación requiere cuidado y la estudiaremos gradualmente.

La regresión logística se llama “regresión” por su formulación matemática, pero se utiliza principalmente para clasificación. Su idea central es estimar una probabilidad entre 0 y 1, y luego convertir esa probabilidad en una clase mediante un umbral de decisión. Esta forma de trabajar la vuelve especialmente útil cuando no solo queremos saber qué clase predice el modelo, sino también con qué grado de probabilidad estima esa pertenencia.

La idea de estimar una probabilidad

La regresión logística es un modelo de clasificación, pero su salida principal no tiene por qué pensarse solamente como una etiqueta. Su aporte más importante es que permite estimar una **probabilidad** asociada a una clase. Luego, esa probabilidad puede convertirse en una decisión.

En clasificación binaria, esto resulta especialmente claro. Supongamos que queremos predecir si un cliente abandonará un servicio. Podemos definir dos clases:



Definimos dos clases.

La regresión logística puede estimar la probabilidad de que el cliente pertenezca a la clase positiva, que en este caso sería “abandona”. Para un cliente determinado, el modelo podría producir una salida como esta:



Salida para un cliente determinado.

Esto significa que, según los patrones aprendidos a partir de los datos disponibles, el modelo estima una probabilidad relativamente alta de abandono. Todavía no estamos hablando necesariamente de una decisión final. Estamos hablando de una estimación.

Esta diferencia es importante. Una probabilidad no es lo mismo que una etiqueta. La etiqueta resume la decisión en una categoría; la probabilidad conserva más información sobre el grado de riesgo o pertenencia estimado por el modelo.

Podemos pensarlo así:



Una probabilidad no es lo mismo que una etiqueta.

En cambio, otro cliente podría recibir una probabilidad de abandono de 0.51. Si usamos un umbral de 0.5, también sería clasificado como “abandona”. Pero el caso no parece igual al anterior. Una probabilidad de 0.73 sugiere una señal más fuerte que una probabilidad apenas superior a 0.5.



Diferente intensidad de señal, mismo resultado.

Los dos clientes reciben la misma clase final, pero no la misma intensidad de señal. Esa diferencia puede ser muy útil. En lugar de tratar todas las predicciones positivas como equivalentes, podemos ordenar los casos según su probabilidad estimada, priorizar algunos o decidir que ciertos casos requieren revisión adicional.

En abandono de clientes, una empresa podría usar esas probabilidades para organizar acciones de retención. Si tiene recursos limitados para contactar clientes, quizás empiece por aquellos con mayor probabilidad estimada de abandono. No necesita ver solamente una columna con “abandona” o “permanece”; puede trabajar con una escala de riesgo.

En riesgo crediticio, una probabilidad estimada de incumplimiento permite distinguir entre solicitudes con niveles de riesgo diferentes. Dos solicitudes pueden superar el umbral que las marca como riesgosas, pero una puede tener una probabilidad

estimada de incumplimiento de 0.55 y otra de 0.90. La decisión final podría ser distinta: revisión adicional, condiciones diferentes, solicitud de más información o rechazo, dependiendo del contexto y de las políticas aplicables.

En diagnóstico asistido, la salida probabilística también puede ser importante. Un sistema puede señalar casos que requieren revisión prioritaria, pero no todos los casos señalados tienen la misma urgencia. Una probabilidad estimada más alta puede ayudar a ordenar la revisión, siempre bajo supervisión profesional y con criterios adecuados.

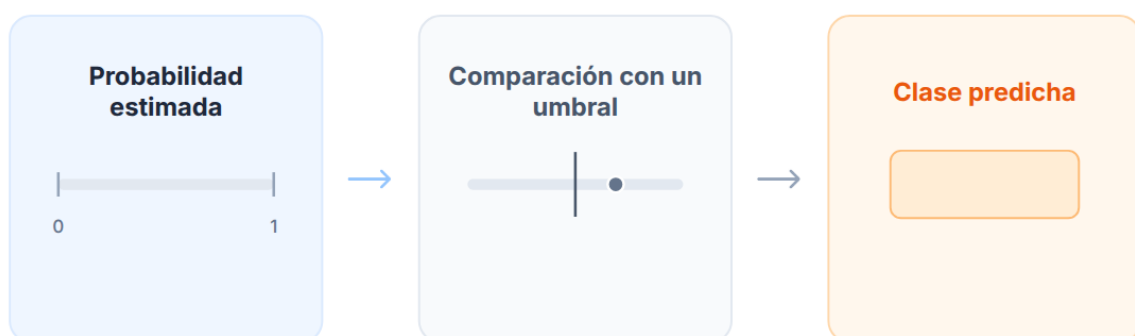
La regresión logística produce estas probabilidades a partir de las variables de entrada. Cada caso tiene valores en X : edad, ingresos, frecuencia de uso, cantidad de reclamos, monto solicitado, historial de pagos u otras variables según el problema. El modelo combina esa información para estimar la probabilidad de la clase positiva.

La estructura conceptual es:



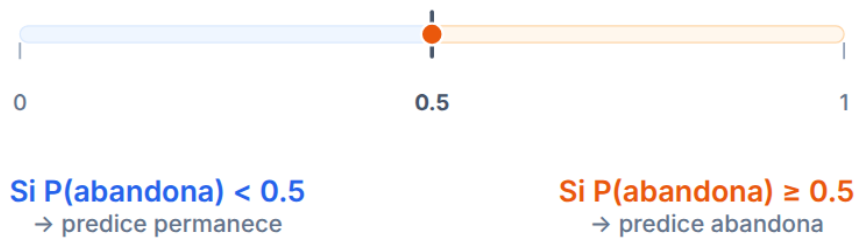
El modelo estima la probabilidad de la clase positiva.

Luego, si necesitamos una clase final, aplicamos una regla de decisión:



Regla de decisión.

El umbral más habitual para empezar es 0.5. Si la probabilidad estimada es mayor o igual que 0.5, se predice la clase positiva. Si es menor que 0.5, se predice la clase negativa.



El umbral más habitual para empezar es 0.5.

Sin embargo, ese umbral no es obligatorio ni universal. Es una decisión que puede ajustarse según el problema. Si queremos detectar la mayor cantidad posible de casos positivos, podemos bajar el umbral. Si queremos ser más estrictos antes de marcar un caso como positivo, podemos subirlo. Esta idea será muy importante cuando estudiemos métricas de clasificación, porque cambiar el umbral modifica el tipo de errores que comete el modelo. Por ahora, lo esencial es separar tres elementos:



Tres elementos diferentes.

Estos tres elementos están relacionados, pero no son lo mismo. La regresión logística estima una probabilidad. La clase final aparece cuando decidimos cómo convertir esa probabilidad en una categoría.

También debemos ser prudentes al interpretar las probabilidades. Una probabilidad estimada por el modelo no es una certeza absoluta. Depende de los datos usados para entrenar, de las variables disponibles, de la calidad de las etiquetas, de la forma del modelo y del contexto en el que se aplica. Si los datos no representan bien la realidad actual, la probabilidad puede ser engañosa. Si faltan variables importantes, el modelo puede estimar con información incompleta. Si las etiquetas históricas contienen errores, la probabilidad reflejará esas limitaciones.

Por eso, cuando decimos que un cliente tiene una probabilidad estimada de abandono de 0.73, no estamos afirmando que “seguro abandonará”. Estamos diciendo que, según el modelo y los datos disponibles, ese caso se parece a patrones asociados con abandono en una medida que el modelo traduce como 0.73.

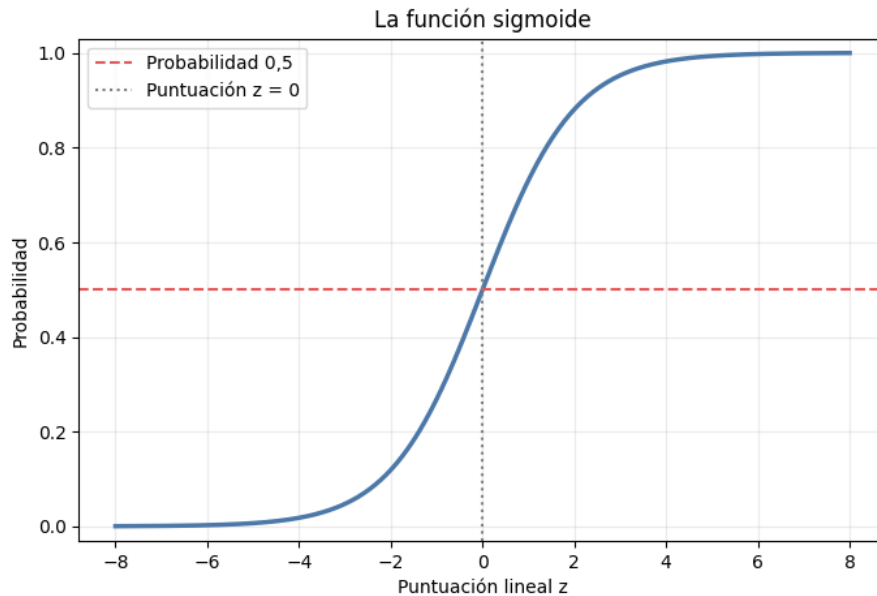
Esta distinción es especialmente importante en contextos sensibles. Una probabilidad de riesgo no debería convertirse automáticamente en una decisión definitiva sin revisión. En muchos casos, debe entenderse como una señal para priorizar, analizar o acompañar una decisión más amplia.

La regresión logística es valiosa porque hace explícita esa escala probabilística. En lugar de entregar solamente una clase, permite trabajar con grados. Esto la vuelve muy útil como modelo inicial de clasificación, porque combina una estructura relativamente simple con una salida fácil de interpretar.

La regresión logística clasifica estimando primero una probabilidad. En clasificación binaria, esa probabilidad suele asociarse a la clase positiva. Luego, mediante un umbral de decisión, se convierte en una etiqueta final. Esta separación entre probabilidad y decisión permite interpretar mejor los resultados, priorizar casos y adaptar el modelo a las necesidades del problema.

La función sigmoide

Para que la regresión logística pueda estimar probabilidades, necesita transformar el resultado interno del modelo en un número entre 0 y 1. Esa transformación se realiza mediante una función llamada **sigmoide**, también conocida como **función logística**.



La curva tiene forma de S. Las puntuaciones muy negativas se acercan a 0 y las muy positivas se acercan a 1, pero la función nunca sale del intervalo de probabilidades.

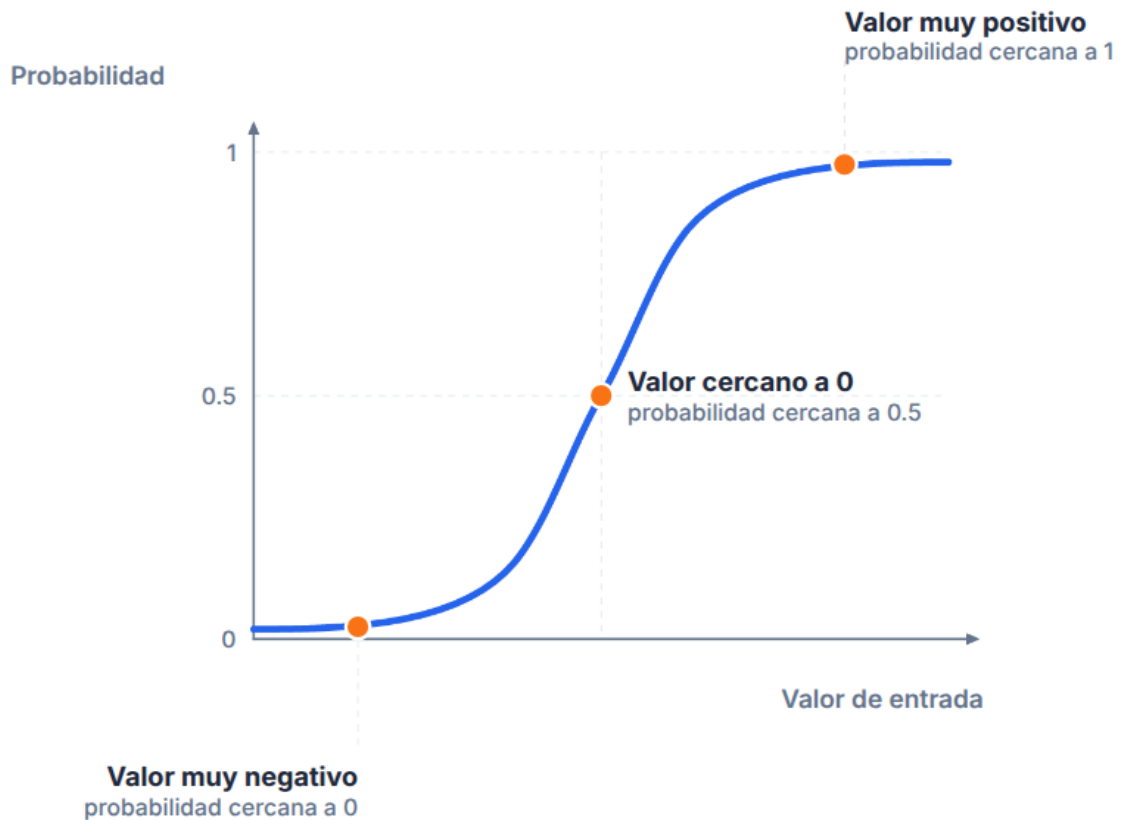
La idea es la siguiente. El modelo primero combina las variables de entrada para producir un valor numérico. Ese valor puede ser negativo, positivo, pequeño o muy grande. Por sí solo, no es una probabilidad. Una probabilidad debe estar limitada entre 0 y 1. La función sigmoide toma ese valor numérico y lo convierte en una salida dentro de ese intervalo.

Podemos pensarlo así:



La función sigmoide.

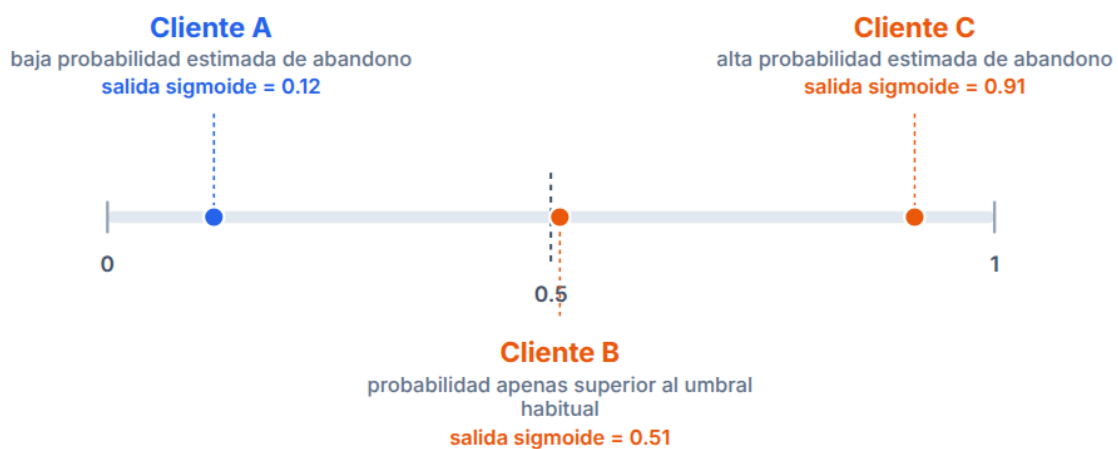
La función sigmoide tiene una forma característica. Cuando el valor de entrada es muy negativo, la salida se acerca a 0. Cuando el valor de entrada es muy positivo, la salida se acerca a 1. Cuando el valor de entrada está cerca de 0, la salida está cerca de 0.5.



Forma de la función sigmoide.

Esto encaja muy bien con la clasificación binaria. Si estamos estimando la probabilidad de abandono, una salida cercana a 0 indica que el modelo ve poco riesgo de abandono. Una salida cercana a 1 indica que el modelo ve mucho riesgo de abandono. Una salida cercana a 0.5 indica una zona más incierta, donde el modelo no se inclina con mucha fuerza hacia ninguna de las dos clases.

Por ejemplo:

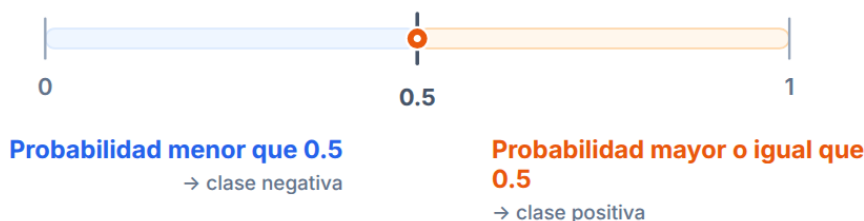


Una salida cercana a 0.5 indica una zona incierta.

La sigmoide no decide por sí sola la clase final. Su tarea es producir una probabilidad. La decisión aparece después, cuando comparamos esa probabilidad con un umbral. Si usamos el umbral habitual de 0.5, entonces una probabilidad de 0.51 alcanza para predecir la clase positiva, mientras que una probabilidad de 0.49 no alcanza. Pero ambas están muy cerca de la frontera de decisión.

Esta observación es importante porque muestra que no todas las predicciones tienen la misma claridad. Un caso con probabilidad 0.98 está muy lejos del umbral de 0.5; un caso con probabilidad 0.52 está apenas por encima. Aunque ambos puedan recibir la misma etiqueta final, el segundo caso debería interpretarse con más cautela.

La función sigmoide también ayuda a entender por qué la regresión logística construye una frontera de decisión. En clasificación binaria, si usamos un umbral de 0.5, la frontera aparece en los puntos donde la probabilidad estimada cambia de estar por debajo a estar por encima de ese valor. De un lado de la frontera, el modelo predice la clase negativa; del otro, predice la clase positiva. En otras palabras:



Frontera.

Detrás de esa probabilidad hay una combinación de variables. Por ejemplo, en un problema de riesgo crediticio, el modelo puede combinar ingresos, monto solicitado, atrasos previos y relación cuota-ingreso. Esa combinación produce un valor interno. La sigmoide transforma ese valor en una probabilidad estimada de incumplimiento. Luego, el umbral convierte esa probabilidad en una decisión de clasificación.

La sigmoide tiene una ventaja importante: nunca produce valores menores que 0 ni mayores que 1. Esto la hace adecuada para representar probabilidades. Si usáramos directamente una combinación lineal de variables, podríamos obtener resultados como -3 o 2.4, que no pueden interpretarse como probabilidades. La sigmoide comprime esos valores en un rango válido.

Sin embargo, conviene no confundir la forma matemática con una garantía de verdad. Que la salida esté entre 0 y 1 no significa que sea automáticamente una probabilidad perfecta. Sigue siendo una estimación basada en datos, variables, etiquetas y supuestos del modelo. Si los datos tienen problemas, la salida de la sigmoide también reflejará esos problemas.

Tampoco necesitamos, en esta etapa, memorizar la fórmula de la función sigmoide. Lo importante es comprender su papel dentro del modelo. La regresión logística no entrega cualquier número: entrega una probabilidad porque aplica una transformación que convierte el valor interno del modelo en una escala entre 0 y 1. Podemos resumirlo así:



Funcionamiento del modelo.

Esta cadena permite separar el funcionamiento del modelo en partes comprensibles. Primero, las variables aportan señales. Luego, el modelo las combina. Después, la sigmoide transforma el resultado en una probabilidad. Finalmente, una regla de decisión produce la etiqueta final.

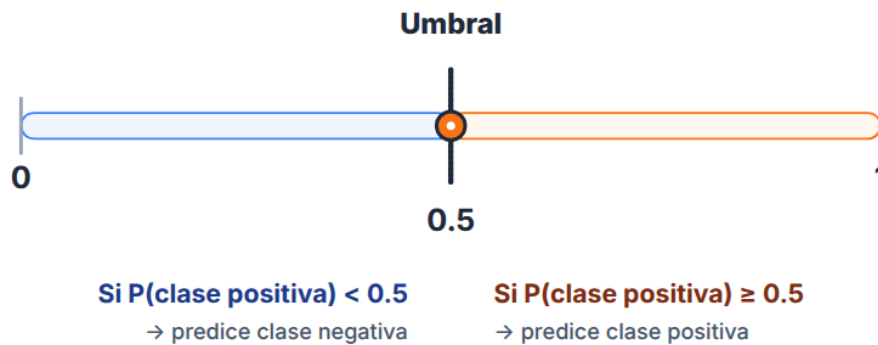
La función sigmoide es el componente que permite a la regresión logística transformar un valor numérico interno en una probabilidad entre 0 y 1. Gracias a ella, el modelo puede expresar qué tan probable estima que un caso pertenezca a la clase positiva. Esa probabilidad no es todavía la decisión final, pero es la base sobre la cual luego se aplica un umbral para clasificar.

El umbral de decisión

La regresión logística estima una probabilidad, pero muchas veces necesitamos una clase final. En un problema de clasificación binaria, no alcanza con decir que un cliente tiene una probabilidad estimada de abandono de 0.62; en algún momento, si el sistema debe actuar, habrá que decidir si ese caso será tratado como “abandona” o como “permanece”. Para pasar de la probabilidad a la clase usamos un **umbral de decisión**.

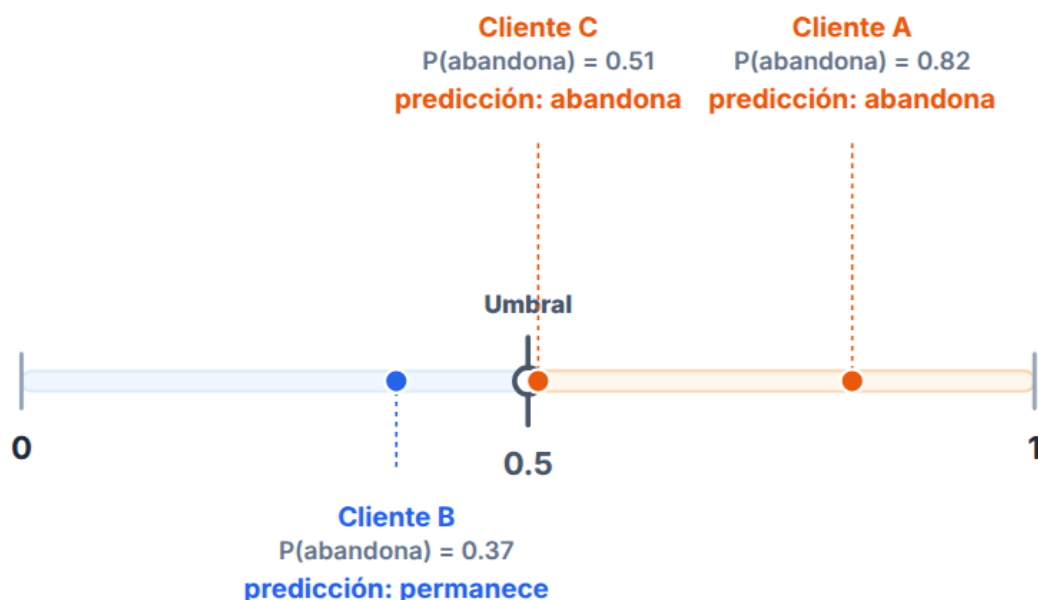
El umbral es el valor a partir del cual consideramos que un caso pertenece a la clase positiva. En muchos ejemplos iniciales se usa 0.5. Si la probabilidad estimada es

mayor o igual que 0.5, se predice la clase positiva. Si es menor que 0.5, se predice la clase negativa.



El valor del umbral determina la predicción.

Por ejemplo, si la clase positiva es “abandona”, podríamos tener:



Cientes con diferentes situaciones, pero con la misma etiqueta.

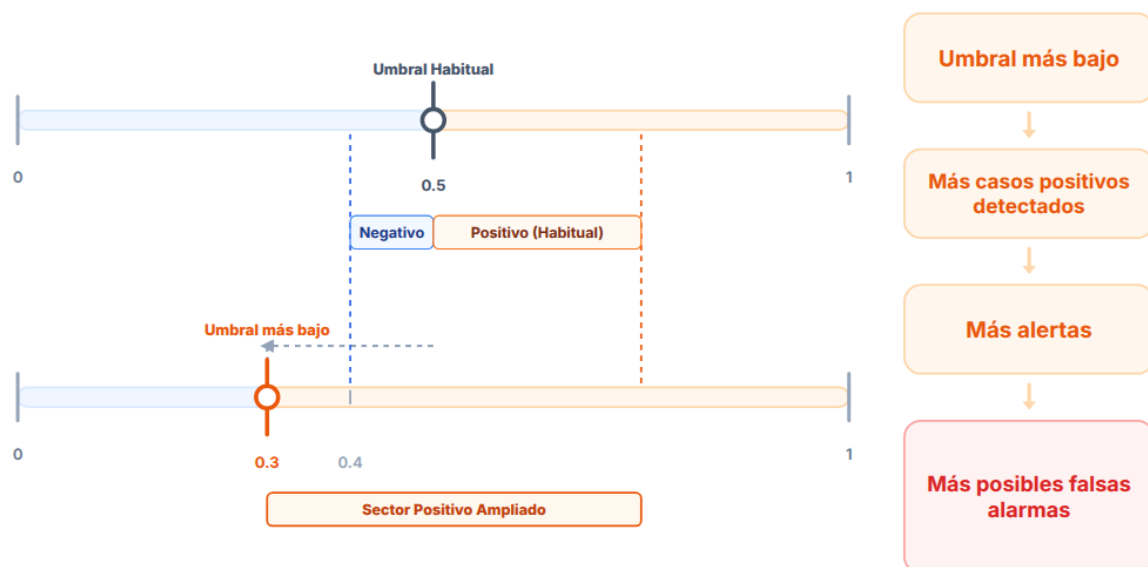
Los clientes A y C reciben la misma etiqueta final, pero no representan la misma situación. El cliente A está bastante por encima del umbral. El cliente C está apenas por encima. Esto muestra que la etiqueta final simplifica la información: convierte una escala continua de probabilidades en una categoría.

El umbral de 0.5 es útil para empezar porque parece natural: si la probabilidad estimada de la clase positiva supera a la de la clase negativa, elegimos la clase positiva. Pero no debemos pensar que 0.5 es siempre el mejor umbral. Es una decisión inicial, no una ley universal.

La elección del umbral depende del problema y de las consecuencias de los errores. En clasificación binaria, hay dos tipos generales de error: marcar como positivo un caso que en realidad era negativo, o marcar como negativo un caso que en realidad era positivo. Estos errores no suelen tener el mismo costo.

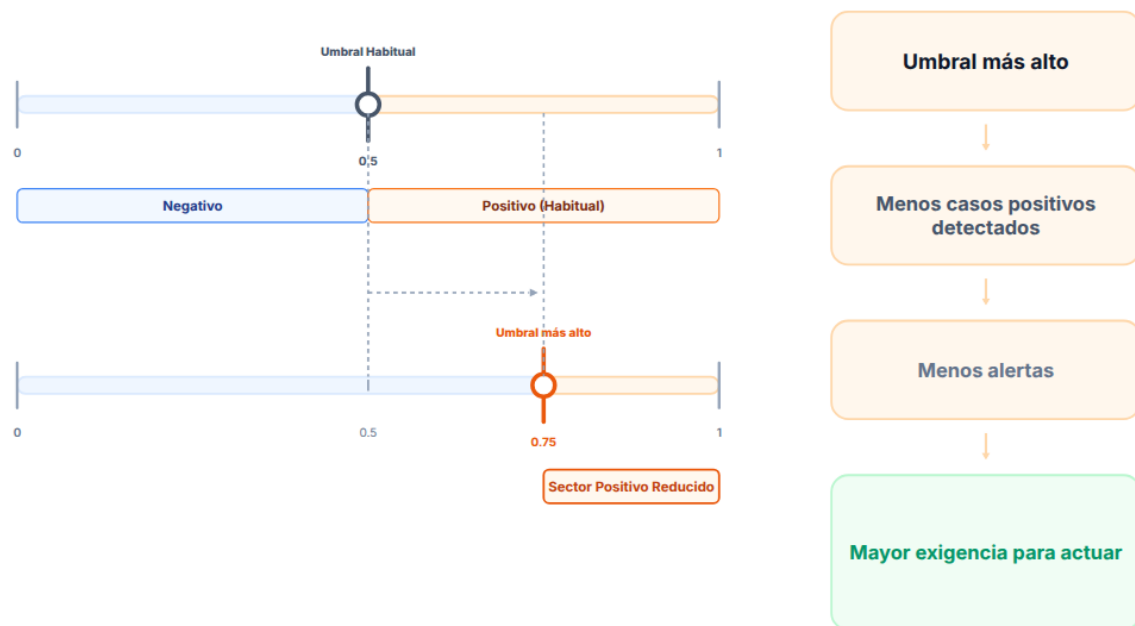
Imaginemos un sistema que intenta detectar clientes con riesgo de abandono. Si el modelo clasifica como “abandona” a un cliente que en realidad habría permanecido, la empresa tal vez le ofrezca una promoción innecesaria o lo contacte sin necesidad. Ese error tiene un costo. Pero si el modelo clasifica como “permanece” a un cliente que realmente estaba por abandonar, la empresa pierde la oportunidad de intervenir. Ese error también tiene un costo, y quizás sea más importante según la estrategia del negocio.

En ese caso, podríamos bajar el umbral. En lugar de exigir una probabilidad de abandono de 0.5, podríamos intervenir desde 0.4 o 0.3. Esto hará que más clientes sean clasificados como “abandona”. Detectaremos más posibles abandonos, pero también aumentará la cantidad de falsas alarmas.



Umbral más bajo → más posibles falsas alarmas.

En cambio, si intervenir tiene un costo alto y queremos actuar solo cuando el modelo muestra una señal fuerte, podríamos subir el umbral. Por ejemplo, podríamos contactar solamente a clientes con probabilidad estimada de abandono mayor o igual que 0.75. Esto reducirá la cantidad de acciones, pero también puede dejar afuera casos que terminarán abandonando.

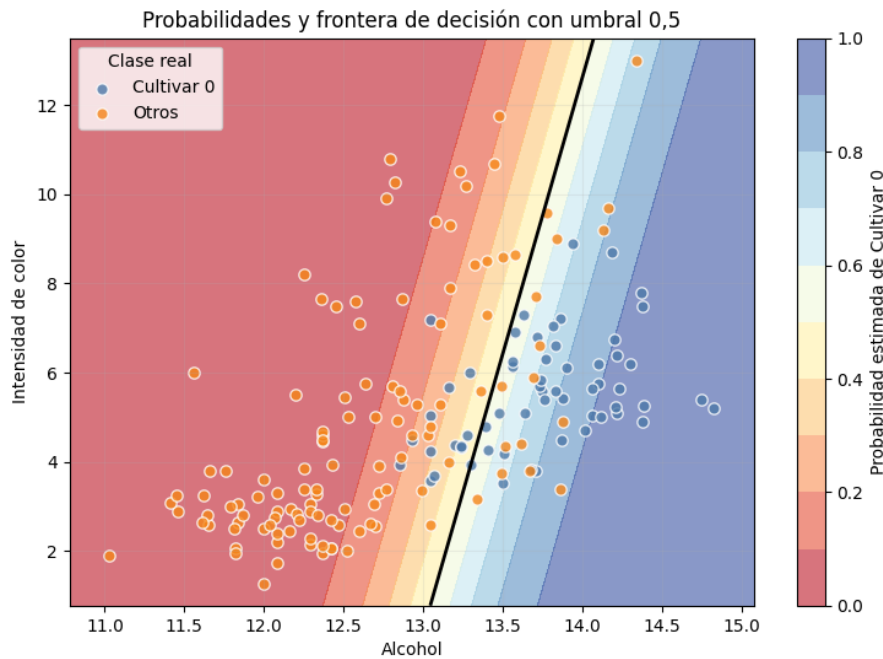


Umbral más alto → mayor exigencia para actuar.

La misma idea aparece en riesgo crediticio. Si el modelo estima la probabilidad de incumplimiento, un umbral bajo marcará más solicitudes como riesgosas. Esto puede reducir ciertos riesgos financieros, pero también puede generar rechazos o revisiones innecesarias. Un umbral alto marcará menos solicitudes como riesgosas, pero puede dejar pasar casos que luego incumplan. La elección no es puramente técnica: depende de políticas, regulaciones, objetivos y criterios de equidad.

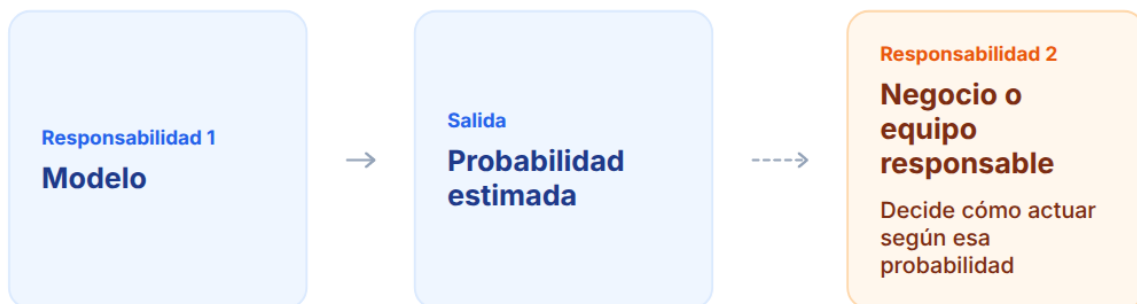
En diagnóstico asistido, el umbral es especialmente sensible. Si el objetivo es detectar casos que requieren revisión, bajar el umbral puede ayudar a no dejar pasar casos importantes. Pero también puede aumentar la cantidad de revisiones innecesarias. Subir el umbral reduce esas falsas alarmas, pero puede aumentar el riesgo de no detectar casos que sí requerían atención. Por eso, en contextos de salud, el umbral debe definirse con muchísimo cuidado y con participación profesional.

Esto muestra una idea central: el modelo estima probabilidades, pero el umbral expresa una decisión. Dos personas pueden usar el mismo modelo y elegir umbrales diferentes porque tienen objetivos distintos. Una puede priorizar detectar la mayor cantidad posible de casos positivos. Otra puede priorizar reducir falsas alarmas. El modelo puede ser el mismo; la regla de decisión cambia.



Una frontera simple no separa necesariamente todos los casos.

También podemos usar las probabilidades sin convertirlas inmediatamente en clases. En algunos proyectos, en lugar de decir “positivo” o “negativo”, se ordenan los casos por probabilidad. Por ejemplo, una empresa puede generar una lista de clientes ordenada de mayor a menor riesgo de abandono y contactar a los primeros mil. En ese caso, el umbral puede estar determinado por la capacidad operativa: cuántos casos se pueden atender, revisar o contactar.



El modelo decide cómo actuar según esa probabilidad.

En `scikit-learn`, cuando usamos un clasificador como regresión logística, normalmente podemos obtener dos tipos de salida. Con `predict`, obtenemos la clase final según el umbral usado por el modelo. Con `predict_proba`, obtenemos las probabilidades estimadas para las clases. No necesitamos profundizar todavía en el código, pero conviene distinguir ambas ideas.

```
modelo.predict(X_test)      # devuelve clases predichas
modelo.predict_proba(X_test) # devuelve probabilidades estimadas
```

Esta diferencia es importante porque muchas veces la clase final oculta información que sí aparece en la probabilidad. Si queremos ajustar umbrales, priorizar casos o analizar el comportamiento del modelo con más detalle, necesitamos mirar las probabilidades y no solo las etiquetas predichas.

Por supuesto, cambiar el umbral no mejora mágicamente el modelo. No hace que las probabilidades sean más correctas ni que los datos sean mejores. Lo que cambia es la manera en que convertimos esas probabilidades en decisiones. Por eso, cuando modifiquemos un umbral, tendremos que evaluar cómo cambian los errores. Más adelante estudiaremos métricas como precisión, recall, F1 y matriz de confusión, que nos ayudarán a analizar estos efectos.

El umbral de decisión es el punto de corte que transforma una probabilidad estimada en una clase final. El valor 0.5 es un punto de partida habitual, pero no siempre es el más adecuado. Elegir un umbral implica decidir qué tipo de errores estamos dispuestos a aceptar y qué consecuencias tienen esas decisiones en el contexto real. La regresión logística estima probabilidades; el umbral convierte esas probabilidades en acciones o categorías.

Interpretar la influencia de las variables

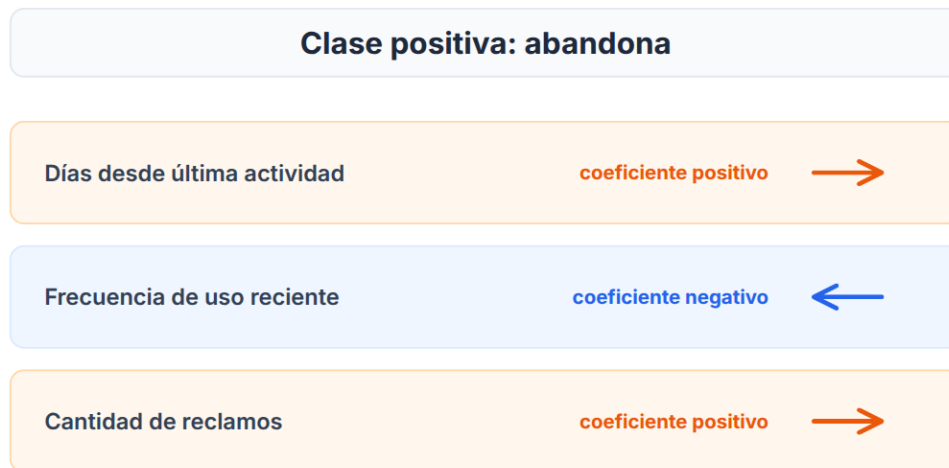
Una de las razones por las que la regresión logística es tan utilizada es que, además de producir predicciones, permite analizar cómo se relacionan las variables de entrada con la probabilidad estimada de la clase positiva. Esta posibilidad la vuelve atractiva cuando no solo queremos clasificar, sino también comprender qué señales está usando el modelo.

En KNN, la predicción depende de los vecinos cercanos. Podemos mirar qué casos fueron vecinos de una observación particular, pero no obtenemos una regla global sencilla sobre el peso de cada variable. En la regresión logística, en cambio, el modelo aprende una combinación de las variables. Cada variable participa con un coeficiente, y ese coeficiente indica cómo cambia la salida interna del modelo cuando esa variable aumenta, manteniendo las demás constantes.

No necesitamos profundizar todavía en la matemática completa de esos coeficientes, pero sí podemos entender la idea general. Si una variable tiene un coeficiente positivo, entonces valores más altos de esa variable empujan la probabilidad estimada hacia la clase positiva. Si una variable tiene un coeficiente negativo, valores más altos empujan la probabilidad estimada en sentido contrario, hacia la clase negativa.

Por ejemplo, en un problema de abandono de clientes, podríamos tener variables como días desde la última actividad, cantidad de reclamos, antigüedad como cliente y frecuencia de uso. Si la clase positiva es “abandona”, es razonable imaginar que más días de inactividad podrían aumentar la probabilidad estimada de abandono. En ese caso, esa variable podría tener un coeficiente positivo. En cambio, una mayor frecuencia de uso reciente podría asociarse con menor probabilidad de abandono, por lo que podría tener un coeficiente negativo.

Podemos pensarlo así:



Problema de abandono de clientes.

Esta lectura debe hacerse siempre con cuidado. Un coeficiente positivo no significa necesariamente que la variable “cause” la clase positiva. Significa que, dentro del modelo y según los datos usados, mayores valores de esa variable están asociados con una mayor probabilidad estimada de la clase positiva, considerando también las otras variables incluidas.

La diferencia entre asociación y causalidad es fundamental. Si un modelo aprende que cierta variable está asociada con mayor riesgo, eso no prueba que esa variable sea la causa del riesgo. Puede ser una señal indirecta, una consecuencia de otro proceso o una variable relacionada con factores que no están en el dataset. La regresión logística puede ayudar a interpretar asociaciones, pero no convierte automáticamente un proyecto predictivo en un estudio causal.

Otro aspecto importante es que la interpretación de los coeficientes depende de cómo estén representadas las variables. Si usamos variables numéricas escaladas, el coeficiente se refiere a cambios en la escala transformada, no en la unidad original. Por ejemplo, si estandarizamos ingresos, el coeficiente ya no corresponde a “un peso más de ingreso”, sino a una variación en términos de desviaciones estándar. Esto puede ser útil para comparar variables, pero requiere recordar que el modelo no está trabajando con las unidades originales.

En variables categóricas transformadas con One-Hot Encoding, la interpretación también cambia. Cada categoría se convierte en una columna binaria. El coeficiente de esa columna indica cómo se relaciona pertenecer a esa categoría con la probabilidad estimada de la clase positiva, en comparación con la referencia implícita que deja la codificación. No conviene interpretar esas columnas como cantidades continuas, sino como presencia o ausencia de una categoría.

Por ejemplo, si una variable “tipo de plan” se transforma en columnas como `plan_basico`, `plan_premium` y `plan_empresa`, cada columna indica si el cliente pertenece o no a ese tipo de plan. Un coeficiente positivo en `plan_basico` podría indicar que, en los datos y según el modelo, ese plan está asociado con mayor probabilidad estimada de abandono que la categoría de referencia o que otras configuraciones del modelo.

`plan_basico = 1` El cliente tiene plan básico

`plan_basico = 0` El cliente no tiene plan básico

Cada categoría se convierte en una columna binaria.

La interpretación de variables categóricas requiere revisar cómo se hizo la codificación. Si no sabemos cómo se prepararon las columnas, podemos sacar conclusiones equivocadas. Por eso, cuando usamos `Pipeline` y `ColumnTransformer`, es importante documentar qué transformaciones se aplicaron y qué columnas finales vio el modelo.

También debemos tener cuidado con la magnitud de los coeficientes. Un coeficiente más grande en valor absoluto indica una influencia más fuerte sobre la salida interna del modelo, pero esa comparación solo es razonable si las variables están en escalas comparables. Si una variable está medida en pesos, otra en años y otra en conteos pequeños, comparar directamente sus coeficientes puede ser engañoso. El escalado ayuda a que esa comparación sea más razonable, aunque no elimina la necesidad de interpretar con contexto.

Además, los coeficientes pueden cambiar cuando agregamos o quitamos variables. Si dos variables están muy relacionadas entre sí, el modelo puede repartir la información entre ambas de una manera difícil de interpretar. Por ejemplo, ingresos mensuales y monto máximo de crédito disponible podrían estar correlacionados. Si

ambas aparecen en el modelo, sus coeficientes deben analizarse con prudencia, porque cada una se interpreta considerando que la otra también está presente.

Esta es una diferencia importante entre mirar una variable de manera aislada y mirar un coeficiente dentro de un modelo. El coeficiente no dice simplemente “esta variable se relaciona con la clase”. Dice algo más específico: “dentro de esta combinación de variables, esta variable empuja la predicción en tal dirección”. Esa frase es más larga, pero también más honesta.

La regresión logística suele considerarse más interpretable que otros modelos porque sus coeficientes permiten ver la dirección de la asociación entre variables y probabilidad estimada. Sin embargo, “más interpretable” no significa “automáticamente clara”. La interpretación depende de la preparación de datos, de la escala, de la codificación de categorías, de la relación entre variables y del conocimiento del dominio.

En contextos sensibles, esta interpretación debe ser especialmente cuidadosa. Si un modelo de riesgo crediticio muestra que ciertas variables empujan la probabilidad de incumplimiento hacia arriba, debemos revisar si esas variables son pertinentes, si están permitidas, si introducen sesgos o si funcionan como sustitutos de información problemática. Interpretar un modelo no es solo mirar números; también es preguntarse si el modelo está usando señales razonables desde el punto de vista del problema.

También puede ocurrir que una variable tenga una influencia estadística clara pero no sea conveniente usarla. Por ejemplo, puede estar disponible en el dataset histórico pero no en el momento real de predicción, o puede estar relacionada con decisiones posteriores al evento que queremos anticipar. En esos casos, aunque el coeficiente parezca útil, la variable debería excluirse para evitar fuga de datos o decisiones inapropiadas.

En `scikit-learn`, después de entrenar una regresión logística, es posible acceder a los coeficientes aprendidos. En un cuaderno práctico, esto nos permitirá armar una tabla con nombres de variables y coeficientes para observar qué señales empujan la predicción hacia una clase u otra. Pero esa tabla no debe leerse de manera automática; debe interpretarse junto con el preprocesamiento y el contexto del problema.

```
modelo.coef_
```

Cuando el modelo forma parte de un `Pipeline`, acceder a los nombres finales de las columnas puede requerir algunos pasos adicionales, especialmente si hubo One-Hot

Encoding. Lo importante, por ahora, no es memorizar el código, sino comprender qué representan esos coeficientes y qué cuidados exige su lectura.

La regresión logística permite analizar la influencia de las variables mediante coeficientes. Un coeficiente positivo empuja la probabilidad estimada hacia la clase positiva; uno negativo la empuja en sentido contrario. Pero esta interpretación depende de la escala, la codificación, las demás variables incluidas y el contexto del problema. Los coeficientes ayudan a entender el modelo, siempre que recordemos que muestran asociaciones aprendidas, no verdades causales absolutas.

Ventajas del modelo

La regresión logística es uno de los modelos más utilizados para comenzar a trabajar con clasificación. Su popularidad no se debe a que sea el modelo más complejo ni el más flexible, sino a que ofrece un equilibrio muy valioso entre simplicidad, utilidad e interpretación.

Una primera ventaja es que permite obtener probabilidades. En muchos problemas, no alcanza con saber qué clase predice el modelo; también queremos conocer qué tan fuerte parece esa predicción. La regresión logística no se limita a devolver una etiqueta final, sino que estima la probabilidad de pertenecer a la clase positiva. Esa probabilidad puede usarse para ordenar casos, priorizar acciones, ajustar umbrales o tomar decisiones con diferentes niveles de exigencia.

Por ejemplo, en abandono de clientes, una empresa puede ordenar a sus clientes según la probabilidad estimada de abandono y contactar primero a quienes presentan mayor riesgo. En riesgo crediticio, una institución puede analizar solicitudes con distintos niveles de probabilidad estimada de incumplimiento. En detección de casos que requieren revisión, un equipo puede priorizar aquellos con mayor probabilidad estimada. En todos estos escenarios, la salida probabilística aporta más información que una etiqueta aislada.

Otra ventaja importante es su relativa interpretabilidad. La regresión logística aprende coeficientes para las variables de entrada. Esos coeficientes permiten analizar qué variables empujan la probabilidad estimada hacia la clase positiva y cuáles la empujan en sentido contrario. Esta lectura no debe hacerse de manera automática ni causal, pero ofrece una ventana útil para comprender el comportamiento del modelo.

En comparación con KNN, esta diferencia es clara. KNN clasifica según vecinos cercanos; puede explicar una predicción individual mostrando qué casos cercanos

influyeron en ella. La regresión logística, en cambio, ofrece una estructura global más resumida: una combinación de variables con pesos aprendidos. Esa estructura puede facilitar la comunicación del modelo, especialmente cuando necesitamos explicar qué señales generales está usando.

También es un modelo relativamente simple desde el punto de vista computacional. Una vez entrenado, predecir nuevos casos suele ser rápido. El modelo no necesita comparar cada nuevo caso contra todos los registros de entrenamiento, como ocurre en KNN. Solo aplica la combinación de variables aprendida, transforma el resultado en una probabilidad y luego, si corresponde, aplica un umbral para obtener la clase final.

Esto puede ser una ventaja práctica cuando el volumen de datos crece o cuando necesitamos hacer muchas predicciones. En sistemas que clasifican solicitudes, clientes, transacciones o registros de manera frecuente, la rapidez de predicción puede ser importante. La regresión logística suele ser eficiente y fácil de integrar en flujos de trabajo.

Otra ventaja es que funciona bien como modelo base o modelo de referencia. En muchos proyectos de clasificación, entrenar una regresión logística inicial permite tener un punto de comparación razonable. Si un modelo más complejo no supera claramente a una regresión logística bien preparada, tal vez la complejidad adicional no esté justificada. Por eso, aunque existan modelos más avanzados, la regresión logística sigue siendo una herramienta muy valiosa.

También ayuda a detectar problemas en los datos o en la formulación del problema. Si una regresión logística simple obtiene resultados razonables, puede indicar que existe una señal relativamente clara en las variables. Si obtiene resultados muy pobres, puede ser una señal para revisar la variable objetivo, las variables predictoras, la calidad de los datos, el preprocesamiento o la separación entre entrenamiento y prueba.

La regresión logística también se adapta bien a datos tabulares. Muchos problemas prácticos de machine learning comienzan con tablas: una fila por cliente, solicitud, transacción, estudiante, producto o registro; una columna por variable. En ese contexto, la regresión logística suele ser una opción natural, especialmente cuando combinamos variables numéricas y categóricas correctamente preprocesadas.

Con un `Pipeline`, podemos integrar imputación de valores faltantes, escalado de variables numéricas, codificación de variables categóricas y el modelo de regresión logística en un flujo ordenado. Esto permite entrenar y evaluar el modelo evitando errores comunes de preprocesamiento y reduciendo el riesgo de fuga de datos.

Otra ventaja es que el modelo permite trabajar de manera clara con el umbral de decisión. Como la salida es una probabilidad, podemos analizar qué ocurre cuando el umbral cambia. Esto resulta muy útil en problemas donde los errores tienen costos distintos. Podemos preguntarnos si conviene ser más sensible para detectar más casos positivos o más estricto para reducir falsas alarmas.

Por ejemplo, en un problema de detección de riesgo, un equipo puede usar un umbral bajo para generar más alertas y luego revisar manualmente los casos. En otro contexto, puede usar un umbral alto para actuar solo cuando la probabilidad estimada sea muy elevada. La regresión logística facilita esta discusión porque separa la estimación probabilística de la decisión final.

También suele ser más fácil de comunicar que modelos más complejos. Decir que el modelo estima una probabilidad a partir de una combinación de variables y que luego se aplica un umbral puede resultar comprensible para muchas audiencias. Esta claridad no elimina la necesidad de explicar limitaciones, métricas y errores, pero ayuda a construir confianza y a discutir el uso del modelo de manera más transparente.

Desde el punto de vista educativo, la regresión logística es especialmente útil porque conecta muchos conceptos fundamentales: variables predictoras, clase positiva, probabilidad, umbral, coeficientes, frontera de decisión, evaluación y trade-off entre tipos de error. Estudiarla permite comprender ideas que luego aparecerán en modelos más avanzados.

Sin embargo, sus ventajas no deben confundirse con una garantía de buen desempeño. La regresión logística tiene supuestos y limitaciones. Puede quedarse corta si la relación entre variables y clase es muy compleja, si las clases están separadas por patrones muy irregulares o si hay interacciones importantes que el modelo no captura. Estas limitaciones serán importantes al decidir cuándo usarla y cómo compararla con otros modelos.

La regresión logística es valiosa porque estima probabilidades, permite ajustar umbrales, ofrece una interpretación mediante coeficientes, predice de manera eficiente y funciona muy bien como modelo base para clasificación. Su sencillez no es una debilidad, sino una de sus principales fortalezas: nos permite construir modelos comprensibles, evaluables y útiles antes de avanzar hacia técnicas más complejas.

Limitaciones

La regresión logística tiene muchas ventajas, pero también presenta limitaciones importantes. Comprenderlas nos ayuda a usarla con criterio y a no esperar de ella más de lo que puede ofrecer.

Una primera limitación es que la regresión logística aprende una relación relativamente simple entre las variables de entrada y la probabilidad estimada. En su forma básica, combina las variables de manera lineal y luego aplica la función sigmoide. Esto significa que el modelo intenta separar las clases mediante una frontera de decisión lineal en el espacio de variables transformado.

En problemas donde esa separación es razonable, la regresión logística puede funcionar muy bien. Pero si las clases se distribuyen de manera muy irregular, o si la frontera real entre ellas es muy compleja, el modelo puede quedarse corto. Hay situaciones en las que una línea, un plano o una combinación lineal no alcanza para capturar el patrón presente en los datos.

Por ejemplo, si queremos clasificar clientes y la probabilidad de abandono depende de combinaciones muy específicas de comportamiento, la regresión logística puede no detectar todos esos matices por sí sola. Tal vez el riesgo aumente mucho solo cuando se combinan inactividad reciente, caída del consumo y reclamos frecuentes. Si esas interacciones no están representadas adecuadamente en las variables, el modelo puede simplificar demasiado el fenómeno.

Esto no significa que la regresión logística sea inútil en esos casos, sino que puede necesitar ayuda. A veces podemos mejorarla creando variables derivadas, incorporando interacciones o transformando algunas columnas. Pero si el problema requiere fronteras muy flexibles, otros modelos pueden adaptarse mejor.

Otra limitación es que la interpretación de los coeficientes puede ser más delicada de lo que parece. Aunque solemos presentar la regresión logística como un modelo interpretable, esa interpretación depende de varios factores: el escalado de las variables, la codificación de categorías, la presencia de variables correlacionadas, el preprocesamiento aplicado y el contexto del problema.

Un coeficiente positivo o negativo no debe leerse como una verdad absoluta. Indica una asociación aprendida por el modelo bajo ciertas condiciones. Si cambiamos las variables incluidas, si modificamos el preprocesamiento o si entrenamos con otros datos, los coeficientes pueden cambiar. Además, como vimos antes, asociación no significa causalidad.

Por ejemplo, en un modelo de riesgo crediticio, una variable puede aparecer asociada con mayor probabilidad de incumplimiento. Pero eso no prueba que esa variable

cause el incumplimiento. Puede estar relacionada con otros factores no observados, con decisiones históricas del sistema o con sesgos presentes en los datos. Interpretar coeficientes exige cuidado técnico y conocimiento del dominio.

La regresión logística también puede verse afectada por variables muy correlacionadas entre sí. Si dos columnas contienen información parecida, el modelo puede repartir el peso entre ellas de una manera inestable o difícil de interpretar. Esto no siempre destruye la capacidad predictiva, pero puede complicar el análisis de la influencia de cada variable.

Imaginemos un dataset con ingresos mensuales, capacidad de pago estimada y monto máximo preaprobado. Es posible que estas variables estén relacionadas. Si aparecen juntas en el modelo, interpretar el coeficiente de una de ellas de manera aislada puede ser engañoso, porque su efecto se entiende considerando que las otras también están presentes.

Otra limitación aparece cuando los datos contienen valores extremos o errores importantes. Como la regresión logística ajusta una combinación de variables, ciertos valores atípicos pueden influir en el aprendizaje, especialmente si el dataset es pequeño o si esos casos tienen etiquetas particulares. Por eso, antes de entrenar, sigue siendo necesario explorar los datos, revisar distribuciones y detectar valores sospechosos.

También debemos recordar que la regresión logística no resuelve por sí sola el problema de las clases desbalanceadas. Si una clase aparece con mucha más frecuencia que otra, el modelo puede tender a favorecer la clase mayoritaria. Por ejemplo, en detección de fraude, detección de incumplimiento o identificación de casos que requieren revisión, la clase positiva puede ser minoritaria. En esos contextos, una buena exactitud general puede ocultar un mal desempeño sobre la clase que más nos interesa.

Esto nos lleva a una idea importante: no alcanza con entrenar una regresión logística y mirar una métrica global. Debemos evaluar qué ocurre con cada clase, qué tipos de errores comete y si el umbral de decisión es adecuado para el problema. La regresión logística facilita el ajuste de umbrales porque produce probabilidades, pero no decide automáticamente cuál es el umbral correcto.

Otra limitación tiene que ver con la calidad de las probabilidades estimadas. Aunque la regresión logística devuelve valores entre 0 y 1, eso no garantiza que esas probabilidades sean perfectas. La calidad de esas estimaciones depende de los datos, de la formulación del problema, de la representatividad del entrenamiento y de si la relación aprendida por el modelo es adecuada. Una probabilidad de 0.80 debe interpretarse como una estimación del modelo, no como una certeza.

Además, la regresión logística puede verse afectada si hay fuga de datos. Si incluimos variables que revelan directa o indirectamente el resultado, el modelo puede parecer excelente durante la evaluación, pero fallar en uso real. Esta limitación no es exclusiva de la regresión logística, pero es importante recordarla porque un modelo simple con datos filtrados puede producir resultados artificialmente altos.

Por ejemplo, si queremos predecir abandono de clientes y dejamos dentro de `X` una columna como `fecha_baja`, `motivo_cancelacion` o `estado_final`, el modelo aprenderá de información que no estaría disponible al momento de anticipar el abandono. La regresión logística no puede detectar por sí sola que una variable no debería usarse. Esa responsabilidad sigue siendo parte del diseño del proyecto.

Otra limitación aparece cuando la relación entre las variables y la clase cambia con el tiempo. Un modelo entrenado con datos históricos puede perder validez si cambian las condiciones del negocio, el comportamiento de los usuarios, las políticas de crédito, los criterios de diagnóstico o la forma de registrar la información. La regresión logística, como cualquier modelo supervisado, aprende del pasado. Si el futuro cambia demasiado, sus predicciones pueden deteriorarse.

También puede quedarse corta cuando hay interacciones complejas entre variables. En su forma básica, el modelo considera el aporte de cada variable dentro de una combinación lineal. Pero muchas veces el efecto de una variable depende del valor de otra. Por ejemplo, una cantidad determinada de reclamos puede tener distinto significado según la antigüedad del cliente, el tipo de plan o el historial previo. Si esas interacciones son importantes y no están representadas en los datos, el modelo puede no capturarlas bien.

Finalmente, aunque la regresión logística sea más interpretable que muchos modelos complejos, no debe confundirse interpretabilidad con simplicidad absoluta. Cuando hay muchas variables, muchas categorías codificadas, transformaciones previas y pipelines extensos, el análisis del modelo puede volverse menos directo. La interpretación sigue siendo posible, pero requiere ordenar cuidadosamente los nombres de variables, revisar las transformaciones y contextualizar los coeficientes.

La regresión logística es un modelo potente como punto de partida, pero tiene límites. Puede no capturar relaciones muy complejas, depende de una buena preparación de datos, puede verse afectada por variables correlacionadas, valores extremos, clases desbalanceadas, fuga de datos y cambios en el tiempo. Sus probabilidades y coeficientes son útiles, pero deben interpretarse con prudencia. Como ocurre con cualquier modelo, su valor depende de que sus supuestos y su nivel de complejidad sean adecuados para el problema que queremos resolver.

Clasificar estimando probabilidades

La regresión logística nos permitió estudiar una forma distinta de clasificar. A diferencia de KNN, que decide mirando vecinos cercanos, la regresión logística aprende una combinación de variables para estimar la probabilidad de pertenecer a una clase. Esa probabilidad luego puede transformarse en una etiqueta final mediante un umbral de decisión.

Esta separación entre probabilidad y clase es una de las ideas más importantes del capítulo. Un modelo puede estimar que un cliente tiene una probabilidad de abandono de 0.72, que una solicitud tiene una probabilidad de incumplimiento de 0.35 o que un caso tiene una probabilidad de requerir revisión de 0.81. Esas probabilidades contienen más información que una etiqueta aislada. Permiten ordenar casos, priorizar acciones, analizar niveles de riesgo y ajustar decisiones según el contexto.

También vimos que la función sigmoide cumple un papel central. La combinación interna de variables puede producir valores numéricos sin límite, pero la sigmoide los transforma en una escala entre 0 y 1. Gracias a esa transformación, la regresión logística puede expresar su salida como una probabilidad estimada. Luego, el umbral de decisión convierte esa probabilidad en una clase.

El umbral más habitual para comenzar es 0.5, pero no debe entenderse como una regla universal. Cambiar el umbral modifica el comportamiento del clasificador. Un umbral más bajo tiende a detectar más casos positivos, pero también puede generar más falsas alarmas. Un umbral más alto vuelve la decisión más exigente, pero puede dejar pasar casos importantes. Elegir un umbral implica pensar en el costo de los errores y en el uso real de las predicciones.

Otra idea importante fue la interpretación de los coeficientes. La regresión logística permite observar qué variables empujan la probabilidad estimada hacia la clase positiva y cuáles la empujan en sentido contrario. Esto puede ser muy útil para comprender el modelo y comunicar sus resultados. Sin embargo, esa interpretación exige cuidado: los coeficientes dependen del preprocesamiento, de la escala de las variables, de la codificación de categorías y de las demás variables incluidas. Además, indican asociaciones aprendidas, no causalidad.

La regresión logística tiene ventajas claras. Es relativamente simple, eficiente, interpretable y muy útil como modelo base. Funciona bien en muchos problemas tabulares y permite trabajar de manera explícita con probabilidades y umbrales. Por eso sigue siendo una herramienta importante, incluso cuando existen modelos más complejos.

Pero también tiene límites. Puede quedarse corta cuando las relaciones entre variables y clases son muy complejas, cuando hay interacciones importantes no representadas, cuando las clases están desbalanceadas o cuando los datos contienen fuga, ruido, sesgos o cambios temporales. Como cualquier modelo, no reemplaza la formulación cuidadosa del problema ni la evaluación rigurosa.

La regresión logística clasifica estimando probabilidades. Su valor está en que permite pasar de las variables de entrada a una probabilidad interpretable y, a partir de ella, a una decisión. Esta forma de pensar será muy importante en los próximos capítulos, cuando evaluemos modelos de clasificación y analicemos no solo cuántas predicciones aciertan, sino también qué tipos de errores cometen.

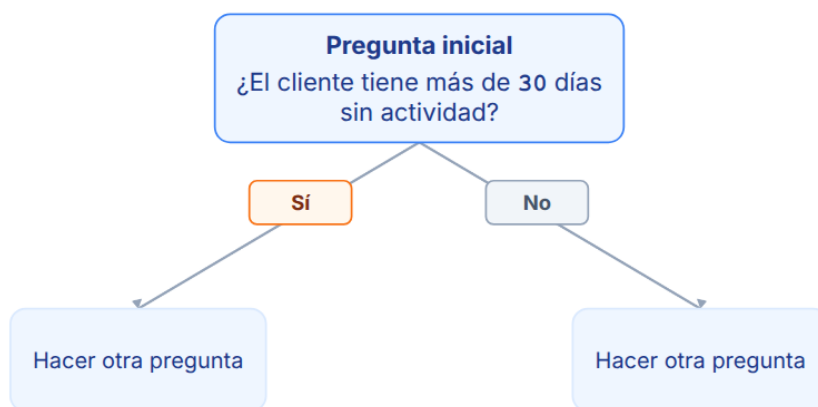
Capítulo 10 · Árboles de Decisión

Decidir mediante preguntas

Un árbol de decisión es un modelo que clasifica casos siguiendo una secuencia de preguntas. En lugar de buscar vecinos cercanos, como hacía KNN, o de estimar una probabilidad mediante una combinación de variables, como hacía la regresión logística, un árbol intenta dividir los datos paso a paso a partir de condiciones simples.

La idea general es muy intuitiva. Para tomar una decisión, el modelo formula una pregunta sobre una variable. Según la respuesta, sigue por un camino u otro. Luego puede hacer otra pregunta, y después otra, hasta llegar a una decisión final.

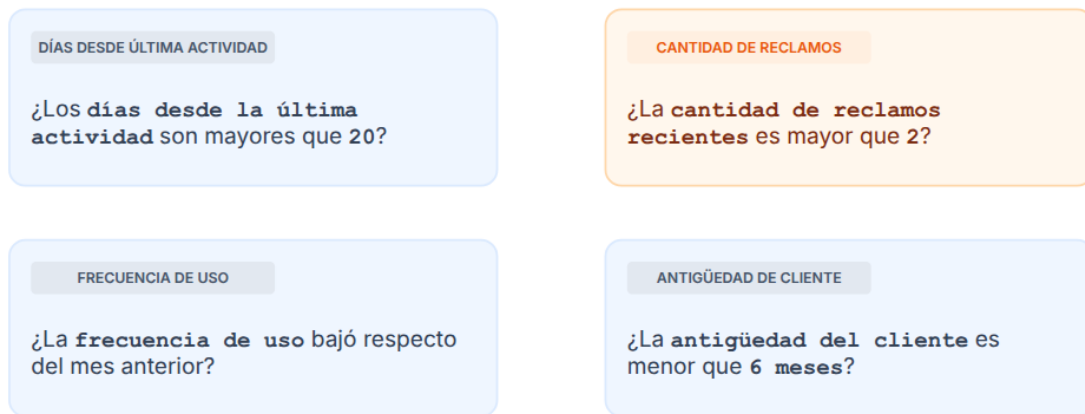
Podemos pensarlo como una serie de bifurcaciones:



Ninguna rama produce todavía una predicción.

Cada pregunta separa los casos en grupos más pequeños. El objetivo del árbol es que, después de varias divisiones, los grupos resultantes queden lo más organizados posible respecto de la clase que queremos predecir.

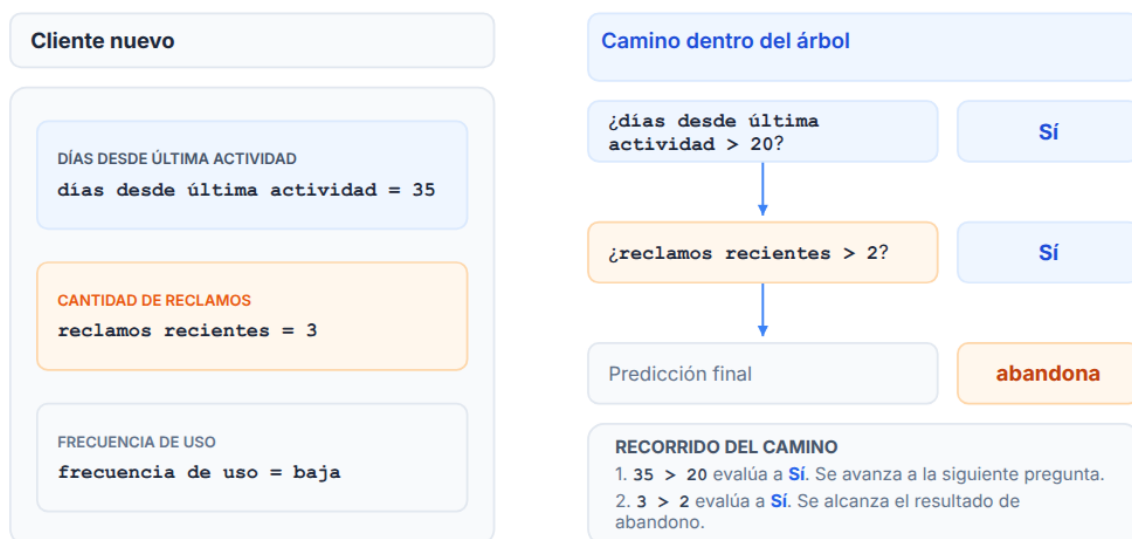
Imaginemos un problema de abandono de clientes. Tenemos variables como días desde la última actividad, cantidad de reclamos recientes, frecuencia de uso y antigüedad como cliente. La variable objetivo indica si el cliente abandonó o permaneció. Un árbol de decisión podría aprender preguntas como estas:



Posible preguntas para un árbol de decisión.

A partir de esas preguntas, el árbol va separando los clientes en grupos. Tal vez descubra que los clientes con muchos días sin actividad y varios reclamos tienen mayor probabilidad de abandono. También podría encontrar que los clientes con pocos días de inactividad y uso frecuente suelen permanecer. La decisión final se construye recorriendo un camino dentro del árbol.

Por ejemplo:



La decisión final se construye recorriendo un camino.

Este tipo de razonamiento se parece bastante a muchas decisiones humanas. Cuando una persona evalúa una situación, no siempre calcula una fórmula compleja. Muchas veces sigue una serie de criterios: si ocurre esto, entonces miro aquello; si además ocurre esto otro, tomo cierta decisión. Los árboles de decisión formalizan esa lógica dentro de un modelo entrenado con datos.

Sin embargo, hay una diferencia importante. En un árbol de decisión de machine learning, las preguntas no las escribe manualmente una persona una por una. El

algoritmo las aprende a partir de los datos de entrenamiento. Busca divisiones que ayuden a separar mejor las clases.

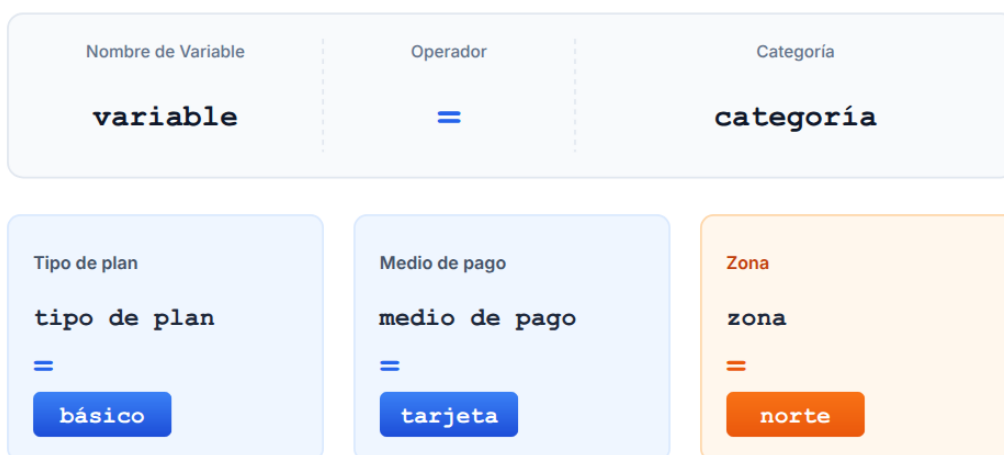
Esto significa que el modelo decide qué variable usar en cada pregunta y qué punto de corte conviene aplicar. Por ejemplo, no parte necesariamente de una regla humana como “más de 30 días sin actividad”. Puede descubrir que, en los datos disponibles, una división útil aparece en 18 días, 24 días o 42 días. El punto de corte depende del patrón que el algoritmo encuentre en el conjunto de entrenamiento.

En una variable numérica, la pregunta suele tener la forma de una comparación:



Variable numérica.

En una variable categórica, la pregunta puede separar categorías. Por ejemplo:



Variable categórica.

En la práctica, cuando trabajamos con `scikit-learn`, muchas variables categóricas suelen transformarse previamente mediante técnicas como One-Hot Encoding. En ese caso, el árbol no ve directamente una categoría como texto, sino columnas numéricas que indican presencia o ausencia de ciertas categorías. Este detalle

pertenece al preprocesamiento, pero conviene recordarlo: el árbol también depende de cómo le presentamos los datos.

La estructura de preguntas sucesivas permite que los árboles capturen relaciones no lineales. Esto quiere decir que no necesitan separar las clases con una única línea recta. Pueden construir reglas por partes, dividiendo distintas regiones del espacio de variables. Esa flexibilidad los vuelve muy útiles para muchos problemas tabulares.

Por ejemplo, el abandono de clientes puede no depender de una sola variable. Un cliente con 25 días sin actividad puede estar en riesgo si además tuvo reclamos recientes, pero no necesariamente si tiene una alta antigüedad y un historial de uso estable. El árbol puede representar este tipo de combinación mediante preguntas encadenadas.



El árbol puede representar este tipo de combinación mediante preguntas encadenadas.

Esta capacidad de combinar condiciones es una de las razones por las que los árboles resultan atractivos. No obligan a que una variable tenga el mismo efecto en todos los casos. El significado de una condición puede depender del camino recorrido antes.

Un mismo valor puede interpretarse de forma diferente según otras variables. Por ejemplo, una baja frecuencia de uso podría ser preocupante en un cliente nuevo, pero menos grave en un cliente antiguo con comportamiento históricamente irregular. Un árbol puede dividir primero por antigüedad y luego aplicar preguntas diferentes en cada grupo.

Esto hace que los árboles sean más flexibles que algunos modelos lineales simples. Mientras la regresión logística básica aprende una combinación global de variables, el árbol puede construir reglas locales en distintas partes de los datos. Esa flexibilidad puede ser una ventaja, pero también puede convertirse en un problema si

el árbol crece demasiado y empieza a aprender detalles particulares del conjunto de entrenamiento.

Por ahora, lo importante es comprender la intuición básica: un árbol de decisión clasifica formulando preguntas sucesivas. Cada respuesta dirige el caso hacia una rama diferente, hasta llegar a una predicción final.

Podemos resumir su funcionamiento así:



Cada respuesta dirige el caso hacia una rama diferente.

Esta forma de clasificar tiene una ventaja inmediata: suele ser fácil de explicar. Podemos seguir el camino que tomó un caso y ver qué condiciones llevaron a la predicción. Esto no significa que todos los árboles sean simples o fáciles de interpretar. Un árbol muy profundo puede volverse tan grande que resulte difícil de leer. Pero, en principio, la lógica de un árbol es más transparente que la de muchos otros modelos.

En un problema de aprobación de créditos, por ejemplo, un árbol podría clasificar una solicitud siguiendo preguntas sobre relación cuota-ingreso, historial de pagos, monto solicitado y antigüedad laboral. En un problema educativo, podría clasificar estudiantes en riesgo a partir de asistencia, entregas pendientes, participación reciente y resultados previos. En un sistema de derivación de reclamos, podría decidir el área correspondiente según palabras clave, tipo de usuario, canal de contacto o categoría seleccionada.

En todos los casos, el árbol convierte los datos en una secuencia de decisiones condicionales. No inventa la respuesta desde cero: aprende preguntas que, según los datos históricos, ayudan a separar las clases.

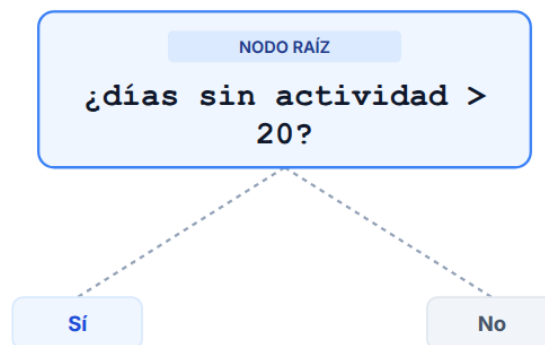
Un árbol de decisión clasifica casos mediante preguntas encadenadas. Cada pregunta divide los datos en grupos más pequeños, y cada respuesta conduce por una rama diferente hasta llegar a una predicción. Esta lógica lo convierte en un modelo intuitivo, visual e interpretable, especialmente útil para comprender cómo un modelo puede tomar decisiones a partir de reglas aprendidas desde los datos.

Nodos, ramas y hojas

Para comprender cómo funciona un árbol de decisión, necesitamos conocer las partes que forman su estructura. Los términos más importantes son **nodos**, **ramas** y **hojas**. Estos nombres ayudan a describir el recorrido que hace un caso desde el inicio del árbol hasta la predicción final.

Un árbol comienza en un primer nodo, llamado **nodo raíz**. Este nodo contiene la primera pregunta que el modelo utiliza para separar los datos. A partir de esa pregunta, los casos se dividen en dos o más caminos posibles. Cada camino recibe el nombre de **rama**.

Podemos imaginarlo así:



Un árbol comienza en un primer nodo, llamado nodo raíz.

El nodo raíz es importante porque representa la primera división del conjunto de datos. No se elige al azar. Durante el entrenamiento, el algoritmo busca una pregunta que permita separar los casos de manera útil para la predicción. Si estamos clasificando abandono de clientes, buscará una división que ayude a distinguir entre clientes que abandonan y clientes que permanecen.

Después del nodo raíz pueden aparecer otros nodos. A estos se los suele llamar **nodos internos**. Cada nodo interno contiene una nueva pregunta. Esa pregunta se aplica solamente a los casos que llegaron hasta allí.

Por ejemplo:



Después del nodo raíz pueden aparecer otros nodos.

En este esquema, el árbol primero pregunta por los días sin actividad. Si la respuesta es “sí”, sigue por una rama y pregunta por los reclamos recientes. Si la respuesta es “no”, sigue por otra rama y pregunta por la frecuencia de uso. Es decir, las preguntas posteriores dependen del camino recorrido.

Esta es una característica fundamental de los árboles de decisión. No todos los casos responden las mismas preguntas. El árbol aplica preguntas diferentes según las respuestas anteriores. Por eso, un árbol puede construir reglas específicas para distintos grupos de casos.

Las **ramas** son los caminos que salen de cada nodo. Representan las respuestas posibles a una pregunta. En un árbol binario, cada nodo suele dividirse en dos ramas: una para los casos que cumplen la condición y otra para los que no la cumplen.

Por ejemplo:



Las ramas son los caminos que salen de cada nodo.

Si un cliente tiene 3 reclamos recientes, seguirá la rama “Sí”. Si tiene 0, 1 o 2 reclamos, seguirá la rama “No”. De esta manera, cada rama conduce a otro nodo o a una decisión final.

Las **hojas**, también llamadas **nodos terminales**, son los puntos finales del árbol. En una hoja ya no se hacen más preguntas. Allí el modelo produce una predicción.



Las hojas son los puntos finales del árbol.

En este ejemplo, algunas hojas predicen “abandona” y otras predicen “permanece”. Un caso nuevo recorre el árbol desde la raíz hasta una hoja. La hoja a la que llega determina la clase predicha.

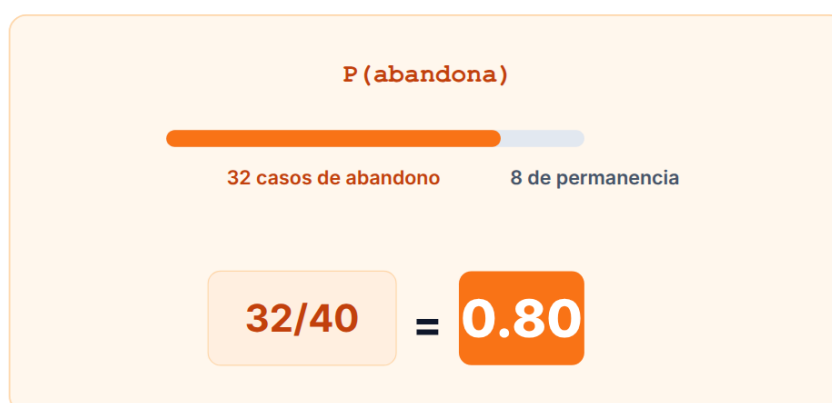
Una hoja no necesariamente contiene un único caso de entrenamiento. Generalmente representa un grupo de casos que llegaron al mismo punto después de responder una serie de preguntas. La predicción de la hoja se decide observando las clases de esos casos durante el entrenamiento.

Supongamos que en una hoja llegaron 40 clientes del conjunto de entrenamiento. De esos 40, 32 abandonaron y 8 permanecieron. En un problema de clasificación, el árbol podría asignar a esa hoja la clase mayoritaria: “abandona”.



Predicción: "abandona".

También puede estimar una proporción o probabilidad basada en esa distribución. En este caso, la hoja tiene 32 abandonos sobre 40 casos, es decir, una proporción de 0.80. Esto puede interpretarse como una probabilidad estimada de abandono dentro de esa región del árbol.



32 abandonos sobre 40 casos, una proporción de 0.80.

Esta idea conecta los árboles con lo que ya vimos sobre probabilidades. Aunque muchas veces hablamos de un árbol como una secuencia de decisiones, sus hojas pueden contener información sobre la distribución de clases. La clase final suele ser la más frecuente, pero las proporciones también pueden aportar una idea de confianza.

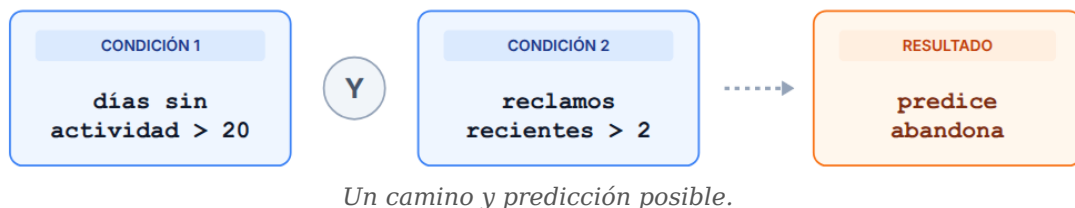
Veamos otro ejemplo con riesgo crediticio. Supongamos que un árbol usa variables como relación cuota-ingreso, historial de atrasos y monto solicitado. Una hoja podría quedar formada por solicitudes con alta relación cuota-ingreso y varios atrasos previos. Si la mayoría de las solicitudes históricas en esa hoja terminó en incumplimiento, el árbol clasificará nuevas solicitudes que lleguen allí como de mayor riesgo.



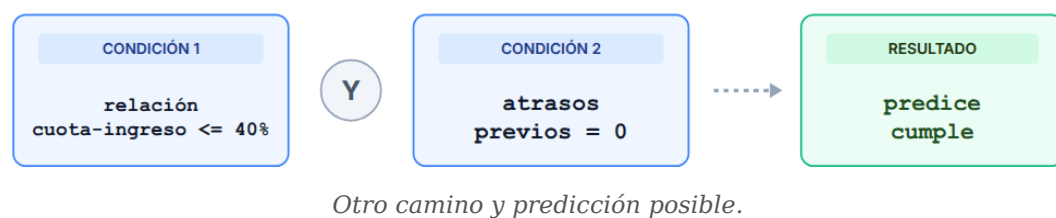
Otra hoja podría representar solicitudes con baja relación cuota-ingreso, sin atrasos previos y monto solicitado moderado. Si la mayoría de esos casos cumplió, la predicción será “cumple”.

Cada hoja resume, entonces, una región de los datos definida por las preguntas anteriores. El camino desde la raíz hasta la hoja puede leerse como una regla.

Por ejemplo:



O bien:

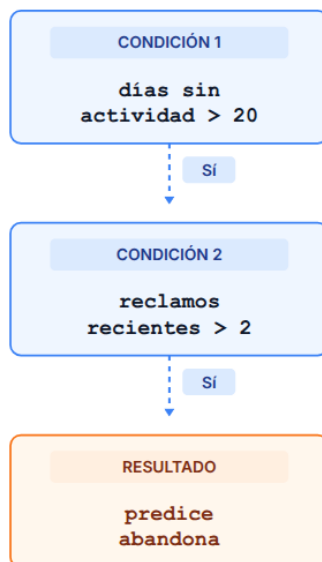


Esta posibilidad de leer caminos como reglas es una de las razones por las que los árboles son considerados modelos interpretables. Podemos observar qué preguntas llevaron a una predicción y explicar el resultado en términos relativamente comprensibles.

Sin embargo, esta interpretabilidad depende del tamaño del árbol. Un árbol pequeño, con pocos nodos y hojas, puede ser fácil de leer. Un árbol muy grande, con muchas

divisiones, puede volverse difícil de interpretar. Si tiene demasiadas ramas y hojas, la explicación deja de ser simple.

Por ejemplo, una regla como esta puede ser comprensible:



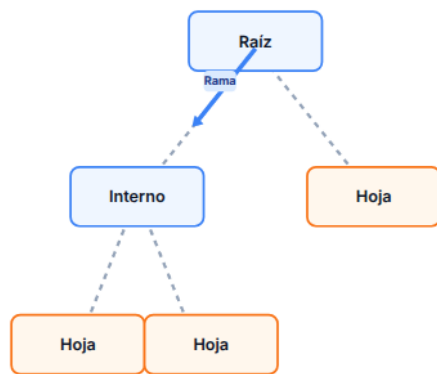
Arbol pequeño, fácil de leer.

Pero si una hoja depende de diez o quince condiciones encadenadas, la interpretación puede volverse menos clara. Además, reglas muy largas pueden indicar que el árbol está capturando detalles demasiado específicos del conjunto de entrenamiento, un problema que estudiaremos al hablar de sobreajuste.

Los nodos también pueden analizarse según cuántos casos llegan a ellos. Al principio, en la raíz, está todo el conjunto de entrenamiento. Después de cada división, los casos se reparten entre ramas. A medida que bajamos en el árbol, cada nodo suele contener menos casos. Las hojas más profundas pueden contener grupos muy pequeños.

Esto es importante porque una hoja basada en muchos casos suele ser más estable que una hoja basada en pocos casos. Si una hoja se formó con 200 ejemplos y la mayoría pertenece a una clase, esa decisión puede ser más confiable que una hoja formada con solo 2 o 3 ejemplos. Cuando un árbol crece demasiado, puede crear hojas muy pequeñas que memorizan casos particulares.

Podemos resumir la estructura de un árbol así:



Partes de un árbol de decisión.

Cuando un caso nuevo entra al árbol, no se compara con todos los casos como en KNN. Tampoco se calcula una combinación global de variables como en la regresión logística. El caso simplemente responde preguntas sucesivas. En cada nodo, una condición decide hacia dónde continúa. Al llegar a una hoja, obtiene su predicción.

La estructura completa puede verse como una forma de dividir el espacio de datos. Cada pregunta parte un grupo en subgrupos. Cada hoja representa una región final. En clasificación, esa región queda asociada con una clase predominante.

Los árboles de decisión están formados por nodos, ramas y hojas. Los nodos contienen preguntas, las ramas representan las respuestas posibles y las hojas producen las predicciones finales. Un caso nuevo recorre el árbol desde la raíz hasta una hoja, siguiendo las condiciones que cumple. Esta estructura permite leer el modelo como una serie de reglas, lo que hace que los árboles sean especialmente útiles para visualizar y explicar decisiones, siempre que no crezcan demasiado.

Criterios de división

Un árbol de decisión aprende haciendo divisiones. En cada nodo, el modelo debe elegir una pregunta que separe los datos en grupos más útiles para predecir la clase. Esa elección no se hace al azar: el algoritmo evalúa distintas divisiones posibles y selecciona aquella que, según cierto criterio, mejora más la organización de los datos.

La idea central es que una buena división debería formar grupos más “puros”. En clasificación, un grupo es más puro cuando sus casos pertenecen mayoritariamente a una misma clase. Si mezclamos muchos casos de clases distintas, la decisión es menos clara. Si logramos separar los datos en grupos donde predomina una clase, el árbol puede clasificar con más seguridad.

Imaginemos un problema de abandono de clientes. En el nodo raíz tenemos todos los clientes del conjunto de entrenamiento. Algunos abandonaron y otros permanecieron. Antes de hacer ninguna pregunta, las clases pueden estar bastante mezcladas. El árbol podría probar una pregunta como:



Partes de un árbol de decisión.

Después de aplicar esa división, tal vez los datos queden así:



La división es útil porque separa bien los casos.

Esta división parece útil porque separa bastante bien los casos. En la rama “Sí” predominan los abandonos. En la rama “No” predominan los clientes que permanecen. No es una separación perfecta, pero los grupos resultantes están más ordenados que el grupo inicial.

En cambio, otra pregunta podría producir una separación menos útil. Por ejemplo:



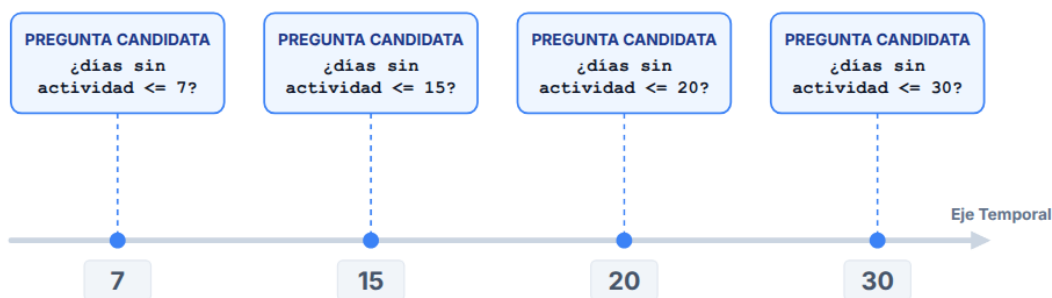
Supongamos que genera estos grupos:



Ambas ramas conservan proporciones similares (40% vs. 60%). La separación es poco útil.

Aquí las proporciones son muy parecidas a las del nodo inicial. La pregunta dividió los datos, pero no ayudó demasiado a separar las clases. Para un árbol de decisión, esta división sería menos atractiva.

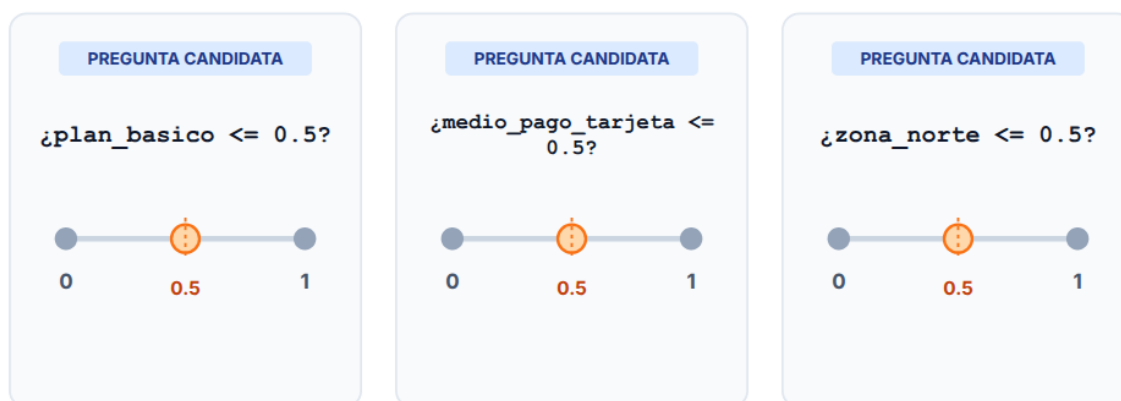
El algoritmo compara muchas preguntas posibles. Para variables numéricas, puede probar distintos puntos de corte. Por ejemplo, si una variable es “días sin actividad”, podría evaluar divisiones como:



El árbol de decisión prueba distintos umbrales de división candidatos a lo largo de una única recta numérica.

Cada punto de corte produce grupos diferentes. El árbol busca aquel que deje los datos mejor separados según la clase objetivo.

Para variables categóricas, la división depende de cómo estén representadas. Si la variable fue transformada con One-Hot Encoding, el árbol puede hacer preguntas sobre columnas binarias, como:



El árbol de decisión puede realizar divisiones basadas en características binarias.

Aunque estas preguntas parezcan menos naturales que preguntar directamente por el texto de la categoría, conceptualmente están evaluando si una condición categórica está presente o no.

Para decidir qué división es mejor, los árboles usan criterios matemáticos. Dos criterios muy comunes son **Gini** y **entropía**. No necesitamos estudiar sus fórmulas en profundidad en este momento, pero sí conviene entender qué intentan medir: ambos evalúan qué tan mezcladas están las clases dentro de un nodo.

Si un nodo contiene casos de una sola clase, es un nodo muy puro. Si contiene una mezcla equilibrada de clases, es un nodo más impuro. El árbol intenta elegir divisiones que reduzcan esa impureza.

Podemos pensarlo así:



Nodos mas o menos puros.

Por ejemplo, un nodo con 50 clientes que abandonan y 50 que permanecen está muy mezclado. No hay una clase predominante clara. En cambio, un nodo con 95 clientes que abandonan y 5 que permanecen está mucho más orientado hacia una clase. Si un caso nuevo llega a ese nodo, la predicción “abandona” parece más razonable.



Mezcla y pureza.

El criterio de división mide justamente esa idea. Una buena pregunta es aquella que, al separar los datos, produce hijos más puros que el nodo original. No alcanza con dividir por dividir. La división tiene que acercar al árbol a una clasificación más clara.

En `scikit-learn`, cuando usamos un árbol de decisión para clasificación, el criterio por defecto suele ser `gini`. También podemos elegir `entropy`. La decisión práctica entre ambos no suele ser lo más importante al comenzar. En muchos problemas producen resultados parecidos. Lo fundamental es comprender que el árbol está buscando divisiones que mejoren la separación de clases.

```
from sklearn.tree import DecisionTreeClassifier

modelo = DecisionTreeClassifier(criterion="gini")
```


O bien:

```
modelo = DecisionTreeClassifier(criterion="entropy")
```

Estos criterios no le dicen al árbol qué variable es importante desde el punto de vista humano. Solo le indican cómo medir la calidad de una separación dentro de los datos de entrenamiento. La interpretación del resultado sigue siendo responsabilidad de quien analiza el modelo.

Esto es importante porque una división puede parecer estadísticamente útil y, aun así, requerir revisión. Tal vez el árbol elija una variable que está muy relacionada con la clase objetivo, pero que no debería usarse porque contiene fuga de datos. Por ejemplo, si queremos predecir abandono y una columna indica “motivo de baja”, el árbol podría usarla rápidamente porque separa muy bien las clases. Pero esa variable no estaría disponible antes de que el cliente abandone. El criterio de división no puede saberlo.

Por eso, aunque el árbol aprenda preguntas automáticamente, no debemos delegar toda la responsabilidad en el algoritmo. Las variables disponibles en X deben haber sido elegidas correctamente antes del entrenamiento. El árbol buscará las mejores divisiones dentro de lo que le permitamos ver.

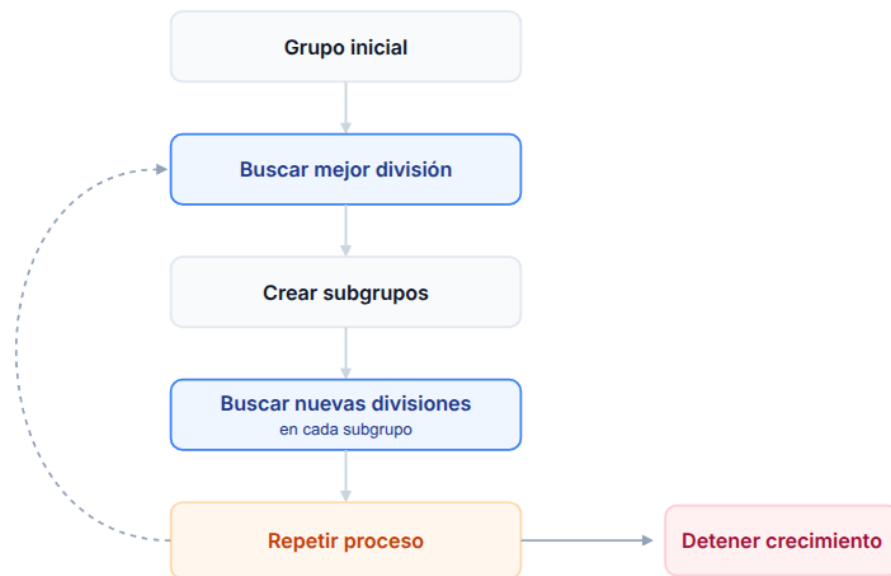
También puede ocurrir que una división sea muy útil en el conjunto de entrenamiento, pero no generalice bien. Si el árbol tiene libertad total, puede encontrar cortes muy específicos que separen perfectamente algunos casos particulares. Por ejemplo, podría usar un punto de corte extraño que funciona bien en los datos históricos, pero que no representa un patrón estable.

Este problema se relaciona con el sobreajuste. Un árbol demasiado profundo puede seguir dividiendo los datos hasta formar hojas muy pequeñas y aparentemente puras. En el entrenamiento, eso puede mejorar mucho los resultados. Pero en datos nuevos, esas reglas demasiado específicas pueden fallar. Por eso, más adelante hablaremos de la profundidad del árbol y de cómo controlar su crecimiento.

Los criterios de división, entonces, ayudan al árbol a decidir qué pregunta hacer en cada nodo. Pero no garantizan por sí solos que el árbol resultante sea el mejor modelo posible. Son una herramienta interna del algoritmo, no un reemplazo de la evaluación con datos de prueba.

Una forma útil de entender el proceso es imaginar que el árbol se construye de manera recursiva. Primero busca la mejor pregunta para dividir todo el conjunto de entrenamiento. Luego toma cada grupo resultante y vuelve a buscar la mejor

pregunta dentro de ese grupo. Después repite el proceso en los nuevos subgrupos, hasta que se cumple alguna condición de detención.



El proceso se repite hasta que se cumple alguna condición específica.

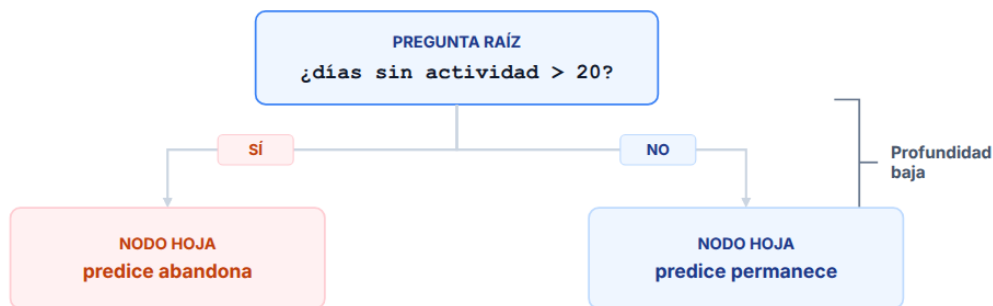
La detención puede ocurrir por varias razones: porque el nodo ya es suficientemente puro, porque tiene pocos casos, porque se alcanzó una profundidad máxima o porque nuevas divisiones ya no aportan una mejora suficiente. Estas condiciones son importantes para evitar que el árbol crezca sin control.

Los criterios de división permiten que un árbol decida qué pregunta hacer en cada nodo. El objetivo es separar los datos en grupos más puros, donde predomine una clase. Criterios como Gini y entropía miden qué tan mezcladas están las clases y ayudan a elegir divisiones útiles. Sin embargo, una buena división matemática no garantiza una buena decisión para el problema real. La calidad del árbol depende también de las variables elegidas, del control del sobreajuste y de una evaluación adecuada sobre datos no vistos.

Profundidad del árbol

La **profundidad** de un árbol de decisión indica cuántos niveles de preguntas puede tener desde el nodo raíz hasta una hoja. Un árbol muy poco profundo hace pocas preguntas antes de decidir. Un árbol muy profundo puede encadenar muchas preguntas y construir reglas más específicas.

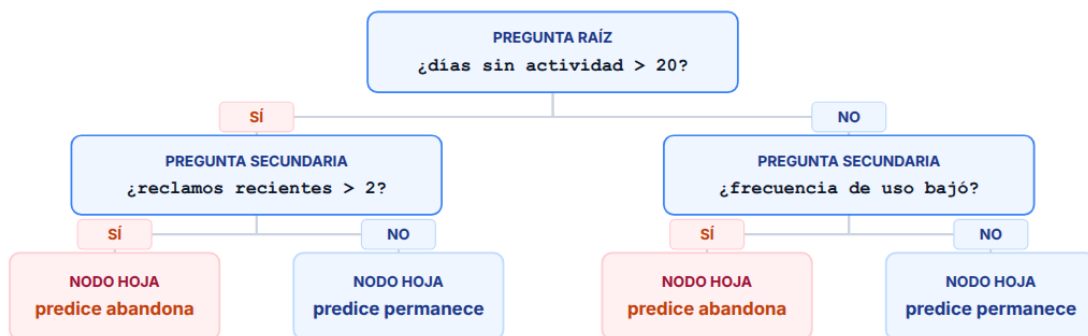
Podemos imaginar un árbol simple de profundidad baja así:



Así podemos imaginar un árbol simple de profundidad baja.

Este árbol tiene una sola pregunta antes de llegar a una hoja. Es fácil de entender, pero quizá demasiado simple para capturar todos los patrones del problema. Tal vez algunos clientes con pocos días de inactividad abandonen por otros motivos, o algunos clientes con muchos días sin actividad permanezcan porque tienen un patrón de uso irregular. Si el árbol solo puede hacer una pregunta, no tendrá forma de distinguir esos casos.

Un árbol más profundo puede refinar la decisión:



Un árbol más profundo puede refinar la decisión.

Ahora el modelo no decide solamente por una variable. Primero separa por días de inactividad, pero luego aplica preguntas distintas dentro de cada grupo. Esto le permite capturar relaciones más detalladas.

La profundidad controla, entonces, el grado de complejidad del árbol. Un árbol de poca profundidad produce reglas generales. Un árbol de mucha profundidad puede producir reglas más específicas.



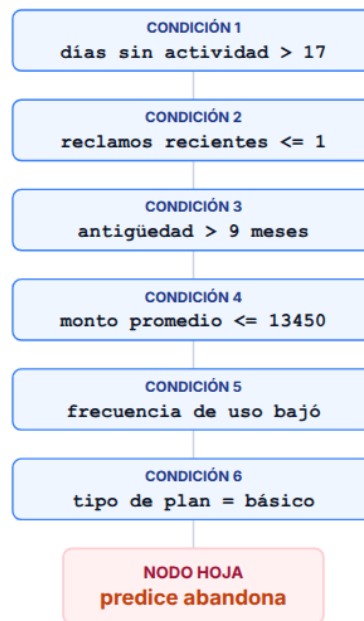
Arbol simple vs. árbol profundo.

Esta diferencia tiene consecuencias importantes. Si el árbol es demasiado simple, puede no captar patrones relevantes de los datos. En ese caso, cometerá errores porque sus reglas son insuficientes. Esto se parece a lo que más adelante llamaremos **subajuste**: el modelo no tiene suficiente capacidad para representar la relación entre las variables y la clase.

Por ejemplo, si el abandono de clientes depende de varias combinaciones de comportamiento, un árbol con una sola pregunta puede quedarse corto. Tal vez necesite considerar inactividad, reclamos, caída de uso, antigüedad y tipo de plan. Si no le permitimos crecer lo suficiente, el modelo tomará decisiones demasiado generales.

Pero el problema contrario también es muy importante. Si dejamos que el árbol crezca demasiado, puede empezar a crear reglas muy específicas para casos particulares del conjunto de entrenamiento. Puede dividir una y otra vez hasta formar hojas con muy pocos ejemplos, incluso con un solo caso. En entrenamiento, eso puede producir resultados aparentemente excelentes. Pero en datos nuevos, esas reglas pueden fallar.

Imaginemos una hoja construida a partir de una regla muy larga:



Se puede llegar a una hoja a través de una regla muy larga.

Esta regla puede haber funcionado muy bien para unos pocos clientes del conjunto de entrenamiento. Pero quizá no represente un patrón general. Tal vez sea una coincidencia, ruido o una particularidad de esos datos. Si el árbol aprende demasiadas reglas de este tipo, puede perder capacidad de generalización.

La profundidad del árbol está directamente relacionada con este equilibrio. Necesitamos un árbol lo suficientemente profundo como para capturar patrones útiles, pero no tan profundo como para memorizar detalles accidentales.

En `scikit-learn`, uno de los hiperparámetros más importantes para controlar esto es `max_depth`. Este parámetro limita la profundidad máxima del árbol.

```
from sklearn.tree import DecisionTreeClassifier

modelo = DecisionTreeClassifier(max_depth=3, random_state=42)
```

Con `max_depth=3`, el árbol puede hacer como máximo tres niveles de preguntas desde la raíz hasta una hoja. Esto restringe su complejidad y puede ayudar a evitar árboles demasiado grandes.

Si no limitamos la profundidad, el árbol puede crecer hasta que cumpla otros criterios de detención. En algunos datasets, esto puede producir árboles muy profundos, con muchas hojas y reglas difíciles de interpretar. Por eso, en la práctica, rara vez conviene aceptar sin análisis un árbol completamente libre.

La profundidad también afecta la interpretabilidad. Un árbol pequeño puede dibujarse y leerse con relativa facilidad. Podemos seguir sus ramas, observar sus

preguntas y explicar por qué un caso llegó a cierta predicción. En cambio, un árbol con muchos niveles puede volverse tan grande que deja de ser útil como explicación.

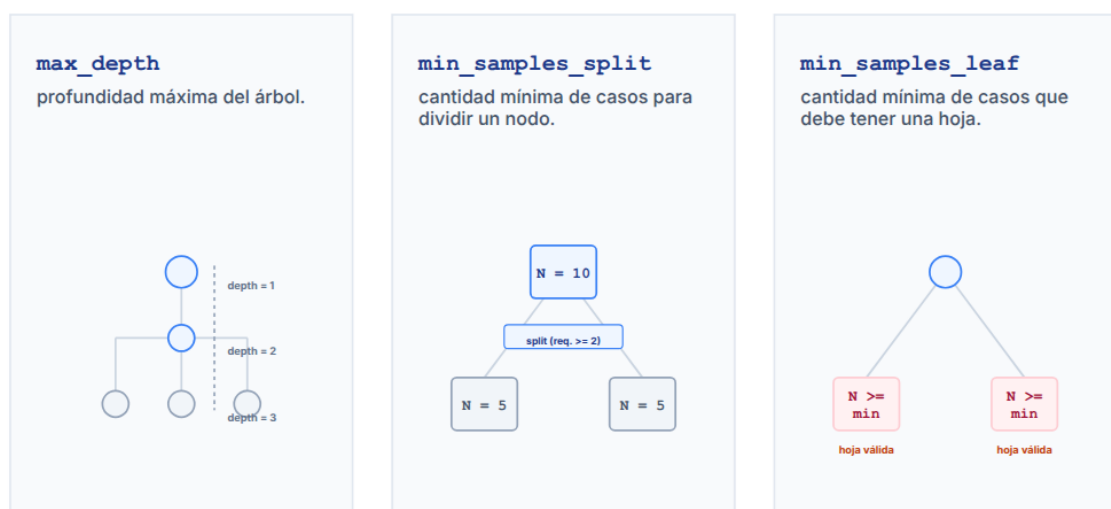
Esto es importante porque una de las ventajas de los árboles es su transparencia. Pero esa ventaja se pierde si el árbol crece sin control. Un árbol de tres o cuatro niveles puede ser una herramienta clara de comunicación. Un árbol de veinte niveles puede ser casi imposible de interpretar de manera global.

La profundidad también influye en la frontera de decisión. Un árbol poco profundo genera separaciones más simples. Un árbol profundo puede construir fronteras más irregulares, porque divide el espacio de variables muchas veces. Esto puede ayudar si la estructura real de los datos es compleja, pero también puede producir fronteras excesivamente adaptadas al entrenamiento.

Podemos relacionarlo con lo visto en KNN. Allí, un valor de `k` muy pequeño podía generar decisiones muy sensibles a casos particulares. En árboles, una profundidad muy grande puede tener un efecto parecido: el modelo se ajusta demasiado a los detalles locales del entrenamiento. En ambos casos, la flexibilidad excesiva puede perjudicar la generalización.

Además de `max_depth`, existen otros hiperparámetros que controlan el crecimiento del árbol. Por ejemplo, podemos exigir una cantidad mínima de casos para dividir un nodo o una cantidad mínima de casos para crear una hoja. No necesitamos profundizar en todos ahora, pero conviene saber que el árbol no se controla solamente por profundidad.

Algunos parámetros frecuentes son:



Varios hiperparámetros que controlan el crecimiento del árbol.

Estos parámetros ayudan a evitar que el árbol construya reglas basadas en grupos demasiado pequeños. Si exigimos que cada hoja tenga una cantidad mínima de ejemplos, reducimos la posibilidad de que el árbol memorice casos individuales.

La elección de la profundidad no debe hacerse mirando solamente el desempeño en entrenamiento. Un árbol más profundo casi siempre puede mejorar sus resultados sobre los datos con los que aprendió. Pero eso no significa que sea mejor. Lo importante es evaluar cómo funciona sobre datos de prueba o mediante estrategias de validación.

Por ejemplo, podemos entrenar árboles con distintas profundidades y comparar su desempeño en datos no vistos. Quizás un árbol de profundidad 2 sea demasiado simple, uno de profundidad 4 funcione bien y uno de profundidad 12 obtenga resultados excelentes en entrenamiento pero peores en prueba. Ese patrón indicaría sobreajuste.

En el cuaderno Colab de este capítulo, será muy útil visualizar árboles con distintas profundidades. Ver un árbol pequeño y luego uno más profundo ayuda a comprender cómo crece la cantidad de preguntas, cómo se multiplican las hojas y cómo cambia la interpretación del modelo.

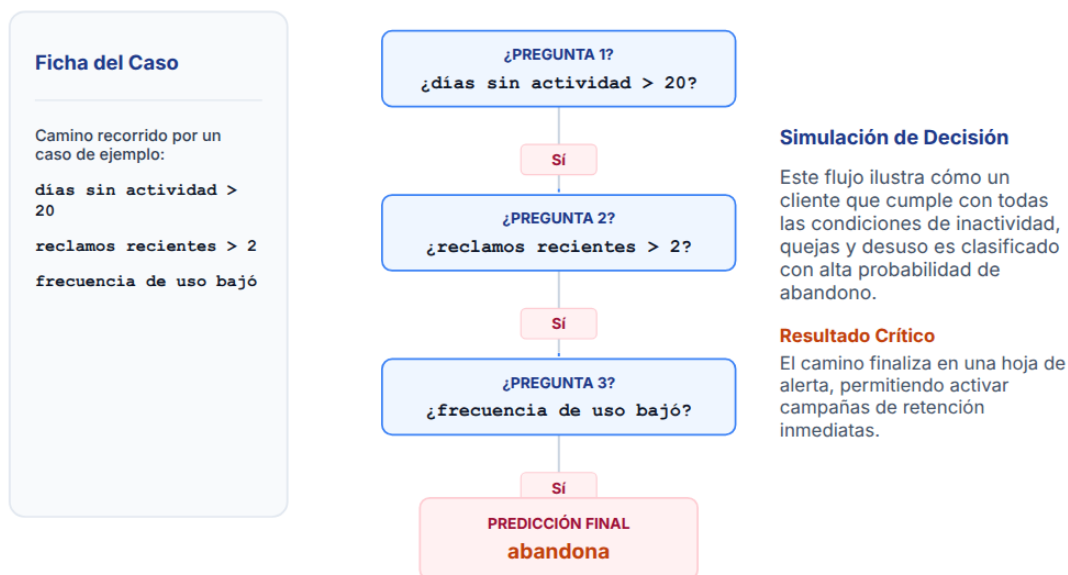
La profundidad del árbol determina cuántos niveles de preguntas puede hacer antes de llegar a una predicción. Una profundidad baja produce modelos simples, fáciles de interpretar, pero posiblemente insuficientes. Una profundidad alta permite capturar patrones más complejos, pero aumenta el riesgo de sobreajuste y reduce la claridad del modelo. Controlar la profundidad es una de las decisiones más importantes al trabajar con árboles de decisión.

Interpretabilidad

Una de las características más atractivas de los árboles de decisión es que suelen ser más fáciles de interpretar que otros modelos. Su estructura se parece a una serie de reglas condicionales: si ocurre cierta condición, seguimos por una rama; si no ocurre, seguimos por otra. Al final del recorrido, llegamos a una predicción.

Esta forma de funcionamiento permite mirar el modelo y reconstruir, al menos en principio, cómo llegó a una decisión. Para un caso particular, podemos seguir el camino desde el nodo raíz hasta la hoja final y observar qué preguntas respondió el caso en cada paso.

Por ejemplo, en un modelo de abandono de clientes, un caso podría recorrer este camino:

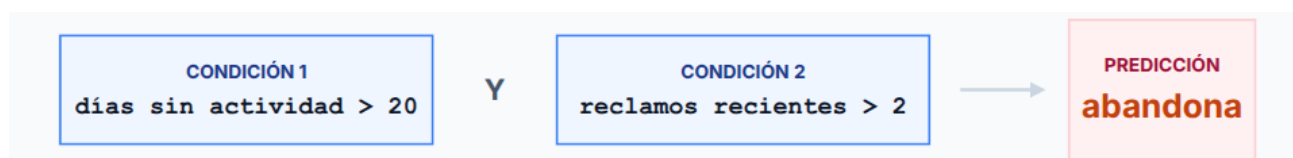


Ejemplo: modelo de abandono de clientes.

Esta explicación resulta bastante comprensible. El modelo no aparece como una caja completamente opaca. Podemos decir que clasificó al cliente como abandono porque tenía muchos días sin actividad, varios reclamos recientes y una caída en su frecuencia de uso. Esa explicación no garantiza que la predicción sea correcta, pero ayuda a entender el criterio que siguió el modelo.

En comparación con KNN, la diferencia es clara. KNN puede justificar una predicción mostrando vecinos cercanos, pero no produce una regla general sencilla. La regresión logística permite interpretar coeficientes, pero exige comprender cómo esos coeficientes influyen en una probabilidad. El árbol de decisión, en cambio, puede representarse como un conjunto de preguntas y respuestas. Esto lo vuelve especialmente útil para comunicar modelos a personas que no necesariamente tienen formación técnica.

Otra ventaja interpretativa es que cada camino desde la raíz hasta una hoja puede leerse como una regla. Por ejemplo:

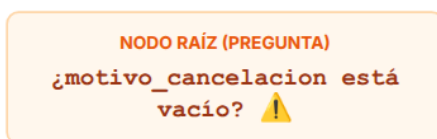


Ejemplo: modelo de abandono de clientes.

Estas reglas permiten discutir el modelo con conocimiento del dominio. Una persona que conoce el negocio, la institución, el sistema educativo o el contexto clínico puede revisar si las condiciones parecen razonables. Puede detectar si el árbol está usando variables esperables, si alguna división resulta sospechosa o si ciertas reglas no tienen sentido práctico.

Esto es muy valioso. La interpretabilidad no sirve solo para explicar el modelo después de entrenarlo. También sirve para revisar si el modelo está aprendiendo patrones razonables. Si un árbol usa como primera división una variable que no debería estar disponible al momento de predecir, eso puede alertarnos sobre una posible fuga de datos. Si usa una variable administrativa sin significado real, tal vez debamos revisar el dataset. Si genera reglas difíciles de justificar, puede ser necesario analizar mejor el problema.

Por ejemplo, supongamos que queremos predecir abandono de clientes y el árbol usa como primera pregunta:



Possible primera pregunta.

Esa división podría separar muy bien los datos históricos, pero sería una señal de alarma. El motivo de cancelación probablemente se conoce después del abandono, no antes. El árbol parecería muy efectivo, pero estaría usando información que no estaría disponible en una predicción real. La interpretabilidad del árbol nos ayuda a detectar este problema.

Sin embargo, debemos tener cuidado: que un árbol sea interpretable no significa que todas sus conclusiones sean correctas o justas. El árbol puede aprender patrones presentes en los datos, pero esos patrones pueden reflejar sesgos, errores de registro, decisiones históricas discutibles o relaciones que no deberían usarse para tomar decisiones.

Por ejemplo, en un problema de aprobación de créditos, un árbol podría usar variables que están asociadas con incumplimiento en los datos históricos. Pero si esas variables reflejan desigualdades previas, criterios de evaluación antiguos o información sensible indirecta, interpretar el árbol no alcanza. También debemos preguntarnos si esas variables son pertinentes y aceptables para el uso previsto.

La interpretabilidad debe ir acompañada de responsabilidad. Un modelo fácil de leer puede producir decisiones problemáticas si los datos o las variables son problemáticos. Ver la regla no convierte automáticamente la regla en buena.

Además, la interpretabilidad de los árboles depende mucho de su tamaño. Un árbol pequeño puede ser claro. Un árbol con pocas preguntas y algunas hojas puede dibujarse, explicarse y revisarse. Pero un árbol muy profundo, con muchas ramas y decenas o cientos de hojas, puede volverse difícil de entender.

Una regla corta como esta es fácil de comunicar:



Regla corta.

Pero una regla muy larga pierde claridad:



Regla muy larga.

Aunque esta regla sea técnicamente legible, puede ser difícil de justificar. Cuando un árbol acumula demasiadas condiciones, la explicación se vuelve menos útil. Además, las reglas largas suelen estar asociadas con hojas pequeñas, que pueden representar casos muy específicos del conjunto de entrenamiento. Esto nos acerca al problema del sobreajuste.

Por eso, en árboles de decisión, interpretabilidad y control de complejidad van juntos. No basta con decir que los árboles son interpretables. Debemos procurar que el árbol tenga un tamaño razonable si queremos usarlo como herramienta explicativa. Parámetros como `max_depth`, `min_samples_leaf` y `min_samples_split` no solo afectan el desempeño, sino también la claridad del modelo.

También es importante distinguir entre interpretación global e interpretación local. La interpretación global intenta comprender el modelo completo: qué variables usa, qué divisiones hace, qué reglas aparecen y cómo organiza el espacio de datos. La

interpretación local se concentra en un caso particular: qué camino siguió ese caso y por qué llegó a cierta hoja.

Un árbol pequeño puede ofrecer ambas lecturas. Podemos entender el modelo completo y también explicar casos individuales. En un árbol grande, la interpretación local puede seguir siendo posible, porque todavía podemos recorrer el camino de un caso. Pero la interpretación global puede volverse difícil, porque el árbol completo tiene demasiadas partes.

Los árboles también permiten analizar la importancia de las variables. Si una variable aparece en divisiones cercanas a la raíz o contribuye mucho a separar los datos, puede considerarse importante para el modelo. Sin embargo, esta idea también requiere cautela. La importancia de variables en árboles depende del criterio de división, de las variables disponibles, de la forma en que se codificaron las categorías y de la presencia de variables relacionadas entre sí. No debe interpretarse como prueba causal.

En `scikit-learn`, los árboles pueden visualizarse con herramientas como `plot_tree`, lo que permite observar sus nodos, condiciones, clases y cantidades de casos. En el cuaderno Colab asociado a este capítulo, la visualización será especialmente útil para conectar la teoría con la práctica. Ver el árbol dibujado ayuda a comprender cómo se organizan sus preguntas y cómo llega a sus predicciones.

```
from sklearn.tree import plot_tree

plot_tree(modelo)
```

La visualización, sin embargo, también muestra una advertencia: si el árbol es demasiado grande, el gráfico se vuelve ilegible. Esto no es solo un problema estético. Es una señal de que quizá el modelo perdió una de sus principales ventajas: la posibilidad de ser comprendido con facilidad.

Los árboles de decisión son modelos interpretables porque clasifican mediante preguntas, ramas y hojas que pueden leerse como reglas. Esta estructura permite explicar predicciones, revisar el comportamiento del modelo y detectar posibles problemas en las variables utilizadas. Pero la interpretabilidad depende del tamaño y de la calidad del árbol. Un árbol pequeño puede ser claro y útil; uno demasiado profundo puede volverse difícil de leer y propenso a aprender detalles particulares del entrenamiento. Interpretar un árbol no es solo mirar sus reglas, sino analizarlas con conocimiento del problema, de los datos y de sus posibles consecuencias.

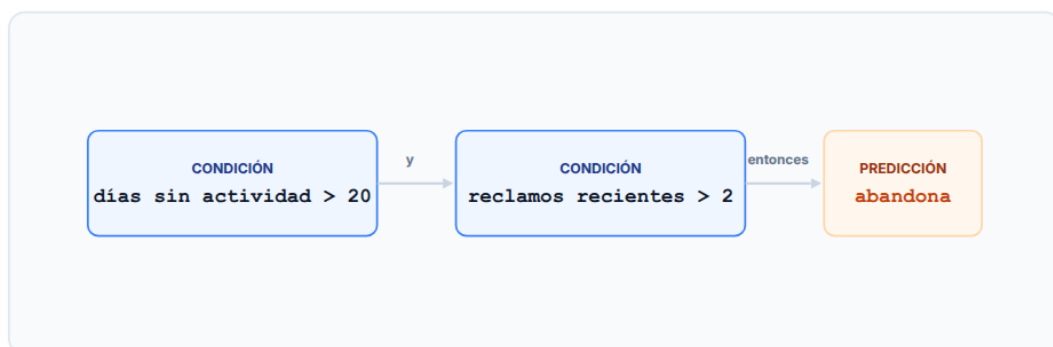
Sobreajuste en árboles

Los árboles de decisión son modelos flexibles. Esa flexibilidad les permite capturar patrones interesantes, construir reglas específicas y adaptarse a distintas regiones de los datos. Pero también introduce un riesgo importante: el **sobreajuste**.

El sobreajuste ocurre cuando un modelo aprende demasiado bien los detalles del conjunto de entrenamiento, incluyendo ruido, casos atípicos o particularidades que no representan patrones generales. En lugar de aprender regularidades útiles, el modelo empieza a memorizar los datos con los que fue entrenado. Como consecuencia, puede funcionar muy bien en entrenamiento y bastante peor sobre datos nuevos.

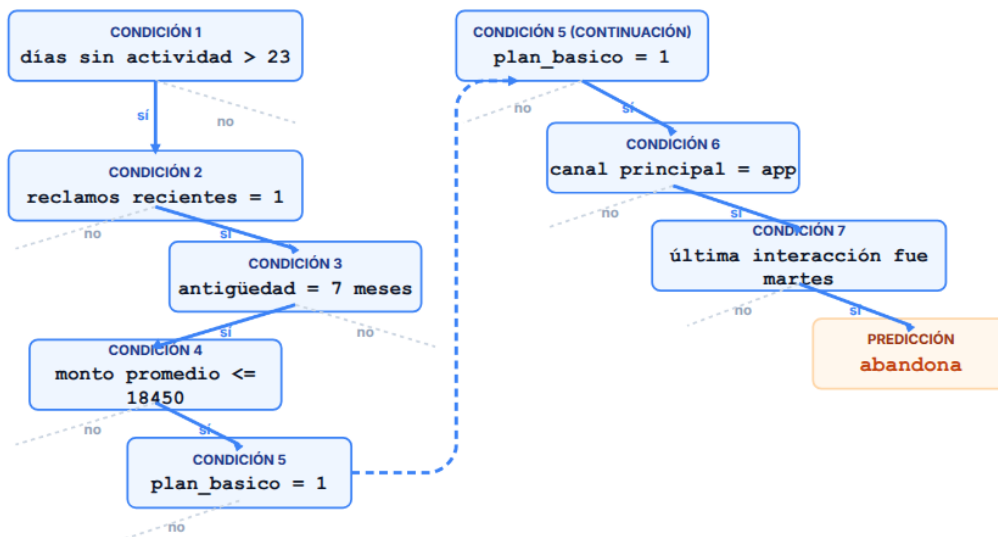
En los árboles de decisión, este problema aparece con mucha claridad. Si dejamos que un árbol crezca sin control, puede seguir dividiendo los datos una y otra vez hasta formar hojas muy pequeñas. Algunas hojas pueden contener pocos casos, o incluso un único caso. En ese punto, el árbol puede clasificar perfectamente muchos ejemplos de entrenamiento, pero sus reglas pueden ser demasiado específicas.

Imaginemos un problema de abandono de clientes. Un árbol pequeño podría aprender una regla general como esta:



Arbol pequeño.

Esa regla es razonable y puede representar un patrón útil. Pero un árbol demasiado profundo podría terminar aprendiendo reglas como esta:



Arbol muy profundo.

Esta regla puede haber separado muy bien algunos casos del conjunto de entrenamiento. Pero quizá no represente una regularidad confiable. Tal vez funciona solo para unos pocos clientes históricos, por casualidad o por ruido. Cuando llegue un cliente nuevo, esa regla puede no generalizar.

El árbol sobreajustado parece muy inteligente porque conoce muchos detalles del entrenamiento. Pero justamente ese es el problema: conoce demasiado bien esos datos particulares y no necesariamente entiende patrones más amplios.

Podemos pensarlo con una analogía. Un estudiante que memoriza todas las respuestas de una guía puede obtener un resultado perfecto si el examen repite exactamente las mismas preguntas. Pero si el examen presenta situaciones nuevas, puede fallar. No aprendió el concepto general; memorizó los ejemplos. Un árbol demasiado profundo puede comportarse de manera parecida.

En términos de evaluación, una señal típica de sobreajuste es una gran diferencia entre el desempeño en entrenamiento y el desempeño en prueba. Si el árbol obtiene resultados excelentes sobre `X_train` pero resultados mucho peores sobre `X_test`, probablemente se adaptó demasiado al conjunto de entrenamiento.



Muy buen resultado en entrenamiento + resultado claramente peor en prueba → posible sobreajuste.

Por ejemplo, un árbol sin restricciones podría obtener 99% de accuracy en entrenamiento y 72% en prueba. Esa diferencia indica que el modelo aprendió reglas que funcionan muy bien para los datos conocidos, pero no se trasladan con la misma calidad a datos no vistos.

El sobreajuste no significa que el modelo “hizo trampa” necesariamente. Puede ocurrir aun cuando no haya fuga de datos. El árbol simplemente usó demasiada libertad para ajustar sus divisiones a detalles específicos. Por eso es tan importante controlar su complejidad.

Uno de los controles más importantes es la profundidad máxima, mediante `max_depth`. Si limitamos la profundidad, impedimos que el árbol encadene demasiadas preguntas. Esto puede reducir el sobreajuste, aunque también puede volver el modelo demasiado simple si el límite es excesivo.

```
modelo = DecisionTreeClassifier(max_depth=3, random_state=42)
```

Con un árbol de profundidad 3, el modelo solo puede hacer una cantidad limitada de divisiones antes de llegar a una hoja. Esto obliga al árbol a aprender reglas más generales. Puede cometer más errores en entrenamiento, pero mejorar su capacidad de generalizar.

Otro control importante es exigir una cantidad mínima de casos para dividir un nodo. Esto se hace con `min_samples_split`. Si un nodo tiene muy pocos casos, tal vez no convenga seguir dividiéndolo, porque las nuevas reglas se basarían en evidencia débil.

```
modelo = DecisionTreeClassifier(  
    max_depth=4,  
    min_samples_split=20,
```

```
random_state=42
)
```

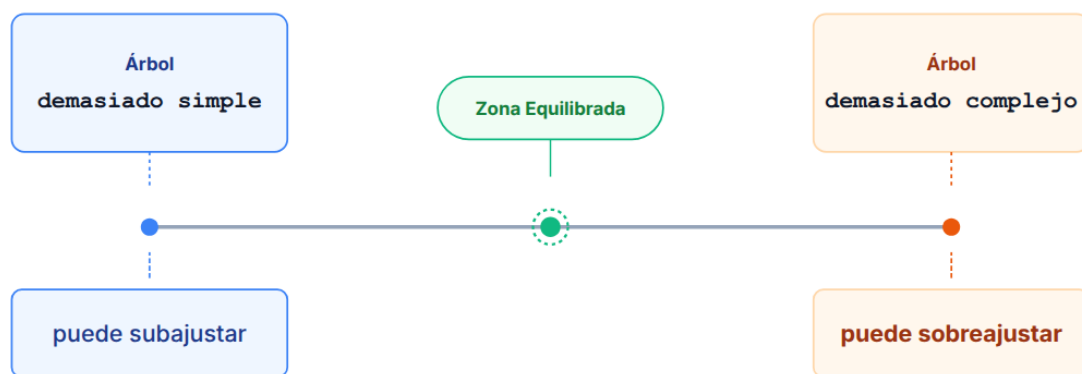
También podemos controlar la cantidad mínima de casos que debe tener una hoja con `min_samples_leaf`. Este parámetro evita que el árbol cree hojas basadas en muy pocos ejemplos.

```
modelo = DecisionTreeClassifier(
    max_depth=4,
    min_samples_leaf=10,
    random_state=42
)
```

Si cada hoja debe tener al menos 10 casos, el árbol no podrá crear una regla final para un único ejemplo aislado. Esto ayuda a que las hojas representen grupos más estables.

Estos parámetros no son simples detalles técnicos. Expresan una idea conceptual: queremos evitar que el árbol construya decisiones basadas en fragmentos demasiado pequeños del entrenamiento. Un árbol que decide a partir de grupos más amplios suele ser menos sensible al ruido.

Sin embargo, controlar el sobreajuste implica un equilibrio. Si restringimos demasiado el árbol, puede producirse el problema contrario: el modelo puede quedar demasiado simple y no capturar patrones importantes. Un árbol de profundidad 1 quizá sea fácil de interpretar, pero insuficiente para un problema donde la clase depende de varias condiciones combinadas.



Controlar el sobreajuste implica un equilibrio.

La tarea consiste en encontrar una complejidad razonable. No queremos un árbol que memorice el entrenamiento, pero tampoco uno que ignore relaciones importantes. Esta tensión entre simplicidad y flexibilidad aparecerá muchas veces en machine learning.

La forma correcta de analizar este equilibrio es evaluar el modelo sobre datos no vistos. No debemos decidir que un árbol es bueno solo porque funciona bien en entrenamiento. Necesitamos comparar su desempeño en prueba, o usar estrategias de validación que estudiaremos más adelante. Si una profundidad mayor mejora entrenamiento pero empeora prueba, probablemente estamos cruzando la frontera hacia el sobreajuste.

También es útil visualizar el árbol. Un árbol muy grande, con muchas ramas y hojas, puede ser una señal de alerta. Si resulta difícil de leer, si contiene reglas muy largas o si muchas hojas tienen pocos casos, conviene revisar si el modelo está creciendo demasiado.

La visualización no reemplaza la evaluación, pero ayuda a comprender el comportamiento del modelo. En el cuaderno Colab de este capítulo, comparar árboles de distinta profundidad permitirá ver cómo cambia la estructura. Un árbol pequeño tendrá pocas reglas y será más claro. Un árbol profundo tendrá más divisiones, más hojas y probablemente una frontera de decisión más irregular.

El sobreajuste en árboles también puede relacionarse con variables de poca calidad. Si incluimos muchas variables irrelevantes, el árbol puede encontrar divisiones accidentales que parecen útiles en entrenamiento. Cuantas más oportunidades tenga de probar cortes y columnas, más posibilidades tiene de encontrar patrones casuales. Por eso, la selección y revisión de variables sigue siendo importante.

Por ejemplo, si agregamos columnas administrativas sin significado real, códigos internos o variables con valores casi únicos por registro, el árbol podría usarlas para separar casos de manera artificial. El resultado puede parecer bueno en entrenamiento, pero fallar al generalizar. Una vez más, el algoritmo trabaja con lo que le damos; no puede saber por sí solo qué variables son conceptualmente válidas.

También debemos tener cuidado con la fuga de datos. Aunque la fuga de datos no es lo mismo que sobreajuste, ambos problemas pueden producir resultados demasiado buenos para ser ciertos. Si un árbol usa una variable que revela el resultado, puede obtener un desempeño excelente, pero ese desempeño no será realista. La interpretabilidad del árbol puede ayudarnos a detectar divisiones sospechosas, pero la prevención debe comenzar antes, al construir **X**.

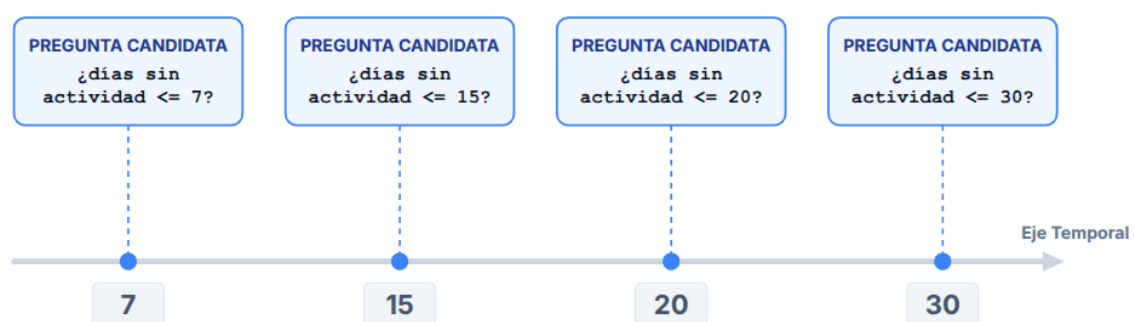
Los árboles de decisión pueden sobreajustar cuando crecen demasiado y aprenden reglas excesivamente específicas del conjunto de entrenamiento. Esto puede generar resultados muy buenos en entrenamiento pero débiles en datos nuevos. Para controlar este riesgo, usamos límites como `max_depth`, `min_samples_split` y `min_samples_leaf`, además de evaluar siempre sobre datos no vistos. Un buen árbol no es el que memoriza cada caso, sino el que encuentra reglas suficientemente generales para clasificar nuevos ejemplos con criterio.

Ventajas y limitaciones

Los árboles de decisión tienen varias ventajas que los convierten en modelos muy atractivos para comenzar a trabajar con clasificación. Al mismo tiempo, presentan limitaciones importantes que debemos conocer para usarlos con criterio. Como ocurre con cualquier modelo, no se trata de decidir si “son buenos” o “son malos” en general, sino de entender en qué condiciones pueden aportar valor y en qué situaciones conviene tener cuidado.

Una de sus principales ventajas es la intuición. Un árbol clasifica mediante preguntas sucesivas, y esa forma de decidir resulta fácil de comprender. No necesitamos imaginar una fórmula abstracta ni una distancia en un espacio de muchas dimensiones. Podemos seguir el recorrido de un caso desde la raíz hasta una hoja y observar qué condiciones llevaron a la predicción.

Por ejemplo, en un problema de abandono de clientes, una predicción puede explicarse mediante un camino como este:



Un árbol clasifica mediante preguntas sucesivas.

Esta estructura permite comunicar el modelo de manera relativamente clara. En contextos educativos, comerciales, administrativos o institucionales, puede ser muy valioso mostrar qué preguntas está usando el árbol y cómo llega a una decisión. Esto hace que los árboles sean especialmente útiles cuando necesitamos explicar el

funcionamiento general del modelo a personas que no necesariamente trabajan con machine learning.

Otra ventaja es que los árboles pueden capturar relaciones no lineales. A diferencia de modelos lineales simples, no necesitan que el efecto de una variable sea igual en todo el espacio de datos. Pueden aplicar una pregunta en una rama y otra pregunta diferente en otra rama. De esta manera, una misma variable puede importar en ciertos grupos de casos y no en otros.

Por ejemplo, la cantidad de reclamos puede ser una señal fuerte de abandono entre clientes nuevos, pero menos importante entre clientes antiguos que históricamente realizaron muchos reclamos sin cancelar el servicio. Un árbol puede dividir primero por antigüedad y luego analizar los reclamos dentro de cada grupo. Esta capacidad de construir reglas por partes le permite representar patrones más flexibles.

Los árboles también manejan bastante bien combinaciones de variables. Un caso puede llegar a una predicción no por una sola condición, sino por una secuencia de condiciones. Esto resulta útil cuando la clase depende de la interacción entre varios factores.

Otra ventaja práctica es que los árboles no requieren escalado de variables numéricas del mismo modo que KNN o algunos modelos lineales. Como el árbol toma decisiones mediante cortes del tipo “variable menor o igual que cierto valor”, no le afecta que una variable esté medida en pesos, otra en años y otra en cantidad de reclamos de la misma forma en que afectaría a un modelo basado en distancias.

Esto no significa que los datos no necesiten preparación. Los valores faltantes, las variables categóricas, los errores de registro y la fuga de datos siguen siendo problemas importantes. Pero el escalado numérico no suele ser una condición central para que un árbol pueda funcionar.

Otra ventaja es que los árboles pueden trabajar tanto con clasificación binaria como con clasificación multiclase. En una hoja, el modelo puede asignar la clase mayoritaria entre varias categorías posibles. Por eso, la lógica de preguntas y ramas se adapta naturalmente a problemas con dos o más clases.

También pueden ofrecer información sobre la importancia de las variables. Si una variable se usa en divisiones relevantes y contribuye mucho a separar las clases, el modelo puede considerarla importante. Esta información puede ayudar a explorar el problema, detectar señales fuertes y comunicar qué columnas parecen influir más en las predicciones.

Sin embargo, esta ventaja debe interpretarse con cuidado. La importancia de variables en un árbol no prueba causalidad. Tampoco garantiza que una variable sea

correcta o ética para usar en una decisión. Solo indica que, dentro del modelo y los datos disponibles, esa variable ayudó a construir divisiones útiles.

La mayor limitación de los árboles de decisión es su tendencia al sobreajuste. Si el árbol crece demasiado, puede aprender reglas muy específicas del conjunto de entrenamiento. Puede crear hojas pequeñas, memorizar casos particulares y obtener resultados excelentes sobre datos conocidos, pero fallar ante datos nuevos.

Este riesgo es especialmente importante porque un árbol puede parecer muy convincente. Sus reglas pueden leerse y explicarse, pero eso no significa que sean generales. Una regla larga y específica puede sonar razonable después de verla, aunque haya surgido de una coincidencia en los datos de entrenamiento.

Por eso, al trabajar con árboles, es fundamental controlar su complejidad. Parámetros como `max_depth`, `min_samples_split` y `min_samples_leaf` ayudan a evitar que el árbol crezca sin control. También es imprescindible evaluar el modelo sobre datos de prueba o mediante estrategias de validación.

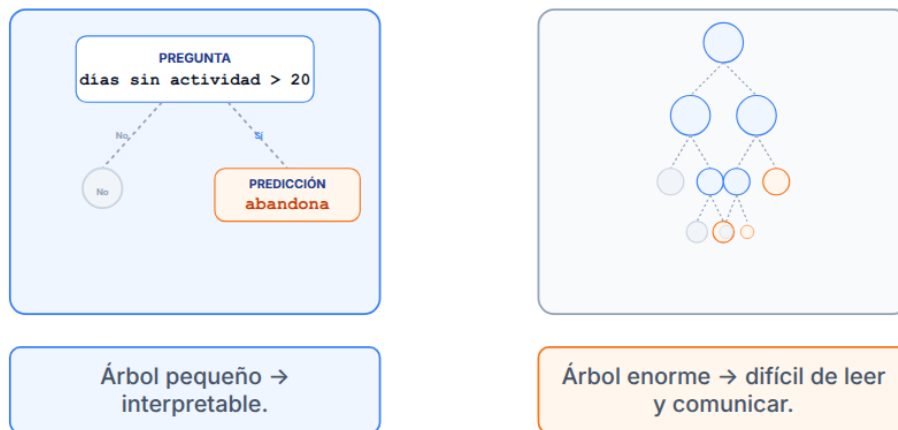
Otra limitación es que los árboles pueden ser inestables. Pequeños cambios en los datos pueden generar árboles bastante diferentes. Si una división alternativa produce una mejora similar, el algoritmo puede elegir un camino distinto, y eso puede modificar varias ramas posteriores. Esta inestabilidad no siempre impide obtener buenas predicciones, pero puede dificultar la interpretación si esperamos que el árbol sea una representación única y definitiva del problema.

También debemos tener cuidado con árboles demasiado simples. Limitar mucho la profundidad puede evitar el sobreajuste, pero también puede impedir que el modelo capture patrones importantes. Un árbol de profundidad baja puede ser fácil de leer, pero tal vez no tenga suficiente capacidad para representar el problema. Como siempre, necesitamos equilibrar simplicidad y desempeño.

Los árboles también pueden verse afectados por variables con muchas posibilidades de división. Algunas variables, especialmente si tienen muchos valores distintos, pueden ofrecer muchas oportunidades para encontrar cortes que parecen útiles en entrenamiento. Esto puede favorecer divisiones accidentales si no se controla el crecimiento del árbol o si no se evalúa correctamente.

Otra limitación aparece con variables categóricas de alta cardinalidad, es decir, variables con muchas categorías diferentes. Si se transforman mediante One-Hot Encoding, pueden generar muchas columnas. El árbol puede terminar usando algunas de esas columnas para construir reglas muy específicas. Esto puede ser útil si las categorías tienen significado real, pero también puede favorecer el sobreajuste si las categorías son demasiado particulares.

Además, aunque los árboles sean interpretables en teoría, no siempre lo son en la práctica. Un árbol pequeño puede ser claro. Un árbol grande puede tener tantas ramas, nodos y hojas que su visualización se vuelve ilegible. En ese caso, perdemos una de sus principales ventajas. Podemos seguir explicando una predicción individual, pero entender el árbol completo se vuelve difícil.



Los árboles no siempre son interpretables en la práctica.

Otra limitación es que los árboles dividen el espacio mediante cortes sucesivos sobre variables individuales. Esto les permite construir fronteras flexibles, pero también puede producir separaciones escalonadas o poco suaves. En algunos problemas, otros modelos pueden representar mejor ciertos patrones. No existe una forma de frontera que sea ideal para todos los casos.

También debemos recordar que los árboles no corrigen problemas de los datos. Si hay fuga de datos, el árbol puede aprovecharla. Si hay sesgos históricos, puede aprenderlos. Si hay errores de registro, puede incorporarlos en sus reglas. Si faltan variables importantes, el árbol no puede inventarlas. La claridad del modelo no elimina la necesidad de una buena preparación y revisión de los datos.

En contextos sensibles, esta advertencia es fundamental. Un árbol de decisión puede producir una regla fácil de leer, pero esa regla puede estar basada en variables problemáticas o en patrones históricos injustos. La interpretabilidad ayuda a detectar y discutir esos problemas, pero no los resuelve automáticamente.

Los árboles de decisión también son importantes porque sirven como base para modelos más potentes. Más adelante estudiaremos Random Forest, que combina muchos árboles para reducir algunas de las limitaciones de un árbol individual, especialmente su tendencia al sobreajuste y su inestabilidad. Pero antes de llegar a esos modelos, es importante comprender bien cómo funciona un árbol simple.

Los árboles de decisión son intuitivos, visuales, interpretables y capaces de capturar relaciones no lineales mediante preguntas sucesivas. No suelen requerir escalado de variables numéricas y pueden aplicarse a clasificación binaria o multiclase. Sin embargo, pueden sobreajustar con facilidad, volverse inestables, perder interpretabilidad si crecen demasiado y aprender patrones problemáticos si los datos contienen errores, sesgos o fuga de información. Su utilidad depende de controlar su complejidad, evaluar sobre datos no vistos e interpretar sus reglas dentro del contexto del problema.

Clasificar siguiendo un camino de decisiones

Los árboles de decisión nos permitieron estudiar una forma de clasificación muy distinta a las anteriores. KNN clasifica observando vecinos cercanos. La regresión logística estima probabilidades a partir de una combinación de variables. El árbol de decisión, en cambio, clasifica siguiendo un camino de preguntas.

Esa diferencia hace que los árboles sean especialmente intuitivos. Cada caso comienza en el nodo raíz, responde una pregunta, sigue una rama, responde otra pregunta y continúa hasta llegar a una hoja. En esa hoja se produce la predicción final. El modelo convierte los datos en una secuencia de decisiones condicionales.

La estructura del árbol se organiza mediante nodos, ramas y hojas. Los nodos contienen preguntas, las ramas representan las respuestas posibles y las hojas contienen las predicciones. Cada camino desde la raíz hasta una hoja puede leerse como una regla. Por eso, los árboles suelen ser más fáciles de visualizar y explicar que otros modelos.

También vimos que el árbol no inventa esas preguntas manualmente. Durante el entrenamiento, el algoritmo busca divisiones que separen mejor las clases. Para hacerlo, utiliza criterios como Gini o entropía, que miden qué tan mezcladas están las clases dentro de un nodo. Una buena división produce grupos más puros, donde predomina una clase.

La profundidad del árbol controla cuántos niveles de preguntas puede hacer. Un árbol poco profundo genera reglas simples y generales. Un árbol más profundo puede capturar patrones más detallados, pero también corre el riesgo de aprender particularidades del conjunto de entrenamiento. Por eso, la profundidad es una decisión clave.

La interpretabilidad es una de las grandes ventajas de los árboles. Podemos recorrer el camino que siguió un caso y ver qué condiciones llevaron a la predicción. También podemos revisar el árbol completo, si no es demasiado grande, para entender qué

variables usa y qué reglas construye. Esta transparencia ayuda a comunicar el modelo y a detectar posibles problemas.

Sin embargo, la interpretabilidad no garantiza que el modelo sea correcto. Un árbol puede aprender reglas basadas en datos con errores, sesgos o fuga de información. También puede construir reglas demasiado específicas si crece sin control. Por eso, interpretar un árbol implica revisar sus condiciones con conocimiento del problema y no aceptar automáticamente todo lo que el modelo aprendió.

El sobreajuste es una limitación central de los árboles de decisión. Un árbol demasiado profundo puede obtener resultados excelentes en entrenamiento, pero funcionar peor con datos nuevos. Para controlar ese riesgo usamos parámetros como `max_depth`, `min_samples_split` y `min_samples_leaf`, además de evaluar siempre sobre datos no vistos.

Los árboles tienen ventajas claras: son intuitivos, visuales, interpretables, no suelen requerir escalado de variables numéricas y pueden capturar relaciones no lineales mediante preguntas sucesivas. Pero también tienen límites: pueden sobreajustar, volverse inestables, perder claridad si crecen demasiado y aprender patrones problemáticos si las variables no fueron bien elegidas.

Un árbol de decisión clasifica siguiendo un camino de decisiones. Su fuerza está en transformar los datos en preguntas comprensibles; su riesgo está en construir demasiadas preguntas y terminar memorizando el entrenamiento. Usado con control y evaluado correctamente, es un modelo muy valioso tanto para aprender machine learning como para construir clasificadores interpretables. Más adelante veremos que esta idea de árbol también será la base de modelos más potentes, como Random Forest.

Capítulo 11 · Accuracy y matriz de confusión

¿Qué mide la accuracy?

Cuando evaluamos un modelo de clasificación, una de las primeras métricas que aparece es la **accuracy**, también llamada exactitud. Su idea es sencilla: mide qué proporción de predicciones fueron correctas sobre el total de casos evaluados.

Si un modelo clasifica 100 casos y acierta 80, su accuracy es 0.80, o 80%. Dicho de otro modo, el modelo acertó 80 de cada 100 predicciones.

La fórmula puede expresarse así:

$$\text{accuracy} = \frac{\text{predicciones correctas}}{\text{total de predicciones}}$$

Accuracy.

Esta métrica compara la clase predicha por el modelo con la clase real registrada en los datos de prueba. Si ambas coinciden, cuenta como acierto. Si no coinciden, cuenta como error.

Por ejemplo, supongamos que entrenamos un modelo para predecir si un cliente abandonará o permanecerá. Evaluamos el modelo sobre 200 clientes del conjunto de prueba. Si el modelo clasifica correctamente a 160 de ellos, la accuracy será:

The diagram illustrates the calculation of accuracy. On the left, a box contains the fraction $\frac{160}{200}$ with the word "accuracy" written above it. An arrow points from this box to a box on the right. The right box is labeled "RESULTADO" and contains the text "accuracy =" followed by the value "0.80" in a large, bold font.

Podemos decir que el modelo tuvo una exactitud del 80%.

Podemos decir que el modelo tuvo una exactitud del 80%. Esa lectura es directa y fácil de comunicar. Por eso la accuracy suele ser una métrica atractiva para comenzar: resume el desempeño general del clasificador en un único número.

En un problema de clasificación binaria, como “abandona” o “permanece”, la accuracy cuenta todos los aciertos sin distinguir a qué clase pertenecen. Si el modelo predice correctamente clientes que abandonan, suma aciertos. Si predice

correctamente clientes que permanecen, también suma aciertos. Ambos tipos de acierto valen lo mismo para esta métrica.

Lo mismo ocurre en clasificación multiclase. Si un modelo clasifica reclamos en categorías como “facturación”, “soporte técnico”, “envíos” y “devoluciones”, la accuracy mide la proporción total de reclamos cuya categoría fue predicha correctamente. No importa cuál sea la clase: cada predicción correcta suma un acierto.

Por ejemplo:

Reclamos correctamente clasificados

$$\frac{410}{500}$$

Total de reclamos evaluados

→

RESULTADO

accuracy = 0.82

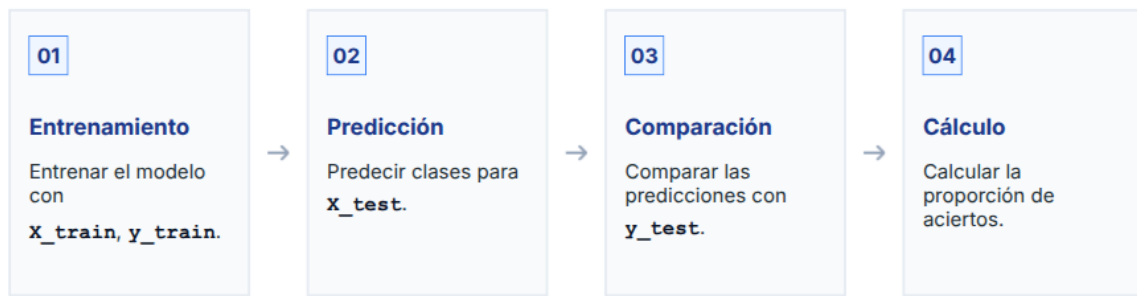
Podemos decir que el modelo tuvo una exactitud del 80%.

En este caso, la accuracy sería del 82%. El número indica que, en general, el modelo acertó en una proporción importante de casos.

La accuracy también puede interpretarse como una primera medida de desempeño global. Nos da una respuesta inicial a la pregunta: “¿cuántas veces acierta el modelo?”. Esa pregunta no es incorrecta. De hecho, en algunos problemas puede ser muy útil. Pero es solo una parte de la evaluación.

Para calcular accuracy correctamente, debemos usar datos que el modelo no haya visto durante el entrenamiento. Si medimos la accuracy sobre los mismos datos con los que entrenamos, el resultado puede ser demasiado optimista. El modelo podría haber aprendido detalles específicos del entrenamiento y no necesariamente generalizar bien. Por eso, la accuracy suele calcularse sobre `X_test` e `y_test`, o mediante estrategias de validación que estudiaremos más adelante.

El flujo básico sería:



Flujo básico.

En `scikit-learn`, muchos modelos tienen un método llamado `score` que, en problemas de clasificación, suele devolver la accuracy. También podemos calcularla explícitamente con `accuracy_score`.

```
from sklearn.metrics import accuracy_score

y_pred = modelo.predict(X_test)

accuracy_score(y_test, y_pred)
```

Aquí `y_test` contiene las clases reales y `y_pred` contiene las clases predichas por el modelo. La función compara ambos valores caso por caso y calcula la proporción de coincidencias.

Aunque el cálculo sea simple, la interpretación exige cuidado. Una accuracy de 0.90 puede parecer excelente, pero su significado depende del problema. Si las clases están balanceadas y los errores tienen costos parecidos, puede indicar un buen desempeño. Pero si las clases están muy desbalanceadas, una accuracy alta puede ser engañosa. Veremos ese problema en detalle más adelante.

Por ahora, conviene retener la idea básica: la accuracy mide aciertos totales. No distingue entre tipos de error, no nos dice qué clase se predice mejor y no muestra si el modelo está fallando en la clase más importante. Solo resume la proporción general de predicciones correctas. La accuracy responde: ¿qué porcentaje total de casos clasificó bien el modelo?

Esa pregunta es útil, pero incompleta. En clasificación, muchas veces necesitamos saber más. Queremos entender qué errores cometió el modelo, si detectó bien la clase positiva, si generó muchas falsas alarmas o si se limitó a predecir la clase más frecuente.

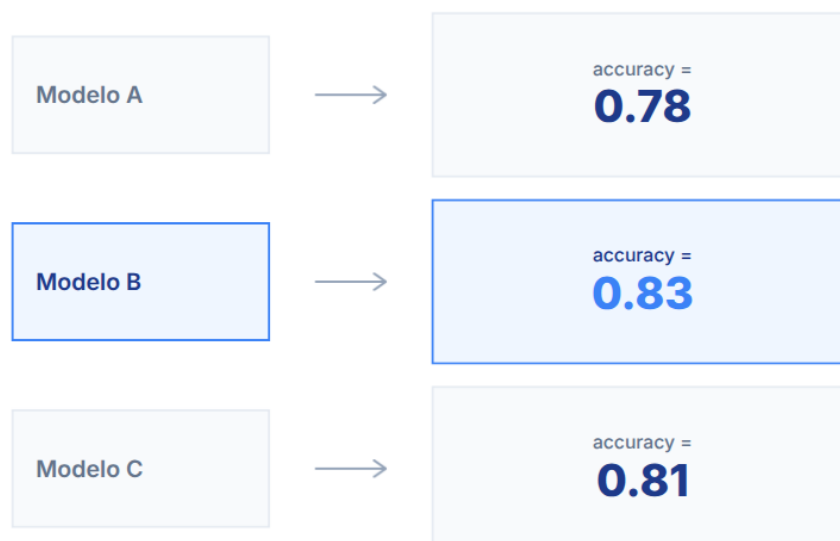
La accuracy es una métrica simple y fácil de interpretar que mide la proporción total de predicciones correctas. Es un buen punto de partida para evaluar clasificadores, especialmente cuando las clases están balanceadas y los errores tienen consecuencias similares. Sin embargo, no alcanza para comprender por completo el comportamiento de un modelo. Para eso necesitaremos mirar la matriz de confusión y analizar los distintos tipos de aciertos y errores.

¿Cuándo puede ser útil?

La accuracy puede ser una métrica útil cuando queremos tener una primera idea general del desempeño de un clasificador. Su principal ventaja es que resulta fácil de calcular, fácil de interpretar y fácil de comunicar. Si decimos que un modelo tiene una accuracy del 85%, la lectura inicial es directa: acertó 85 de cada 100 casos evaluados.

Esa simplicidad la convierte en una buena métrica de entrada. Cuando estamos explorando un problema por primera vez, puede ayudarnos a comparar rápidamente si un modelo funciona mejor que otro o si un cambio en el preprocesamiento produjo una mejora general. También puede servir como una referencia inicial antes de analizar métricas más específicas.

Por ejemplo, si entrenamos tres modelos distintos sobre el mismo conjunto de entrenamiento y los evaluamos sobre el mismo conjunto de prueba, la accuracy puede darnos una primera comparación:

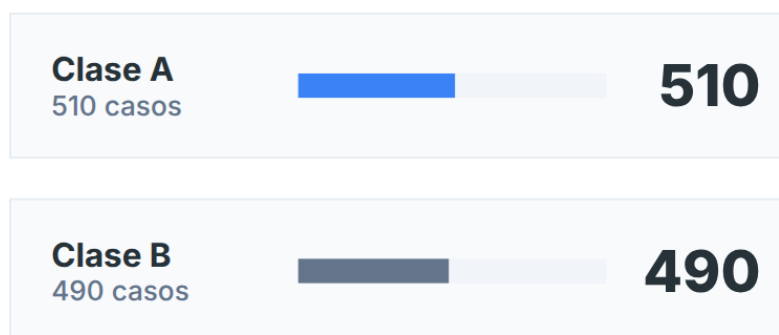


Tres modelos distintos sobre el mismo conjunto de entrenamiento.

En una primera lectura, el Modelo B parece tener mejor desempeño global. Esto no significa que sea automáticamente el mejor para el problema, pero nos da una señal inicial para investigar con más detalle.

La accuracy puede ser especialmente útil cuando las clases están relativamente balanceadas. Esto significa que las categorías aparecen en proporciones parecidas. Por ejemplo, si estamos clasificando casos en dos clases y aproximadamente la mitad pertenece a una clase y la mitad a la otra, la proporción total de aciertos puede representar mejor el desempeño general.

Imaginemos un dataset con 1000 casos:



Dataset con 1000 casos.

En una situación así, un modelo no puede obtener una accuracy muy alta simplemente prediciendo siempre una sola clase. Si siempre predijera “Clase A”, acertaría alrededor del 51%. Para alcanzar una accuracy del 85%, necesitaría clasificar correctamente muchos casos de ambas clases. Por eso, cuando las clases están balanceadas, la accuracy suele ser más informativa que en contextos desbalanceados.

También puede ser útil cuando los errores tienen consecuencias parecidas. Supongamos que un modelo clasifica imágenes de frutas entre manzana, pera y naranja para una actividad educativa. Si confundir una manzana con una pera tiene una importancia similar a confundir una naranja con una manzana, la proporción total de aciertos puede ser una medida razonable para comenzar.

Lo mismo podría ocurrir en ciertos problemas de clasificación de objetos, categorías de productos o tareas de reconocimiento donde ninguna clase es mucho más crítica que otra y donde todos los errores tienen un costo similar. En esos contextos, la accuracy responde una pregunta relevante: ¿qué tan seguido clasifica bien el modelo?

Sin embargo, incluso en estos casos, conviene revisar otras métricas si el problema crece en importancia. La accuracy puede ser útil como resumen general, pero no

muestra qué clases se confunden entre sí. En clasificación multiclase, un modelo puede tener buena accuracy global y aun así fallar bastante en una clase particular. Si esa clase empieza a ser importante, necesitaremos mirar la matriz de confusión.

La accuracy también puede servir como comparación contra una línea de base. Una línea de base es una estrategia simple que usamos como referencia. Por ejemplo, en clasificación, una referencia básica puede ser predecir siempre la clase más frecuente. Si nuestro modelo no supera claramente esa estrategia, probablemente no esté aprendiendo patrones útiles.

Supongamos que tenemos un problema con dos clases balanceadas. La clase A representa el 52% de los casos y la clase B el 48%. Un modelo que siempre predice la clase A tendría una accuracy de 52%. Si entrenamos un modelo y obtiene 75%, podemos decir que mejora claramente esa referencia simple. En este caso, la accuracy ayuda a medir esa mejora.



Modelo ingenuo vs. modelo entrenado.

Esta comparación es importante porque una métrica aislada puede ser difícil de interpretar. Una accuracy del 75% puede parecer buena o regular según el problema. Pero si la referencia básica era 52%, entonces el modelo parece haber aprendido algo. Si la referencia básica era 72%, la mejora sería mucho menos impresionante.

La accuracy también puede ser práctica cuando necesitamos una comunicación inicial para públicos no técnicos. En un informe, decir que un modelo acierta cierta proporción de casos puede ser una forma simple de introducir el desempeño. Luego, esa explicación debería completarse con detalles sobre errores, clases y limitaciones. Pero como punto de entrada, la accuracy puede ayudar.

Por ejemplo, una presentación responsable podría decir:

El modelo clasificó correctamente el 84% de los casos del conjunto de prueba.

Sin embargo, para evaluar si es adecuado, también analizamos la matriz de confusión y el desempeño por clase.

Esta forma de comunicar evita dos extremos. No descarta la accuracy, porque efectivamente aporta información. Pero tampoco la convierte en la única medida del modelo.

En problemas de clasificación multiclase con clases equilibradas, la accuracy también puede ser útil como resumen general. Si tenemos cuatro categorías con cantidades similares y errores de importancia parecida, una accuracy alta puede indicar que el modelo está funcionando razonablemente bien en conjunto. Aun así, la matriz de confusión seguirá siendo necesaria para saber cuáles son las confusiones más frecuentes.

La accuracy también es útil durante etapas tempranas de experimentación. Cuando estamos probando si el flujo completo funciona, si las variables están bien armadas, si el modelo entrena correctamente o si el preprocesamiento no genera errores, la accuracy puede ofrecer una señal rápida. No reemplaza la evaluación final, pero ayuda a orientarnos.

Por ejemplo, si un modelo obtiene una accuracy extremadamente baja, puede haber un problema en la preparación de datos, en la codificación de la variable objetivo, en la separación de entrenamiento y prueba o en el entrenamiento. Si obtiene una accuracy extremadamente alta, también puede ser una señal de alerta: tal vez haya fuga de datos o una variable que revela la respuesta. En ambos casos, la accuracy funciona como un primer indicador que invita a revisar.

La accuracy puede ser útil cuando necesitamos una medida general de aciertos, especialmente si las clases están balanceadas y los errores tienen consecuencias similares. También sirve como primera comparación entre modelos, como referencia frente a una estrategia simple y como forma inicial de comunicación. Pero su utilidad depende del contexto. Debe entenderse como una puerta de entrada a la evaluación, no como una conclusión definitiva.

¿Cuándo puede engañar?

La accuracy puede ser útil, pero también puede llevarnos a conclusiones equivocadas. El problema aparece cuando el porcentaje total de aciertos oculta qué está ocurriendo con cada clase o con cada tipo de error. Un modelo puede tener una accuracy alta y, sin embargo, ser poco útil para el objetivo real del problema.

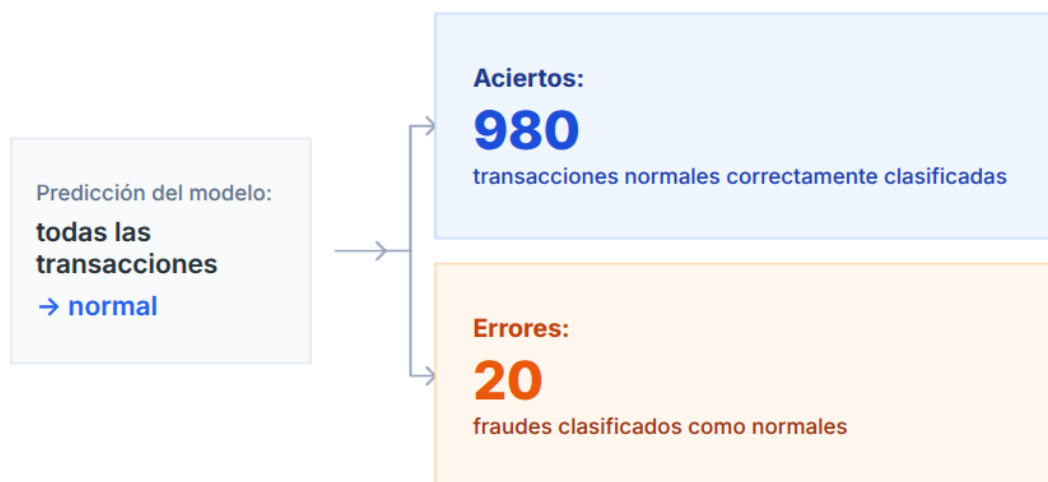
La situación más común ocurre cuando las clases están desbalanceadas. Si una clase aparece mucho más que otra, un modelo puede lograr una accuracy elevada simplemente prediciendo casi siempre la clase mayoritaria. En ese caso, el modelo acierta muchos casos no porque haya aprendido patrones valiosos, sino porque aprovecha la distribución desigual de las clases.

Imaginemos un problema de detección de fraude. Tenemos 1000 transacciones:



Detección de fraude, sobre 1000 casos.

Ahora supongamos que construimos un modelo muy simple que siempre predice “normal”. Ese modelo no detecta ningún fraude. Sin embargo, acertará las 980 transacciones normales.



Predicción del modelo.

La accuracy será:



accuracy del modelo.

Una accuracy del 98% parece excelente. Pero el modelo no detectó ningún caso de la clase que más nos interesaba. Para un sistema de detección de fraude, ese resultado sería inaceptable. La métrica se ve bien, pero el modelo no cumple su objetivo.

Este ejemplo muestra por qué la accuracy puede ser engañosa: trata todos los aciertos como equivalentes y todos los errores como equivalentes. No distingue si el modelo está acertando la clase fácil, la clase mayoritaria o la clase menos importante. Tampoco muestra si está fallando justamente en los casos críticos.

Algo parecido puede ocurrir en abandono de clientes. Supongamos que el 90% de los clientes permanece y el 10% abandona. Un modelo que siempre predice “permanece” tendrá una accuracy del 90%. Pero si el objetivo era detectar clientes con riesgo de abandono, ese modelo no sirve. No identifica a nadie para intervenir.



Resultados del modelo.

La accuracy también puede engañar cuando los errores tienen costos diferentes. En muchos problemas, no es lo mismo equivocarse en una dirección que en otra. Marcar como positivo un caso negativo puede generar una falsa alarma. Marcar como negativo un caso positivo puede dejar pasar un caso importante. Ambos errores reducen la accuracy de la misma manera, pero sus consecuencias pueden ser muy distintas.

En diagnóstico asistido, por ejemplo, no es equivalente marcar para revisión un caso que finalmente no requería atención que no detectar un caso que sí la necesitaba. En ambos casos hay un error, pero uno puede tener consecuencias mucho más graves que el otro. La accuracy los cuenta simplemente como predicciones incorrectas.

En riesgo crediticio ocurre algo similar. Clasificar como riesgosa una solicitud que habría cumplido puede afectar a una persona y generar una oportunidad perdida. Clasificar como segura una solicitud que terminará en incumplimiento puede generar una pérdida financiera. No podemos evaluar ambos errores únicamente como “un fallo más” sin considerar el contexto.

La accuracy también puede ocultar problemas en clasificación multiclase. Imaginemos un modelo que clasifica reclamos en cuatro categorías: “consulta general”, “facturación”, “soporte técnico” y “baja del servicio”. Si la mayoría de los reclamos pertenece a “consulta general”, el modelo puede obtener una buena accuracy clasificando bien esa categoría frecuente, pero fallar mucho en “baja del servicio”, que quizá sea una categoría menos frecuente pero muy importante.

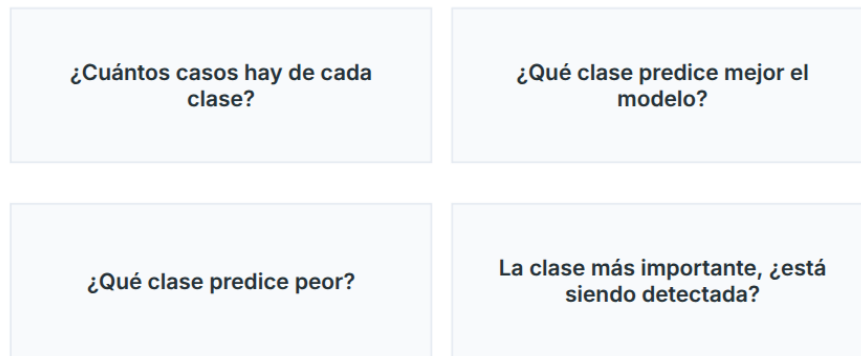
Por ejemplo:



Ejemplo.

Si el modelo funciona muy bien con “consulta general” pero casi nunca detecta “baja del servicio”, la accuracy global puede seguir siendo alta. Sin embargo, desde el punto de vista del negocio, fallar en esa categoría puede ser grave. La métrica general puede esconder el mal desempeño en clases minoritarias.

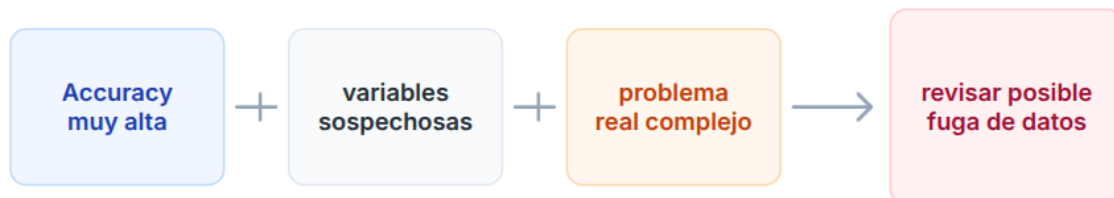
Por eso, una accuracy alta siempre debe leerse junto con la distribución de clases. Antes de interpretar el número, conviene preguntar:



Algunas preguntas útiles para interpretar la accuracy.

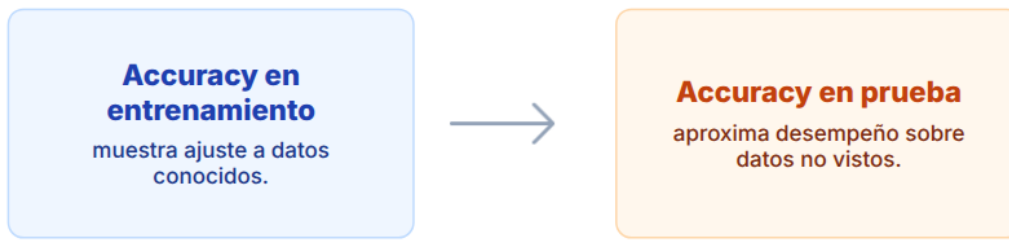
Otra señal de alerta aparece cuando la accuracy es demasiado buena para ser cierta. Si un modelo obtiene resultados casi perfectos en un problema real complejo, no debemos celebrarlo inmediatamente. Puede haber una explicación legítima, pero también puede haber fuga de datos, evaluación incorrecta o algún error en el armado de `X` e `y`.

Por ejemplo, si queremos predecir abandono de clientes y dejamos dentro de las variables predictoras una columna como `fecha_baja`, `motivo_cancelacion` o `estado_final`, el modelo puede alcanzar una accuracy altísima. Pero no porque haya aprendido a anticipar el abandono, sino porque recibió información que revela el resultado. En uso real, esas columnas no estarían disponibles antes de la predicción.



También puede ocurrir que la evaluación se haya hecho sobre los mismos datos usados para entrenar. En ese caso, la accuracy puede reflejar memoria del entrenamiento más que capacidad de generalización. Un árbol de decisión muy profundo, por ejemplo, puede obtener una accuracy casi perfecta sobre `X_train`, pero rendir mucho peor sobre `X_test`.

Por eso, una accuracy debe interpretarse siempre preguntando dónde fue calculada. No es lo mismo accuracy de entrenamiento que accuracy de prueba. La que más nos interesa para evaluar generalización es la obtenida sobre datos no vistos.



La accuracy debe interpretarse siempre preguntando dónde fue calculada.

La accuracy también puede engañar cuando el umbral de decisión no fue elegido con criterio. En modelos que producen probabilidades, como la regresión logística, la clase final depende de un umbral. Si usamos automáticamente 0.5, quizá obtengamos una accuracy aceptable, pero no necesariamente el comportamiento que necesitamos. En problemas desbalanceados, un umbral de 0.5 puede hacer que el modelo prediga muy pocos casos positivos.

Esto significa que la accuracy puede parecer razonable mientras el modelo deja pasar muchos positivos reales. Para detectar ese problema, necesitamos mirar otras métricas y la matriz de confusión.

La accuracy puede engañar cuando las clases están desbalanceadas, cuando los errores tienen costos diferentes, cuando una clase minoritaria es especialmente importante, cuando evaluamos sobre datos de entrenamiento o cuando hay fuga de datos. Una accuracy alta no garantiza que el modelo sea útil. Para interpretar correctamente un clasificador, necesitamos mirar cómo se distribuyen sus aciertos y errores. Esa será justamente la función de la matriz de confusión.

Matriz de confusión

Para entender mejor el desempeño de un clasificador, necesitamos mirar algo más detallado que la proporción total de aciertos. La **matriz de confusión** cumple justamente esa función: organiza las predicciones comparando la clase real con la clase predicha.

El nombre puede sonar extraño al principio. No significa que la matriz sea confusa, sino que muestra en qué casos el modelo confunde una clase con otra. Nos permite ver cuántas veces acertó, pero también qué tipo de errores cometió.

En cualquier problema de clasificación, cada caso evaluado tiene dos valores importantes:



Valores posibles.

Si ambos valores coinciden, el modelo acertó. Si no coinciden, el modelo se equivocó. La matriz de confusión cuenta todas esas combinaciones.

Imaginemos un modelo que predice abandono de clientes. Definimos dos clases: "permanece" y "abandona".

Para cada cliente del conjunto de prueba, sabemos qué ocurrió realmente y qué predijo el modelo. Algunos clientes que abandonaron fueron correctamente clasificados como "abandona". Otros clientes que permanecieron fueron correctamente clasificados como "permanece". Pero también puede haber errores: clientes que permanecieron y fueron marcados como abandono, o clientes que abandonaron y no fueron detectados.

La matriz de confusión organiza esos resultados en una tabla:

		Predicción	
		permanece	abandona
Real:	permanece	50	10
	abandona	15	25

Matriz de confusión.

Esta matriz se lee comparando filas y columnas. Las filas representan la clase real. Las columnas representan la clase predicha. En este ejemplo, hubo 50 clientes que realmente permanecieron y que el modelo predijo como "permanece". También hubo 25 clientes que realmente abandonaron y que el modelo predijo como "abandona". Esas dos celdas son aciertos.

Los errores aparecen fuera de esa diagonal principal. Hay 10 clientes que realmente permanecieron, pero el modelo predijo “abandona”. También hay 15 clientes que realmente abandonaron, pero el modelo predijo “permanece”.

La matriz nos permite ver mucho más que la accuracy. Si sumamos los aciertos, tenemos:

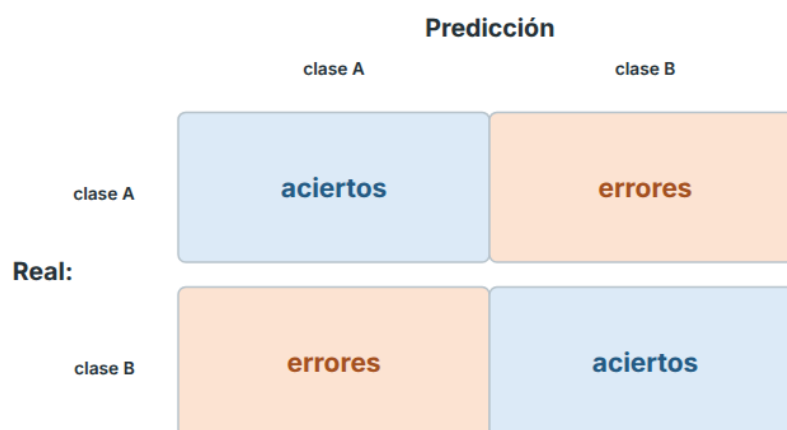


Suma de los aciertos.

La accuracy es 75%. Pero la matriz nos dice cómo se compone ese resultado. Sabemos que el modelo detectó 25 abandonos, dejó pasar 15 y generó 10 falsas alarmas. Esa información es mucho más útil para decidir si el modelo sirve.

Por ejemplo, si el objetivo de la empresa es detectar clientes en riesgo para intervenir a tiempo, los 15 abandonos no detectados pueden ser un problema importante. Si cada intervención comercial tiene un costo alto, los 10 clientes marcados incorrectamente como abandono también importan. La matriz de confusión permite discutir estos errores de manera concreta.

La estructura general en clasificación binaria puede representarse así:



Estructura general en clasificación binaria.

Los valores de la diagonal principal son predicciones correctas. Los valores fuera de la diagonal son confusiones entre clases.

Esta idea también sirve para clasificación multiclase. Supongamos que un modelo clasifica reclamos en tres categorías: “facturación”, “soporte técnico” y “envíos”. La matriz de confusión podría verse así:

		Predicción		
		facturación	soporte	envíos
Real:	facturación	80	12	8
	soporte técnico	10	70	20
	envíos	5	15	60

Matriz de confusión con tres categorías.

En este caso, la diagonal principal muestra los aciertos: reclamos de facturación predichos como facturación, reclamos de soporte técnico predichos como soporte técnico y reclamos de envíos predichos como envíos. Los demás valores muestran errores. Por ejemplo, hay 20 reclamos reales de soporte técnico que fueron clasificados como envíos, y 15 reclamos reales de envíos que fueron clasificados como soporte técnico.

Esta lectura es especialmente útil porque no todos los errores son igualmente frecuentes ni igualmente importantes. Tal vez el modelo confunde mucho soporte técnico con envíos, pero casi no confunde facturación. La accuracy global podría ocultar esa diferencia. La matriz de confusión la muestra de manera directa.

En problemas multiclase, una matriz de confusión puede ayudar a responder preguntas como estas:

1

¿Qué clase clasifica mejor el modelo?

2

¿Qué clases confunde con más frecuencia?

3

¿Hay alguna clase que casi nunca detecta?

4

¿Los errores más frecuentes son aceptables para el problema?

Preguntas útiles en problemas multiclase.

La matriz de confusión también es muy útil cuando las clases están desbalanceadas. Volvamos al ejemplo de detección de fraude. Si tenemos 1000 transacciones, 980 normales y 20 fraudulentas, un modelo que siempre predice “normal” tendría esta matriz:

		Predicción	
		normal	fraude
Real:	normal	980	0
	fraude	20	0

Preguntas útiles en problemas multiclase.

La accuracy sería 98%, pero la matriz muestra claramente que el modelo no detectó ningún fraude. Todos los casos fraudulentos aparecen en la celda donde el modelo predijo “normal”. Sin la matriz de confusión, podríamos quedarnos con la impresión equivocada de que el modelo funciona muy bien.

Este ejemplo muestra por qué la matriz de confusión es una herramienta central. Nos obliga a mirar los errores. Nos muestra si el modelo está detectando la clase importante o si solamente acierta porque la clase mayoritaria domina el dataset.

En `scikit-learn`, la matriz de confusión puede calcularse con `confusion_matrix`:

```
from sklearn.metrics import confusion_matrix

y_pred = modelo.predict(X_test)

confusion_matrix(y_test, y_pred)
```

El primer argumento, `y_test`, contiene las clases reales. El segundo, `y_pred`, contiene las clases predichas. Este orden es importante porque la matriz compara realidad y predicción. Si invertimos los argumentos o interpretamos mal filas y columnas, podemos sacar conclusiones equivocadas.

Una convención habitual es:



Convención habitual de la matriz de confusión.

Aun así, siempre conviene revisar cómo la herramienta está ordenando las clases. Si las etiquetas son texto, puede ordenarlas alfabéticamente. Si son números, puede ordenarlas de menor a mayor. En algunos casos, podemos especificar explícitamente el orden de las etiquetas para evitar confusiones.

```
confusion_matrix(y_test, y_pred, labels=["permanece", "abandona"])
```

Esto ayuda a leer la matriz de manera coherente con el problema. No debemos interpretar una celda solo por su posición visual sin confirmar qué clase corresponde a cada fila y a cada columna.

La matriz de confusión no reemplaza a las métricas, pero las sostiene. Muchas métricas importantes se calculan a partir de sus celdas. En clasificación binaria, de la matriz surgen los verdaderos positivos, falsos positivos, verdaderos negativos y falsos negativos. A partir de esos valores podremos calcular precisión, recall y otras métricas que estudiaremos en el siguiente capítulo.

Por ahora, lo más importante es comprender su función: la matriz de confusión abre la caja de la accuracy. En lugar de decir solamente “el modelo acertó el 75%”, nos muestra qué aciertos y qué errores componen ese 75%.

Podemos resumirlo así:

Accuracy:

resume cuántos casos acertó el modelo.



Matriz de confusión:

muestra cómo se distribuyen los aciertos y errores entre las clases.

	Clase A	Clase B
A	aciertos	errores
B	errores	aciertos

accuracy y matriz de confusión.

Esta diferencia es esencial. Un mismo valor de accuracy puede corresponder a modelos con comportamientos muy distintos. Dos modelos pueden acertar el 80% de los casos, pero uno puede detectar bien la clase importante y otro puede ignorarla casi por completo. La matriz permite distinguir esas situaciones.

La matriz de confusión organiza los resultados de un clasificador comparando clases reales y clases predichas. Permite ver aciertos y errores por clase, detectar confusiones frecuentes, analizar problemas de desbalance y comprender mejor el comportamiento del modelo. Es una herramienta indispensable para evaluar clasificadores con más criterio que usando únicamente la accuracy.

Verdaderos positivos

Para leer una matriz de confusión con más precisión, necesitamos nombrar sus celdas. En clasificación binaria, una de las celdas más importantes corresponde a los **verdaderos positivos**.

Un verdadero positivo ocurre cuando el caso real pertenece a la clase positiva y el modelo también predice la clase positiva. Es decir, el modelo detecta correctamente un caso que nos interesaba identificar.

La idea puede resumirse así: el caso era positivo y el modelo predijo positivo.

La palabra “verdadero” indica que la predicción fue correcta. La palabra “positivo” indica que el modelo predijo la clase positiva. Por eso, un verdadero positivo es un acierto sobre la clase positiva.

La clase positiva no significa necesariamente algo bueno. En machine learning, llamamos clase positiva a la clase que queremos detectar o analizar especialmente. En un problema de abandono de clientes, la clase positiva podría ser “abandona”. En detección de fraude, podría ser “fraude”. En riesgo crediticio, podría ser “incumple”. En diagnóstico asistido, podría ser “requiere revisión”.

Por ejemplo:

Problema:
abandono de clientes

Clase positiva:
abandona

Cliente real
abandona

Predicción del modelo
abandona

Resultado
verdadero positivo

Verdadero positivo.

En este caso, el modelo hizo exactamente lo que queríamos para ese cliente: identificó correctamente un abandono. Si la empresa usa el modelo para priorizar acciones de retención, este cliente habría sido señalado como un caso sobre el cual actuar.

En detección de fraude, un verdadero positivo sería una transacción fraudulenta correctamente marcada como sospechosa:

Transacción real:
fraude

Predicción del modelo:
fraude

Resultado
verdadero positivo

Verdadero positivo.

En diagnóstico asistido, un verdadero positivo podría ser un caso que realmente requería revisión y que el modelo marcó para revisar. En mantenimiento predictivo, podría ser una falla real correctamente anticipada. En todos los ejemplos, la idea es la misma: el modelo detecta correctamente un caso de la clase de interés.

Dentro de la matriz de confusión, los verdaderos positivos se ubican en la celda donde la clase real es positiva y la predicción también es positiva.

Si usamos el ejemplo de abandono:

		Predicción	
		permanece	abandona
Real:	permanece	50	10
	abandona	15	25

Matriz de confusión.

Como la clase positiva es “abandona”, los verdaderos positivos son los 25 casos de la esquina inferior derecha. Son clientes que realmente abandonaron y que el modelo predijo como abandono.

Este número es muy importante porque aparece en varias métricas. La precision, el recall y el F1-score dependen de los verdaderos positivos. En general, cuando queremos que un modelo detecte una clase importante, nos interesa que la cantidad de verdaderos positivos sea alta.

Sin embargo, no alcanza con mirar este número de manera aislada. Si un modelo obtiene muchos verdaderos positivos, también debemos mirar cuántos falsos positivos produjo y cuántos positivos reales dejó sin detectar. Un modelo podría detectar muchos abandonos, pero generar demasiadas falsas alarmas. O podría detectar algunos abandonos con mucha precisión, pero dejar pasar muchos otros.

Por eso, los verdaderos positivos son una parte central de la evaluación, pero no toda la evaluación. Nos dicen cuántos casos de interés fueron correctamente identificados. Para entender el comportamiento completo del modelo, necesitamos mirar también las otras celdas de la matriz.

Los verdaderos positivos son los casos positivos reales que el modelo predijo correctamente como positivos. Representan detecciones correctas de la clase de interés. Son especialmente importantes en problemas donde queremos encontrar eventos relevantes, como abandono, fraude, incumplimiento, fallas o casos que requieren revisión.

Falsos positivos

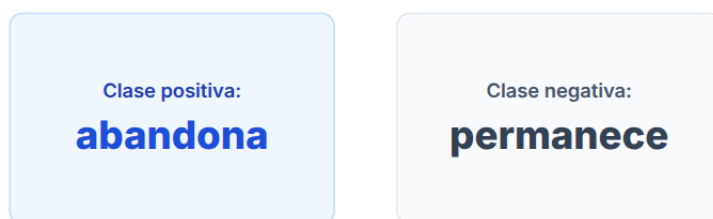
Un **falso positivo** ocurre cuando el modelo predice la clase positiva, pero el caso real pertenece a la clase negativa. Es decir, el modelo genera una alerta o una detección positiva que no corresponde con la realidad.

La idea puede resumirse así: el caso era negativo pero el modelo predijo positivo.

La palabra “falso” indica que la predicción fue incorrecta. La palabra “positivo” indica que el modelo predijo la clase positiva. Por eso, un falso positivo es una predicción positiva equivocada.

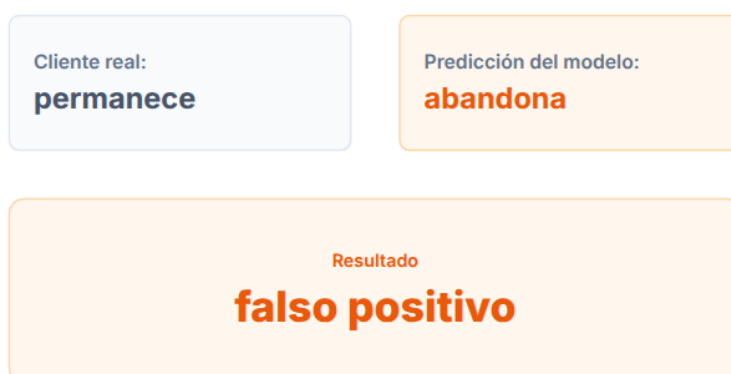
En muchos contextos, podemos pensar el falso positivo como una **falsa alarma**. El modelo señala un caso como si perteneciera a la clase de interés, pero en realidad no era así.

Tomemos nuevamente el ejemplo de abandono de clientes. Definimos:



Definición.

Un falso positivo sería un cliente que realmente permanece, pero que el modelo clasifica como “abandona”.



Falso positivo.

En este caso, el modelo generó una alerta de abandono innecesaria. Si la empresa usa esa predicción para intervenir, quizás contacte al cliente, le ofrezca una promoción o destine recursos de retención a alguien que no estaba por abandonar.

Esto no significa que el error sea siempre grave. Depende del contexto. Si contactar al cliente tiene bajo costo y no produce molestias importantes, algunos falsos positivos pueden ser aceptables. Pero si cada intervención es costosa, limitada o sensible, demasiados falsos positivos pueden volver poco práctico al modelo.

En detección de fraude, un falso positivo ocurre cuando una transacción normal es marcada como sospechosa.



Este error puede generar una revisión innecesaria, bloquear temporalmente una operación legítima o incomodar al usuario. Si el sistema produce demasiados falsos positivos, puede saturar a los equipos de revisión y afectar la confianza en el servicio.

En diagnóstico asistido, un falso positivo podría ser un caso marcado para revisión que finalmente no requería atención especial. En algunos contextos, esto puede ser aceptable si el objetivo es no dejar pasar casos importantes. Pero también puede generar ansiedad, sobrecarga de trabajo o uso innecesario de recursos. Nuevamente, el impacto depende del problema.

Dentro de la matriz de confusión, los falsos positivos se ubican en la celda donde la clase real es negativa, pero la predicción es positiva.

En el ejemplo de abandono:

		Predicción	
		permanece	abandona
Real:	permanece	50	10
	abandona	15	25

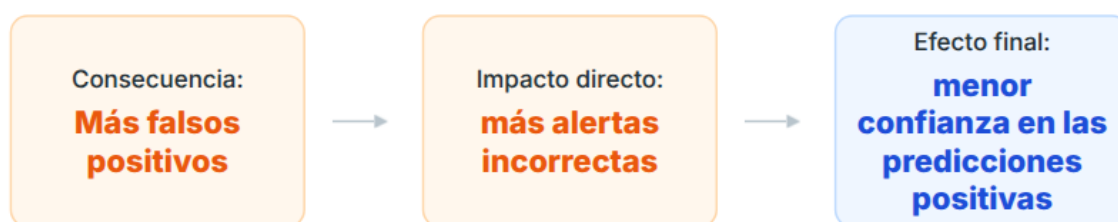
Matriz de confusión.

Como la clase positiva es “abandona”, los falsos positivos son los 10 clientes que realmente permanecieron pero fueron predichos como abandono.

Este número es especialmente importante para entender cuántas alertas incorrectas genera el modelo. Si el modelo predice muchos casos como positivos, pero muchos de ellos son falsos positivos, entonces sus predicciones positivas serán poco confiables.

Esta idea se relaciona directamente con la métrica de **precision**, que estudiaremos en el próximo capítulo. La precision responde la pregunta: de todos los casos que el modelo marcó como positivos, ¿cuántos eran realmente positivos? Cuando hay muchos falsos positivos, la precision tiende a bajar.

Podemos verlo de manera intuitiva:



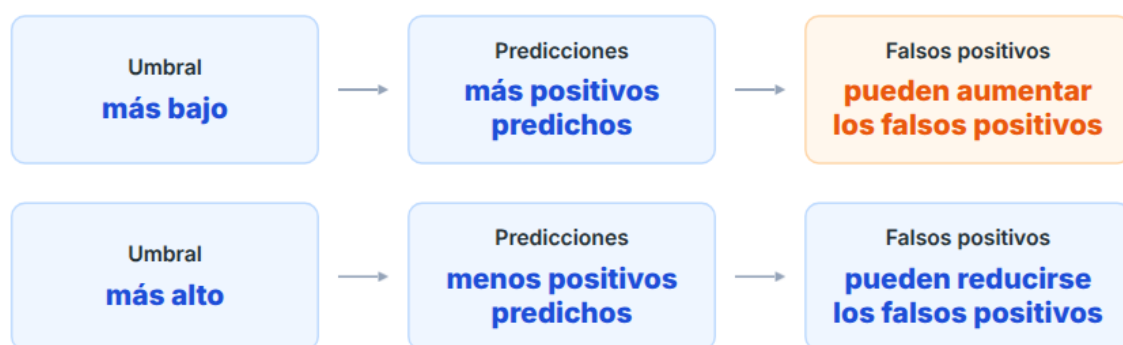
Cuando hay muchos falsos positivos, la precision suele bajar.

Pero reducir falsos positivos no siempre debe ser el único objetivo. Un modelo muy conservador puede generar pocas falsas alarmas, pero también dejar pasar muchos casos positivos reales. Por ejemplo, si un sistema de fraude solo marca como sospechosas las transacciones extremadamente evidentes, puede tener pocos falsos positivos, pero no detectar muchos fraudes reales.

Por eso, los falsos positivos deben analizarse junto con los falsos negativos. En algunos problemas, preferimos aceptar más falsos positivos para reducir los casos positivos no detectados. En otros, preferimos ser más estrictos y generar menos alertas, aunque eso implique perder algunos casos positivos.

La decisión depende del costo de cada tipo de error. En una campaña comercial, tal vez podamos tolerar algunas falsas alarmas si el costo de contacto es bajo. En un sistema financiero, demasiadas falsas alarmas pueden afectar operaciones legítimas. En salud, una revisión adicional puede ser aceptable en algunos contextos, pero no si satura el sistema o genera consecuencias innecesarias.

También debemos recordar que el número de falsos positivos puede cambiar si modificamos el umbral de decisión. En modelos que producen probabilidades, bajar el umbral suele aumentar la cantidad de casos predichos como positivos. Eso puede ayudar a detectar más positivos reales, pero también puede aumentar los falsos positivos. Subir el umbral suele reducir falsas alarmas, pero puede aumentar los positivos no detectados.



Umbral y falsos positivos.

Esta relación será especialmente importante cuando estudiemos precisión, recall y umbrales. Por ahora, lo esencial es reconocer qué representa esta celda de la matriz de confusión.

Los falsos positivos son casos negativos que el modelo predijo incorrectamente como positivos. Representan falsas alarmas. Son importantes cuando actuar sobre una predicción positiva tiene costo, genera molestias, requiere revisión o puede producir consecuencias injustificadas. Un buen análisis de clasificación debe revisar cuántos falsos positivos genera el modelo y si esa cantidad es aceptable para el problema real.

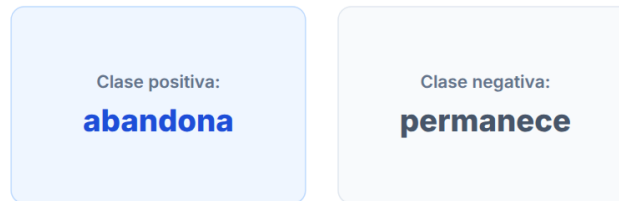
Verdaderos negativos

Un **verdadero negativo** ocurre cuando el caso real pertenece a la clase negativa y el modelo también predice la clase negativa. Es decir, el modelo descarta correctamente un caso que no pertenece a la clase de interés.

La idea puede resumirse así: el caso era negativo y el modelo predijo negativo.

La palabra “verdadero” indica que la predicción fue correcta. La palabra “negativo” indica que el modelo predijo la clase negativa. Por eso, un verdadero negativo es un acierto sobre la clase negativa.

Volvamos al ejemplo de abandono de clientes. Definimos:



Verdaderos negativos.

Un verdadero negativo sería un cliente que realmente permanece y que el modelo clasifica correctamente como “permanece”.



Verdaderos negativos.

En este caso, el modelo no generó una alerta de abandono porque, según su predicción, el cliente no estaba en riesgo. Como el cliente efectivamente permaneció, la decisión fue correcta.

En detección de fraude, un verdadero negativo sería una transacción normal correctamente clasificada como normal:



En diagnóstico asistido, podría ser un caso que no requería revisión especial y que el modelo no marcó como alerta. En mantenimiento predictivo, podría ser un equipo que no iba a fallar y que el modelo clasificó como funcionamiento normal.

Dentro de la matriz de confusión, los verdaderos negativos se ubican en la celda donde la clase real es negativa y la predicción también es negativa.

En el ejemplo de abandono:

		Predicción	
		permanece	abandona
Real	permanece	50	10
	abandona	15	25

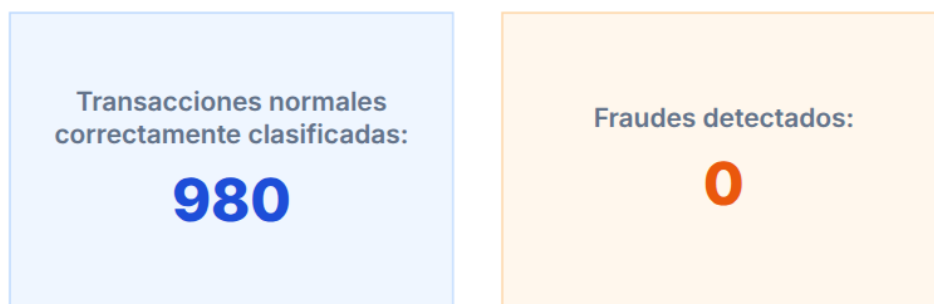
Los verdaderos negativos se ubican en la celda donde la clase real es negativa.

Como la clase negativa es “permanece”, los verdaderos negativos son los 50 clientes que realmente permanecieron y que el modelo predijo como “permanece”.

Los verdaderos negativos son importantes porque muestran cuántos casos negativos fueron correctamente descartados. En muchos problemas, esto permite evitar acciones innecesarias. Si el modelo reconoce correctamente que una transacción es normal, no la envía a revisión. Si reconoce que un cliente no está en riesgo de abandono, no consume recursos de retención. Si reconoce que un caso no requiere alerta, evita generar trabajo adicional.

Sin embargo, una cantidad alta de verdaderos negativos no siempre significa que el modelo sea bueno. Si la clase negativa es muy mayoritaria, un modelo puede acumular muchos verdaderos negativos simplemente prediciendo negativo casi siempre. Esto puede elevar la accuracy aunque el modelo no detecte bien los casos positivos.

Por ejemplo, en un problema de fraude con 1000 transacciones, de las cuales 980 son normales y 20 fraudulentas, un modelo que siempre predice “normal” tendrá 980 verdaderos negativos.



980 verdaderos negativos.

El número de verdaderos negativos es muy alto, pero el modelo no detecta ningún fraude. Si el objetivo era encontrar transacciones sospechosas, ese modelo no es útil. Por eso, los verdaderos negativos deben interpretarse junto con las demás celdas de la matriz.

Esta situación muestra una advertencia importante: en problemas desbalanceados, los verdaderos negativos pueden dominar la evaluación. Si hay muchos más casos negativos que positivos, un modelo puede parecer exitoso porque acierta muchos negativos, pero fallar en la clase que más interesa.

Podemos pensarlo así: Muchos verdaderos negativos no garantizan buena detección de positivos.

Esto no significa que los verdaderos negativos no importen. En muchos problemas sí importan, y mucho. Si un sistema de fraude marcara demasiadas transacciones normales como sospechosas, generaría molestias y trabajo innecesario. Si un modelo de abandono señalara a demasiados clientes que en realidad permanecerían, podría gastar recursos sin sentido. Para evitar esos problemas, necesitamos que el modelo reconozca correctamente una buena parte de los casos negativos.

Pero el equilibrio depende del objetivo. En algunos contextos, queremos maximizar la detección de positivos aunque eso reduzca la cantidad de verdaderos negativos. En otros, queremos ser más cuidadosos para no marcar demasiados negativos como positivos. La matriz de confusión nos permite observar esa tensión.

Los verdaderos negativos también aparecen en algunas métricas que estudiaremos más adelante, como la especificidad. La especificidad mide qué proporción de los casos negativos reales fueron correctamente identificados como negativos. Aunque no profundizaremos todavía en esa métrica, conviene saber que los verdaderos negativos son fundamentales cuando queremos evaluar la capacidad del modelo para descartar correctamente casos que no pertenecen a la clase positiva.

En clasificación binaria, puede ser útil recordar la relación entre verdaderos negativos y falsos positivos. Ambos parten de casos que realmente eran negativos. La diferencia está en lo que predijo el modelo.



Relación entre verdaderos negativos y falsos positivos.

En otras palabras, los verdaderos negativos son negativos correctamente descartados. Los falsos positivos son negativos marcados por error como positivos. Cuando el modelo reduce falsos positivos, suele aumentar su capacidad de conservar verdaderos negativos, aunque esa relación también depende del umbral y del comportamiento general del clasificador.

Volvamos al ejemplo de abandono. Los clientes que realmente permanecen son 60 en total:



Relación entre verdaderos negativos y falsos positivos en el ejemplo.

De esos 60 clientes negativos reales, 50 fueron correctamente identificados como negativos. Los otros 10 fueron falsas alarmas. Esta lectura nos permite analizar qué tan bien el modelo evita intervenir sobre clientes que no estaban en riesgo.

Los verdaderos negativos son los casos negativos reales que el modelo predijo correctamente como negativos. Representan aciertos al descartar casos que no pertenecen a la clase de interés. Son importantes para evitar falsas alarmas y acciones innecesarias, pero no deben interpretarse de manera aislada. Un modelo puede tener muchos verdaderos negativos y aun así fallar en la detección de los casos positivos que realmente importan.

Falsos negativos

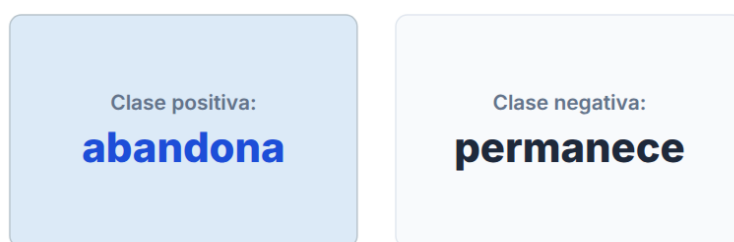
Un **falso negativo** ocurre cuando el caso real pertenece a la clase positiva, pero el modelo predice la clase negativa. Es decir, el modelo no detecta un caso que sí pertenecía a la clase de interés.

La idea puede resumirse así: el caso era positivo pero el modelo predijo negativo.

La palabra “falso” indica que la predicción fue incorrecta. La palabra “negativo” indica que el modelo predijo la clase negativa. Por eso, un falso negativo es una predicción negativa equivocada.

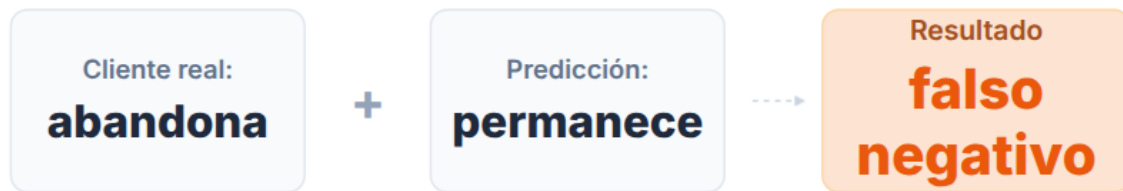
En muchos problemas, los falsos negativos son especialmente importantes porque representan casos que el modelo dejó pasar. No generó alerta, no marcó riesgo, no recomendó revisión o no identificó el evento que queríamos detectar.

Volvamos al ejemplo de abandono de clientes. Definimos:



Definiciones.

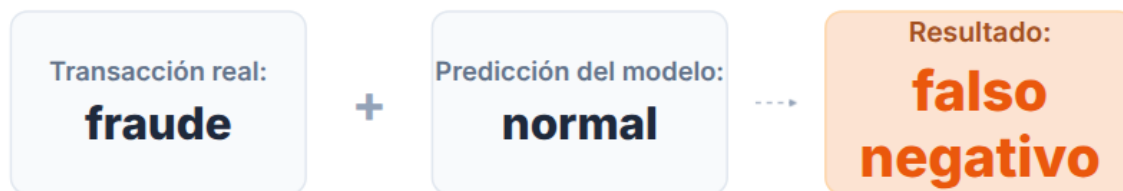
Un falso negativo sería un cliente que realmente abandona, pero que el modelo clasifica como “permanece”.



Falsos negativos.

En este caso, el modelo no detectó a un cliente que estaba en riesgo. Si la empresa usa el modelo para decidir a quién contactar, este cliente no recibiría ninguna acción de retención. La consecuencia posible es perder la oportunidad de intervenir antes del abandono.

En detección de fraude, un falso negativo sería una transacción fraudulenta clasificada como normal.



Falsos negativos.

Este error puede ser grave porque el fraude no fue detectado. La transacción podría aprobarse sin revisión y generar una pérdida económica o un problema de seguridad.

En diagnóstico asistido, un falso negativo puede ser un caso que realmente requería revisión, pero que el modelo no marcó. Este tipo de error suele ser muy sensible, porque puede implicar que un caso importante no reciba atención oportuna. Por eso, en muchos contextos de salud se presta especial atención a reducir falsos negativos, aunque eso pueda aumentar la cantidad de falsos positivos.

Dentro de la matriz de confusión, los falsos negativos se ubican en la celda donde la clase real es positiva, pero la predicción es negativa. En el ejemplo de abandono:

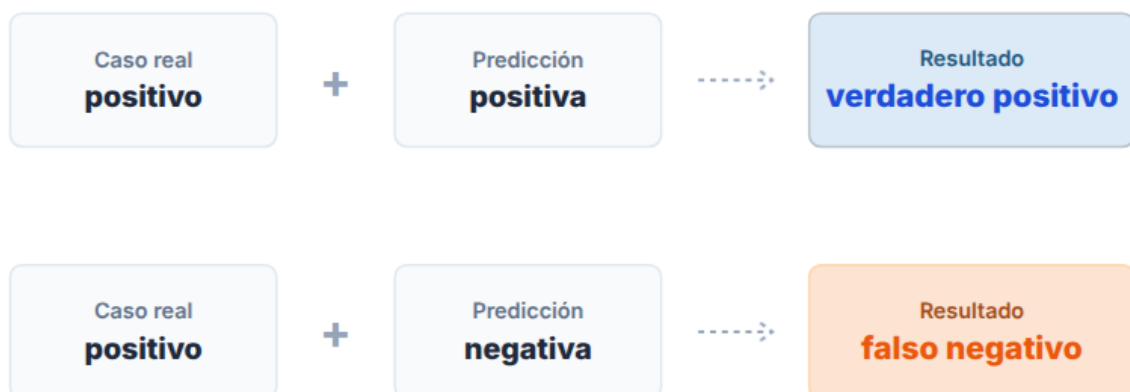
		Predicción	
		permanece	abandona
Real:	permanece	50	10
	abandona	15	25

Matriz de confusión, en el ejemplo de abandono.

Como la clase positiva es “abandona”, los falsos negativos son los 15 clientes que realmente abandonaron, pero que el modelo predijo como “permanece”.

Este número nos dice cuántos casos positivos reales no fueron detectados. En problemas donde la clase positiva representa un evento importante, los falsos negativos pueden ser una de las partes más críticas de la evaluación.

La relación entre verdaderos positivos y falsos negativos es muy importante. Ambos parten de casos que realmente eran positivos. La diferencia está en la predicción del modelo.



Relación entre verdaderos positivos y falsos negativos.

En abandono, todos los clientes que realmente abandonaron se dividen en dos grupos: los que el modelo detectó y los que dejó pasar. En la matriz anterior, hubo 40 abandonos reales en total:

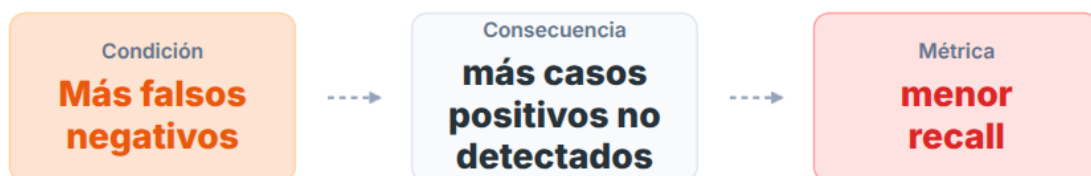


Relación entre verdaderos positivos y falsos negativos.

El modelo detectó 25 y dejó pasar 15. Esta información es mucho más útil que saber únicamente la accuracy. Nos permite preguntar si la cantidad de abandonos no detectados es aceptable para el objetivo del proyecto.

Los falsos negativos están directamente relacionados con la métrica de **recall** o sensibilidad, que estudiaremos en el próximo capítulo. El recall responde la pregunta: de todos los casos que realmente eran positivos, ¿cuántos logró detectar el modelo? Cuando hay muchos falsos negativos, el recall tiende a bajar.

Podemos pensarlo así:



Cuando hay muchos falsos negativos, el recall tiende a bajar.

Reducir falsos negativos suele ser prioritario cuando no detectar un caso positivo tiene consecuencias importantes. En fraude, puede significar dejar pasar operaciones peligrosas. En mantenimiento predictivo, puede significar no anticipar una falla. En salud, puede significar no revisar un caso que lo necesitaba. En abandono de clientes, puede significar perder oportunidades de retención.

Sin embargo, reducir falsos negativos no es gratis. En muchos modelos, para detectar más positivos debemos ser menos exigentes al marcar un caso como positivo. Esto puede lograrse bajando el umbral de decisión cuando el modelo produce probabilidades. Pero al hacerlo, también podemos aumentar los falsos positivos.

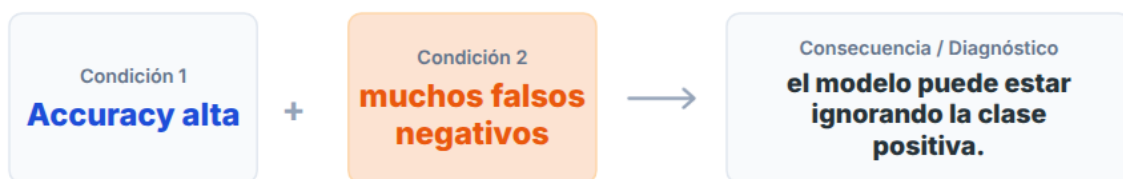


Reducir falsos negativos no es gratis.

Esta tensión es central en evaluación de clasificadores. No se trata simplemente de eliminar un tipo de error, sino de decidir qué equilibrio es más adecuado para el problema real. Un sistema que detecta todos los positivos pero genera demasiadas falsas alarmas puede ser impracticable. Un sistema que genera pocas falsas alarmas pero deja pasar muchos positivos puede ser insuficiente.

Por eso, los falsos negativos deben analizarse junto con los falsos positivos. En algunos contextos, aceptaremos más falsas alarmas para reducir positivos no detectados. En otros, necesitaremos limitar las alertas y aceptar que algunos positivos reales no serán identificados. La decisión depende del costo de cada error, de los recursos disponibles y de las consecuencias de actuar o no actuar.

También es importante recordar que un modelo puede tener una accuracy alta y muchos falsos negativos si la clase positiva es minoritaria. Por ejemplo, en un problema donde solo el 5% de los casos son positivos, un modelo que predice casi siempre negativo puede acertar mucho en general, pero dejar pasar la mayoría de los positivos. La accuracy puede verse bien, mientras que los falsos negativos revelan el problema.



El modelo puede estar ignorando la clase positiva.

Esta situación es frecuente en problemas desbalanceados. Por eso, cuando la clase positiva es rara pero importante, siempre debemos revisar cuántos falsos negativos produce el modelo. No alcanza con mirar aciertos totales.

En clasificación multiclase, la idea de falso negativo puede analizarse para una clase específica. Por ejemplo, si tenemos clases “riesgo bajo”, “riesgo medio” y “riesgo alto”, podemos tomar “riesgo alto” como clase de interés. Un falso negativo para “riesgo alto” sería un caso que realmente era de riesgo alto, pero fue clasificado como riesgo medio o bajo. Esta lectura por clase permite detectar si el modelo está dejando pasar categorías importantes.

Los falsos negativos son casos positivos reales que el modelo predijo incorrectamente como negativos. Representan casos importantes no detectados. Son especialmente relevantes cuando la clase positiva corresponde a eventos que queremos encontrar, como fraude, abandono, incumplimiento, fallas o casos que requieren revisión. Analizar los falsos negativos nos ayuda a entender qué está dejando pasar el modelo y si ese comportamiento es aceptable para el problema real.

Problemas con clases desbalanceadas

Las clases desbalanceadas aparecen cuando una clase tiene muchos más casos que otra. En clasificación binaria, esto suele traducirse en una clase mayoritaria y una clase minoritaria. El desbalance no es un problema raro; aparece en muchos escenarios reales. Lo importante es entender que puede afectar profundamente la manera en que interpretamos la evaluación de un modelo.

Imaginemos un problema de detección de fraude. En un conjunto de 10 000 transacciones, quizá 9900 sean normales y solo 100 sean fraudulentas.



Problema de detección de fraude.

La clase “normal” representa el 99% de los casos. La clase “fraude” representa apenas el 1%. Si entrenamos un modelo y miramos solamente la accuracy, podríamos llegar a conclusiones equivocadas.

Un modelo que siempre predice “normal” tendría esta matriz de confusión:

		Predicción	
		normal	fraude
Real	normal	9900	0
	fraude	100	0

Matriz de confusión.

La accuracy sería:

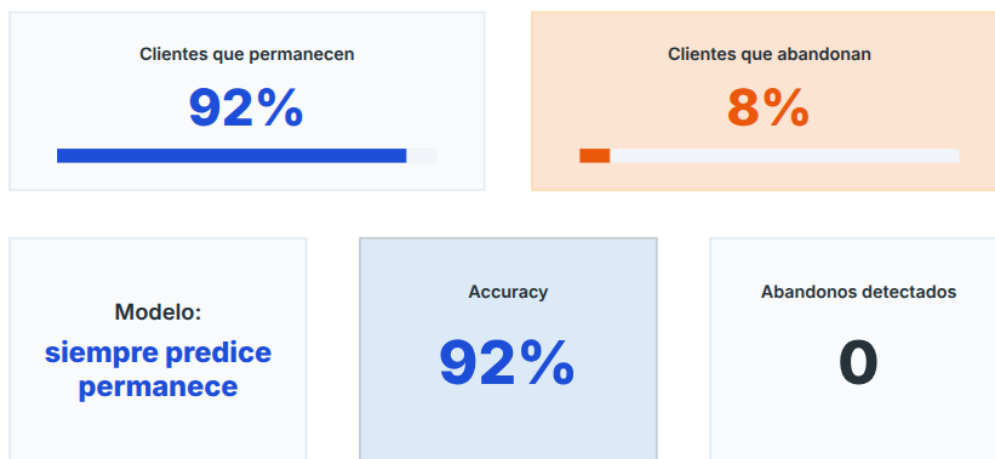
$$\begin{array}{c}
 \text{accuracy} \\
 \hline
 \frac{9900}{10000}
 \end{array}
 =
 \begin{array}{c}
 \text{accuracy} \\
 \hline
 0.99
 \end{array}$$

Accuracy.

Una exactitud del 99% parece excelente. Pero el modelo no detectó ningún fraude. Todos los casos fraudulentos quedaron como falsos negativos. Para el objetivo del problema, ese modelo es inútil, aunque su accuracy sea muy alta.

Este ejemplo muestra el peligro central del desbalance: la clase mayoritaria puede dominar la métrica global. El modelo puede acertar muchos casos simplemente porque predice la clase más frecuente. Pero si la clase minoritaria es la que realmente nos interesa, la evaluación queda distorsionada.

Esto también puede ocurrir en abandono de clientes. Si solo el 8% de los clientes abandona durante el período observado, un modelo que siempre predice “permanece” tendrá una accuracy del 92%. Sin embargo, no detectará a ningún cliente en riesgo.



Abandono de clientes.

Lo mismo puede aparecer en incumplimiento crediticio, fallas de máquinas, detección de enfermedades, alertas de seguridad, casos que requieren revisión o reclamos críticos. En muchos problemas, los eventos más importantes son menos frecuentes que los eventos normales.

Por eso, cuando las clases están desbalanceadas, no alcanza con mirar la accuracy. Debemos revisar la matriz de confusión y observar qué ocurre con la clase minoritaria o clase de interés. ¿El modelo la detecta? ¿La ignora? ¿Genera demasiadas falsas alarmas? ¿Cuántos casos positivos reales quedan sin detectar?

En un problema desbalanceado, los falsos negativos pueden quedar ocultos detrás de muchos verdaderos negativos. Si hay muchísimos casos negativos, acertarlos puede hacer que el desempeño general parezca bueno. Pero si el modelo falla sobre los positivos, puede no cumplir su propósito.

Podemos resumirlo así: Muchas clases negativas correctamente clasificadas pueden ocultar una mala detección de la clase positiva.

La matriz de confusión nos obliga a mirar esa estructura. En lugar de ver solo un 99% de accuracy, vemos que la celda de verdaderos positivos está en cero y que todos los positivos reales quedaron como falsos negativos. Esa información cambia por completo la interpretación.

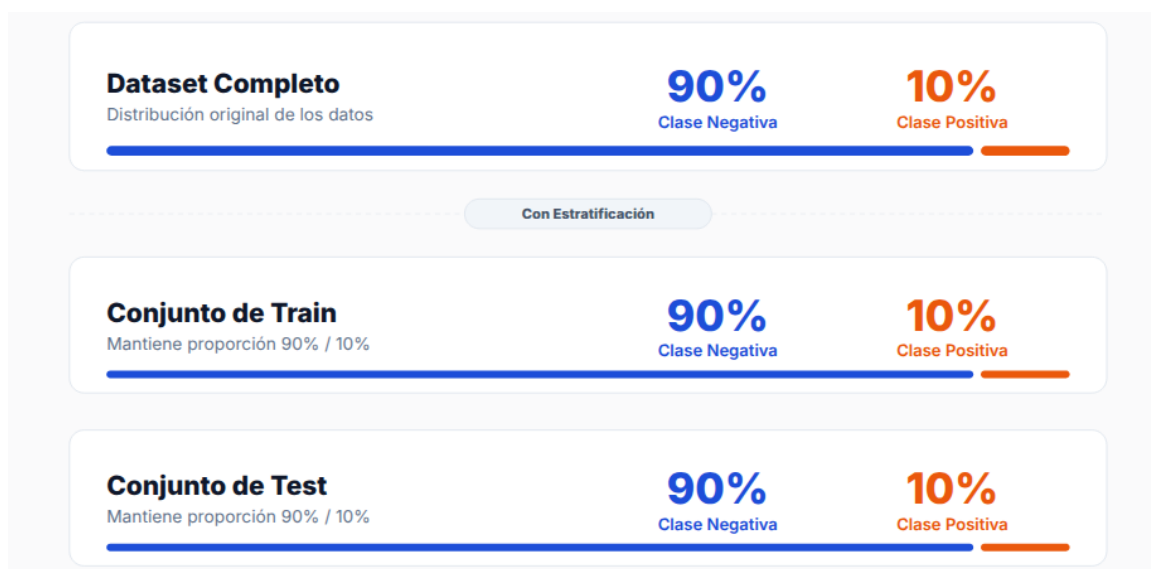
El desbalance también puede afectar el entrenamiento. Si la clase mayoritaria tiene muchos más ejemplos, el modelo puede aprender a favorecerla. Esto no significa que el algoritmo “decida” ignorar la clase minoritaria de manera consciente. Simplemente, al intentar reducir errores generales, puede encontrar más fácil acertar la clase abundante y fallar en la clase rara.

Este problema se vuelve más grave cuando la clase minoritaria tiene pocos ejemplos para aprender. Si hay solo 20 casos positivos en todo el dataset, el modelo tendrá

poca información para reconocer patrones asociados a esa clase. Además, la evaluación será inestable: unos pocos aciertos o errores pueden cambiar mucho las métricas.

El desbalance también afecta la separación entre entrenamiento y prueba. Si dividimos los datos al azar sin cuidado, puede ocurrir que una clase minoritaria quede muy poco representada en alguno de los conjuntos. Por ejemplo, si tenemos pocos casos positivos, una mala división podría dejar casi todos en entrenamiento y muy pocos en prueba, o al revés.

Para reducir ese riesgo, en clasificación usamos con frecuencia la **estratificación** al separar los datos. La estratificación intenta mantener proporciones similares de clases en entrenamiento y prueba.



Usamos estratificación al separar los datos.

En `scikit-learn`, esto puede hacerse con el argumento `stratify` en `train_test_split`:

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

La estratificación no soluciona el desbalance, pero evita que la separación lo empeore. Ayuda a que la evaluación sea más representativa de la distribución original de clases.

Cuando hay desbalance, también es importante comparar el modelo con una línea de base. Si la clase mayoritaria representa el 95% de los casos, un modelo que obtiene 94% de accuracy no necesariamente es bueno. De hecho, puede ser peor que predecir siempre la clase mayoritaria. Por eso, antes de interpretar una métrica, conviene preguntarse cuánto obtendría una estrategia simple.

Si siempre predigo la clase mayoritaria, ¿qué accuracy obtengo?

Esta pregunta ayuda a poner los resultados en contexto. Un 90% de accuracy puede ser excelente en un problema balanceado, pero pobre en uno donde la clase mayoritaria representa el 92%.

Además de la accuracy, en problemas desbalanceados necesitamos métricas que miren el desempeño sobre la clase de interés. En el próximo capítulo estudiaremos precision, recall y F1-score. Por ahora, alcanza con anticipar la idea: el recall nos ayudará a medir cuántos positivos reales detecta el modelo, y la precision nos ayudará a medir qué tan confiables son sus predicciones positivas.

Estas métricas son especialmente importantes cuando la clase positiva es minoritaria. Por ejemplo, en fraude, el recall puede decirnos qué proporción de fraudes reales detectamos. La precision puede decirnos qué proporción de las alertas generadas eran realmente fraude. Ambas preguntas son más informativas que la accuracy en ese contexto.

El desbalance también obliga a pensar en el umbral de decisión. En modelos que producen probabilidades, como la regresión logística, usar un umbral de 0.5 puede no ser adecuado si la clase positiva es rara. Una probabilidad de 0.20 puede ser baja en términos absolutos, pero alta si el evento ocurre solo en el 1% de los casos. Si usamos automáticamente 0.5, quizá casi nunca predigamos la clase positiva.

Esto no significa que debemos bajar siempre el umbral. Significa que debemos evaluarlo. Bajar el umbral puede ayudar a detectar más casos positivos, pero también puede aumentar los falsos positivos. La decisión depende del problema, del costo de las alertas y de la importancia de no dejar pasar casos positivos.

En algunos proyectos también se aplican técnicas específicas para manejar el desbalance. Algunas modifican los datos de entrenamiento, reduciendo casos de la clase mayoritaria o aumentando ejemplos de la clase minoritaria. Otras ajustan el peso de las clases para que los errores sobre la clase minoritaria tengan más impacto durante el entrenamiento. Estas estrategias son útiles, pero deben aplicarse con cuidado y evaluarse correctamente.

En este punto del libro, lo fundamental no es memorizar todas esas técnicas, sino reconocer el problema: cuando las clases están desbalanceadas, una evaluación

superficial puede ser engañosa. La accuracy deja de ser suficiente y la matriz de confusión se vuelve indispensable.

También debemos tener presente que el desbalance no siempre es un error del dataset. Muchas veces refleja la realidad del fenómeno. El fraude es raro. Las fallas críticas son raras. Algunas enfermedades son menos frecuentes. Algunos tipos de reclamo ocurren poco. No se trata de forzar artificialmente que todas las clases tengan la misma cantidad, sino de evaluar y entrenar con conciencia de esa distribución.

En clasificación multiclase, el desbalance puede aparecer cuando algunas categorías tienen muchos ejemplos y otras muy pocos. Un modelo puede clasificar bien las clases frecuentes y mal las raras, manteniendo una accuracy global aceptable. Por eso, también en multiclase conviene mirar la matriz de confusión y revisar el desempeño por clase.

Las clases desbalanceadas pueden hacer que un modelo parezca bueno aunque ignore la clase más importante. La accuracy puede ser engañosa porque favorece la clase mayoritaria. Para evaluar con criterio, debemos revisar la matriz de confusión, observar verdaderos positivos, falsos positivos, verdaderos negativos y falsos negativos, usar separación estratificada cuando corresponda y comparar contra líneas de base simples. En problemas desbalanceados, la pregunta no es solo cuántos casos acierta el modelo, sino si detecta los casos que realmente importan.

Mirar más allá del porcentaje de aciertos

La accuracy es una métrica simple y útil para comenzar a evaluar un clasificador, pero este capítulo mostró por qué no alcanza con mirar solamente el porcentaje total de aciertos. En clasificación, un modelo puede acertar muchas veces y aun así fallar en los casos más importantes para el problema.

La accuracy responde una pregunta general: qué proporción de casos fueron clasificados correctamente. Esa información puede ser valiosa cuando las clases están balanceadas y los errores tienen consecuencias similares. También puede servir como primera comparación entre modelos o como referencia inicial frente a una estrategia simple.

Pero la accuracy puede engañar cuando las clases están desbalanceadas o cuando distintos tipos de error tienen costos diferentes. Un modelo que siempre predice la clase mayoritaria puede alcanzar una exactitud alta sin detectar la clase minoritaria. En problemas como fraude, abandono, incumplimiento, fallas o casos que requieren

revisión, esto puede ser especialmente grave, porque la clase menos frecuente suele ser justamente la más importante.

Por eso incorporamos la matriz de confusión. Esta herramienta organiza los resultados comparando la clase real con la clase predicha. En lugar de resumir todo en un único número, muestra cómo se distribuyen los aciertos y errores entre las clases. Esa información permite ver si el modelo está detectando la clase de interés, si genera muchas falsas alarmas o si deja pasar demasiados casos importantes.

En clasificación binaria, la matriz de confusión nos llevó a distinguir cuatro situaciones:



Matriz de confusión en clasificación binaria.

Esta distinción es fundamental porque los errores no son equivalentes. Un falso positivo puede generar una alerta innecesaria, una revisión extra o una acción costosa. Un falso negativo puede dejar pasar un fraude, un abandono, una falla o un caso que requería atención. Cada problema define qué tipo de error es más preocupante.

También vimos que los verdaderos negativos pueden dominar la evaluación cuando la clase negativa es muy frecuente. En esos casos, una accuracy alta puede esconder un mal desempeño sobre la clase positiva. Por eso, ante clases desbalanceadas, siempre debemos revisar la matriz de confusión y no confiar únicamente en una métrica global.

La separación estratificada ayuda a mantener proporciones similares de clases en entrenamiento y prueba, pero no soluciona por sí sola el desbalance. La evaluación debe seguir preguntando qué ocurre con cada clase. Un modelo no es bueno solo porque acierta muchos casos; es bueno si acierta los casos que importan de acuerdo con el objetivo del proyecto.

Este capítulo prepara el terreno para el siguiente. Una vez que entendemos la matriz de confusión, podemos construir métricas más específicas. La precision nos ayudará a responder qué tan confiables son las predicciones positivas. El recall nos permitirá medir cuántos positivos reales logra detectar el modelo. El F1-score combinará ambas ideas para analizar el equilibrio entre falsas alarmas y casos no detectados.

Evaluar un clasificador exige mirar más allá del porcentaje de aciertos. La accuracy puede ser un punto de partida, pero la matriz de confusión muestra la estructura real de los resultados. Nos permite comprender qué clases predice bien el modelo, qué errores comete y si esos errores son aceptables para el problema. Una evaluación responsable no se queda con un número atractivo: analiza cómo se comporta el modelo frente a las decisiones que realmente importan.

Capítulo 12 · Precision, recall y F1-score

¿Qué mide precision?

En el capítulo anterior vimos que la matriz de confusión permite separar los distintos tipos de aciertos y errores de un clasificador. Esa estructura es la base para construir métricas más específicas que la accuracy. Una de las más importantes es **precision**.

La precision responde una pregunta concreta: De todos los casos que el modelo predijo como positivos, ¿cuántos eran realmente positivos?

Es decir, precision mira las predicciones positivas del modelo y analiza qué tan confiables fueron. No se concentra en todos los casos del dataset, sino solamente en aquellos que el modelo marcó como pertenecientes a la clase positiva.

En clasificación binaria, precision se calcula con verdaderos positivos y falsos positivos:

$$\text{precision} = \text{VP} / (\text{VP} + \text{FP})$$

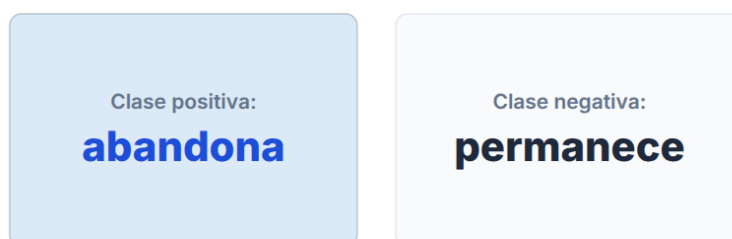
Verdaderos Positivos
Aciertos positivos: casos que pertenecen a la clase y fueron correctamente detectados.

Falsos Positivos
Falsas alarmas: casos que no pertenecen a la clase, pero fueron detectados por error.

Fórmula de cálculo.

Recordemos el significado de estas dos cantidades. Los verdaderos positivos son los casos que el modelo predijo como positivos y que realmente eran positivos. Los falsos positivos son los casos que el modelo predijo como positivos, pero que en realidad eran negativos. Por eso, el denominador contiene todas las predicciones positivas del modelo.

Volvamos al ejemplo de abandono de clientes. Definimos:



Definiciones.

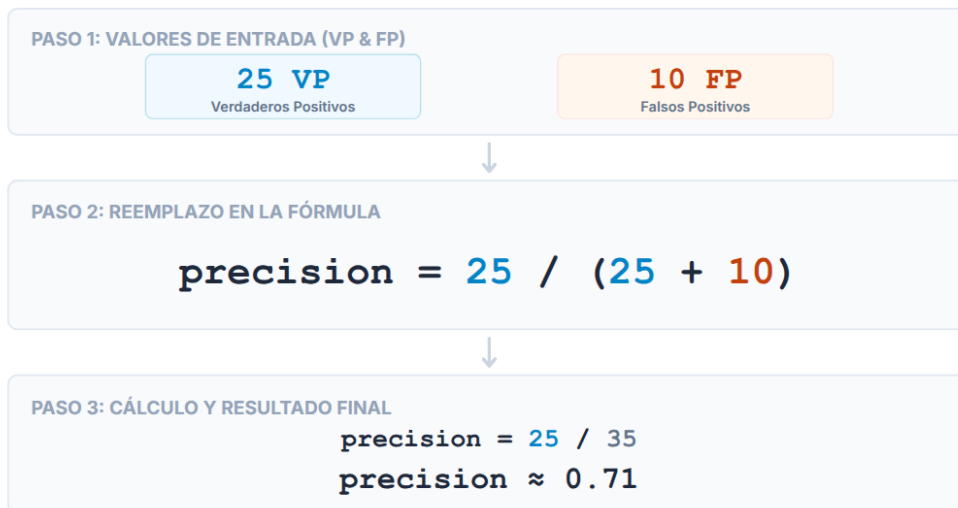
Supongamos que la matriz de confusión es:

		Predicción	
		permanece	abandona
Real:	permanece	50	10
	abandona	15	25

Matriz de confusión.

En esta matriz, los verdaderos positivos son 25 y los falsos positivos son 10. El modelo predijo “abandona” en 35 casos: 25 correspondían a clientes que realmente abandonaron y 10 correspondían a clientes que finalmente permanecieron.

Entonces:



El 71% realmente abandonó.

Esto significa que, de todos los clientes que el modelo marcó como abandono, aproximadamente el 71% realmente abandonó. Otra forma de decirlo es: cuando el modelo genera una alerta de abandono, acierta alrededor de 7 de cada 10 veces.

Esta métrica es especialmente útil cuando nos importa que las predicciones positivas sean confiables. Si cada vez que el modelo predice positivo se activa una acción costosa, delicada o limitada, queremos reducir la cantidad de falsas alarmas.

En abandono de clientes, una falsa alarma puede significar contactar a un cliente que no estaba en riesgo, ofrecerle un beneficio innecesario o dedicar recursos comerciales a un caso que no lo requería. En detección de fraude, puede significar bloquear o revisar una operación legítima. En un sistema de seguridad, puede generar alertas que ocupan tiempo de análisis. En un filtro de spam, puede enviar a la carpeta equivocada un correo legítimo.

Por eso, una precision baja puede ser problemática. Indica que muchas de las predicciones positivas del modelo no corresponden a casos positivos reales. El modelo genera alertas, pero una proporción importante de esas alertas es incorrecta.

Podemos resumirlo así:



Alta y baja precision.

Sin embargo, precision no cuenta toda la historia. Un modelo puede tener una precision muy alta y aun así ser poco útil si detecta muy pocos casos positivos reales.

Imaginemos un modelo de abandono que solo marca como “abandona” a 10 clientes. De esos 10, 9 realmente abandonan. La precision sería:

ALERTAS PREDICHAS

VP	VP	VP	VP	VP	VP	VP	VP	VP	FP
----	----	----	----	----	----	----	----	----	----

$$\text{precision} = 9 / 10 = 0.90$$

¿Bueno o malo?.

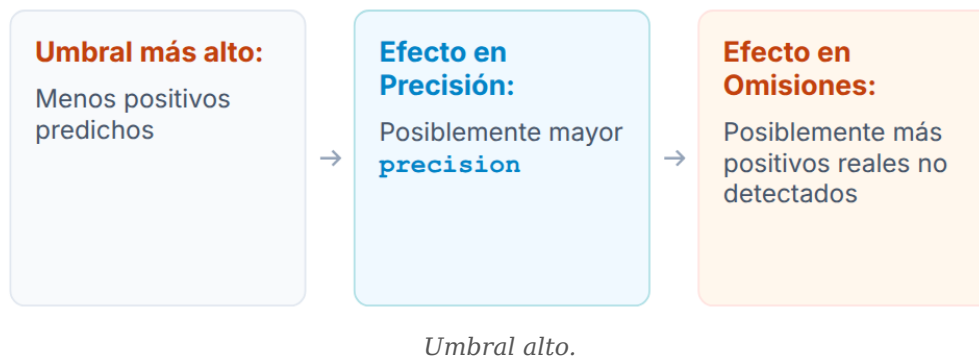
Una precision del 90% parece muy buena. Pero si en realidad había 100 clientes que abandonaron, el modelo detectó solo 9. Dejó pasar 91 abandonos. En una estrategia de retención, eso podría ser insuficiente.

Este ejemplo muestra que precision mide la calidad de las alertas positivas, no la cobertura de todos los positivos reales. Para responder cuántos positivos reales fueron detectados necesitaremos otra métrica: **recall**.

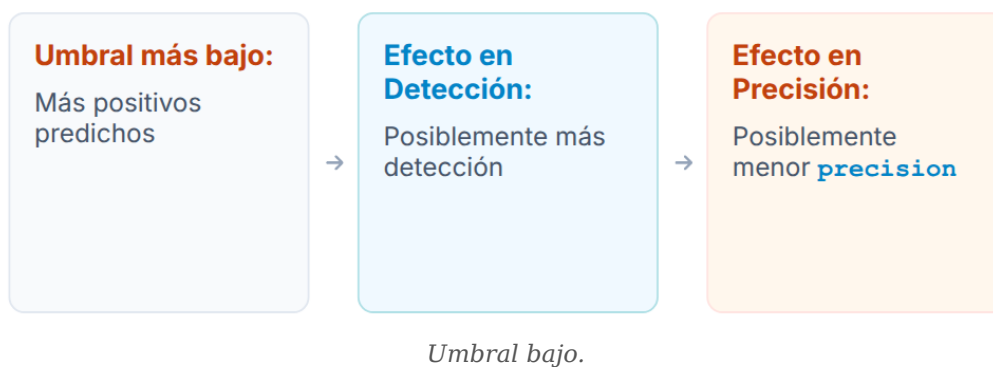
La precision suele ser prioritaria cuando el costo de un falso positivo es alto. Si marcar un caso como positivo implica revisar manualmente, bloquear una operación, contactar a una persona, iniciar una acción comercial o tomar una decisión sensible, queremos que esas predicciones positivas sean lo más confiables posible.

Pero si el objetivo principal es no dejar pasar casos importantes, mirar solo precision puede llevarnos a un modelo demasiado conservador. El modelo puede generar pocas alertas y acertar en la mayoría, pero ignorar muchos positivos reales.

También debemos recordar que precision depende del umbral de decisión. En modelos que producen probabilidades, como la regresión logística, cambiar el umbral modifica cuántos casos se predicen como positivos. Si subimos el umbral, el modelo será más exigente: marcará menos casos como positivos. Eso puede aumentar la precision, porque las alertas serán más selectivas, pero también puede hacer que se detecten menos positivos reales.



Si bajamos el umbral, el modelo marcará más casos como positivos. Esto puede ayudar a detectar más positivos reales, pero también puede aumentar los falsos positivos y reducir la precision.



Por eso, precision debe interpretarse junto con la matriz de confusión, el recall, el objetivo del problema y el costo de los errores. No es una métrica “mejor” que accuracy en todos los casos; responde una pregunta distinta.

En `scikit-learn`, podemos calcular precision con `precision_score`:

```
from sklearn.metrics import precision_score  
  
precision_score(y_test, y_pred)
```

Como siempre, debemos asegurarnos de que la clase positiva esté correctamente definida. Si la biblioteca interpreta como positiva una clase distinta de la que nos interesa, el cálculo puede ser correcto desde el punto de vista matemático, pero mal interpretado desde el punto de vista del problema.

En clasificación multiclase, precision puede calcularse por clase. La pregunta se adapta de esta forma: de todos los casos que el modelo predijo como una clase determinada, ¿cuántos realmente pertenecían a esa clase? Por ejemplo, si el modelo predice “riesgo alto”, precision para esa clase indica qué proporción de esos casos eran realmente de riesgo alto.

Esta lectura por clase es muy útil cuando algunas categorías son más importantes que otras. Un promedio general puede ocultar que el modelo tiene buena precisión en clases frecuentes y baja precisión en una clase crítica.

Precision mide qué tan confiables son las predicciones positivas del modelo. Responde la pregunta: de todos los casos marcados como positivos, ¿cuántos eran realmente positivos? Es una métrica clave cuando queremos reducir falsas alarmas o cuando actuar sobre una predicción positiva tiene un costo importante. Sin embargo, no indica cuántos positivos reales fueron detectados. Para eso necesitamos estudiar recall.

¿Qué mide recall?

Precision nos dice qué tan confiables son las predicciones positivas del modelo. Pero muchas veces necesitamos mirar el problema desde el punto de vista contrario: no queremos saber solamente si las alertas positivas fueron correctas, sino cuántos de los casos positivos reales logró encontrar el modelo.

Esa es la pregunta que responde **recall**, también llamado **sensibilidad**: De todos los casos que realmente eran positivos, ¿cuántos logró detectar el modelo?

Mientras precision mira los positivos predichos, recall mira los positivos reales. Su foco está en la capacidad del modelo para encontrar la clase positiva.

En clasificación binaria, recall se calcula con verdaderos positivos y falsos negativos:

$$\text{recall} = \text{VP} / (\text{VP} + \text{FN})$$

VP

positivos detectados

FN

positivos reales que quedaron sin detectar

Fórmula de cálculo.

El denominador representa todos los casos que realmente pertenecían a la clase positiva. Algunos fueron detectados correctamente por el modelo: esos son los verdaderos positivos. Otros fueron clasificados como negativos aunque eran positivos: esos son los falsos negativos.

Volvamos al ejemplo de abandono de clientes. La clase positiva es “abandona” y la matriz de confusión era:

		Predicción	
		permanece	abandona
Real:	permanece	50	10
	abandona	15	25

Matriz de confusión.

En esta matriz, los verdaderos positivos son 25 y los falsos negativos son 15. Los clientes que realmente abandonaron fueron 40 en total: 25 detectados correctamente y 15 no detectados.

Entonces:



El 62,5% de los clientes que realmente abandonaron.

Esto significa que el modelo detectó el 62,5% de los clientes que realmente abandonaron. Dicho de otra manera, de cada 100 clientes que iban a abandonar, el modelo habría identificado aproximadamente 63.

Esta métrica es especialmente importante cuando nos preocupa dejar pasar casos positivos. En abandono de clientes, un falso negativo es un cliente que realmente

abandona pero que el modelo clasifica como “permanece”. Si el objetivo del modelo es anticipar abandono para intervenir, esos casos no detectados son muy relevantes.

En detección de fraude, recall indica qué proporción de fraudes reales detectó el modelo. Un recall bajo significa que muchos fraudes pasan desapercibidos. En diagnóstico asistido, indica qué proporción de casos que requerían revisión fueron efectivamente marcados. En mantenimiento predictivo, puede indicar qué proporción de fallas reales fueron anticipadas.

Podemos resumirlo así:



Recall alto vs bajo.

Recall es clave cuando el costo de un falso negativo es alto. Si no detectar un caso positivo puede tener consecuencias graves, necesitamos prestarle mucha atención. Esto ocurre en problemas como detección de enfermedades, alertas de seguridad, fraude, fallas críticas, abandono de clientes importantes o incumplimientos de alto impacto.

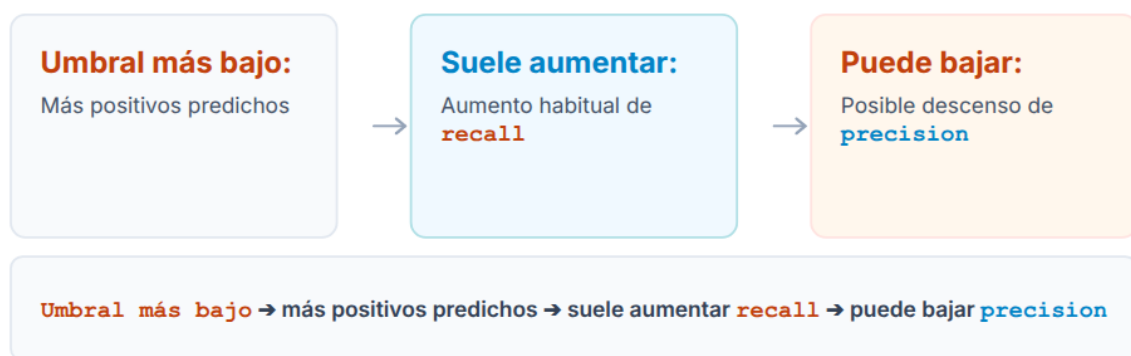
Sin embargo, igual que ocurría con precision, recall no cuenta toda la historia. Un modelo puede tener recall muy alto simplemente marcando muchos casos como positivos. Si predice positivo casi todo el tiempo, detectará muchos positivos reales, pero también generará muchas falsas alarmas.

Imaginemos un modelo que clasifica todos los clientes como “abandona”. Si había 40 clientes que realmente abandonaron, el modelo detectará los 40. Su recall será: $40 / 40 = 1.00$

Pero también habrá marcado como “abandona” a todos los clientes que en realidad permanecieron. El recall sería perfecto, pero la precision podría ser muy baja. El modelo detecta todos los positivos, pero a costa de generar demasiadas alertas.

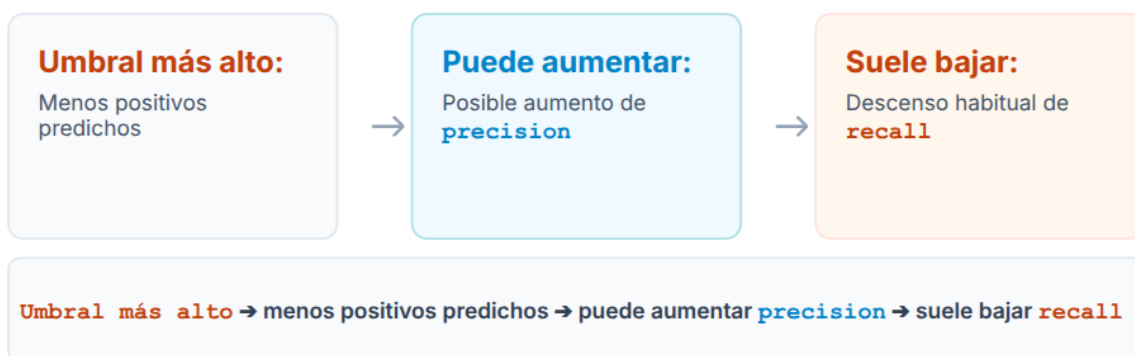
Este ejemplo muestra por qué no conviene mirar recall de manera aislada. Un recall alto es valioso cuando queremos encontrar positivos reales, pero debe interpretarse junto con la cantidad de falsos positivos que el modelo produce. Detectar todo puede ser útil en algunos contextos, pero inviable en otros si genera demasiadas revisiones innecesarias.

La relación entre precision y recall suele implicar un equilibrio. Si bajamos el umbral de decisión, el modelo clasifica más casos como positivos. Esto puede aumentar recall, porque se detectan más positivos reales. Pero también puede reducir precision, porque aumentan las falsas alarmas.



Umbral bajo.

Si subimos el umbral, el modelo será más exigente para predecir positivo. Esto puede aumentar precision, porque las alertas serán más selectivas. Pero también puede reducir recall, porque algunos positivos reales quedarán sin detectar.



Umbral alto.

Este equilibrio no se resuelve de manera automática. Depende del problema. En diagnóstico asistido, puede ser preferible tener recall alto, aunque eso genere más casos para revisión. En una campaña comercial costosa, quizá se prefiera una

precision más alta para no gastar recursos en demasiados clientes que no estaban realmente en riesgo. En fraude, la decisión dependerá del costo de revisar transacciones frente al costo de dejar pasar fraudes.

Por eso, la pregunta no es simplemente si queremos más recall o más precision. La pregunta correcta es: qué tipo de error es más costoso en este problema, cuántas falsas alarmas podemos tolerar, cuántos casos positivos no detectados son aceptables y qué capacidad operativa tenemos para actuar sobre las alertas.

En `scikit-learn`, recall puede calcularse con `recall_score`:

```
from sklearn.metrics import recall_score

recall_score(y_test, y_pred)
```

Como siempre, debemos verificar cuál es la clase positiva. Si la clase positiva está mal definida, recall medirá la detección de una clase que tal vez no sea la que realmente nos interesa. Esto es especialmente importante cuando las etiquetas están codificadas como `0` y `1`, o cuando usamos textos como `"si"` y `"no"`, `"riesgo"` y `"normal"`.

En clasificación multiclase, recall también puede analizarse por clase. Para una clase determinada, responde: de todos los casos que realmente pertenecían a esa clase, ¿cuántos fueron clasificados correctamente como esa clase? Esto permite detectar si el modelo ignora alguna categoría o si confunde sistemáticamente una clase con otra.

Por ejemplo, en un modelo de riesgo con clases “bajo”, “medio” y “alto”, podríamos calcular recall para la clase “alto”. Si ese recall es bajo, significa que muchos casos de riesgo alto están siendo clasificados como medio o bajo. Aunque la accuracy general parezca razonable, este problema podría ser grave si la clase “alto” es la más importante para la toma de decisiones.

Recall nos obliga a mirar el modelo desde el punto de vista de los casos reales que queremos encontrar. No pregunta cuán confiables son las alertas, sino cuántos casos importantes logró capturar. Por eso es una métrica indispensable en problemas donde la clase positiva representa eventos críticos, raros o especialmente relevantes.

Recall mide la capacidad del modelo para detectar los casos positivos reales. Se calcula como la proporción de verdaderos positivos sobre todos los positivos reales. Es una métrica clave cuando queremos reducir falsos negativos y evitar que casos importantes pasen desapercibidos. Sin embargo, un recall alto puede venir acompañado de muchas falsas alarmas, por lo que debe analizarse junto con precision, la matriz de confusión y el contexto del problema.

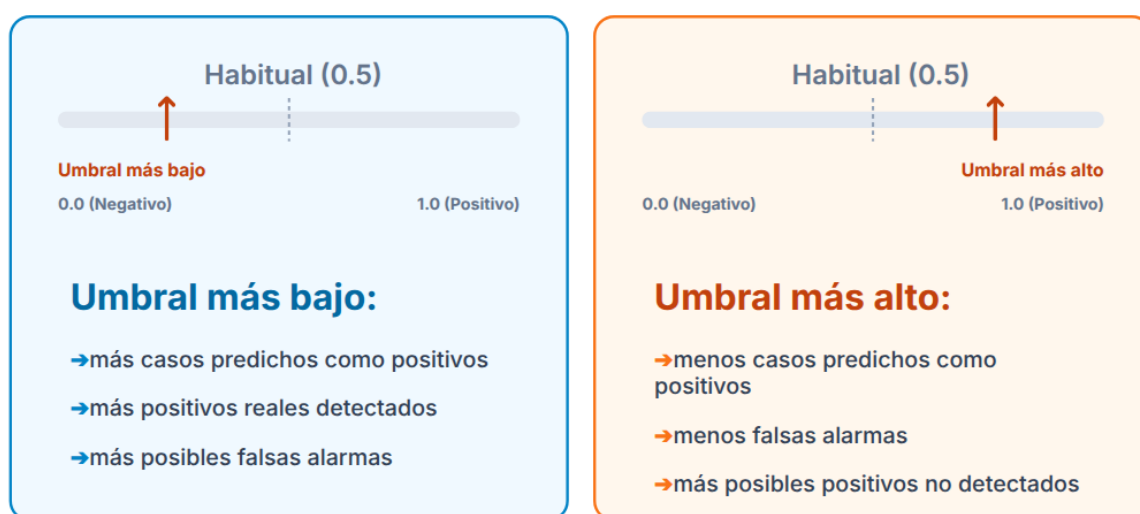
El equilibrio entre ambas

Precision y recall miden aspectos distintos del comportamiento de un clasificador. Precision se concentra en la confiabilidad de las predicciones positivas. Recall se concentra en la capacidad para detectar los positivos reales. Ambas métricas son útiles, pero muchas veces se mueven en direcciones opuestas.

Esta tensión aparece con claridad cuando un modelo produce probabilidades y debemos elegir un umbral de decisión. Si bajamos el umbral, el modelo predice más casos como positivos. Esto suele aumentar recall, porque detecta más positivos reales. Pero también puede generar más falsos positivos, lo que puede reducir precision.

Si subimos el umbral, ocurre lo contrario. El modelo se vuelve más exigente para predecir positivo. Esto puede aumentar precision, porque las alertas que genera tienden a ser más confiables. Pero también puede reducir recall, porque algunos positivos reales ya no superan el umbral y quedan sin detectar.

Podemos resumirlo así:



Umbral bajo / alto.

Este equilibrio es una de las ideas más importantes en evaluación de clasificadores. No siempre podemos maximizar precision y recall al mismo tiempo. Mejorar una de ellas puede empeorar la otra. Por eso, la pregunta no es cuál métrica es “mejor” en abstracto, sino qué tipo de error es más importante controlar en el problema real.

Volvamos al ejemplo de abandono de clientes. Si una empresa quiere detectar la mayor cantidad posible de clientes en riesgo, puede preferir un modelo con recall alto. Esto significa aceptar que algunos clientes marcados como “abandona”

finalmente permanecerán. Habrá falsas alarmas, pero la empresa reducirá la cantidad de abandonos no detectados.

En cambio, si cada acción de retención es costosa, limitada o delicada, quizá la empresa prefiera mayor precision. En ese caso, quiere que los clientes señalados por el modelo tengan alta probabilidad real de abandonar. Se generarán menos alertas, pero posiblemente se dejarán pasar algunos clientes que también estaban en riesgo.

Priorizar recall:

- detectar más abandonos reales,
- aunque haya más falsas alarmas.

Priorizar precision:

- actuar sobre alertas más confiables,
- aunque se detecten menos abandonos reales.

¿Qué priorizamos?.

En detección de fraude ocurre algo parecido. Si el objetivo es evitar que pasen operaciones fraudulentas, puede priorizarse recall. El sistema marcará más transacciones como sospechosas para capturar más fraudes reales. Pero eso puede generar muchas revisiones sobre operaciones legítimas.

Si, en cambio, bloquear o revisar una transacción normal genera mucho costo, molestia o riesgo de perder clientes, puede priorizarse precision. El sistema será más cuidadoso antes de marcar una operación como sospechosa. Pero algunos fraudes podrían pasar sin ser detectados.

En medicina o diagnóstico asistido, muchas veces se presta gran atención al recall cuando la clase positiva representa casos que requieren revisión. Un falso negativo puede significar que un caso importante no sea detectado. Por eso, puede ser aceptable generar más falsos positivos si eso ayuda a reducir los casos positivos no detectados. Pero tampoco se pueden ignorar las falsas alarmas: demasiadas revisiones innecesarias pueden saturar el sistema y afectar la atención.

El equilibrio entre precision y recall no se decide mirando solo números. Se decide entendiendo el contexto. Debemos preguntarnos qué ocurre cuando el modelo se equivoca de cada manera. Por ejemplo:

- ¿Qué pasa si el modelo genera una falsa alarma?
- ¿Qué pasa si el modelo no detecta un caso positivo?
- ¿Qué error es más costoso?
- ¿Cuántas alertas podemos revisar?
- ¿Qué consecuencias tiene actuar sobre una predicción positiva?

Estas preguntas conectan la evaluación técnica con el uso real del modelo. Un clasificador no existe solo para obtener una métrica alta. Existe para apoyar una decisión, priorizar casos, automatizar una parte de un proceso o generar información útil. La métrica debe reflejar ese objetivo.

También es importante entender que el equilibrio no siempre se resuelve con una única respuesta. En algunos proyectos podemos construir distintos escenarios. Por ejemplo, un equipo puede evaluar qué ocurre con un umbral conservador, uno intermedio y uno más sensible. En cada caso, observará cómo cambian precision, recall, falsos positivos y falsos negativos.

- **Umbral 0.30** : más detección, más alertas.
- **Umbral 0.50** : equilibrio inicial.
- **Umbral 0.70** : menos alertas, más exigencia.

Esta comparación permite decidir con más criterio. No se trata de aceptar automáticamente el umbral por defecto, sino de comprender qué comportamiento produce el modelo bajo distintas reglas de decisión.

Sin embargo, no todos los modelos o situaciones se resuelven ajustando umbrales. A veces el problema está en las variables, en los datos, en la calidad de las etiquetas, en el desbalance de clases o en el propio modelo elegido. Si las probabilidades no separan bien los casos positivos de los negativos, cambiar el umbral solo moverá el problema de lugar. Puede aumentar recall sacrificando mucha precision, o aumentar precision dejando casi sin detectar positivos.

Por eso, antes de buscar un equilibrio, necesitamos mirar la matriz de confusión, revisar ejemplos concretos de errores y comparar el modelo con referencias simples. Precision y recall son métricas, pero la evaluación completa requiere interpretación.

También conviene recordar que el equilibrio entre ambas puede variar por clase en problemas multiclase. Un modelo puede tener buena precision para una clase y bajo recall para otra. Por ejemplo, puede ser muy confiable cuando predice “riesgo alto”, pero detectar solo una parte de los casos que realmente son de riesgo alto. Según el problema, eso puede ser aceptable o no.

En problemas multiclase, mirar promedios generales puede ocultar estos matices. Si una clase es crítica, debemos revisar precision y recall específicamente para esa clase. No basta con mirar el promedio global.

El equilibrio entre precision y recall también se relaciona con la capacidad operativa. Supongamos que un equipo puede revisar 100 alertas por día. Un modelo con recall muy alto pero que genera 5000 alertas diarias puede ser impracticable. En cambio, un modelo con precision muy alta pero solo 5 alertas por día puede ser demasiado

conservador si deja pasar muchos casos importantes. La evaluación debe considerar qué se puede hacer con las predicciones.

Precision y recall ofrecen dos miradas complementarias. Precision nos ayuda a controlar falsas alarmas; recall nos ayuda a controlar casos positivos no detectados. Muchas veces mejorar una implica empeorar la otra, especialmente cuando modificamos el umbral de decisión. El equilibrio adecuado depende del problema, del costo de los errores, de la capacidad para actuar sobre las alertas y de las consecuencias reales de las predicciones. Evaluar un clasificador implica decidir qué tipo de error importa más, no simplemente buscar la métrica más alta.

F1-score

Precision y recall nos dan dos miradas complementarias sobre el desempeño de un clasificador. Precision nos dice qué tan confiables son las predicciones positivas. Recall nos dice cuántos positivos reales logró detectar el modelo. Pero muchas veces necesitamos una métrica que combine ambas ideas en un único valor. Para eso se utiliza el **F1-score**.

El F1-score busca resumir el equilibrio entre precision y recall. No reemplaza el análisis detallado de ambas métricas, pero puede ser útil cuando queremos comparar modelos considerando, al mismo tiempo, falsas alarmas y positivos no detectados.

Podemos pensarlo así:

precision

¿cuántas alertas
positivas eran
correctas?

$$VP / (VP + FP)$$

recall

¿cuántos positivos
reales fueron
detectados?

$$VP / (VP + FN)$$

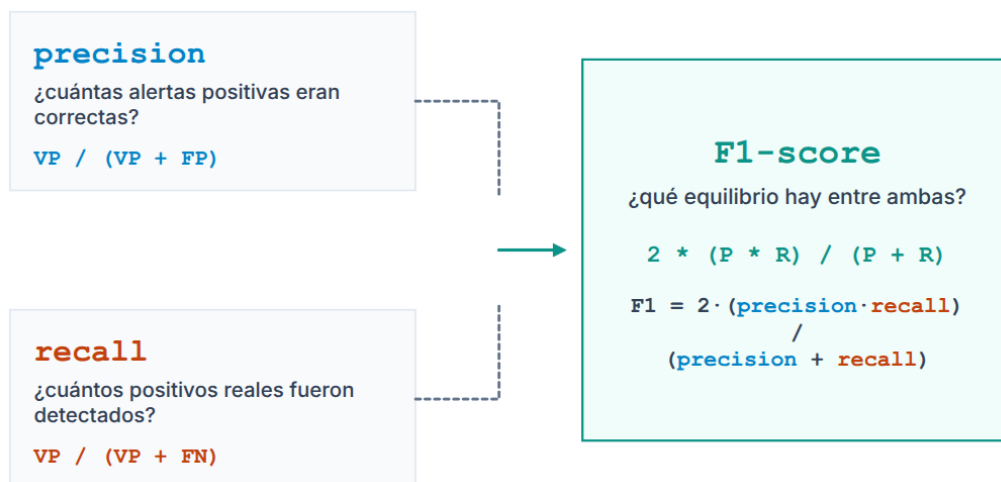
F1-score

¿qué equilibrio hay
entre ambas?

$$2 * (P * R) / (P + R)$$

El F1-score busca resumir el equilibrio entre precision y recall.

La fórmula del F1-score es:



F1-score.

No es necesario memorizar la fórmula para interpretar la métrica, pero sí conviene entender su comportamiento. El F1-score será alto cuando precision y recall sean altas al mismo tiempo. Si una de las dos es baja, el F1-score también se ve afectado.

Esto es importante porque evita que un modelo parezca bueno solo por destacarse en una dimensión. Un clasificador puede tener una precision muy alta porque genera pocas alertas y casi todas son correctas, pero puede tener recall bajo porque deja pasar muchos positivos reales. Otro modelo puede tener recall muy alto porque detecta casi todos los positivos, pero precision baja porque produce demasiadas falsas alarmas. El F1-score penaliza estos desequilibrios.

Volvamos al ejemplo de abandono de clientes. La matriz de confusión era:

		Predicción	
		permanece	abandona
Real:	permanece	50	10
	abandona	15	25

Matriz de confusión.

Habíamos calculado:

precision $VP / (VP + FP)$ cuando VP = 25, FP = 10 $25 / (25 + 10)$ ≈ 0.71	recall $VP / (VP + FN)$ cuando VP = 25, FN = 15 $25 / (25 + 15)$ $= 0.625$
---	---

precision y recall

El F1-score combina ambas métricas:

Paso 1: Sustitución de precision y recall

$$F1 \approx 2 \cdot (0.71 \cdot 0.625) / (0.71 + 0.625)$$

Paso 2: Resultado Final

$$F1 \approx 0.66$$

F1-score

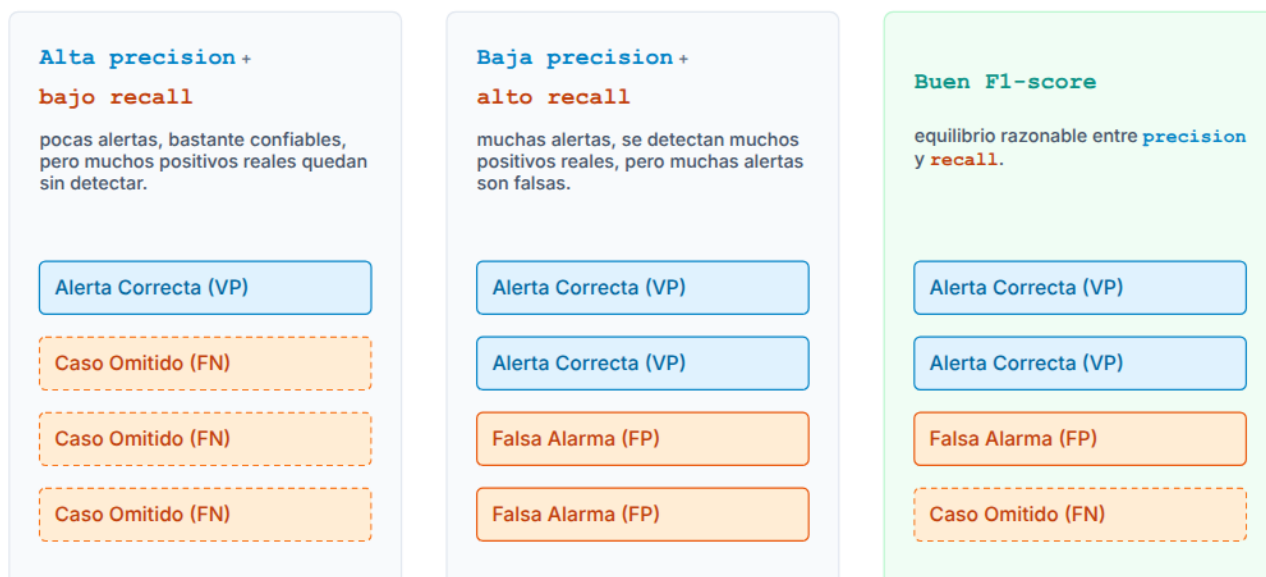
Este valor resume el desempeño del modelo sobre la clase positiva considerando dos aspectos: qué tan confiables son las alertas de abandono y qué proporción de abandonos reales fueron detectados.

El F1-score resulta especialmente útil cuando las clases están desbalanceadas. En esos casos, la accuracy puede ser engañosa, porque un modelo puede acertar muchos casos de la clase mayoritaria y fallar en la clase minoritaria. Como el F1-score se apoya en verdaderos positivos, falsos positivos y falsos negativos, pone el foco sobre la clase positiva o clase de interés.

Imaginemos un modelo de detección de fraude que casi nunca marca transacciones como sospechosas. Cuando lo hace, suele acertar. Su precision puede ser alta. Pero si deja pasar la mayoría de los fraudes reales, su recall será bajo. El F1-score reflejará ese problema porque no basta con generar pocas alertas correctas; también importa detectar una proporción razonable de los fraudes existentes.

También puede ocurrir lo contrario. Un modelo puede marcar muchísimas transacciones como sospechosas y detectar casi todos los fraudes. Su recall será alto. Pero si la mayoría de sus alertas son falsas, la precision será baja. El F1-score también penalizará esa situación.

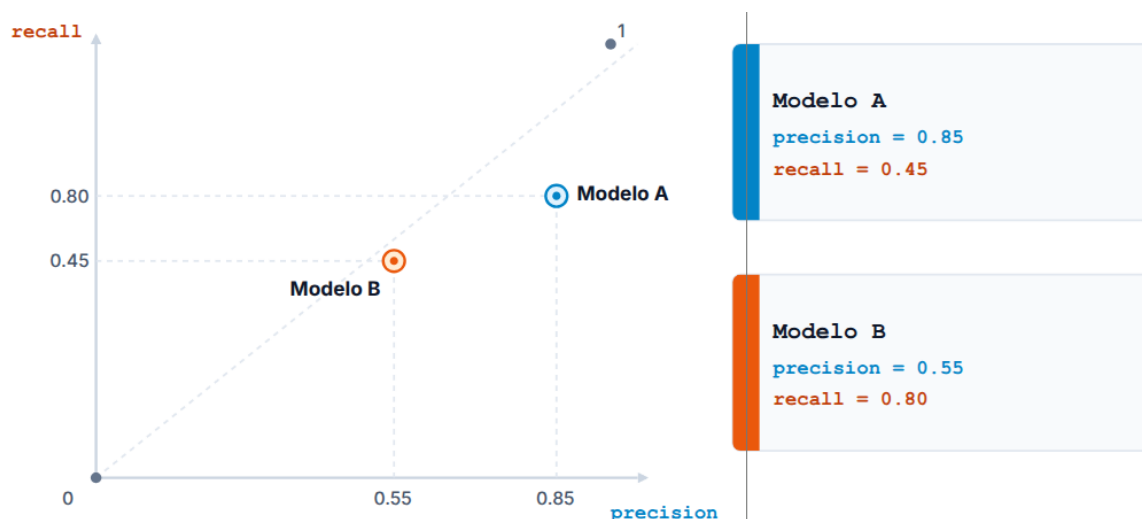
Podemos resumir estos casos así:



Resumen.

Sin embargo, el F1-score no es una métrica perfecta. Al resumir precision y recall en un solo número, también puede ocultar diferencias importantes. Dos modelos pueden tener F1-score parecido y comportamientos muy distintos. Uno puede ser más preciso pero menos sensible; otro puede detectar más positivos reales pero generar más falsas alarmas.

Por ejemplo, supongamos dos modelos para detectar clientes con riesgo de abandono:



El F1-score no es una métrica perfecta.

El Modelo A genera alertas más confiables, pero deja pasar muchos abandonos reales. El Modelo B detecta más abandonos, pero genera más falsas alarmas. Aunque sus F1-score pudieran estar relativamente cerca, la elección entre ellos dependería del objetivo. Si la empresa tiene pocos recursos para contactar clientes, tal vez

prefiera el Modelo A. Si quiere detectar la mayor cantidad posible de abandonos, tal vez prefiera el Modelo B.

Esto muestra que el F1-score es útil para comparar, pero no debe reemplazar la interpretación. Siempre conviene mirar precision y recall por separado, junto con la matriz de confusión y el contexto del problema.

El F1-score suele ser una buena métrica cuando queremos un equilibrio entre falsos positivos y falsos negativos, especialmente si la clase positiva es importante y las clases están desbalanceadas. Pero si sabemos que un tipo de error es claramente más costoso que el otro, quizá convenga priorizar directamente precision o recall en lugar de buscar equilibrio.

En diagnóstico asistido, por ejemplo, puede ser preferible un modelo con mayor recall aunque su F1-score no sea el máximo, porque dejar pasar casos importantes puede ser grave. En una campaña comercial muy costosa, puede ser preferible un modelo con mayor precision, aunque detecte menos positivos. La métrica debe seguir al problema, no al revés.

El F1-score también depende del umbral de decisión. Si cambiamos el umbral, cambian las clases predichas. Si cambian las clases predichas, cambian los verdaderos positivos, falsos positivos y falsos negativos. Por lo tanto, también cambian precision, recall y F1-score.

Esto significa que un mismo modelo puede tener distintos valores de F1-score según el umbral que usemos para convertir probabilidades en clases. El umbral de 0.5 puede ser un punto de partida, pero no siempre produce el mejor equilibrio. En algunos proyectos, podemos evaluar varios umbrales y observar dónde se obtiene un balance más adecuado entre precision y recall.

En `scikit-learn`, el F1-score puede calcularse con `f1_score`:

```
from sklearn.metrics import f1_score

f1_score(y_test, y_pred)
```

Como ocurre con precision y recall, debemos verificar cuál es la clase positiva. Si la clase positiva no corresponde a la clase que realmente queremos analizar, el F1-score puede ser interpretado de manera equivocada.

En clasificación multiclase, el F1-score puede calcularse por clase. Esto permite ver si el modelo funciona bien para algunas categorías y mal para otras. Por ejemplo, en un problema de riesgo con clases “bajo”, “medio” y “alto”, podríamos tener buen F1-

score para “bajo” y mal F1-score para “alto”. Si la clase “alto” es la más importante, el promedio general no alcanza para decidir si el modelo es adecuado.

Cuando se resumen métricas en problemas multiclase, aparecen distintas formas de promedio, como macro, micro o weighted. No necesitamos profundizar todavía en esos detalles, pero sí conviene saber que los promedios pueden ocultar diferencias entre clases. Siempre que el problema lo justifique, debemos revisar las métricas clase por clase.

El F1-score es especialmente valioso como medida de comparación inicial cuando nos interesan tanto las falsas alarmas como los positivos no detectados. Pero la elección final de un modelo no debería depender únicamente de él. Debe relacionarse con el objetivo del proyecto, el costo de los errores, la distribución de clases y la forma en que se usarán las predicciones.

El F1-score combina precision y recall en una sola métrica. Es alto cuando el modelo logra un buen equilibrio entre generar alertas confiables y detectar una proporción importante de los casos positivos reales. Es útil en problemas con clases desbalanceadas o cuando necesitamos considerar al mismo tiempo falsos positivos y falsos negativos. Sin embargo, debe interpretarse junto con precision, recall, matriz de confusión y contexto real del problema.

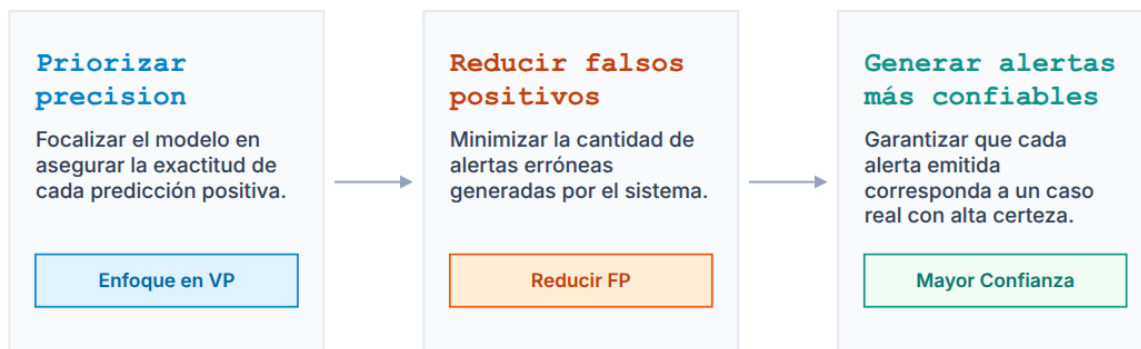
¿Cuándo conviene priorizar precision?

Conviene priorizar precision cuando el problema exige que las predicciones positivas sean muy confiables. Esto ocurre cuando marcar un caso como positivo genera una acción costosa, delicada, molesta o difícil de revertir. En esas situaciones, no queremos que el modelo produzca demasiadas falsas alarmas.

Recordemos la pregunta que responde precision: De todos los casos que el modelo predijo como positivos, ¿cuántos eran realmente positivos?

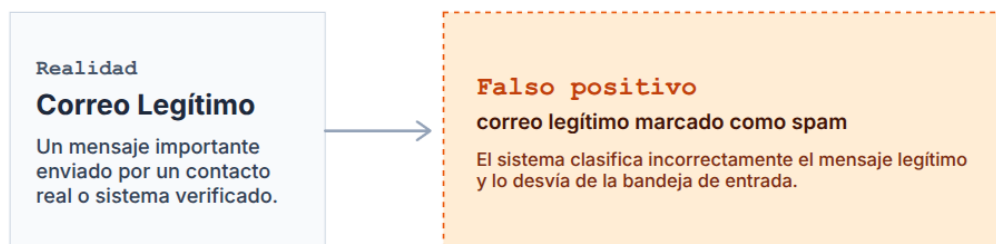
Si la precision es alta, significa que la mayoría de las alertas positivas corresponden a casos positivos reales. Si la precision es baja, significa que muchas alertas son falsos positivos.

Por eso, priorizar precision significa intentar reducir los falsos positivos. No se trata simplemente de obtener una métrica alta, sino de controlar un tipo particular de error: los casos negativos que el modelo marca por error como positivos.



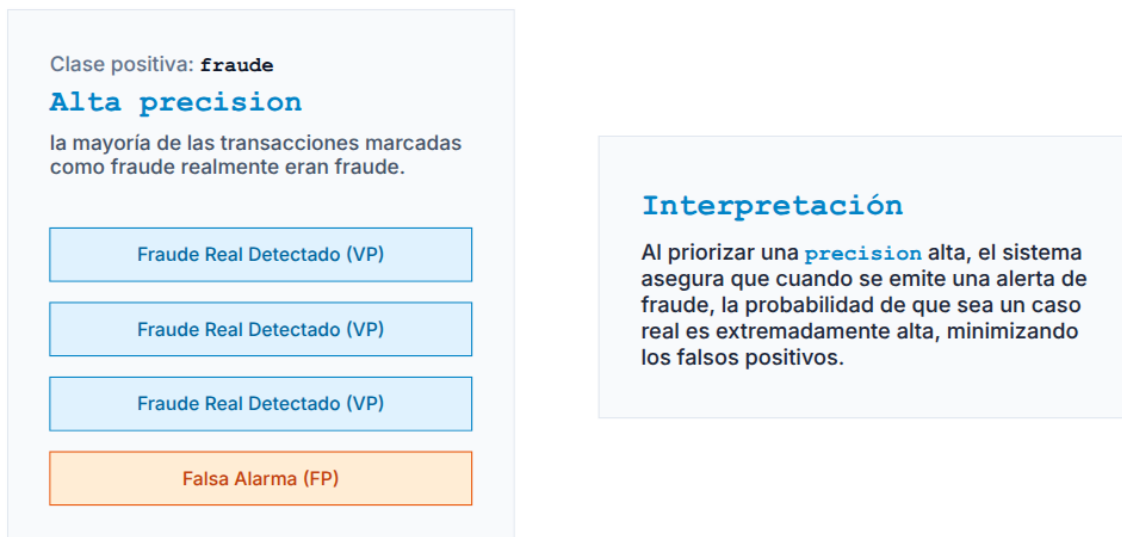
Un ejemplo claro aparece en filtros de spam. Si un sistema clasifica un correo legítimo como spam, el usuario puede no verlo a tiempo. Tal vez sea un mensaje importante de trabajo, una confirmación de turno, una comunicación bancaria o una respuesta esperada. En este contexto, un falso positivo puede ser bastante molesto o incluso perjudicial.

Por eso, un filtro de spam no debería enviar alegremente correos legítimos a la carpeta de spam. Puede ser preferible que algunos correos no deseados lleguen a la bandeja de entrada antes que ocultar mensajes importantes. En este caso, interesa que los correos marcados como spam sean realmente spam. Es decir, interesa una precisión alta para la clase “spam”.



Algo similar ocurre en sistemas de fraude cuando una predicción positiva puede bloquear una operación. Si el modelo marca una transacción normal como fraudulenta, puede impedir una compra legítima, generar una revisión innecesaria o afectar la experiencia del usuario. Si esto ocurre con demasiada frecuencia, las personas pueden perder confianza en el sistema.

En ese contexto, puede ser importante que las transacciones marcadas como sospechosas tengan alta probabilidad de ser realmente problemáticas. Esto no significa que recall deje de importar. Detectar fraudes reales también es fundamental. Pero si cada alerta tiene consecuencias fuertes, precisión se vuelve una métrica central.



También puede convenir priorizar precision cuando la capacidad de revisión es limitada. Supongamos que un equipo de seguridad puede revisar solo 100 alertas por día. Si el modelo genera miles de alertas con baja precision, el equipo se saturará y muchas alertas serán inútiles. En ese caso, puede ser mejor recibir menos alertas, pero más confiables.

Esta situación aparece en muchos contextos: revisión de transacciones, moderación de contenido, análisis de incidentes de seguridad, auditorías, derivación de reclamos o selección de casos para inspección. Cuando los recursos para actuar son limitados, una baja precision puede volver impracticable el uso del modelo.

También en campañas comerciales puede ser importante priorizar precision. Si contactar a un cliente tiene un costo alto, si se ofrece un beneficio económico importante o si se quiere evitar molestar a personas que no necesitan la intervención, conviene que los casos seleccionados sean lo más confiables posible. Un modelo con alta precision permite concentrar los recursos en un grupo más pequeño, pero con mayor proporción de casos realmente relevantes.

Por ejemplo, si una empresa solo puede ofrecer una promoción especial a 500 clientes, quizás no quiera contactar a todos los que tienen una mínima señal de riesgo. Preferirá seleccionar aquellos con mayor probabilidad de abandono real. En ese caso, puede aceptar detectar menos abandonos totales a cambio de que los clientes contactados sean más pertinentes.



Otro caso aparece cuando una predicción positiva puede tener consecuencias injustas o sensibles. En riesgo crediticio, marcar una solicitud como riesgosa puede derivar en revisión adicional, condiciones menos favorables o rechazo. Si el modelo produce muchos falsos positivos, muchas personas o solicitudes que no habrían presentado problemas pueden verse afectadas innecesariamente.

En estos contextos, no alcanza con decir que el modelo detecta muchos casos de riesgo. También debemos mirar si las predicciones positivas son confiables, si las variables usadas son pertinentes y si el proceso de decisión incorpora revisión adecuada. Una alta precisión puede ser importante, aunque no elimina otras responsabilidades éticas y legales.

Priorizar precisión suele implicar usar un umbral de decisión más alto cuando el modelo produce probabilidades. Al exigir una probabilidad mayor para predecir la clase positiva, el modelo se vuelve más selectivo. Marca menos casos como positivos, pero esos casos pueden ser más confiables.

Pero esta estrategia tiene un costo. Al subir el umbral, algunos positivos reales pueden quedar por debajo del corte y ser clasificados como negativos. Es decir, podemos reducir falsos positivos, pero aumentar falsos negativos. Por eso, priorizar precisión puede reducir recall.

Este punto es clave. Priorizar precisión no significa mejorar el modelo en todos los sentidos. Significa aceptar un equilibrio: queremos predicciones positivas más confiables, aunque eso implique detectar menos casos positivos reales.

Por ejemplo, un modelo puede pasar de este comportamiento: con un umbral bajo detecta muchos abandonos, pero genera muchas falsas alarmas; a este otro: con un umbral alto genera menos alertas, las alertas son más confiables, pero deja pasar más abandonos reales.

La decisión será adecuada o no según el problema. Si el objetivo es actuar solo sobre casos muy seguros, priorizar precision puede tener sentido. Si el objetivo es no dejar pasar casi ningún caso positivo, puede ser peligroso.

Por eso, antes de priorizar precision conviene preguntarse:

- ¿Qué costo tiene una falsa alarma?
- ¿Cuántas alertas podemos revisar?
- ¿Qué consecuencias tiene marcar un caso como positivo?
- ¿Es aceptable dejar pasar algunos positivos reales?

Estas preguntas ayudan a evitar una interpretación automática. Una precision alta puede sonar siempre deseable, pero puede lograrse a costa de un recall demasiado bajo. En algunos problemas, eso sería inaceptable.

Un modelo que predice positivo muy pocas veces puede tener precision alta simplemente porque solo se anima a marcar los casos más evidentes. Pero si deja sin detectar la mayoría de los positivos reales, su utilidad puede ser limitada. Por eso, aun cuando prioricemos precision, debemos seguir mirando recall y la matriz de confusión.

Conviene priorizar precision cuando los falsos positivos son costosos, cuando las alertas deben ser confiables, cuando la capacidad de revisión es limitada o cuando actuar sobre una predicción positiva tiene consecuencias importantes. Priorizar precision ayuda a reducir falsas alarmas, pero puede aumentar los positivos no detectados. Por eso, debe elegirse con conciencia del problema real y no como una preferencia automática.

¿Cuándo conviene priorizar recall?

Conviene priorizar recall cuando el problema exige detectar la mayor cantidad posible de casos positivos reales. Esto ocurre cuando dejar pasar un caso positivo puede tener consecuencias importantes. En esas situaciones, el error que más preocupa no es la falsa alarma, sino el caso que el modelo no detectó.

Recordemos la pregunta que responde recall: De todos los casos que realmente eran positivos, ¿cuántos logró detectar el modelo?

Si el recall es alto, significa que el modelo encuentra una gran proporción de los casos positivos reales. Si el recall es bajo, significa que muchos casos importantes quedan clasificados como negativos y pasan desapercibidos.

Por eso, priorizar recall significa intentar reducir los falsos negativos. No se trata simplemente de obtener un número alto, sino de controlar un tipo de error específico: los casos positivos que el modelo deja sin detectar.

Un ejemplo claro aparece en diagnóstico asistido. Si la clase positiva representa “requiere revisión”, un falso negativo sería un caso que realmente necesitaba ser revisado, pero que el modelo clasificó como si no requiriera atención especial. En muchos contextos de salud, este tipo de error puede ser especialmente grave.

En una situación así, puede ser preferible que el modelo genere más alertas, aunque algunas sean falsas, antes que dejar pasar casos importantes. Esto no significa que los falsos positivos no importen. Demasiadas alertas pueden saturar al equipo médico, producir demoras o generar revisiones innecesarias. Pero si el costo de no detectar un caso relevante es muy alto, recall se vuelve una métrica central.

También conviene priorizar recall en ciertos problemas de fraude. Si una transacción fraudulenta no es detectada, puede generar una pérdida económica, un problema de seguridad o un daño para el usuario. Un sistema con bajo recall puede parecer ordenado porque genera pocas alertas, pero si deja pasar muchos fraudes reales, no cumple su función principal.

En estos casos, el objetivo puede ser detectar la mayor cantidad posible de fraudes, aun aceptando una cantidad mayor de transacciones normales enviadas a revisión. La decisión dependerá del costo de revisar operaciones legítimas frente al costo de dejar pasar fraudes reales.

Otro ejemplo aparece en seguridad informática. Si un modelo intenta detectar accesos sospechosos, malware, intentos de intrusión o comportamientos anómalos, un falso negativo puede significar que una amenaza real no sea identificada. En contextos donde el riesgo de no detectar es alto, puede priorizarse recall para aumentar la cobertura de detección.

Sin embargo, también aquí hay un equilibrio. Un sistema de seguridad que genera demasiadas alertas falsas puede saturar a los analistas. Cuando hay demasiadas alertas, las personas pueden empezar a ignorarlas o no tener tiempo para revisarlas con cuidado. Por eso, priorizar recall no significa aceptar cualquier cantidad de falsos positivos sin análisis. Significa reconocer que, en ese problema, dejar pasar positivos reales es especialmente costoso.

En abandono de clientes, priorizar recall puede tener sentido si la organización quiere detectar la mayor cantidad posible de clientes en riesgo. Un falso negativo sería un cliente que realmente abandona pero que el modelo clasifica como

“permanece”. Si la empresa depende fuertemente de la retención, esos casos no detectados pueden representar oportunidades perdidas.

En este escenario, la empresa puede aceptar contactar a algunos clientes que finalmente no habrían abandonado, si eso permite cubrir mejor al conjunto de clientes realmente en riesgo. Pero si cada contacto o beneficio ofrecido tiene un costo elevado, no podrá priorizar recall sin considerar precision.

Priorizar `recall` suele implicar utilizar un umbral de decisión más bajo cuando el modelo produce probabilidades. De ese modo, más casos superan el punto de corte y son clasificados como positivos, lo que permite detectar una mayor cantidad de positivos reales.

Sin embargo, al ampliar la cantidad de casos clasificados como positivos, también suelen aumentar los falsos positivos. En otras palabras, un umbral bajo ayuda a reducir los casos positivos que pasan inadvertidos, pero genera más falsas alarmas. Por el contrario, un umbral alto produce menos alertas incorrectas, aunque aumenta el riesgo de no detectar algunos positivos reales. Por eso, priorizar `recall` puede reducir `precision`.

La elección depende del problema. En un sistema destinado a detectar casos críticos, puede ser razonable aceptar más falsas alarmas para reducir los falsos negativos. En cambio, si cada alerta implica una consecuencia importante o costosa, esa estrategia podría resultar inviable.

Por eso, antes de priorizar recall conviene preguntarse:

- ¿Qué costo tiene no detectar un positivo real?
- ¿Cuántas falsas alarmas podemos tolerar?
- ¿Qué capacidad tenemos para revisar alertas?
- ¿Qué ocurre si el modelo marca demasiados casos?

Estas preguntas son necesarias porque un recall alto puede lograrse de una manera muy poco útil: prediciendo positivo casi siempre. En ese caso, el modelo detectará muchos positivos reales, pero generará tantas falsas alarmas que tal vez no sea práctico.

Por ejemplo, si un modelo de fraude marca como sospechosas casi todas las transacciones, probablemente tenga recall alto. Pero si la mayoría de las alertas son falsas, el sistema puede volverse inutilizable. Detectar mucho no alcanza si la cantidad de alertas supera la capacidad de revisión o si afecta demasiado a los usuarios.

Por eso, aun cuando prioricemos recall, debemos seguir mirando precision y la matriz de confusión. El objetivo no es maximizar una métrica sin límite, sino encontrar un comportamiento útil para el contexto real.

En los problemas multiclase también es importante analizar el recall de cada clase por separado. Supongamos que un modelo clasifica incidentes de seguridad como de nivel “bajo”, “medio” o “alto”. Si la categoría “alto” representa incidentes críticos, su recall será especialmente relevante.

Un recall bajo para esa clase indica que muchos incidentes críticos están siendo clasificados erróneamente como de gravedad media o baja. Aunque la accuracy global del modelo sea buena, este comportamiento puede resultar inaceptable.

Por eso, cuando una clase tiene una importancia particular o los errores asociados a ella pueden provocar consecuencias graves, no alcanza con observar el rendimiento general del modelo: es necesario analizar explícitamente su recall.

Conviene priorizar recall cuando los falsos negativos son especialmente costosos y necesitamos detectar la mayor cantidad posible de casos positivos reales. Esto ocurre en contextos como diagnóstico asistido, fraude, seguridad, fallas críticas, abandono de clientes importantes o riesgos que no queremos dejar pasar. Priorizar recall ayuda a reducir positivos no detectados, pero puede aumentar falsas alarmas. Por eso, debe equilibrarse con precision, capacidad operativa y consecuencias reales de las predicciones.

Ejemplos reales: medicina, fraude, spam y seguridad

Precision, recall y F1-score se entienden mejor cuando los relacionamos con problemas concretos. Las fórmulas son importantes, pero la elección de una métrica depende siempre del contexto. No evaluamos un clasificador solo para obtener números, sino para decidir si sus errores son aceptables en una situación real.

Un primer ejemplo es el **diagnóstico asistido** o la detección de casos que requieren revisión médica. Supongamos que un modelo analiza información clínica y clasifica cada caso como “requiere revisión” o “no requiere revisión”. Si definimos como clase positiva “requiere revisión”, entonces un falso negativo sería un caso que realmente necesitaba revisión, pero que el modelo no marcó.

Clase positiva: requiere revisión

Falso negativo:

el caso requería revisión,
pero el modelo no lo detectó.

En este contexto, los falsos negativos pueden ser especialmente graves. Si un caso importante no se revisa a tiempo, las consecuencias pueden ser significativas. Por eso, muchas veces se prioriza recall: interesa detectar la mayor proporción posible de casos que realmente requieren atención.

Sin embargo, esto no significa que precision no importe. Si el modelo marca demasiados casos como “requiere revisión” y muchos de ellos no lo necesitaban, el sistema puede saturarse. Los profesionales podrían recibir demasiadas alertas, aumentarían los tiempos de revisión y se podrían generar estudios o derivaciones innecesarias. En un contexto de salud, más alertas no siempre significa mejor atención.

Por eso, en medicina, la evaluación debe pensarse con mucho cuidado. Un recall alto puede ser deseable cuando queremos evitar que casos importantes pasen desapercibidos, pero necesitamos controlar la precision para que el sistema sea viable y no produzca una carga excesiva. Además, cualquier modelo de este tipo debe entenderse como una herramienta de apoyo, no como reemplazo del criterio profesional.

Un segundo ejemplo es la **detección de fraude**. En este caso, la clase positiva podría ser “fraude” o “transacción sospechosa”. Un verdadero positivo sería un fraude correctamente detectado. Un falso negativo sería un fraude que el modelo dejó pasar como transacción normal. Un falso positivo sería una transacción legítima marcada como sospechosa.

Clase positiva: fraude

Falso negativo:
fraude real clasificado como normal.

Falso positivo:
transacción normal marcada como fraude.

Si el objetivo principal es evitar pérdidas o incidentes de seguridad, puede interesar aumentar recall para detectar la mayor cantidad posible de fraudes reales. Pero si el modelo genera demasiados falsos positivos, puede bloquear operaciones legítimas, molestar a los usuarios y saturar a los equipos de revisión.

En fraude, entonces, precision y recall suelen estar en tensión. Un sistema muy sensible puede detectar más fraudes, pero generar muchas alertas falsas. Un sistema muy estricto puede generar alertas más confiables, pero dejar pasar fraudes reales.

La decisión adecuada depende del costo de cada tipo de error, del volumen de operaciones, de la capacidad de revisión y de las consecuencias para los usuarios.

En algunos casos, no se usa el modelo para tomar una decisión automática inmediata, sino para ordenar transacciones por nivel de sospecha. Un equipo puede revisar primero las operaciones con mayor probabilidad estimada de fraude. Allí las probabilidades predichas y los umbrales de decisión se vuelven tan importantes como la clase final.

Un tercer ejemplo es el **filtro de spam**. Aquí la clase positiva suele ser “spam”. Un verdadero positivo es un correo no deseado correctamente enviado a la carpeta de spam. Un falso negativo es un correo spam que llega a la bandeja de entrada. Un falso positivo es un correo legítimo que el filtro envía por error a spam.

Clase positiva: spam

Falso positivo:
correo legítimo marcado como spam.

Falso negativo:
correo spam que llega a la bandeja de entrada.

En este problema, los falsos positivos pueden ser muy molestos. Si un correo importante se oculta en la carpeta de spam, el usuario puede perder una comunicación laboral, una factura, una confirmación de turno o un mensaje personal relevante. Por eso, en muchos filtros de spam es importante mantener una precisión alta: cuando el sistema marca algo como spam, conviene que realmente lo sea.

Pero recall también importa. Si el filtro deja pasar demasiado spam, la bandeja de entrada se llena de mensajes no deseados y el sistema pierde utilidad. La diferencia está en que, para muchos usuarios, puede ser más grave perder un correo legítimo que recibir algunos correos no deseados. En ese contexto, se suele preferir un equilibrio que evite falsos positivos excesivos, aunque eso implique tolerar algunos falsos negativos.

Este ejemplo muestra que la clase positiva no siempre es algo que queremos “aceptar”. En spam, fraude o enfermedad, la clase positiva representa algo problemático que queremos detectar. Por eso, los términos positivo y negativo son técnicos: no indican algo bueno o malo, sino la clase que estamos analizando.

Un cuarto ejemplo aparece en **seguridad informática**. Un modelo puede clasificar accesos, eventos de red, archivos, comportamientos de usuarios o alertas del sistema como “normal” o “sospechoso”. Si la clase positiva es “sospechoso”, un falso negativo

puede significar que una amenaza real no fue detectada. Un falso positivo puede significar que una actividad normal fue marcada como amenaza.

En seguridad informática, los falsos negativos pueden resultar muy costosos. Una intrusión, un malware o un acceso indebido que pasa inadvertido puede provocar daños importantes. Por eso, en muchos contextos de seguridad se valora un recall alto, ya que interesa detectar la mayor cantidad posible de amenazas reales.

Sin embargo, los falsos positivos también representan un problema serio. Un sistema con baja precisión genera demasiadas alertas falsas, lo que puede saturar a los equipos de seguridad y dificultar la identificación de los eventos verdaderamente importantes. Incluso puede producirse fatiga de alertas: cuando la mayoría de las advertencias no corresponde a incidentes reales, las personas pueden comenzar a ignorarlas.

El equilibrio es especialmente delicado. Un sistema que no detecta amenazas reales es peligroso, pero uno que alerta ante casi cualquier evento puede volverse impracticable. Por eso, su evaluación no debe limitarse a las métricas: también debe considerar la capacidad disponible para analizar alertas, la criticidad de los eventos y las consecuencias de cada tipo de error.

Estos ejemplos muestran que no hay una métrica universalmente correcta. En medicina y seguridad, muchas veces se presta especial atención al recall para no dejar pasar casos críticos. En spam, puede ser muy importante la precisión para no ocultar mensajes legítimos. En fraude, el equilibrio depende del costo de dejar pasar operaciones fraudulentas frente al costo de bloquear o revisar operaciones legítimas.

También muestran que el F1-score puede ser útil cuando necesitamos una medida de equilibrio, pero no reemplaza la interpretación. Dos modelos pueden tener F1-score parecido y comportamientos distintos. En un contexto puede ser mejor el que tiene mayor recall; en otro, el que tiene mayor precisión.

Por eso, al evaluar un clasificador, conviene volver siempre a las preguntas centrales:

- ¿Qué representa la clase positiva?
- ¿Qué costo tiene un falso positivo?
- ¿Qué costo tiene un falso negativo?
- ¿Qué capacidad tenemos para revisar alertas?
- ¿La predicción se usará automáticamente o como apoyo?

Las métricas no deciden por nosotros. Nos ayudan a expresar con números el comportamiento del modelo. La decisión sobre qué métrica priorizar debe surgir del problema real.

Precision, recall y F1-score adquieren sentido cuando los conectamos con casos concretos. En medicina, dejar pasar un caso importante puede ser más grave que generar revisiones adicionales. En fraude, necesitamos equilibrar detección y falsas alarmas. En spam, suele ser crítico no ocultar correos legítimos. En seguridad, el desafío es detectar amenazas sin saturar de alertas falsas. Evaluar bien un modelo significa entender qué errores produce y qué consecuencias tienen en el contexto donde será usado.

Elegir la métrica según el error que más importa

Precision, recall y F1-score nos permiten evaluar clasificadores con más detalle que la accuracy. Mientras la accuracy resume el porcentaje total de aciertos, estas métricas se concentran en la clase positiva y en los errores que más suelen preocupar en problemas reales: las falsas alarmas y los casos importantes no detectados.

Precision responde qué tan confiables son las predicciones positivas. Si el modelo marca un caso como positivo, precision nos ayuda a saber qué proporción de esas alertas realmente correspondía a casos positivos. Por eso, es especialmente importante cuando actuar sobre una predicción positiva tiene costo, genera molestias, requiere revisión o puede producir consecuencias delicadas.

Recall, en cambio, responde cuántos casos positivos reales logró detectar el modelo. Su foco está en no dejar pasar casos importantes. Es una métrica clave cuando los falsos negativos son costosos: fraudes que no se detectan, casos médicos que no se revisan, amenazas de seguridad que pasan desapercibidas, clientes importantes que abandonan sin ser identificados o fallas críticas que no se anticipan.

El F1-score combina precision y recall en una sola medida. Puede ser útil cuando necesitamos un equilibrio entre alertas confiables y buena detección de positivos reales. Sin embargo, no reemplaza el análisis de cada métrica por separado. Dos modelos con F1-score parecido pueden tener comportamientos distintos: uno puede ser más preciso pero detectar menos casos; otro puede detectar más positivos pero generar más falsas alarmas.

Por eso, la elección de la métrica no debe hacerse de manera automática. No se trata de buscar el número más alto, sino de decidir qué tipo de error importa más para el problema. Si los falsos positivos son especialmente costosos, conviene mirar con atención precision. Si los falsos negativos son más graves, recall será central. Si ambos tipos de error importan y queremos una medida de compromiso, F1-score puede ser una referencia útil.

Esta decisión siempre debe conectarse con el uso real del modelo. Un clasificador puede servir para tomar decisiones automáticas, para priorizar revisiones, para ordenar casos por riesgo o para apoyar a una persona que toma la decisión final. Cada uso cambia la forma en que interpretamos los errores. No es lo mismo una alerta que solo invita a revisar que una predicción que bloquea una operación, rechaza una solicitud o deja sin atención un caso importante.

También debemos considerar la capacidad operativa. Un modelo con recall muy alto puede generar tantas alertas que un equipo no pueda revisarlas. Un modelo con precision muy alta puede generar pocas alertas confiables, pero dejar pasar demasiados casos relevantes. Las métricas deben interpretarse junto con los recursos disponibles y con las consecuencias de actuar o no actuar.

Los ejemplos reales muestran esta variedad. En medicina, muchas veces preocupa no dejar pasar casos que requieren revisión, por lo que recall puede ser prioritario. En spam, puede ser especialmente grave ocultar correos legítimos, por lo que precision gana importancia. En fraude y seguridad, el equilibrio puede ser delicado: queremos detectar amenazas o fraudes reales, pero sin saturar el sistema con falsas alarmas.

Precision, recall y F1-score no son métricas rivales, sino herramientas complementarias. Cada una responde una pregunta distinta sobre el clasificador. Evaluar bien implica comprender esas preguntas, relacionarlas con la matriz de confusión y elegir la métrica principal según el error que más importa. Un buen modelo no es simplemente el que obtiene el valor más alto en una tabla, sino el que comete errores aceptables para el contexto en el que será utilizado.

Capítulo 13 · Probabilidades, umbrales, ROC y AUC

Predicción de clases vs. predicción de probabilidades

Cuando utilizamos un modelo de clasificación, solemos pensar en su salida como una etiqueta final. El modelo recibe las variables de entrada de un caso y lo clasifica, por ejemplo, como “abandona”, “permanece”, “fraude”, “normal”, “requiere revisión” o “no requiere revisión”. Esta es la forma más directa de interpretar una clasificación.

Sin embargo, muchos clasificadores también pueden estimar la probabilidad asociada a cada clase. En lugar de indicar solamente que un cliente abandonará, el modelo puede estimar una probabilidad de abandono de 0,73. Del mismo modo, puede determinar que una transacción tiene una probabilidad de fraude de 0,18 o que un caso presenta una probabilidad de 0,91 de requerir revisión.

En los problemas de clasificación binaria, normalmente nos concentramos en la probabilidad de la clase positiva. La etiqueta final se obtiene al comparar esa probabilidad con un umbral de decisión. Si el umbral es 0,5, los casos con una probabilidad igual o superior a ese valor se clasifican como positivos, mientras que los restantes se consideran negativos. Por ejemplo, una probabilidad de abandono de 0,73 conduce a predecir que el cliente abandonará, mientras que una probabilidad de 0,31 produce la predicción contraria.

Esto permite entender la clasificación como un proceso compuesto por varias etapas. El modelo recibe las variables de entrada y estima una probabilidad. Luego, esa probabilidad se compara con el umbral de decisión para obtener la clase predicha.

La clase final, sin embargo, puede ocultar información importante. Dos clientes con probabilidades de abandono de 0,92 y 0,53 serían clasificados como “abandona” si el umbral fuera 0,5, aunque el primer resultado parece mucho más claro que el segundo. De manera similar, dos clientes con probabilidades de 0,48 y 0,05 recibirían la etiqueta “permanece”, pero el primero se encuentra muy cerca del límite entre ambas clases, mientras que el segundo muestra una señal de abandono mucho menor.

Por eso, en muchos proyectos conviene analizar las probabilidades y no solamente las etiquetas finales. Estas permiten ordenar casos, establecer prioridades, ajustar el umbral de decisión y comprender con mayor detalle el comportamiento del modelo.

En una campaña de retención, por ejemplo, quizá no sea posible contactar a todos los clientes clasificados como propensos a abandonar. Si el equipo comercial solo puede

comunicarse con 500 personas, puede priorizar aquellas que tengan las probabilidades estimadas más altas.

En la detección de fraude también puede resultar útil graduar la respuesta. Una transacción con una probabilidad de fraude de 0,95 podría requerir una acción inmediata. Otra con una probabilidad de 0,12 quizá no supere el umbral general, pero podría revisarse si involucra un monto elevado o aparece acompañada por otras señales sospechosas.

En un sistema de diagnóstico asistido, las probabilidades pueden utilizarse para ordenar los casos según su prioridad de revisión. El modelo no debe reemplazar el criterio profesional, pero puede ayudar a destacar aquellos en los que estima un riesgo mayor. En todos estos ejemplos, la probabilidad aporta información que una clase final aislada no permite observar.

En `scikit-learn`, esta diferencia suele aparecer en dos métodos distintos. `predict` devuelve la clase predicha. `predict_proba` devuelve las probabilidades estimadas para cada clase.

```
modelo.predict(X_test)
```

```
modelo.predict_proba(X_test)
```

Si usamos clasificación binaria, `predict_proba` suele devolver dos columnas: una para la probabilidad de la clase negativa y otra para la probabilidad de la clase positiva. Por ejemplo, podría devolver 0.27 para $P(\text{permanece})$ y 0.73 para $P(\text{abandona})$.

Como ambas clases son complementarias en un problema binario, sus probabilidades suman 1. Si la probabilidad de abandono es 0.73, la probabilidad de permanencia será 0.27.

En clasificación multiclase, el modelo puede devolver una probabilidad para cada clase. Por ejemplo, en un problema de riesgo, podría estimar una probabilidad de 0.15 para “riesgo bajo”, 0.30 para “riesgo medio” y 0.55 para “riesgo alto”.

La clase final suele ser la de mayor probabilidad estimada. En este caso, el modelo predice “riesgo alto”. Pero las otras probabilidades también aportan información. Si “riesgo medio” tiene 0.30, el caso no parece completamente alejado de esa categoría. Esta información puede ser útil para interpretación o revisión.

De todos modos, debemos ser prudentes al interpretar probabilidades. Una probabilidad predicha no es una certeza absoluta. Es una estimación producida por

un modelo entrenado con ciertos datos. Si los datos son incompletos, están desactualizados, contienen sesgos o no representan bien la situación actual, las probabilidades también pueden ser poco confiables.

Además, no todos los modelos producen probabilidades igualmente bien calibradas. Que un modelo devuelva 0.80 no garantiza automáticamente que, de cada 100 casos con esa probabilidad, exactamente 80 correspondan a la clase positiva. La calibración de probabilidades es un tema importante, pero más avanzado. Por ahora, alcanza con recordar que las probabilidades son estimaciones útiles, no verdades perfectas.

Esta advertencia es especialmente relevante cuando las probabilidades se usan para tomar decisiones sensibles. En riesgo crediticio, salud, seguridad o educación, una probabilidad no debería convertirse automáticamente en una decisión definitiva sin considerar contexto, revisión, consecuencias y criterios éticos. El modelo aporta una señal, no una respuesta incuestionable.

La diferencia entre clase y probabilidad también nos prepara para estudiar las curvas ROC y AUC. Estas herramientas no se basan solamente en una clase predicha con un umbral fijo. Analizan cómo cambia el comportamiento del modelo cuando modificamos el umbral de clasificación. Para eso, necesitamos mirar las probabilidades o puntuaciones que el modelo asigna a los casos.

Un clasificador puede devolver una clase final o una probabilidad estimada. La clase responde qué etiqueta asigna el modelo. La probabilidad muestra qué tan fuerte parece la señal hacia una clase. La clase final depende de un umbral de decisión, mientras que la probabilidad permite analizar el modelo con más matices, ordenar casos, ajustar decisiones y estudiar cómo cambia el desempeño al modificar el umbral.

Umbral de clasificación

Cuando un modelo devuelve probabilidades, todavía necesitamos decidir cómo convertir esas probabilidades en clases finales. Esa conversión se realiza mediante un **umbral de clasificación**.

En clasificación binaria, el umbral es el valor mínimo que debe alcanzar la probabilidad de la clase positiva para que el modelo prediga esa clase. Si la probabilidad estimada supera o iguala el umbral, el caso se clasifica como positivo. Si queda por debajo, se clasifica como negativo.

La regla general es:

Si $P(\text{clase positiva}) \geq \text{umbral} \rightarrow \text{predice clase positiva.}$
Si $P(\text{clase positiva}) < \text{umbral} \rightarrow \text{predice clase negativa.}$

El umbral más habitual para comenzar es 0.5. Por ejemplo, si estamos prediciendo abandono de clientes y la clase positiva es “abandona”, podemos usar esta regla:

Si $P(\text{abandona}) \geq 0.5 \rightarrow \text{predice abandona.}$
Si $P(\text{abandona}) < 0.5 \rightarrow \text{predice permanece.}$

Supongamos tres clientes:

Cliente A $\rightarrow P(\text{abandona}) = 0.82$
Cliente B $\rightarrow P(\text{abandona}) = 0.57$
Cliente C $\rightarrow P(\text{abandona}) = 0.21$

Con un umbral de 0.5, los clientes A y B serán clasificados como “abandona”, mientras que el cliente C será clasificado como “permanece”.

Cliente A $\rightarrow 0.82 \geq 0.5 \rightarrow \text{abandona}$
Cliente B $\rightarrow 0.57 \geq 0.5 \rightarrow \text{abandona}$
Cliente C $\rightarrow 0.21 < 0.5 \rightarrow \text{permanece}$

Esta regla parece natural, pero no debemos olvidar que el umbral de 0.5 es solo un punto de partida. No siempre es el más adecuado. La elección del umbral depende del problema, del costo de los errores y del uso que se dará a las predicciones.

Cambiar el umbral modifica las clases predichas. No cambia las probabilidades estimadas por el modelo, pero sí cambia la decisión que tomamos a partir de ellas. Este detalle es muy importante: el mismo modelo puede producir comportamientos distintos según el umbral elegido.

Si bajamos el umbral, más casos serán clasificados como positivos. Por ejemplo, si usamos un umbral de 0.3 en lugar de 0.5, cualquier cliente con probabilidad de abandono mayor o igual que 0.3 será marcado como “abandona”.

Umbral = 0.3

Cliente A $\rightarrow P(\text{abandona}) = 0.82 \rightarrow \text{abandona}$
Cliente B $\rightarrow P(\text{abandona}) = 0.57 \rightarrow \text{abandona}$
Cliente C $\rightarrow P(\text{abandona}) = 0.21 \rightarrow \text{permanece}$
Cliente D $\rightarrow P(\text{abandona}) = 0.35 \rightarrow \text{abandona}$

Con un umbral más bajo, el modelo se vuelve más sensible. Detecta más posibles positivos, pero también puede generar más falsas alarmas. En términos de la matriz

de confusión, bajar el umbral suele reducir falsos negativos, pero puede aumentar falsos positivos.

Umbral más bajo

- más casos positivos predichos
- más verdaderos positivos posibles
- menos falsos negativos posibles
- más falsos positivos posibles

Si subimos el umbral, ocurre lo contrario. El modelo se vuelve más exigente para clasificar un caso como positivo. Por ejemplo, con un umbral de 0.8, solo los casos con probabilidad estimada muy alta serán marcados como positivos.

Umbral = 0.8

- Cliente A → $P(\text{abandona}) = 0.82$ → abandona
- Cliente B → $P(\text{abandona}) = 0.57$ → permanece
- Cliente C → $P(\text{abandona}) = 0.21$ → permanece
- Cliente D → $P(\text{abandona}) = 0.35$ → permanece

Con un umbral más alto, el modelo genera menos alertas positivas. Esto puede reducir falsas alarmas y aumentar la precisión, pero también puede dejar pasar casos positivos reales. En términos de la matriz de confusión, subir el umbral suele reducir falsos positivos, pero puede aumentar falsos negativos.

Umbral más alto

- menos casos positivos predichos
- menos falsos positivos posibles
- más falsos negativos posibles

Esta relación conecta directamente con precisión y recall. Cuando bajamos el umbral, suele aumentar el recall, porque detectamos más positivos reales. Pero la precisión puede bajar, porque también aumentan las falsas alarmas. Cuando subimos el umbral, puede aumentar la precisión, porque las alertas son más selectivas, pero puede bajar el recall, porque dejamos pasar más positivos reales.

Por eso, el umbral expresa una decisión sobre qué errores estamos dispuestos a tolerar. No es un detalle técnico menor. Es una parte central del uso del modelo.

En un sistema de diagnóstico asistido, si la clase positiva es “requiere revisión”, puede ser razonable usar un umbral más bajo para no dejar pasar casos importantes. Esto generará más alertas, pero puede reducir la cantidad de falsos negativos. La decisión debe tomarse con criterios profesionales y considerando la capacidad de revisión.

En un filtro de spam, en cambio, tal vez convenga ser más exigente antes de enviar un correo a la carpeta de spam. Un falso positivo puede ocultar un mensaje legítimo importante. Por eso, podría preferirse un umbral más alto para marcar algo como spam, aceptando que algunos correos no deseados lleguen a la bandeja de entrada.

En fraude, la elección del umbral dependerá del equilibrio entre detectar operaciones sospechosas y evitar bloquear transacciones legítimas. Un umbral bajo puede detectar más fraudes, pero generar más revisiones. Un umbral alto puede reducir molestias, pero dejar pasar operaciones peligrosas.

Esto muestra que no existe un umbral universalmente correcto. El valor adecuado depende del problema real. No elegimos el umbral porque sí. Lo elegimos según el costo de los errores y la forma en que se usarán las predicciones.

También es posible que el modelo no se use para tomar una decisión binaria inmediata. A veces no necesitamos decir “positivo” o “negativo” para todos los casos. Podemos ordenar los casos según su probabilidad y actuar sobre los primeros.

Por ejemplo, una empresa puede decidir contactar a los 1000 clientes con mayor probabilidad de abandono, sin fijar inicialmente un umbral como 0.5. En la práctica, el umbral queda determinado por la capacidad operativa: se atienden los casos que están por encima del punto de corte necesario para formar ese grupo de 1000.

Esta forma de trabajar es muy común cuando los recursos son limitados. En lugar de buscar una clasificación absoluta, usamos las probabilidades para priorizar.

En `scikit-learn`, si usamos `predict`, el modelo devuelve clases finales según su regla interna de decisión. Para muchos clasificadores binarios, esa regla se corresponde con un umbral de 0.5 sobre las probabilidades, aunque siempre conviene revisar el comportamiento del modelo específico. Si queremos elegir nuestro propio umbral, necesitamos trabajar con `predict_proba`.

```
probabilidades = modelo.predict_proba(X_test)[: , 1]
```

En este ejemplo, `[ : , 1]` suele seleccionar la probabilidad de la clase positiva, si las clases están ordenadas de la forma esperada. Luego podemos crear nuestras propias predicciones:

```
umbral = 0.3  
y_pred_umbral = (probabilidades >= umbral)
```

En un cuaderno práctico, esta idea será muy útil para observar cómo cambian las predicciones al modificar el umbral. Podemos comparar la matriz de confusión, precisión y recall para distintos valores.

Por ahora, lo más importante es la idea conceptual: el umbral transforma probabilidades en clases. Al moverlo, cambiamos el equilibrio entre falsos positivos y falsos negativos.

También debemos recordar que ajustar el umbral no arregla todos los problemas. Si el modelo asigna probabilidades poco útiles, si las variables no contienen información relevante o si hay fuga de datos, cambiar el umbral no convierte automáticamente al modelo en bueno. Solo modifica cómo usamos sus salidas.

Un buen umbral debe elegirse después de evaluar el modelo. No alcanza con probar muchos valores hasta encontrar uno que parezca conveniente en un único conjunto de prueba. Más adelante, cuando estudiemos validación cruzada y búsqueda de hiperparámetros, veremos formas más cuidadosas de comparar decisiones. En esta etapa, basta con entender que el umbral debe analizarse con criterio y no aceptarse sin reflexión.

El umbral de clasificación es el punto de corte que convierte una probabilidad estimada en una clase final. Un umbral bajo tiende a detectar más casos positivos, pero puede generar más falsas alarmas. Un umbral alto produce predicciones positivas más exigentes, pero puede dejar pasar más casos reales. Elegir un umbral implica decidir qué tipo de error es más importante controlar en el contexto del problema.

Sensibilidad y especificidad

Cuando modificamos el umbral de clasificación, cambia la forma en que el modelo reparte los casos entre positivos y negativos. Para entender ese cambio con más precisión, conviene estudiar dos métricas muy relacionadas con la matriz de confusión: **sensibilidad** y **especificidad**.

La **sensibilidad** es otro nombre para el recall. Mide qué proporción de los casos positivos reales logró detectar el modelo. Ya vimos esta idea en el capítulo anterior, pero ahora la retomamos porque es una de las piezas centrales para comprender la curva ROC.

La pregunta que responde la sensibilidad es: De todos los casos realmente positivos, ¿cuántos fueron clasificados como positivos?

Su fórmula es:

$$\text{sensibilidad} = \text{VP} / (\text{VP} + \text{FN})$$

VP

verdaderos positivos

FN

falsos negativos

Fórmula de cálculo.

Donde **VP** son los verdaderos positivos y **FN** los falsos negativos. Si la sensibilidad es alta, el modelo deja pasar pocos casos positivos. Si es baja, muchos positivos reales quedan sin detectar.

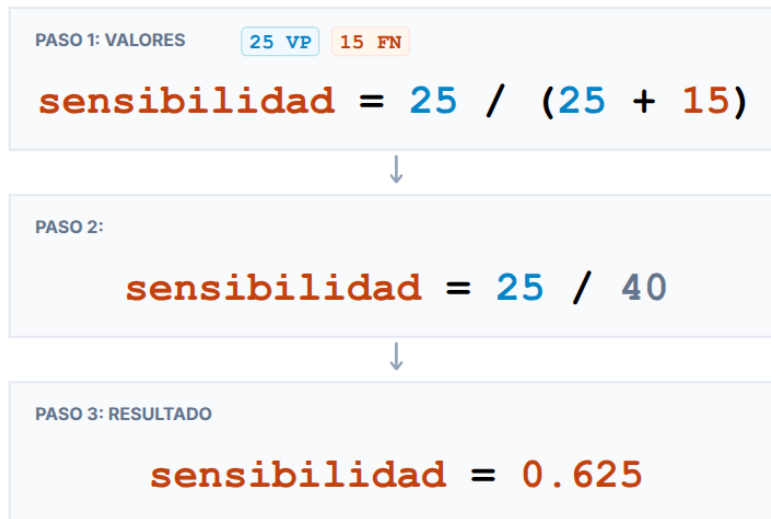
Por ejemplo, en un problema de detección de fraude, una sensibilidad alta significa que el modelo detecta una gran parte de las transacciones fraudulentas. En diagnóstico asistido, una sensibilidad alta significa que el modelo marca para revisión una gran parte de los casos que realmente lo requerían. En abandono de clientes, significa que detecta muchos de los clientes que efectivamente abandonarían.

Supongamos esta matriz de confusión para abandono de clientes:

		Predicción	
		permanece	abandona
Real:	permanece	50	10
	abandona	15	25

Matriz de confusión para abandono de clientes.

Como la clase positiva es “abandona”, tenemos:



El modelo detectó el 62,5% de los abandonos reales.

La **especificidad**, en cambio, mira la clase negativa. Mide qué proporción de los casos negativos reales fueron correctamente clasificados como negativos.

La pregunta que responde la especificidad es: De todos los casos realmente negativos, ¿cuántos fueron clasificados como negativos?

Su fórmula es:

$$\text{especificidad} = \text{VN} / (\text{VN} + \text{FP})$$

VN verdaderos negativos	FP falsos positivos
-----------------------------------	-------------------------------

Fórmula de cálculo.

Donde **VN** son los verdaderos negativos y **FP** los falsos positivos. Si la especificidad es alta, el modelo descarta correctamente muchos negativos. Si es baja, marca como positivos demasiados casos que en realidad eran negativos.

En el mismo ejemplo de abandono, la clase negativa es “permanece”. En la matriz:



Fórmula de cálculo.

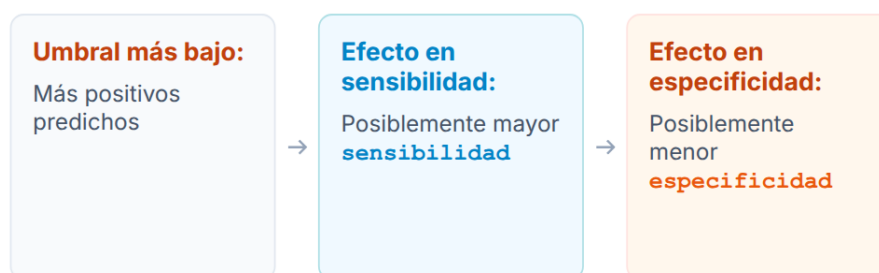
Esto significa que el modelo clasificó correctamente como “permanece” aproximadamente el 83% de los clientes que realmente permanecieron.

Sensibilidad y especificidad miran lados distintos del problema. La sensibilidad se concentra en los positivos reales. La especificidad se concentra en los negativos reales:

- **Sensibilidad:** qué tan bien detectamos los positivos.
- **Especificidad:** qué tan bien descartamos los negativos.

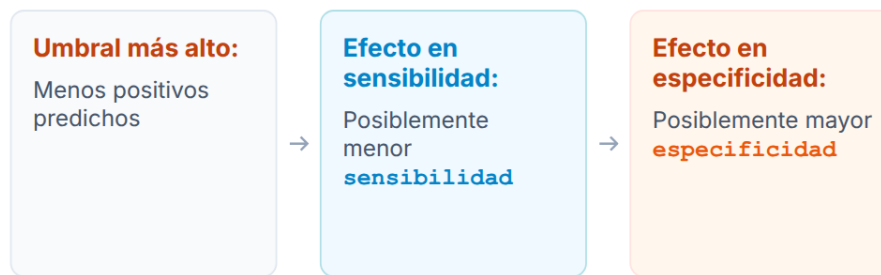
Esta diferencia es importante porque un modelo puede tener alta sensibilidad y baja especificidad, o alta especificidad y baja sensibilidad. No son lo mismo.

Si bajamos el umbral de clasificación, el modelo predice más casos como positivos. Eso suele aumentar la sensibilidad, porque detecta más positivos reales. Pero también puede reducir la especificidad, porque más negativos reales serán marcados por error como positivos.



Bajo umbral de clasificación.

Si subimos el umbral, el modelo predice menos casos como positivos. Eso puede aumentar la especificidad, porque descarta mejor los negativos. Pero también puede reducir la sensibilidad, porque algunos positivos reales quedarán sin detectar.



Alto umbral de clasificación.

Este equilibrio aparece en muchos problemas reales. En diagnóstico asistido, si queremos detectar la mayor cantidad posible de casos que requieren revisión, podemos priorizar sensibilidad. Pero si el modelo marca demasiados casos que no lo requerían, la especificidad será baja y el sistema puede generar una sobrecarga de revisiones.

En filtros de spam, si queremos evitar que correos legítimos terminen en spam, nos interesa una buena especificidad para la clase “no spam”. Si el sistema marca como spam muchos correos legítimos, está fallando al reconocer negativos reales. Pero si somos demasiado estrictos, puede bajar la sensibilidad y dejar pasar demasiado spam.

En fraude, una sensibilidad alta permite detectar más operaciones fraudulentas. Una especificidad alta permite no molestar o bloquear innecesariamente demasiadas operaciones legítimas. El valor adecuado depende del costo de cada tipo de error.

La especificidad también se relaciona con los falsos positivos. Si el modelo genera muchos falsos positivos, la especificidad baja. Por eso, a veces se usa una medida complementaria llamada **tasa de falsos positivos**.

La tasa de falsos positivos responde: De todos los casos realmente negativos, ¿cuántos fueron clasificados por error como positivos?

Su fórmula es:

$$\text{tasa de falsos positivos} = \frac{FP}{FP+VN}$$

FP
falsos positivos

VN
verdaderos negativos

Tasa de falsos positivos.

Esta tasa es el complemento de la especificidad:

$$\text{tasa de falsos positivos} \\ = 1 - \text{especificidad}$$

Tasa de falsos positivos.

En el ejemplo anterior, la especificidad era aproximadamente 0.83. Por lo tanto, la tasa de falsos positivos es aproximadamente 0.17. Esto significa que alrededor del 17% de los clientes que realmente permanecieron fueron marcados erróneamente como abandono.

Esta idea será clave para entender la curva ROC. La curva ROC compara, para distintos umbrales, la sensibilidad con la tasa de falsos positivos. Es decir, analiza cuánto aumenta la detección de positivos y cuánto cuesta eso en términos de falsas alarmas sobre los negativos.

Por ahora, lo importante es reconocer las dos fuerzas principales:

- **Sensibilidad alta:** detectamos muchos positivos reales.
- **Especificidad alta:** descartamos correctamente muchos negativos reales.

En algunos problemas queremos priorizar sensibilidad. En otros, especificidad. Muchas veces necesitamos un equilibrio entre ambas.

No debemos interpretar estas métricas como valores abstractos separados del problema. Una sensibilidad de 0.90 puede ser excelente en un contexto y todavía insuficiente en otro. Una especificidad de 0.80 puede ser aceptable si las falsas alarmas son baratas, pero problemática si cada falso positivo produce consecuencias graves.

También debemos recordar que estas métricas dependen del umbral. No son propiedades fijas del modelo en todo sentido. Si cambiamos la regla para convertir probabilidades en clases, cambian los verdaderos positivos, falsos positivos, verdaderos negativos y falsos negativos. Por lo tanto, cambian sensibilidad y especificidad.

La sensibilidad mide la capacidad del modelo para detectar casos positivos reales. La especificidad mide la capacidad para descartar correctamente casos negativos reales. Al modificar el umbral de clasificación, suele aparecer un equilibrio entre ambas: aumentar la sensibilidad puede reducir la especificidad, y aumentar la especificidad puede reducir la sensibilidad. Esta relación será la base para comprender la curva ROC.

Curva ROC

La curva ROC es una herramienta para analizar cómo se comporta un clasificador binario cuando modificamos el umbral de decisión. En lugar de evaluar el modelo con un único umbral, como 0.5, la curva ROC observa qué ocurre con muchos umbrales posibles.

Esto es importante porque, como vimos, el umbral cambia las clases predichas. Si bajamos el umbral, el modelo detecta más positivos, pero también puede generar más falsos positivos. Si subimos el umbral, el modelo genera menos falsas alarmas, pero puede dejar pasar más positivos reales.

La curva ROC permite visualizar ese equilibrio.

ROC significa **Receiver Operating Characteristic**. El nombre proviene de otro contexto histórico, pero en Machine Learning se usa para estudiar la relación entre dos cantidades: la sensibilidad o tasa de verdaderos positivos frente a la tasa de falsos positivos.

La sensibilidad, también llamada recall, mide qué proporción de positivos reales detecta el modelo:

$$\text{sensibilidad} = \text{VP} / (\text{VP} + \text{FN})$$

VP

verdaderos positivos

FN

falsos negativos

Fórmula de cálculo.

La tasa de falsos positivos mide qué proporción de negativos reales fueron clasificados incorrectamente como positivos:

$$\text{tasa de falsos positivos} = \text{FP} / (\text{FP} + \text{VN})$$

FP

falsos positivos

VN

verdaderos negativos

Tasa de falsos positivos.

La curva ROC coloca estas dos medidas en un gráfico. En el eje vertical se ubica la sensibilidad. En el eje horizontal se ubica la tasa de falsos positivos.

Cada punto de la curva representa el comportamiento del modelo para un umbral determinado. Si elegimos un umbral muy alto, el modelo será muy exigente para predecir positivo. Tendrá pocos positivos predichos. En ese caso, puede tener pocos falsos positivos, pero también pocos verdaderos positivos.

Si elegimos un umbral más bajo, el modelo clasificará más casos como positivos. Aumentará la cantidad de positivos reales detectados, pero también puede aumentar la cantidad de negativos marcados por error como positivos.

La curva ROC muestra justamente cómo cambia esa relación a medida que el umbral se mueve.

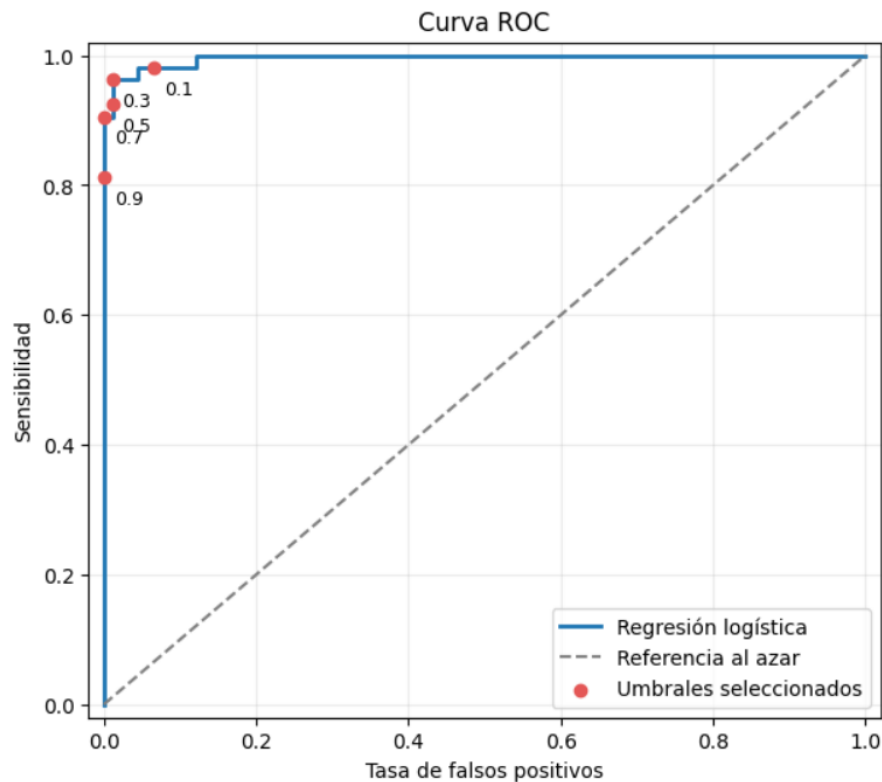
Podemos imaginarlo con un modelo que predice la probabilidad de abandono de clientes. Para cada cliente, el modelo calcula una probabilidad estimada de abandono. Luego podríamos probar distintos umbrales:

- umbral = 0.90
- umbral = 0.70
- umbral = 0.50
- umbral = 0.30
- umbral = 0.10

Con cada umbral obtenemos una matriz de confusión distinta. A partir de cada matriz calculamos sensibilidad y tasa de falsos positivos. Cada par de valores genera un punto en la curva ROC.

Por ejemplo:

- **Umbral 0.90:** baja sensibilidad, baja tasa de falsos positivos.
- **Umbral 0.50:** sensibilidad intermedia, tasa de falsos positivos intermedia.
- **Umbral 0.10:** alta sensibilidad, alta tasa de falsos positivos.



Curva ROC.

La curva ROC no muestra directamente precision, recall y F1-score. Muestra sensibilidad y tasa de falsos positivos. Como sensibilidad es lo mismo que recall, una parte de lo que ya estudiamos aparece en la curva. Pero el otro eje no es precision, sino la proporción de negativos reales que el modelo marca por error como positivos.

Esto tiene una consecuencia importante: la curva ROC se concentra en la capacidad del modelo para separar positivos y negativos a lo largo de distintos umbrales. No se limita a una decisión puntual.

Un modelo ideal tendría una curva que se acerca mucho a la esquina superior izquierda del gráfico. Esa zona representa alta sensibilidad y baja tasa de falsos positivos. Es decir, detecta muchos positivos reales sin marcar demasiados negativos como positivos.

Buen comportamiento: alta sensibilidad y baja tasa de falsos positivos

En cambio, un modelo que no logra distinguir entre positivos y negativos tendrá una curva cercana a una diagonal. Esa diagonal representa un comportamiento parecido al azar. Si el modelo no ordena los casos mejor que una selección aleatoria, no logra una separación útil.

No hace falta interpretar la curva ROC como un dibujo exacto en esta etapa, pero sí conviene entender su lectura general:

- Cuanto más cerca esté la curva de la esquina superior izquierda, mejor separa el modelo las clases.
- Cuanto más cerca esté de la diagonal, menos capacidad tiene para distinguir positivos y negativos.

La curva ROC es especialmente útil cuando queremos comparar modelos sin quedarnos atados a un único umbral. Dos modelos pueden tener accuracy parecida usando umbral 0.5, pero comportarse de manera distinta si modificamos ese umbral. La curva ROC permite ver esa diferencia.

Supongamos dos modelos para detectar fraude. Con umbral 0.5, ambos obtienen una accuracy similar. Pero al mirar sus probabilidades, uno de ellos ordena mejor los casos: suele asignar probabilidades más altas a los fraudes reales y más bajas a las operaciones normales. Ese modelo tendrá una curva ROC mejor, porque permite elegir umbrales con buena sensibilidad y baja tasa de falsos positivos.

Esta idea de “ordenar bien” es central. La curva ROC no pregunta solamente si una clase final fue correcta con un umbral determinado. Pregunta si el modelo tiende a dar puntuaciones más altas a los positivos reales que a los negativos reales. Si lo hace, podremos encontrar umbrales útiles. Si no lo hace, cambiar el umbral no ayudará demasiado.

Por eso, la curva ROC trabaja con probabilidades o puntuaciones, no solo con clases finales. En `scikit-learn`, normalmente usamos las probabilidades de la clase positiva:

```
probabilidades = modelo.predict_proba(X_test)[: , 1]
```

Luego, con esas probabilidades y las etiquetas reales, podemos calcular los puntos de la curva ROC:

```
from sklearn.metrics import roc_curve

fpr, tpr, thresholds = roc_curve(y_test, probabilidades)
```

En este código, `fpr` (false positive rate) representa la tasa de falsos positivos y `tpr` (true positive rate) representa la tasa de verdaderos positivos, es decir, la sensibilidad. `thresholds` (umbrales) contiene los umbrales evaluados.

En un cuaderno práctico, estos valores pueden graficarse para visualizar la curva. El eje horizontal será `fpr` y el eje vertical será `tpr`.

La curva ROC también ayuda a comprender que no existe un único comportamiento del modelo. Un mismo modelo puede ser más conservador o más sensible según el umbral que elijamos. La curva muestra el abanico de comportamientos posibles.

Sin embargo, la curva ROC no decide automáticamente qué umbral debemos usar. Puede mostrar que un modelo tiene buena capacidad para separar clases, pero la elección del umbral sigue dependiendo del problema. Si queremos reducir falsos negativos, elegiremos una zona con mayor sensibilidad. Si queremos reducir falsas alarmas, necesitaremos controlar la tasa de falsos positivos.

La curva ROC ayuda a comparar y entender el modelo. La elección del umbral depende del contexto.

También debemos tener cuidado cuando las clases están muy desbalanceadas. En esos casos, la curva ROC puede verse razonablemente buena aunque el desempeño práctico sobre la clase minoritaria no sea tan útil. Por eso, más adelante mencionaremos brevemente la curva Precision-Recall, que puede ser más informativa en ciertos problemas con clases raras o muy desbalanceadas.

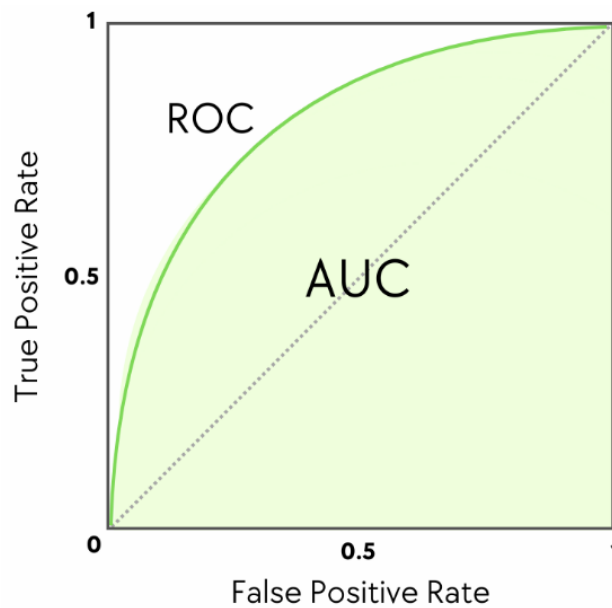
La curva ROC muestra cómo cambia la relación entre sensibilidad y tasa de falsos positivos cuando variamos el umbral de clasificación. Cada punto de la curva corresponde a un umbral distinto. Una curva cercana a la esquina superior izquierda indica mejor capacidad para separar positivos y negativos. Una curva cercana a la diagonal indica un comportamiento parecido al azar. La curva ROC no reemplaza la matriz de confusión ni la elección contextual del umbral, pero permite analizar el modelo más allá de una única decisión fija.

AUC

La curva ROC nos permite ver cómo cambia el comportamiento de un clasificador cuando modificamos el umbral. Pero a veces queremos resumir esa curva en un solo valor para comparar modelos de manera más rápida. Para eso se utiliza **AUC**.

AUC significa **Area Under the Curve**, es decir, área bajo la curva. En este contexto, se refiere al área bajo la curva ROC. Por eso muchas veces hablamos de **ROC-AUC**.

El valor de AUC suele estar entre 0 y 1. Cuanto más alto sea, mejor capacidad tiene el modelo para separar casos positivos y negativos.



Curva ROC y AUC.

Un valor cercano a 1 indica que el modelo tiende a asignar puntuaciones más altas a los casos positivos reales que a los negativos reales. Un valor cercano a 0.5 indica un comportamiento parecido al azar. En la práctica, un modelo con AUC de 0.5 no está separando las clases mejor que una selección aleatoria.

- **AUC $\approx$ 1.0:** muy buena separación entre clases.
- **AUC $\approx$ 0.5:** separación parecida al azar.

Para entender la idea, imaginemos un modelo que predice la probabilidad de abandono de clientes. Si tomamos al azar un cliente que realmente abandonó y un cliente que realmente permaneció, un modelo con buen AUC tenderá a asignar mayor probabilidad de abandono al cliente que realmente abandonó.

Esta interpretación es muy útil: AUC mide qué tan bien el modelo ordena los casos, asignando puntuaciones más altas a los positivos reales que a los negativos reales.

No se trata únicamente de acertar una clase con un umbral fijo. AUC evalúa la capacidad de separación del modelo a través de todos los umbrales posibles. Por eso puede ser útil cuando todavía no decidimos qué umbral usaremos o cuando queremos comparar modelos antes de fijar una regla de decisión.

Supongamos dos modelos para detectar fraude. Ambos pueden tener una accuracy parecida con umbral 0.5. Pero uno de ellos asigna puntuaciones más altas a la mayoría de los fraudes reales y puntuaciones más bajas a la mayoría de las transacciones normales. Ese modelo tendrá un AUC mayor, porque ordena mejor los casos.

Esta idea de ordenamiento es central. Un modelo puede no usar todavía el umbral perfecto, pero si ordena bien los casos, tenemos margen para elegir un umbral útil. En cambio, si las puntuaciones de positivos y negativos están muy mezcladas, cualquier umbral tendrá dificultades: al intentar detectar más positivos, también aparecerán muchos falsos positivos; al intentar reducir falsas alarmas, se dejarán pasar muchos positivos reales.

AUC resume esa capacidad general de separación.

En `scikit-learn`, podemos calcular ROC-AUC con `roc_auc_score`. En clasificación binaria, normalmente usamos las probabilidades de la clase positiva:

```
from sklearn.metrics import roc_auc_score

probabilidades = modelo.predict_proba(X_test)[: , 1]
roc_auc_score(y_test, probabilidades)
```

Es importante notar que aquí no usamos `y_pred`, sino las probabilidades o puntuaciones del modelo. Esto se debe a que AUC no evalúa solamente las clases finales para un umbral determinado. Evalúa cómo se ordenan los casos a lo largo de muchos umbrales posibles.

Para accuracy, precision, recall y F1-score usamos clases predichas. Para ROC-AUC usamos probabilidades o puntuaciones.

Esta diferencia suele ser una fuente de confusión al comenzar. Si usamos solo `predict`, ya perdimos parte de la información necesaria para construir la curva ROC. Necesitamos las puntuaciones que permiten mover el umbral.

AUC puede servir para comparar modelos. Si entrenamos una regresión logística, un árbol de decisión y otro clasificador, podemos calcular el AUC de cada uno sobre el conjunto de prueba. El modelo con mayor AUC será, en principio, el que mejor separa las clases según sus puntuaciones.

Pero esa comparación debe hacerse con cuidado. Un AUC más alto no significa automáticamente que el modelo sea mejor para cualquier uso. Tal vez un modelo tenga muy buen AUC general, pero no funcione tan bien en la zona de umbrales que realmente nos interesa. Por ejemplo, en un sistema donde necesitamos muy pocos falsos positivos, nos importa especialmente el comportamiento del modelo en esa región, no solo el área total bajo la curva.

También puede ocurrir que un modelo con AUC menor sea preferible porque es más simple, más interpretable, más estable o más adecuado para el contexto. Las métricas ayudan a decidir, pero no reemplazan el análisis completo.

Una ventaja de AUC es que no depende de un umbral único. Esto permite evaluar la capacidad general de separación del modelo antes de elegir el punto de corte. Es especialmente útil en etapas de comparación inicial, cuando queremos saber si un modelo tiene señal predictiva.

Otra ventaja es que ayuda a detectar modelos que, aunque tengan una accuracy razonable, no ordenan bien los casos. Esto puede ocurrir cuando las clases están desbalanceadas o cuando el umbral elegido produce una impresión incompleta del desempeño.

Sin embargo, AUC también puede ser malinterpretado. Un AUC alto no significa que las probabilidades estén perfectamente calibradas. Un modelo puede ordenar bien los casos, pero asignar probabilidades que no correspondan exactamente con frecuencias reales. Por ejemplo, puede ubicar correctamente los casos de mayor riesgo por encima de los de menor riesgo, pero sobreestimar o subestimar los valores de probabilidad.

AUC evalúa ordenamiento y separación. No garantiza calibración perfecta de probabilidades. Esto significa que AUC puede ser alto aunque el valor 0.80 no signifique literalmente que 80 de cada 100 casos similares serán positivos. Para decisiones que dependen de probabilidades interpretables, la calibración puede ser importante, aunque no profundizaremos en ese tema en este capítulo.

También debemos tener cuidado con clases muy desbalanceadas. En algunos problemas con pocos positivos reales, ROC-AUC puede parecer aceptable aunque el modelo no produzca una lista de alertas suficientemente precisa. Por eso, en esos casos puede convenir mirar también precision, recall, F1-score y, en ciertos contextos, curvas Precision-Recall.

AUC no reemplaza la matriz de confusión. La matriz de confusión nos muestra qué ocurre con un umbral concreto. AUC nos muestra la capacidad general de separación a través de umbrales. Son miradas complementarias. Matriz de confusión evalúa una decisión concreta. ROC-AUC evalúa la separación general del modelo.

Por ejemplo, si una empresa ya decidió que solo puede contactar a clientes con probabilidad mayor a 0.75, la matriz de confusión para ese umbral será muy importante. Pero si todavía está comparando modelos y quiere saber cuál ordena mejor a los clientes según riesgo de abandono, AUC puede ser una métrica útil.

AUC mide el área bajo la curva ROC y resume la capacidad del modelo para separar positivos y negativos a través de distintos umbrales. Un AUC alto indica que el modelo tiende a asignar puntuaciones más altas a los positivos reales que a los negativos reales. Es útil para comparar modelos y evaluar capacidad general de ordenamiento, pero no decide por sí solo el umbral adecuado, no reemplaza la matriz de confusión y no garantiza que las probabilidades estén perfectamente calibradas.

Limitaciones de ROC-AUC

ROC-AUC es una métrica muy útil para evaluar la capacidad general de un modelo para separar clases. Permite comparar clasificadores sin depender de un único umbral y resume, en un solo valor, el comportamiento de la curva ROC. Sin embargo, como toda métrica, tiene límites. Usarla sin interpretación puede llevar a conclusiones incompletas o equivocadas.

La primera limitación importante es que ROC-AUC no nos dice qué ocurre con un umbral concreto. Evalúa el comportamiento del modelo a través de muchos umbrales posibles, pero en la práctica casi siempre necesitamos tomar una decisión específica.

Por ejemplo, un sistema de abandono de clientes puede terminar usando un umbral determinado para decidir a quién contactar. Un sistema de fraude puede usar un umbral para enviar operaciones a revisión. Un filtro de spam puede usar un umbral para mover correos a la carpeta de spam. En todos esos casos, necesitamos saber cuántos falsos positivos y falsos negativos se producen con ese umbral particular.

Mientras que ROC-AUC evalúa separación general, la matriz de confusión evalúa una decisión concreta. Un modelo puede tener buen AUC y, aun así, no producir el equilibrio deseado en el umbral que realmente necesitamos. Por eso, después de comparar modelos con AUC, debemos revisar matrices de confusión, precision, recall y F1-score para los umbrales relevantes.

La segunda limitación es que ROC-AUC no elige el umbral por nosotros. Un AUC alto indica que el modelo ordena razonablemente bien los casos positivos por encima de los negativos. Pero decidir dónde cortar sigue siendo una decisión del proyecto.

Si queremos detectar casi todos los positivos reales, elegiremos un umbral más bajo y aceptaremos más falsas alarmas. Si queremos pocas falsas alarmas, elegiremos un umbral más alto y aceptaremos dejar pasar más positivos. ROC-AUC ayuda a

entender el potencial del modelo, pero no resuelve el costo de los errores. AUC alto no responde automáticamente:

- ¿Qué umbral usamos?
- ¿Cuántas alertas podemos revisar?
- ¿Qué error es más costoso?

La tercera limitación es que ROC-AUC puede ser poco informativo cuando las clases están muy desbalanceadas. Esto ocurre, por ejemplo, cuando la clase positiva es rara: fraude, fallas críticas, ciertos eventos médicos, intrusiones o abandonos poco frecuentes.

En estos casos, puede haber muchísimos negativos y muy pocos positivos. La tasa de falsos positivos se calcula sobre todos los negativos reales:

$$\text{tasa de falsos positivos} = \text{FP} / (\text{FP} + \text{VN})$$

FP
falsos positivos

VN
verdaderos negativos

Tasa de falsos positivos.

Cuando hay una gran cantidad de verdaderos negativos, una cantidad considerable de falsos positivos puede representar una tasa pequeña. El gráfico ROC puede verse razonablemente bien, aunque en la práctica el modelo genere demasiadas alertas falsas para que el sistema sea útil.

Supongamos un problema con 10 000 transacciones normales y 100 transacciones fraudulentas. Si un modelo produce 300 falsos positivos, la tasa de falsos positivos será:

$$\text{FP} / (\text{FP} + \text{VN}) = 300 / 10\,000 = 0.03$$

Tasa de falsos positivos.

Una tasa de falsos positivos de 0.03 puede parecer baja. Pero 300 transacciones legítimas enviadas a revisión pueden ser demasiadas, según la capacidad del equipo o el impacto sobre los usuarios. En problemas muy desbalanceados, una tasa pequeña puede representar un volumen operativo grande.

Por eso, en este tipo de problemas conviene mirar también precisión. La precisión responde qué proporción de las alertas positivas eran realmente positivas. Si la clase

positiva es rara, esta pregunta suele ser muy importante, porque necesitamos saber cuántas de las alertas generadas son útiles.

En clases muy desbalanceadas ROC-AUC puede verse bien, pero precision puede revelar demasiadas falsas alarmas.

La cuarta limitación es que ROC-AUC resume toda la curva, aunque quizás solo nos interese una parte. En algunos problemas, no nos da lo mismo cualquier región de la curva ROC. Tal vez solo nos interesan umbrales con tasa de falsos positivos muy baja, porque no podemos tolerar muchas falsas alarmas. O quizá nos importan especialmente umbrales con sensibilidad muy alta, porque no queremos dejar pasar positivos reales.

AUC calcula un área general. Puede ser alto aunque el modelo no sea especialmente bueno en la región operativa que nos interesa. Por eso, si el problema exige trabajar en una zona específica, debemos observar esa zona de la curva y no quedarnos únicamente con el valor global.

Por ejemplo, en un sistema de seguridad, puede ser inaceptable una tasa de falsos positivos mayor a cierto valor porque el equipo no podría revisar tantas alertas. En ese caso, nos interesa comparar modelos dentro de ese rango, no en toda la curva. Un modelo con AUC general apenas menor podría ser mejor en la zona realmente útil.

La quinta limitación es que ROC-AUC no garantiza probabilidades bien calibradas. Como vimos, AUC mide principalmente ordenamiento: si los positivos reales tienden a recibir puntuaciones más altas que los negativos reales. Pero no nos asegura que una probabilidad de 0.80 signifique realmente que 80 de cada 100 casos similares serán positivos.

Esto puede parecer una diferencia sutil, pero es importante. Para ordenar casos por prioridad, un buen AUC puede ser suficiente en muchas situaciones. Pero si una decisión depende de interpretar la probabilidad como riesgo absoluto, necesitamos tener más cuidado.

Con un AUC alto el modelo ordena bien los casos. No implica necesariamente probabilidades perfectamente interpretables.

Por ejemplo, un modelo puede ubicar correctamente a los clientes con mayor riesgo de abandono por encima de los de menor riesgo, pero sobreestimar los valores. Puede asignar 0.90 a muchos clientes que, en realidad, abandonan con una frecuencia mucho menor. El ranking puede ser útil, pero la probabilidad como número debe interpretarse con cautela.

La sexta limitación es que ROC-AUC puede ocultar diferencias entre grupos o clases de interés. Un modelo puede tener buen desempeño general, pero funcionar peor para ciertos segmentos de la población, ciertos tipos de casos o ciertas condiciones particulares. Una métrica global no revela automáticamente esos problemas.

Por ejemplo, en un sistema de riesgo, el modelo puede separar bien casos en promedio, pero fallar más en ciertos grupos de clientes. En diagnóstico asistido, puede funcionar mejor para algunas edades o condiciones que para otras. En seguridad, puede detectar ciertos tipos de amenazas y fallar en otros. Un AUC global alto no elimina la necesidad de analizar errores concretos.

Este punto también se aplica a problemas multiclase. ROC-AUC puede extenderse a clasificación multiclase usando distintas estrategias, pero su interpretación se vuelve menos directa. En esos casos, conviene mirar métricas por clase, matrices de confusión y análisis de errores, no solo un promedio.

Finalmente, ROC-AUC puede transmitir una falsa sensación de seguridad si se usa como única métrica. Un valor alto parece resumir mucho, pero no responde todas las preguntas importantes. No nos dice si el modelo es justo, si los datos tenían fuga, si las variables son aceptables, si el comportamiento será estable en el tiempo, si el umbral elegido es adecuado o si las alertas resultantes pueden gestionarse.

Por eso, conviene usar ROC-AUC como una herramienta dentro de una evaluación más amplia. ROC-AUC ayuda a comparar modelos, pero no reemplaza a la matriz de confusión, precision, recall, análisis de umbrales, revisión de errores, o el criterio del problema.

ROC-AUC mide la capacidad general de separación del modelo, pero no decide el umbral, no describe una decisión concreta, puede ser engañoso en clases muy desbalanceadas, puede ocultar problemas en regiones específicas de la curva y no garantiza probabilidades bien calibradas. Es una métrica poderosa, pero debe interpretarse junto con otras métricas y con el contexto real en el que se usará el clasificador.

Breve mención a Precision-Recall Curve

La curva ROC no es la única forma de estudiar el comportamiento de un clasificador al variar el umbral. Otra herramienta muy útil es la **curva Precision-Recall**, especialmente cuando trabajamos con clases desbalanceadas o cuando la clase positiva es rara y muy importante.

Como su nombre indica, la curva Precision-Recall muestra la relación entre **precision** y **recall** para distintos umbrales de decisión. En lugar de comparar sensibilidad contra tasa de falsos positivos, como hace la curva ROC, compara directamente dos métricas centradas en la clase positiva: mientras la curva ROC muestra la sensibilidad frente a la tasa de falsos positivos, la curva Precision-Recall muestra la **precision** frente al **recall**.

Esta diferencia es importante. Cuando la clase positiva es rara, muchas veces nos interesa saber qué tan útiles son las alertas positivas. No alcanza con detectar una buena proporción de positivos reales; también necesitamos saber cuántas de las alertas generadas realmente corresponden a casos positivos.

Por ejemplo, en detección de fraude, la mayoría de las transacciones suelen ser normales y solo una pequeña proporción corresponde a fraude. En ese contexto, un modelo puede lograr una tasa de falsos positivos baja y aun así generar muchas alertas falsas en términos absolutos, porque la cantidad de transacciones normales es enorme. La curva ROC puede verse razonablemente bien, pero la **precision** puede mostrar que muchas alertas positivas no son realmente fraude.

La curva Precision-Recall pone el foco justamente en ese problema. Para cada umbral, observa cuántos positivos reales detectamos, mediante el **recall**, y qué proporción de las alertas positivas eran correctas, mediante la **precision**.

Si bajamos el umbral, el modelo predice más casos como positivos. Esto suele aumentar **recall**, porque detecta más casos positivos reales. Pero también puede reducir **precision**, porque aparecen más falsas alarmas.

Si subimos el umbral, el modelo predice menos casos como positivos. Esto puede aumentar **precision**, porque las alertas son más selectivas, pero puede reducir **recall**, porque deja pasar más positivos reales.

La curva Precision-Recall muestra ese intercambio de manera directa. Un umbral bajo suele producir más **recall**, pero puede reducir la **precision**. Por el contrario, un umbral alto puede aumentar la **precision**, pero reducir el **recall**.

Esta curva puede ser más informativa que ROC cuando la clase positiva es minoritaria y lo que más nos importa es la calidad de las alertas. En problemas como fraude, detección de fallas raras, eventos médicos poco frecuentes, intrusiones de seguridad o abandono de clientes de alto valor, mirar **precision** y **recall** a distintos umbrales puede dar una imagen más clara del uso práctico del modelo.

Supongamos un modelo de fraude que detecta muchos fraudes reales, pero produce tantas alertas falsas que solo una de cada veinte alertas corresponde realmente a fraude. Su **recall** puede parecer atractivo, pero la **precision** indica que el equipo

deberá revisar muchas operaciones normales para encontrar pocos fraudes. La curva Precision-Recall permite ver cómo cambia esa situación al mover el umbral.

En `scikit-learn`, podemos obtener los valores de una curva Precision-Recall con `precision_recall_curve`:

```
from sklearn.metrics import precision_recall_curve

precision, recall, thresholds = precision_recall_curve(y_test, probabilidades)
```

Como en la curva ROC, usamos las probabilidades o puntuaciones del modelo, no solo las clases finales. Esto permite analizar distintos umbrales posibles.

También existe una métrica resumida asociada a esta curva: el área bajo la curva Precision-Recall. En `scikit-learn`, una medida frecuente para resumir este comportamiento es `average_precision_score`.

```
from sklearn.metrics import average_precision_score

average_precision_score(y_test, probabilidades)
```

No profundizaremos en esta métrica en este capítulo, porque el objetivo principal es comprender ROC y AUC. Pero conviene saber que, en problemas con clases muy desbalanceadas, las curvas Precision-Recall pueden ser una herramienta muy valiosa para complementar el análisis.

Lo importante es no pensar estas curvas como competidoras absolutas. ROC y Precision-Recall responden preguntas distintas. ROC ayuda a estudiar qué tan bien separa el modelo los casos positivos de los negativos. Precision-Recall, en cambio, se concentra en qué tan bien detecta los positivos reales y qué tan confiables son las alertas positivas generadas.

En muchos proyectos conviene mirar ambas, junto con la matriz de confusión y las métricas ya estudiadas. Una curva puede mostrar una buena capacidad general de separación, mientras que la otra revela si el modelo produce alertas útiles en un contexto desbalanceado.

La curva Precision-Recall muestra cómo cambian precision y recall al variar el umbral de clasificación. Es especialmente útil cuando la clase positiva es rara o cuando la calidad de las alertas positivas es central para el problema. No reemplaza a ROC-AUC, pero la complementa. En problemas reales, elegir una métrica o una curva depende de qué pregunta necesitamos responder y de qué tipo de error tiene más consecuencias.

Evaluar probabilidades antes de tomar decisiones

En este capítulo dimos un paso más en la evaluación de clasificadores. Hasta ahora habíamos trabajado principalmente con clases predichas: positivo o negativo, abandona o permanece, fraude o normal. Pero muchos modelos también producen probabilidades o puntuaciones, y esas salidas permiten analizar el comportamiento del clasificador con más detalle.

La idea central es que la clase final depende de un umbral. Si usamos un umbral de 0.5, una probabilidad mayor o igual que 0.5 se transforma en clase positiva. Pero ese umbral no es obligatorio ni universal. Podemos moverlo para obtener un modelo más sensible o más conservador, según el problema.

Un umbral más bajo hace que se predigan más casos positivos. Esto suele aumentar el recall o la sensibilidad, aunque también puede generar más falsos positivos. Por el contrario, un umbral más alto reduce la cantidad de casos predichos como positivos y puede aumentar la especificidad, pero también producir más falsos negativos.

Modificar el umbral no cambia el modelo entrenado, pero sí cambia la decisión que tomamos a partir de sus probabilidades. Por eso, evaluar un clasificador no consiste solamente en mirar qué accuracy obtiene con el umbral por defecto. También implica preguntarse si ese umbral produce una matriz de confusión adecuada para el uso real del modelo.

La sensibilidad y la especificidad ayudan a describir este equilibrio. La sensibilidad mide qué proporción de positivos reales detecta el modelo. La especificidad mide qué proporción de negativos reales descarta correctamente. Cuando movemos el umbral, suele aparecer una tensión entre ambas.

La curva ROC muestra cómo cambia esa relación entre sensibilidad y tasa de falsos positivos a medida que variamos el umbral. En lugar de analizar una única decisión, nos permite observar muchos puntos de corte posibles. Esto ayuda a comprender si el modelo logra separar positivos y negativos de manera general.

AUC resume esa curva en un solo valor. Un AUC alto indica que el modelo tiende a asignar puntuaciones más altas a los positivos reales que a los negativos reales. Por eso puede ser útil para comparar modelos antes de elegir un umbral específico. Sin embargo, AUC no decide por nosotros. No reemplaza la matriz de confusión, no elige el punto de corte y no garantiza que las probabilidades estén perfectamente calibradas.

También vimos que ROC-AUC puede ser insuficiente cuando las clases están muy desbalanceadas. En esos casos, conviene mirar con atención precision y recall, porque necesitamos saber no solo cuántos positivos detectamos, sino también

cuántas alertas positivas son realmente útiles. La curva Precision-Recall permite analizar directamente ese intercambio.

Las probabilidades nos permiten evaluar clasificadores de una manera más flexible que las clases finales. El umbral transforma probabilidades en decisiones; la curva ROC muestra cómo cambia el desempeño al mover ese umbral; AUC resume la capacidad general de separación; y la curva Precision-Recall puede ser especialmente valiosa cuando la clase positiva es rara o crítica.

La idea más importante es que ninguna métrica decide sola. Un modelo puede tener buen AUC, buena accuracy o buen F1-score, y aun así no ser adecuado para un uso concreto. Evaluar correctamente significa conectar las métricas con el problema real: qué error es más costoso, qué alertas pueden revisarse, qué consecuencias tiene actuar sobre una predicción y qué nivel de incertidumbre estamos dispuestos a aceptar.

A partir de aquí, ya contamos con una base sólida para evaluar clasificadores. Sabemos mirar aciertos globales, matrices de confusión, precision, recall, F1-score, probabilidades, umbrales, ROC y AUC. En los próximos capítulos podremos avanzar hacia modelos y técnicas más potentes, sin perder de vista una idea esencial: un modelo no se juzga solo por producir predicciones, sino por producir predicciones útiles, evaluadas con criterios adecuados.

Capítulo 14 · Regresión: predecir valores numéricos

Diferencia entre clasificación y regresión

Hasta ahora trabajamos principalmente con problemas de clasificación. En esos problemas, el modelo debe elegir una categoría: abandona o permanece, fraude o normal, spam o no spam, requiere revisión o no requiere revisión. La salida del modelo es una clase.

Pero no todos los problemas de Machine Learning consisten en elegir entre categorías. Muchas veces queremos predecir un **valor numérico**: el precio de una vivienda, las ventas del próximo mes, la temperatura de mañana, la cantidad de energía que consumirá un edificio, el tiempo estimado de entrega o la demanda de un producto.

Ese tipo de problema se llama **regresión**.

La diferencia central es la naturaleza de la variable objetivo: en problemas de clasificación, la variable objetivo es una categoría; en problemas de regresión, la variable objetivo es un valor numérico.

En clasificación, el modelo aprende a asignar etiquetas. En regresión, aprende a estimar cantidades.

Por ejemplo:

Clasificación: ¿Este cliente abandonará? Respuesta posible: sí / no	Regresión: ¿Cuánto gastará este cliente el próximo mes? Respuesta posible: 14500 pesos
---	--

Otro ejemplo:

Clasificación: ¿Esta vivienda pertenece a la categoría "precio alto"? Respuesta posible: sí / no	Regresión: ¿Cuál será el precio estimado de esta vivienda? Respuesta posible: 128000 dólares
--	--

En ambos casos podemos estar usando datos parecidos, pero el tipo de pregunta cambia. Y cuando cambia la pregunta, cambia también el tipo de modelo, la forma de evaluar los resultados y la interpretación de los errores.

En clasificación, una predicción puede ser correcta o incorrecta según coincida o no con la clase real. Si el modelo predice “fraude” y la transacción era fraude, acierta. Si predice “normal” y era fraude, se equivoca. Por eso usamos métricas como accuracy, matriz de confusión, precision, recall y F1-score.

En regresión, el error no suele ser simplemente “correcto” o “incorrecto”. Si un modelo predice que una vivienda cuesta 130000 dólares y el precio real era 132000, la predicción no es exacta, pero está bastante cerca. Si predice 220000, el error es mucho mayor. En regresión importa **cuánto se equivocó** el modelo.

Clasificación:

¿la clase predicha coincide con la clase real?

Regresión:

¿qué tan lejos está el valor predicho del valor real?

Esto cambia por completo la evaluación. En regresión no construimos una matriz de confusión, porque no hay clases positivas y negativas. Tampoco hablamos de falsos positivos o falsos negativos en el sentido habitual. En su lugar, medimos diferencias numéricas entre valores reales y valores predichos.

Por ejemplo:

Precio real:

150000

Precio predicho:

147000

Error:

3000

O también:

Ventas reales:

820

Vent. predichas:

760

Error:

60

En los próximos capítulos estudiaremos métricas específicas para este tipo de problemas, como MAE, MSE, RMSE y R^2 . Todas ellas intentan responder, de distintas

maneras, qué tan grandes son los errores del modelo cuando predice valores numéricos.

También cambia la forma de interpretar la salida del modelo. En clasificación, una probabilidad puede convertirse en una clase mediante un umbral. En regresión, el modelo devuelve directamente un número estimado. No necesitamos elegir entre “positivo” y “negativo”, sino analizar si el número predicho es razonable.

Clasificador:

$P(\text{abandona}) = 0.73$
umbral = 0.5
predicción final = **abandona**

Regresor:

precio estimado = **128000**

Esto no significa que la regresión sea más simple. Predecir un valor numérico puede ser difícil porque el resultado puede tomar muchos valores posibles. En clasificación binaria, el modelo elige entre dos clases. En regresión, una pequeña diferencia numérica ya genera un error.

Por ejemplo, si predecimos la temperatura máxima de mañana, no basta con decir “alta” o “baja”. El modelo debe estimar un valor: 23 grados, 24.5 grados, 31 grados. Cuanto más alejada esté la predicción del valor real, mayor será el error.

En regresión, los errores también deben interpretarse en unidades reales. Si predecimos precios de viviendas, el error estará en unidades monetarias. Si predecimos temperatura, estará en grados. Si predecimos demanda, estará en unidades vendidas. Esta característica hace que los resultados puedan ser muy concretos, pero también exige cuidado: un error de 10 puede ser pequeño en un problema y enorme en otro.

Error de 10 dólares:

puede ser irrelevante en el precio de una casa.

Error de 10 grados:

puede ser enorme en una predicción de temperatura.

Por eso, en regresión no alcanza con mirar el número de la métrica. Necesitamos entender la escala del problema. Un modelo puede tener un error promedio de 500, pero ese valor puede ser excelente si estamos prediciendo ventas de cientos de miles de unidades, o muy malo si estamos prediciendo consumos pequeños.

También debemos distinguir entre una variable numérica verdadera y una categoría codificada con números. Esta diferencia es muy importante.

Supongamos una variable objetivo con estos valores:

0 = riesgo bajo
1 = riesgo medio
2 = riesgo alto

Aunque esos valores sean números, el problema no necesariamente es de regresión. Si los números representan categorías, estamos ante un problema de clasificación. El modelo no debería interpretar que “riesgo alto” es simplemente el doble de “riesgo medio” o que puede predecir un valor como 1.6 de manera directa.

En cambio, si la variable objetivo representa una cantidad real, como precio, edad, temperatura, consumo o ingresos, entonces sí estamos frente a un problema de regresión.

Números que representan categorías:

clasificación.

Números que representan cantidades:

regresión.

Esta distinción evita muchos errores al plantear proyectos. No debemos elegir regresión solo porque la columna objetivo contiene números. Debemos preguntarnos qué significan esos números.

También puede ocurrir que un mismo fenómeno se pueda formular como clasificación o como regresión, según el objetivo. Por ejemplo, con viviendas podríamos plantear dos problemas distintos:

Clasificación:

predecir si la vivienda será barata, media o cara.

Regresión:

predecir el precio exacto o aproximado de la vivienda.

Con clientes:

Clasificación:

predecir si el cliente será de alto valor o no.

Regresión:

predecir cuánto gastará un cliente.

Con demanda:

Clasificación:

predecir si habrá demanda alta, media o baja.

Regresión:

predecir cuántas unidades se venderán.

La elección depende de la decisión que queremos tomar. Si necesitamos estimar cantidades para planificar inventario, presupuesto o capacidad, probablemente necesitemos regresión. Si necesitamos asignar casos a grupos para tomar acciones diferenciadas, tal vez convenga clasificación.

Clasificación y regresión son dos tipos fundamentales de aprendizaje supervisado. En clasificación, la variable objetivo es categórica y el modelo predice clases. En regresión, la variable objetivo es numérica y el modelo predice cantidades. Esta diferencia afecta la forma de entrenar, evaluar e interpretar los modelos. Antes de elegir un algoritmo, debemos asegurarnos de que la pregunta esté bien formulada: no es lo mismo decidir entre categorías que estimar un valor.

Variable objetivo continua

En los problemas de regresión, la variable objetivo suele ser una **variable numérica continua**. Esto significa que el valor que queremos predecir representa una cantidad y puede tomar muchos valores posibles dentro de cierto rango.

Por ejemplo, si queremos predecir el precio de una vivienda, la variable objetivo puede ser:

```
precio = 85000  
precio = 123500  
precio = 240000
```

Si queremos predecir temperatura:

```
temperatura = 18.5  
temperatura = 24.0  
temperatura = 31.7
```

Si queremos predecir consumo eléctrico:

```
consumo = 420.8 kWh  
consumo = 615.3 kWh  
consumo = 902.1 kWh
```

muchos valores numéricos posibles.

En todos estos casos, la variable objetivo no representa una clase, sino una cantidad. El modelo no debe elegir entre etiquetas predefinidas, sino estimar un valor numérico.

La palabra “continua” se usa porque, al menos conceptualmente, la variable puede tomar muchos valores intermedios. Entre 100000 y 101000 pesos puede haber muchos precios posibles. Entre 20 y 21 grados puede haber 20.1, 20.2, 20.5 o 20.87. La salida del modelo no está limitada a unas pocas categorías.

Clasificación:

valores posibles limitados a clases.

Regresión:

muchos valores numéricos posibles.

En la práctica, no todas las variables de regresión son perfectamente continuas. Algunas son cantidades discretas, como número de ventas, cantidad de productos demandados, número de llamadas recibidas o cantidad de pasajeros. Aunque sean enteros, muchas veces se tratan como regresión porque representan cantidades y porque el interés está en estimar un valor.

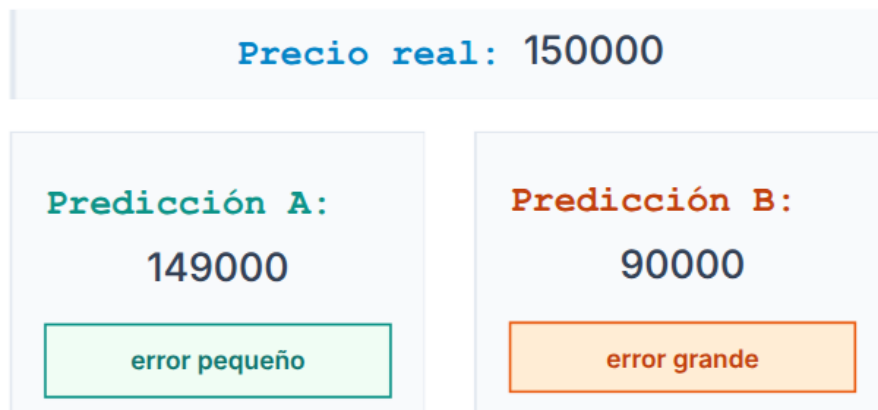
Por ejemplo:

```
unidades vendidas mañana = 138  
llamadas recibidas por hora = 42  
pasajeros estimados = 1260
```

Estos valores no son categorías. Son conteos. Puede tener sentido tratarlos como regresión si queremos estimar la magnitud de la cantidad. La diferencia no está en si el número tiene decimales, sino en qué significa.

Una buena forma de distinguir regresión de clasificación es preguntarse si tiene sentido calcular distancias entre valores de la variable objetivo.

Si el precio real de una vivienda es 150000 y el modelo predice 145000, podemos decir que el error fue de 5000. Si predice 149000, el error fue menor. Si predice 90000, el error fue mucho mayor. La distancia numérica tiene significado.



En cambio, si las clases son “spam” y “no spam”, no tiene sentido decir que una clase está numéricamente cerca de la otra. O si las clases son “bajo”, “medio” y “alto”, puede haber un orden conceptual, pero no necesariamente una distancia matemática clara. En esos casos, debemos pensar cuidadosamente si el problema es realmente de regresión o de clasificación.

También puede haber variables objetivo que parecen numéricas pero no deberían tratarse como continuas. Un código postal, un número de cliente, un identificador de producto o un código de categoría pueden estar compuestos por números, pero no representan cantidades.

Código postal: 1425
ID de cliente: 839102
Código de producto: 5017

Estos valores no deberían ser tratados como objetivos de regresión. Predecir un ID no tiene sentido como estimación numérica. Un error de 10 en un código de producto no significa que la predicción esté “cerca”. Son identificadores, no cantidades.

Por eso, al definir la variable objetivo, no alcanza con mirar el tipo de dato en la tabla. Debemos interpretar el significado de la columna. No toda columna numérica es una variable de regresión. Una variable de regresión representa una cantidad con sentido.

En un DataFrame, podríamos tener una columna numérica llamada `precio`, otra llamada `cantidad_vendida` y otra llamada `id_producto`. Las tres pueden verse como números, pero solo las dos primeras podrían ser objetivos naturales de regresión.

<code>precio</code>	→ cantidad monetaria	→ posible regresión
<code>cantidad_vendida</code>	→ conteo	→ posible regresión
<code>id_producto</code>	→ identificador	→ no es regresión

Esta interpretación es parte del planteo del problema. Antes de entrenar un modelo, debemos saber exactamente qué queremos predecir y por qué ese valor es útil.

La variable objetivo continua también afecta la forma en que entendemos el error. En clasificación, si el modelo predice una clase incorrecta, se cuenta como error. En regresión, casi ninguna predicción será exactamente igual al valor real. Esto es normal. Lo importante es saber si el error es aceptable.

Por ejemplo, si un modelo predice el precio de una vivienda con un error de 1000 dólares, puede ser muy bueno. Si predice la temperatura con un error de 1000 grados, no tiene sentido. La escala de la variable objetivo define cómo interpretamos el desempeño. El error siempre debe interpretarse en relación con la variable que estamos prediciendo.

También debemos considerar la variabilidad natural del fenómeno. Algunos valores son difíciles de predecir con exactitud porque dependen de muchos factores, algunos de los cuales no están en los datos. Las ventas futuras pueden depender de promociones, cambios económicos, clima, competencia o eventos inesperados. La demanda puede cambiar por razones que el modelo no observa. El precio de un producto puede variar por condiciones del mercado.

Por eso, una variable objetivo continua no implica que el modelo pueda predecir con precisión perfecta. La regresión trabaja con estimaciones, no con certezas.

En muchos casos, además, la variable objetivo puede tener valores extremos. En precios de viviendas, puede haber propiedades muy caras que se alejan del resto. En consumo eléctrico, puede haber edificios con consumos muy superiores al promedio. En ventas, puede haber días excepcionales por promociones o eventos especiales.

Estos valores extremos pueden influir mucho en el entrenamiento y en la evaluación de modelos de regresión. No siempre deben eliminarse, porque pueden representar casos reales importantes. Pero sí deben revisarse y comprenderse. Pueden ser errores de carga, casos excepcionales, o situaciones reales relevantes.

La variable objetivo continua también puede requerir transformaciones en algunos proyectos. Por ejemplo, precios o ingresos suelen tener distribuciones muy asimétricas: muchos valores bajos o medios y pocos valores muy altos. En ciertos casos, aplicar transformaciones puede ayudar al modelo. No profundizaremos en ese tema ahora, pero conviene saber que la forma de la variable objetivo también importa.

Otra cuestión importante es el momento en que la variable objetivo se conoce. Igual que en clasificación, la variable objetivo debe representar algo que conocemos en los datos históricos, pero que queremos predecir para casos nuevos. Si queremos

predecir ventas del próximo mes, en los datos históricos necesitaremos meses pasados con sus ventas reales. Para el futuro, esas ventas todavía no estarán disponibles.

Datos históricos:

conocemos X y conocemos y.

Caso nuevo:

conocemos X, pero queremos estimar y.

Este esquema es el mismo que en aprendizaje supervisado. La diferencia está en que **y** ahora no es una etiqueta categórica, sino un valor numérico.

En `scikit-learn`, la estructura sigue siendo familiar:

```
X = df[["superficie", "ambientes", "antiguedad"]]  
y = df["precio"]
```

En este ejemplo, **X** contiene las variables de entrada y **y** contiene la variable objetivo continua: el precio. El modelo aprenderá relaciones entre las características de las viviendas y sus precios observados.

Cuando luego reciba una vivienda nueva, usará sus variables de entrada para estimar un precio:

```
precio_estimado = modelo.predict(X_nuevo)
```

La salida será un número, no una clase.

En regresión la variable objetivo representa una cantidad numérica que queremos estimar. Puede ser una variable continua en sentido estricto, como temperatura o precio, o una cantidad discreta, como unidades vendidas, si el objetivo es predecir magnitudes. Lo importante es que los valores tengan significado numérico real y que los errores puedan interpretarse como distancias entre cantidades. Antes de tratar un problema como regresión, debemos asegurarnos de que la variable objetivo no sea simplemente una categoría codificada, un identificador o un número sin sentido cuantitativo.

Error de predicción

En regresión, el modelo no suele acertar exactamente el valor real. Esto es normal. Si intentamos predecir el precio de una vivienda, las ventas de un producto o la

temperatura de mañana, es esperable que exista alguna diferencia entre el valor predicho y el valor observado.

Esa diferencia se llama **error de predicción**.

Podemos pensarlo de manera sencilla:

```
error = valor real - valor predicho
```

O, si solo nos interesa la magnitud de la diferencia:

```
error absoluto =  
|valor real - valor predicho|
```

Supongamos que un modelo predice el precio de una vivienda:

```
precio real      = 150000  
precio predicho = 147000  
  
diferencia = 150000 - 147000  
           = 3000
```

El modelo se equivocó por 3000 unidades monetarias. En este caso, la predicción fue menor que el valor real. Decimos que el modelo **subestimó** el precio.

Si, en cambio, el modelo hubiera predicho 158000, el error sería:

```
precio real      = 150000  
precio predicho = 158000  
  
diferencia = 150000 - 158000  
           = -8000
```

El signo negativo indica que el modelo predijo un valor mayor que el real. Es decir, **sobreestimó** el precio. Si miramos solo la magnitud del error, diríamos que se equivocó por 8000.

Esta diferencia entre error con signo y error absoluto es importante. El error con signo nos dice en qué dirección se equivocó el modelo: si predijo por encima o por debajo. El error absoluto nos dice qué tan grande fue la equivocación, sin importar la dirección.

- **Error positivo:** el modelo predijo por debajo del valor real.
- **Error negativo:** el modelo predijo por encima del valor real.

- **Error absoluto:** tamaño del error, sin importar la dirección.

En regresión, no basta con saber si una predicción fue exacta. De hecho, en muchos problemas sería poco realista esperar exactitud perfecta. Lo importante es saber si el error es pequeño o grande para el contexto del problema.

Por ejemplo, imaginemos un modelo que predice la temperatura máxima de un día:

Temperatura real:	28.0 °C
Temperatura predicha:	27.5 °C
Error absoluto:	0.5 °C

Este error podría ser aceptable para muchos usos. Pero si el modelo predice:

Temperatura real:	28.0 °C
Temperatura predicha:	18.0 °C
Error absoluto:	10.0 °C

el error ya es mucho más importante. Aunque ambas predicciones sean “incorrectas” en sentido estricto, no son igualmente malas.

Lo mismo ocurre con ventas. Si una tienda esperaba vender 1000 unidades y el modelo predijo 980, el error puede ser pequeño. Si predijo 300, el error puede afectar seriamente la planificación de stock.



Esta es una diferencia esencial entre clasificación y regresión. En clasificación, una predicción suele contarse como correcta o incorrecta. En regresión, las predicciones se ubican en una escala de error. Algunas están muy cerca del valor real; otras están lejos; otras pueden ser inaceptables.

También es importante analizar si el modelo tiende a equivocarse siempre en la misma dirección. Si en muchos casos predice valores menores que los reales, puede estar subestimando sistemáticamente. Si predice valores mayores, puede estar sobreestimando.

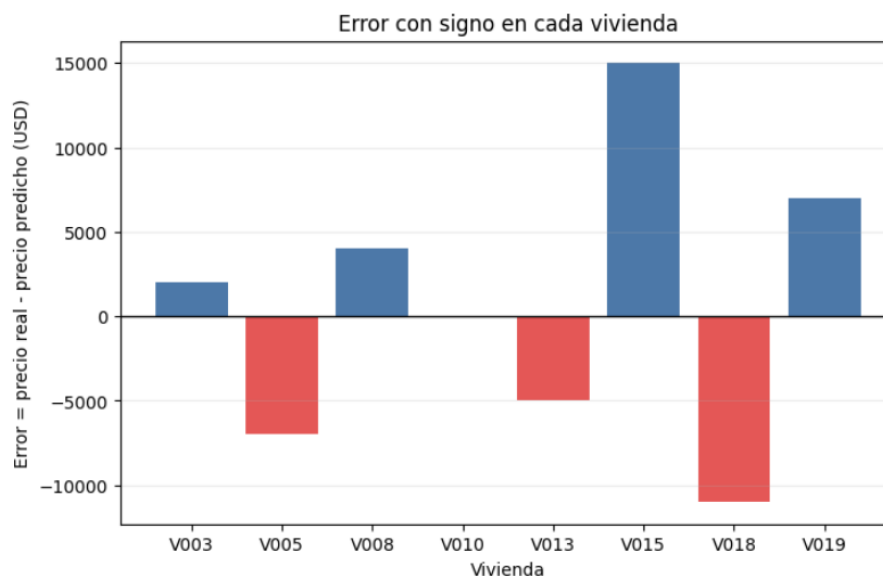
Por ejemplo, un modelo de demanda que siempre predice menos ventas de las reales puede producir faltantes de stock. Un modelo que siempre predice más ventas puede llevar a comprar demasiado inventario. En ambos casos, el error promedio puede tener consecuencias distintas según la dirección.

- **Subestimar demanda:** puede generar faltantes.
- **Sobreestimar demanda:** puede generar exceso de stock.

Por eso, al evaluar regresión, no solo importa el tamaño del error. También puede importar su dirección. En algunos problemas, sobreestimar y subestimar tienen costos parecidos. En otros, no.

Supongamos un modelo que predice tiempos de entrega. Si predice que un pedido tardará 30 minutos y tarda 35, el cliente esperará un poco más de lo informado. Si predice 50 minutos y tarda 35, quizá el cliente reciba una estimación conservadora. Ambos errores son de 5 o 15 minutos, pero pueden tener efectos distintos sobre la experiencia del usuario.

En precios también puede ocurrir algo similar. Sobreestimar el precio de venta de una vivienda puede hacer que tarde más en venderse. Subestimarlos puede hacer que se venda rápido, pero por menos dinero del posible. El mismo tamaño de error puede tener consecuencias diferentes.



Las barras por encima de cero representan subestimaciones y las que quedan por debajo, sobreestimaciones. (Fuente: Cuaderno Colab del capítulo).

Otra característica del error de predicción es que se interpreta en las unidades de la variable objetivo. Si predecimos precios, el error está en pesos, dólares u otra moneda. Si predecimos temperatura, el error está en grados. Si predecimos consumo, está en kWh. Si predecimos tiempo, está en minutos, horas o días.

El error de regresión conserva las unidades del problema cuando lo medimos como diferencia directa o absoluta.

Esto facilita la interpretación, pero también exige cuidado. Un error de 100 puede ser pequeño o grande según el problema. No tiene sentido evaluar ese número sin conocer la escala.

Por ejemplo:

- Error de 100 pesos en el precio de una vivienda: probablemente pequeño.
- Error de 100 unidades en una venta diaria de 120: probablemente enorme.

Por eso, muchas veces conviene comparar el error con valores típicos de la variable objetivo. Si el precio promedio de las viviendas es 200000 y el error promedio es 5000, estamos hablando de un 2,5% aproximado. Si el valor promedio es 8000 y el error promedio es 5000, la situación es muy distinta.

También conviene comparar el modelo con una referencia simple. Por ejemplo, podríamos preguntarnos qué pasaría si siempre predijéramos el promedio de la variable objetivo. Si nuestro modelo sofisticado no mejora mucho esa estrategia básica, tal vez no esté aportando demasiado.

- **Referencia simple:** predecir siempre el promedio.
- **Modelo útil:** debería mejorar esa referencia de manera clara y consistente.

Esta comparación es importante porque una métrica aislada puede ser difícil de interpretar. Saber que un modelo tiene un error promedio de 20 no dice demasiado si no sabemos si una estrategia simple tendría un error de 25, 50 o 200.

En regresión, los errores también pueden distribuirse de manera desigual. El modelo puede funcionar bien para valores medios y mal para valores extremos. Por ejemplo, puede predecir razonablemente viviendas de precio medio, pero fallar mucho con propiedades muy caras. Puede estimar bien la demanda habitual, pero fallar en días de promociones. Puede predecir bien temperaturas normales, pero peor en eventos extremos.

Por eso, además de calcular métricas generales, conviene revisar errores individuales y observar patrones. Si los errores se concentran en ciertos grupos, rangos o situaciones, esa información puede ser más útil que un promedio global.

Preguntas útiles:

- ¿Dónde se equivoca más el modelo?

- ¿Falla en valores altos?
- ¿Falla en valores bajos?
- ¿Falla en ciertos grupos o condiciones?

Estas preguntas conectan la evaluación de regresión con el análisis exploratorio de datos. Un modelo no debe evaluarse únicamente con una fórmula. También necesitamos mirar casos concretos y entender qué tipo de errores comete.

En `scikit-learn`, una vez entrenado un modelo de regresión, usamos `predict` para obtener valores estimados:

```
y_pred = modelo.predict(X_test)
```

Luego podemos comparar esos valores con los valores reales:

```
errores = y_test - y_pred
```

O calcular errores absolutos:

```
errores_absolutos = abs(y_test - y_pred)
```

Estas operaciones muestran la idea básica detrás de muchas métricas de regresión. MAE, MSE, RMSE y R^2 , que veremos en los próximos capítulos, son formas de resumir estos errores para evaluar el comportamiento general del modelo.

Pero antes de estudiar esas métricas, debemos conservar la intuición principal: en regresión, evaluar significa medir distancias entre números reales y números predichos. La pregunta no es simplemente si el modelo acertó, sino cuánto se equivocó y si ese error es aceptable.

El error de predicción en regresión es la diferencia entre el valor real y el valor estimado por el modelo. Puede analizarse con signo, para saber si el modelo sobreestima o subestima, o en valor absoluto, para medir la magnitud del error. Su interpretación depende de las unidades, la escala del problema, la dirección del error y las consecuencias prácticas. Evaluar regresión implica comprender no solo cuánto se equivoca el modelo en promedio, sino también dónde, cómo y con qué impacto se equivoca.

Ejemplos reales: precios, ventas, demanda, temperaturas y consumo

Los problemas de regresión aparecen cada vez que necesitamos estimar una cantidad. A diferencia de la clasificación, donde el modelo elige entre categorías, en regresión el resultado esperado es un número. Ese número puede representar dinero, unidades, grados, energía, tiempo, distancia, volumen o cualquier otra magnitud medible.

Un ejemplo clásico es la **predicción de precios**. Podemos querer estimar el precio de una vivienda a partir de su superficie, ubicación, cantidad de ambientes, antigüedad, estado general y cercanía a ciertos servicios. En este caso, la variable objetivo es el precio.

Variables de entrada:

superficie, ambientes, ubicación, antigüedad, estado.

Variable objetivo:

precio de la vivienda.

El modelo no debe decir si la vivienda es “cara” o “barata”, salvo que formulemos el problema como clasificación. En regresión, debe estimar un valor numérico: por ejemplo, 95000, 180000 o 320000 dólares.

Este tipo de predicción puede servir como apoyo para tasaciones, análisis de mercado o comparación de propiedades. Pero también exige cuidado. El precio de una vivienda puede depender de factores difíciles de medir, como el estado real de conservación, el contexto del barrio, la urgencia del vendedor, la situación económica o características particulares que no aparecen en la tabla.

Por eso, una predicción de precio debe interpretarse como una estimación, no como un valor definitivo. Un modelo puede ayudar a orientar, pero no reemplaza el análisis del contexto.

Otro ejemplo frecuente es la **predicción de ventas**. Una empresa puede querer estimar cuántas unidades venderá de un producto durante la próxima semana o el próximo mes. Para eso podría usar variables como ventas históricas, precio, promociones, temporada, día de la semana, feriados, clima o campañas publicitarias.

Variables de entrada:

ventas previas, precio, promociones, fecha, temporada.

Variable objetivo:

unidades vendidas.

Aunque las unidades vendidas sean números enteros, el problema puede tratarse como regresión porque el objetivo es estimar una cantidad. Si el modelo predice 820 unidades y se venden 790, podemos medir el error como diferencia entre cantidades.

La predicción de ventas es importante para planificar stock, compras, logística y personal. Un modelo que subestima las ventas puede generar faltantes. Un modelo que las sobreestima puede producir exceso de inventario. En este caso, la dirección del error importa tanto como su tamaño.

- **Subestimar ventas:** puede generar faltantes.
- **Sobreestimar ventas:** puede generar exceso de stock.

Un problema muy relacionado es la **predicción de demanda**. Aunque a veces se usa como sinónimo de ventas, demanda puede referirse a una necesidad esperada antes de que ocurra la venta final. Por ejemplo, una empresa de transporte puede estimar cuántos pasajeros habrá en una franja horaria. Un servicio de comida puede estimar cuántos pedidos recibirá. Una compañía eléctrica puede estimar cuánta energía será requerida por una zona.

En estos casos, la variable objetivo suele ser una cantidad esperada:

pasajeros esperados
pedidos estimados
unidades solicitadas
energía requerida

La demanda puede cambiar por patrones regulares, como horarios, días de la semana o estaciones del año, pero también por eventos especiales. Un recital, una tormenta, una promoción, un feriado o un cambio económico pueden modificar el comportamiento esperado. Si esas variables no están en los datos, el modelo puede tener dificultades para anticipar esos cambios.

Esto muestra un punto importante: un modelo de regresión solo puede aprender a partir de la información disponible. Si los datos no contienen señales sobre un evento, el modelo no puede adivinarlo mágicamente.

Otro ejemplo claro es la **predicción de temperaturas**. En este caso, el modelo puede intentar estimar la temperatura máxima de mañana, la temperatura promedio de una hora o la temperatura esperada en una región. La variable objetivo se expresa en grados.

Variables de entrada:

temperaturas previas, presión, humedad, viento, nubosidad, fecha.

Variable objetivo:

temperatura estimada.

En este tipo de problema, el error se interpreta de manera muy directa. Un error de 1 grado puede ser aceptable para ciertos usos generales. Un error de 8 grados puede ser grave. Sin embargo, el nivel de error aceptable depende de la aplicación: no es lo mismo informar una estimación general al público que tomar decisiones agrícolas, energéticas o de seguridad.

La predicción de temperaturas también ilustra que la regresión no siempre busca valores económicos. Cualquier magnitud continua puede ser objetivo de regresión si queremos estimarla a partir de datos.

Un quinto ejemplo es la **predicción de consumo**, como consumo eléctrico, consumo de agua, consumo de combustible o uso de datos móviles. En un edificio, por ejemplo, podríamos querer estimar el consumo eléctrico diario a partir de la temperatura exterior, el día de la semana, la cantidad de personas, el tipo de actividad y consumos anteriores.

Variables de entrada:

temperatura, día, ocupación, actividad, consumo histórico.

Variable objetivo:

consumo eléctrico.

Este tipo de predicción puede ayudar a planificar recursos, detectar desviaciones, optimizar costos o anticipar picos de demanda. Pero, nuevamente, el error debe interpretarse en unidades reales. Un error de 50 kWh puede ser pequeño para una planta industrial y grande para una vivienda.

Estos ejemplos muestran que la regresión no se define por el algoritmo utilizado, sino por la pregunta que hacemos. Si la pregunta apunta a estimar una cantidad, probablemente estamos ante un problema de regresión.

- ¿Cuánto costará?
- ¿Cuántas unidades se venderán?
- ¿Cuánta energía se consumirá?
- ¿Qué temperatura habrá?
- ¿Cuánto tardará?

Todas estas preguntas esperan respuestas numéricas.

También muestran que las variables de entrada pueden ser muy variadas. Podemos tener variables numéricas, categóricas, fechas, indicadores de eventos, información histórica o datos externos. Como vimos en capítulos anteriores, esas variables deben prepararse correctamente antes de entrenar el modelo. Los valores faltantes, las categorías y las escalas siguen importando.

Por ejemplo, para predecir ventas, una columna como `día_semana` puede ser categórica. Para predecir precios, una columna como `barrio` también puede ser categórica. Para predecir consumo, una columna como `temperatura` será numérica. El hecho de trabajar con regresión no elimina la necesidad de preparar los datos.

Regresión no significa usar solo variables numéricas. La variable objetivo es numérica, pero las variables de entrada pueden ser de distintos tipos.

Otra idea importante es que los errores no tienen siempre el mismo impacto. En precios, equivocarse por 5000 puede ser poco o mucho según el valor del bien. En ventas, equivocarse por 100 unidades puede ser aceptable o problemático según el volumen. En temperatura, equivocarse por 2 grados puede ser razonable para algunas decisiones, pero no para otras. En consumo energético, un error puede afectar costos, planificación o capacidad de suministro.

Por eso, al plantear un problema de regresión debemos preguntar qué nivel de error es tolerable. No basta con decir “queremos predecir ventas” o “queremos predecir precios”. Necesitamos saber para qué se usará la predicción y qué error puede aceptarse.

Una predicción de regresión debe evaluarse según el uso que tendrá el número estimado.

También conviene preguntarse si realmente necesitamos un valor exacto o si bastaría con una categoría. A veces, para tomar una decisión, no necesitamos predecir el monto preciso. Tal vez alcanza con saber si la demanda será baja, media o alta. En ese caso, podríamos transformar el problema en clasificación. Pero esa transformación implica perder detalle numérico y debe justificarse.

Los problemas de regresión aparecen en muchos contextos reales: precios, ventas, demanda, temperaturas, consumo, tiempos, ingresos, costos y cantidades. En todos ellos, el modelo intenta estimar un valor numérico a partir de variables de entrada. La utilidad de la predicción depende de la calidad de los datos, de la relación entre las variables y la cantidad objetivo, y de si el error resultante es aceptable para la decisión que se quiere tomar.

¿Cuándo un problema no debería tratarse como clasificación?

A veces, un problema que naturalmente podría formularse como regresión se transforma en clasificación. Esto ocurre cuando tomamos una variable numérica y la convertimos en categorías. Por ejemplo, en lugar de predecir el precio de una vivienda, podemos clasificarla como “barata”, “media” o “cara”. En lugar de predecir la cantidad de ventas, podemos clasificar la demanda como “baja”, “media” o “alta”.

Esta transformación puede ser útil en algunos casos, pero también puede ser una mala decisión si se hace sin criterio. No todo problema numérico debería convertirse en clasificación.

Supongamos que queremos estimar el precio de una vivienda. La variable objetivo original es numérica:

```
precio = 85000  
precio = 120000  
precio = 210000
```

Podríamos convertir esos precios en categorías:

```
menos de 100000      → precio bajo  
entre 100000 y 180000 → precio medio  
más de 180000       → precio alto
```

A partir de esa transformación, el modelo ya no intentaría predecir el precio exacto o aproximado. Solo intentaría predecir una categoría. Esto puede simplificar el problema, pero también elimina información.

Una vivienda de 99000 y una de 50000 quedarían en la misma categoría: “precio bajo”. Una vivienda de 181000 y una de 400000 quedarían en “precio alto”. Para algunas decisiones, esa clasificación puede ser demasiado gruesa.

Clasificación:

reduce el valor a una categoría

Regresión:

conserva el valor numérico.

La pregunta es si esa pérdida de detalle es aceptable. Si el objetivo es hacer una tasación aproximada, probablemente necesitamos regresión. Si solo queremos separar propiedades en rangos generales para un tablero exploratorio, la clasificación puede alcanzar. La decisión depende del uso del resultado.

Algo parecido ocurre con ventas. Si una empresa necesita planificar inventario, quizá no le alcance saber que la demanda será “alta”. Necesita estimar cuántas unidades venderá para decidir cuánto comprar, cuánto producir o cuánta capacidad logística preparar.

Clasificación:

demanda alta

Regresión:

*demanda estimada =
1250 unidades*

La categoría puede orientar, pero la cantidad permite tomar decisiones más precisas. Si la empresa necesita comprar insumos, reservar espacio o planificar distribución, un valor numérico suele ser más útil que una etiqueta.

Un problema no debería tratarse como clasificación cuando el valor exacto o aproximado de la variable objetivo importa para la decisión. Si la acción depende de la magnitud, convertir el problema en categorías puede ocultar diferencias importantes.

Por ejemplo:

- **Predecir consumo eléctrico:** si necesitamos comprar energía o anticipar picos, la cantidad estimada importa.
- **Predecir temperatura:** si debemos ajustar sistemas de riego o calefacción, los grados estimados importan.
- **Predecir tiempo de entrega:** si informamos al usuario, los minutos estimados importan.

En todos estos casos, una categoría puede ser insuficiente. Decir “consumo alto” no es lo mismo que estimar 500 kWh o 900 kWh. Decir “temperatura baja” no es lo mismo que estimar 8 grados o 2 grados. Decir “entrega lenta” no es lo mismo que estimar 45 minutos o 3 horas.

Otro riesgo de convertir una variable numérica en categorías es que los límites elegidos pueden ser arbitrarios. Si definimos “demanda alta” como más de 1000 unidades, entonces 999 unidades será “demanda media” y 1001 será “demanda alta”. Sin embargo, en la realidad esos dos valores son casi iguales.

999 unidades → demanda media
1001 unidades → demanda alta

La clasificación crea una frontera artificial. Cerca de esa frontera, pequeñas diferencias numéricas pueden producir cambios de categoría. Esto puede volver el problema menos natural y generar errores difíciles de interpretar.

En regresión, en cambio, el modelo puede equivocarse por una cantidad pequeña. Si predice 990 cuando el valor real era 1010, el error es de 20 unidades. En clasificación, si esos valores caen en categorías distintas, la predicción puede contarse como incorrecta aunque numéricamente esté muy cerca.

Este punto es importante. Al transformar una variable continua en categorías, podemos perder la noción de cercanía. Un error pequeño cerca del límite puede parecer tan grave como un error enorme entre categorías alejadas.

Valor real: 1001
Predicción: 999

Como números: error pequeño.
Como categorías: puede ser una clase incorrecta.

Por eso, no conviene convertir automáticamente valores numéricos en clases solo para simplificar. La simplificación puede facilitar la explicación, pero también puede empobrecer el análisis.

Otra situación problemática aparece cuando las categorías resultantes quedan desbalanceadas. Si dividimos precios, ventas o consumos en rangos, puede ocurrir que casi todos los casos queden en una categoría y muy pocos en las demás. Entonces volvemos a enfrentar problemas de clasificación desbalanceada, aunque el fenómeno original era una cantidad continua.

Por ejemplo:

demanda baja: 8500 casos
demanda media: 1300 casos
demanda alta: 200 casos

En este caso, el modelo podría aprender a predecir casi siempre “demanda baja” y obtener una accuracy aparentemente razonable, pero ser poco útil para detectar demanda alta. El problema de evaluación se vuelve más complejo por una decisión de formulación.

También debemos tener cuidado cuando las categorías no tienen un significado claro para quienes usarán el modelo. Si decimos “precio medio”, “riesgo moderado” o “consumo elevado”, debemos definir qué significan esos términos y por qué esos límites son relevantes. Si los rangos no se relacionan con decisiones reales, la clasificación puede ser confusa.

Una buena regla es formular el problema según la decisión que queremos apoyar. Si la decisión requiere una cantidad, usemos regresión. Si la decisión requiere asignar una categoría, usemos clasificación.

Si necesitamos estimar cuánto: **regresión**.

Si necesitamos decidir a qué grupo pertenece: **clasificación**.

Por ejemplo, si una empresa de logística necesita informar un tiempo estimado de llegada, la regresión parece natural. Si solo necesita decidir si un pedido llegará “a tiempo” o “tarde”, la clasificación puede ser adecuada. Son preguntas distintas.



Ambas formulaciones pueden ser válidas, pero no responden lo mismo. Incluso podrían coexistir en un mismo proyecto. Primero podríamos estimar el tiempo de entrega y luego, a partir de ese valor, decidir si supera un límite aceptable. Pero en ese caso debemos ser conscientes de que estamos transformando una estimación numérica en una regla de decisión.

También hay situaciones donde tratar un problema como clasificación puede ser conveniente. Si el valor exacto es difícil de predecir y la decisión solo necesita rangos amplios, clasificar puede ser suficiente. Por ejemplo, para priorizar zonas de demanda “alta”, “media” o “baja”, tal vez no se necesite una estimación exacta. Pero esa elección debe estar justificada por el uso del resultado, no por comodidad.

El peligro aparece cuando se convierte una variable numérica en categorías solo porque parece más sencillo evaluar accuracy, construir una matriz de confusión o explicar etiquetas. Esa comodidad puede producir un modelo menos útil. Elegir el

tipo de problema no debería depender de la métrica que conocemos mejor, sino de la pregunta real que queremos responder.

Un problema no debería tratarse como clasificación cuando la magnitud del valor objetivo es importante, cuando los límites entre categorías serían arbitrarios, cuando se perdería información relevante o cuando la decisión necesita una estimación numérica. La clasificación puede ser útil si las categorías tienen sentido práctico, pero no debe usarse para reemplazar automáticamente una regresión. Antes de elegir el enfoque, debemos preguntarnos si necesitamos saber “qué clase es” o “cuánto vale”.

Predecir cantidades, no categorías

En este capítulo presentamos los problemas de regresión. A diferencia de la clasificación, donde el modelo debe elegir una categoría, en regresión el objetivo es estimar un valor numérico. Esto cambia la forma de plantear el problema, interpretar las predicciones y evaluar los resultados.

La diferencia central está en la variable objetivo. Si y representa una clase, estamos ante un problema de clasificación. Si y representa una cantidad con significado numérico real, estamos ante un problema de regresión.

Clasificación:

$y = \text{categoría}$

Regresión:

$y = \text{cantidad numérica}$

Una variable objetivo de regresión puede ser continua, como precio, temperatura o consumo, o puede ser un conteo, como unidades vendidas o cantidad de pedidos, siempre que el objetivo sea estimar una magnitud. Lo importante no es si el número tiene decimales, sino si los valores representan cantidades comparables y si las diferencias entre ellos tienen sentido.

También vimos que no toda columna numérica debe tratarse como regresión. Un identificador, un código postal o una categoría codificada con números no representan cantidades. En esos casos, el número es solo una forma de nombrar o clasificar, no una magnitud que el modelo deba aproximar.

En regresión, el error de predicción es la diferencia entre el valor real y el valor estimado. No solemos preguntar si el modelo acertó exactamente, porque en muchos problemas eso sería poco realista. En cambio, preguntamos cuánto se equivocó, en qué dirección se equivocó y si ese error es aceptable para el contexto.

Valor real:	150000
Valor predicho:	147000
Error:	3000

El error debe interpretarse en las unidades del problema. Un error de 10 puede ser pequeño en una predicción de precios, pero enorme en una predicción de temperatura. Por eso, una métrica de regresión nunca debe leerse sin conocer la escala de la variable objetivo y el uso que tendrá la predicción.

Los ejemplos reales muestran la variedad de situaciones donde aparece la regresión. Podemos predecir precios de viviendas, ventas futuras, demanda de servicios, temperaturas, consumo eléctrico, tiempos de entrega, ingresos, costos o cualquier otra cantidad. En todos los casos, el modelo intenta aprender una relación entre variables de entrada y un valor numérico objetivo.

También analizamos cuándo no conviene transformar un problema numérico en clasificación. Convertir precios en “bajo”, “medio” y “alto”, o demanda en “baja”, “media” y “alta”, puede ser útil si esas categorías responden a una decisión concreta. Pero si necesitamos la magnitud del valor, esa transformación elimina información y puede crear fronteras arbitrarias.

- Si necesitamos saber cuánto: conviene pensar en **regresión**.
- Si necesitamos asignar un grupo: puede tener sentido pensar en **clasificación**.

La elección entre clasificación y regresión no debería depender de la comodidad ni de la métrica que conocemos mejor. Debe depender de la pregunta real que queremos responder. No es lo mismo estimar cuántas unidades se venderán que decidir si la demanda será alta. No es lo mismo estimar el precio de una vivienda que clasificarla como cara o barata.

La regresión permite predecir cantidades. Sus predicciones se evalúan midiendo distancias entre valores reales y valores estimados. Para trabajar correctamente con regresión, debemos asegurarnos de que la variable objetivo represente una magnitud, interpretar los errores en unidades reales y formular el problema según la decisión que queremos apoyar. En el próximo capítulo estudiaremos uno de los modelos más conocidos para este tipo de tareas: la regresión lineal.

Capítulo 15 · Regresión lineal

La idea de ajustar una recta o un plano

La regresión lineal es uno de los modelos más conocidos para predecir valores numéricos. Su idea central es sencilla: buscar una relación aproximadamente lineal entre las variables de entrada y la variable objetivo.

Si trabajamos con una sola variable de entrada, el modelo intenta ajustar una **recta**. Por ejemplo, podríamos usar la superficie de una vivienda para estimar su precio. La intuición sería que, en general, viviendas con mayor superficie tienden a tener mayor precio.

`superficie -> precio`

Si representamos cada vivienda como un punto en un gráfico, la superficie puede ubicarse en el eje horizontal y el precio en el eje vertical. Cada punto corresponde a una vivienda observada. La regresión lineal busca una recta que pase lo más cerca posible de esos puntos.

No se espera que la recta atraviese todos los puntos. En datos reales eso casi nunca ocurre. Dos viviendas con la misma superficie pueden tener precios distintos por ubicación, antigüedad, estado, orientación, servicios cercanos u otros factores. La recta no pretende explicar cada caso a la perfección, sino capturar una tendencia general. La regresión lineal no busca acertar todos los puntos. Busca una tendencia que reduzca los errores de predicción.

Una recta puede interpretarse como una regla de predicción. Si conocemos la superficie de una vivienda, usamos la recta para estimar su precio. Cuanto más adecuada sea la recta para representar la relación entre superficie y precio, mejores serán las predicciones.

La forma general de una regresión lineal simple puede escribirse así:

$$y = a \cdot x + b$$

Donde `x` es la variable de entrada, `y` es el valor que queremos predecir, `a` representa la pendiente de la recta y `b` representa el intercepto. Veremos estos términos con más detalle en la próxima sección. Por ahora, alcanza con entender que la recta transforma un valor de entrada en una predicción numérica.

Por ejemplo, un modelo podría aprender una relación como esta:

$$p_estimado = 1100 \cdot superficie + 15000$$

Según esta regla, una vivienda de 80 metros cuadrados tendría un precio estimado de:

$$\begin{aligned} p_estimado &= 1100 \cdot 80 + 15000 \\ p_estimado &= 103000 \end{aligned}$$

Este valor no debe interpretarse como una verdad exacta. Es una estimación basada en la relación aprendida por el modelo. Algunas viviendas de 80 metros cuadrados costarán más y otras menos. La regresión lineal ofrece una predicción promedio esperada según las variables incluidas.

Cuando usamos más de una variable de entrada, la idea se mantiene, pero ya no hablamos de una recta en un plano de dos dimensiones. Si usamos dos variables, como superficie y antigüedad, el modelo puede representarse como un **plano** en un espacio tridimensional.

$$superficie + antigüedad \rightarrow precio$$

La ecuación podría tener una forma como esta:

$$precio_estimado = a \cdot superficie + antigüedad + b$$

Aquí el modelo combina dos variables para estimar el precio. La superficie puede aumentar la predicción y la antigüedad podría aumentarla o reducirla, según lo que los datos muestren.

Si usamos más de dos variables, la representación geométrica se vuelve más difícil de visualizar. Ya no podemos dibujar fácilmente el modelo como una recta o un plano en una hoja. Pero la idea matemática sigue siendo la misma: el modelo calcula una combinación lineal de las variables de entrada.

```
predicción =  
coeficiente_1 · variable_1 +  
coeficiente_2 · variable_2 +  
coeficiente_3 · variable_3 +  
...  
b
```

Aunque no podamos visualizarlo, seguimos hablando de regresión lineal. En dimensiones más altas, la generalización de una recta o un plano se llama hiperplano. No necesitamos profundizar en ese término, pero conviene saber que el modelo conserva la misma lógica: combinar variables mediante coeficientes para producir una predicción numérica.

La palabra “lineal” no significa que el problema sea simple ni que las predicciones sean siempre buenas. Significa que el modelo asume una forma particular de relación: cada variable aporta de manera proporcional y acumulativa a la predicción. Si la superficie aumenta, el modelo suma o resta una cantidad fija por cada unidad adicional, según el coeficiente aprendido. Si la antigüedad cambia, ocurre algo similar.

Esta suposición puede funcionar razonablemente bien en muchos problemas, especialmente como primer modelo. Pero también puede ser limitada. En datos reales, las relaciones pueden ser curvas, tener interacciones o cambiar según el rango de valores. Por ejemplo, sumar 10 metros cuadrados puede no tener el mismo impacto en una vivienda pequeña que en una vivienda muy grande. La regresión lineal básica no captura automáticamente ese tipo de comportamiento.

Aun así, la regresión lineal es muy valiosa porque es fácil de entender, rápida de entrenar y ofrece resultados interpretables. Nos permite ver cómo se relaciona cada variable con la predicción, al menos dentro de las limitaciones del modelo. Por eso suele ser un buen punto de partida para problemas de regresión.

También es útil como referencia. Antes de usar modelos más complejos, conviene saber cómo funciona un modelo simple. Si una regresión lineal obtiene resultados razonables, quizás no haga falta comenzar con técnicas más difíciles de interpretar. Si funciona mal, sus errores pueden ayudarnos a entender si necesitamos más variables, mejores transformaciones o modelos más flexibles.

En `scikit-learn`, la regresión lineal se implementa con `LinearRegression`:

```
from sklearn.linear_model import LinearRegression  
  
modelo = LinearRegression()
```

```
modelo.fit(X_train, y_train)
y_pred = modelo.predict(X_test)
```

La estructura es la misma que ya vimos en otros modelos: crear el modelo, entrenarlo con `fit` y obtener predicciones con `predict`. Lo que cambia es el tipo de problema: ahora `y` contiene valores numéricos, no clases.

La regresión lineal también se integra con `Pipeline` y con transformaciones previas. Si las variables de entrada incluyen categorías, valores faltantes o escalas diferentes, seguimos necesitando preparar los datos correctamente. El hecho de usar un modelo lineal no elimina la importancia del preprocesamiento.

La regresión lineal intenta ajustar una recta cuando hay una sola variable de entrada, un plano cuando hay dos, o una combinación lineal en dimensiones mayores cuando hay varias variables. Su objetivo es capturar una tendencia general que permita predecir valores numéricos. Es un modelo simple, interpretable y útil como punto de partida, aunque sus resultados dependen de qué tan razonable sea asumir una relación lineal entre las variables y la variable objetivo.

Pendiente e intercepto

En una regresión lineal simple, el modelo intenta representar la relación entre una variable de entrada y una variable objetivo mediante una recta. Esa recta suele escribirse de esta forma:

$$y = a \cdot x + b$$

En esta expresión, `x` es la variable de entrada, `y` es el valor que queremos predecir, `a` es la **pendiente** y `b` es el **intercepto**.

La pendiente indica cuánto cambia la predicción cuando la variable de entrada aumenta en una unidad. El intercepto indica cuál sería la predicción cuando la variable de entrada vale cero.

Supongamos que queremos predecir el precio de una vivienda usando solo su superficie. Un modelo podría aprender esta relación:

$$p_estimado = 1100 \cdot superficie + 15000$$

En este ejemplo, la pendiente es `1100` y el intercepto es `15000`.

La pendiente nos dice que, según el modelo, por cada metro cuadrado adicional, el precio estimado aumenta en 1100 unidades monetarias.

Pendiente = 1100

Interpretación: por cada 1 m² adicional, el precio estimado aumenta en 1100.

Si comparamos una vivienda de 80 m² con una de 81 m², el modelo aumentará la predicción en 1100. Si comparamos una de 80 m² con una de 90 m², la diferencia estimada será de 11000.

10 m² adicionales

→ $10 \cdot 1100$

→ 11000 unidades más en la predicción

La pendiente representa entonces la inclinación de la recta. Si la pendiente es positiva, la recta sube: a medida que aumenta x , aumenta la predicción. Si la pendiente es negativa, la recta baja: a medida que aumenta x , disminuye la predicción. Si la pendiente es cercana a cero, la variable tiene poca influencia lineal sobre la predicción.

Por ejemplo, si estamos prediciendo precio de viviendas, es razonable esperar que la superficie tenga una pendiente positiva. A mayor superficie, mayor precio estimado, al menos como tendencia general.

Pendiente positiva:

x aumenta -> la predicción aumenta

En cambio, si usamos la antigüedad de una vivienda, podríamos encontrar una pendiente negativa, según el mercado y el dataset. Una mayor antigüedad podría asociarse con un precio menor, si todo lo demás se mantiene igual.

Pendiente negativa:

x aumenta -> la predicción disminuye

Pero debemos ser cuidadosos con la interpretación. La pendiente describe la relación aprendida por el modelo en los datos disponibles. No demuestra por sí sola una relación causal. Si la pendiente de una variable es positiva, significa que el modelo encontró una asociación positiva con la variable objetivo, no necesariamente que esa variable sea la causa directa del cambio.

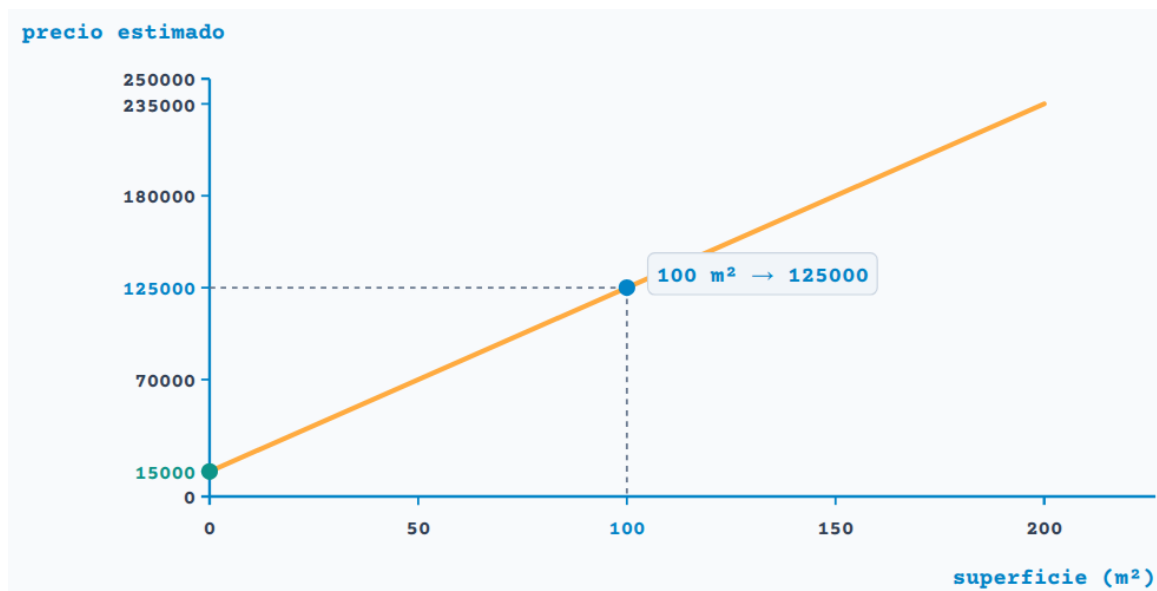
El intercepto, por su parte, es el valor que predice el modelo cuando x vale cero. En la ecuación:

$$p_{\text{estimado}} = 1100 \cdot \text{superficie} + 15000$$

si la superficie fuera cero, la predicción sería:

$$\begin{aligned} p_{\text{estimado}} &= 1100 \cdot 0 + 15000 \\ p_{\text{estimado}} &= 15000 \end{aligned}$$

Por eso, el intercepto es 15000.



Gráfica de la recta.

En algunos problemas, el intercepto tiene una interpretación directa. En otros, no tanto. Una vivienda con superficie cero no tiene sentido como caso real, por lo que no deberíamos interpretar literalmente ese valor como el precio de una vivienda de 0 m². En este contexto, el intercepto es necesario para ubicar la recta, pero no representa necesariamente una situación realista.

El intercepto ayuda a ubicar la recta. No siempre tiene una interpretación práctica directa.

Esto ocurre muchas veces en Machine Learning. Los parámetros de un modelo pueden tener una interpretación matemática clara, pero su interpretación en el mundo real depende del contexto. Si $x = 0$ está fuera del rango razonable de los datos, el intercepto puede ser poco significativo como valor aislado.

Por ejemplo, si entrenamos un modelo con viviendas entre 40 m² y 300 m², preguntar qué predice para 0 m² es extrapolar muy lejos de los datos observados. La regresión lineal puede calcularlo, pero eso no significa que sea una predicción útil.

Esta idea también se aplica a predicciones fuera del rango de entrenamiento. Si el modelo aprendió a partir de viviendas de tamaño habitual, puede no ser confiable para estimar el precio de una propiedad extremadamente grande o extremadamente pequeña. La recta se extiende matemáticamente, pero el fenómeno real puede no seguir la misma relación en todos los rangos.

La recta puede extenderse más allá de los datos. Pero las predicciones fuera del rango observado deben interpretarse con cuidado.

Cuando tenemos más de una variable de entrada, el modelo ya no tiene una sola pendiente. Tiene un coeficiente para cada variable. Cada coeficiente cumple un papel parecido a la pendiente: indica cómo cambia la predicción cuando esa variable aumenta en una unidad, manteniendo constantes las demás variables.

Por ejemplo:

```
precio_estimado =  
1200 · superficie  
- 500 · antigüedad  
+ 8000 · ambientes  
+ 10000
```

En esta ecuación, el modelo usa tres variables: superficie, antigüedad y ambientes. Cada una tiene su propio coeficiente.

```
superficie -> 1200  
ambientes -> 8000  
antigüedad -> -500  
intercepto -> 10000
```

La interpretación sería:

Por cada 1 m² adicional, el precio estimado aumenta en 1200, manteniendo constantes las demás variables.

Por cada año adicional de antigüedad, el precio estimado disminuye en 500, manteniendo constantes las demás variables.

Por cada ambiente adicional, el precio estimado aumenta en 8000, manteniendo constantes las demás variables.

La frase “manteniendo constantes las demás variables” es muy importante. En regresión lineal múltiple, un coeficiente intenta expresar la contribución de una variable cuando las otras variables no cambian. En la práctica, esa interpretación puede complicarse si las variables están muy relacionadas entre sí.

Por ejemplo, superficie y cantidad de ambientes suelen estar correlacionadas. Las viviendas más grandes tienden a tener más ambientes. En ese caso, separar con precisión cuánto aporta cada variable puede ser difícil. El modelo puede seguir siendo útil para predecir, pero la interpretación individual de los coeficientes requiere cautela.

También debemos considerar la escala de las variables. Un coeficiente depende de la unidad en que se mide la variable. Si medimos superficie en metros cuadrados, la pendiente expresa el cambio por cada metro cuadrado. Si la medimos en decenas de metros cuadrados, el coeficiente cambiará.

La interpretación de un coeficiente depende de la unidad de medida de la variable.

Por eso, no conviene comparar coeficientes directamente si las variables están en escalas muy distintas. Un coeficiente grande no siempre significa que una variable sea más importante que otra. Puede deberse simplemente a la unidad de medida.

En `scikit-learn`, después de entrenar un modelo `LinearRegression`, podemos acceder a los coeficientes y al intercepto:

```
modelo.coef_
```

```
modelo.intercept_
```

En una regresión lineal simple, `modelo.coef_` tendrá un único valor. En una regresión lineal múltiple, tendrá un coeficiente por cada variable de entrada.

Si usamos un DataFrame de Pandas para construir X , podemos asociar cada coeficiente con el nombre de su columna. Esto ayuda a interpretar el modelo, aunque siempre debemos hacerlo con las precauciones mencionadas.

```
coeficientes = pd.Series(modelo.coef_, index=X.columns)
```

Este tipo de análisis puede ser muy útil para entender cómo el modelo está usando las variables. Pero no debemos confundir interpretabilidad con verdad absoluta. Los coeficientes dependen de los datos, de las variables incluidas, de las transformaciones realizadas y de la presencia de relaciones entre variables.

La pendiente indica cómo cambia la predicción cuando cambia una variable de entrada. El intercepto indica el valor de la predicción cuando las variables valen cero, aunque no siempre tenga una interpretación práctica directa. En regresión lineal múltiple, cada variable tiene su propio coeficiente, que puede interpretarse como una pendiente parcial. Estos valores hacen que la regresión lineal sea un modelo relativamente interpretable, siempre que recordemos sus límites y el contexto de los datos.

Regresión lineal simple

La regresión lineal simple es el caso más básico de regresión lineal. Se llama “simple” porque usa una sola variable de entrada para predecir una variable objetivo numérica.

La estructura del problema es:

```
variable de entrada -> variable objetivo
```

Por ejemplo:

```
superficie -> precio
horas de estudio -> puntaje
temperatura -> consumo eléctrico
inversión publicitaria -> ventas
```

En todos estos casos, el modelo intenta encontrar una relación lineal entre una variable x y una variable objetivo y .

La ecuación general es:

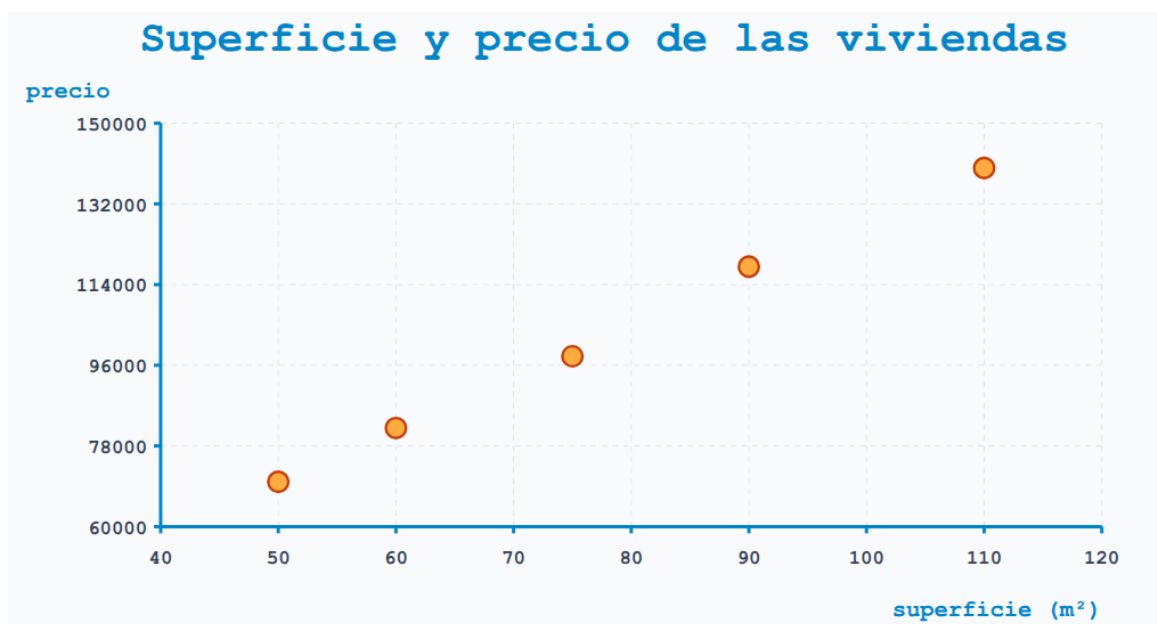
$$y = a \cdot x + b$$

Donde x es la variable de entrada, y es la predicción, a es la pendiente y b es el intercepto.

Supongamos que tenemos un pequeño conjunto de datos sobre viviendas. Para cada vivienda conocemos su superficie y su precio. La pregunta es si podemos usar la superficie para estimar el precio.

superficie_m2	precio
50	70000
60	82000
75	98000
90	118000
110	140000

Si representamos estos datos en un gráfico, cada vivienda aparece como un punto:



Superficie y precio de las viviendas.

La superficie se ubica en el eje horizontal y el precio en el eje vertical. Si las viviendas más grandes tienden a ser más caras, los puntos mostrarán una tendencia ascendente.

La regresión lineal simple busca una recta que represente esa tendencia.

- **Puntos observados:** viviendas reales del dataset.
- **Recta ajustada:** tendencia aprendida por el modelo.

La recta no tiene que pasar exactamente por todos los puntos. En datos reales, eso sería poco probable. El objetivo es encontrar una recta que produzca errores razonablemente pequeños en conjunto.

Para cada vivienda, el modelo puede calcular un precio estimado. La diferencia entre el precio real y el precio estimado es el error de predicción para ese caso.

$$\text{valor real} - \text{precio estimado} = \text{error}$$

El entrenamiento de la regresión lineal consiste en elegir la pendiente y el intercepto de manera que la recta se ajuste lo mejor posible a los datos. Dicho de otra manera, el modelo busca los valores de **a** y **b** que reducen los errores de predicción.

No necesitamos estudiar todavía el procedimiento matemático completo, pero sí conviene entender la idea: el modelo prueba una recta posible, observa los errores, y el método de ajuste encuentra la recta que minimiza una medida global de esos errores.

Entrenar una regresión lineal simple significa: elegir la recta que mejor resume la relación entre **x** y **y**.

Por ejemplo, el modelo podría aprender una ecuación como esta:

$$\text{precio estimado} = 1150 \cdot \text{superficie} + 12000$$

Con esta recta, una vivienda de 80 m² tendría una predicción de:

$$\begin{aligned}\text{precio estimado} &= 1150 \cdot 80 + 12000 \\ \text{precio estimado} &= 104000\end{aligned}$$

El modelo no está diciendo que todas las viviendas de 80 m² cuesten exactamente 104000. Está diciendo que, según la tendencia observada en los datos, ese es el valor estimado para una vivienda con esa superficie, considerando solo esa variable.

Esta última aclaración es clave. En regresión lineal simple, el modelo solo ve una variable de entrada. Si usamos superficie para predecir precio, el modelo ignora ubicación, estado, antigüedad, orientación, servicios, barrio y cualquier otro factor que no hayamos incluido.

Regresión lineal simple: predice usando una sola variable. Si hay factores importantes fuera de esa variable, el modelo no puede considerarlos.

Por eso, la regresión lineal simple es fácil de interpretar, pero puede ser limitada. Puede servir para estudiar una relación básica o como primer acercamiento, pero muchos problemas reales requieren más variables.

Aun así, tiene un gran valor pedagógico. Permite visualizar qué significa ajustar un modelo, entender la idea de error, interpretar pendiente e intercepto y observar cómo una tendencia puede convertirse en una regla de predicción.

También permite detectar si una relación lineal simple parece razonable. Si los puntos del gráfico se distribuyen aproximadamente alrededor de una recta, la regresión lineal simple puede ser una buena primera aproximación. Si los puntos forman una curva, grupos separados o una nube sin tendencia clara, una recta puede ser insuficiente.

Por ejemplo, si al aumentar la superficie el precio aumenta de manera bastante regular, una recta puede capturar parte de la relación. Pero si el precio crece muy rápido para superficies grandes, o si hay barrios con precios completamente diferentes para la misma superficie, la recta puede quedarse corta.

Una recta funciona mejor cuando la relación entre x e y es aproximadamente lineal.

Esto no significa que los puntos deban estar perfectamente alineados. La variabilidad es normal. Lo que buscamos es una tendencia lineal general.

Una forma sencilla de evaluar visualmente una regresión lineal simple es graficar los puntos reales y la recta ajustada. Si la recta pasa por el centro de la nube de puntos y los errores parecen razonables, el modelo puede estar capturando una relación útil. Si la recta queda claramente mal ubicada o los errores muestran un patrón fuerte, quizás el modelo no sea adecuado.

En un cuaderno Colab práctico, como el que acompaña a este capítulo, este tipo de gráfico será muy útil. Podremos ver los datos, entrenar el modelo y dibujar la recta de predicción. Esa visualización ayuda a conectar la ecuación con la intuición.

En `scikit-learn`, una regresión lineal simple se entrena igual que cualquier otro modelo supervisado. La diferencia es que X debe contener una sola variable de entrada.

```
from sklearn.linear_model import LinearRegression

X = df[["superficie"]]
y = df["precio"]

modelo = LinearRegression()
modelo.fit(X, y)
```

Es importante notar que `X` se escribe como `DataFrame`, no como `Serie`, por eso usamos doble corchete:

```
X = df[["superficie"]]
```

Aunque haya una sola variable, `scikit-learn` espera una matriz de entrada con filas y columnas. Cada fila representa un caso; cada columna, una variable predictora.

Luego podemos predecir nuevos valores:

```
modelo.predict([[80]])
```

Esta predicción representa el precio estimado para una vivienda de 80 m², según la recta aprendida.

Después de entrenar el modelo, también podemos consultar la pendiente y el intercepto:

```
modelo.coef_
modelo.intercept_
```

En regresión lineal simple, `modelo.coef_` contiene un único coeficiente: la pendiente de la recta. `modelo.intercept_` contiene el intercepto.

Si el coeficiente es positivo, la relación aprendida es ascendente. Si es negativo, la relación es descendente. Si está cerca de cero, la variable de entrada aporta poca variación lineal a la predicción.

La regresión lineal simple también puede mostrar claramente el problema de extrapolar. Si entrenamos el modelo con viviendas entre 40 y 150 m², una predicción para 80 m² está dentro del rango observado. Pero una predicción para 500 m² puede ser poco confiable. La recta puede calcular un valor, pero el modelo no tiene evidencia suficiente sobre ese rango.

- Predecir dentro del rango observado: más razonable.
- Predecir lejos del rango observado: más riesgoso.

Esto es importante porque una recta se extiende indefinidamente, pero los fenómenos reales no siempre mantienen la misma relación en todos los rangos. La relación entre superficie y precio puede cambiar en propiedades muy grandes, zonas exclusivas o casos especiales.

La regresión lineal simple usa una sola variable de entrada para predecir una variable objetivo numérica. Ajusta una recta que intenta representar la tendencia general entre ambas variables. Es un modelo fácil de visualizar e interpretar, útil para comprender la lógica de la regresión, aunque limitado cuando el fenómeno depende de muchos factores o cuando la relación no es aproximadamente lineal.

Regresión lineal múltiple

La regresión lineal simple usa una sola variable de entrada para predecir una variable objetivo numérica. La regresión lineal múltiple extiende esa idea: usa varias variables de entrada al mismo tiempo.

En lugar de intentar predecir el precio de una vivienda solo con su superficie, podríamos usar también la cantidad de ambientes, la antigüedad, la ubicación, el estado general o la distancia a ciertos servicios. El modelo combina toda esa información para producir una predicción.

Regresión lineal simple:
una variable de entrada -> una predicción

Regresión lineal múltiple:
varias variables de entrada -> una predicción

Por ejemplo, si queremos predecir el precio de una vivienda, podríamos tener:

Variables de entrada:
superficie, ambientes, antigüedad

Variable objetivo:
precio

La ecuación podría escribirse de esta manera:


```
precio_estimado =  
    a · superficie  
    + b · ambientes  
    + c · antigüedad  
    + d
```

En esta expresión, cada variable tiene su propio coeficiente. El modelo aprende cuánto aporta cada variable a la predicción, dentro de la forma lineal que está usando.

Un ejemplo concreto podría ser:

```
precio_estimado =  
    1200 · superficie  
    + 9000 · ambientes  
    - 500 · antigüedad  
    + 15000
```

Según esta ecuación, la superficie y la cantidad de ambientes aumentan el precio estimado, mientras que la antigüedad lo reduce. El último valor, `15000`, es el intercepto.

Si queremos estimar el precio de una vivienda de 80 m², 3 ambientes y 20 años de antigüedad, el modelo calcularía:

```
precio_estimado =  
    1200 · 80  
    + 9000 · 3  
    - 500 · 20  
    + 15000
```

```
precio_estimado =  
    96000  
    + 27000  
    - 10000  
    + 15000
```

```
precio_estimado = 128000
```

La predicción surge de sumar los aportes de todas las variables. Esta es la idea central de la regresión lineal múltiple: cada variable contribuye con una parte, y el modelo combina esas partes para estimar el valor final.

```
Predicción =  
    aporte de variable 1  
+ aporte de variable 2  
+ aporte de variable 3  
    ...  
+ intercepto
```

La ventaja de usar varias variables es que muchos fenómenos reales no dependen de un único factor. El precio de una vivienda no depende solo de la superficie. Las ventas no dependen solo del precio. El consumo eléctrico no depende solo de la temperatura. Al incluir más variables relevantes, el modelo puede capturar una parte más rica del problema.

Por ejemplo, para predecir ventas podríamos usar:

```
precio del producto  
descuento aplicado  
día de la semana  
mes  
inversión publicitaria  
ventas anteriores
```

Para predecir consumo eléctrico:

```
temperatura exterior  
día de la semana  
hora  
cantidad de personas  
tipo de actividad  
consumo histórico
```

Para predecir tiempo de entrega:

```
distancia  
tráfico estimado  
tipo de vehículo  
horario  
cantidad de pedidos activos  
zona de entrega
```

En todos los casos, la regresión lineal múltiple intenta aprender una relación lineal entre varias variables de entrada y una cantidad objetivo.

Sin embargo, agregar variables no garantiza mejorar el modelo. Una variable puede aportar información útil, pero también puede introducir ruido, errores o relaciones engañosas. Si agregamos columnas irrelevantes, mal medidas o no disponibles al momento de predicción, el modelo puede empeorar o volverse poco confiable.

Más variables no siempre significa mejor modelo. Importa que las variables sean relevantes, válidas y disponibles.

Esta idea conecta con lo trabajado en capítulos anteriores. Antes de elegir variables, debemos preguntarnos si estarían disponibles cuando el modelo se use en casos nuevos, si contienen información legítima, si podrían producir fuga de datos y si tienen sentido para el problema.

En regresión lineal múltiple, además, la interpretación de los coeficientes requiere más cuidado que en la regresión simple. Cada coeficiente representa el cambio estimado en la predicción cuando esa variable aumenta en una unidad, manteniendo constantes las demás variables.

Por ejemplo, en la ecuación:

```
precio_estimado =  
    1200 · superficie  
    + 9000 · ambientes  
    - 500 · antigüedad  
    + 15000
```

el coeficiente de `superficie` es `1200`. Podemos interpretarlo así:

Si la superficie aumenta en 1 m², y las demás variables se mantienen constantes, el precio estimado aumenta en 1200.

El coeficiente de `antigüedad` es `-500`:

Si la antigüedad aumenta en 1 año, y las demás variables se mantienen constantes, el precio estimado disminuye en 500.

La expresión “manteniendo constantes las demás variables” es fundamental. El modelo intenta aislar el aporte de cada variable dentro del conjunto de variables incluidas. Pero en datos reales, las variables pueden estar relacionadas entre sí. Por ejemplo, viviendas con más ambientes suelen tener mayor superficie. En ese caso, interpretar por separado el efecto de cada variable puede ser más difícil.

Esta relación entre variables predictoras se conoce como colinealidad o multicolinealidad cuando es fuerte. No necesitamos profundizar todavía en el concepto, pero sí conviene saber que puede afectar la interpretación de los

coeficientes. El modelo puede seguir prediciendo razonablemente, pero no siempre será sencillo decir cuánto “aporta” cada variable individualmente.

Otra diferencia importante con la regresión simple es que la regresión múltiple no siempre puede visualizarse fácilmente. Con una sola variable de entrada, podemos dibujar una recta en un gráfico. Con dos variables, podríamos imaginar un plano. Con tres o más variables, ya no podemos representar el modelo completo en una visualización simple.

Aun así, la lógica del modelo no cambia. Sigue siendo una combinación lineal de variables.

Aunque no podamos dibujarlo fácilmente, el modelo sigue sumando aportes lineales de las variables de entrada.

En `scikit-learn`, entrenar una regresión lineal múltiple es muy similar a entrenar una regresión lineal simple. La diferencia está en que `X` contiene varias columnas.

```
from sklearn.linear_model import LinearRegression

X = df[["superficie", "ambientes", "antiguedad"]]
y = df["precio"]

modelo = LinearRegression()
modelo.fit(X, y)
```

Luego podemos obtener predicciones:

```
y_pred = modelo.predict(X_test)
```

Y también consultar los coeficientes:

```
modelo.coef_
modelo.intercept_
```

Si `X` es un `DataFrame`, resulta útil asociar cada coeficiente con el nombre de la columna correspondiente:

```
coeficientes = pd.Series(modelo.coef_, index=X.columns)
```

Esto puede producir una salida conceptual como:

```
superficie -> 1200  
ambientes -> 9000  
antigüedad -> -500
```

Esta tabla ayuda a interpretar el modelo, pero no debe leerse de manera automática. Los coeficientes dependen de las unidades de medida, de las variables incluidas, de las transformaciones realizadas y de las relaciones entre variables.

Por ejemplo, un coeficiente de 1200 para superficie está expresado por metro cuadrado. Si la superficie se midiera en decenas de metros cuadrados, el coeficiente cambiaría. Si agregamos o quitamos variables relacionadas, los coeficientes también pueden cambiar.

Los coeficientes no son propiedades absolutas del mundo. Son valores aprendidos por el modelo según los datos y las variables utilizadas.

También debemos recordar que la regresión lineal múltiple sigue siendo lineal. Aunque use muchas variables, el modelo básico no captura automáticamente curvas, interacciones complejas o cambios de comportamiento según rangos. Si el efecto de una variable depende de otra, el modelo no lo representará por sí solo, salvo que construyamos variables adicionales o usemos modelos más flexibles.

Por ejemplo, el efecto de la superficie sobre el precio puede depender del barrio. Diez metros cuadrados adicionales pueden tener un impacto distinto en una zona que en otra. Una regresión lineal básica no captura esa interacción si no la incorporamos explícitamente.

Esto no vuelve inútil al modelo. Al contrario, la regresión lineal múltiple puede ser muy potente como primer enfoque. Permite combinar varias fuentes de información, producir predicciones numéricas e interpretar parcialmente el aporte de las variables. Pero debemos conocer sus límites.

La regresión lineal múltiple usa varias variables de entrada para predecir una variable objetivo numérica. El modelo aprende un coeficiente para cada variable y combina esos aportes con un intercepto para generar la predicción. Es más flexible que la regresión lineal simple porque puede incorporar más información, pero su interpretación requiere cuidado y sigue dependiendo de la suposición de una relación lineal entre las variables y el objetivo.

Coeficientes

En regresión lineal, los coeficientes son los valores que el modelo aprende para combinar las variables de entrada y producir una predicción. Cada variable predictora tiene asociado un coeficiente. Ese coeficiente indica cómo participa esa variable en el cálculo del valor estimado.

En una regresión lineal simple, hay una sola variable de entrada y, por lo tanto, un solo coeficiente. En una regresión lineal múltiple, hay varias variables de entrada y el modelo aprende un coeficiente para cada una. Cada coeficiente responde:

¿Cómo cambia la predicción cuando esta variable aumenta en una unidad, manteniendo las demás constantes?

Supongamos un modelo para estimar el precio de una vivienda:

```
precio_estimado =  
    1200 · superficie  
    + 9000 · ambientes  
    - 500 · antigüedad  
    + 15000
```

En esta ecuación, los coeficientes son:

```
superficie -> 1200  
ambientes -> 9000  
antigüedad -> -500
```

El intercepto es 15000.

Cada coeficiente acompaña a una variable. Para calcular la predicción, el modelo multiplica cada valor de entrada por su coeficiente y luego suma todos esos aportes.

Por ejemplo, para una vivienda de 80 m², 3 ambientes y 20 años de antigüedad:

```
precio_estimado =  
    1200 · 80  
    + 9000 · 3  
    - 500 · 20  
    + 15000
```

El aporte de la superficie es:

$$1200 \cdot 80 = 96000$$

El aporte de los ambientes es:

$$9000 \cdot 3 = 27000$$

El aporte de la antigüedad es:

$$-500 \cdot 20 = -10000$$

Luego se suma el intercepto:

```
precio_estimado =  
96000 + 27000 - 10000 + 15000  
  
precio_estimado = 128000
```

Esta forma de cálculo muestra que los coeficientes definen el peso lineal de cada variable dentro del modelo. Algunas variables empujan la predicción hacia arriba; otras pueden empujarla hacia abajo.

Un coeficiente positivo significa que, según el modelo, al aumentar esa variable también aumenta la predicción, manteniendo constantes las demás variables.

- **Coeficiente positivo:** la variable aumenta → la predicción aumenta.

Un coeficiente negativo significa que, según el modelo, al aumentar esa variable disminuye la predicción, manteniendo constantes las demás variables.

- **Coeficiente negativo:** la variable aumenta → la predicción disminuye.

Un coeficiente cercano a cero indica que, dentro de ese modelo y con esos datos, la variable tiene poca relación lineal con la predicción, al menos después de considerar las demás variables incluidas.

Pero esta interpretación debe hacerse con cuidado. Un coeficiente no es una verdad absoluta sobre el mundo. Es un valor aprendido a partir de un conjunto de datos, con ciertas variables, ciertas transformaciones y ciertas limitaciones.

Por ejemplo, si el coeficiente de `superficie` es `1200`, no significa necesariamente que cada metro cuadrado adicional “cause” exactamente 1200 unidades más de

precio. Significa que, en el modelo entrenado, esa es la asociación lineal estimada entre superficie y precio, manteniendo constantes las demás variables del modelo.

- **Coefficiente:** asociación aprendida por el modelo. No necesariamente "causa-efecto" real.

Esta diferencia entre asociación y causalidad es muy importante. Un modelo puede encontrar que una variable está relacionada con el precio, las ventas o el consumo, pero eso no significa que modificar esa variable produzca directamente el cambio estimado. Para hablar de causalidad se necesita otro tipo de análisis y más información sobre el proceso que genera los datos.

También debemos considerar la escala de las variables. Los coeficientes dependen de las unidades de medida. Si `superficie` está medida en metros cuadrados, el coeficiente indica el cambio estimado por cada metro cuadrado adicional. Si la misma variable se mide en decenas de metros cuadrados, el coeficiente tendrá otro valor.

Misma información, distinta unidad: puede cambiar el coeficiente numérico.

Por ejemplo, supongamos que el modelo usa superficie en metros cuadrados:

```
precio_estimado = 1200 · superficie_m2 + ...
```

Si en lugar de medir superficie en metros cuadrados usamos decenas de metros cuadrados, una vivienda de 80 m² se representaría como 8. Para producir predicciones equivalentes, el coeficiente debería cambiar:

```
precio_estimado = 1200 · superficie_en_decenas + ...
```

El fenómeno no cambió. Cambió la unidad. Por eso, no conviene comparar coeficientes de variables medidas en escalas distintas como si fueran directamente equivalentes.

Un coeficiente de `9000` para ambientes puede parecer más grande que un coeficiente de `1200` para superficie, pero eso no significa automáticamente que `ambientes` sea "más importante". La superficie cambia de a muchos metros cuadrados; ambientes cambia en valores mucho más pequeños. La interpretación depende de la escala y del rango de cada variable.

Coefficiente grande no siempre significa variable más importante.

Cuando las variables se escalan, por ejemplo con `StandardScaler`, los coeficientes pueden ser más comparables porque las variables pasan a estar expresadas en una escala común. Sin embargo, incluso en ese caso, la interpretación debe hacerse con cuidado y teniendo en cuenta el contexto.

Otro punto importante es la relación entre variables predictoras. Si dos variables contienen información parecida, sus coeficientes pueden volverse inestables o difíciles de interpretar. En viviendas, superficie y cantidad de ambientes suelen estar relacionadas. En ventas, precio y descuento pueden estar vinculados. En consumo eléctrico, temperatura y estación del año pueden moverse juntas.

Cuando esto ocurre, el modelo puede repartir la explicación entre variables relacionadas de maneras que dependen del dataset. Si agregamos o quitamos una variable, los coeficientes de otras pueden cambiar.

Variables muy relacionadas entre sí pueden dificultar la interpretación de los coeficientes individuales.

Esto no significa necesariamente que el modelo no sirva para predecir. Puede ocurrir que sus predicciones sean razonables, aunque la interpretación exacta de cada coeficiente sea delicada. La utilidad predictiva y la interpretabilidad no siempre son lo mismo.

Los coeficientes también dependen de las variables incluidas en el modelo. Supongamos que primero entrenamos un modelo de precios usando solo superficie. El coeficiente de superficie puede absorber parte de la información que, en realidad, está relacionada con otras variables no incluidas, como ubicación o cantidad de ambientes. Si luego agregamos esas variables, el coeficiente de superficie puede cambiar.

Esto ocurre porque cada coeficiente se interpreta dentro del conjunto completo de variables del modelo. No existe aislado.

Un coeficiente siempre debe interpretarse en relación con las demás variables incluidas.

También debemos tener cuidado con variables categóricas codificadas mediante One-Hot Encoding. Si una variable como `barrio` se transforma en varias columnas binarias, cada columna tendrá su propio coeficiente. Esos coeficientes se interpretan respecto de una categoría de referencia o respecto de cómo quedó construida la codificación.

Por ejemplo, si tenemos columnas como:

```
barrio_Centro  
barrio_Norte  
barrio_Sur
```

cada coeficiente indica cómo se ajusta la predicción cuando esa columna vale 1, en comparación con el esquema de referencia usado por el modelo. Esta interpretación puede ser útil, pero requiere conocer cómo se codificaron las categorías.

En `scikit-learn`, después de entrenar una regresión lineal, podemos consultar los coeficientes con:

```
modelo.coef_
```

Y el intercepto con:

```
modelo.intercept_
```

Si `X` es un `DataFrame` con nombres de columnas, podemos crear una serie para ver cada coeficiente junto a su variable:

```
coeficientes = pd.Series(modelo.coef_, index=X.columns)  
coeficientes
```

Esto puede ayudar a revisar el modelo. Por ejemplo, podemos observar qué variables tienen coeficientes positivos, cuáles negativos y cuáles cercanos a cero. Pero esa lectura debe acompañarse con conocimiento del problema, revisión de escalas, análisis de correlaciones y evaluación de errores.

Los coeficientes son una de las razones por las que la regresión lineal se considera un modelo interpretable. A diferencia de modelos más complejos, permite ver explícitamente cómo cada variable participa en la predicción. Pero esa interpretabilidad no debe confundirse con simplicidad absoluta. Interpretar bien un coeficiente requiere entender los datos.

Los coeficientes son los valores que la regresión lineal aprende para combinar las variables de entrada. Indican la dirección y magnitud de la asociación lineal entre cada variable y la predicción, dentro del modelo entrenado. Un coeficiente positivo aumenta la predicción; uno negativo la reduce. Pero su interpretación depende de la escala, de las variables incluidas, de las relaciones entre ellas y del contexto. Los coeficientes hacen que la regresión lineal sea transparente, pero no eliminan la necesidad de análisis cuidadoso.

Supuestos básicos

La regresión lineal es un modelo simple e interpretable, pero esa simplicidad se apoya en algunas ideas sobre la relación entre las variables de entrada y la variable objetivo. Estas ideas suelen llamarse **supuestos**. No hace falta estudiarlos todavía con todo el detalle estadístico, pero sí conviene conocerlos para entender cuándo una regresión lineal puede funcionar razonablemente y cuándo puede quedarse corta.

El primer supuesto es que la relación entre las variables de entrada y la variable objetivo es aproximadamente lineal. Esto significa que el modelo espera que los cambios en las variables se traduzcan en cambios proporcionales en la predicción.

Por ejemplo, si usamos superficie para predecir precio, una regresión lineal simple supone que cada metro cuadrado adicional suma una cantidad aproximadamente constante al precio estimado. Si el coeficiente de superficie es 1200:

```
1m2 adicional -> 1200 más en la predicción
10m2 adicionales -> 12000 más en la predicción
```

Esta idea puede ser una buena aproximación en algunos rangos, pero no siempre. En muchos fenómenos reales, las relaciones no son perfectamente lineales. Puede haber curvas, saturaciones, efectos que cambian según el rango o interacciones entre variables.

Por ejemplo, en ventas, bajar el precio de un producto puede aumentar la demanda hasta cierto punto, pero no indefinidamente. En consumo eléctrico, la temperatura puede afectar el consumo de manera distinta en días templados que en días extremadamente fríos o calurosos. En viviendas, sumar metros cuadrados puede tener distinto impacto según la ubicación o el tipo de propiedad.

La regresión lineal funciona mejor cuando una recta o una combinación lineal representa razonablemente la tendencia.

El segundo supuesto importante es que los errores no deberían mostrar un patrón claro. Después de entrenar un modelo, podemos comparar los valores reales con los valores predichos. La diferencia entre ambos es el error o residuo.

$$\text{residuo} = \text{valor real} - \text{valor predicho}$$

Si el modelo lineal es adecuado, esos residuos deberían distribuirse de manera relativamente desordenada alrededor de cero. No esperamos errores perfectos, pero sí que no aparezca una forma sistemática.

Por ejemplo, si el modelo subestima siempre las viviendas caras y sobreestima siempre las viviendas baratas, hay un patrón. Eso puede indicar que la relación no está bien capturada por una recta o que faltan variables importantes.

- **Errores sin patrón claro:** señal razonable.
- **Errores con patrón fuerte:** posible problema del modelo o de los datos.

Este análisis de residuos es una herramienta muy útil en regresión. No alcanza con mirar una métrica promedio. Dos modelos pueden tener errores promedio parecidos, pero uno puede equivocarse de manera más sistemática en ciertos rangos. Revisar los residuos ayuda a detectar esos problemas.

El tercer supuesto se relaciona con la variabilidad de los errores. En muchos enfoques clásicos de regresión lineal se espera que los errores tengan una dispersión más o menos constante a lo largo de los valores predichos. Dicho de manera más simple: el modelo no debería tener errores pequeños para un rango y errores enormes para otro, al menos no de forma sistemática.

Supongamos un modelo de precios de viviendas. Si predice bastante bien propiedades de precio medio, pero comete errores cada vez más grandes a medida que el precio aumenta, puede haber un problema. Quizás las viviendas caras tienen dinámicas diferentes, faltan variables relevantes o la relación deja de ser lineal en ese rango.

Errores que crecen mucho en ciertos rangos pueden indicar que el modelo no representa bien todas las zonas del problema.

El cuarto supuesto es que las observaciones deberían ser razonablemente independientes entre sí. Esto significa que cada fila del dataset debería aportar información propia, no ser una repetición disfrazada o una medición fuertemente dependiente de otra fila.

En algunos datasets, esta idea puede no cumplirse. Por ejemplo, si tenemos mediciones horarias de consumo eléctrico, el consumo de una hora puede estar muy relacionado con el consumo de la hora anterior. Si tenemos ventas diarias, las ventas de un día pueden depender de las ventas del día previo, de la temporada o de una promoción en curso. En esos casos, el orden temporal importa.

Cuando los datos tienen dependencia temporal, una separación aleatoria puede ser engañosa.

No profundizaremos todavía en series temporales, pero conviene recordar que muchos modelos simples asumen que las filas pueden tratarse como ejemplos relativamente independientes. Cuando esa suposición no es razonable, debemos tener más cuidado con la forma de entrenar y evaluar.

El quinto supuesto práctico es que no debería haber valores extremos que dominen el ajuste sin ser comprendidos. La regresión lineal puede ser sensible a outliers. Unos pocos casos muy alejados pueden inclinar la recta o modificar los coeficientes de manera importante.

Por ejemplo, en un dataset de viviendas, una propiedad extremadamente cara puede afectar el modelo si se aleja mucho del resto. Tal vez sea un caso real y relevante; tal vez sea un error de carga; tal vez pertenezca a un segmento distinto del mercado. Lo importante es revisarlo.

Un valor extremo no siempre debe eliminarse. Pero sí debe entenderse.

Eliminar outliers sin criterio puede ser tan problemático como ignorarlos. Si representan casos reales importantes, quitarlos puede empobrecer el modelo. Si son errores de datos, mantenerlos puede distorsionar el entrenamiento. La decisión depende del contexto.

El sexto punto se relaciona con la colinealidad entre variables. En regresión lineal múltiple, si dos o más variables predictoras están muy relacionadas entre sí, la interpretación de los coeficientes puede volverse inestable. Esto no siempre destruye la capacidad predictiva del modelo, pero puede dificultar la lectura de “cuánto aporta” cada variable.

Por ejemplo, en viviendas, superficie y cantidad de ambientes suelen estar relacionadas. En ventas, precio final y porcentaje de descuento pueden estar conectados. En consumo eléctrico, temperatura y estación del año pueden moverse

juntas. Si dos variables cuentan una historia parecida, el modelo puede repartir el peso entre ellas de una forma difícil de interpretar.

Variables predictoras muy relacionadas pueden complicar la interpretación de los coeficientes.

Este punto es especialmente importante cuando usamos la regresión lineal no solo para predecir, sino también para explicar. Si el objetivo principal es interpretación, debemos revisar con más cuidado las relaciones entre variables.

También conviene mencionar una idea sobre normalidad de los errores. En cursos de estadística, la regresión lineal suele estudiarse junto con supuestos sobre la distribución de los residuos. Para Machine Learning aplicado, especialmente cuando el objetivo principal es predecir, no siempre empezamos por ese nivel de detalle. Pero sí nos importa que los errores no estén completamente desordenados por un problema de formulación, que no haya patrones evidentes y que las métricas de evaluación sean honestas sobre datos no vistos.

En Machine Learning nos interesa especialmente que el modelo generalice bien y que los errores sean aceptables en datos no vistos.

Por eso, los supuestos no deben verse como una lista rígida que invalida automáticamente el modelo. Son señales para interpretar sus resultados. Si una relación no es perfectamente lineal, la regresión lineal puede seguir siendo útil como aproximación. Si hay outliers, quizá podamos revisarlos o usar métricas menos sensibles. Si hay variables relacionadas, tal vez el modelo prediga bien aunque sus coeficientes sean difíciles de interpretar.

Lo importante es no usar la regresión lineal como una caja mágica. Su simplicidad permite entenderla, pero también exige reconocer sus límites.

En un flujo de trabajo práctico, podemos revisar estos supuestos con preguntas como:

- ¿La relación entre variables parece aproximadamente lineal?
- ¿Los errores muestran algún patrón claro?
- ¿El modelo falla especialmente en valores altos o bajos?
- ¿Hay valores extremos que influyen demasiado?
- ¿Las variables predictoras están muy relacionadas entre sí?
- ¿La evaluación respeta la forma en que se usarán los datos?

Estas preguntas no requieren una respuesta perfecta antes de entrenar el primer modelo. Muchas veces se entrena una regresión lineal como punto de partida y luego se analizan sus errores. Si el modelo funciona razonablemente, puede ser una solución suficiente o una referencia útil. Si falla de manera sistemática, esos fallos orientan los próximos pasos.

La regresión lineal supone que la relación entre variables y objetivo puede aproximarse mediante una combinación lineal. También requiere prestar atención a los errores, los valores extremos, la independencia de las observaciones y las relaciones entre variables predictoras. Estos supuestos no deben memorizarse como reglas abstractas, sino usarse como criterios para interpretar cuándo el modelo es adecuado, cuándo sus coeficientes son confiables y cuándo conviene buscar otro enfoque.

Ventajas y limitaciones

La regresión lineal es uno de los modelos más importantes para comenzar a estudiar regresión. No porque sea siempre el mejor modelo posible, sino porque combina tres cualidades muy valiosas: es simple, es rápida y es interpretable.

Una de sus principales ventajas es que resulta fácil de entender. La idea de ajustar una recta, un plano o una combinación lineal permite conectar el modelo con una intuición clara. Cada variable aporta una parte a la predicción y ese aporte está representado por un coeficiente.

predicción = suma de aportes de las
variables + **intercepto**

Esta estructura hace que el modelo sea más transparente que otros métodos más complejos. Podemos revisar los coeficientes, observar si una variable empuja la predicción hacia arriba o hacia abajo y analizar si esas relaciones tienen sentido en el contexto del problema.

Por ejemplo, en un modelo de precios de viviendas, esperaríamos que la superficie tenga un coeficiente positivo. Si el modelo aprende que más superficie se asocia con menor precio, tal vez haya una explicación en los datos, pero también podría ser una señal de que algo merece revisión: variables faltantes, datos mezclados, errores de carga o relaciones confusas entre columnas. Los coeficientes pueden ayudar a detectar relaciones esperadas o sospechosas.

Otra ventaja es que la regresión lineal suele ser un buen modelo de referencia. Antes de usar modelos más complejos, conviene entrenar un modelo simple y evaluar qué

tan lejos llega. Si una regresión lineal logra errores aceptables, quizá no sea necesario complicar el enfoque. Si funciona mal, sus errores pueden mostrar qué tipo de dificultad tiene el problema.

En este sentido, la regresión lineal sirve como línea de base. Nos permite preguntarnos si un modelo más avanzado realmente mejora lo suficiente como para justificar su mayor complejidad. Un modelo más complejo debería mejorar a un modelo simple de manera clara y útil.

También es un modelo rápido de entrenar y de usar para predecir. En muchos datasets tabulares, una regresión lineal puede ajustarse en muy poco tiempo. Esto facilita la experimentación inicial, la comparación de variables y la creación de primeras versiones del flujo de trabajo.

La regresión lineal también se integra muy bien con las herramientas que venimos usando. Puede incluirse dentro de un `Pipeline`, combinarse con transformaciones de datos y evaluarse con métricas de regresión. Su uso en `scikit-learn` mantiene la misma estructura que ya conocemos:

```
from sklearn.linear_model import LinearRegression

modelo = LinearRegression()
modelo.fit(X_train, y_train)
y_pred = modelo.predict(X_test)
```

Otra ventaja importante es que puede trabajar con varias variables a la vez. Aunque su forma sea lineal, no está limitada a una sola columna. Podemos incluir variables numéricas, variables categóricas codificadas, fechas transformadas, indicadores y otras características construidas durante la preparación de datos.

Sin embargo, la regresión lineal también tiene limitaciones claras. La más importante es que asume una relación lineal entre las variables de entrada y la variable objetivo. Si la relación real es curva, cambia según el rango o depende de interacciones complejas, una regresión lineal básica puede no capturarla bien.

Por ejemplo, el efecto del precio sobre las ventas puede no ser constante. Bajar un precio un poco puede aumentar la demanda, pero a partir de cierto punto el efecto puede estabilizarse. La relación entre temperatura y consumo eléctrico también puede ser no lineal: el consumo puede aumentar tanto en días muy fríos como en días muy calurosos, pero ser menor en temperaturas templadas.

Si la relación tiene forma curva o cambia por rangos, una recta puede ser insuficiente.

Otra limitación es que la regresión lineal básica no captura interacciones automáticamente. Una interacción aparece cuando el efecto de una variable depende del valor de otra. Por ejemplo, la superficie puede influir en el precio de una vivienda de manera distinta según el barrio. Una promoción puede impactar las ventas de forma distinta según el tipo de producto. La temperatura puede afectar el consumo de manera diferente según el horario o el tipo de edificio.

La regresión lineal puede incorporar interacciones si las construimos explícitamente como nuevas variables, pero no las descubre por sí sola en su forma básica.

- **Regresión lineal básica:** suma efectos individuales.
- **Interacciones:** requieren variables adicionales o modelos más flexibles.

También puede ser sensible a valores extremos. Algunos outliers pueden modificar la recta o los coeficientes de manera importante, especialmente si están muy alejados del resto de los datos. Esto no significa que debamos eliminar todos los valores extremos, pero sí que debemos revisarlos.

En un dataset de precios de viviendas, unas pocas propiedades extremadamente caras pueden influir mucho en el ajuste. En ventas, un día excepcional por una promoción puede alejarse del comportamiento habitual. En consumo, un evento inusual puede producir valores muy altos. Si esos casos son reales, quizá deban mantenerse; si son errores, pueden distorsionar el modelo.

La regresión lineal puede verse afectada por casos muy alejados del patrón general.

Otra limitación aparece cuando las variables predictoras están muy relacionadas entre sí. Como vimos, esto puede complicar la interpretación de los coeficientes. Si dos columnas contienen información similar, el modelo puede repartir el peso entre ellas de una forma poco estable. Al cambiar ligeramente los datos o agregar una nueva variable, los coeficientes pueden modificarse bastante.

Esto es especialmente importante si queremos usar el modelo para explicar relaciones. Para predecir, el modelo puede funcionar de manera aceptable. Para interpretar el impacto individual de cada variable, debemos ser más cautelosos.

La regresión lineal también puede producir predicciones poco razonables fuera del rango de los datos. Como la recta o el hiperplano se extienden indefinidamente, el modelo puede calcular valores para casos muy diferentes de los observados. Pero esos valores no siempre tienen sentido.

Por ejemplo, si entrenamos un modelo con viviendas de 40 a 200 m², una predicción para 1000 m² puede ser poco confiable. Si entrenamos con temperaturas habituales, una predicción bajo condiciones extremas puede fallar. El modelo extrapola linealmente, aunque el fenómeno real no necesariamente continúe igual.

La regresión lineal puede extrapolar, pero extrapolar no siempre es confiable.

Además, la regresión lineal puede producir valores imposibles si el problema tiene límites naturales. Por ejemplo, puede predecir ventas negativas, consumo negativo o tiempos negativos si las variables de entrada llevan la ecuación a esos valores. Matemáticamente, el modelo puede calcularlos; en el mundo real, no tienen sentido. Algunos problemas tienen límites naturales:

```
ventas ≥ 0  
consumo ≥ 0  
tiempo ≥ 0
```

La regresión lineal básica no impone esos límites automáticamente. Esto no significa que la regresión lineal no pueda usarse en esos problemas, pero sí que debemos revisar sus predicciones y evaluar si los errores o valores extremos resultan aceptables.

Otra limitación es que un modelo lineal puede quedarse corto cuando hay patrones complejos. Modelos como árboles, Random Forest, Gradient Boosting u otros enfoques pueden capturar relaciones no lineales e interacciones de manera más flexible. Esos modelos aparecerán más adelante en el libro. Por ahora, alcanza con reconocer que la regresión lineal es un punto de partida, no el límite de lo posible.

Aun así, no conviene descartar la regresión lineal demasiado rápido. En muchos problemas, un modelo simple puede ser suficiente. Si los datos tienen una relación aproximadamente lineal, si la interpretación es importante y si el desempeño es aceptable, la regresión lineal puede ser una excelente opción.

También puede ser útil aunque no sea el modelo final. Sus coeficientes, errores y predicciones ayudan a comprender el problema. Puede mostrar qué variables parecen relevantes, dónde aparecen errores grandes y si hay señales suficientes para predecir la variable objetivo.

La regresión lineal puede servir como:

- modelo final simple,
- modelo de referencia,

- herramienta de interpretación,
- primer diagnóstico del problema.

La elección dependerá del objetivo. Si necesitamos un modelo interpretable, rápido y fácil de comunicar, la regresión lineal tiene ventajas claras. Si necesitamos capturar patrones complejos, relaciones curvas o interacciones difíciles, probablemente necesitemos modelos más flexibles.

La regresión lineal es simple, interpretable, rápida y útil como punto de partida para problemas de regresión. Permite analizar coeficientes y construir una primera referencia de desempeño. Sus principales limitaciones aparecen cuando la relación no es lineal, hay interacciones complejas, valores extremos, variables muy relacionadas, extrapolaciones riesgosas o límites naturales que el modelo no respeta. Usarla bien implica aprovechar su claridad sin olvidar sus supuestos.

Una primera herramienta para predecir valores

La regresión lineal nos permitió dar el primer paso dentro de los modelos de regresión. Su objetivo es predecir valores numéricos a partir de una relación lineal entre las variables de entrada y la variable objetivo.

Cuando usamos una sola variable predictora, la regresión lineal ajusta una recta. Cuando usamos dos variables, podemos imaginar un plano. Cuando usamos más variables, el modelo sigue la misma lógica, aunque ya no podamos visualizarlo fácilmente: combina los valores de entrada mediante coeficientes y suma un intercepto para producir una predicción.

```
predicción =  
coeficiente_1 · variable_1 +  
coeficiente_2 · variable_2 +  
coeficiente_3 · variable_3 +  
...  
b
```

Esta estructura hace que la regresión lineal sea relativamente fácil de interpretar. Los coeficientes nos permiten analizar si una variable aumenta o reduce la predicción, siempre dentro de las limitaciones del modelo y de los datos utilizados. Por eso, la regresión lineal suele ser una herramienta valiosa cuando necesitamos no solo predecir, sino también comprender de manera inicial cómo se relacionan las variables.

También vimos que los coeficientes deben interpretarse con cuidado. Dependen de la escala de las variables, de las columnas incluidas en el modelo, de las

transformaciones realizadas y de las relaciones entre variables predictoras. Un coeficiente positivo o negativo muestra una asociación aprendida por el modelo, pero no demuestra causalidad por sí solo.

La regresión lineal se apoya en una idea fuerte: que la relación entre las variables y el objetivo puede aproximarse mediante una combinación lineal. Esto puede funcionar bien en muchos casos, pero no en todos. Si el fenómeno tiene curvas, interacciones complejas, valores extremos influyentes o patrones que cambian según el rango de valores, el modelo puede quedarse corto.

Por eso, la regresión lineal debe evaluarse con datos no vistos y con métricas adecuadas para regresión. No alcanza con que la ecuación parezca razonable o con que los coeficientes sean fáciles de explicar. Necesitamos medir los errores, revisar si son aceptables y observar si el modelo falla de manera sistemática en ciertos tipos de casos.

Aun con sus limitaciones, la regresión lineal es un excelente punto de partida. Es simple, rápida, interpretable y permite construir una primera referencia de desempeño. Si funciona bien, puede ser suficiente para algunos problemas. Si no funciona bien, sus errores nos ayudan a entender qué necesita mejorar: más variables, mejores transformaciones, otro tipo de modelo o una formulación distinta del problema.

La regresión lineal es una primera herramienta para predecir valores numéricos. Su fuerza está en la claridad: muestra cómo una predicción puede construirse como suma de aportes de distintas variables. Su límite está en la misma simplicidad: no siempre puede representar la complejidad de los datos reales. Aprender a usarla y evaluarla correctamente nos prepara para comparar modelos de regresión con más criterio.

Capítulo 16 · Métricas para regresión

Error absoluto

En los problemas de regresión, el modelo predice valores numéricos. Por eso, evaluar una predicción no consiste simplemente en decir si fue correcta o incorrecta. Lo importante es medir qué tan lejos quedó el valor predicho del valor real.

Supongamos que un modelo predice el precio de una vivienda:

```
precio real      = 150000
precio predicho  = 147000

diferencia = 150000 - 147000
           = 3000
```

El modelo se equivocó por 3000 unidades monetarias. En este caso, la predicción fue menor que el valor real. Pero si lo que nos interesa es medir el tamaño del error, no necesitamos conservar el signo. Solo necesitamos saber cuán grande fue la diferencia.

Para eso usamos el **error absoluto**.

```
precio real      = 150000
precio predicho  = 147000

diferencia = 150000 - 147000
           = 3000
```

Las barras verticales indican valor absoluto. Esto significa que el resultado siempre se toma como positivo, aunque la diferencia original haya sido negativa.

Por ejemplo:

```
precio real      = 150000
precio predicho  = 147000

diferencia = 150000 - 147000
           = 3000
```

Si el modelo hubiera predicho por encima del valor real, también obtendríamos un error absoluto positivo:

```
Valor real      = 150000
Valor predicho  = 158000

Error absoluto:
|150000 - 158000| = 8000
```

En el primer caso, el modelo subestimó el precio por 3000. En el segundo, lo sobreestimó por 8000. El error absoluto no se concentra en la dirección del error, sino en su magnitud.

- **Error con signo:** indica si el modelo sobreestimó o subestimó.
- **Error absoluto:** indica cuánto se equivocó, sin importar la dirección.

Esta distinción es útil porque, si sumáramos errores con signo, algunos errores positivos y negativos podrían compensarse entre sí. Un modelo que sobreestima en algunos casos y subestima en otros podría parecer mejor de lo que realmente es si solo miramos la suma de diferencias.

Imaginemos tres predicciones:

```
Caso 1: error = 100
Caso 2: error = -100
Caso 3: error = 20
```

Si sumamos los errores con signo:

```
100 + (-100) + 20 = 20
```

Podría parecer que el error total es pequeño. Pero eso oculta que hubo dos errores de magnitud 100 que se cancelaron entre sí. Si usamos errores absolutos, la situación se ve mejor:

```
|100| + |-100| + |20| = 220
```

El error absoluto evita que las equivocaciones en direcciones opuestas se anulen. Por eso es una base importante para evaluar regresión.

También tiene una ventaja práctica: se interpreta en las mismas unidades de la variable objetivo. Si predecimos precios, el error absoluto está en pesos, dólares u otra moneda. Si predecimos temperatura, está en grados. Si predecimos ventas, está en unidades vendidas. Si predecimos tiempos, está en minutos u horas.

Precio: error absoluto en unidades monetarias.
Temperatura: error absoluto en grados.
Ventas: error absoluto en unidades vendidas.

Esto hace que el error absoluto sea muy fácil de comunicar. Decir que una predicción se equivocó por 5 grados, 20 unidades o 3000 pesos es más claro que hablar de una medida abstracta sin unidades.

Sin embargo, el error absoluto se calcula para una predicción individual. Si queremos evaluar un modelo sobre muchos casos, necesitamos resumir los errores absolutos de todas las predicciones. Esa idea nos llevará a la métrica MAE, que veremos en la próxima sección.

Antes de avanzar, conviene mirar algunos ejemplos. Supongamos que tenemos un modelo que predice ventas diarias:

Día	Ventas reales	Ventas predichas
1	100	95
2	120	130
3	80	70

Los errores absolutos son:

Día 1: $|100 - 95| = 5$
Día 2: $|120 - 130| = 10$
Día 3: $|80 - 70| = 10$

El modelo se equivocó por 5 unidades el primer día, por 10 unidades el segundo y por 10 unidades el tercero. No importa aquí si predijo de más o de menos. Estamos midiendo la distancia entre cada valor real y cada valor predicho.

En un problema de temperatura:

Temperatura real: 24.0 °C
Temperatura predicha: 22.5 °C

Error absoluto:
 $|24.0 - 22.5| = 1.5$ °C

Este valor puede ser aceptable o no según el uso. Para una estimación general del clima, 1.5 °C puede ser razonable. Para una aplicación industrial muy sensible, quizá sea demasiado. La interpretación del error siempre depende del contexto.

En un problema de tiempos de entrega:

```
Tiempo real:      42 minutos  
Tiempo predicho: 35 minutos  
  
Error absoluto:  
|42 - 35| = 7 minutos
```

El modelo se equivocó por 7 minutos. Este valor puede ser útil para evaluar la calidad de la estimación, pero también debemos considerar si subestimar o sobreestimar tiene consecuencias distintas. El error absoluto no distingue esa dirección.

Ese es uno de sus límites. Al convertir todo en magnitudes positivas, perdemos información sobre si el modelo tiende a predecir por encima o por debajo. Para evaluar el tamaño de los errores, esto es adecuado. Para analizar sesgos sistemáticos, necesitamos mirar también los errores con signo o revisar gráficos de residuos.

El error absoluto mide tamaño del error. No muestra si el modelo se equivoca sistemáticamente hacia arriba o hacia abajo.

Por ejemplo, dos modelos pueden tener errores absolutos parecidos, pero comportamientos distintos. Uno puede sobreestimar casi siempre y otro puede alternar entre sobreestimaciones y subestimaciones. Según el problema, esa diferencia puede importar.

En inventario, sobreestimar ventas puede llevar a comprar demasiada mercadería. Subestimar ventas puede producir faltantes. En tiempos de entrega, prometer menos tiempo del real puede generar frustración. Prometer más tiempo del real puede ser conservador, pero quizás reduzca conversiones. El tamaño del error importa, pero la dirección también puede ser relevante.

Aun así, el error absoluto es una idea fundamental porque permite medir la distancia básica entre predicción y realidad. Muchas métricas de regresión parten de esta lógica: comparar valores reales con valores predichos y resumir esas diferencias.

En Python, si tenemos los valores reales y predichos, podemos calcular errores absolutos de forma directa:

```
errores_absolutos = abs(y_test - y_pred)
```

Cada elemento de `errores_absolutos` representa cuánto se equivocó el modelo en un caso del conjunto de prueba.

Esta operación es sencilla, pero conceptualmente muy importante. Nos recuerda que la evaluación de regresión no se basa en contar clases correctas, sino en medir distancias numéricas.

El error absoluto es la magnitud de la diferencia entre un valor real y un valor predicho. Ignora la dirección del error y evita que las sobreestimaciones y subestimaciones se cancelen entre sí. Se interpreta en las mismas unidades de la variable objetivo y permite entender cuánto se equivocó el modelo en cada caso. Para evaluar el desempeño general de un modelo, el próximo paso será promediar estos errores absolutos mediante MAE.

MAE

El **MAE** es una de las métricas más simples e intuitivas para evaluar modelos de regresión. Su nombre viene de **Mean Absolute Error**, que en español significa **error absoluto medio**.

La idea es directa: calculamos el error absoluto de cada predicción y luego promediamos esos errores.

MAE = promedio de los errores absolutos

Es decir, si para cada caso medimos cuánto se equivocó el modelo, sin importar si predijo por encima o por debajo, el MAE nos dice cuál fue el error promedio.

Podemos expresarlo de manera conceptual así:

1. Calculamos el error absoluto de cada predicción.
2. Sumamos todos esos errores absolutos.
3. Dividimos por la cantidad de casos.

Supongamos un modelo que predice ventas diarias:

Día	Ventas reales	Ventas predichas
1	100	95
2	120	130
3	80	70
4	90	100

Calculamos el error absoluto de cada día:

```
Día 1: |100 - 95| = 5
Día 2: |120 - 130| = 10
Día 3: |80 - 70| = 10
Día 3: |90 - 100| = 10
```

Luego promediamos:

```
MAE = (5 + 10 + 10 + 10) / 4
MAE = 35 / 4
MAE = 8.75
```

El MAE es 8.75 unidades. Esto significa que, en promedio, el modelo se equivocó por 8.75 unidades vendidas.

Esta interpretación es una de sus grandes ventajas. El MAE se expresa en las mismas unidades que la variable objetivo. Si predecimos ventas, el MAE está en unidades vendidas. Si predecimos precios, está en pesos, dólares u otra moneda. Si predecimos temperatura, está en grados. Si predecimos tiempo, está en minutos, horas o días.

```
MAE de 8.75 en ventas:
error promedio de 8.75 unidades.

MAE de 8.75 en temperatura:
error promedio de 8.75 grados.

MAE de 8.75 en precios:
error promedio de 8.75 unidades monetarias.
```

El número es el mismo, pero su significado depende completamente del problema. Por eso, al comunicar MAE, siempre debemos mencionar la variable objetivo y sus unidades.

Imaginemos un modelo de precios de viviendas con:

```
MAE = 5000 dólares
```

Esto indica que, en promedio, las predicciones se alejan del precio real en 5000 dólares. Para viviendas de 200000 dólares, ese error podría ser aceptable en una primera estimación. Para productos de 7000 dólares, sería enorme. La métrica no puede interpretarse sin contexto.

En un problema de temperatura:

```
MAE = 1.2 °C
```

Esto significa que, en promedio, el modelo se equivoca por 1.2 grados. La interpretación dependerá del uso. Puede ser razonable para una predicción general, pero no para una aplicación que requiere control muy preciso.

El MAE también es fácil de comparar entre modelos si todos fueron evaluados sobre el mismo problema y el mismo conjunto de datos. Si dos modelos predicen ventas diarias sobre el mismo conjunto de prueba, un MAE menor indica un error promedio menor.

```
Modelo A: MAE = 12.4 unidades
Modelo B: MAE = 8.1 unidades

En promedio, el Modelo B se equivoca menos.
```

Sin embargo, esta comparación solo es justa si ambos modelos se evaluaron sobre los mismos datos, con la misma variable objetivo y bajo la misma separación de entrenamiento y prueba. Comparar MAE entre problemas distintos no suele tener sentido.

MAE de precios de viviendas no se compara directamente con MAE de temperatura.

Una ventaja importante del MAE es que trata todos los errores de forma proporcional. Un error de 20 cuenta el doble que un error de 10. Un error de 100 cuenta diez veces más que un error de 10. Esto hace que la métrica sea bastante intuitiva.

```
Error de 20 -> pesa el doble que error de 10.
Error de 100 -> pesa diez veces más que error de 10.
```

Esta propiedad diferencia al MAE de otras métricas que penalizan con más fuerza los errores grandes, como MSE o RMSE. El MAE no exagera tanto el impacto de los errores extremos. Por eso puede ser una buena opción cuando queremos una medida robusta y fácil de interpretar del error típico.

Supongamos estos errores absolutos:

5, 6, 7, 8, 100

El error de 100 afecta el promedio, por supuesto, pero no se eleva al cuadrado ni se magnifica más allá de su tamaño original. En problemas donde existen algunos valores extremos, el MAE puede ofrecer una lectura más estable que métricas basadas en errores cuadrados.

Esto no significa que el MAE ignore errores grandes. Un error grande aumenta el promedio. Pero lo hace de manera lineal, no de manera cuadrática. MAE penaliza errores grandes, pero de forma proporcional.

El MAE también tiene límites. Al ser un promedio, puede ocultar diferencias entre casos. Un modelo con MAE de 10 puede equivocarse más o menos de manera uniforme, o puede acertar muchos casos y fallar muchísimo en algunos pocos. El mismo promedio puede surgir de comportamientos distintos.

Por ejemplo:

```
Modelo A, errores absolutos:  
10, 10, 10, 10  
  
MAE = 10
```

```
Modelo B, errores absolutos:  
0, 0, 0, 40  
  
MAE = 10
```

Ambos modelos tienen el mismo MAE, pero no se comportan igual. El Modelo A comete errores moderados en todos los casos. El Modelo B acierta perfectamente tres casos y falla mucho en uno. Según el problema, uno u otro puede ser más conveniente.

Esto muestra que el MAE resume el desempeño, pero no reemplaza el análisis de errores individuales. Siempre conviene revisar dónde se equivoca el modelo, si los errores se concentran en ciertos rangos y si existen casos especialmente problemáticos.

Otro límite del MAE es que no muestra la dirección del error. Como se basa en valores absolutos, no permite saber si el modelo tiende a sobreestimar o subestimar. Un modelo puede tener MAE aceptable y aun así estar sesgado hacia arriba o hacia abajo en ciertos grupos.

Por ejemplo, en predicción de demanda, un modelo podría tener un MAE razonable, pero subestimar sistemáticamente los días de alta demanda. Ese comportamiento puede generar problemas de stock aunque la métrica promedio parezca buena. MAE mide tamaño promedio del error. No indica si el modelo sobreestima o subestima.

Por eso, junto con MAE, puede ser útil revisar residuos con signo, gráficos de valores reales vs. predichos o errores por segmentos. La métrica nos da una síntesis; el análisis de errores nos da comprensión.

En `scikit-learn`, el MAE se calcula con `mean_absolute_error`:

```
from sklearn.metrics import mean_absolute_error

mae = mean_absolute_error(y_test, y_pred)
mae
```

Aquí `y_test` contiene los valores reales del conjunto de prueba y `y_pred` contiene las predicciones del modelo. El resultado será el error absoluto medio.

La interpretación debe escribirse en palabras. Por ejemplo:

- *El modelo tiene un MAE de 8.75 unidades. Esto significa que, en promedio, sus predicciones se alejan 8.75 unidades del valor real.*

Esta forma de comunicar es importante porque evita que la métrica quede como un número aislado. Un MAE siempre debe traducirse al problema.

También es útil comparar el MAE del modelo con una referencia simple. Por ejemplo, si predecir siempre el promedio de ventas produce un MAE de 18 unidades, y nuestro modelo obtiene un MAE de 8.75, entonces el modelo aporta una mejora clara. Si la diferencia fuera pequeña, habría que preguntarse si el modelo realmente está aprendiendo algo útil.

Esta comparación será importante cada vez que evaluemos modelos. Una métrica solo tiene sentido cuando podemos interpretarla en relación con el problema, con una referencia y con alternativas.

MAE es el promedio de los errores absolutos. Mide cuánto se equivoca el modelo, en promedio, en las mismas unidades de la variable objetivo. Es fácil de interpretar, útil para comparar modelos sobre el mismo problema y relativamente robusto frente a algunos errores extremos. Sin embargo, no muestra la dirección del error ni revela por sí solo cómo se distribuyen las equivocaciones. Por eso debe complementarse con análisis de errores y con otras métricas cuando el problema lo requiera.

MSE

El **MSE** es otra métrica muy usada para evaluar modelos de regresión. Su nombre viene de **Mean Squared Error**, que en español significa **error cuadrático medio**.

La idea es calcular el error de cada predicción, elevarlo al cuadrado y luego promediar esos valores.

MSE = promedio de los errores al cuadrado

Si el error de una predicción es:

error = **valor real** - **valor predicho**

entonces el error cuadrático es:

error cuadrático =
(**valor real** - **valor predicho**)²

El MSE calcula ese valor para todos los casos y luego obtiene el promedio.

Supongamos un modelo que predice ventas diarias:

Día	Ventas reales	Ventas predichas
1	100	95
2	120	130
3	80	70
4	90	100

Primero calculamos los errores:

Día 1: 100 - 95 = 5
Día 2: 120 - 130 = -10
Día 3: 80 - 70 = 10
Día 4: 90 - 100 = -10

Luego elevamos cada error al cuadrado:

Día 1: 5² = 25
Día 2: (-10)² = 100
Día 3: 10² = 100
Día 4: (-10)² = 100

Finalmente promediamos:

MSE = (25 + 100 + 100 + 100) / 4
MSE = 325 / 4
MSE = 81.25

El MSE de este modelo es 81.25.

A primera vista, esta métrica puede parecer menos intuitiva que el MAE. El MAE del mismo ejemplo era 8.75 unidades vendidas. En cambio, el MSE es 81.25, pero ya no está expresado en unidades vendidas, sino en unidades vendidas al cuadrado.

MAE: se interpreta en las unidades originales.

MSE: se interpreta en unidades al cuadrado.

Esta es una de las principales dificultades del MSE. Si predecimos ventas, el MSE queda expresado en “unidades vendidas al cuadrado”. Si predecimos precios, queda en “pesos al cuadrado” o “dólares al cuadrado”. Esa unidad no suele tener una interpretación directa en el problema real.

Entonces, ¿por qué se usa tanto el MSE?

La razón principal es que penaliza con más fuerza los errores grandes. Al elevar los errores al cuadrado, las equivocaciones grandes crecen mucho más que las pequeñas.

Por ejemplo:

```
Error = 2  → error cuadrático = 4
Error = 5  → error cuadrático = 25
Error = 10 → error cuadrático = 100
Error = 50 → error cuadrático = 2500
```

Un error de 50 no pesa simplemente cinco veces más que un error de 10. Al elevarlo al cuadrado, pesa veinticinco veces más. Esto hace que el MSE sea muy sensible a errores grandes. MSE penaliza especialmente las predicciones muy alejadas del valor real.

Esta característica puede ser una ventaja o una desventaja, según el problema.

Puede ser una ventaja cuando los errores grandes son especialmente graves. Por ejemplo, en predicción de demanda, equivocarse por 5 unidades puede ser aceptable, pero equivocarse por 500 puede generar una decisión muy mala. En predicción de consumo eléctrico, un error grande puede afectar la planificación de recursos. En predicción de tiempos críticos, una desviación grande puede ser mucho más problemática que varias pequeñas.

En esos casos, una métrica que castigue fuertemente los errores grandes puede ser útil. Si los errores grandes son mucho más costosos que los errores pequeños, MSE puede ser una métrica informativa. Pero también puede ser una desventaja cuando hay valores extremos o casos atípicos. Unos pocos errores muy grandes pueden

dominar el MSE y hacer que la métrica parezca muy mala, aunque el modelo funcione razonablemente bien para la mayoría de los casos.

Supongamos dos modelos con estos errores:

Modelo A: 5, 6, 7, 8, 9

Modelo B: 1, 2, 3, 4, 35

El Modelo B tiene errores muy pequeños en cuatro casos, pero un error grande en uno. El MSE castigará mucho ese error de 35, porque al elevarlo al cuadrado se convierte en 1225. Según el problema, eso puede ser correcto o puede exagerar demasiado el impacto de un caso particular.

Por eso, el MSE debe interpretarse con cuidado. No solo nos dice si el modelo se equivoca mucho. También refleja cuánto pesan los errores grandes dentro de la evaluación.

A diferencia del MAE, que penaliza los errores de forma proporcional, el MSE los penaliza de forma cuadrática.

MAE: error de 20 pesa el doble que error de 10.

MSE: error de 20 pesa cuatro veces más que error de 10.

Esto hace que MAE y MSE puedan preferir modelos distintos. Un modelo con errores moderados pero constantes puede tener un MAE parecido a otro que casi siempre acierta, pero falla mucho en algunos casos. El MSE favorecerá al modelo que evite errores extremos, si esos extremos son grandes.

El MSE también aparece con frecuencia en el entrenamiento de modelos. Muchos métodos de regresión lineal se ajustan minimizando una suma o promedio de errores cuadrados. Esto se debe a propiedades matemáticas convenientes: los errores cuadrados son suaves, diferenciables y permiten encontrar soluciones eficientes. No necesitamos profundizar en esos detalles, pero ayuda saber por qué esta métrica aparece tan seguido.

La regresión lineal clásica suele buscar la recta o el plano que minimiza errores cuadrados. Sin embargo, que una métrica sea conveniente matemáticamente no significa que siempre sea la mejor para comunicar resultados. En un informe, decir que el MSE es 81.25 puede ser menos claro que decir que el MAE es 8.75 unidades. Por eso, muchas veces se calcula MSE para comparación técnica, pero se complementa con métricas más interpretables.

En `scikit-learn`, el MSE se calcula con `mean_squared_error`:

```
from sklearn.metrics import mean_squared_error

mse = mean_squared_error(y_test, y_pred)
mse
```

Aquí `y_test` contiene los valores reales y `y_pred` contiene las predicciones del modelo.

La interpretación debe aclarar que se trata de errores al cuadrado. Por ejemplo:

El modelo tiene un MSE de 81.25. Esto resume el promedio de los errores cuadrados, por lo que penaliza especialmente las predicciones más alejadas del valor real.

No conviene comunicarlo como si fuera un error promedio en unidades originales. Para eso será más útil el RMSE, que estudiaremos en la siguiente sección. RMSE se obtiene a partir del MSE, pero vuelve a expresarse en las unidades de la variable objetivo.

También conviene comparar MSE entre modelos solo cuando fueron evaluados sobre el mismo problema y el mismo conjunto de datos. Como ocurre con MAE, no tiene sentido comparar directamente el MSE de un modelo de precios con el MSE de un modelo de temperatura.

Además, al comparar modelos con MSE, debemos recordar que la métrica puede estar muy influida por errores extremos. Si un modelo tiene peor MSE que otro, puede deberse a que comete errores grandes en algunos casos particulares. Eso puede ser una señal importante, pero también requiere revisar los datos y los errores individuales.

Por ejemplo, si un modelo de ventas tiene MSE alto por unos pocos días excepcionales de promoción, quizás el problema no sea el modelo en general, sino la falta de variables que indiquen promociones. Si un modelo de precios falla mucho en propiedades de lujo, tal vez necesite tratar ese segmento de manera distinta o incorporar variables adicionales.

Un MSE alto puede indicar errores grandes, outliers, segmentos mal modelados, variables faltantes, o casos especiales.

Por eso, MSE no debe leerse solo como un número final. Debe motivar preguntas sobre dónde se producen los errores más grandes y por qué.

MSE es el promedio de los errores elevados al cuadrado. Penaliza con fuerza las predicciones muy alejadas del valor real y por eso resulta útil cuando los errores grandes son especialmente importantes. Sin embargo, sus unidades al cuadrado dificultan la interpretación directa y su sensibilidad a outliers puede hacer que unos pocos casos dominen la métrica. Por eso suele complementarse con MAE, RMSE y análisis de errores individuales.

RMSE

El **RMSE** es otra métrica muy utilizada en problemas de regresión. Su nombre viene de **Root Mean Squared Error**, que en español puede traducirse como **raíz del error cuadrático medio**.

El RMSE está directamente relacionado con el MSE. Primero se calcula el promedio de los errores al cuadrado, y luego se toma la raíz cuadrada de ese valor.

$$\text{RMSE} = \text{raíz cuadrada del MSE}$$

Conceptualmente, el proceso es:

1. Calculamos el error de cada predicción.
2. Elevamos cada error al cuadrado.
3. Promediamos esos errores cuadrados.
4. Tomamos la raíz cuadrada del resultado.

La razón de tomar la raíz cuadrada es volver a una escala más interpretable. Recordemos que el MSE queda expresado en unidades al cuadrado. Si predecimos ventas, el MSE queda en unidades vendidas al cuadrado. Si predecimos precios, queda en pesos al cuadrado o dólares al cuadrado. Eso dificulta la comunicación.

El RMSE corrige parcialmente ese problema, porque vuelve a expresarse en las unidades originales de la variable objetivo.

$$\begin{aligned}\text{MSE} &= \text{unidades al cuadrado.} \\ \text{RMSE} &= \text{unidades originales.}\end{aligned}$$

Supongamos el mismo ejemplo de ventas diarias:

Día	Ventas reales	Ventas predichas
1	100	95
2	120	130
3	80	70
4	90	100

Ya habíamos calculado los errores:

Día 1: $100 - 95 = 5$
Día 2: $120 - 130 = -10$
Día 3: $80 - 70 = 10$
Día 4: $90 - 100 = -10$

Luego los errores al cuadrado:

Día 1: $5^2 = 25$
Día 2: $(-10)^2 = 100$
Día 3: $10^2 = 100$
Día 4: $(-10)^2 = 100$

El MSE era:

$MSE = (25 + 100 + 100 + 100) / 4$
 $MSE = 325 / 4$
 $MSE = 81.25$

Para obtener el RMSE, tomamos la raíz cuadrada:

$RMSE = \sqrt{81.25}$
 $RMSE \approx 9.01$

Entonces, el RMSE es aproximadamente 9.01 unidades vendidas. Esto significa que el error típico del modelo, interpretado en la escala original, está alrededor de 9 unidades, con una penalización mayor para errores grandes.

Esta última aclaración es importante. Aunque el RMSE vuelve a las unidades originales, no se interpreta exactamente igual que el MAE. El MAE era el promedio directo de los errores absolutos. El RMSE, en cambio, surge de elevar errores al cuadrado antes de promediarlos. Por eso, los errores grandes influyen más en el resultado.

MAE: promedia errores absolutos.
RMSE: penaliza más los errores grandes antes de volver a la escala original.

Si todos los errores son parecidos, MAE y RMSE pueden tener valores relativamente cercanos. Pero si hay algunos errores muy grandes, el RMSE suele crecer más que el MAE. Esa diferencia puede darnos una pista sobre la presencia de errores extremos.

Supongamos dos modelos:

Modelo A, errores absolutos: 8, 9, 10, 9, 8

Modelo B, errores absolutos: 2, 3, 4, 5, 30

El Modelo B tiene errores pequeños en varios casos, pero uno muy grande. El MAE puede mostrar un promedio razonable, pero el RMSE aumentará más por ese error de 30. Esto puede ser útil si queremos detectar modelos que ocasionalmente fallan mucho.

Cuando RMSE es bastante mayor que MAE, puede haber errores grandes influyendo en la evaluación. Esta comparación entre MAE y RMSE es muy útil en la práctica. No da una explicación completa, pero puede orientar el análisis. Si ambos valores son parecidos, los errores podrían ser relativamente uniformes. Si RMSE es mucho mayor, conviene revisar casos con errores grandes.

Por ejemplo, en un modelo de precios de viviendas:

MAE = 8000 dólares

RMSE = 22000 dólares

Esta diferencia sugiere que, aunque el error absoluto promedio sea de 8000 dólares, existen algunas predicciones con errores mucho más grandes que elevan el RMSE. Puede tratarse de propiedades atípicas, errores de datos, segmentos mal representados o variables faltantes.

En cambio:

MAE = 8000 dólares

RMSE = 9500 dólares

Aquí los errores parecen más consistentes. No significa que el modelo sea necesariamente bueno, pero no hay una señal tan fuerte de errores extremos dominando la métrica.

El RMSE puede ser una buena opción cuando los errores grandes son especialmente problemáticos. Si en un sistema de demanda un error pequeño no afecta demasiado,

pero un error enorme puede generar una decisión costosa, RMSE puede reflejar mejor esa preocupación que MAE.

En predicción de consumo energético, por ejemplo, errores grandes pueden causar problemas de planificación. En tiempos de entrega, una estimación extremadamente equivocada puede afectar mucho la experiencia del usuario. En precios, errores muy grandes pueden ser más graves que varios errores pequeños. RMSE es útil cuando queremos penalizar con más fuerza las grandes equivocaciones.

Sin embargo, esa misma sensibilidad puede ser una limitación. Si el dataset contiene outliers o casos muy particulares, el RMSE puede quedar dominado por ellos. Eso puede ser deseable si esos casos son importantes, o problemático si se trata de errores de carga o situaciones que no representan el comportamiento habitual.

Por eso, un RMSE alto no debe interpretarse automáticamente como “el modelo es malo”. Debe motivar preguntas:

- ¿Hay errores extremos?
- ¿Aparecen en ciertos segmentos?
- ¿Son casos reales o errores de datos?
- ¿Faltan variables para explicar esos casos?
- ¿El modelo falla en valores altos o bajos?

El RMSE no reemplaza la revisión de errores individuales. Resume información, pero no explica por sí solo la causa de los errores.

En `scikit-learn`, podemos calcular RMSE a partir de `mean_squared_error`. Una forma general es calcular el MSE y luego aplicar raíz cuadrada:

```
from sklearn.metrics import mean_squared_error
import numpy as np

mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
rmse
```

En versiones recientes de `scikit-learn`, también puede existir una función específica para calcular la raíz del error cuadrático medio. De todos modos, calcularlo como raíz cuadrada del MSE permite entender claramente de dónde sale.

La interpretación debe hacerse en las unidades de la variable objetivo. Por ejemplo:

RMSE = 9.01 unidades vendidas.

Interpretación: el error típico del modelo está alrededor de 9 unidades, con mayor penalización para errores grandes.

O, en precios:

RMSE = 15000 dólares

Interpretación: los errores del modelo, considerando especialmente las equivocaciones grandes, se ubican alrededor de 15000 dólares en la escala original.

Conviene evitar una interpretación demasiado rígida. El RMSE no significa que todos los errores sean de ese tamaño. Tampoco significa que el modelo se equivoque exactamente esa cantidad en cada caso. Es una medida resumida que combina todos los errores, dando más peso a los grandes.

Como ocurre con MAE y MSE, el RMSE debe compararse entre modelos solo si fueron evaluados sobre el mismo conjunto de datos y la misma variable objetivo. Un RMSE menor indica mejor desempeño bajo esta métrica, pero no necesariamente mejor modelo para todos los usos.

Aun así, la elección entre MAE y RMSE depende del criterio del problema. Si queremos una medida simple del error promedio, MAE puede ser más fácil de comunicar. Si queremos penalizar fuertemente errores grandes, RMSE puede ser más adecuado. Muchas veces se calculan ambas métricas para obtener una imagen más completa. MAE es más directo e interpretable, RMSE es más sensible a errores grandes.

RMSE es la raíz cuadrada del MSE. Se expresa en las unidades originales de la variable objetivo, pero conserva la propiedad de penalizar con más fuerza los errores grandes. Es útil cuando las grandes equivocaciones son especialmente importantes y cuando queremos complementar el MAE con una métrica más sensible a outliers. Sin embargo, debe interpretarse junto con el contexto, la distribución de errores y la revisión de casos individuales.

R²

Además de medir el tamaño de los errores, en regresión suele usarse una métrica llamada **R²**, también conocida como **coeficiente de determinación**. A diferencia de MAE, MSE y RMSE, R² no expresa el error en las unidades originales de la variable

objetivo. Su objetivo es dar una idea de qué tan bien el modelo explica la variabilidad de los datos en comparación con una referencia simple.

La intuición es la siguiente: si no tuviéramos ningún modelo, una estrategia muy básica sería predecir siempre el promedio de la variable objetivo. Por ejemplo, si queremos predecir precios de viviendas, podríamos ignorar todas las variables de entrada y decir que todas las viviendas valen el precio promedio del dataset.

Esa estrategia no usa superficie, antigüedad, ambientes ni ninguna otra variable. Es una línea de base muy simple. R^2 compara el modelo entrenado contra esa referencia.

Si el modelo predice mucho mejor que el promedio, R^2 será alto. Si el modelo no mejora al promedio, R^2 estará cerca de 0. Si el modelo es peor que predecir siempre el promedio, R^2 puede ser negativo.

R^2 alto: el modelo mejora claramente la predicción del promedio.
 R^2 cercano a 0: el modelo no mejora mucho al promedio.
 R^2 negativo: el modelo funciona peor que predecir siempre el promedio.

Esta idea es útil porque no mira solamente el error absoluto, sino cuánto logra aprovechar el modelo la información de las variables de entrada. Si las variables predictoras contienen señal útil, el modelo debería mejorar la referencia del promedio.

Supongamos un problema de precios. Si el precio promedio de las viviendas es 150000, una referencia simple podría predecir 150000 para todas las viviendas. Esa referencia cometerá errores: algunas viviendas reales valdrán 90000, otras 180000, otras 300000. El modelo de regresión debería usar las variables de entrada para acercarse mejor a cada caso.

Referencia: todas las viviendas → 150000

Modelo:

vivienda pequeña → 95000

vivienda mediana → 150000

vivienda grande → 240000

Si el modelo logra reducir mucho los errores respecto de la referencia, R^2 será mayor. Si sus predicciones no son mejores que usar siempre el promedio, R^2 será bajo.

Una forma habitual de interpretar R^2 es como la proporción de variabilidad de la variable objetivo que el modelo logra explicar. Por ejemplo, un $R^2 = 0.80$ puede interpretarse, de manera aproximada, como que el modelo explica el 80% de la variabilidad observada en la variable objetivo. Esta interpretación debe hacerse con cuidado, pero sirve como primera intuición.

$R^2 = 1.00 \rightarrow$ ajuste perfecto en los datos evaluados.
 $R^2 = 0.80 \rightarrow$ el modelo explica gran parte de la variabilidad.
 $R^2 = 0.00 \rightarrow$ no mejora al promedio.
 $R^2 < 0 \rightarrow$ peor que predecir el promedio.

Un R^2 de 1 indica que las predicciones coinciden perfectamente con los valores reales en el conjunto evaluado. En datos reales, esto es poco frecuente y, si aparece en un problema complejo, conviene revisar si hay fuga de datos o si estamos evaluando sobre los mismos datos usados para entrenar.

Un R^2 de 0 no significa que el modelo no tenga errores. Significa que, según esta métrica, no mejora la referencia de predecir siempre el promedio. Puede estar produciendo predicciones distintas, pero no mejores en conjunto.

Un R^2 negativo puede sorprender al principio, porque muchas personas esperan que R^2 esté siempre entre 0 y 1. En ciertos contextos estadísticos puede presentarse así, pero en evaluación predictiva sobre datos de prueba puede ser negativo. Esto ocurre cuando el modelo comete más error que la estrategia simple de predecir el promedio. El modelo no solo es malo, sino que es peor que una referencia muy simple.

Esto puede suceder si el modelo está mal ajustado, si las variables no contienen señal útil, si hay un problema de preprocesamiento, si se evaluó en datos muy distintos a los de entrenamiento o si el modelo sobreajustó y luego generalizó mal.

R^2 es útil porque ofrece una medida relativa. Mientras MAE y RMSE nos dicen cuánto se equivoca el modelo en unidades reales, R^2 nos dice qué tan bien lo hace en comparación con una referencia básica. Son preguntas distintas.

MAE / RMSE: ¿cuánto se equivoca el modelo?
 R^2 : ¿cuánto mejora el modelo respecto de predecir el promedio?

Por ejemplo, un MAE de 5000 dólares puede parecer pequeño o grande según el problema. Si las viviendas tienen precios muy variables, tal vez sea excelente. Si todas las viviendas tienen precios parecidos, quizás no sea tan bueno. R^2 ayuda a ubicar el desempeño del modelo en relación con la variabilidad de los datos.

Supongamos dos problemas distintos:

Problema A:
precios entre 100000 y 500000
MAE = 10000

Problema B:
precios entre 100000 y 120000
MAE = 10000

El mismo MAE puede significar cosas muy diferentes. En el Problema A, un error de 10000 puede ser relativamente pequeño frente a la variabilidad general. En el Problema B, puede ser enorme. R^2 puede ayudar a reflejar esa diferencia porque considera cuánto varía la variable objetivo.

Sin embargo, R^2 también tiene limitaciones. La primera es que no está en unidades reales. Decir que un modelo tiene R^2 de 0.72 no nos dice directamente cuántos pesos, grados, unidades o minutos se equivoca. Para comunicar el impacto práctico del error, necesitamos métricas como MAE o RMSE.

R^2 ayuda a comparar capacidad explicativa. MAE y RMSE ayudan a interpretar errores concretos.

La segunda limitación es que un R^2 alto no garantiza que el modelo sea adecuado para todas las decisiones. Un modelo puede tener buen R^2 general y, aun así, fallar mucho en ciertos rangos. Por ejemplo, puede predecir bien viviendas de precio medio y mal viviendas muy caras. Puede funcionar bien en días normales y mal en días de promoción. Puede explicar buena parte de la variabilidad general, pero equivocarse donde más nos importa.

Por eso, R^2 no reemplaza el análisis de errores. Debemos seguir revisando valores reales frente a valores predichos, errores por segmentos y posibles patrones en los residuos.

La tercera limitación es que R^2 puede aumentar al agregar variables, especialmente en el conjunto de entrenamiento, aunque esas variables no aporten una mejora real sobre datos nuevos. Si agregamos muchas columnas, el modelo puede ajustarse mejor a los datos de entrenamiento, pero eso no significa que generalice mejor.

Esta es una razón más para evaluar sobre datos de prueba y, más adelante, usar validación cruzada. No nos interesa solamente ajustar bien los datos que ya conocemos, sino predecir bien casos no vistos. Un R^2 alto en entrenamiento no garantiza buen desempeño en datos nuevos.

La cuarta limitación es que R^2 puede ser difícil de interpretar cuando el problema tiene una variabilidad muy baja. Si los valores reales son todos muy parecidos, pequeñas diferencias pueden producir comportamientos extraños en la métrica. En esos casos, MAE o RMSE pueden ser más claros para comunicar el error.

En `scikit-learn`, R^2 se calcula con `r2_score`:

```
from sklearn.metrics import r2_score
```

```
r2 = r2_score(y_test, y_pred)
r2
```

También es frecuente que el método `score` de algunos modelos de regresión devuelva R^2 por defecto:

```
modelo.score(X_test, y_test)
```

Esto puede ser cómodo, pero conviene recordar qué está devolviendo. En clasificadores, `score` suele devolver accuracy. En regresores, suele devolver R^2 . Usar el mismo método no significa que la métrica sea la misma.

```
modelo.score en clasificación: frecuentemente accuracy.
modelo.score en regresión:     frecuentemente  $R^2$ .
```

Al comunicar R^2 , conviene acompañarlo con otras métricas. Por ejemplo:

El modelo obtuvo: $MAE = 8500$ dólares $RMSE = 14000$ dólares $R^2 = 0.76$

Esta combinación informa tres cosas distintas: el error absoluto promedio, la penalización de errores grandes y la proporción de variabilidad explicada respecto del promedio. Juntas ofrecen una visión más completa que cualquiera de ellas por separado.

También conviene comparar R^2 con una referencia simple. Si el modelo tiene R^2 cercano a 0, probablemente no está aportando demasiado sobre la predicción del promedio. Si tiene R^2 negativo, debemos revisar seriamente el modelo, los datos o la forma de evaluación. Si tiene R^2 muy alto, debemos verificar que no haya fuga de datos y que la evaluación se haya hecho correctamente.

R^2 mide qué tan bien el modelo mejora la referencia de predecir siempre el promedio de la variable objetivo. Un valor cercano a 1 indica gran capacidad para explicar la variabilidad de los datos; un valor cercano a 0 indica poca mejora respecto del promedio; y un valor negativo indica peor desempeño que esa referencia simple. Es una métrica útil para comparar modelos y entender capacidad explicativa, pero no reemplaza métricas en unidades reales como MAE o RMSE ni el análisis de errores concretos.

Interpretar errores en unidades reales

Las métricas de regresión no tienen sentido si se leen como números aislados. Un MAE de 10, un RMSE de 25 o un R^2 de 0.70 pueden parecer buenos o malos según el problema, pero no podemos interpretarlos correctamente sin conocer qué variable estamos prediciendo, en qué unidades está medida y para qué se usará la predicción.

Esta idea es especialmente importante con MAE y RMSE, porque ambas métricas se expresan en las unidades de la variable objetivo. Si predecimos ventas, el error estará en unidades vendidas. Si predecimos precios, estará en pesos, dólares u otra moneda. Si predecimos temperatura, estará en grados. Si predecimos tiempo, estará en minutos, horas o días.

MAE = 10 Puede significar:

- 10 unidades vendidas,
- 10 grados,
- 10 minutos,
- 10 pesos,
- 10 kWh.

El número es el mismo, pero su significado cambia por completo.

Supongamos un modelo que predice temperatura máxima diaria y obtiene MAE = 2 °C. Esto significa que, en promedio, las predicciones se alejan 2 grados del valor real. Para una aplicación meteorológica general, puede ser un error razonable. Pero para un proceso industrial muy sensible a la temperatura, quizá no sea suficiente.

Ahora imaginemos un modelo que predice precios de viviendas y obtiene MAE = 2000 dólares. En este caso, un error promedio de 2000 dólares podría ser pequeño si las viviendas valen alrededor de 200000 dólares. Pero sería enorme si estuviéramos prediciendo productos cuyo precio habitual ronda los 3000 dólares.

Error de 2000 dólares en una vivienda: puede ser pequeño.
Error de 2000 dólares en un producto de 3000: puede ser enorme.

Por eso, interpretar errores en unidades reales exige conocer la escala del problema. No alcanza con mirar si el valor de la métrica es “grande” o “chico”. Debemos preguntarnos grande o chico en relación con qué.

Una forma útil de interpretar el error es compararlo con valores típicos de la variable objetivo. Por ejemplo, si predecimos ventas diarias y el MAE es de 15 unidades,

necesitamos saber si las ventas habituales son 50, 500 o 5000 unidades. Si MAE: 15 unidades, el error promedio representa una parte importante del valor típico.

En cambio, si las ventas promedio son de 5000 unidades y MAE: 15 unidades, el error promedio parece muy pequeño respecto del valor típico. El mismo MAE puede contar historias completamente distintas.

También conviene comparar el error con la variabilidad natural de los datos. Si la variable objetivo cambia mucho de un caso a otro, un error determinado puede ser aceptable. Si los valores reales están muy concentrados, ese mismo error puede ser problemático.

Supongamos dos problemas de precios:

```text

Problema A:

precios entre 100000 y 500000

MAE = 10000

Problema B:

precios entre 100000 y 120000

MAE = 10000

En el Problema A, un MAE de 10000 puede ser relativamente moderado frente a un rango de precios amplio. En el Problema B, ese mismo error puede ser muy alto, porque representa una parte importante de toda la variación posible.

Esta es una de las razones por las que conviene mirar varias métricas. MAE y RMSE nos dan errores en unidades reales.  $R^2$  nos ayuda a entender cuánto mejora el modelo respecto de predecir siempre el promedio. Juntas, esas métricas permiten una interpretación más equilibrada.

MAE y RMSE: ¿cuánto se equivoca el modelo?

$R^2$ : ¿cuánto mejora respecto de una referencia simple?

Otra comparación importante es contra un modelo base. Un modelo base puede ser una estrategia muy simple, como predecir siempre el promedio o la mediana de la variable objetivo. Si nuestro modelo obtiene un MAE de 20, ese número puede parecer bueno. Pero si predecir siempre el promedio obtiene un MAE de 22, la mejora es pequeña. Si la referencia obtiene un MAE de 80, la mejora es mucho más clara.

Esta comparación evita sobrevalorar modelos que apenas mejoran una regla sencilla. En Machine Learning, un modelo no debería considerarse útil solo porque produce predicciones. Debería aportar una mejora significativa para el problema.

Además, la interpretación del error depende del costo de equivocarse. En algunos problemas, un error pequeño puede tener consecuencias importantes. En otros, un error relativamente grande puede ser tolerable.

Por ejemplo, en predicción de stock, subestimar la demanda puede dejar clientes sin producto. Sobreestimar puede generar exceso de inventario. En tiempos de entrega, subestimar puede producir frustración si el pedido tarda más de lo prometido. En consumo energético, un error grande puede afectar la planificación de recursos.

El error no solo se mide, también se interpreta según sus consecuencias. Esto significa que dos modelos con errores similares pueden ser evaluados de manera distinta según cómo se distribuyan esos errores. Un modelo puede tener buen MAE general, pero fallar mucho justo en los casos más importantes.

Supongamos un modelo de ventas que funciona bien en días normales, pero falla en días de promoción. Si las promociones son críticas para la planificación, el MAE global puede ocultar un problema importante. Lo mismo ocurre si un modelo de precios predice bien viviendas comunes, pero falla mucho en propiedades de alto valor.

Un error promedio aceptable puede ocultar errores graves en segmentos importantes. Por eso, además de leer métricas generales, conviene revisar errores por grupos o rangos. Podemos separar los casos según valor real bajo, medio y alto; por zonas; por fechas; por tipo de producto; por segmento de cliente; o por cualquier variable relevante para el problema.

Preguntas útiles:

- ¿El error es parecido en todos los rangos?
- ¿El modelo falla más en valores altos?
- ¿Falla más en ciertos grupos?
- ¿Falla más en ciertos momentos?

Estas preguntas ayudan a interpretar los errores en términos reales, no solo estadísticos.

También debemos tener cuidado con RMSE. Como penaliza más los errores grandes, suele ser mayor que MAE. Si RMSE es mucho mayor que MAE, puede indicar que algunos casos tienen errores muy grandes. Esa diferencia debe investigarse.

MAE = 8  
RMSE = 10

Errores relativamente moderados.

MAE = 8  
RMSE = 30

Posible presencia de errores grandes.

Esta comparación no explica por sí sola el problema, pero señala que conviene mirar los casos con mayor error.

En cambio,  $R^2$  debe interpretarse de otra manera. Como no está en unidades reales, no nos dice directamente cuánto se equivoca el modelo. Un  $R^2$  de 0.80 puede sonar bien, pero necesitamos acompañarlo con MAE o RMSE para entender la magnitud del error. Un modelo puede explicar buena parte de la variabilidad y aun así tener errores demasiado grandes para el uso práctico.

$R^2$  alto no garantiza que el error sea aceptable para la decisión real. Por ejemplo, un modelo de precios puede tener  $R^2$  alto porque diferencia bien propiedades baratas y caras, pero cometer errores de decenas de miles de dólares. Para algunas decisiones, eso puede ser demasiado. Para otras, puede ser una primera estimación útil.

La comunicación de métricas de regresión debe traducir los números al lenguaje del problema. No basta con escribir:

MAE = 12.4  
RMSE = 18.9  
 $R^2$  = 0.76

Es mejor acompañar esos valores con una interpretación:

*El modelo se equivoca, en promedio, por 12.4 unidades. Los errores grandes elevan el RMSE a 18.9 unidades. El modelo mejora claramente la referencia del promedio, con un  $R^2$  de 0.76.*

Y todavía mejor si agregamos contexto:

*Como las ventas diarias suelen estar entre 300 y 500 unidades, un error promedio de 12.4 unidades parece moderado para una estimación inicial de stock.*

Esta forma de comunicar conecta las métricas con la toma de decisiones.

También debemos recordar que las métricas se calculan sobre un conjunto de datos específico. Un MAE de 12 en el conjunto de prueba indica cómo funcionó el modelo

en esos casos no vistos, pero no garantiza que funcionará igual en el futuro si cambian las condiciones. Si los datos nuevos son distintos, los errores pueden aumentar.

Las métricas describen el desempeño en los datos evaluados. No garantizan comportamiento idéntico en cualquier situación futura. Por eso, en problemas reales, la evaluación debe complementarse con seguimiento, actualización de datos y revisión periódica del modelo.

Interpretar errores en unidades reales significa traducir las métricas al contexto del problema. MAE y RMSE deben leerse en las unidades de la variable objetivo y compararse con valores típicos, variabilidad, referencias simples y consecuencias prácticas.  $R^2$  ayuda a entender cuánto mejora el modelo respecto del promedio, pero no reemplaza la interpretación del error concreto. Una buena evaluación de regresión no termina en el cálculo de métricas: debe responder si los errores son aceptables para la decisión que el modelo pretende apoyar.

## Por qué no existe una única métrica perfecta

---

En regresión usamos métricas para resumir los errores del modelo. MAE, MSE, RMSE y  $R^2$  ayudan a mirar el desempeño desde ángulos distintos. Cada una responde una pregunta útil, pero ninguna cuenta toda la historia por sí sola.

Esto es importante porque, al comenzar, puede aparecer la tentación de buscar “la mejor métrica” y usarla siempre. Pero en Machine Learning no existe una métrica universal que sea perfecta para todos los problemas. La métrica adecuada depende de qué estamos prediciendo, cómo se usará la predicción, qué errores son más costosos y qué comportamiento queremos evitar.

MAE es fácil de interpretar porque se expresa en las unidades originales de la variable objetivo. Si predecimos ventas y el MAE es 12 unidades, podemos comunicarlo de manera directa: el modelo se equivoca, en promedio, por 12 unidades. Esta claridad lo vuelve muy útil para informes, comparaciones iniciales y comunicación con personas no técnicas.

Pero MAE también tiene límites. Al promediar errores absolutos, no muestra si el modelo tiende a sobreestimar o subestimar. Tampoco penaliza de manera especial los errores grandes. Un error de 100 pesa diez veces más que un error de 10, no cien veces más. Si en el problema las grandes equivocaciones son especialmente graves, MAE puede no reflejar esa preocupación con suficiente fuerza. MAE es fácil de interpretar, pero no destaca tanto los errores grandes.

MSE, en cambio, penaliza mucho más los errores grandes porque eleva cada error al cuadrado. Esto puede ser útil cuando queremos evitar predicciones muy alejadas del valor real. Si un error grande es mucho más costoso que varios errores pequeños, MSE puede ayudarnos a detectar y comparar modelos que fallan demasiado en algunos casos.

Pero MSE tiene un problema importante: sus unidades quedan elevadas al cuadrado. Si predecimos precios, el MSE queda en unidades monetarias al cuadrado. Si predecimos temperatura, en grados al cuadrado. Esa interpretación no es cómoda para comunicar resultados en términos reales. MSE penaliza errores grandes, pero es menos intuitivo por sus unidades al cuadrado.

RMSE intenta resolver parte de esa dificultad. Como es la raíz cuadrada del MSE, vuelve a expresarse en las unidades originales de la variable objetivo. Esto lo hace más interpretable que MSE, pero conserva la sensibilidad a errores grandes. Por eso, RMSE puede ser una buena métrica cuando queremos una medida en unidades reales que también castigue fuertemente las grandes equivocaciones.

Sin embargo, esa sensibilidad también puede ser una desventaja. Si hay outliers, errores de carga o casos muy particulares, RMSE puede crecer mucho y quedar dominado por unos pocos ejemplos. A veces eso es deseable, porque esos errores grandes importan. Otras veces puede distorsionar la evaluación general del modelo. RMSE vuelve a unidades reales, pero sigue siendo sensible a errores extremos.

$R^2$  responde una pregunta diferente. No mide directamente el error en unidades reales, sino cuánto mejora el modelo respecto de una referencia simple: predecir siempre el promedio de la variable objetivo. Esto es útil para saber si el modelo está aprovechando las variables de entrada y explicando parte de la variabilidad de los datos.

Pero  $R^2$  tampoco es suficiente por sí solo. Un  $R^2$  alto no nos dice cuántos pesos, grados, unidades o minutos se equivoca el modelo. Un modelo puede tener buen  $R^2$  y aun así producir errores demasiado grandes para una aplicación concreta. También puede comportarse bien en promedio y fallar en segmentos importantes.  $R^2$  ayuda a comparar contra una referencia, pero no expresa el error en unidades reales.

Estas diferencias muestran que cada métrica ilumina una parte del problema. MAE mira el error absoluto promedio. MSE y RMSE prestan más atención a errores grandes.  $R^2$  compara el modelo contra una referencia basada en el promedio. Ninguna de estas miradas es completa.

Podemos resumirlo así:

- **MAE:** ¿cuánto se equivoca el modelo en promedio?



- **MSE:** ¿qué tan fuertes son los errores al cuadrado?
- **RMSE:** ¿cuánto se equivoca el modelo, penalizando más los errores grandes, pero en unidades reales?
- **R<sup>2</sup>** ¿cuánto mejora el modelo respecto de predecir el promedio?

La elección de una métrica depende del costo de los errores. Si todos los errores importan de manera aproximadamente proporcional, MAE puede ser una opción muy clara. Si los errores grandes son mucho más graves que los pequeños, RMSE puede ser más adecuado. Si queremos saber si el modelo aporta algo respecto de una referencia simple, R<sup>2</sup> es útil. Si estamos comparando modelos durante una etapa técnica, MSE también puede ser informativo.

Supongamos un modelo para predecir demanda de productos. Si equivocarse por 20 unidades no es mucho peor que equivocarse dos veces por 10, MAE puede representar bien el problema. Pero si una gran subestimación puede dejar a una tienda sin stock durante varios días, quizá nos interese una métrica más sensible a errores grandes, como RMSE.

En precios de viviendas, un error promedio puede ser fácil de comunicar con MAE. Pero si el modelo falla mucho en propiedades muy caras, RMSE puede revelar que hay errores extremos que el MAE suaviza. En temperatura, MAE puede ser suficiente para una predicción general, pero para aplicaciones críticas quizá convenga mirar también errores máximos o comportamiento en condiciones extremas.

Esto nos lleva a una idea clave: una métrica puede ser correcta matemáticamente y, aun así, no ser la más adecuada para el objetivo del proyecto. Las métricas no deben elegirse por costumbre, sino por su relación con la decisión que se quiere apoyar.

No elegimos métricas solo porque sean conocidas. Las elegimos porque responden preguntas importantes para el problema.

También debemos recordar que una métrica promedio puede ocultar diferencias internas. Un modelo puede tener buen MAE general y fallar mucho en valores altos. Puede tener buen RMSE y aun así sobreestimar sistemáticamente ciertos grupos. Puede tener buen R<sup>2</sup> y producir errores inaceptables para una decisión específica.

Por eso, además de calcular métricas, debemos revisar errores individuales, gráficos de valores reales contra predichos y errores por segmentos. Las métricas resumen; el análisis de errores explica. Las métricas responden ¿cómo funcionó el modelo en conjunto?. El análisis de errores responde ¿dónde y cómo se equivocó?

Otra razón por la que no existe una métrica perfecta es que distintos usuarios pueden tener prioridades distintas. Un equipo técnico puede querer comparar modelos con una métrica estable. Un equipo de negocio puede necesitar entender el error en unidades reales. Un área operativa puede preocuparse por los peores casos. Una persona responsable de comunicar resultados puede necesitar una explicación clara y prudente.

Un mismo modelo puede verse distinto según la métrica elegida. Un modelo puede tener mejor MAE pero peor RMSE que otro. Eso significa que, en promedio, se equivoca menos, pero quizás comete algunos errores grandes. Otro puede tener peor MAE pero mejor RMSE, si evita grandes fallas a costa de errores moderados más frecuentes. La elección dependerá de qué comportamiento preferimos. La mejor elección depende de qué errores queremos evitar.

Por eso, en la práctica, rara vez conviene mirar una sola métrica. Lo más razonable es usar un conjunto pequeño de métricas complementarias. Por ejemplo, MAE para interpretación directa, RMSE para sensibilidad a errores grandes y  $R^2$  para comparar contra una referencia. Luego, junto con esas métricas, revisar casos concretos.

Tampoco conviene llenar un informe con demasiadas métricas sin explicación. Muchas métricas pueden confundir si no se aclara qué aporta cada una. Es mejor elegir pocas, interpretarlas bien y relacionarlas con el problema.

No existe una única métrica perfecta para regresión porque cada una responde una pregunta distinta. MAE es claro y directo, RMSE penaliza más los errores grandes, MSE es útil técnicamente pero menos interpretable, y  $R^2$  compara el modelo contra una referencia simple. La evaluación adecuada depende del contexto, del costo de los errores, de la escala de la variable objetivo y de la forma en que se usarán las predicciones. Evaluar bien no significa elegir una métrica universal, sino elegir métricas coherentes con el problema y complementarlas con análisis de errores.

## Comparación entre modelos de regresión

---

Una vez que sabemos calcular métricas de regresión, podemos usarlas para comparar modelos. Esta comparación es una parte central del trabajo en Machine Learning: no entrenamos un modelo para mirarlo de forma aislada, sino para decidir si funciona mejor que una referencia, si mejora a otras alternativas y si sus errores son aceptables para el problema.

Supongamos que queremos predecir ventas diarias. Podemos entrenar una regresión lineal y obtener estas métricas en el conjunto de prueba:

```
Regresión lineal:
MAE = 14.2 unidades
RMSE = 21.8 unidades
R2 = 0.72
```

Luego probamos otro modelo y obtenemos:

```
Modelo alternativo:
MAE = 11.5 unidades
RMSE = 18.1 unidades
R2 = 0.80
```

Si ambos modelos fueron evaluados sobre el mismo conjunto de prueba, el segundo parece funcionar mejor: tiene menor MAE, menor RMSE y mayor  $R^2$ . En promedio se equivoca menos, penaliza menos los errores grandes y explica mejor la variabilidad de los datos. Pero esta conclusión solo es válida si la comparación es justa.

Para comparar modelos de regresión, todos deben evaluarse sobre los mismos datos de prueba. Si un modelo se evalúa sobre un conjunto más fácil y otro sobre uno más difícil, las métricas no son comparables. La separación entre entrenamiento y prueba debe ser la misma, o al menos seguir un procedimiento equivalente y controlado.

Para que la comparación sea justa, debemos considerar el mismo problema, la misma variable objetivo y el mismo conjunto de prueba, además de usar las mismas métricas.

También deben usarse las mismas transformaciones de datos de manera correcta. Si un modelo recibe variables limpias, escaladas y codificadas correctamente, y otro recibe datos mal preparados, la comparación no evalúa solo el modelo, sino todo el flujo de preprocesamiento. En la práctica, eso puede estar bien si comparamos pipelines completos, pero debemos ser conscientes de qué estamos comparando.

Un modelo no es solo el algoritmo. En sentido práctico, incluye los datos usados, el preprocesamiento, el algoritmo, su configuración y, si corresponde, el umbral o la regla de uso. Es el conjunto de decisiones que abarca variables, limpieza, transformaciones, entrenamiento y evaluación.

En regresión, una comparación básica podría incluir una referencia simple. Por ejemplo, predecir siempre el promedio de `y` en el conjunto de entrenamiento. Esta referencia no usa variables predictoras, pero sirve como punto de partida. El modelo base predice siempre el promedio, mientras que el modelo entrenado utiliza las variables de entrada para intentar mejorar esa predicción.

Si un modelo de regresión no mejora claramente esa referencia, probablemente no esté aprendiendo una relación útil. Puede tener una ecuación, coeficientes y predicciones distintas, pero si sus errores no son mejores que los de una regla simple, su valor práctico es limitado.

Por ejemplo:

|                   |            |
|-------------------|------------|
| Modelo base:      | MAE = 18.0 |
| Regresión lineal: | MAE = 17.4 |

La regresión lineal mejora apenas al modelo base. Tal vez esa mejora no justifique la complejidad, especialmente si el modelo requiere más datos, mantenimiento o explicación.

En cambio:

|                   |            |
|-------------------|------------|
| Modelo base:      | MAE = 18.0 |
| Regresión lineal: | MAE = 9.5  |

Aquí la mejora es mucho más clara. El modelo está usando información de las variables de entrada para reducir el error de manera importante.

Esta comparación con una referencia simple es una de las formas más sanas de evaluar. Evita que nos impresionemos con un modelo solo porque produce números o gráficos. La pregunta debe ser: ¿mejora algo que podríamos hacer de manera mucho más simple?

También podemos comparar varios modelos entre sí. Por ejemplo:

| Modelo            | MAE  | RMSE | R <sup>2</sup> |
|-------------------|------|------|----------------|
| Modelo base       | 18.0 | 25.0 | 0.00           |
| Regresión lineal  | 11.2 | 16.8 | 0.68           |
| Árbol de decisión | 10.5 | 20.4 | 0.61           |

La lectura no siempre es inmediata. El árbol de decisión tiene menor MAE que la regresión lineal, pero mayor RMSE y menor R<sup>2</sup>. Esto sugiere que, en promedio absoluto, puede equivocarse un poco menos, pero quizás comete algunos errores grandes que elevan el RMSE. La regresión lineal parece más estable frente a errores grandes, aunque su MAE sea levemente mayor.

¿Cuál elegir? Depende del problema. Si nos importa reducir el error promedio típico, podríamos mirar MAE con más atención. Si nos preocupan especialmente las grandes equivocaciones, RMSE puede pesar más. Si queremos una referencia global de

variabilidad explicada,  $R^2$  puede ayudar. La comparación no se resuelve mirando una única columna sin pensar.

El modelo con menor MAE presenta un menor error absoluto promedio; el que tiene menor RMSE recibe una penalización menor por los errores grandes; y el que alcanza un  $R^2$  mayor logra una mejor mejora respecto de predecir siempre el promedio.

Además, las diferencias pequeñas deben interpretarse con prudencia. Si un modelo tiene MAE de 10.42 y otro de 10.39, no conviene concluir automáticamente que el segundo es superior de manera importante. Esa diferencia puede ser insignificante frente a la variabilidad de los datos, la incertidumbre de la separación train/test o el ruido natural del problema.

No toda diferencia numérica es una diferencia práctica importante. Por eso, más adelante estudiaremos validación cruzada. La validación cruzada permite comparar modelos usando varias divisiones de los datos y observar no solo el promedio de las métricas, sino también su variabilidad. Por ahora, con una única separación train/test, debemos ser especialmente prudentes.

También es importante comparar modelos teniendo en cuenta su simplicidad. Un modelo más complejo puede mejorar una métrica, pero quizá sea más difícil de explicar, más costoso de mantener o más sensible a cambios en los datos. Si la mejora es pequeña, un modelo simple puede ser preferible.

Por ejemplo:

```
Regresión lineal:
MAE = 10.8
modelo simple e interpretable

Modelo complejo:
MAE = 10.4
modelo más difícil de explicar
```

La diferencia de 0.4 unidades puede no justificar la complejidad adicional. En cambio, si el modelo complejo reduce el MAE de 10.8 a 6.2, la mejora puede ser importante, siempre que el modelo generalice bien y sea usable en el contexto real.

La comparación entre modelos también debe incluir análisis de errores. Dos modelos con métricas similares pueden fallar en lugares distintos. Uno puede funcionar mejor para valores bajos y peor para valores altos. Otro puede ser más estable en valores extremos, pero menos preciso en el centro de la distribución.

Para compararlos, podemos preguntarnos dónde se equivoca cada uno, si falla más en valores altos o bajos, si comete errores extremos, si funciona igual en todos los segmentos y si sus errores son aceptables para la decisión.

Estas preguntas pueden cambiar la elección. Un modelo con mejor métrica global puede no ser el más adecuado si falla en los casos más importantes. Por ejemplo, en predicción de demanda, puede ser crítico acertar días de alta demanda. En precios, puede interesar especialmente no cometer errores grandes en propiedades de alto valor. En consumo energético, puede ser clave anticipar picos.

Por eso, comparar modelos no es una competencia abstracta de métricas. Es una decisión contextual.

También debemos revisar si algún modelo está sobreajustando. Si un modelo tiene errores muy bajos en entrenamiento pero errores altos en prueba, está aprendiendo demasiado bien los datos conocidos y generalizando mal. Este tema será central en capítulos posteriores, pero ya podemos anticipar la idea: no nos interesa el modelo que mejor memoriza, sino el que mejor predice datos no vistos. El buen desempeño en entrenamiento no alcanza. Necesitamos buen desempeño en datos no vistos.

En regresión, podemos comparar métricas de entrenamiento y prueba para obtener una primera señal. Si el MAE de entrenamiento es muy bajo y el MAE de prueba mucho más alto, puede haber sobreajuste. Si ambos son altos, puede haber subajuste o falta de señal en las variables. Estos conceptos se desarrollarán más adelante.

En `scikit-learn`, una comparación sencilla puede construirse calculando las mismas métricas para varios modelos:

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

y_pred = modelo.predict(X_test)

mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)
```

Luego podemos organizar los resultados en una tabla:

| Modelo            | MAE  | RMSE | R <sup>2</sup> |
|-------------------|------|------|----------------|
| Modelo base       | 18.0 | 25.0 | 0.00           |
| Regresión lineal  | 11.2 | 16.8 | 0.68           |
| Árbol de decisión | 10.5 | 20.4 | 0.61           |

Esta tabla permite comparar, pero no decide por sí sola. Debemos interpretarla. Una buena práctica es acompañar la tabla con una explicación breve: qué modelo mejora la referencia, qué métrica se prioriza, qué errores preocupan y qué limitaciones quedan.

También conviene evitar elegir un modelo solo porque gana en una métrica por una diferencia mínima. La comparación debe considerar el desempeño, los errores, la simplicidad, la interpretabilidad, la estabilidad, el costo de mantenimiento y la adecuación al uso real.

Comparar modelos de regresión consiste en evaluar alternativas sobre los mismos datos, con las mismas métricas y bajo condiciones justas. MAE, RMSE y  $R^2$  permiten mirar el desempeño desde perspectivas complementarias. Un modelo debe compararse contra una referencia simple y contra otras alternativas, pero la elección final no debería depender solo de una tabla de números. Debe considerar qué errores son importantes, dónde falla cada modelo, si la mejora es práctica y si el modelo es adecuado para el contexto en el que será usado.

## Evaluar cuánto se equivoca un modelo

En este capítulo estudiamos las principales métricas para evaluar modelos de regresión. A diferencia de la clasificación, donde contamos aciertos y errores entre clases, en regresión medimos distancias entre valores reales y valores predichos.

La idea básica es el error de predicción. Para cada caso, podemos comparar el valor real con el valor estimado por el modelo. Si solo nos interesa la magnitud de la diferencia, usamos el error absoluto. Esta noción simple es la base de varias métricas importantes.

```
Valor real - Valor predicho = Error
|Valor real - Valor predicho| = Error absoluto
```

MAE resume esos errores absolutos calculando su promedio. Es una métrica clara, directa y fácil de comunicar porque se expresa en las mismas unidades de la variable objetivo. Si predecimos ventas, el MAE está en unidades vendidas. Si predecimos precios, está en moneda. Si predecimos temperatura, está en grados.

MSE también parte de los errores, pero los eleva al cuadrado antes de promediarlos. Esto hace que penalice con más fuerza las grandes equivocaciones. Su desventaja es

que queda expresado en unidades al cuadrado, lo que dificulta su interpretación directa.

RMSE toma la raíz cuadrada del MSE. Por eso vuelve a las unidades originales de la variable objetivo, pero conserva la sensibilidad a errores grandes. Puede ser especialmente útil cuando las predicciones muy alejadas del valor real son mucho más problemáticas que los errores pequeños.

$R^2$ , en cambio, responde una pregunta distinta. No expresa el error en unidades reales, sino cuánto mejora el modelo respecto de una referencia simple: predecir siempre el promedio de la variable objetivo. Un  $R^2$  alto indica que el modelo explica una parte importante de la variabilidad de los datos. Un  $R^2$  cercano a 0 indica que apenas mejora al promedio. Un  $R^2$  negativo indica que funciona peor que esa referencia básica.

En síntesis, MAE representa el error absoluto promedio; MSE, el promedio de los errores elevados al cuadrado; RMSE, la raíz del MSE expresada en las unidades originales; y  $R^2$ , la mejora obtenida respecto de predecir siempre el promedio.

También vimos que ninguna métrica es perfecta. MAE es fácil de interpretar, pero no destaca tanto los errores grandes. RMSE penaliza más las grandes equivocaciones, pero puede ser muy sensible a outliers.  $R^2$  ayuda a comparar contra una referencia, pero no dice cuánto se equivoca el modelo en unidades reales. Por eso, evaluar regresión exige mirar varias métricas y entender qué pregunta responde cada una.

La interpretación en unidades reales es fundamental. Un MAE de 10 no significa nada por sí solo. Puede ser 10 pesos, 10 grados, 10 unidades vendidas o 10 minutos. Además, ese error puede ser pequeño o grande según la escala del problema, la variabilidad de los datos y la decisión que se quiere apoyar.

Una métrica de regresión debe leerse junto con:

- la unidad de la variable objetivo;
- los valores típicos;
- la variabilidad de los datos;
- el costo de equivocarse;
- el uso real de la predicción.

También destacamos la importancia de comparar modelos de manera justa. Para decidir si un modelo es mejor que otro, deben evaluarse sobre el mismo conjunto de prueba, con las mismas métricas y bajo condiciones equivalentes. Además, siempre conviene incluir una referencia simple, como predecir el promedio o la mediana, para saber si el modelo realmente aporta valor.



Una tabla de métricas puede ayudar a comparar, pero no decide sola. Dos modelos pueden tener resultados parecidos y errores muy distintos. Uno puede funcionar mejor en valores bajos y otro en valores altos. Uno puede tener menor MAE, pero mayor RMSE. Uno puede ser más simple e interpretable, mientras que otro mejora levemente una métrica, pero es más difícil de explicar.

Por eso, la evaluación de regresión debe combinar números y análisis. Las métricas resumen el desempeño general. La revisión de errores individuales, residuos y segmentos ayuda a comprender dónde falla el modelo y si esos errores son aceptables.

Evaluar modelos de regresión significa medir cuánto se alejan las predicciones de los valores reales. MAE, MSE, RMSE y  $R^2$  ofrecen miradas complementarias. La elección de métricas depende del problema, del costo de los errores y del uso que tendrán las predicciones. Un buen modelo de regresión no es simplemente el que obtiene el número más atractivo, sino el que produce errores comprensibles, comparables y aceptables para la decisión que se quiere tomar.

## Cierre del Tomo I

---

Llegamos a un punto clave del recorrido. Hasta aquí construimos una base para entender qué es Machine Learning, cómo se organizan los datos, cómo se separan los conjuntos de entrenamiento y prueba, cómo se preparan las variables y cómo se entrenan y evalúan los primeros modelos de clasificación y regresión.

Este primer tomo se concentró en los fundamentos. La idea fue avanzar paso a paso, desde las preguntas iniciales hasta la construcción de modelos capaces de predecir categorías o valores numéricos. También trabajamos métricas que permiten evaluar esos modelos y comprender mejor sus aciertos y errores.

A partir de este punto, el camino continúa con una mirada más profunda. En el siguiente tomo vamos a estudiar cómo saber si un modelo realmente generaliza, cómo detectar sobreajuste y subajuste, cómo usar validación cruzada, cómo ajustar hiperparámetros y cómo controlar la complejidad de los modelos.

También avanzaremos hacia modelos más potentes, como Random Forest, Gradient Boosting, Support Vector Machines y Naive Bayes. Luego abriremos la puerta al aprendizaje no supervisado, con clustering y reducción de dimensionalidad, y dedicaremos una parte importante a la interpretación de modelos, los sesgos, los errores comunes y la comunicación responsable de resultados.

Este cierre no marca el final del aprendizaje, sino un cambio de etapa. Ya contamos con las herramientas básicas para construir y evaluar modelos iniciales. En el tomo siguiente vamos a profundizar en cómo mejorarlos, compararlos, interpretarlos y usarlos con mayor criterio.

MATERIAL GRATUITO

## Última versión y apoyo al proyecto

Este documento forma parte de VintaBytes y se comparte de manera gratuita.

Escaneá el código QR para acceder a la última versión disponible, encontrar otros recursos del proyecto o colaborar de forma voluntaria.

Si este material te resultó útil, cualquier aporte ayuda a seguir preparando libros, cuadernos y ejemplos en español. También ayuda compartirlo, recomendarlo o avisarme si encontrás algo para mejorar.

Y si querés colaborar con el café que acompaña muchas de estas páginas, prometo usarlo con responsabilidad pedagógica.

Muchas gracias por formar parte de VintaBytes.



Escaneá para ir a la página oficial

También podés entrar desde:

<https://vintabytes.github.io/>